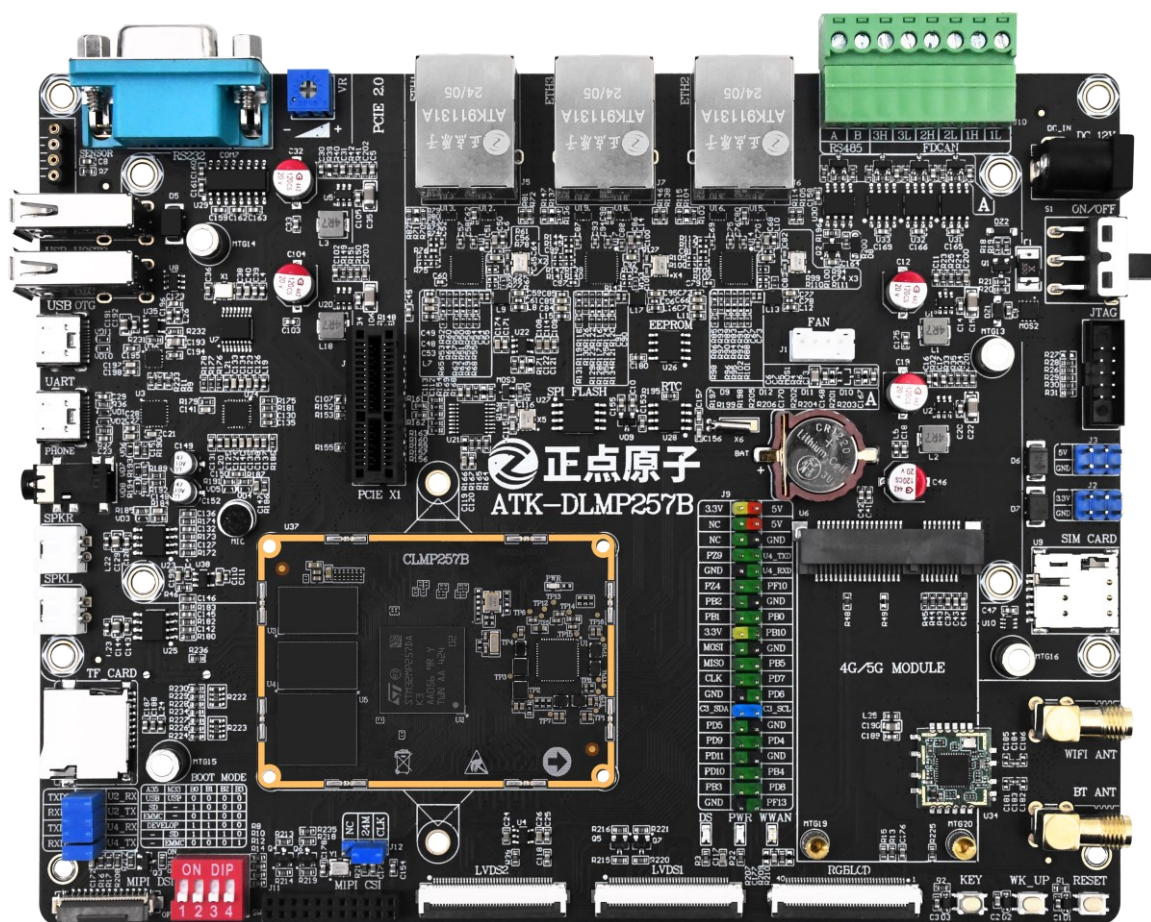


ATK-DLMP257B 嵌入式

Linux 驱动开发指南

V1.1





正点原子公司名称 : 广州市星翼电子科技有限公司

原子哥在线教学平台 : www.yuanzige.com

开源电子网 / 论坛 : <http://www.openedv.com/forum.php>

正点原子淘宝店铺 : <https://openedv.taobao.com>

正点原子官方网站 : www.alientek.com

正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。



扫码关注正点原子公众号



免责声明

本文档所提及的产品规格和使用说明仅供参考，如有内容更新，恕不另行通知；除非有特殊约定，本文档仅作为产品指导，所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有，未经公司的书面许可，任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息，请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	正式发布	正点原子	正点原子	2025.04.01
V1.1	新增 4.4 小节字符设备驱动开发实验	正点原子	正点原子	2025.04.21

目录

第一章 ATK-DLMP257B 开发板简介	1
1.1 STM32MP257 处理器简介	1
1.2 ATK-DL257B 开发板硬件接口说明	2
1.3 ATK-CLMP257B 核心板硬件接口说明	8
1.4 开发板网盘资料说明	9
第二章 开发环境搭建	10
2.1 从零搭建开发环境虚拟机	10
2.2 使用正点原子搭建好的虚拟机（推荐）	10
2.3 网络相关环境搭建	10
第三章 内核源码基础使用说明	11
3.1 内核源码目录介绍	11
3.2 内核源码编译	12
3.2.1 单独编译内核 Image.gz	16
3.2.2 单独编译设备树 dtb	17
3.2.3 单独编译内核模块	18
3.3 设备树说明	19
3.4 内核模块说明	20
3.5 添加内核配置及菜单选项	21
3.5.1 内核菜单 menuconfig	21
3.5.2 stm32mp257_atk_defconfig 说明	25
3.5.3 配置文件的使用	25
3.5.4 修改并保存 menuconfig	26
第四章 嵌入式 Linux 内核驱动基础	29
4.1 嵌入式 Linux 内核思路	29
4.1.1 分层思想	29
4.1.2 sysfs 接口	30
4.2 驱动程序框架	32
4.3 设备模型	33
4.4 字符设备驱动开发实验	34
4.4.1 字符设备驱动编写	34
4.4.2 编写测试 APP	39
4.4.3 编译驱动程序	41
4.4.4 编译测试应用程序	43
4.5 Platform 平台驱动	43
第五章 Pinctrl 子系统	45
5.1 Pinctrl 子系统简介	45
5.1.1 主要特点	45
5.1.2 组成部分	45
5.1.3 应用场景	46
5.2 Pinctrl 和内核其他子系统的关系	46
5.3 拓展: Pinctrl 使用案例	47

第六章 LED 子系统.....	48
6.1 LED 基础知识.....	48
6.2 LED 子系统简介.....	49
6.2.1 主要特点.....	49
6.2.2 组成部分.....	49
6.2.3 应用场景.....	51
6.3 开发板 LED 设备.....	52
6.3.1 原理图分析.....	52
6.3.2 实物硬件.....	53
6.4 内核 LED 子系统使能.....	53
6.5 设备树文件分析.....	54
6.6 LED 测试方法.....	55
6.6.1 使用 sysfs 方式操控 LED.....	55
第七章 PWM 子系统.....	57
7.1 PWM 基础知识.....	57
7.2 PWM 子系统简介.....	58
7.2.1 主要特点.....	58
7.2.2 组成部分.....	58
7.2.3 应用场景.....	59
7.3 MP257 的 PWM 说明.....	59
7.3.1 文档和资源.....	59
7.3.2 特性说明.....	60
7.4 开发板 PWM 接口.....	61
7.4.1 原理图分析.....	61
7.4.2 实物硬件.....	61
7.5 内核 PWM 子系统使能.....	62
7.6 设备树文件分析.....	63
7.7 PWM 测试方法.....	65
7.7.1 使用 sysfs 方式操控 PWM.....	65
7.7.2 目标管脚 PWM 测试.....	68
7.8 拓展: 移除 PB10 的 PWM 功能.....	70
第八章 GPIO 子系统.....	72
8.1 GPIO 基础知识.....	72
8.2 GPIO 子系统简介.....	73
8.2.1 主要特点.....	73
8.2.2 组成部分.....	73
8.2.3 应用场景.....	73
8.3 MP257 的 GPIO 说明.....	74
8.3.1 文档和资源.....	74
8.3.2 特性说明.....	74
8.4 开发板 GPIO 接口.....	77
8.4.1 原理图分析.....	77
8.4.2 实物硬件.....	78



8.5 内核 GPIO 子系统使能	78
8.6 设备树文件分析	79
8.7 GPIO 测试工具-libgpiod	79
8.7.1 gpiodetect	81
8.7.2 gpioinfo	82
8.7.3 gpioget	85
8.7.4 gpioset	86
8.7.5 gpiomon	88
8.7.6 gpionotify	90
8.8 拓展: 新增 GPIO 接口配置实验	92
8.8.1 确认目标 GPIO 信息	92
8.8.2 内核 GPIO 子系统使能	92
8.8.3 源码编译及固件更新	92
8.8.4 GPIO 测试	92
第九章 Input 子系统	95
9.1 Input 子系统简介	95
9.1.1 主要特点	95
9.1.2 组成部分	95
9.1.3 应用场景	95
9.2 开发板 Input 设备	96
9.2.1 原理图分析	96
9.2.2 实物硬件	97
9.3 内核 Input 子系统使能	97
9.4 设备树文件分析	98
9.5 按键 Input 测试方法	99
第十章 ADC 子系统	102
10.1 ADC 基础知识	102
10.2 ADC 子系统简介	103
10.2.1 主要特点	103
10.2.2 组成部分	103
10.2.3 应用场景	103
10.3 MP257 的 ADC 说明	104
10.3.1 文档和资源	104
10.3.2 特性说明	105
10.4 开发板 ADC 设备	106
10.4.1 原理图分析	106
10.4.2 实物硬件	106
10.5 内核 ADC 子系统使能	107
10.6 设备树文件分析	107
10.7 ADC 测试方法	110
第十一章 IIO 子系统	111
11.1 IIO 子系统简介	111
11.1.1 主要特点	111

11.1.2 组成部分.....	111
11.2 开发板 IIO 设备.....	112
11.3 内核 IIO 子系统使能.....	112
第十二章 I2C 子系统.....	114
12.1 I2C 基础知识.....	114
12.2 I2C 子系统简介.....	115
12.3 主要特点.....	115
12.4 组成部分.....	115
12.5 MP257 的 I2C 说明.....	117
12.5.1 文档和资源.....	117
12.5.2 特性说明.....	118
12.6 开发板 I2C 设备.....	119
12.6.1 原理图分析.....	119
12.6.2 实物硬件.....	119
12.7 内核 I2C 子系统使能.....	120
12.8 AT24C64 驱动配置.....	121
12.9 设备树文件分析.....	122
12.10 EEPROM 的使用.....	124
12.10.1 EEPROM 读写测试.....	125
12.10.2 读取 EEPROM 内容.....	125
12.10.3 清除 EEPROM 中的数据.....	126
12.10.4 写入数据到 EEPROM.....	126
12.11 I2C 测试工具.....	127
12.11.1 i2cdetect.....	127
12.11.2 i2cdump.....	129
12.11.3 i2cset.....	132
12.11.4 i2cget.....	133
12.11.5 i2ctransfer.....	133
第十三章 SPI 子系统.....	136
13.1 SPI 基础知识.....	136
13.2 SPI 子系统简介.....	137
13.3 主要特点.....	137
13.4 组成部分.....	138
13.5 MP257 的 SPI 说明.....	139
13.5.1 文档和资源.....	139
13.5.2 特性说明.....	140
13.6 开发板 SPI 设备.....	141
13.6.1 原理图分析.....	141
13.6.2 实物硬件.....	142
13.7 内核 SPI 子系统使能.....	142
13.8 设备树文件分析.....	143
13.9 SPI 调试工具.....	145
13.9.1 spidev_test.....	145



13.9.2 spidev_fdx.....	146
第十四章 MTD 子系统.....	147
14.1 MTD 子系统简介.....	147
14.2 主要特点.....	147
14.3 组成部分.....	147
14.4 开发板 MTD 设备.....	148
14.4.1 原理图分析.....	148
14.4.2 实物硬件.....	149
14.5 SPI FLASH 驱动配置.....	150
14.6 设备树文件分析.....	151
14.7 MTD 设备作为块设备使用.....	152
14.8 MTD 调试工具.....	153
14.8.1 mtdinfo.....	153
14.8.2 mtd_debug.....	154
14.8.3 mtdpart.....	156
14.8.4 Sysfs 接口.....	157
第十五章 RTC 子系统.....	159
15.1 RTC 子系统简介.....	159
15.2 主要特点.....	159
15.3 组成部分.....	159
15.4 开发板 RTC 设备.....	160
15.4.1 原理图分析.....	160
15.4.2 实物硬件.....	161
15.5 内核 RTC 子系统使能.....	162
15.6 RTC 驱动配置.....	163
15.7 设备树文件分析.....	164
15.8 RTC 调试工具.....	165
15.8.1 注意事项.....	165
15.8.2 dmesg 命令.....	165
15.8.3 hwclock 工具.....	166
15.8.4 timedatectl.....	169
15.8.5 rtcwake 工具.....	172
第十六章 UART 子系统.....	178
16.1 UART 子系统简介.....	178
16.2 主要特点.....	178
16.3 组成部分.....	178
16.4 MP257 的 UART 说明.....	179
16.4.1 文档和资源.....	179
16.4.2 特性说明.....	180
16.5 开发板 UART 设备.....	183
16.5.1 原理图分析.....	183
16.5.2 实物硬件.....	184
16.6 内核 UART 子系统使能.....	185

16.7 CH342F 驱动配置	185
16.8 设备树文件分析	186
16.9 系统接口	188
16.9.1 /dev/ttySTM*	188
16.9.2 /sys/class/tty/ttySTM*	188
16.9.3 /sys/devices/platform/xxx	189
16.10 USART 调试工具	191
16.10.1 dmesg	191
16.10.2 stty	191
16.10.3 minicom	193
第十七章 RS232/RS485 子系统	194
17.1 RS485 子系统简介	194
17.2 主要特点	194
17.3 组成部分	194
17.4 开发板 RS232 设备	195
17.5 开发板 RS485 设备	197
17.5.1 原理图分析	197
17.5.2 实物硬件	198
17.6 内核 RS232、RS485 子系统使能	198
17.7 系统接口	198
17.8 RS232、RS485 调试工具	199
17.8.1 stty	199
17.8.2 minicom	199
17.9 具体使用示例	203
17.9.1 RS232	203
17.9.2 RS485	205
第十八章 CAN 子系统	208
18.1 CAN 子系统简介	208
18.2 主要特点	208
18.3 组成部分	208
18.4 MP257 的 CAN 说明	209
18.4.1 文档和资源	209
18.4.2 特性说明	210
18.5 开发板 CAN 设备	211
18.5.1 原理图分析	211
18.5.2 实物硬件	212
18.6 内核 CAN 子系统使能	212
18.7 设备树分析	213
18.8 系统接口	216
18.9 CAN 调试工具	217
18.9.1 使能 CAN 接口	217
18.9.2 candump	220
18.9.3 cansend	222



18.9.4 cangen	225
18.9.5 canplayer	228
第十九章 MMC 子系统	233
19.1 MMC 子系统简介	233
19.2 主要特点	233
19.3 组成部分	234
19.4 MP257 的 MMC 说明	236
19.4.1 文档和资源	236
19.4.2 特性说明	237
19.5 开发板 TF Card 接口	239
19.5.1 原理图分析	239
19.5.2 硬件实物	241
19.6 内核 MMC 子系统使能	241
19.7 设备树分析	242
19.8 系统接口	244
第二十章 Display 子系统	246
20.1 Display 子系统简介	246
20.2 主要特点	246
20.3 组成部分	246
20.4 MP257 的 Display 说明	247
20.4.1 文档和资源	247
20.4.2 特性说明	248
20.5 开发板 RGBLCD 接口	250
20.5.1 原理图分析	250
20.6 开发板 MIPI DSI 接口	252
20.6.1 原理图分析	252
20.6.2 实物硬件	253
20.7 开发板 LVDS 接口	255
20.7.1 原理图分析	255
20.7.2 实物硬件	256
20.8 内核显示功能使能	257
20.8.1 RGBLCD 使能	257
20.8.2 MIPI DSI 使能	258
20.8.3 LVDS 使能	259
20.9 设备树分析	259
20.9.1 RGB 设备树	261
20.9.2 MIPI 屏幕设备树	266
20.9.3 LVDS 设备树	271
20.10 Display 工具	276
20.10.1 fbset	276
20.10.2 modetest	277
20.10.3 drmdevice	280
第二十一章 Ethernet 子系统	283



21.1 Ethernet 子系统简介	283
21.2 组成部分	283
21.3 主要功能	283
21.4 MP257 的 Ethernet 说明	284
21.4.1 文档和资源	284
21.4.2 特性说明	284
21.5 开发板 Ethernet 接口	285
21.5.1 原理图分析	285
21.5.2 硬件实物	287
21.6 内核 Ethernet 使能	288
21.6.1 网络功能使能	288
21.6.2 通用 PHY 使能	289
21.6.3 STM32 DWMAC 使能	290
21.7 设备树分析	290
21.8 ETH 工具	296
21.8.1 dmesg	296
21.8.2 ifconfig	297
21.8.3 udhcpc	298
21.8.4 ip	301
第二十二章 Camera 子系统	306
22.1 Camera 子系统简介	306
22.2 组成部分	306
22.3 MP257 的 Camera 说明	306
22.3.1 文档和资源	306
22.3.2 特性说明	307
22.4 开发板 Camera 接口	309
22.4.1 原理图分析	309
22.4.2 硬件实物	311
22.5 内核 Camera 相关使能	312
22.5.1 DCMIPP 驱动使能	312
22.5.2 IMX335 驱动使能	313
22.6 设备树分析	314
22.7 Camera 工具	318
22.7.1 dmesg	318
22.7.2 v4l2-sysfs-path	319
22.7.3 v4l2-compliance	319
22.7.4 v4l2-ctl	321
22.7.5 gst-launch-1.0	327
第二十三章 Audio 子系统	329
23.1 Audio 子系统简介	329
23.2 组成部分	329
23.3 MP257 的 Audio 说明	330
23.3.1 文档和资源	330

23.3.2 特性说明.....	331
23.4 开发板 Audio 接口.....	331
23.4.1 原理图分析.....	331
23.4.2 硬件实物.....	337
23.5 内核 Audio 相关使能.....	339
23.6 设备树分析.....	340
23.7 ALSA 工具.....	346
23.7.1 alsamixer.....	346
23.7.2 amixer.....	347
23.8 音频测试.....	356
23.8.1 播放音频测试.....	356
23.8.2 录音测试.....	357
第二十四章 WiFi 子系统.....	359
24.1 WiFi 子系统简介.....	359
24.2 组成部分.....	359
24.3 开发板 WiFi 说明.....	360
24.3.1 原理图分析.....	360
24.3.2 硬件实物.....	361
24.4 内核 WiFi 相关使能.....	361
24.4.1 无线框架使能.....	361
24.4.2 WIFI 驱动使能.....	362
24.5 WiFi 工具.....	363
24.5.1 connmanctl.....	363
24.5.2 iw.....	368
第二十五章 Bluetooth 子系统.....	371
25.1 Bluetooth 子系统简介.....	371
25.2 组成部分.....	371
25.3 开发板 Bluetooth 说明.....	372
25.3.1 原理图分析.....	372
25.3.2 硬件实物.....	373
25.4 内核 Bluetooth 相关使能.....	374
25.4.1 蓝牙框架使能.....	374
25.5 板载蓝牙测试.....	374
25.5.1 蓝牙配对.....	374
第二十六章 4G/5G 子系统.....	379
26.1 4G/5G 子系统简介.....	379
26.2 组成部分.....	379
26.3 开发板 4G/5G 接口说明.....	381
26.3.1 原理图说明.....	381
26.3.2 硬件实物.....	382
26.4 内核 4G/5G 相关使能.....	383
26.5 4G 模块 EC20 测试.....	384
26.5.1 quectel-CM 拨号上网.....	385

26.5.2 pppd 拨号上网.....	387
26.5.3 GPS 功能测试	389
26.6 4G 模块 ME3630-W 测试.....	389
26.6.1 ECM 上网.....	390
26.6.2 pppd 拨号上网.....	391
26.6.3 GPS 功能测试	393

第一章 ATK-DLMP257B 开发板简介

1.1 STM32MP257 处理器简介

STM32MP257 系列处理器是一款高性能、低功耗的异构多核处理器，集成 Cortex-A35 + Cortex-M33，提供强大的计算能力，并具备丰富的连接接口（以太网、CAN、PCIe、USB、I²C、SPI、UART）。它支持 AI 硬件加速、3D GPU、多媒体视频编解码、ISP 和摄像头接口，适用于图像处理和嵌入式计算。完善的安全机制（TrustZone、SHA、HMAC、防篡改）确保数据安全，广泛适用于工业控制、AI 计算、实时控制和电机驱动等应用场景。

STM32MP257 系列处理器简化框图如下：



图 1.1-1 STM32MP257 处理器框图

1.2 ATK-DL257B 开发板硬件接口说明

首先看一下 ATK-DLMP257B 开发板的底板资源图，如下图所示：

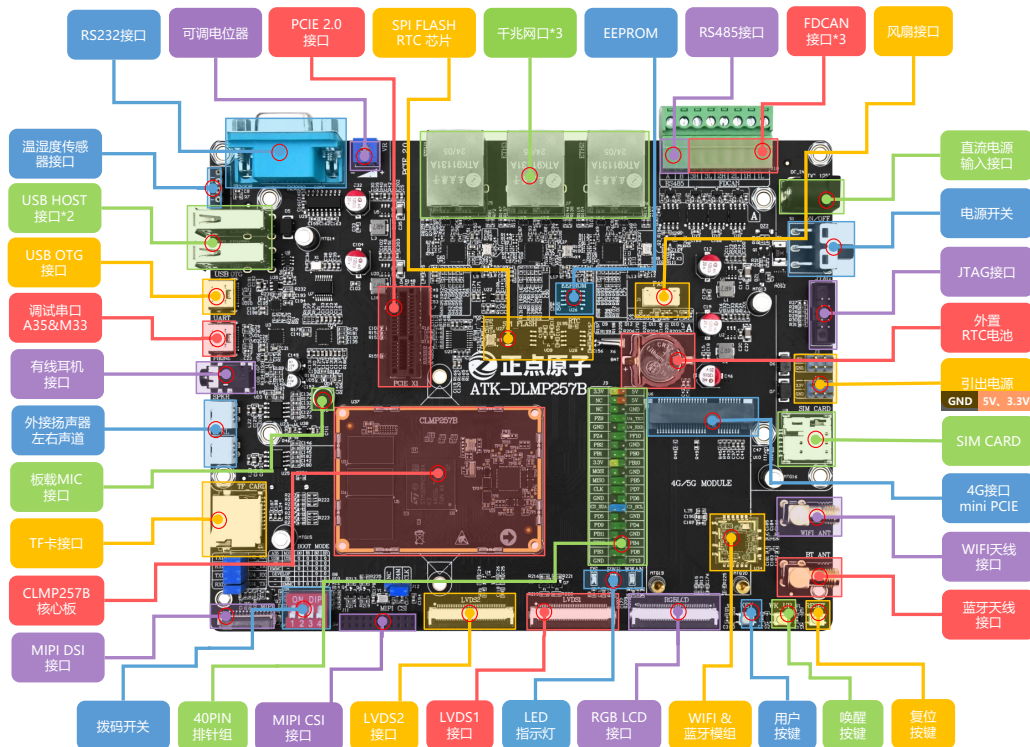


图 1.2-1 ATK-DLMP257B 开发板底板资源图（正面）

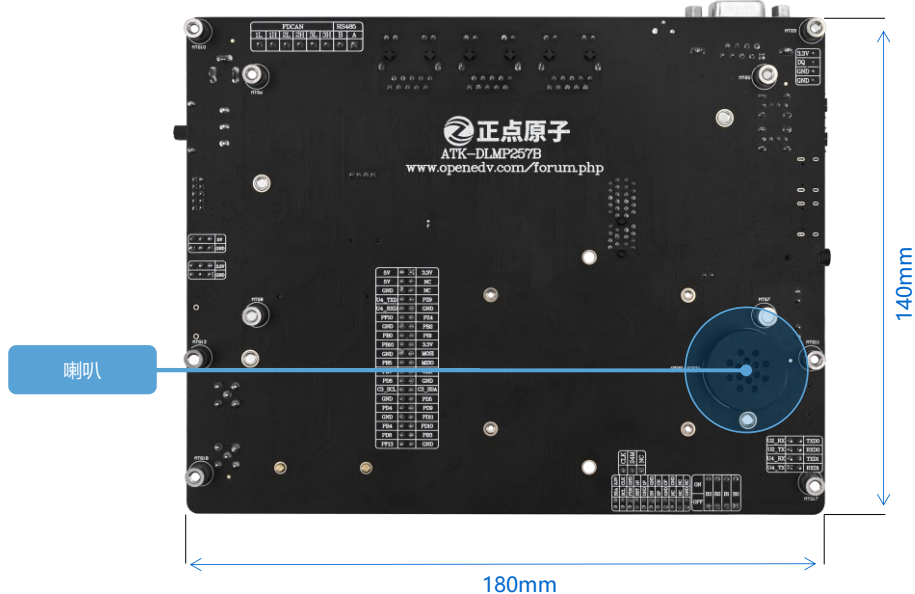


图 1.2-2 ATK-DLMP257B 开发板底板资源图（背面）

从上图可以看出，ATK-DLMP257B 开发板底板资源十分丰富，扩充了丰富的接口和功能模块。开发板的外形尺寸为 180mm*140mm 大小，布局充分考虑了人性化设计，便于开发使用。

正点原子 ATK-DLMP257B 开发板底板板载资源如下：

- ◆ 1 组核心板连接座接口，支持 ATK-CLMP257B 核心板
- ◆ 1 路 DC12V 电源输入接口



- ◆ 1 个电源开关, 控制整板电源
- ◆ 1 路 JTAG 调试接口
- ◆ 1 组 5V 电源输出口
- ◆ 1 组 3.3V 电源输出口
- ◆ 1 路 Mini PCIE 接口
- ◆ 1 路 Nano SIM 卡接口
- ◆ 1 路 WIFI 天线接口
- ◆ 1 路 BT 蓝牙天线接口
- ◆ 1 个 USB WIFI&BT 模组
- ◆ 1 个复位按键
- ◆ 1 个唤醒按键
- ◆ 1 个用户按键
- ◆ 1 个 4G 模块工作指示灯
- ◆ 1 个蓝色电源指示灯
- ◆ 1 个红色用户指示灯
- ◆ 1 路 RGB 屏幕接口
- ◆ 2 路 LVDS 屏幕接口
- ◆ 1 路 MIPI CSI 摄像头接口
- ◆ 1 个启动模式拨码开关
- ◆ 1 路 MIPI DSI 屏幕接口
- ◆ 1 组调试串口排针组
- ◆ 1 路 TF 卡接口
- ◆ 1 个高性能音频编解码芯片, ES8388
- ◆ 2 路扬声器外接接口
- ◆ 1 路耳机接口, 支持 4 段式耳机
- ◆ 1 个驻极体麦克风 MIC
- ◆ 1 个小扬声器 (板卡背面)
- ◆ 1 路 UART 调试串口, Type-C 接口类型
- ◆ 1 路 USB_OTG 接口, Type-C 接口类型
- ◆ 2 路 USB2.0 HOST 接口
- ◆ 1 路温湿度传感器接口
- ◆ 1 路 RS232 接口, 标准 DB9 母头
- ◆ 1 个可调电位器, ADC 电压采集
- ◆ 1 路 PCIE 接口, PCIE2.0
- ◆ 3 路 10M/100M/1000M 以太网接口 (RJ45)
- ◆ 1 路 RS485 接口
- ◆ 3 路 FDCAN 接口
- ◆ 1 路风扇接口
- ◆ 1 个 I2C EEPROM 芯片, 8KB
- ◆ 1 个 SPI NOR FLASH 芯片, 16MB
- ◆ 1 个 RTC 实时时钟芯片
- ◆ 1 个 RTC 纽扣电池座, 带电池
- ◆ 1 组 2×20P 2.54mm 间距的排针组, 引出数十个 IO



正点原子 ATK-DLMP257B 开发板底板的特点包括:

- 1)、接口丰富。开发板提供多种标准接口, 可以方便地进行各种外设的实验和开发。
- 2)、设计灵活。采用核心板+底板形式, 板上很多资源都可以灵活配置, 以满足不同条件下的使用。核心板引出了 STM32MP257DAK3 处理器几乎所有通信接口, 包含 UART、I2C、I3C、SPI、FDCAN、SDMMC、USB2.0、USB3.0、PCIE、MIPI DSI、MIPI CSI、LVDS、RGB、ADC、千兆以太网等, 极大地方便用户扩展及使用。
- 3)、资源丰富。板载高性能音频编解码芯片、千兆网卡、EEPROM 存储芯片以及各种接口芯片, 满足多种应用需求。
- 4)、人性化设计。各个接口都有丝印标注, 使用起来一目了然; 部分常用外设大丝印标出, 方便查找; 接口位置设计合理, 方便使用。

这里详细介绍 ATK-DLMP257B 开发板的各个部分, 笔者将按逆时针的顺序依次介绍:

1、DC12V 电源输入

开发板板载 1 个外部电源输入口 (DC_IN), 采用标准直流电源插座, 给开发板供电时需输入 12V 直流电源。开发板板载多个 DC-DC 电源芯片, 用于提供高效、稳定的 5V、3.3V 等。

2、电源开关

开发板板载 1 个电源开关 (S1)。该开关用于控制整个开发板的供电。这是一个两段式拨动开关, 拨到上边打开电源, 整个开发板开始供电, 电源指示灯 (PWR) 点亮。拨到下边关闭电源, 整个开发板都将断电, 电源指示灯 (PWR) 会随之熄灭。

3、JTAG 接口

这是一个 10P, 2.0mm 间距的 JTAG 接口, 可连接 ST-LINK 仿真调试器, 一般可用来调试 Cortex-M33 核。

4、5V 电源输出

开发板板载 1 组 5V 电源输出排针 (2*3, J3), 用于给外部提供 5V 电源, 具备防反接功能。用户在实验时可能会因为没有 5V 电源而苦恼不已, 有了这组 5V 排针, 就可以很方便地拥有一组简单的 5V 电源。

5、3.3V 电源输出

开发板板载 1 组 3.3V 电源输出排针 (2*3, J2), 用于给外部提供 3.3V 电源, 具备防反接功能。用户在实验时如果需要 3.3V 电源, 那么就可以使用此电源。注意不能接功率太大的设备, 最大电流建议不超过 1000mA, 一般用来接一些传感器之类的小功率器件或模块。

6、Nano SIM 卡接口

开发板板载 1 路 Nano SIM 卡接口, 如果需要使用 4G/5G 模块进行无线数据传输, 就需要在此接口插入 Nano SIM 卡。

7、Mini PCIE 接口

开发板板载 1 个 Mini PCIE 座, 实质上是使用 USB 接口, 连接到板载 USB HUB 芯片上。此接口可以连接 4G 或 5G RedCap 模组, 比如高新兴物联 ME3630 4G 模组、移远 EC20 4G 模组、广和通 5G RedCap FG132 模组等。此处补充说明, 对于 5G RedCap 模组, 开发板目前仅适配广和通 5G RedCap FG132 模组, 若用户使用其他品牌模组, 需检查该接口是否硬件兼容。

8、WIFI 天线接口

开发板板载 1 路 WIFI 天线接口, 为开发板板载 WIFI & Bluetooth 一体模组提供 WIFI 无线连接信号。该模组使用瑞昱 (REALTEK) 公司的 RTL8733BU 芯片, USB2.0 接口, 支持 WIFI4 无线双频 2.4G/5.8G 和蓝牙 5.2 版本协议。该模块支持双天线, 也支持 WiFi 和蓝牙共用一根天线, 开发板采用双天线设计。由于该模组支持 5G 频段 WIFI, 故使用 WIFI 功能时建议使用 2.4G &

5G 双频天线。

9、蓝牙天线接口

开发板板载 1 路蓝牙天线接口，为开发板板载 WIFI & Bluetooth 一体模组提供蓝牙无线连接信号，该接口可连接 2.4G 天线。

10、复位按键

开发板板载 1 个复位按键(RESET)，当用户需要将处理器复位重启时，可使用该按键复位处理器。

11、休眠唤醒按键

开发板板载 1 个休眠唤醒按键(WK_UP)，当用户对处理器执行休眠操作后需要唤醒时，可使用该按键唤醒处理器。

12、用户按键

开发板板载 1 个用户按键(KEY)，用户可使用该按键进行自定义按键操作。

13、WWAN 指示灯

开发板板载 1 个 4G\5G 模块工作指示灯(WWAN)，当 4G\5G 模块处于不同工作状态时，该指示灯亮灭状态不同。

14、电源指示灯

开发板板载 1 个电源指示灯(PWR)，当开发板正常供电 3.3V 时，该指示灯为常亮状态。

15、用户指示灯

开发板板载 1 个用户指示灯(DS)，用户可使用该指示灯自定义指示程序运行状态。

16、RGB 接口

开发板板载 1 路 RGB LCD 屏幕接口，可连接正点原子 RGB LCD 触摸屏幕，已适配支持 4.3 寸 800x480、7 寸 800x480、7 寸 1024x600、10.1 寸 1280x800 分辨率屏幕。注意，由于开发板默认设计功能较多，该 RGB 接口受限于 GPIO 引脚数量，仅使用 RGB R3~R7、G2~G7、B3~B7 共 16 个 IO 作显示接口，故该接口仅支持 RGB LCD 屏幕 RGB565 格式。

17、LVDS1 接口

开发板板载 2 路 4-lanes LVDS 接口，分别为 LVDS1、LVDS2 接口，均为 4 通道，可连接正点原子 LVDS LCD 触摸屏幕，已适配支持 10.1 寸 1280x800 分辨率 LVDS 屏幕。该 LVDS1 接口可单独使用，独立接入单块 LVDS 屏幕进行显示。

18、LVDS2 接口

开发板支持 LVDS2 接口与 LVDS1 接口共同使用，支持 LVDS1 和 LVDS2 双屏同显，即 LVDS2 接口屏幕能够显示和 LVDS1 接口屏幕一致的内容。另外 LVDS2 接口支持与 LVDS1 接口形成 8 通道，接入单块 8 通道 LVDS 屏幕进行显示。

19、MIPI CSI 接口

开发板板载 1 路 2-lanes MIPI CSI 摄像头接口，支持带 ISP 的 MIPI CSI 摄像头，用于图像采集、处理，可将图像采集提供给 NPU 处理器进行 AI 模型运算处理等。该接口适配支持正点原子 IMX335 摄像头模块，可将摄像头模块插入到该接口使用，镜头方向向外侧。

20、启动模式拨码开关

开发板板载 1 个 4x2P 红色拨码开关，用于选择 STM32MP257 处理器启动模式。处理器支持从 SD 卡、EMMC、USB 多种方式启动，用户将拨码开关引脚拨动至对应的启动模式即可。启动模式配置已标记在开发板丝印上，便于用户查看选择。需注意的是，处理器 STM32MP257 为异核处理器，支持运行 Cortex-A35 核和 Cortex-M33 核，如果跑 Linux 系统，则只需设置 Cortex-A35 核启动模式。

21、MIPI DSI 接口

开发板板载 1 路 4-lanes MIPI DSI 屏幕接口, 可连接正点原子 MIPI LCD 触摸屏, 已适配支持 5.5 寸 720x1280、5.5 寸 1080x1920、10.1 寸 800x1280 分辨率 MIPI 屏幕。

22、调试串口排针组

开发板板载一组调试串口排针组(2*4, J4), 为预留跳线接口, 供用户测试使用。

23、TF 卡接口

开发板板载 1 路标准 TF 卡接口 (TF_CARD), 采用小型 TF 卡接口, SDIO 方式驱动。有了这个 TF 卡接口, 就可以满足大容量数据存储需求。另外, STM32MP2 处理器支持 SD 卡启动模式, 可运行烧写在 TF 卡内的系统镜像。

24、扬声器外接接口

开发板板载 2 路扬声器外接接口, 其中一路扬声器接口 SPKL 已在开发板接入一个小扬声器, 方便用户进行音频播放测试。

25、耳机接口

开发板板载 1 个四段式耳机接口。耳机座规格为 PJ-342 单柱带 T, 该接口可以插入四段式 3.5mm 的耳机, 支持录音与放音。放音时, 支持耳机热插拔; 录音时, 支持耳机录音。

26、扬声器 (在背面)

开发板板载 1 个外置扬声器 (背面), 扬声器型号为 GSPK2307P-8R1W, 功率为 8Ω、1W, 供用户进行音频播放测试。当音频芯片 ES8388 播放音频时, 开发板支持从扬声器播放及耳机播放。在不插入耳机时, 开发板自动从外置扬声器进行播放; 当插入耳机时, 会自动切换到耳机播放音频, 此时扬声器自动停止播放。该设计符合用户常规使用体验。

27、驻极体麦克风 MIC

开发板板载 1 个驻极体麦克风, 即录音输入口 (MIC), 可以用来实现录音功能。

28、USB 调试串口

开发板板载 1 路 USB 调试串口, 为 USB Type-C 接口, 用于 USB 连接 CH342F 芯片, 从而实现 1 路 USB 转 2 路串口功能, 作为系统调试串口。在开发板中, USART2 为 Cortex-A35 核调试串口, UART4 为 Cortex-M33 核调试串口。

29、USB_OTG 接口

开发板板载 1 路 USB OTG 接口, 为 USB Type-C 接口。此接口通过使用 USB Type-C 控制器芯片 FUSB302MPX, 实现 USB OTG 功能。可以使用 USB Type-C 线将开发板连接到电脑上, 此时开发板作为一个从机 Slave, 进行系统烧写。也可以使用 USB OTG 线连接其他 USB 设备, 比如 U 盘等, 此时开发板就作为主机 Host, 对设备进行读写操作。

30、USB HOST 接口

开发板板载 2 路 USB Type-A 座接口, 可接入 U 盘、键盘、鼠标、USB 摄像头等设备。开发板通过 USB HUB 芯片将 STM32MP257 处理器的 USBH 总线扩展成 4 路 USB HOST, 其中 1 路用于连接 4G/5G 模块, 另外 1 路用于连接 WIFI&蓝牙模组, 剩下 2 路 USB 接口通过 USB Type-A 座引出。

31、温湿度传感器接口

开发板板载 1 路温湿度传感器接口(SENSOR), 可接入 DHT11 和 DS18B20 温湿度传感器进行温湿度测试。

32、RS232 接口

开发板板载 1 路 RS232 接口, 采用标准 DB9 母头接口, 可进行 RS232 数据收发通信。

33、可调电位器

开发板板载 1 个可调电位器, 电阻阻值为 10K, 用于进行 ADC 电压采集测试。

34、PCIE 接口

开发板板载 1 路 PCIE 接口, 接口为 36PIN, 支持 PCIE 2.0 版本协议, 可接入 PCIE 接口设备, 比如 PCIE 接口 WIFI 模块、PCIE 接口硬盘、PCIE 接口千兆以太网模块等。

35、千兆以太网接口

开发板板载 3 路千兆以太网接口, 支持 10M/100M/1000M 速率。STM32MP257 处理器内置双端口交换机, 其中网口 1 和网口 3 为交换机接口, 网口 2 为独立千兆以太网接口。支持 TSN 时间敏感网络等功能。

36、RS485 接口

开发板板载 1 路 RS485 总线接口(RS485), 具备 ESD 防护功能。RS485 通信时, 需要板载端口 A 接对方设备端口 A, 板载端口 B 接对方设备端口 B。

37、FDCAN 接口

开发板板载 3 路 FDCAN 总线接口(支持 CAN 和 CAN-FD), 具备 ESD 防护功能。CAN、CAN-FD 设备通信时, 必须 CANH 接 CANH, CANL 接 CANL, 否则不能正常通信。

38、I2C EEPROM

开发板板载一个 EEPROM 存储芯片, 型号为 AT24C64, 容量为 8KB, IIC 接口, 供用户存储测试使用。

39、SPI NOR FLASH

开发板板载一个 SPI NOR FLASH 存储芯片, 型号为 W25Q128, 容量为 16MB, SPI 接口, 供用户存储测试使用。

40、RTC 实时时钟

开发板板载 1 个 RTC 实时时钟芯片 AT8563T, IIC 接口。STM32MP257 处理器内部也有 RTC 外设, 但是当 RTC 纽扣电池供电 STM32MP257 处理器内置 RTC 时, 不仅仅只供电 RTC 部分, 还会对 STM32MP257 处理器 VSW 电源域进行供电, 从而导致纽扣电池耗电量增加。因此开发板外置一个 RTC 实时时钟芯片, 供用户测试使用。

41、RTC 纽扣电池座

开发板板载 1 个 RTC 纽扣电池座, 安装上纽扣电池后, 给外置 RTC 实时时钟芯片供电, 在开发板断电后能够维持 RTC 时钟芯片计时。开发板出厂时, 默认安装上一颗纽扣电池 CR1220, 供测试使用。需注意, 在开发板电路设计中, 该纽扣电池在开发板断电后也同时供电 STM32MP257 处理器内置 RTC 外设等, 因此相对于独立供电外置 RTC 芯片 AT8563T, 耗电量会有所增加, 供电电流大于 10uA。

42、风扇接口

开发板板载一路风扇接口, 可按散热需要接入小风扇对核心板处理器、存储芯片等进行散热, 供用户测试使用。该风扇接口电路对外供电 12V, 接入风扇时需检查风扇设备供电电压范围及控制 IO 口位置。

43、引出 IO 排针组

开发板板载一组引出 IO 排针组(J9), 2x20PIN, 共引出 28 个 GPIO 引脚及 5V、3.3V、GND 引脚。需注意该引出 IO 口非独立使用, 大多数与开发板所板载功能电路引脚共用。故用户在使用这些引出 IO 时, 需检查是否与板载功能电路存在冲突情况, 比如是否存在上下拉电阻、IO 是否已连接到其他功能芯片等, 从而导致 IO 电平值不对、外接模块通信失败等。如果受到影响, 则需要将开发板板载电路、芯片进行暂时焊接移除处理。

44、ATK-CLMP257B 核心板接口

开发板板载一组核心板接口, 由 4 个 2*40、0.4mm 间距的贴片板对板连接座公座组成, 共 320PIN。该接口用来连接正点原子 ATK-CLMP257B 核心板, 从而进行 STM32MP257 处理器的系统开发、产品评估。

1.3 ATK-CLMP257B 核心板硬件接口说明

下面来看 ATK-CLMP257B 核心板，如下图所示：

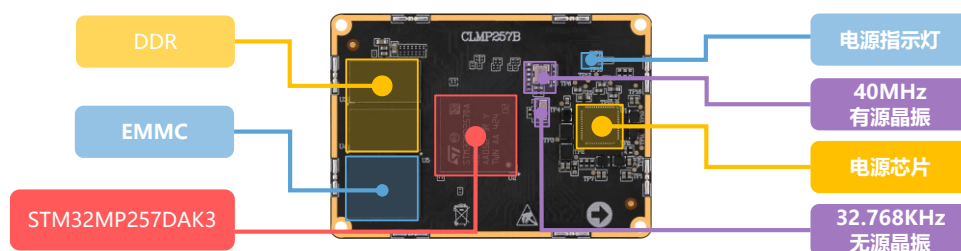


图 1.3-1 ATK-CLMP257B 核心板正面

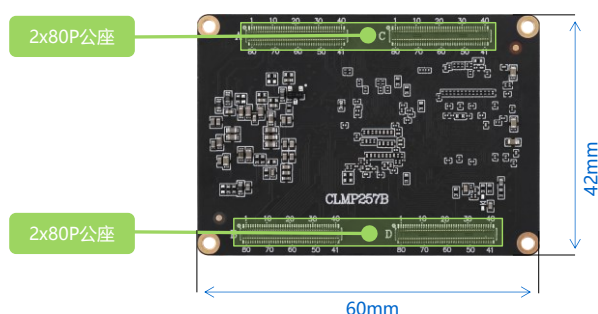


图 1.3-2 ATK-CLMP257B 核心板背面

整个核心板的外形尺寸为 60mm*42mm 大小，非常小巧，采用了 0.4mm 间距贴片板对板连接器，与底板 BTB 连接座的合高为 1.5mm，可以很方便地应用在各种项目上。

正点原子 ATK-CLMP257B 核心板板载资源如下：

- ◆ CPU：处理器 STM32MP257DAK3，双核 Cortex-A35(主频 1.5GHz)，单核 Cortex-M33 (400MHz)，VFBGA424 封装，14x14mm
- ◆ GPU：900MHz
- ◆ NPU：900MHz，1.35TOPS
- ◆ 内存 DDR4：容量为 1GB\2GB
- ◆ 存储 EMMC：容量为 8GB\16GB
- ◆ 高稳定性电源管理 PMIC：STPMIC25
- ◆ 四个 2*40P BTB 连接座，共计 320PIN

正点原子 ATK-CLMP257B 核心板的特点包括：

- 1)、体积小。核心板仅 60mm*42mm 大小，方便使用到各种项目。
- 2)、集成方便。核心板使用 4 个 2*40P BTB 连接座，可以非常方便地集成到客户 PCB 上，更换简单，方便维修测试。
- 3)、资源丰富。核心板板载不同容量 DDR4 内存、EMMC 存储器，可以满足多种应用需求。
- 4)、性能稳定。核心板采用 10 层板设计，单独地层、电源层，所有关键信号采用等长线走线，元器件布局合理；另采用 ST 针对 STM32MP2x MPU 系列应用专门设计的低功耗、高效率电源管理芯片 PMIC 进行稳定供电，保证核心板长期运行稳定、可靠。
- 5)、人性化设计。按功能分区引出 IO 口，方便布线。
- 6)、防反插设计。BTB 连接座位置错开，BTB 连接座防反插设计。

1.4 开发板网盘资料说明

在【正点原子】STM32MP257 开发板（A 盘）-基础资料\ATK-DLMP257B 资料脑图和学习路线.pdf 中有资料说明。

更多资料说明请下载开发板网盘资料。

<http://www.openedv.com/docs/boards/arm-linux/mp257.html>

第二章 开发环境搭建

2.1 从零搭建开发环境虚拟机

参考【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B Linux 基础学习手册.pdf 的 Ubuntu 系统安装章节，以及【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 开发环境搭建手册.pdf

编译文件之前需要提前安装【正点原子】STM32MP257 开发板（A 盘）-基础资料\05、开发工具\01、交叉编译器

另外需在 ubuntu24.04 环境中安装以下工具，避免编译时提示缺少工具库：

```
sudo apt-get install make gcc libssl-dev g++ git libncurses5-dev libncursesw5-dev libyaml-dev  
sudo apt-get install u-boot-tools python3-pyelftools device-tree-compiler bison flex expect bzip2
```

2.2 使用正点原子搭建好的虚拟机（推荐）

为了方便用户进行基于 ATK-DLMP257B 开发板的研发，正点原子团队特地打包了一个基础的虚拟机镜像，帮助用户顺利学习和开发。

虚拟机下载地址：<http://www.openedv.com/docs/boards/arm-linux/mp257.html>

正点原子虚拟机安装和使用方法请参考【正点原子】STM32MP257 开发板（A 盘）-基础资料\10、用户手册\【正点原子】ATK-DLMP257B 虚拟机使用参考手册.pdf

2.3 网络相关环境搭建

在开发过程中常常会用到网络文件传输，用户可以参考【正点原子】STM32MP257 开发板（A 盘）-基础资料\10、用户手册\【正点原子】Linux 开发板网络环境搭建手册.pdf 进行基础的开发环境网络搭建。

TFTP 环境搭建参考【正点原子】STM32MP257 开发板（A 盘）-基础资料\10、用户手册\【正点原子】ATK-DLMP257B 出厂系统 TFTP 搭建手册.pdf

NFS 环境搭建参考【正点原子】STM32MP257 开发板（A 盘）-基础资料\10、用户手册\【正点原子】ATK-DLMP257B 出厂系统 NFS 搭建手册.pdf

第三章 内核源码基础使用说明

3.1 内核源码目录介绍

把【正点原子】STM32MP257 开发板（A 盘）-基础资料\01、程序源码\01、正点原子 Linux 出厂系统源\linux-6.6.48-版本号.tar.bz2 拷贝到虚拟机。执行解压命令，并进入内核源码目录；（请确认自己下载的源码版本，不要直接复制下面解压命令）

```
tar -xjf linux-6.6.48-v1.0.tar.bz2 //加-v 参数可以看解压详情
sync
cd linux/linux-6.6.48
```

进入到 ATK-DLMP257B 内核源码的目录下，执行 ls 指令可以查看内核源码子目录。

```
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux$ tar -xjf linux-6.6.48-v1.0.tar.bz2
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux$ ls
linux  linux-6.6.48-v1.0.tar.bz2
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux$ cd linux/
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux$ ls
linux-6.6.48
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux$ cd linux-6.6.48/
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ ls
arch          CODE_OF_CONDUCT.md  Documentation      io_uring  lib          net          scripts      usr
block         CONTRIBUTING.md     drivers            ipc       LICENSES    pack_bootfs.sh  security     virt
bootfs_config COPYING             fs                Kbuild   MAINTAINERS  README        SECURITY.md
build_kernel.sh CREDITS             include            Kconfig  Makefile     rust          sound
certs         crypto              init              kernel   mm           samples      tools
```

图 3.1-1 内核源码文件

子目录的介绍如下：

文件/路径	说明
arch	包含不同处理器架构的特定实现代码和配置。
block	主要包含了块设备相关的驱动程序和核心子系统代码。
bootfs_config	bootfs 配置
build_kernel.sh	ATK-DLMP257B 开发板内核编译脚本。
certs	包含了一些安全相关的证书文件。
COPYING	包含 Linux 内核的开源许可证文件。
LICENSES	
CREDITS	对 Linux 内核贡献过的开发者名单。
crypto	包含了加密算法和密码学相关的内核组件。
Documentation	存放内核的文档和说明文件。
drivers	包含各种硬件设备驱动程序的源码。
fs	文件系统相关代码，包含多种 Linux 支持的文件系统的实现。
include	定义了内核的各种数据结构、常量、宏、全局变量声明及函数原型等。
init	初始化相关代码，例如内核启动时的一些基本设置。
io_uring	Linux 内核异步 I/O 接口相关的代码。
ipc	包含了实现内核级进程间通信机制的相关代码。
Kbuild	一种高度自定义的 Makefile 系统，用于指导内核及模块的编译过程。

Kconfig	内核配置文件，用户可以通过配置工具选择内核编译选项。
kernel	内核核心部分的源码，包括调度器、内存管理等。
lib	内核公用函数库和数据结构的源码。
LICENSES	包含了一系列与内核源码相关的许可协议文件。
MAINTAINERS	各个内核子系统、驱动程序和其他组件的主要维护者及其联系信息。
Makefile	控制整个内核编译过程的核心脚本。
mm	包含了内核内存管理子系统的源代码。
net	网络子系统相关代码。
README	包含该版本 Linux 内核的基本信息、内核特性等内容。
pack_bootfs.sh	打包 bootfs.ext4 的脚本
rust	与 Rust 语言相关的内核组件。
samples	包含一些示例代码或内核模块样本。
scripts	包含用于构建、配置、测试、转换以及其他辅助内核编译的脚本文件。
security	安全相关的内核子系统代码。
sound	音频子系统相关代码。
tools	辅助工具集，用于内核开发和分析。
usr	包含了一些与用户空间相关的链接或配置文件。
virt	包含与虚拟化相关的代码和子系统。

3.2 内核源码编译

编译出厂源码时，用户不必修改任何内容。适配于 ATK-DLMP257B 开发板硬件资源的 Linux 内核源码设备树路径为 arch/arm64/boot/dts/st，以 2GB 版本为例，设备树文件为 stm32mp257d-atk-ddr-2GB.dts、stm32mp257d-atk-ddr-2GB-mipi.dts、stm32mp257d-atk-ddr-2GB-rgb.dts、stm32mp257d-atk-ddr-2GB-lvds-1xSingleLink.dts、stm32mp257d-atk-ddr-2GB-lvds-2xSingleLink.dts、stm32mp257d-atk-ddr-2GB-lvds-dualLink.dts。

为方便大家的编译，正点原子提供了一按钮编译脚本 build_kernel.sh，第一次编译的时候可以直接运行脚本编译，脚本如下所示：

```

1  #!/bin/bash
2
3  #判断交叉编译工具链是否存在
4  if [ ! -e "/opt/st/stm32mp2/5.0.3-snapshot/environment-setup-
cortexa35-ostl-linux" ]; then
5      echo "未在该路径下找到交叉编译器:/opt/st/stm32mp2/5.0.3-snapshot/"
6      echo "请先安装交叉编译器:atk-image-openstlinux-weston-
stm32mp2.rootfs-x86_64-toolchain-5.0.3-snapshot*.sh"
7      exit 1
8  fi
9
10 #配置交叉编译器
11 source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-
ostl-linux
12

```



```
13 #检查输入内存容量参数: 1:DDR_1GB; 2:DDR_2GB
14 if [ $# -eq 0 ]; then
15     read -p "请选择 DDR 内存容量, 输入数字 1 或 2, 按 Enter 键确认, 开始编译:"
16     1.DDR_1GB
17     2.DDR_2GB
18     输入数字: " ddr_size_number
19     if [ $ddr_size_number -ne 1 ] && [ $ddr_size_number -ne 2 ]; then
20         echo "error:unsupport"
21         exit 1
22     fi
23 else
24     echo "usage example:./build_kernel.sh"
25     exit 1
26 fi
27
28 #编译前先清理上一次的编译结果, 可选
29 #由于 linux 内核源码编译时间较长, 选择不删除上一次编译结果
30 # if [ -e $PWD/../build ];then
31 #     rm -rf $PWD/../build
32 # fi
33
34 #执行 linux 内核源码配置文件 stm32mp257_atk_defconfig
35 make stm32mp257_atk_defconfig O="$PWD/../build"
36
37 #若需要更改 menuconfig 配置, 请更改后更新 stm32mp257_atk_defconfig 配置
38 #示例:
39 #make menuconfig O="$PWD/../build"
40 #cp $PWD/../build/.config
arch/arm64/configs/stm32mp257_atk_defconfig
41
42 #编译 linux 内核源码
43 make Image.gz vmlinux dtbs LOADADDR=0xC2000040 O="$PWD/../build" -j4
44 #若编译内核源码失败, 则退出编译
45 if [ ! $? -eq 0 ]; then
46     exit 1
47 fi
48
49 #编译 linux 内核模块
50 make modules O="$PWD/../build" -j4
51 #若编译内核模块失败, 则退出编译
52 if [ ! $? -eq 0 ]; then
53     exit 1
54 fi
```

```

55
56 #在上层目录新建一个 build_image 目录，用于存放编译后的目标文件：内核镜像、设备树
    文件、内核驱动模块
57 if [ ! -e "$PWD/../build_image" ]; then
58     mkdir $PWD/../build_image
59 else
60     #如果已存在该目录，则清空该目录
61     rm -rf $PWD/../build_image/*
62 fi
63
64 #指定 linux 内核模块安装目录 build_image，安装模块，并移除模块内部调试信息
65 make INSTALL_MOD_PATH="$PWD/../build_image" modules_install
O="$PWD/../build" INSTALL_MOD_STRIP=1 -j4
66 #若安装内核模块失败，则退出安装
67 if [ ! $? -eq 0 ]; then
68     exit 1
69 fi
70
71 #移除模块目录下的无用链接目录
72 rm $PWD/../build_image/lib/modules/6.6.48/build -rf
73 rm $PWD/../build_image/lib/modules/6.6.48/source -rf
74
75 #复制编译后的内核镜像到 build_image 目录下
76 cp $PWD/../build/arch/arm64/boot/Image.gz $PWD/../build_image/
77
78 #复制编译后的设备树文件到 build_image 目录下
79 #DDR 1GB 设备树（包含多个设备树文件）
80 devicetree_dds_1GB=stm32mp257d-atk-ddr-1GB*.dtb
81 #DDR 2GB 设备树（包含多个设备树文件）
82 devicetree_dds_2GB=stm32mp257d-atk-ddr-2GB*.dtb
83
84 if [ $dds_size_number -eq 1 ]; then
85     #DDR 1GB
86     cp $PWD/../build/arch/arm64/boot/dts/st/$devicetree_dds_1GB
$PWD/../build_image/
87 elif [ $dds_size_number -eq 2 ]; then
88     #DDR 2GB
89     cp $PWD/../build/arch/arm64/boot/dts/st/$devicetree_dds_2GB
$PWD/../build_image/
90 else
91     #非正确选项
92     echo "error:unsupport"
93     exit 1

```

```

94 fi
95
96 #提示编译结束，请查看具体编译信息
97 echo "---Compile finish---"

```

上面的脚本可以单条命令运行(必须有.config 文件)，不用每次都使用此脚本。比如笔者需要清除内核编译的时候可以拷贝脚本的第 28~32 行运行。直接运行脚本即可编译，代码如下所示：

```
./build_kernel.sh
```

编译结果如下图所示：

```

.48/./build_image/lib/modules/6.6.48/kernel/net/tsn_sw_edge_lkm/edgx_pfm_lkm.ko
STRIP /home/alientek/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6
.48/./build_image/lib/modules/6.6.48/kernel/net/qrtr/qrtr.ko
STRIP /home/alientek/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6
.48/./build_image/lib/modules/6.6.48/kernel/net/qrtr/qrtr-tun.ko
STRIP /home/alientek/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6
.48/./build_image/lib/modules/6.6.48/kernel/net/qrtr/qrtr-mhi.ko
STRIP /home/alientek/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6
.48/./build_image/lib/modules/6.6.48/kernel/net/tsn_sw_edge_lkm/edgx_pfm_lkm.ko
DEPMOD /home/alientek/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6
.48/./build_image/lib/modules/6.6.48
make[1]: 离开目录"/home/alientek/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/b
uild"
---Compile finish---
alientek@ubuntu:~/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6.48$

```

图 3.2-1 linux 源码编译结果

如果不想用 build_kernel.sh 编译，可以参考下面的编译方法进行单独编译。

3.2.1 单独编译内核 Image.gz

单独编译内核源码并生成 Image.gz，步骤如下：

```
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
```

```
make stm32mp257_atk_defconfig
```

```
make Image.gz -j16
```

```
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ make stm32mp257_atk_defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
#
# configuration written to .config
#
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ make Image.gz -j16
WRAP arch/arm64/include/generated/uapi/asm/kvm_para.h
WRAP arch/arm64/include/generated/uapi/asm/errno.h
WRAP arch/arm64/include/generated/uapi/asm/ioctl.h
```

图 3.2-2 编译 Image.gz

source: 这个命令用于执行指定脚本中的命令，在当前会话中设置环境变量。它不会启动新的 shell，因此在当前 shell 中执行的所有命令都将使用这些新的环境设置。

路径: `source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux` 是一个脚本路径，这个脚本用于配置交叉编译环境。通过执行此脚本会设置包括编译器（如 GCC）、库路径、系统根路径（sysroot）等在内的多个环境变量，为后续交叉编译内核或其他应用准备环境。

make: 是一个建构管理工具，用于自动化编译和构建过程。

stm32mp257_atk_defconfig: 这是一个 Makefile 目标，用于生成针对特定硬件优化的默认配置文件。这个配置文件定义了哪些功能和驱动将被编译进内核。

Image.gz: 这是构建 Linux 内核的标准目标之一，特别是对于 ARM 架构。生成的 Image.gz 文件是一个压缩的内核映像，用于引导系统。。

-j16: 这个选项告诉 make 并行执行多个线程。-j16 意味着同时运行 16 个线程，这可以显著加快编译过程，特别是在多核处理器上。笔者电脑处理器支持 16 线程，用户可根据使用电脑处理器的性能修改线程数。

编译出来的 Image.gz 镜像在源码的 arch/arm64/boot 目录。

```
SORTTAB vmlinux
OBJCOPY arch/arm64/boot/Image
GZIP arch/arm64/boot/Image.gz
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ cd arch/arm64/boot/
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48/arch/arm64/boot$ ls
dts Image Image.gz install.sh Makefile
```

图 3.2-3 编译完成生成 Image.gz

3.2.2 单独编译设备树 dtb

单独编译源码并生成指定设备树文件，步骤如下：

```
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
make stm32mp257_atk_defconfig
make st/stm32mp257d-atk-ddr-2GB.dtb
make dtbs //编译全部设备树
```

```
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ make stm32mp257_atk_defconfig
#
# No change to .config
#
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ make st/stm32mp257d-atk-ddr-2GB.dtb
DTC      arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dtb
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ make dtbs
DTC      arch/arm64/boot/dts/st/stm32mp215f-dk.dtb
DTC      arch/arm64/boot/dts/st/stm32mp235f-dk.dtb
DTC      arch/arm64/boot/dts/st/stm32mp257f-dk.dtb
DTC      arch/arm64/boot/dts/st/stm32mp257f-ev1.dtb
```

以下省略

图 3.2-4 编译设备树

`source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux` 这条命令用于设置环境变量，使得后续的构建步骤可以使用交叉编译工具和其他必要的工具。它配置了编译器、库路径和其他重要的环境变量。

`make stm32mp257_atk_defconfig` 这条命令用于生成 ATK-DLMP257B 的默认配置文件。这是开始编译内核或设备树前的准备步骤，确保所有默认设置都被正确配置。

`make st/stm32mp257d-atk-ddr-2GB.dtb` 构建针对 ATK-DLMP257B（2GB 版本）的标准设备树二进制文件。这个设备树描述了开发板硬件的基础配置和接口，使得系统能够正确地管理和使用硬件资源。其他设备树同理。

`make dtbs` 这个命令会构建所有在 Makefile 中指定的设备树二进制文件。通常在需要批量构建多个设备树时使用。

编译出来的 ATK-DLMP257B 需要的设备树 dtb 文件在源码的 `arch/arm64/boot/dts/st` 目录。

3.2.3 单独编译内核模块

内核模块是 Linux 系统中的一个程序或功能代码片段，可以在系统运行时按需加载和卸载，而不需要重新编译整个内核或重启系统。这些模块通常用来添加设备驱动程序、文件系统类型或其他扩展内核功能的代码。

下面是单独编译内核模块的步骤：

```
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
make stm32mp257_atk_defconfig
make modules -j16
```

make modules: 这部分命令指示 make 构建自动化工具来编译和准备所有的内核模块。编译完的内核模块在对应的路径下，具体看打印信息提示。

内核模块打包方式参考 build.sh 脚本内容。

```
#在当前目录下新建一个 tmp 目录，用于存放编译后的目标文件
mkdir tmp

#将编译好的模块安装到 tmp 目录，通过 INSTALL_MOD_STRIP=1 移除模块调试信息
make modules_install INSTALL_MOD_PATH=tmp INSTALL_MOD_STRIP=1 -j16

#删除模块目录下的 source 目录
rm -rf tmp/lib/modules/6.6.48/source

#删除模块的目录下的 build 目录
rm -rf tmp/lib/modules/6.6.48/build

#跳转到模块目录
cd tmp/lib/modules

#压缩内核模块
tar -jcvf ../../modules.tar.bz2 .
```

3.3 设备树说明

设备树名字	部分节点	说明
stm32mp257d-atk-ddr-2GB.dts	memory	内存配置
	gpio-leds	LED 配置
	gpio_keys	按键配置
	dht11	DHT11 温湿度传感器配置
	ds18b20	DS18B20 传感器配置
	adc_12	ADC 配置
	csi	CSI 摄像头配置
	eth1	ETH1 网口配置
	eth2	ETH2 网口配置
	gpu	GPU 配置
	i2c3	I2C3 配置
	i2c4	I2C4 配置
	eeprom	EEPROM 配置
	imx335	IMX335 摄像头配置
	es8388	ES8388 音频芯片配置
	i2c8	I2C8 配置
	sai1	SAI1 配置
	timers17	定时器 17 配置
	m0_rproc	M0 核配置
	m33_rprpc	M33 核配置
	m_can1	CAN1 配置
	m_can2	CAN2 配置
	m_can3	CAN3 配置
	pcie_rc	PCIE 配置
	rtc	RTC 配置
	sdmmc1	SD 卡配置
	sdmmc2	EMMC 配置
	spi8	SPI8 配置
	switch0	交换机配置
	usart1	串口 1 配置
	usbh	USB 配置
stm32mp257d-atk-ddr-2GB-mipi.dts	dsi	MIPI DSI 配置
stm32mp257d-atk-ddr-2GB-rgb.dts	panel_rgb	RGB 配置
stm32mp257d-atk-ddr-2GB-lvds-1xSingleLink.dts	lvds	LVDS 配置
stm32mp257d-atk-ddr-2GB-lvds-2xSingleLink.dts	lvds	LVDS 配置
stm32mp257d-atk-ddr-2GB-lvds-dualLink.dts	lvds	LVDS 配置

出厂系统有多个设备树兼容正点原子的多款屏幕, 以 2GB 版本为例, stm32mp257d-atk-ddr-2GB.dts 是基树, 其他设备树文件都包含此设备树, 用户需要修改设备树时需要从 stm32mp257d-atk-ddr-2GB.dts 设备树文件修改, 并编译对应的设备树文件更新到开发板上使用。

3.4 内核模块说明

内核模块是操作系统内核的一部分，它可以在系统运行时动态加载和卸载，而不需要重新编译整个内核或重新启动系统。内核模块通常用于添加设备驱动程序、文件系统或其他系统级功能。这些模块使得开发人员能够根据需要扩展操作系统的功能，而无需对内核进行永久性修改。

内核模块的主要特点包括：

动态加载和卸载：内核模块可以在系统运行时被加载进内核或从内核中移除，提供了极大的灵活性和便利。

功能扩展：通过加载模块，可以根据需要临时添加新的功能，比如支持新的硬件设备或者新的网络协议。

独立开发：模块可以独立于主内核开发和编译，使得开发者可以专注于特定的功能，而不影响整个系统的稳定性。

节省资源：只有当需要特定功能时才加载相应的模块，可以节省系统资源，如内存和处理器时间。

在 Linux 操作系统中，内核模块通常以.ko（kernel object）文件形式存在。开发人员可以使用命令如 insmod、rmmod、modprobe 等来管理这些模块，分别用于加载、卸载和智能处理依赖关系和配置选项的加载。

lsmod 命令是最直接的方式来查看当前加载的内核模块。它实际上是/proc/modules 文件的格式化输出，这个文件包含了所有当前加载的内核模块的信息。开发板出厂系统执行 lsmod，示例如下：

```
root@ATK-DLMP257:~# lsmod
Module                Size  Used by
bridge                266240  0
stp                   12288  1 bridge
llc                   16384  2 bridge,stp
edgx_pfm_lkm          221184  0
rpmsg_tty             16384  0
rpmsg_ctrl            12288  0
rpmsg_char            20480  1 rpmsg_ctrl
ipv6                  577536  25 bridge
aes_neon_bs           24576  1
aes_neon_blk          24576  3 aes_neon_bs
algif_skcipher        12288  1
8733bu                1855488  0
rtk_btusb             73728  0
bluetooth             761856  22 rtk_btusb
cfg80211              425984  1 8733bu
ecdh_generic          16384  1 bluetooth
ecc                   36864  1 ecdh_generic
rfkill                32768  5 bluetooth,cfg80211
```

以下省略

图 3.4-1 出厂系统内核模块

通过 find 指令可以查找对应的 ko 驱动文件在出厂系统/boot/6.6.48/kernel 目录。

```
root@ATK-DLMP257:~# cd /
root@ATK-DLMP257:/# find -name *8733bu.ko*
./boot/6.6.48/kernel/drivers/net/wireless/realtek/rtl8733bu/8733bu.ko
root@ATK-DLMP257:/#
```

图 3.4-2 驱动文件路径

决定是否将驱动程序编译进内核或者编译为模块，主要取决于几个关键因素，包括系统的需求、资源限制、安全考虑和可维护性。以下是一些常见情况，用以判断如何处理驱动程序：

将驱动程序编译进内核

必需的启动设备：如果一个驱动程序是用于启动设备（如系统盘的文件系统或者网络启动所需的网络接口），通常需要将其编译进内核，以确保系统启动时可以立即使用这些设备。

性能要求：当对性能有极高要求时，将驱动程序编译进内核可以减少模块加载所需的额外处理和可能的延迟，尽管这种影响通常非常小。

稳定性和安全性：对于一些关键的安全功能，如加密或安全硬件访问，直接集成到内核中可以降低被动态修改的风险。

将驱动程序编译为模块

硬件兼容性：如果系统需要支持多种硬件配置，或者需要能够在不同硬件之间迁移，使用模块可以提供更高的灵活性。模块可以根据需要加载和卸载，不需要为每种配置重新编译内核。

资源限制：在资源受限的环境（如嵌入式系统）中，使用模块可以在不需要的时候卸载它们，从而节省宝贵的内存和存储空间。

开发和测试：开发新的驱动程序或者对现有驱动进行测试时，使用模块可以避免重复编译整个内核。这样可以快速迭代和测试修改。

系统维护和更新：模块可以在不重新启动系统的情况下被更新或替换，这使得系统维护和更新变得更加灵活和方便。

3.5 添加内核配置及菜单选项

3.5.1 内核菜单 menuconfig

内核配置及菜单选项是指在编译操作系统内核之前，通过一套配置界面来选择和定制内核的功能和模块的过程。这种配置允许开发者按需启用或禁用特定的内核功能，从而构建一个满足特定需求的内核。这对于优化系统性能、增加新的支持设备或简化内核以减少资源消耗等目的非常有用。

在 Linux 系统中，内核配置通常是通过 `make menuconfig`，这是一个基于文本的图形界面，允许用户在类似菜单的结构中浏览不同的内核选项。它适用于大多数终端环境，因为它不需要图形界面。

打开内核菜单 `menuconfig` 的步骤：

1. 安装必需的软件包

在开始配置内核之前，你需要确保已经安装了编译内核所需的软件包。

```
sudo apt install make build-essential libncurses-dev bison flex libssl-dev bc
```

2. 获取 Linux 内核源代码，这里笔者使用的是 ATK-DLMP257B 开发板出厂内核源码，位于【正点原子】STM32MP257 开发板（A 盘）-基础资料\01、程序源码\01、正点原子 Linux 出厂系统源码\linux-6.6.48.tar.bz2

安装交叉编译器【正点原子】STM32MP257 开发板（A 盘）-基础资料\05、开发工具\01、出厂系统交叉编译器\atk-image-openstlinux-weston-stm32mp2.rootfs-x86_64-toolchain-5.0.3-snapshot.sh

相关安装操作可以查看【正点原子】STM32MP257 开发板（A 盘）-基础资料\10、用户手册\【正点原子】ATK-DLMP257B 出厂系统源码使用指南.pdf

3. 设置内核配置

在准备 Linux 内核进行编译和配置时，需要使用特定的默认配置命令加载一个预设的配置文件，这个配置文件针对特定的硬件平台，对于 ATK-DLMP257B 开发板出厂内核源码而言，内核配置文件是 `arch/arm64/configs/stm32mp257_atk_defconfig`。此配置目的是提供一个良好的出发点，确保出厂系统所有必需的驱动和选项已经被启用。

```
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
```

```
make stm32mp257_atk_defconfig
```

执行完上面指令后，配置文件会被加载到 `.config` 文件中，此文件中包含了所有的配置选项和它们的设置。

```
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ make stm32mp257_atk_defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
#
# No change to .config
#
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ ls -a
.      .cocciconfig  drivers      ipc         Makefile    samples     .version
..     CODE_OF_CONDUCT.md fs           Kbuild      mm          scripts     virt
arch   .config       .get_maintainer.ignore Kconfig     Module.symvers security
block  CONTRIBUTING.md .gitattributes kernel       net         SECURITY.md
```

图 3.5-1 内核配置 `stm32mp257_atk_defconfig`

在 Linux 内核配置过程中，`stm32mp257_atk_defconfig` 和 `.config` 文件都扮演着重要的角色，但他们的功能和用途有所不同。

stm32mp257_atk_defconfig

定义：`stm32mp257_atk_defconfig` 是一个预设的配置文件，专为特定的硬件平台（如基于 `stm32mp257` 处理器，正点原子在这个文件基础上添加了 ATK-DLMP257B 开发板的配置）优化。这个文件通常包含了为该硬件平台推荐的内核选项和模块设置。

用途：使用 `make stm32mp257_atk_defconfig` 命令，可以将这个预设配置加载到当前的内核构建环境中。这通常是编译适用于特定硬件的内核的第一步，确保所有必需的硬件支持都已启用，并作为基础配置存在。

位置：这些默认配置文件通常存储在内核源代码树的 `arch/<arch>/configs/` 目录下，其中 `<arch>` 是特定的架构，例如对于 ARM64 架构，路径可能是 `arch/arm64/configs/`。

.config

定义：`.config` 文件是当前内核构建环境的实际配置文件。它包含了所有内核选项的当前设置，这些设置定义了哪些功能将被编译进内核，哪些将作为模块编译，以及哪些被完全禁用。

用途：这个文件是内核编译过程中使用的直接输入，内核的编译工具（如 `make`）根据 `.config` 文件中的设置来编译内核和模块。在使用诸如 `make menuconfig` 或其他配置工具修改内核设置时，更改将被保存在这个文件中。

生成和修改：`.config` 文件可以通过执行类似 `make defconfig`、`make oldconfig` 或特定的 `make xxx_defconfig` 命令来生成或更新。它也可以手动编辑，或者通过图形化或文本界面工具进行更新。

区别和联系



区别: `stm32mp257_atk_defconfig` 是一个为特定硬件预设的模板或基线配置文件, 而 `.config` 是根据这些预设模板或通过其他方式生成的实际配置文件, 用于控制具体的内核编译过程。

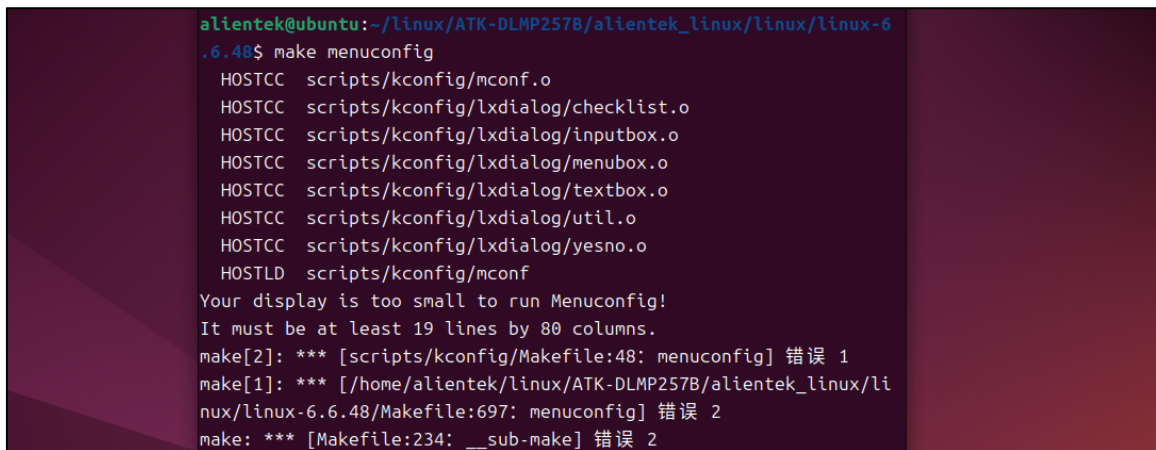
联系: 执行 `make stm32mp257_atk_defconfig` 会将 `stm32mp257_atk_defconfig` 的内容复制或导入到 `.config` 文件中。之后, `.config` 可以进一步修改以满足特定的需求或偏好。

4. 运行 `make menuconfig`

在内核源代码的根目录中, 运行 `make menuconfig` 命令启动配置菜单:

```
make stm32mp257_atk_defconfig
make menuconfig
```

在第 1 步中, 笔者已经安装好启动配置菜单需要的相关工具, 正常情况下使能交叉编译器、设置内核配置文件后就可以启动配置菜单了。`menuconfig` 是一个菜单界面, 对终端界面大小有最低限制要求 19 行和 80 列, 否则会有 `Your display is too small to run Menuconfig! It must be at least 19 lines by 80 columns.` 相关报错提示。



```
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6
.6.48$ make menuconfig
HOSTCC scripts/kconfig/mconf.o
HOSTCC scripts/kconfig/lxdialog/checklist.o
HOSTCC scripts/kconfig/lxdialog/inputbox.o
HOSTCC scripts/kconfig/lxdialog/menubox.o
HOSTCC scripts/kconfig/lxdialog/textbox.o
HOSTCC scripts/kconfig/lxdialog/util.o
HOSTCC scripts/kconfig/lxdialog/yesno.o
HOSTLD scripts/kconfig/mconf
Your display is too small to run Menuconfig!
It must be at least 19 lines by 80 columns.
make[2]: *** [scripts/kconfig/Makefile:48: menuconfig] 错误 1
make[1]: *** [/home/alientek/linux/ATK-DLMP257B/alientek_linux/li
nux/linux-6.6.48/Makefile:697: menuconfig] 错误 2
make: *** [Makefile:234: __sub-make] 错误 2
```

图 3.5-2 终端界面太小导致无法打开菜单

正常打开 `menuconfig` 界面后如下:

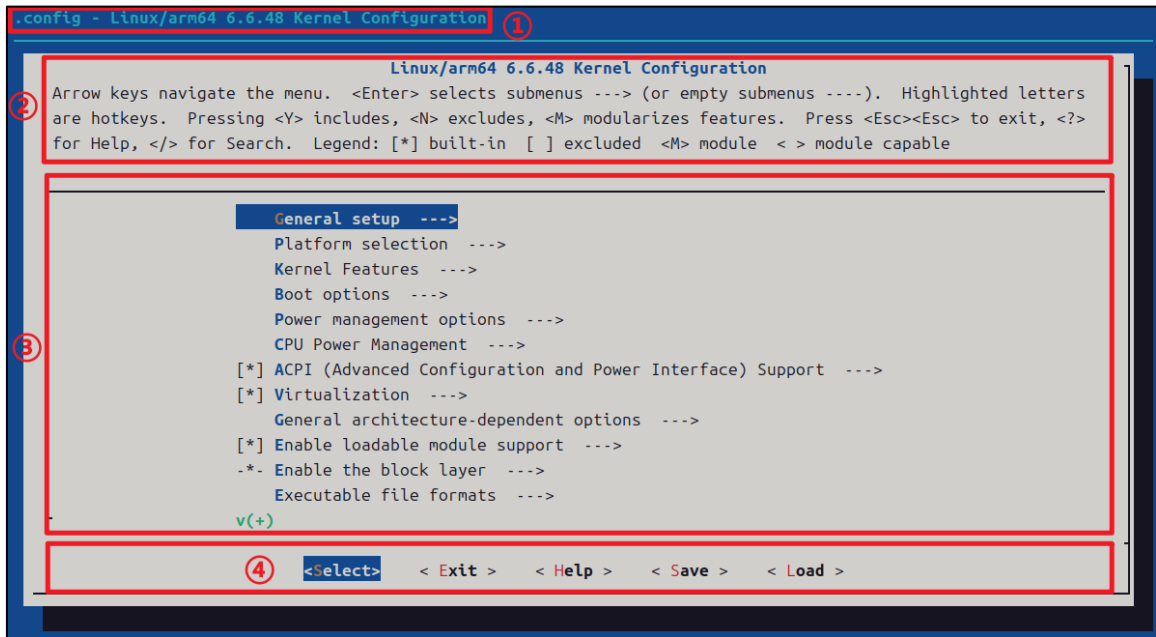


图 3.5-3 menuconfig 界面

对于界面的分析如下:

1、顶部栏 (.config - Linux/arm64 6.6.48 Kernel Configuration):

这部分显示了当前的配置文件名称 (.config)，目标的内核架构 (arm64)，以及内核的版本号 (6.6.48)。这提供了关于当前配置会话的基本信息，包括用于正在为哪个架构配置哪个版本的内核。

2、帮助说明区域:

这一区域提供了如何在菜单中导航和操作的基本指南。例如，箭头键用于在菜单中上下移动，回车键 (Enter) 用来选择子菜单或更改设置等。它还说明了界面底部的快捷键和图例的含义，比如 [] 表示可以切换的选项，<*> 表示内建到内核中的功能，<M> 表示编译为模块的功能。

3、主菜单区域:

这里显示了所有可配置的内核选项的主菜单。这些选项通常按功能类别进行分组，例如“General setup”、“Device Drivers”和“File systems”等。用户可以在这些类别中进一步导航以查看和配置更详细的选项。

4、底部按钮区域:

Select: 选择当前高亮的菜单项或选项。

Exit: 退出当前菜单，如果是在最顶层菜单将退出 menuconfig，会提示你是否保存更改。

Help: 显示当前选中的选项或子菜单的帮助信息。

Save: 保存当前的配置到 .config 文件中。

Load: 加载先前保存的配置。

这些区域的组合为用户提供了一个强大且直观的接口，用于配置和构建 Linux 内核。用户可以通过这些工具精确控制内核的构建过程，确保内核配置与其特定的需求和硬件兼容。

在 make menuconfig 的内核配置界面中，有一系列的键盘操作用于导航和更改配置选项。以下是一些基本的操作:

键盘按键	说明
------	----

方向键	上/下箭头键	在菜单项之间上下移动。
	左/右箭头键	在某些上下文中移动，在有可选值的配置选项中进行切换。
回车键		选择菜单项或确认更改。
字母键	Y 键	将选项设置为内建（编译进内核）。
	M 键	将选项设置为模块（在需要时可动态加载）。
	N 键	禁用选项，不构建此模块。
空格键		切换选项的状态（未选中、选中）。
符号键	/键	打开搜索配置选项。
	?键	获取关于当前选中项的详细帮助信息。
Tab 键		在界面不同区域间切换，比如从配置选项列表到底部按钮区域。
Esc 键		退出当前视图或对话框。连续按两次 Esc（Esc Esc）通常会退出当前菜单或者 menuconfig 系统本身。

这些基本操作允许用户在 menuconfig 界面中有效地浏览和修改内核的配置选项。不同的内核版本和不同的配置工具可能会有一些变化，但大部分操作都保持一致。在界面的底部通常会有一行简短的说明，提醒用户可以使用哪些键。

3.5.2 stm32mp257_atk_defconfig 说明

前面笔者提到出厂内核源码中的 stm32mp257_atk_defconfig 是正点原子 ATK-DLMP257B 开发板的配置文件，现在打开此文件看下具体内容并简要解析：

```

1 #
2 # Automatically generated file; DO NOT EDIT.
3 #
4 #
5 CONFIG_CC_VERSION_TEXT="aarch64-ostl-linux-gcc (GCC) 13.3.0"
6 CONFIG_CC_IS_GCC=y
7 .....
8 CONFIG_RTL8733BU=m
9 .....
```

文件中的每一行都代表一个配置项，它定义了一个特定功能是否被编译进内核，作为模块，或者不被包括。（除#符号注释外）

CONFIG_CC_VERSION_TEXT="aarch64-poky-linux-gcc (GCC) 13.3.0": 这行显示了用于编译内核的 GCC 编译器的版本和类型。这里，编译器是为 aarch64 架构的 poky Linux 环境配置的 GCC 13.3.0。

CONFIG_CC_IS_GCC=y: 这个选项表明内核是使用 GCC（GNU 编译器集合）编译的。y 代表“是”，表示此选项被激活。

CONFIG_RTL8733BU=m: 这表示 Realtek 8733BU Wi-Fi 驱动将被构建为一个模块（m）。构建为模块意味着该驱动不会直接编译进内核，而是可以在运行时按需加载。

3.5.3 配置文件的使用

当管理员或开发者需要编译内核时，这个.config 文件会被 make 工具读取，根据文件中定义的选项来编译内核。

如果需要更改这些设置，通常会通过运行 `make menuconfig` 或类似的工具来交互式地调整选项，而不是直接编辑 `.config` 文件。这样可以确保所有依赖关系得到正确处理，并且所有选项都保持一致。

3.5.4 修改并保存 `menuconfig`

通过 `menuconfig` 修改内核配置并保存为 `stm32mp257_atk_defconfig` 文件或者自定义文件，这是内核裁剪的基础操作。下面是内核裁剪 ES8388 的示例。

进入出厂内核源码目录下，打开 `menuconfig` 配置。

```
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
make stm32mp257_atk_defconfig
make menuconfig
搜索关键词 es8388。
```

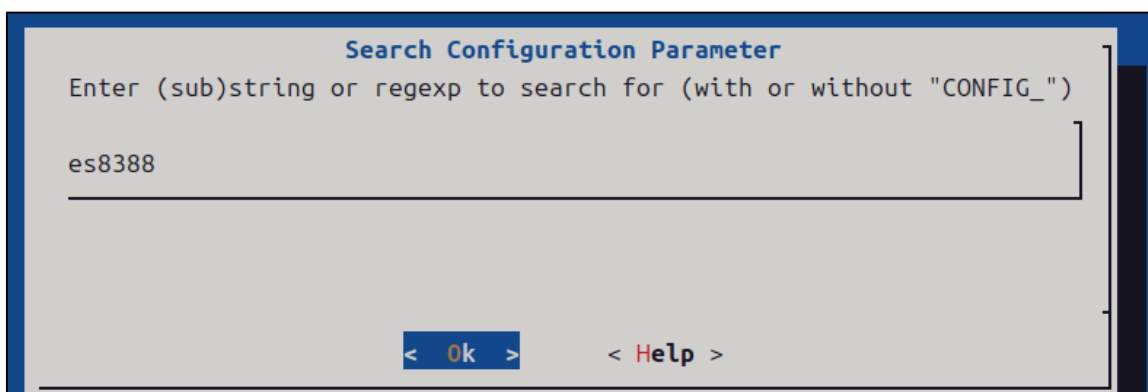


图 3.5-4 `menuconfig` 中搜索 ES8388

得到 `es8388` 相关配置和路径信息。

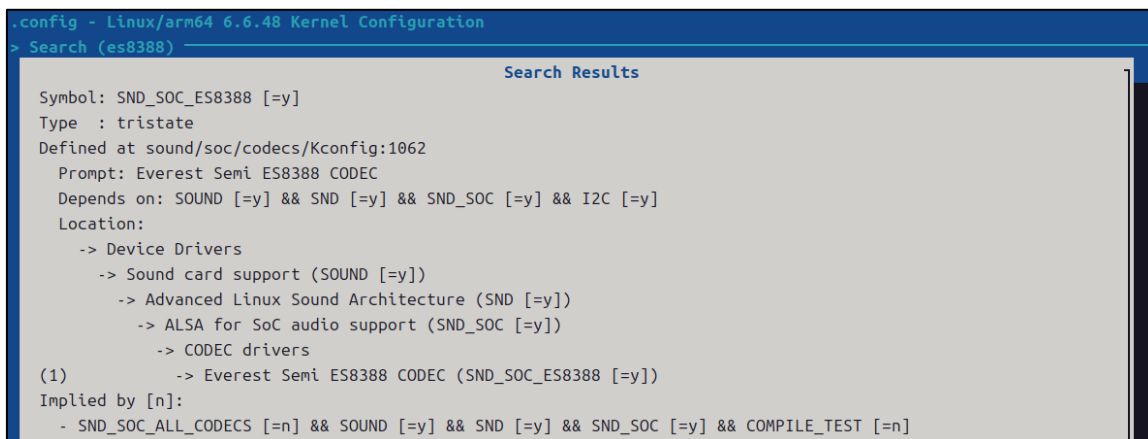


图 3.5-5 ES8388 的信息

默认是选中 `<*>ES8388`，用方向键将光标选到 `ES8388` 后，按 `N` 键取消选中 `<>`。然后用方向键将光标选到 `Save` 后，保存配置文件。

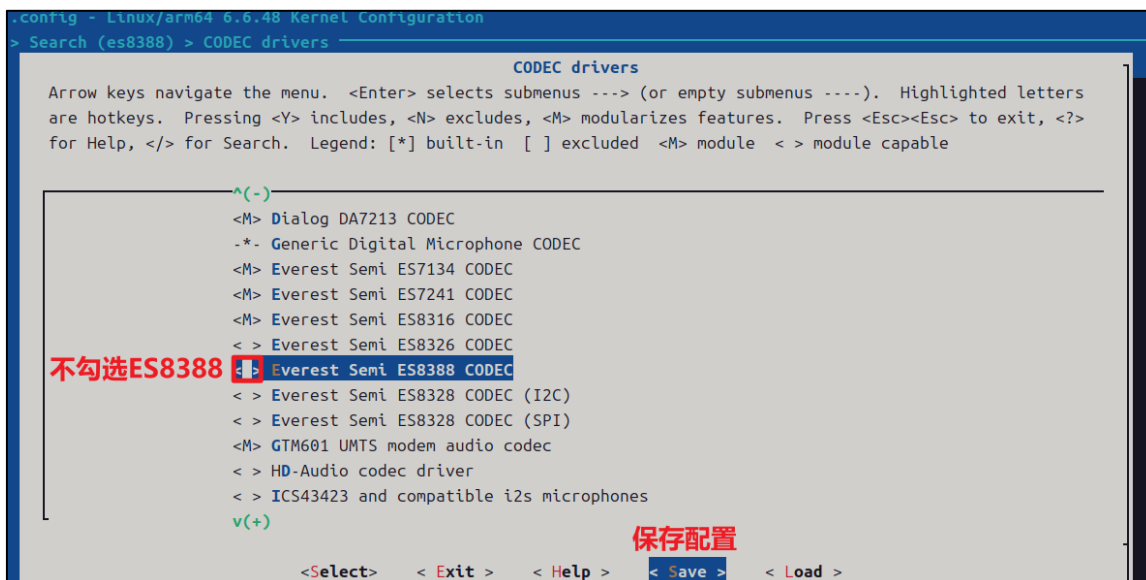


图 3.5-6 不选中 ES8388

将配置文件保存为./arch/arm64/configs/del_audio_stm32mp257_atk_defconfig

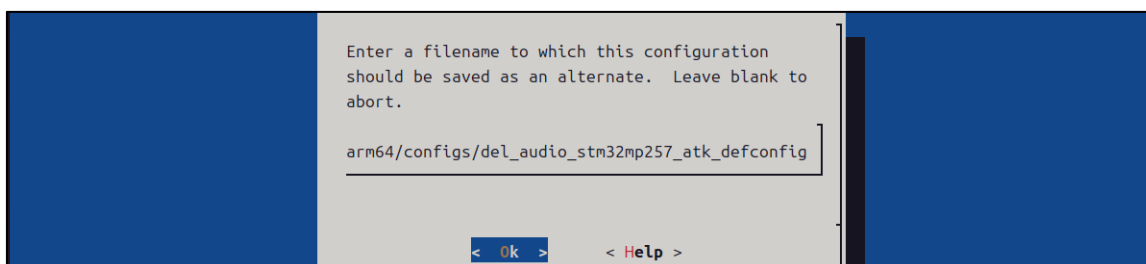


图 3.5-7 保存配置文件

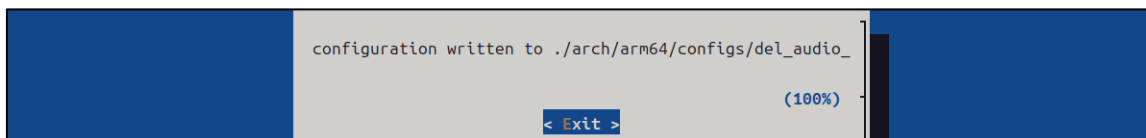


图 3.5-8 确认将配置写入文件

按回车键确认将配置写入文件后，双击 ESC 键退出 menuconfig 界面。

源码中 arch/arm64/configs/del_audio_stm32mp257_atk_defconfig 是默认的配置，前面基于此配置修改好后另存为的是 del_audio_stm32mp257_atk_defconfig，使能此配置然后重新编译内核，编译出来的内核就是不带 ES8388 驱动的。

```
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
make del_audio_stm32mp257_atk_defconfig
make Image.gz -j16
```

```
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ source /opt/st/stm32mp2/5.0.3-snapshot/environme
nt-setup-cortexa35-ostl-linux
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ make del_audio_stm32mp257_atk_defconfig
#
# configuration written to .config
#
alientek@ubuntu:~/linux/ATK-DLMP257B/alientek_linux/linux/linux-6.6.48$ make Image.gz -j16
SYNC      include/config/auto.conf.cmd
HOSTCC    scripts/dtc/dtc.o
HOSTCC    scripts/dtc/flattree.o
```

图 3.5-9 编译内核

如果是编译成模块，则使用 `make modules -j16`。

第四章 嵌入式 Linux 内核驱动基础

4.1 嵌入式 Linux 内核思路

4.1.1 分层思想

嵌入式 Linux 系统的设计通常采用分层思想,以便更好地组织和管理系统的不同组成部分。这种分层结构不仅有助于明确每层的功能和责任,还可以简化开发和维护工作,提高系统的模块化和可扩展性。以下是嵌入式 Linux 系统的典型分层架构:

1. 硬件层

这是系统的最底层,包括所有的物理设备,如处理器、内存、输入/输出端口和其他外围设备。硬件层是所有软件操作的基础,它直接与物理设备交互。

2. 驱动层

位于硬件层之上的是驱动层,它包括操作系统的所有硬件驱动程序。这些驱动程序负责管理硬件设备的具体操作,如数据读写、设备控制和状态监控。驱动层的作用是为上层提供一个抽象的、统一的接口,使上层软件不需要关心硬件的具体细节。

3. 操作系统内核层

内核是嵌入式 Linux 系统的核心,负责管理系统资源(如 CPU、内存和设备)、处理系统调用以及执行基本的任务调度和内存管理。内核层还包括各种内核模块和内核级服务,如网络协议栈、文件系统管理等。

4. 中间件层

中间件层位于内核层和应用层之间,提供了更高级别的服务和库,如数据库、多媒体处理、网络通信框架和设备协议。中间件层的目的是简化应用程序的开发,提供跨平台的服务和接口,以及封装复杂的功能。

5. 应用层

应用层包含运行在嵌入式设备上的所有用户级应用程序。这些应用程序利用下层提供的接口和服务来执行特定的任务,如用户界面展示、数据处理、设备控制等。应用层是与用户直接交互的层面,其性能和功能直接影响用户的体验。

6. 用户界面层

在某些复杂的嵌入式系统中,还可能有一个专门的用户界面层,用于管理图形用户界面(GUI)或其他形式的交互界面。这一层通常依赖于底层的图形库或 GUI 框架来实现,如 Qt 或其他专门的嵌入式 GUI 库。

这种分层架构使得嵌入式 Linux 系统的每个部分都可以独立开发和测试,提高了系统的可维护性和可扩展性。此外,分层也有助于在系统设计时进行资源和性能的优化,确保系统能够高效地运行在资源受限的嵌入式硬件上。

在操作系统中,尤其是在像 Linux 这样的现代操作系统中,内存被划分为两个主要区域:用户空间和内核空间。这种划分提高了系统的安全性和稳定性,确保了用户程序不能直接访问硬件资源或其他关键的系统资源。

用户空间 (User Space)

用户空间是普通用户程序执行的区域。这些程序包括各种应用软件,如文本编辑器、游戏、网络浏览器等。用户空间的程序通常具有有限的权限,不能直接操作硬件或访问内核数据结构。

这意味着用户程序在执行时更安全，因为即使应用程序崩溃或执行恶意操作，也不会直接危害整个系统。

内核空间（Kernel Space）

内核空间是操作系统内核的运行区域。内核负责管理计算机的硬件资源，如 CPU、内存和 I/O 设备。它也负责进程管理、文件系统操作、网络通信等核心功能。内核空间具有完全的系统访问权限，可以执行硬件级操作。由于内核空间的代码可以直接访问硬件和控制系统操作，因此任何在内核空间中的错误都可能导致严重的系统问题，包括系统崩溃和安全漏洞。

用户空间与内核空间的交互

尽管用户空间和内核空间在权限和功能上有严格的区分，但它们之间仍然需要频繁的交互，以执行诸如读写文件、发送接收网络数据包或访问设备等操作。以下是几种用户空间和内核空间交互的机制：

系统调用：

系统调用是用户空间访问内核服务的主要方式。当用户程序需要执行诸如文件操作、创建进程或者进行网络通信等不能直接完成的任务时，它会通过系统调用请求内核来执行这些操作。

例如，`open()`、`read()`、`write()` 是文件操作相关的系统调用。

虚拟文件系统（如 `/proc` 和 `/sys`）：

Linux 提供了一些虚拟文件系统，例如 `sysfs`，它们以文件系统的形式向用户空间程序提供内核数据的访问。

用户可以通过读写这些特殊的文件来获取系统信息或修改系统行为。

信号：

内核可以向用户空间的进程发送信号，以通知发生了某些系统级事件（如除零错误、访问违规等）。

设备驱动程序：

设备驱动程序在内核空间运行，提供了一种机制，允许用户空间程序通过标准的文件操作接口（如 `open`、`read`、`write`）与硬件设备进行交互。

共享内存：

用户空间程序和内核可以通过共享内存进行通信，这是一种高效的数据交换方法，尤其是在处理大量数据时。

4.1.2 sysfs 接口

在嵌入式 Linux 系统中，`sysfs` 是一个虚拟文件系统，提供了一种机制，通过它可以从用户空间与内核空间的各种对象进行交互。`sysfs` 通过文件系统的方式暴露内核设备、驱动程序和其他内核结构的属性，使得用户和应用程序可以通过读写文件的简单操作来获取系统信息或修改系统设置。

功能和作用

1、设备和驱动程序的动态展示：

`sysfs` 展示了系统中所有设备和驱动程序的结构，每个设备和驱动程序都有一个对应的目录。

设备通常在 `/sys/devices` 中列出，按照物理、虚拟和总线分类。

驱动程序通常在 `/sys/bus/`、`/sys/class/` 或 `/sys/block/` 下列出。

2、属性文件：



在 sysfs 中, 设备和驱动的特性通过属性文件表示。这些文件可以是可读的、可写的或两者都有, 允许用户和应用程序查看和修改设备设置。

例如, 可以读取一个设备的状态, 修改设备的配置, 或者触发设备执行特定操作。

3、动态性:

sysfs 是动态的, 意味着设备被连接或断开时, 相应的 sysfs 目录和文件会实时创建或删除。

使用场景

1、设备管理:

管理员和开发人员可以通过 sysfs 监控和配置硬件设备, 例如查看网络设备的统计数据或设置 LED 的状态。

2、系统监控:

通过 sysfs 文件读取各种硬件传感器的数据 (如温度传感器、电池电量监控等)。

3、调试:

开发人员可以使用 sysfs 来获取有关设备和驱动程序的详细信息, 这对于驱动开发和系统调试非常有用。

编程与操作示例

在 Linux 环境中, 操作 sysfs 可以通过简单的命令行工具 (如 cat、echo) 或在程序中使用文件操作 API 来完成。下面是一些示例:

读取属性:

通过 cat 命令读取属性。

写入属性:

通过 echo 命令写入属性。

编程访问:

在 C 程序中, 可以使用标准的文件操作函数如 open(), read(), write(), 和 close() 来操作 sysfs 文件。具体可以参考【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南.pdf 的 GPIO 应用编程章节。

4.2 驱动程序框架

在 Linux 系统中, 驱动程序框架是一种结构化和标准化的方式, 用来处理各种硬件设备。Linux 的驱动程序架构设计考虑到了模块性、可移植性和安全性, 使得开发人员能够为各种硬件写出高效和稳定的驱动程序。下面是 Linux 驱动程序的一些关键组成部分和概念:

1. 设备驱动类型

Linux 中的设备驱动主要可以分为以下几种:

字符设备驱动: 操作以字符为单位进行数据传输的设备 (如串口、文本显示设备)。

块设备驱动: 操作以数据块为单位传输的设备 (如硬盘驱动器)。

网络设备驱动: 处理网络通信的设备 (如以太网卡)。

2. 设备模型 (Device Model)

Linux 的设备模型提供了一种描述和组织系统中所有设备的方式, 包括那些由设备驱动程序控制的设备。这个模型包括:

设备 (Devices): 代表系统中的物理或虚拟设备。

驱动 (Drivers): 与设备交互的软件组件。

总线 (Buses): 连接设备和驱动的通信通道, 如 PCI、USB 等。

3. 驱动程序接口

Linux 内核为驱动开发者提供了一系列的核心接口和辅助函数, 以简化硬件操作和与系统其他部分的交互。这些接口包括:

文件操作接口: 如 `open()`, `read()`, `write()`, `ioctl()` 等。

设备注册与注销: 使用 `register_chrdev()` 或 `register_blkdev()` 等函数来注册和注销设备。

中断处理: 驱动程序可以注册中断处理函数, 响应硬件中断。

内存管理: 包括直接内存访问 (DMA) 和虚拟内存操作。

4. 设备树 (Device Tree)

在嵌入式 Linux 系统中, 设备树用于描述硬件设备的信息, 它是一个平台独立的数据结构, 用于在不修改内核代码的情况下提供有关硬件的详细信息。

5. 模块化

Linux 支持驱动程序的模块化, 这意味着驱动可以编译为内核模块 (*.ko 文件), 并在系统运行时动态加载和卸载。这提供了极大的灵活性和便利:

`insmod`: 加载模块。

`rmmod`: 卸载模块。

`modprobe`: 智能加载模块, 自动处理依赖。

6. 驱动程序编写和调试

编写: 驱动程序通常用 C 语言编写, 遵循 Linux 内核的编码规范。

调试: 驱动开发可以使用内核调试工具如 `printk` 等。

4.3 设备模型

Linux 设备模型是一个核心的组织架构，它为设备、驱动程序、和总线提供了一种结构化的管理方式。这个模型使得 Linux 内核能够高效地处理硬件设备，支持热插拔，并且能够自动管理设备与驱动程序之间的关系。这里详细解释 Linux 设备模型的核心组件和它们的作用。

核心组件

1、设备 (Devices)

在设备模型中，设备是指代表系统中的一个硬件组件。每个设备由一个 `struct device` 结构表示，这个结构包含了描述设备的各种信息，如其在系统中的位置、状态、和与其关联的资源（如内存、I/O 端口等）。

设备对象通常与一个具体的驱动程序关联，驱动程序负责实现与该设备交互的具体逻辑。

2、驱动程序 (Drivers)

驱动程序是管理一个或多个特定类型设备的软件模块。每个驱动程序由一个 `struct device_driver` 结构表示，其中包含了驱动程序的名称、总线类型、以及一系列回调函数，这些函数用于设备的探测、移除、挂起和恢复等。

驱动程序负责响应来自设备或内核的请求，执行如读写设备、处理中断、管理电源等操作。

3、总线 (Buses)

总线是连接设备和驱动程序的通道。在设备模型中，每种总线类型（如 PCI、USB、I2C 等）都由一个 `struct bus_type` 结构表示。

总线类型负责定义和管理挂载在该总线上的设备和驱动程序，提供方法来匹配设备和它们的驱动程序，并处理总线上的热插拔事件。

4、类 (Classes)

类是一种高级的抽象，用来组织提供相似功能的设备。例如，所有的声卡设备可能都属于一个名为 `sound` 的类。

类由 `struct class` 结构表示，用于创建表示设备功能的统一视图，而不考虑它们连接到哪种类型的总线。

设备模型的功能

1、自动设备注册与注销：当设备被连接到系统（例如通过 USB 接口插入）时，相应的设备对象会被自动创建并注册。当设备断开连接时，设备对象会被自动销毁。

2、驱动程序绑定：内核会自动将设备与合适的驱动程序绑定。这是通过在驱动程序中定义的匹配规则来完成的，匹配规则通常基于设备的 ID、总线类型等信息。

3、热插拔支持：设备模型支持热插拔，允许设备在系统运行时被添加或移除，系统会相应地加载或卸载驱动程序。

4、电源管理：设备模型通过与设备和驱动程序的交互，支持复杂的电源管理操作，如设备的挂起和恢复，这对于移动设备尤其重要。

实现机制

设备模型广泛使用了内核中的 `kobject` 机制，每个 `struct device`、`struct device_driver` 和 `struct bus_type` 都包含一个 `kobject` 结构。这使得设备模型能够利用 `kobject` 提供的功能，如引用计数、层次结构、热插拔事件和 `sysfs` 集成等。

通过 `sysfs`，设备模型的各个组成部分（设备、驱动、总线等）都在文件系统中都有对应的表示，这使得用户和管理员可以通过文件系统接口来查询和控制硬件设备，例如读写设备属性或触发特定操作。这种设计不仅增强了 Linux 的设备管理能力，也极大地方便了系统管理和调试。

4.4 字符设备驱动开发实验

本节我们就以 chrdevbase 这个虚拟设备为例，完整的编写一个字符设备驱动模块。chrdevbase 不是实际存在的一个设备，是笔者为了方便讲解字符设备的开发而引入的一个虚拟设备。chrdevbase 设备有两个缓冲区，一个为读缓冲区，一个为写缓冲区，这两个缓冲区的大小都为 100 字节。在应用程序中可以向 chrdevbase 设备的写缓冲区中写入数据，从读缓冲区中读取数据。chrdevbase 这个虚拟设备的功能很简单，但是它包含了字符设备的最基本功能。

4.4.1 字符设备驱动编写

本实验对应的例程路径为：开发板光盘->2、Linux 驱动例程->04_chrdevbase。

应用程序调用 open 函数打开 chrdevbase 这个设备，打开以后可以使用 write 函数向 chrdevbase 的写缓冲区 writebuf 中写入数据(不超过 100 个字节)，也可以使用 read 函数读取读缓冲区 readbuf 中的数据操作，操作完成以后应用程序使用 close 函数关闭 chrdevbase 设备。

编写实验程序：新建 chrdevbase.c，然后在里面输入如下内容：

```
1 #include <linux/module.h>
2 #include <linux/fs.h>
3 #include <linux/init.h>
4 #include <linux/cdev.h>
5 #include <linux/uaccess.h>
6
7 #define DEVICE_NAME "chrdevbase"
8 static dev_t devno;
9 static struct cdev chr_cdev;
10 static char kernel_buf[100] = "kernel data!";
11
12 static int chrdevbase_open(struct inode *inode, struct file *file)
13 {
14     printk("chrdevbase: device opened\n");
15     return 0;
16 }
17
18 static int chrdevbase_release(struct inode *inode, struct file
*file)
19 {
20     printk("chrdevbase: device closed\n");
21     return 0;
22 }
23
24 static ssize_t chrdevbase_read(struct file *file, char __user *buf,
size_t len, loff_t *offset)
25 {
26     size_t datalen = strlen(kernel_buf);
27     if (len > datalen)
```



```
28     len = datalen;
29
30     if (copy_to_user(buf, kernel_buf, len))
31         return -EFAULT;
32
33     printk("chrdevbase: read %zu bytes\n", len);
34     return len;
35 }
36
37 static ssize_t chrdevbase_write(struct file *file, const char __user
*buf, size_t len, loff_t *offset)
38 {
39     if (len > sizeof(kernel_buf) - 1)
40         len = sizeof(kernel_buf) - 1;
41
42     if (copy_from_user(kernel_buf, buf, len))
43         return -EFAULT;
44
45     kernel_buf[len] = '\0';
46     printk("chrdevbase: received %zu bytes: %s\n", len, kernel_buf);
47     return len;
48 }
49
50 static struct file_operations chrdevbase_fops = {
51     .owner = THIS_MODULE,
52     .open = chrdevbase_open,
53     .release = chrdevbase_release,
54     .read = chrdevbase_read,
55     .write = chrdevbase_write,
56 };
57
58 static int __init chrdevbase_init(void)
59 {
60     int ret;
61
62     ret = alloc_chrdev_region(&devno, 0, 1, DEVICE_NAME);
63     if (ret < 0) {
64         printk("chrdevbase: failed to alloc chrdev region\n");
65         return ret;
66     }
67
68     cdev_init(&chr_cdev, &chrdevbase_fops);
69     ret = cdev_add(&chr_cdev, devno, 1);
```



```

70     if (ret < 0) {
71         unregister_chrdev_region(devno, 1);
72         printk("chrdevbase: failed to add cdev\n");
73         return ret;
74     }
75
76     printk("chrdevbase: module loaded. Major: %d Minor: %d\n",
MAJOR(devno), MINOR(devno));
77     return 0;
78 }
79
80 static void __exit chrdevbase_exit(void)
81 {
82     cdev_del(&chr_cdev);
83     unregister_chrdev_region(devno, 1);
84     printk("chrdevbase: module unloaded\n");
85 }
86
87 module_init(chrdevbase_init);
88 module_exit(chrdevbase_exit);
89
90 MODULE_LICENSE("GPL");
91 MODULE_AUTHOR("Alientek");
92 MODULE_DESCRIPTION("Compatible character device driver for Linux
6.6.48");

```

以下是这份源码的解析:

第一部分: 头文件引入 (第 1~5 行), 这些是编写字符设备驱动所需的基本内核头文件。

```

1  #include <linux/module.h>      // 模块的核心支持, 提供 module_init/module_exit 等
2  #include <linux/fs.h>          // file_operations 结构体定义, 注册字符设备用
3  #include <linux/init.h>        // __init 和 __exit 宏
4  #include <linux/cdev.h>        // 字符设备结构体 struct cdev 定义
5  #include <linux/uaccess.h>     // 用户空间访问函数 copy_to_user、copy_from_user

```

第二部分: 全局变量定义 (第 7~10 行), kernel_buf 是用于测试 read/write 的内核态缓冲区。

```

7  #define DEVICE_NAME "chrdevbase" // 设备名 (用于注册设备)
8  static dev_t devno;              // 设备号 (包含主设备号+次设备号)
9  static struct cdev chr_cdev;     // 字符设备结构体
10 static char kernel_buf[100] = "kernel data!"; // 内核缓冲区, 初始内容为 "kernel data!"

```

第三部分: 设备文件操作函数 (open、release、read、write)

open 函数 (第 12~16 行), 当用户态调用 open("/dev/chrdevbase") 时触发, 打印调试信息。

```

12 static int chrdevbase_open(struct inode *inode, struct file *file)

```



```

13 {
14     printk("chrdevbase: device opened\n");
15     return 0;
16 }

```

release 函数（第 18~22 行），对应 close() 系统调用，释放资源，这里只是打印日志。

```

18 static int chrdevbase_release(struct inode *inode, struct file *file)
19 {
20     printk("chrdevbase: device closed\n");
21     return 0;
22 }

```

read 函数（第 24~35 行），把 kernel_buf 里的内容拷贝到用户空间。重点是 copy_to_user() 的使用，确保内核态安全访问用户空间。

```

24 static ssize_t chrdevbase_read(struct file *file, char __user *buf, size_t len, loff_t *offset)
25 {
26     size_t datalen = strlen(kernel_buf);
27     if (len > datalen)
28         len = datalen;
29
30     if (copy_to_user(buf, kernel_buf, len))
31         return -EFAULT;
32
33     printk("chrdevbase: read %zu bytes\n", len);
34     return len;
35 }

```

write 函数（第 37~48 行），把用户空间数据写入 kernel_buf，并添加结束符 \0。核心在于 copy_from_user()。

```

37 static ssize_t chrdevbase_write(struct file *file, const char __user *buf, size_t len, loff_t *offset)
38 {
39     if (len > sizeof(kernel_buf) - 1)
40         len = sizeof(kernel_buf) - 1;
41
42     if (copy_from_user(kernel_buf, buf, len))
43         return -EFAULT;
44
45     kernel_buf[len] = '\0';
46     printk("chrdevbase: received %zu bytes: %s\n", len, kernel_buf);
47     return len;
48 }

```

第四部分：file_operations 注册（第 50~56 行），标准结构体 file_operations，内核会根据应用层的 open/read/write 调用对应函数。

```

50 static struct file_operations chrdevbase_fops = {
51     .owner = THIS_MODULE,

```



```
52     .open = chrdevbase_open,
53     .release = chrdevbase_release,
54     .read = chrdevbase_read,
55     .write = chrdevbase_write,
56 };
```

第五部分：模块初始化与退出（第 58~85 行）

init 初始化(第 58~78 行), 实现字符设备注册: 动态申请设备号+初始化 cdev+注册 cdev。

```
58 static int __init chrdevbase_init(void)
59 {
60     int ret;
61
62     ret = alloc_chrdev_region(&devno, 0, 1, DEVICE_NAME); // 动态申请设备号
63     if (ret < 0) {
64         printk("chrdevbase: failed to alloc chrdev region\n");
65         return ret;
66     }
67
68     cdev_init(&chr_cdev, &chrdevbase_fops); // 初始化 cdev
69     ret = cdev_add(&chr_cdev, devno, 1); // 添加到内核
70     if (ret < 0) {
71         unregister_chrdev_region(devno, 1);
72         printk("chrdevbase: failed to add cdev\n");
73         return ret;
74     }
75
76     printk("chrdevbase: module loaded. Major: %d Minor: %d\n", MAJOR(devno),
MINOR(devno));
77     return 0;
78 }
```

exit 退出（第 80~85 行），注销字符设备，释放资源。

```
80 static void __exit chrdevbase_exit(void)
81 {
82     cdev_del(&chr_cdev); // 删除 cdev
83     unregister_chrdev_region(devno, 1); // 释放设备号
84     printk("chrdevbase: module unloaded\n");
85 }
```

最后部分：模块声明（第 87~92 行），这是标准模块宏：声明入口/出口函数、授权、作者、描述等信息。

```
87 module_init(chrdevbase_init);
88 module_exit(chrdevbase_exit);
89 MODULE_LICENSE("GPL");
```

```
91 MODULE_AUTHOR("Alientek");
```

```
92 MODULE_DESCRIPTION("Compatible character device driver for Linux 6.6.48");
```

4.4.2 编写测试 APP

编写测试 APP 就是编写 Linux 应用，需要用到 C 库里面和文件操作有关的一些函数，比如 open、read、write 和 close 这四个函数。对应于编程没基础可以查看[网盘资料 09](#)、[文档教程\05【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南](#)

编写测试 APP 程序：驱动编写好以后是需要测试的，一般编写一个简单的测试 APP，测试 APP 运行在用户空间。测试 APP 很简单通过输入相应的指令来对 chrdevbase 设备执行读或者写操作。新建 chrdevbaseApp.c 文件，在此文件中输入如下内容：

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6 #include <stdlib.h>
7 #include <string.h>
8
9 /*****
10 Copyright © ALIENTEK Co., Ltd. 1998-2029. All rights reserved.
11 文件名      : chrdevbaseApp.c
12 作者        : Alientek
13 版本        : V1.1
14 描述        : chrdevbase 测试 APP。
15 其他        : 使用方法: ./chrdevbaseApp /dev/chrdevbase <1>|<2>
16                argv[2] 1: 读驱动
17                argv[2] 2: 写驱动
18 论坛        : www.openedv.com
19 日志        : V1.0 2025/4/21 初版 by Alientek
20
21 *****/
22
23 static char usrdata[] = "usr data from user space.";
24
25 int main(int argc, char *argv[])
26 {
27     int fd, ret;
28     char *filename;
29     char readbuf[100] = {0};
30     char writebuf[100] = {0};
31
32     if (argc != 3) {
33         printf("Usage: %s <device> <1:read | 2:write>\r\n", argv[0]);
```



```
34     return -1;
35 }
36
37 filename = argv[1];
38
39 fd = open(filename, O_RDWR);
40 if (fd < 0) {
41     perror("open");
42     printf("Failed to open device file: %s\r\n", filename);
43     return -1;
44 }
45
46 if (atoi(argv[2]) == 1) {
47     /* 读驱动 */
48     ret = read(fd, readbuf, sizeof(readbuf) - 1);
49     if (ret < 0) {
50         perror("read");
51         printf("Read from %s failed!\r\n", filename);
52     } else {
53         readbuf[ret] = '\0'; // 确保字符串结尾
54         printf("Read %d bytes: \"%s\"\r\n", ret, readbuf);
55     }
56 } else if (atoi(argv[2]) == 2) {
57     /* 写驱动 */
58     strncpy(writebuf, usrdata, sizeof(writebuf) - 1);
59     ret = write(fd, writebuf, strlen(writebuf));
60     if (ret < 0) {
61         perror("write");
62         printf("Write to %s failed!\r\n", filename);
63     } else {
64         printf("Wrote %d bytes: \"%s\"\r\n", ret, writebuf);
65     }
66 } else {
67     printf("Invalid option: %s (1=read, 2=write)\r\n", argv[2]);
68 }
69
70 close(fd);
71 return 0;
72 }
```

这个测试 APP 的作用:

- 1、打开设备文件, 使用 `open()` 打开指定的字符设备路径
- 2、读设备数据, 如果参数为 1, 使用 `read()` 从设备读取数据
- 3、写设备数据, 如果参数为 2, 使用 `write()` 向设备写入一段用户数据

4、输入参数检查，确保用户输入了设备文件路径和操作命令（读或写）

这个 APP 用于验证内核模块 chrdevbase 功能的测试程序，展示了如何从用户空间与字符设备交互。

4.4.3 编译驱动程序

首先编译驱动程序，也就是 chrdevbase.c 这个文件，我们需要将其编译为.ko 模块，创建 Makefile 文件，然后在其中输入如下内容：

```
1 obj-m += chrdevbase.o
2
3 KDIR :=
/home/alientek/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6.48
4 PWD := $(shell pwd)
5
6 all:
7 $(MAKE) -C $(KDIR) M=$(PWD) modules
8
9 clean:
10 $(MAKE) -C $(KDIR) M=$(PWD) clean
```

第 1 行，这是内核模块的入口，告诉内核编译系统你要构建一个名为 chrdevbase.ko 的模块，源文件是 chrdevbase.c。

第 3 行，KDIR 是你的 内核源码路径（Makefile 用它来找到编译环境），**这里填写的是内核源码所在的目录，需要根据自己的内核源码路径进行填写**，笔者的 MP257 出厂系统源码放在了 /home/alientek/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6.48 这个目录，所以就写了这个。

第 4 行，PWD 是当前模块源代码所在目录

第 6~7 行，进入 KDIR 所在目录（即内核源码根目录），使用 M=\$(PWD) 告诉内核“我要编译的模块代码在当前目录下”，modules 表示你只编译模块，不重新编译整个内核。

第 9~10 行，清除中间文件（比如 .o, .mod.c, .ko 等），保持项目干净。

编写完 Makefile 后，我们就可以准备编译内核驱动源码了。内核驱动源码依赖的是内核，我们需要编译内核以及相关模块依赖，才能编译内核驱动和使用。编译就需要用到编译器，编译器的安装方法可以看网盘资料的 **10、用户手册03【正点原子】ATK-DLMP257B 出厂系统源码使用指南**的第二章节，这里不再赘述。安装完编译器后，进入内核源码目录下进行编译，参考如下操作，不需要执行源码自带的编译脚本：

```
cd 内核源码目录
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux //设置交叉编译环境变量
make stm32mp257_atk_defconfig //使用 ATK 提供的默认配置初始化.config 文件
make prepare //为内核模块和外部驱动编译准备头文件
make modules_prepare //为外部模块准备 Module.symvers 等信息
make -j16 //真正开始编译 Linux 内核和模块
```

编译完内核后，返回到驱动源码目录下，这里以 chrdevbase 为例：

```
cd 驱动源码所在目录
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux //设置交叉编译环境变量
make //编译驱动源码
file chrdevbase.ko //查看驱动 ko 文件是否为 aarch64 位格式
```

编译成功后会在当前目录生成一个 chrdevbase.ko 文件，可以查看该文件格式是否符合要求。

```
alientek@ubuntu:~/linux/mp257/04_chrdvbase$ source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
alientek@ubuntu:~/linux/mp257/04_chrdvbase$ make
make -C /home/alientek/linux/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6.48 M=/home/alientek/linux/mp257/04_chrdvbase modules
make[1]: 进入目录 "/home/alientek/linux/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6.48"
CC [M] /home/alientek/linux/mp257/04_chrdvbase/chrdevbase.o
MODPOST /home/alientek/linux/mp257/04_chrdvbase/Module.symvers
CC [M] /home/alientek/linux/mp257/04_chrdvbase/chrdevbase.mod.o
LD [M] /home/alientek/linux/mp257/04_chrdvbase/chrdevbase.ko
make[1]: 离开目录 "/home/alientek/linux/mp257/v6.0_Factory/emmc_factory_files/other/2GB_git/linux/linux-6.6.48"
alientek@ubuntu:~/linux/mp257/04_chrdvbase$ ls
chrdevbaseApp.c  chrdevbase.c  chrdevbase.ko  chrdevbase.mod  chrdevbase.mod.c  chrdevbase.mod.o  chrdevbase.o  Makefile  modules.order  Module.symvers
alientek@ubuntu:~/linux/mp257/04_chrdvbase$ file chrdevbase.ko
chrdevbase.ko: ELF 64-bit LSB relocatable, ARM aarch64, version 1 (SYSV), BuildID[sha1]=e93ab8c0db9183864e2ddd9601c143e56e55fa, with debug info, not stripped
```

图 4.4-1 编译驱动

将编译出来的 chrdevbase.ko 文件拷贝到开发板出厂系统的/lib/modules/6.6.48 目录下，并进行下面指令加载驱动。

```
depmod
modprobe chrdevbase
lsmod | grep chrdevbase
```

```
root@ATK-DLMP257:/lib/modules/6.6.48# ls
chrdevbase.ko  modules.alias  modules.builtin  modules.builtin.bin  modules.dep  modules.devname  modules.softdep  modules.symbols.bin
kernel  modules.alias.bin  modules.builtin.alias.bin  modules.builtin.modinfo  modules.dep.bin  modules.order  modules.symbols
root@ATK-DLMP257:/lib/modules/6.6.48# depmod
root@ATK-DLMP257:/lib/modules/6.6.48# modprobe chrdevbase
[ 4114.468766] chrdevbase: module loaded. Major: 510 Minor: 0
root@ATK-DLMP257:/lib/modules/6.6.48# lsmod | grep chrdevbase
chrdevbase
12288 0
root@ATK-DLMP257:/lib/modules/6.6.48#
```

图 4.4-2 加载驱动

查看当前模块目录，确认模块文件已经放在正确路径下。depmod 用于生成或更新模块依赖关系（modules.dep 文件），这是为了让 modprobe 能自动找到依赖项。

modprobe chrdevbase 作用是将模块加载进内核（会根据 modules.dep 自动处理依赖并加载），输出了 chrdevbase: module loaded. Major: 510 Minor: 0，说明模块成功被加载，主设备号是 510。

lsmod | grep chrdevbase，输出：chrdevbase 12288 0，表示模块已经被加载，大小 12KB，没有被其它模块使用（0）。

驱动加载成功需要在/dev 目录下创建一个与之对应的设备节点文件，应用程序就是通过操作这个设备节点文件来完成对具体设备的操作。输入如下命令创建/dev/chrdevbase 这个设备节点文件，如果你的驱动会自动生成对应的设备节点文件，则不需要手动创建：

```
mknod /dev/chrdevbase c 510 0
```

其中“mknod”是创建节点命令，“/dev/chrdevbase”是要创建的节点文件，“c”表示这是个字符设备，“510”是设备的主设备号，根据实际主设备号进行填写，“0”是设备的次设备号。创建完成以后就会存在/dev/chrdevbase 这个文件，可以使用“ls /dev/chrdevbase -l”命令查看。

```
root@ATK-DLMP257:/lib/modules/6.6.48# mknod /dev/chrdevbase c 510 0
root@ATK-DLMP257:/lib/modules/6.6.48# ls /dev/chrdevbase -l
crw-r--r-- 1 root root 510, 0 Feb 27 18:48 /dev/chrdevbase
root@ATK-DLMP257:/lib/modules/6.6.48#
```

图 4.4-3 创建和查看设备

4.4.4 编译测试应用程序

测试 APP 比较简单, 只有一个文件, 因此就不需要编写 Makefile 了, 直接输入命令编译。因为测试 APP 是要在 ARM 开发板上运行的, 所以需要使用对应的 gcc 工具来编译, 输入如下命令:

```
source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux //设置交叉编译环境变量
${CC} chrdevbaseApp.c -o chrdevbaseApp
```

编译完成以后会生成一个叫做 chrdevbaseApp 的可执行程序, 输入如下命令查看 chrdevbaseAPP 这个程序的文件信息, 也是需要 ARM aarch64 格式:

```
file chrdevbaseApp
alientek@ubuntu:~/linux/mp257/04_chrdevbase$ source /opt/st/stm32mp2/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
alientek@ubuntu:~/linux/mp257/04_chrdevbase$ ${CC} chrdevbaseApp.c -o chrdevbaseApp
alientek@ubuntu:~/linux/mp257/04_chrdevbase$ ls
chrdevbaseApp  chrdevbase.c  chrdevbase.mod  chrdevbase.mod.o  Makefile  Module.symvers
chrdevbaseApp.c  chrdevbase.ko  chrdevbase.mod.c  chrdevbase.o  modules.order
alientek@ubuntu:~/linux/mp257/04_chrdevbase$ file chrdevbaseApp
chrdevbaseApp: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-aarch64.so.1, BuildID[sha1]=e649bb56e4d174418647504c399a9e15beb36336, for GNU/Linux 5.15.0, with debug_info, not stripped
alientek@ubuntu:~/linux/mp257/04_chrdevbase$
```

图 4.4-4 编译测试应用程序

将编译好的测试文件拷贝到开发板出厂系统上, 这里笔者也是拷贝到/lib/modules/6.6.48 目录, 方便测试。

测试指令如下:

```
chmod u+x chrdevbaseApp //赋予 App 可执行权限
./chrdevbaseApp /dev/chrdevbase 1 //读测试
./chrdevbaseApp /dev/chrdevbase 2 //写测试

root@ATK-DLMP257:/lib/modules/6.6.48# ls
chrdevbaseApp kernel modules.alias.bin modules.builtin.alias.bin modules.builtin.modinfo modules.dep.bin modules.order modules.symbols
chrdevbase.ko modules.alias modules.builtin modules.builtin.bin modules.dep modules.devname modules.softdep modules.symbols.b
root@ATK-DLMP257:/lib/modules/6.6.48# chmod u+x chrdevbaseApp
root@ATK-DLMP257:/lib/modules/6.6.48# ./chrdevbaseApp /dev/chrdevbase 1
[ 5697.189595] chrdevbase: device opened
[ 5697.189658] chrdevbase: read 25 bytes
Real 5697.191551] chrdevbase: device closed
d 25 bytes: "usr data from user space."
root@ATK-DLMP257:/lib/modules/6.6.48# ./chrdevbaseApp /dev/chrdevbase 2
[ 5700.131947] chrdevbase: device opened
[ 5700.132021] chrdevbase: received 25 bytes: usr data from user space.
Wrote[ 5700.136847] chrdevbase: device closed
25 bytes: "usr data from user space."
root@ATK-DLMP257:/lib/modules/6.6.48#
```

图 4.4-5 应用程序测试

测试结果:

Read 12 bytes: "kernel data!" 成功从 kernel_buf 中读取内容; 数据一致, 读写链路正常; 读取长度正确, 12 字节是 "kernel data!" 的长度。

Wrote 25 bytes: "usr data from user space." 成功从用户空间写入到 kernel_buf; 内核接收数据正常; 字符串内容完整一致。

测试结果成功。

4.5 Platform 平台驱动

在嵌入式 Linux 开发中, 平台驱动 (Platform Driver) 是一个处理没有专门硬件总线的设备的驱动类型。这类设备通常是系统的一部分, 如处理器上的集成外设, 它们不是通过标准的硬件总线 (如 PCI 或 USB) 来发现 and 管理的。平台驱动的概念允许开发者以一种标准的方式来处理这些设备, 即使它们不符合标准总线架构。

平台设备和平台驱动

平台设备 (Platform Device) 是内核中的一种设备, 它代表那些直接集成在系统板上的硬件组件。这些设备通常包括时钟、中断控制器、GPIO 控制器等。

平台驱动 (Platform Driver) 则是控制这类设备的驱动程序。这些驱动需要能够加载和初始化设备, 处理数据传输, 以及可能的中断处理等。

平台驱动的主要组件

1、设备注册:

平台设备在系统启动时通过硬编码或使用设备树 (Device Tree) 来注册。注册过程包括设备的名称、资源 (如内存区域、IRQ 等) 和其他配置参数。

2、驱动注册:

平台驱动使用 `platform_driver_register()` 函数来注册自己。注册过程中会指定驱动可以处理的设备名称, 以及设备的初始化和退出时调用的函数。

3、驱动和设备的匹配:

当平台设备注册到系统后, 内核会尝试为其找到一个匹配的驱动。如果设备的名称与驱动支持的名称相匹配, 那么该驱动的初始化函数会被调用。

第五章 Pinctrl 子系统

5.1 Pinctrl 子系统简介

Pinctrl (Pin Control) 子系统是 Linux 内核中的一个重要组件, 用于管理处理器芯片上的引脚 (pin) 的配置和控制。处理器芯片的引脚用于连接外部设备或其他芯片, 如 GPIO (通用输入输出)、I2C、SPI、UART 等。Pinctrl 子系统提供了一个统一的接口, 允许设备驱动程序和其他子系统来配置和控制这些引脚。

5.1.1 主要特点

Pinctrl 子系统在 Linux 内核中具有以下特点:

1、灵活性和可配置性: Pinctrl 子系统允许用户通过设备树或其他配置方式灵活地定义和配置处理器芯片上的引脚功能和属性。这种可配置性使得系统能够适应不同的硬件平台和应用需求。

2、统一的接口: Pinctrl 子系统提供了一个统一的接口, 使得设备驱动程序和其他子系统能够以统一的方式配置和控制引脚。这简化了驱动程序开发和维护, 并提高了代码的可移植性。

3、动态配置和控制: Pinctrl 子系统允许设备驱动程序在运行时动态地配置和控制引脚, 包括切换引脚功能模式、设置引脚电平、启用/禁用引脚中断等。这种动态性使得系统能够根据运行时的需求进行灵活的调整和优化。

4、设备树绑定支持: Pinctrl 子系统通过设备树绑定来管理引脚的配置和控制。设备树提供了一种灵活的方式来描述硬件设备和其在系统中的连接关系, 使得设备驱动程序能够在不同的硬件平台上进行重用。

5、多功能支持: Pinctrl 子系统支持多种不同的引脚功能模式, 包括 GPIO、I2C、SPI、UART 等, 使得处理器芯片上的引脚能够适应各种不同的外设和通信接口。

6、硬件抽象层支持: Pinctrl 子系统提供了一种硬件抽象层, 使得驱动程序能够以统一的方式访问不同硬件平台上的引脚。这种抽象层隐藏了硬件细节, 简化了驱动程序的开发和维护。

总的来说, Pinctrl 子系统在 Linux 内核中扮演着至关重要的角色, 它提供了一种灵活、统一的方式来管理处理器芯片上的引脚, 从而提高了系统的可扩展性、可移植性和可维护性。

5.1.2 组成部分

Pinctrl 子系统由以下几个主要组成部分组成:

1、引脚描述符 (Pin Descriptor): 引脚描述符是对处理器芯片上每个引脚的抽象描述, 其中包括引脚编号、功能模式、电气属性等信息。这些描述符通常在设备树中进行定义, 以便内核可以在运行时使用。

2、引脚配置 (Pin Configuration): 引脚配置是指在系统启动时或运行时通过设备树等方式定义引脚的功能模式、电气属性等。这些配置信息告诉内核如何初始化引脚以及它们的初始状态。

3、引脚控制 (Pin Control): 引脚控制是通过内核提供的 API 函数来动态地配置和控制引脚的行为。这包括在运行时更改引脚的功能模式、设置引脚的电平状态、启用或禁用引脚中断等操作。

4、设备树绑定 (Device Tree Binding)： 设备树绑定是描述引脚配置和控制信息的一种方式，它通常用于在设备树中定义引脚的功能模式、电气属性等。内核根据这些绑定信息来初始化和配置引脚。

5、设备树节点 (Device Tree Nodes)： 设备树节点包含了引脚的配置信息，如引脚编号、功能模式、电气属性等。内核在启动时会解析设备树节点，并根据其中的信息来初始化和配置引脚。

6、硬件抽象层 (Hardware Abstraction Layer, HAL)： 硬件抽象层是 Pinctrl 子系统提供的一组抽象接口，它隐藏了不同硬件平台之间的差异，使得驱动程序可以以统一的方式访问不同平台上的引脚。

这些组成部分共同构成了 Pinctrl 子系统，在 Linux 内核中起着管理处理器芯片引脚配置和控制的重要作用。

5.1.3 应用场景

Pinctrl 子系统在 Linux 内核中的应用场景非常广泛，主要包括以下几个方面：

1、设备驱动程序开发： Pinctrl 子系统为设备驱动程序提供了统一的接口，使得驱动程序能够以统一的方式配置和控制处理器芯片上的引脚。这种统一的接口简化了驱动程序的开发和维护，使得驱动程序能够在不同的硬件平台上进行重用。

2、嵌入式系统开发： 在嵌入式系统中，处理器芯片上的引脚通常被用于连接外部设备或其他芯片，如 GPIO、I2C、SPI、UART 等。Pinctrl 子系统可以帮助开发人员灵活地配置和控制这些引脚，以满足系统对外设和通信接口的需求。

3、设备树配置： 设备树是一种描述硬件设备和其在系统中连接关系的数据结构，在嵌入式系统中被广泛应用。Pinctrl 子系统通过设备树绑定来管理引脚的配置和控制，开发人员可以在设备树中定义引脚的功能模式、电气属性等信息。

4、嵌入式系统调试和优化： Pinctrl 子系统允许开发人员在运行时动态地配置和控制引脚，包括切换引脚功能模式、设置引脚电平、启用/禁用引脚中断等。这使得开发人员能够在系统运行时对引脚进行灵活的调试和优化，提高系统的性能和稳定性。

5、嵌入式系统集成： 在嵌入式系统集成过程中，Pinctrl 子系统可以帮助开发人员解决引脚配置和冲突等问题，确保不同的硬件组件能够正确地连接和通信。

总的来说，Pinctrl 子系统在嵌入式系统开发中起着至关重要的作用，它提供了一种灵活、统一的方式来管理处理器芯片上的引脚，从而提高了系统的灵活性、可移植性和可维护性。

5.2 Pinctrl 和内核其他子系统的关系

Pinctrl (Pin Control) 子系统与其他内核外设系统 (如 GPIO、I2C、SPI 等) 相比，有着一些区别和联系：

区别：

1、功能定位不同： Pinctrl 主要负责处理器芯片上引脚的配置和控制，包括将引脚配置为不同的功能模式 (如 GPIO、I2C、SPI 等) 以及设置引脚的电气属性等。而 GPIO、I2C、SPI 等外设系统则是针对具体的外部设备或通信接口，提供了相应的驱动程序和 API 函数，用于实现对这些设备或接口的控制和操作。

2、功能范围不同： Pinctrl 涵盖了处理器芯片上所有引脚的配置和控制，而其他外设系统则专注于特定类型的设备或通信接口，如 GPIO 用于通用输入输出、I2C 用于串行通信、SPI 用于高速串行通信等。



3、抽象层级不同： Pinctrl 提供了一个底层的硬件抽象层，用于管理处理器上的引脚配置和控制，而其他外设系统则提供了更高层次的抽象，用于操作特定类型的外部设备或通信接口。

联系：

1、功能交叉： Pinctrl 可以配置引脚为 GPIO、I2C、SPI 等不同的功能模式，因此它与 GPIO、I2C、SPI 等外设系统存在一定的交叉。例如，可以使用 Pinctrl 将引脚配置为 GPIO 模式，然后使用 GPIO 子系统来进行通用输入输出操作；或者将引脚配置为 I2C 模式，然后使用 I2C 子系统来实现与外部 I2C 设备的通信。

2、设备树配置： 在 Linux 系统中，Pinctrl 和其他外设系统通常都会使用设备树来描述硬件设备和配置信息。因此，它们之间在设备树配置方面存在联系，开发者可以在设备树中定义引脚的配置信息以及外部设备的连接关系，从而由内核根据设备树进行初始化和配置。

总的来说，Pinctrl 与其他内核外设系统在功能定位、功能范围和抽象层级等方面有所不同，但它们之间存在一定的联系，可以共同为嵌入式系统提供灵活、可配置的硬件支持。

5.3 拓展：Pinctrl 使用案例

前面笔者提到，Pinctrl 和其他外设接口是功能交叉的关系，在外设配置的过程中往往会用到 Pinctrl 来将引脚配置成对应的模式，比如后面章节笔者的 GPIO 配置过程中，就用到了 Pinctrl。在后面的其他子系统章节中，也会用到 Pinctrl 的配置。

企业用户如果需要对开发板、核心板外设进行裁剪和配置，也会用到 Pinctrl，具体可以参考【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册。



第六章 LED 子系统

6.1 LED 基础知识

1、LED 的基本概念

LED（发光二极管，Light Emitting Diode）是一种半导体器件，当电流通过它时能够发光。与传统光源相比，LED 具有高效、节能、寿命长等优点，被广泛应用于各种照明和显示设备中。

2、LED 的工作原理

LED 的发光原理基于半导体的 PN 结。当电流从 P 型半导体流向 N 型半导体时，电子和空穴在 PN 结处复合，释放出光子，从而发光。

3、LED 的特性

正向电压（Forward Voltage）：LED 工作时的电压降，通常为 2V 至 3.6V 之间，具体值取决于材料和颜色。

正向电流（Forward Current）：LED 工作时的电流，通常在 10mA 至 30mA 之间，具体值取决于 LED 的规格。

亮度（Luminous Intensity）：LED 的发光强度，通常以毫烛光（mcd）或流明（lm）为单位。

颜色（Color）：LED 发光的颜色，取决于半导体材料的能隙。常见颜色有红、绿、蓝、黄、白等。

4、LED 的应用

照明：如家庭照明、街道照明、车灯等。

显示：如 LED 显示屏、指示灯等。

信号指示：如电子设备的电源指示灯、状态指示灯等。

背光：如液晶显示屏的背光源。

5、LED 的电路设计

在使用 LED 时，需要通过限流电阻来控制电流，避免过流损坏 LED。

6、在嵌入式系统中控制 LED

在嵌入式 Linux 中，可以通过控制 GPIO 来操作 LED。

6.2 LED 子系统简介

嵌入式 Linux 中的 LED 子系统是内核的一部分，用于控制和管理各种 LED 设备。它提供了一个抽象层，使用户和应用程序可以方便地操作 LED，而不需要关心底层硬件的具体实现。

6.2.1 主要特点

嵌入式 Linux 的 LED 子系统具有一系列特点，使其在嵌入式系统中得以广泛应用。以下是 LED 子系统的主要特点：

1、统一抽象层

LED 子系统为各种 LED 设备提供了一个统一的抽象层，隐藏了底层硬件差异，简化了上层应用程序的开发。

2、设备树支持

LED 子系统支持通过设备树（Device Tree）进行配置，这使得在不同硬件平台上配置 LED 设备更加方便和直观。设备树定义了 LED 设备的属性及其关联的 GPIO 引脚，允许在内核启动时自动配置 LED 设备。

3、标准化接口

通过 sysfs 文件系统，LED 子系统向用户空间提供了标准化的接口。用户和应用程序可以通过这些接口方便地控制 LED 的状态和属性。

4、多种触发模式

LED 子系统支持多种触发模式（trigger），可以根据系统状态自动控制 LED 的行为。

6.2.2 组成部分

嵌入式 Linux 的 LED 子系统由多个组成部分构成，这些部分协同工作以提供对 LED 设备的高效管理和控制。以下是 LED 子系统的主要组成部分：

1、LED 类驱动（LED Class Driver）

LED 类驱动是 LED 子系统的核心组件，定义了 LED 设备的通用接口和基本操作。它通过 sysfs 接口向用户空间暴露控制 LED 设备的文件，并提供对这些文件进行操作的方法。

- 文件位置：drivers/leds/led-class.c
- 功能：
 - ◆ 创建和管理 LED 设备节点
 - ◆ 提供统一的接口以便 LED 设备驱动程序注册和使用

2. LED 设备驱动（LED Device Driver）

LED 设备驱动是与具体硬件相关的驱动程序，实现对特定 LED 硬件的控制。它实现了 LED 类驱动定义的接口，并将具体的硬件操作封装起来。

- 文件位置：drivers/leds/
- 功能：
 - ◆ 初始化和配置 LED 硬件
 - ◆ 实现对 LED 设备的控制，如开关、亮度调节等

3. 触发器（LED Triggers）

触发器定义了 LED 的行为模式，根据系统事件或状态自动控制 LED。常见的触发器包括定时器触发、心跳触发等。触发器通过内核中的钩子函数与系统其他部分互动。

- 文件位置：drivers/leds/led-triggers.c
- 功能：



- ◆ 定义触发器类型和行为
- ◆ 实现触发器的注册和注销
- ◆ 处理系统事件并控制 LED

4. sysfs 接口

sysfs 是 Linux 内核提供的虚拟文件系统，通过它可以从用户空间访问和控制 LED 设备。LED 类驱动通过 sysfs 接口向用户空间暴露文件，这些文件用于控制 LED 的亮度、状态和触发器等。

- 典型路径: `/sys/class/leds/<led_name>/`
- 主要文件:
 - ◆ `brightness`: 控制 LED 亮度
 - ◆ `trigger`: 设置 LED 的触发模式
 - ◆ `delay_on` 和 `delay_off`: 在定时器触发模式下设置 LED 的闪烁时间

5. 设备树 (Device Tree)

设备树用于描述硬件信息，LED 设备的配置通常通过设备树进行定义。设备树节点中包括了 LED 设备的属性、关联的 GPIO 引脚等信息。

文件位置: 设备树源文件 (DTS)，这里以 `stm32mp257d-atk-ddr-2GB.dts` 为例:

```
1 gpio-leds {
2     compatible = "gpio-leds";
3     status = "okay";
4
5     led {
6         function = LED_FUNCTION_HEARTBEAT;
7         color = <LED_COLOR_ID_RED>;
8         gpios = <&gpioh 4 GPIO_ACTIVE_LOW>;
9         linux,default-trigger = "heartbeat";
10        default-state = "off";
11    };
12 };
```

6. 内核配置选项

为了启用 LED 子系统及其相关功能，需要在内核配置中启用相应的选项。可以通过 `menuconfig` 工具进行配置。

位置:

```
Device Drivers --->
  [*] LED Support --->
    <*> LED Class Support
    <*> LED Support for GPIO connected LEDs
```

7. 用户空间工具和库

用户空间工具和库可以简化对 LED 设备的控制，提供更高层次的抽象。常见的用户空间工具包括 `shell` 脚本和应用程序。

示例:

```
cat /sys/class/leds/red:heartbeat/trigger //查看 LED 的当前触发方式及支持的触发方式
echo none > /sys/class/leds/red:heartbeat/trigger //改变 LED 的触发方式，设置为 none
```



```
echo 1 > /sys/class/leds/red:heartbeat/brightness //点亮 LED
echo 0 > /sys/class/leds/red:heartbeat/brightness //熄灭 LED
echo heartbeat > /sys/class/leds/red:heartbeat/trigger //设置 LED 的触发方式为心跳 heartbeat
echo timer > /sys/class/leds/red:heartbeat/trigger //将 LED 触发模式设置为定时器模式
echo 500 > /sys/class/leds/red:heartbeat/delay_on //设置 LED 开启（点亮）时间为 500 毫秒
echo 500 > /sys/class/leds/red:heartbeat/delay_off //设置 LED 关闭（熄灭）时间为 500 毫秒
```

6.2.3 应用场景

嵌入式 Linux 的 LED 子系统在各种应用场景中具有广泛的应用。以下是一些常见的应用场景：

1、系统状态指示

电源状态指示、网络状态指示、工作状态指示，例如当设备正常工作时，指示灯亮起或闪烁；当设备出现故障时，指示灯以不同的频率闪烁或变成其他颜色。

2、用户交互反馈

按键反馈：在用户按下按钮时，通过点亮或闪烁 LED 来提供视觉反馈，确认用户操作已被接收。

用户提示：使用 LED 提示用户某些操作需要进行，例如需要插入存储设备、充电完成等。

3、调试和诊断

错误指示：在设备故障时，使用 LED 来指示具体的错误类型或状态。例如，设备可以通过不同的闪烁模式或颜色来指示内存错误、网络错误等。

状态监控：在设备开发和调试过程中，使用 LED 指示设备的当前状态，帮助开发人员实时监控和诊断设备状态。

4、安全和报警

安全报警：在检测到安全问题时，如门禁系统检测到非法入侵，使用 LED 闪烁或变色来发出警报。

环境监测：在环境监控系统中，LED 可以用来指示环境参数是否正常，例如温度过高或烟雾报警。

5、时间指示

定时器：使用 LED 指示定时器的状态，例如倒计时结束时，LED 亮起或闪烁。

心跳指示：在嵌入式系统中，使用 LED 以固定频率闪烁来指示系统的心跳，表明系统正在正常运行。ATK-DLMP257B 开发板的 LED 默认使用此功能。

6、用户界面

简单用户界面：在一些简易设备中，使用 LED 作为用户界面的一部分。例如，通过不同颜色和闪烁模式的组合，向用户传递设备的状态信息。

7、设备功能指示

存储状态指示：在存储设备（如路由器或网络存储设备）中，使用 LED 指示存储设备的读写状态，帮助用户了解存储设备的工作状态。

无线连接指示：在无线设备（如 Wi-Fi 路由器）中，使用 LED 指示无线连接状态，包括连接成功、信号强度等。

8、工业控制

生产线状态指示：在工业生产线中，使用 LED 指示生产线的运行状态、故障状态等，帮助工人实时了解生产情况。

设备状态监控：在工业设备中，使用 LED 指示设备的运行状态和故障状态，方便维护人员进行监控和诊断。

实际应用示例

示例 1：网络路由器

在网络路由器中，LED 子系统可以用来指示以下状态：

- 电源指示灯：指示路由器是否通电。
- 网络连接指示灯：指示 WAN 和 LAN 端口的连接状态。
- 无线信号指示灯：指示 Wi-Fi 状态和信号强度。
- 活动指示灯：指示网络数据传输活动。

示例 2：智能家居设备

在智能家居设备中，LED 子系统可以用来指示以下状态：

- 设备状态指示灯：指示设备是否在线、是否正常工作。
- 报警指示灯：指示烟雾、温度等安全报警状态。
- 用户操作反馈：指示用户操作是否成功，如按键反馈等。

示例 3：工厂生产线

在工厂生产线中，LED 子系统可以用来指示以下状态：

- 生产状态指示灯：指示生产线是否正常运行、是否停机。
- 故障指示灯：指示生产线或设备是否出现故障。
- 操作提示灯：提示工人进行特定操作，如更换材料等。

6.3 开发板 LED 设备

6.3.1 原理图分析

打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 LED0，可以看到 GPIO_IO04 默认用做 LED0。

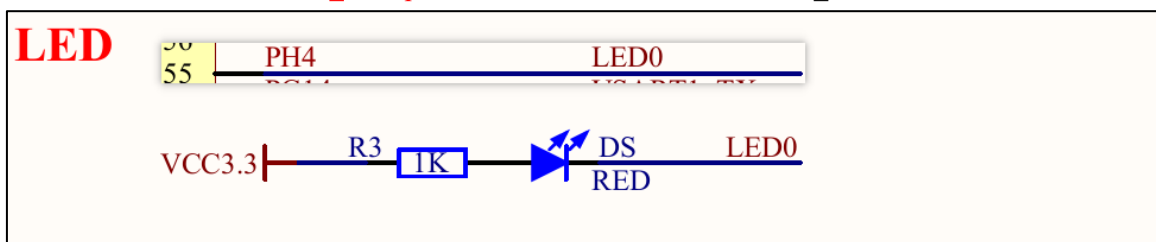


图 6.3-1 开发板 LED 原理图

原理图解析：

LED0：这是 LED 的正极，连接到 GPIO 引脚（GPIOH_IO04，核心板管脚序号 A55）。

R3：这是限流电阻，值为 1K 欧姆（ Ω ），用于限制通过 LED 的电流，以防止 LED 被烧毁。

DS：这是实际的 LED（红色 LED）。

GND：这是电路的接地端。

此图中 LED 的连接方式是通过 GPIO 引脚设置低电平来点亮 LED。这种连接方式通常称为“低电平点亮”，因为当 GPIO 引脚设置低电平时，电流从 3.3V 流过电阻和 LED 到主控，点亮 LED。

6.3.2 实物硬件

开发板上的 LED 如下图所示，丝印为 DS1，图中是没启动系统的状态，启动出厂系统默认作为红色心跳灯闪烁：

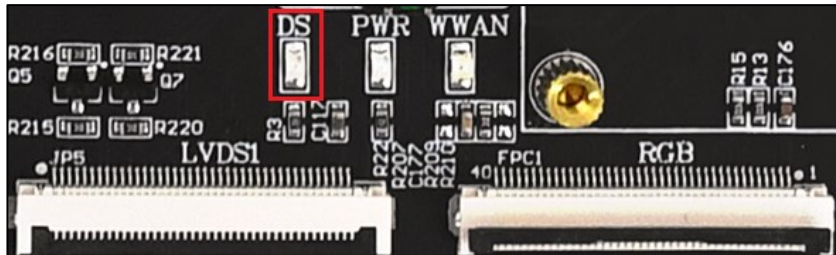


图 6.3-2 开发板 LED 实物

6.4 内核 LED 子系统使能

在 ATK-DLMP257B 内核源码配置文件 `arch/arm64/configs/stm32mp257_atk_defconfig` 中，LED 系统相关配置如下(略有删减，只列举重点部分)：

```
1 CONFIG_NEW_LEDS=y //启用新的 LED 框架支持
2 CONFIG_LEDS_CLASS=y //启用 LED 类支持
3
4 CONFIG_LEDS_GPIO=y //启用 GPIO LED 支持
5 CONFIG_LEDS_PWM=y //启用 PWM LED 支持
6 CONFIG_LEDS_TRIGGERS=y //启用 LED 触发器支持
7 CONFIG_LEDS_TRIGGER_TIMER=y //启用定时器触发器支持
8 CONFIG_LEDS_TRIGGER_HEARTBEAT=y //启用心跳触发器支持
9 CONFIG_LEDS_TRIGGER_DEFAULT_ON=y //启用默认开启触发器支持，
// 允许 LED 在系统启动时默认处于开启状态
10 .....
```

对应的 `menuconfig` 配置如下（篇幅有限，截图有所省略）：

```
-> Device Drivers
  -> LED Support
    -> <*> LED Class Support
    -> <*> LED Support for GPIO connected LEDs
    -> <*> PWM driven LED Support
    -> [*] LED support for LEDs on system controllers
    -> -* LED Trigger support
      -> <*> LED Timer Trigger
      -> [*] LED Disk Trigger
      -> <*> LED Heartbeat Trigger
      -> <*> LED Default ON Trigger
      -> [*] LED Panic Trigger
```

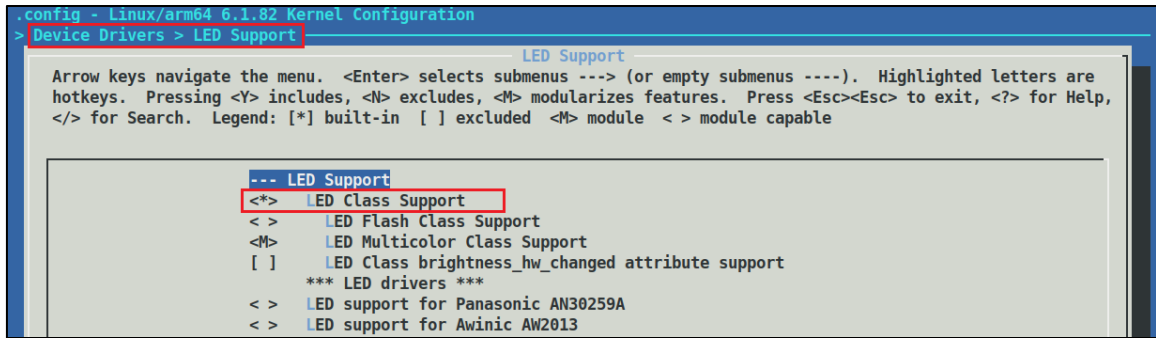


图 6.4-1 LED 配置（节选）

6.5 设备树文件分析

打开内核源码 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 文件，找到 gpio-leds 子节点。

```

1 / {
2     .....
3     gpio-leds {
4         compatible = "gpio-leds";
5         status = "okay";
6
7         led {
8             function = LED_FUNCTION_HEARTBEAT;
9             color = <LED_COLOR_ID_RED>;
10            gpios = <&gpioh 4 GPIO_ACTIVE_LOW>;
11            linux,default-trigger = "heartbeat";
12            default-state = "off";
13        };
14    };
15    .....
16 };
17

```

第 3 行，节点名称：gpio-leds，表示将使用 GPIO 控制的 LED 设备。

第 4 行，compatible: "gpio-leds"，表示这个节点兼容 GPIO 控制的 LED 设备驱动。对应的驱动源码为 drivers/leds/leds-gpio.c

第 5 行：status = "okay"；这个节点的状态为“正常”，表示该节点已经启用，并且应该由操作系统或驱动程序管理。

第 7 行：led 这个节点定义了具体的 LED 配置，通常一个 gpio-leds 节点下可以有多个 led 配置节点。

第 8 行：function = LED_FUNCTION_HEARTBEAT；指定 LED 的功能为心跳指示灯。这个功能通常会使用 LED 以一定频率闪烁，以表示设备的健康状态（例如，系统正在运行且一切正常）。

第 9 行：color = <LED_COLOR_ID_RED>;定义 LED 的颜色，这里指定的是红色。LED 的颜色通常是通过驱动程序来处理的，具体颜色通过定义标识符（如红色、绿色等）来传达。

第 10 行: `gpios = <&gpioh 4 GPIO_ACTIVE_LOW>`;这个属性定义了 LED 与 GPIO 的连接关系。`&gpioh` 指向 GPIOH 端口（设备树中的 GPIO 控制器），4 表示 LED 连接的 GPIO 引脚号（GPIOH 端口的第 4 个引脚）。`GPIO_ACTIVE_LOW` 表示 LED 在低电平时点亮（即当 GPIO 引脚输出低电平时，LED 会亮起）。

第 11 行: `linux,default-trigger = "heartbeat"`;该属性指定 LED 的默认触发事件是“heartbeat”（心跳）。这意味着 LED 会按照心跳信号闪烁，以显示系统健康状态。

第 12 行: `default-state = "off"`;这个属性定义 LED 的初始状态。`off` 表示 LED 在设备启动时默认为关闭状态，直到触发心跳事件。

6.6 LED 测试方法

6.6.1 使用 sysfs 方式操控 LED

在【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南.pdf 的点亮 LED 章节中，有关于 LED 的使用说明。

LED 通过 sysfs 方式进行操控，进入到 `/sys/class/leds` 目录下，如下所示：

```
root@ATK-DLMP257:~# cd /sys/class/leds/
root@ATK-DLMP257:/sys/class/leds# ls
red:heartbeat
root@ATK-DLMP257:/sys/class/leds#
```

图 6.6-1 /sys/class/leds 目录

从上图可以看到该目录下有一个 `red:heartbeat`，这个便是底板上的用户 LED 设备对应的文件夹，进入到文件夹 `red:heartbeat` 目录下，如下所示：

```
root@ATK-DLMP257:/sys/class/leds# ls -l
total 0
lrwxrwxrwx 1 root root 0 Jan  1 2000 red:heartbeat -> ../../devices/platform/gpio-leds/leds/red:heartbeat
root@ATK-DLMP257:/sys/class/leds# cd red:heartbeat/
root@ATK-DLMP257:/sys/class/leds/red:heartbeat# ls
brightness device invert max_brightness power subsystem trigger uevent
root@ATK-DLMP257:/sys/class/leds/red:heartbeat#
```

图 6.6-2 red:heartbeat 文件夹

从上图可知，实际上 `red:heartbeat` 是链接文件，链接到 `/sys/devices/platform/gpio-leds/leds/red:heartbeat` 目录，led 对应板子上的 DS；这里我们主要关注便是 `brightness`、`max_brightness` 以及 `trigger` 三个文件，这三个文件都是 LED 设备的属性文件：

- **brightness:** 翻译过来就是亮度的意思，该属性文件可读可写；所以这个属性文件是用于设置 LED 的亮度等级或者获取当前 LED 的亮度等级，譬如 `brightness` 等于 0 表示 LED 灭，`brightness` 为正整数表示 LED 亮，其值越大、LED 越亮；对于 PWM 控制的 LED 来说，这通常是适用的，因为它存在亮度等级的问题，不同的亮度等级对应不同的占空比，自然 LED 的亮度也是不同的；但对于 GPIO 控制（控制 GPIO 输出高低电平）的 LED 来说，通常不存在亮度等级这样的说法，只有 LED 亮（`brightness` 等于 0）和 LED 灭（`brightness` 为非 0 值的正整数）两种状态，ATK-DLMP257B 开发板上的这颗 LED 就是如此，所以自然就不存在亮度等级一说，只有亮和灭两种亮度等级。
- **max_brightness:** 该属性文件只能被读取，不能写，用于获取 LED 设备的最大亮度等级。
- **trigger:** 触发模式，该属性文件可读可写，读表示获取 LED 当前的触发模式，写表示设置 LED 的触发模式。不同的触发模式其触发条件不同，LED 设备会根据不同的触发条件自动控制其亮、灭状态，通过 `cat` 命令查看该属性文件，可获取 LED 支持的所有触发模式以及 LED 当前被设置的触发模式：

```
root@ATK-DLMP257:/sys/class/leds/red:heartbeat# cat trigger
none rfkill-any rfkill-none kbd-scrolllock kbd-numlock kbd-capslock kbd-kanalock kbd-
shiftlock kbd-altgrlock kbd-ctrllock kbd-altlock kbd-shiftllock kbd-shiftrlock kbd-ct
rllock kbd-ctrlrlock timer disk-activity disk-read disk-write ide-disk [heartbeat] c
pu cpu0 cpu1 default-on panic mmc0 mmc1 rfkill0 rfkill1
root@ATK-DLMP257:/sys/class/leds/red:heartbeat#
```

图 6.6-3 LED 支持的所有触发模式

方括号 ([heartbeat]) 括起来的表示当前 LED 对应的触发模式，none 表示无触发，常用的触发模式包括 none（无触发）、mmc0（当对 mmc0 设备发起读写操作的时候 LED 会闪烁）、timer（LED 会有规律的一亮一灭，被定时器控制住）、heartbeat（心跳呼吸模式，LED 模仿人的心跳呼吸那样亮灭变化）。

通常系统启动之后，会将板子上的一颗 LED 设置为 heartbeat 触发模式，将其作为系统正常运行的指示灯，譬如我们的 ATK-DLMP257B 开发板出厂系统启动之后，底板上的 DS 就会处于心跳呼吸模式，这个大家自己观察便可知道。

通过上面的介绍，已经知道如何去控制开发板底板上的用户 LED 了，譬如通过 echo 命令进行控制：

```
echo timer > trigger    //将 LED 触发模式设置为 timer
echo none > trigger      //将 LED 触发模式设置为 none
echo 1 > brightness      //点亮 LED
echo 0 > brightness      //熄灭 LED
```

更多 LED 使用方法和应用编程请参考【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南.pdf



第七章 PWM 子系统

7.1 PWM 基础知识

PWM（脉宽调制，Pulse Width Modulation）是一种通过调节信号的占空比来控制模拟量的方法。它在嵌入式系统中有广泛的应用，例如控制电机速度、调节 LED 亮度、生成音频信号等。

1、PWM 的基本概念

PWM 通过改变信号的占空比（Duty Cycle）来控制功率传递。占空比是指信号在一个周期内高电平的时间占整个周期时间的比例。

- 占空比（Duty Cycle）：信号高电平持续时间与周期时间的比值。通常用百分比表示。
- 周期（Period）：信号重复的时间长度，通常以秒（s）或毫秒（ms）为单位。
- 频率（Frequency）：信号的重复频率，通常以赫兹（Hz）为单位。频率与周期的关系为：频率 = 1 / 周期。

2、PWM 的工作原理

PWM 信号在高电平和低电平之间快速切换，通过改变高电平的持续时间来控制平均输出电压。例如，50%占空比意味着高电平和低电平的时间相等，平均输出电压是电源电压的一半。

3、PWM 的应用

电机控制：通过改变占空比来调节电机的速度。

LED 调光：通过改变占空比来调节 LED 的亮度。

音频信号生成：通过 PWM 生成不同频率的音频信号。

电源调节：通过 PWM 控制开关电源的输出电压。

7.2 PWM 子系统简介

PWM (Pulse Width Modulation, 脉宽调制) 子系统是 Linux 内核中的一个重要组件, 用于生成脉宽调制信号。脉宽调制信号是一种周期性的数字信号, 其脉冲宽度可以通过调节占空比来控制。PWM 子系统通常与处理器上的定时器硬件模块相关联, 用于生成高精度的脉宽调制信号。

7.2.1 主要特点

PWM 子系统在 Linux 内核中具有以下主要特点:

1、脉宽调制信号生成: PWM 子系统主要用于生成脉宽调制信号, 这是一种周期性的数字信号, 其脉冲宽度可以通过调节占空比来控制。这种信号通常用于控制电机速度、LED 亮度、音频合成等应用。

2、多通道支持: PWM 子系统通常支持多个独立的 PWM 通道, 每个通道都可以独立配置和控制。这使得系统能够同时控制多个外部设备或执行多个不同的 PWM 任务。

3、高精度和稳定性: PWM 子系统通常与处理器上的定时器硬件模块相关联, 能够提供高精度和稳定的脉宽调制信号。这种高精度的输出信号对于一些需要精确控制的应用非常重要。

4、灵活的配置和控制: PWM 子系统提供了一组 API 函数, 允许设备驱动程序和用户空间应用程序对 PWM 通道进行灵活的配置和控制。这包括设置 PWM 频率、占空比、极性 etc 参数, 以及启用/禁用 PWM 输出。

5、设备树绑定支持: PWM 子系统通常通过设备树绑定来管理 PWM 通道的配置和控制。开发者可以在设备树中描述每个 PWM 通道的相关信息, 如引脚连接、时钟源、频率范围等。

6、应用广泛: PWM 子系统在嵌入式系统中有着广泛的应用, 包括电机控制、LED 亮度调节、音频合成、电源管理等领域。

总的来说, PWM 子系统提供了一种灵活、高精度的脉宽调制信号生成功能, 能够满足各种不同应用的需求。它在嵌入式系统中扮演着重要的角色, 为系统提供了精确的时间控制和信号调节能力。

7.2.2 组成部分

PWM 子系统在 Linux 内核中的组成部分通常包括以下几个方面:

1、PWM 核心代码: PWM 子系统的核心代码包括 PWM 框架及相关的数据结构、API 函数等。这些代码提供了对 PWM 硬件模块的抽象, 允许设备驱动程序和用户空间应用程序通过统一的接口来配置和控制 PWM 通道。

2、设备驱动程序: 每个具体的 PWM 硬件模块通常都有相应的设备驱动程序, 用于与硬件进行交互。这些驱动程序通过 PWM 框架提供的接口来注册 PWM 通道, 并实现对 PWM 硬件模块的初始化、配置和控制等功能。

3、设备树绑定: 在 Linux 系统中, PWM 通道的配置信息通常会以设备树的形式进行描述。设备树中的节点包含了 PWM 通道的相关属性, 如引脚连接、时钟源、频率范围等。PWM 子系统会根据设备树中的描述来初始化和配置 PWM 通道。

4、用户空间接口: PWM 子系统提供了一组 API 函数, 允许用户空间应用程序对 PWM 通道进行配置和控制。这些接口通常包括打开/关闭 PWM 通道、设置 PWM 频率、占空比等操作。

5、时钟管理: PWM 通道通常需要一个时钟源来生成脉宽调制信号。因此, PWM 子系统可能会涉及时钟管理模块, 用于配置和控制时钟源的频率和分频等参数。

6、中断处理：一些 PWM 硬件模块支持中断功能，用于在特定事件发生时通知处理器。因此，PWM 子系统可能会涉及到中断处理模块，用于注册和处理 PWM 通道相关的中断。

综上所述，PWM 子系统的组成部分包括核心代码、设备驱动程序、设备树绑定、用户空间接口、时钟管理和中断处理等模块，它们共同构成了对 PWM 硬件模块的抽象和管理，使得系统能够灵活地配置和控制 PWM 通道，满足不同应用的需求。

7.2.3 应用场景

PWM 子系统在嵌入式系统中有着广泛的应用场景，其中一些主要的应用包括：

1、电机控制：PWM 信号常用于控制电机的转速和方向。通过调节 PWM 信号的占空比，可以控制电机的平均电压和功率，从而实现精确的速度调节和方向控制。

2、LED 亮度调节：PWM 信号可以用于控制 LED 的亮度。通过调节 PWM 信号的占空比，可以控制 LED 的平均功率，从而实现 LED 的亮度调节。

3、音频合成：PWM 信号可以用于音频合成和数字音频处理。通过调节 PWM 信号的频率和占空比，可以产生不同频率和幅度的音频信号，从而实现音频合成和音频效果的调节。

4、电源管理：PWM 信号常用于电源管理和电压调节。通过调节 PWM 信号的占空比，可以控制电源输出的电压和电流，从而实现电源管理和电压调节。

5、通信接口：PWM 信号可以用于模拟和数字通信接口。在一些特定的通信协议中，如脉冲编码调制（PCM）、脉冲位置调制（PPM）等，PWM 信号可以作为通信信号进行数据传输。

6、温度控制：PWM 信号可以用于温度控制和温度调节。通过调节 PWM 信号的占空比，可以控制加热元件的功率和温度，从而实现温度控制和温度调节。

综上所述，PWM 子系统在嵌入式系统中具有多种应用场景，包括电机控制、LED 亮度调节、音频合成、电源管理、通信接口和温度控制等领域，它为系统提供了一种灵活、高效的脉宽调制信号生成功能，能够满足各种不同应用的需求。

7.3 MP257 的 PWM 说明

7.3.1 文档和资源

想要获取 MP257 系列处理器的 PWM 详细说明，最直接的方式是参考 ST 官方文档，包括：

1、参考手册（Reference Manual）：提供了详细的寄存器级说明，包括 PWM 模块的寄存器映射以及配置方法等。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南（即本文档）

应用开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图：【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

7.3.2 特性说明

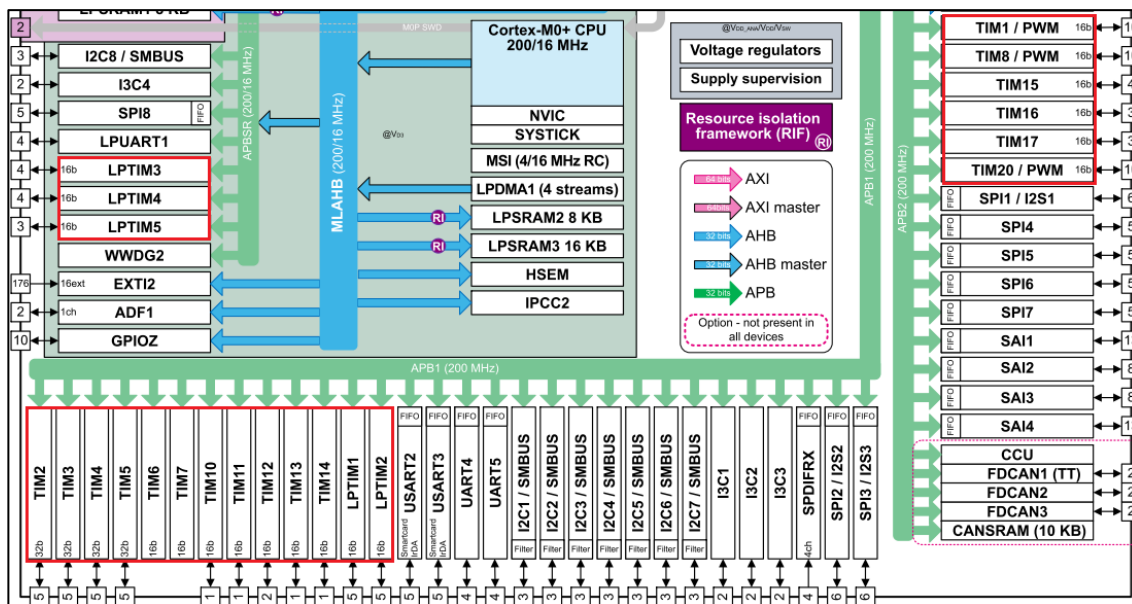


图 7.3-1 MP257 功能框图中的 PWM 部分

根据 MP257 芯片数据手册可知:

TIM1/TIM8/TIM20 高级定时器各有 4 个通道, 每个通道可用于 PWM, 共 12 路 PWM。

TIM2/TIM3/TIM4/TIM5 通用定时器各有 4 个通道, 每个通道可用于 PWM, 共 16 路 PWM。

TIM10/TIM11/TIM13/TIM14/TIM16/TIM17 定时器各具有一个独立通道可以用于 PWM, 而 TIM12 和 TIM15 具有两个独立通道, 可用于 PWM。共 10 路 PWM。

LPTIM1/2/3/4/5 低功耗定时器可以用于 PWM, 共 5 路 PWM。

更新信息请查阅 MP257 芯片数据手册和 MP257 芯片参考手册。

7.4 开发板 PWM 接口

7.4.1 原理图分析

开发板上散热风扇使用到了 PWM。打开开发板上【正点原子】STM32MP257 开发板 (A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf, 搜索 FAN, 可以看到 GPIOB_IO10 默认用做 TIM1_CH4 (即定时器 timers1 的通道 4), 此管脚是 FAN 信号线, 使用 PWM 功能来调节散热风扇。

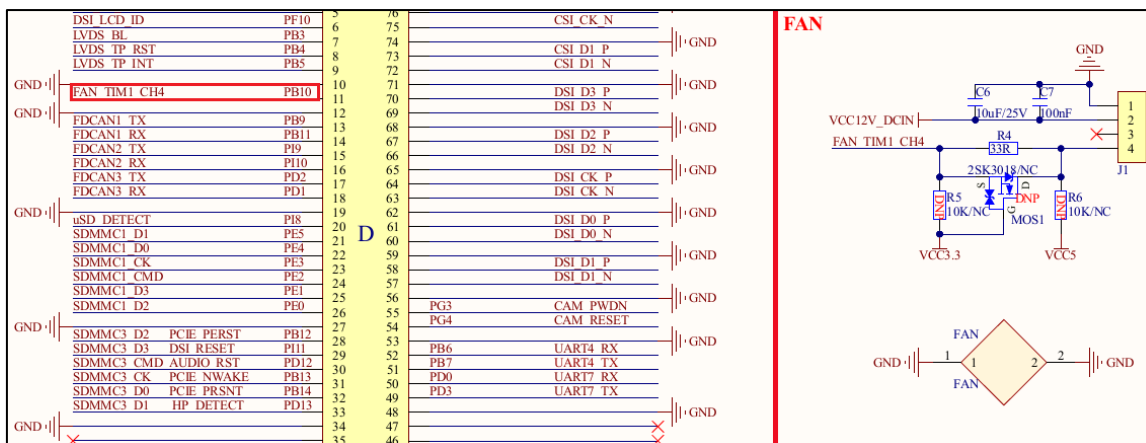


图 7.4-1 FAN TIM1_CH4 管脚

在开发板的 FAN 接口上可以找到对应的 PWM 背光管脚, 但是接口 IO 比较密集, 不方便测试。为了方便测试, ATK-DLMP257B 开发板的 GPIO 接口上引出了 GPIOB_IO10 管脚, 此管脚 GPIO 编号为 GPIOB_IO10, 和开发板上 FAN 接口的 PWM 管脚是共用的。

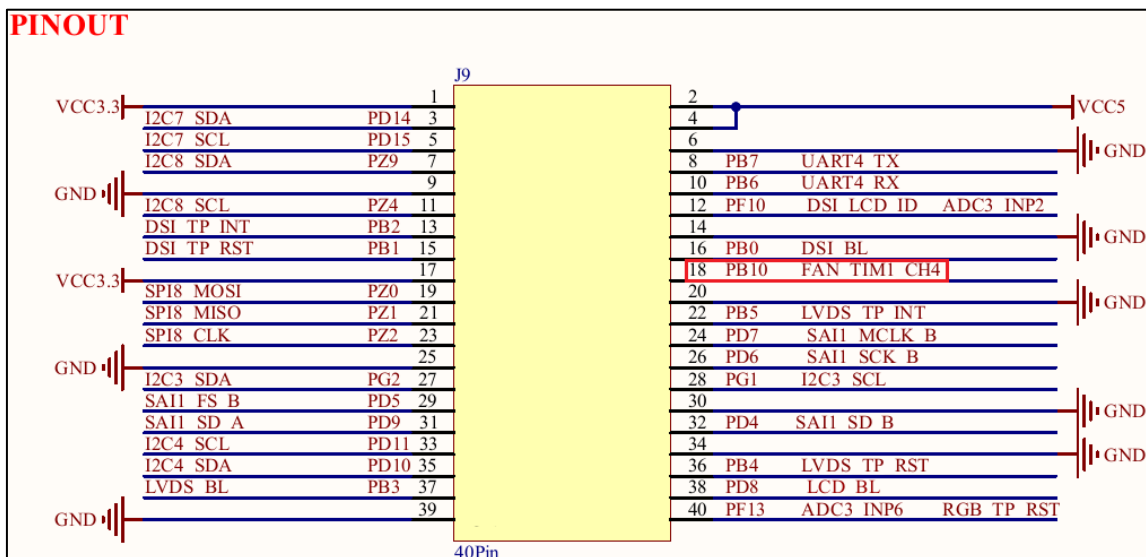


图 7.4-2 GPIOB_IO10 管脚

7.4.2 实物硬件

开发板上的排针和 GPIOB_IO10 如下图所示:

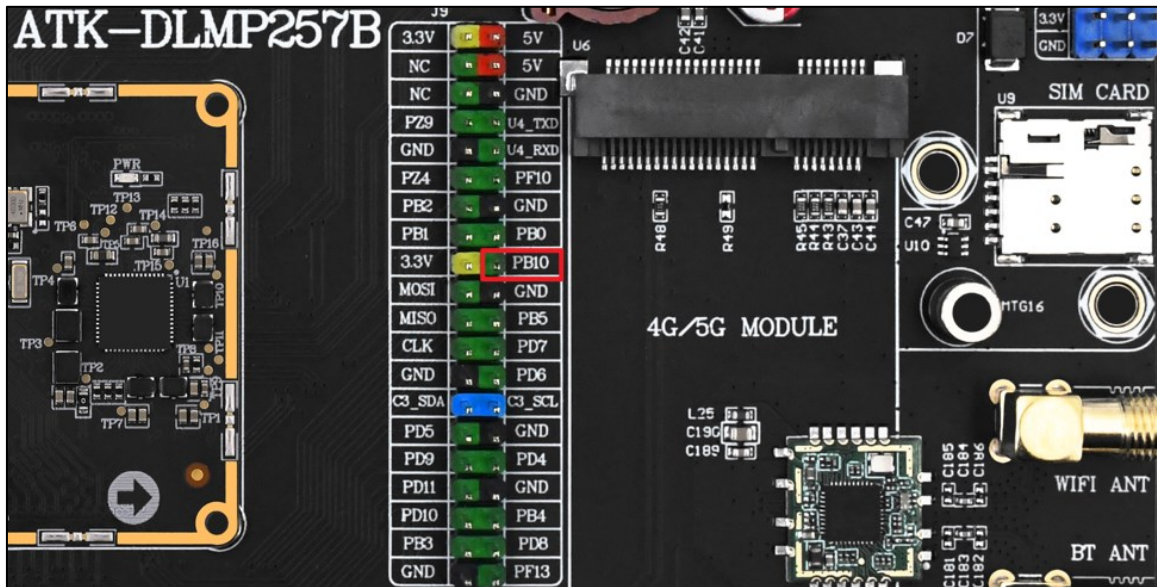


图 7.4-3 GPIOB_IO10 管脚

7.5 内核 PWM 子系统使能

在 ATK-DLMP257B 内核源码配置文件 `arch/arm64/configs/stm32mp257_atk_defconfig` 中, PWM 系统配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_BACKLIGHT_PWM=y      //启用 PWM 背光支持
2 CONFIG_BACKLIGHT_GPIO=y     //允许通过 GPIO 引脚控制背光
3 CONFIG_LEDS_PWM=y           //启用使用 PWM 控制 LED 灯的支持
4 CONFIG_COMMON_CLK_PWM=y     //为 PWM 设置一个公共时钟框架
5 CONFIG_PWM=y                //在内核中启用对 PWM 驱动程序的支持
6 CONFIG_PWM_SYSFS=y          //为 PWM 提供一个 sysfs 接口
7 CONFIG_PWM_STM32=y          //提供 ST 的 PWM 驱动
8 CONFIG_PWM_STM32_LP=y       //提供 ST 的低功耗 PWM 驱动
9 CONFIG_SENSORS_PWM_FAN=m    //风扇 PWM 驱动
10 .....
```

这些配置反映了一个系统的 PWM 支持情况, 包括基础架构、访问接口和一些额外的功能。用户可以使用 `menuconfig` 进行配置, 在 `menuconfig` 中搜索上面相关配置即可查看具体说明。

PWM 对应的 `menuconfig` 配置如下: (篇幅有限, 截图有所省略):

```
-> Device Drivers
  -> [*] Pulse-Width Modulation (PWM) Support
      .....
      -> <*>  STMicroelectronics STM32 PWM
      -> <*>  STMicroelectronics STM32 PWM LP
      .....
```

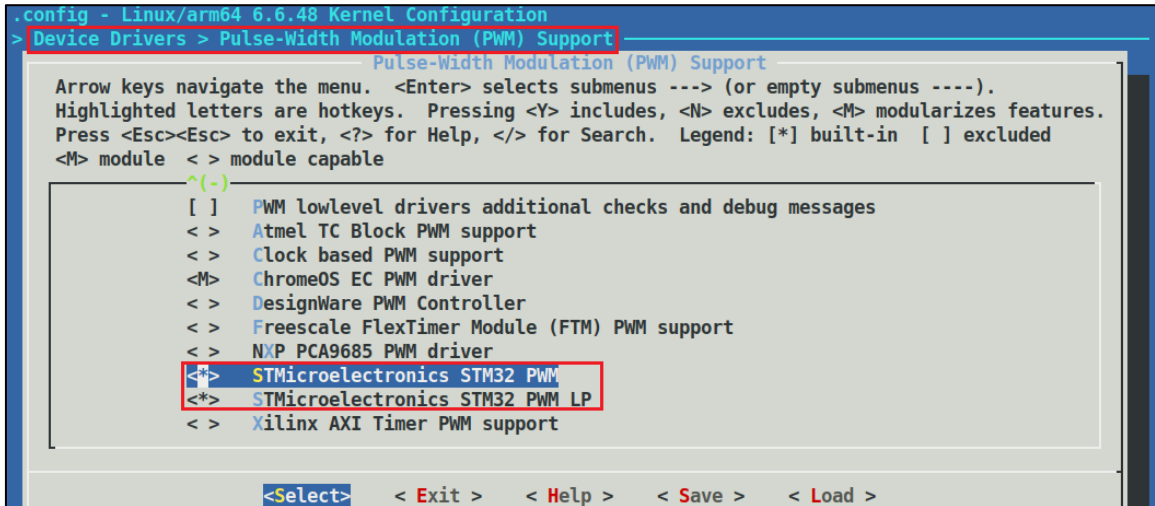


图 7.5-1 PWM 配置

风扇 PWM 的配置如下:

```

-> Device Drivers
    -> Graphics support
        -> Hardware Monitoring support (HWMON [=y])
            .....
            -> PWM fan (SENSORS_PWM_FAN [=m])
    
```

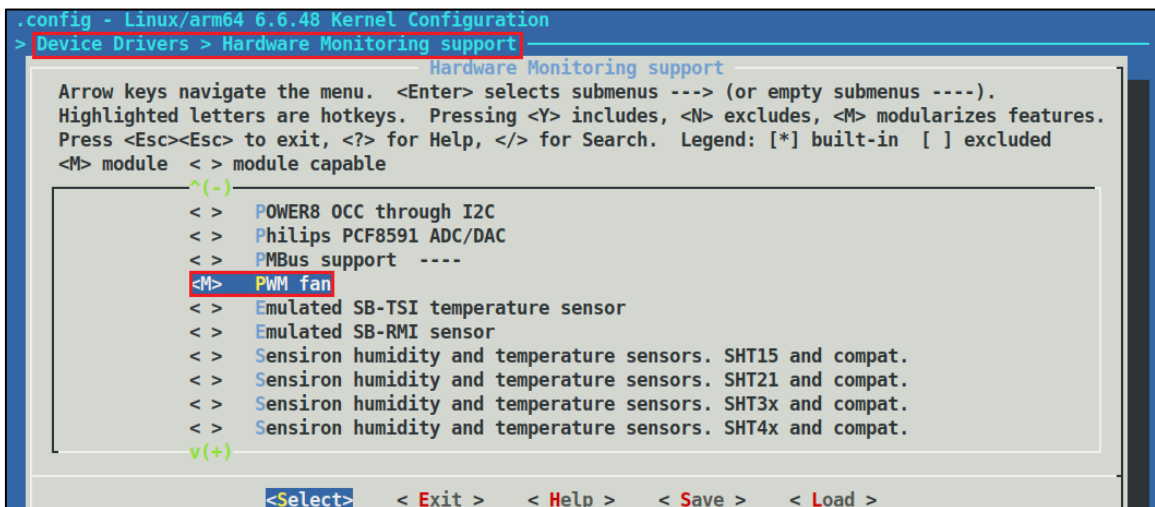


图 7.5-2 风扇 PWM 配置

7.6 设备树文件分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi, 此文件是特定于 MP257 系列芯片的设备树包含文件, 通常不独立使用, 而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置, 而 .dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 timers17 的描述如下:

```

1      timers17: timer@40270000 {
2          compatible = "st,stm32mp25-timers";
3          reg = <0x40270000 0x400>;
    
```

```

4      interrupts = <GIC_SPI 195 IRQ_TYPE_LEVEL_HIGH>;
5      interrupt-names = "global";
6      clocks = <&rcc CK_KER_TIM17>;
7      clock-names = "int";
8      #address-cells = <1>;
9      #size-cells = <0>;
10     access-controllers = <&rifsc 15>;
11     power-domains = <&CLUSTER_PD>;
12     status = "disabled";
13
14     counter {
15         compatible = "st,stm32mp25-timer-counter";
16         status = "disabled";
17     };
18
19     pwm {
20         compatible = "st,stm32mp25-pwm";
21         #pwm-cells = <3>;
22         status = "disabled";
23     };
24
25     timer@16 {
26         compatible = "st,stm32mp25-timer-trigger";
27         reg = <16>;
28         status = "disabled";
29     };
30 };

```

以 timers17 为例，对上述配置进行具体说明：

第 1 行，`timers17: timer@40270000` 定义一个名为 `timers17` 的定时器节点，其基地址为 `0x40270000`。`timers17` 是标签，用于引用该定时器节点。

第 2 行，`compatible = "st,stm32mp25-timers"`；指定该节点兼容 `st,stm32mp25-timers` 驱动，用于识别 STM32MP25 系列的定时器。

第 3 行，`reg = <0x40270000 0x400>`；定义寄存器的地址和大小范围，`0x40270000` 是定时器的起始地址，`0x400` 表示寄存器的大小为 1 KB。

第 4 行，`interrupts = <GIC_SPI 195 IRQ_TYPE_LEVEL_HIGH>`；定义了中断信息，使用 GIC SPI 中断号 195，中断类型为“高电平触发”。

第 5 行，`interrupt-names = "global"`；为中断命名为“global”，便于在代码中引用该中断。

第 6 行，`clocks = <&rcc CK_KER_TIM17>`；指定定时器使用的时钟源，`&rcc` 指向时钟控制器，`CK_KER_TIM17` 表示时钟标识。

第 7 行，`clock-names = "int"`；为时钟源命名为“int”（内部时钟）。

第 8 行，`#address-cells = <1>`；指定在此节点的子节点中使用单个地址单元。

第 9 行，`#size-cells = <0>`；指定子节点中不需要使用大小单元。

第 10 行, `access-controllers = <&rifsc 15>` 指定定时器的特性域, `&rifsc` 是特性控制器的引用。

第 11 行, `power-domains = <&CLUSTER_PD>`;指定电源域, `&CLUSTER_PD` 是电源控制器的引用, 用于管理定时器的电源状态。

第 12 行, `status = "disabled"`;设置该节点状态为“disabled”, 即默认禁用定时器。

第 13-16 行, `counter { ... }`;定义一个子节点 `counter`, 配置为计数器:

`compatible = "st,stm32mp25-timer-counter"`; 兼容 `st,stm32mp25-timer-counter` 驱动, 用于计数。

`status = "disabled"`; 设置计数器的状态为“disabled”。

第 17-21 行, `pwm { ... }`;定义一个子节点 `pwm` (脉冲宽度调制):

`compatible = "st,stm32mp25-pwm"`;表示该节点与 `st,stm32mp25-pwm` 驱动兼容, 用于 PWM 功能, 驱动文件为 `/drivers/pwm/pwm-stm32.c`。

`#pwm-cells = <3>`; 定义 `pwm` 单元数为 3。

`status = "disabled"`; 将 PWM 节点状态设置为“disabled”。

第 22-26 行, `timer@16 { ... }`;定义一个子节点 `timer@16`, 配置为触发定时器:

`compatible = "st,stm32mp25-timer-trigger"`; 表示该节点与 `st,stm32mp25-timer-trigger` 驱动兼容, 用于触发定时器。

`reg = <16>`; 定义子节点寄存器的偏移地址为 16。

`status = "disabled"`; 设置触发定时器的状态为“disabled”。

7.7 PWM 测试方法

7.7.1 使用 sysfs 方式操控 PWM

在【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南.pdf 的 PWM 应用编程章节中, 有关于 PWM 的使用说明。

PWM 通过 sysfs 方式进行操控, 进入到 `/sys/class/pwm` 目录下。

注意事项:

如果开发板接了屏幕, 比如 MIPI 屏幕, 会有背光引脚占用 `pwmchip0`, 即会出现 2 个 PWM 文件, 此时屏幕背光 PWM 占用 `pwmchip0`, 而风扇电路 PWM 会被调整为占用 `pwmchip1`。如果没有接屏幕, 则只有一个 `pwmchip0`, 此时这个就是风扇 PWM。

```
root@ATK-DLMP257:~# ls /sys/class/pwm/
pwmchip0 pwmchip1
root@ATK-DLMP257:~#
```

开发板接了屏幕时的PWM设备

```
root@ATK-DLMP257:~# ls /sys/class/pwm
pwmchip0
root@ATK-DLMP257:~#
```

开发板不接屏幕时的PWM设备

图 7.7-1 /sys/class/pwm 目录下的内容

这里笔者**开发板没有接屏幕**, 列举出了文件夹 `pwmchip0`, 这其实就对应了 ATK-DLMP257B 的 1 个 PWM 控制器。对于我们的出厂系统来说, `pwmchip0` 对应的是 ATK-DLMP257B 的 `timers17`, (注意并不是 `pwmchip0` 对应 `TIM17`, 这个与序号没有直接关联, 具体可以结合设备树里的外设地址查看)。事实上, ATK-DLMP257B 一共有多个 PWM 控制器, 上图中只是列举出了一个, 因为其它的 PWM 控制器外设我们的出厂系统并未使能。

我们进入到 pwmchip0 目录下:

```
root@ATK-DLMP257:~# cd /sys/class/pwm/
root@ATK-DLMP257:/sys/class/pwm# ls
pwmchip0
root@ATK-DLMP257:/sys/class/pwm# cd pwmchip0/
root@ATK-DLMP257:/sys/class/pwm/pwmchip0# ls
device export npwm power subsystem uevent unexport
root@ATK-DLMP257:/sys/class/pwm/pwmchip0#
```

图 7.7-2 pwmchip0 目录下的内容

在这个目录下我们重点关注的是 export、npwm 以及 unexport 这三个属性文件，下面一一进行介绍:

- **npwm:** 这是一个只读属性，读取该文件可以得知该 PWM 控制器下共有几路 PWM 输出，如下所示:

```
root@ATK-DLMP257:/sys/class/pwm/pwmchip0# cat npwm
1
root@ATK-DLMP257:/sys/class/pwm/pwmchip0#
```

图 7.7-3 读取 npwm 属性文件

所以由此可知，该 PWM 控制器 (TIM17) 一共有 1 路 PWM 输出，通过查阅 MP257 参考手册也可知，TIM17 确实支持 1 通道 PWM 输出 (TIM17_CH1N\TIM17_CH1，尾缀 N 代表极性相反)。

- **export:** 与 GPIO 控制一样，在使用 PWM 之前，也需要将其导出，通过 export 属性进行导出，以下所示:

```
echo 0 > export
```

```
root@ATK-DLMP257:/sys/class/pwm/pwmchip0# echo 0 > export
root@ATK-DLMP257:/sys/class/pwm/pwmchip0# ls
device export npwm power pwm0 subsystem uevent unexport
root@ATK-DLMP257:/sys/class/pwm/pwmchip0#
```

图 7.7-4 导出 TIM17_CH1N

数字 0 表示一个编号，注意，每个 PWM 控制器 (pwmchipX) 下，使用 export 属性文件导出 PWM 通道时，编号都是从 0 开始 (对于 TIM17 来说，0 表示 TIM17_CH1\TIM17_CH1N 通道，以此类推)；譬如导出 TIM_CH1N 通道，如下所示:

```
echo 0 > /sys/class/pwm/pwmchip0/export #导出 TIM17_CH1N 通道
```

导出成功后会在 pwmchipX (X 表示数字) 目录下生成一个名为 pwm0 的目录，如图所示，稍后介绍。

- **unexport:** 将导出的 PWM 通道删除。当使用完 PWM 之后，我们需要将导出的 PWM 通道删除，譬如:

```
echo 0 > unexport
```

写入到 unexport 文件中的编号与写入到 export 文件中的编号是相对应的；需要注意的是，export 文件和 unexport 文件都是只写的、没有读权限。

如何控制 PWM

上例中通过 export 导出之后，便会生成 pwm0 这个目录，我们进入到该目录下看看:

```
root@ATK-DLMP257:/sys/class/pwm/pwmchip0# cd pwm0/
root@ATK-DLMP257:/sys/class/pwm/pwmchip0/pwm0# ls
capture duty_cycle enable period polarity power uevent
root@ATK-DLMP257:/sys/class/pwm/pwmchip0/pwm0#
```

图 7.7-5 pwm0 目录下的内容

该目录下也有一些属性文件，我们重点关注 `duty_cycle`、`enable`、`period` 以及 `polarity` 这四个属性文件，接下来一一进行介绍。

- **polarity:** 用于设置极性，可读可写，可写入的值如下：

"normal": 普通；

"inversed": 反转；

```
echo normal > polarity          #默认极性
```

```
echo inversed > polarity        #极性反转
```

很多 SoC 的 PWM 外设其硬件上并不支持极性配置，所以对应的驱动程序中并未实现这个接口，应用层自然也就无法通过 `polarity` 属性文件对 PWM 极性进行配置。

- **period:** 用于配置 PWM 周期，可读可写；写入一个字符串数字值，以 ns（纳秒）为单位，譬如配置 PWM 周期为 10us（微秒）：

```
echo 10000 > period             #PWM 周期设置为 10us (10 * 1000ns)
```

- **duty_cycle:** 用于配置 PWM 的占空比，可读可写；写入一个字符串数字值，同样也是以 ns 为单位，譬如：

```
echo 5000 > duty_cycle          #PWM 占空比设置为 5us
```

- **enable:** 可读可写，写入"0"表示禁止 PWM；写入"1"表示使能 PWM。读取该文件获取 PWM 当前是禁止还是使能状态。

```
echo 0 > enable                 #禁止 PWM 输出
```

```
echo 1 > enable                 #使能 PWM 输出
```

通常配置好 PWM 之后，再使能 PWM。

7.7.2 目标管脚 PWM 测试

在系统中查找设备树 pwm 相关节点，笔者操作如下图所示：

```
ls /sys/firmware/devicetree/base/soc@0/pinctrl@44240000
```

```
root@ATK-DLMP257:~# ls /sys/firmware/devicetree/base/soc@0/pinctrl@44240000
'#address-cells'      gpio@442a0000          m-can3-sleep-0        sdmmc1-b4-0
'#size-cells'         gpio@442b0000          name                  sdmmc1-b4-od-0
adcl-in15             gpio@442c0000          pcie-0                sdmmc1-b4-sleep-0
compatible            gpio@442d0000          pcie-init-0           sdmmc2-b4-0
eth1-mdio-0           gpio@442e0000          phandle               sdmmc2-b4-od-0
eth1-mdio-sleep-0     i2c3-0                pwm1-0                sdmmc2-b4-sleep-0
eth1-rgmii-0          i2c3-sleep-0          pwm1-sleep-0          sdmmc2-d47-0
eth1-rgmii-sleep-0    i2c4-0                pwm16-0               sdmmc2-d47-sleep-0
eth2-rgmii-0          i2c4-sleep-0          pwm16-sleep-0         st,package
eth2-rgmii-sleep-0    interrupt-parent       pwm17-0               uart7-0
eth3-rgmii-0          interrupts-extended    pwm17-sleep-0         uart7-idle-0
eth3-rgmii-sleep-0    ltcd-0                pwm20-0               uart7-sleep-0
gpio@44240000          ltcd-sleep-0          pwm20-sleep-0         usart1-0
gpio@44250000          m-can1-0              ranges                usart1-idle-0
gpio@44260000          m-can1-sleep-0        saila-0               usart1-sleep-0
gpio@44270000          m-can2-0              saila-sleep-0         usart2-0
gpio@44280000          m-can2-sleep-0        sailb-0               usart2-idle-0
gpio@44290000          m-can3-0              sailb-sleep-0         usart2-sleep-0
root@ATK-DLMP257:~#
```

图 7.7-6 系统中的设备树节点信息

将示波器接到开发板的 GPIOB_IO10 接口和 GND，后续将使用示波器查看 PWM 波形。

开发板不接屏幕的情况下，进入到/sys/class/pwm/目录下，可以一个 pwmchip0，进入 pwmchip0 就可以操作对应的 PWM 功能。

```
cd /sys/class/pwm/
cd pwmchip0
```

```
root@ATK-DLMP257:~# cd /sys/class/pwm/
root@ATK-DLMP257:/sys/class/pwm# cd pwmchip0
root@ATK-DLMP257:/sys/class/pwm/pwmchip0#
root@ATK-DLMP257:/sys/class/pwm/pwmchip0#
root@ATK-DLMP257:/sys/class/pwm/pwmchip0# ls
device export npwm power pwm0 subsystem uevent unexport
root@ATK-DLMP257:/sys/class/pwm/pwmchip0#
```

图 7.7-7 进入 pwmchip0

导出 PWM 通道，因为复用的功能是 TIM17_CH1N，极性相反，对应的通道是 0，笔者这里导出通道 0，并进入通道 0，配置 PWM、周期设置为 500us、占空比设置为 50%，默认极性，最后使能 PWM。

```
echo 0 > /sys/class/pwm/pwmchip0/export
echo 500000 > /sys/class/pwm/pwmchip0/pwm0/period
echo 250000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
echo "normal" > /sys/class/pwm/pwmchip0/pwm0/polarity
echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
```

```
root@ATK-DLMP257:~# echo 0 > /sys/class/pwm/pwmchip0/export
root@ATK-DLMP257:~# echo 500000 > /sys/class/pwm/pwmchip0/pwm0/period
root@ATK-DLMP257:~# echo 250000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
root@ATK-DLMP257:~# echo "normal" > /sys/class/pwm/pwmchip0/pwm0/polarity
root@ATK-DLMP257:~# echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
root@ATK-DLMP257:~#
```

图 7.7-8 TIM17_CH1 配置

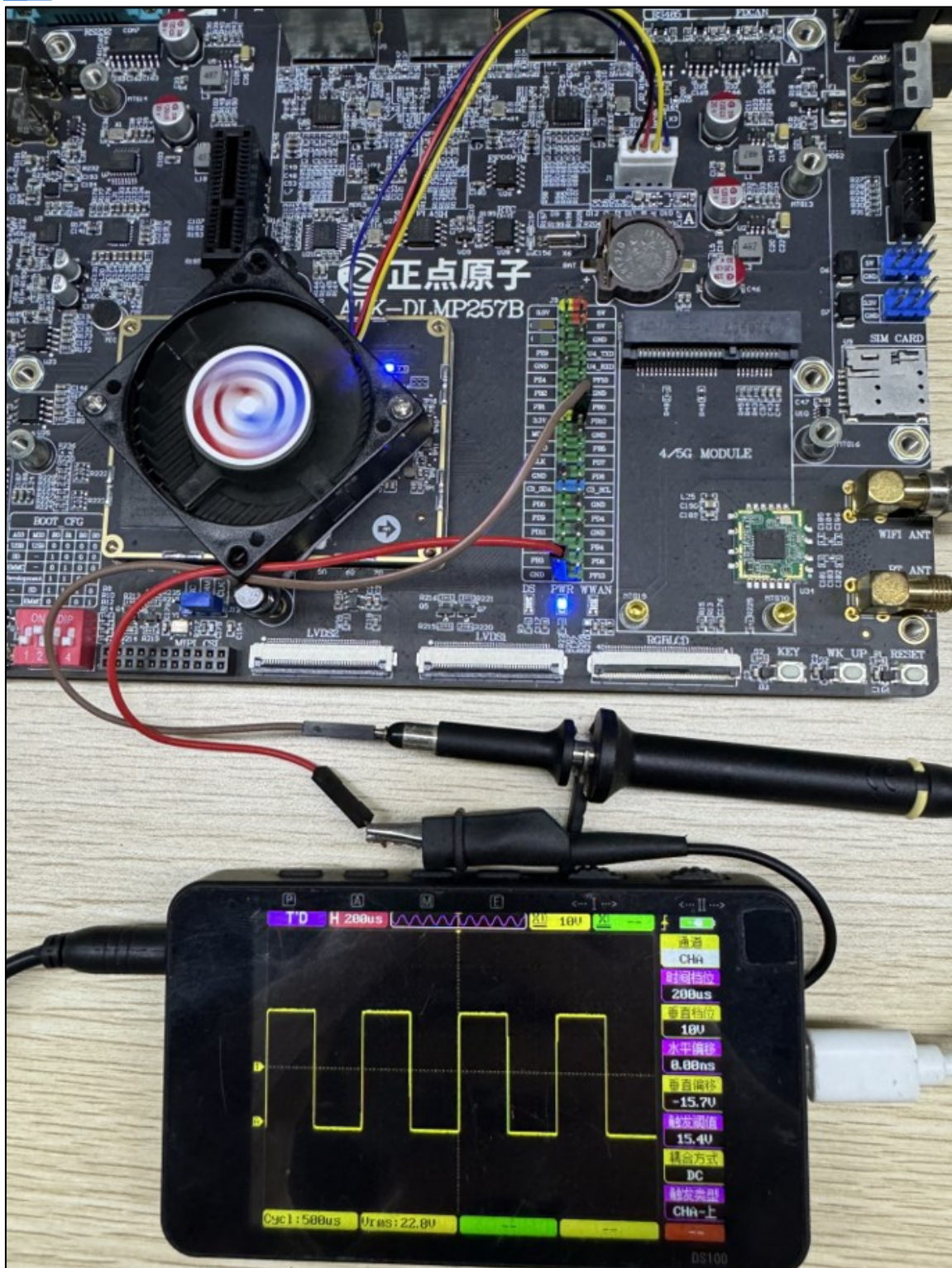


图 7.7-9 实验现象

如果有购买散热套件（即散热风扇）的话，可以通过风扇运行状态来观察 PWM 效果。例如在我们前面的基础上，修改并提高占空比为 86%（差不多 430000）

```
echo 430000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
```


因为散热风扇使用的是 TIM17_CH1N，此极性与 TIM17_CH1 相反。此款散热风扇高电平的频率越高，风扇转速越快，在没有手动修改极性的情况下，越提高占空比，高电平频率越低，风扇转速越慢。效果如下图所示：

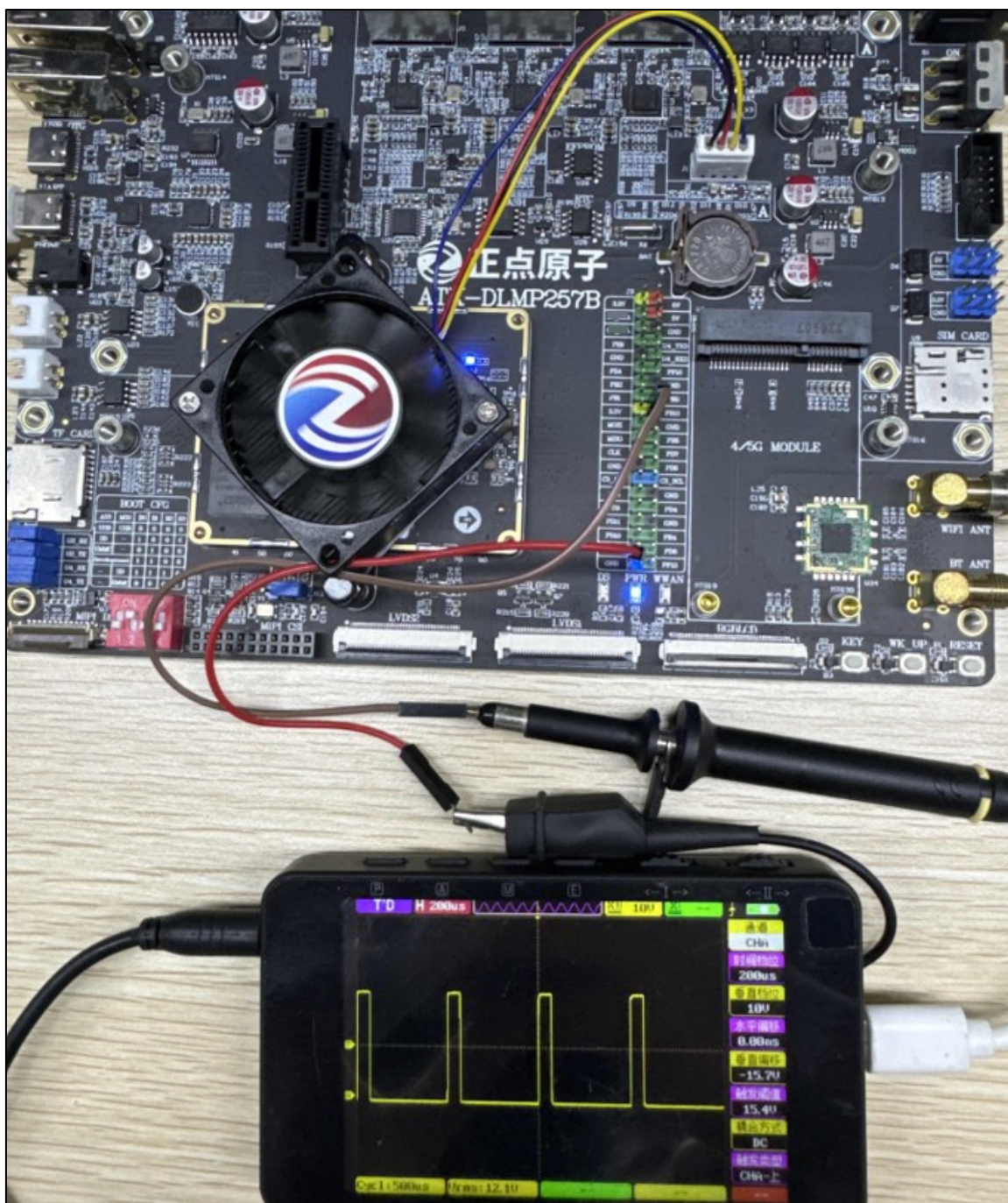


图 7.7-10 降低风扇转速效果

7.8 拓展：移除 PB10 的 PWM 功能

核心板为 1GB 的 DDR 版本的用户打开 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-1GB.dts，核心板为 2GB 的 DDR 版本的用户打开 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts，以下以 2GB 版本为例。

找到 timers17 节点和 pwm17 节点，并将其注释掉，如下所示：



```

520 //timers17 {
521 //     status = "okay";
522 //     /* spare dmas for other usage */
523 //     /delete-property/dmas;
524 //     /delete-property/dma-names;
525 //
526 //     pwm17: pwm {
527 //         pinctrl-0 = <&pwm17_pins_a>;
528 //         pinctrl-1 = <&pwm17_sleep_pins_a>;
529 //         pinctrl-names = "default", "sleep";
530 //         #pwm-cells = <2>;
531 //         status = "okay";
532 //     };
533 //};

```

图 7.8-1 禁止timers17 节点和 pwm17 节点状态

打开 arch/arm64/boot/dts/st/stm32mp25-pinctrl-atk-ddr-2GB.dtsi, 注释掉pinctrl 节点下的 pwm17_pins_a 和 pwm17_sleep_pins_a 节点:

```

651 //     pwm17_pins_a: pwm17-0 {
652 //         pins {
653 //             pinmux = <STM32_PINMUX('B', 10, AF7)>; /* TIM17_CH1N */
654 //             bias-pull-down;
655 //             drive-push-pull;
656 //             slew-rate = <0>;
657 //         };
658 //     };
659 //
660 //     pwm17_sleep_pins_a: pwm17-sleep-0 {
661 //         pins {
662 //             pinmux = <STM32_PINMUX('B', 10, ANALOG)>; /* TIM17_CH1N */
663 //         };
664 //     };

```

图 7.8-2 移除 PB10 的复用成 PWM 功能

保存文件, 根据核心板版本编译内核源码, 这里笔者使用 2GB 版本:

```

./build_kernel.sh
2

```

编译完成后, 返回源码上级路径的 build_image 目录下, 将编译得到的 stm32mp257d-atk-ddr-2GB.dtb 设备树更新到开发板的/boot 目录。

第八章 GPIO 子系统

8.1 GPIO 基础知识

1、GPIO 的基本概念

GPIO (General Purpose Input/Output) 引脚是一种通用的数字信号引脚, 能够在输入和输出之间进行配置, 用于与外部设备进行通信。

2、GPIO 引脚的配置模式

每个 GPIO 引脚可以配置为不同的模式, 根据所需的功能进行设定。主要有以下几种模式:

- 输入模式 (Input Mode): 用于读取外部设备的信号。引脚的状态由外部信号决定。
- 输出模式 (Output Mode): 用于向外部设备发送信号。引脚的状态由程序控制。
- 输入上拉模式 (Input Pull-Up Mode): 在输入模式的基础上, 启用内部上拉电阻, 以确保引脚在未连接外部信号时处于高电平状态。
- 输入下拉模式 (Input Pull-Down Mode): 在输入模式的基础上, 启用内部下拉电阻, 以确保引脚在未连接外部信号时处于低电平状态。

3、GPIO 引脚的电平状态

GPIO 引脚的电平状态有两种:

- 高电平 (High Level): 对应逻辑 1, 通常表示高电压 (如 3.3V 或 5V)
- 低电平 (Low Level): 对应逻辑 0, 通常表示低电压 (如 0V)

4、中断 (Interrupt)

中断是一种重要的机制, 可以在特定事件发生时打断正常的程序执行, 立即执行预先定义的中断服务程序 (ISR)。在 GPIO 中, 中断常用于处理按钮按下、传感器触发等事件。

5、内部上拉和下拉电阻

内部上拉和下拉电阻用于确保在引脚未连接外部信号时, 有一个确定的电平状态。通常用于输入引脚, 以避免浮空引脚的噪声干扰。

- 上拉电阻 (Pull-Up Resistor): 将未连接的引脚拉到高电平。
- 下拉电阻 (Pull-Down Resistor): 将未连接的引脚拉到低电平。

6、GPIO 的高级应用

GPIO 引脚可以用于更多高级应用, 如:

- PWM 输出: 生成脉宽调制信号, 用于控制电机速度、LED 亮度等。
- 模拟信号读取: 通过 ADC 将模拟信号转换为数字信号进行处理。
- 串行通信: 通过 I2C、SPI、UART 等接口实现与其他设备的通信。

7、GPIO 的实际应用

在实际项目中, GPIO 可以用于各种场景, 如:

- 控制 LED: 通过 GPIO 引脚控制 LED 的亮灭。
- 读取传感器数据: 连接温度传感器、光敏传感器等, 读取并处理传感器数据。
- 按钮输入: 检测按钮按下状态, 执行相应操作。
- 驱动继电器: 通过 GPIO 引脚控制继电器, 实现高电压设备的开关控制。

8.2 GPIO 子系统简介

Linux 内核的 GPIO (General Purpose Input/Output, 通用输入输出) 子系统是为了管理和控制硬件上的 GPIO 引脚而设计的。它允许开发者在内核空间和用户空间中控制引脚的状态, 实现输入和输出功能。这个子系统在嵌入式系统和硬件交互中扮演了重要的角色, 使得与外界环境的交互成为可能。

8.2.1 主要特点

1. 通用性: GPIO 子系统提供了一个统一的接口来操作不同硬件上的 GPIO 引脚, 确保了在不同平台间的可移植性和兼容性。
2. 灵活性: 支持将 GPIO 引脚配置为输入或输出模式, 以及配置上拉/下拉电阻, 满足不同的硬件需求。
3. 易用性: 通过提供用户空间的访问接口 (如 sysfs 和 GPIO character device API), 简化了对 GPIO 引脚的操作, 使得开发者可以不编写内核驱动程序即可控制 GPIO。
4. 中断支持: GPIO 子系统允许 GPIO 引脚被配置为中断源, 可以在特定事件 (如电平变化) 发生时通知处理器, 实现高效的事件响应。

8.2.2 组成部分

1. **gpiolib**: 是 Linux 内核中的 GPIO 核心库, 提供了一套标准的 API 来管理和控制 GPIO 引脚。它作为一个抽象层, 屏蔽了不同硬件之间的差异。
2. **GPIO 控制器驱动**: 为特定的硬件平台实现 GPIO 操作的细节。每个控制器驱动管理一组 GPIO 引脚, 并注册到 gpiolib, 以供上层使用。
3. **设备树 (Device Tree)**: 在许多基于 ARM 的系统中, GPIO 的配置 (如引脚编号、功能、默认状态等) 在设备树中静态描述, 内核通过解析设备树来初始化和配置 GPIO 系统。
4. **用户空间接口**:
 - **sysfs**: 早期 Linux 版本通过 sysfs 文件系统提供对 GPIO 引脚的直接访问。虽然简单, 但不推荐在新的开发中使用, 因为它缺乏必要的同步机制和保护。
 - **GPIO character device**: 是一个较新的接口, 提供了一种更加安全和高效的方式来从用户空间访问 GPIO 引脚, 是目前推荐的方法。

8.2.3 应用场景

GPIO 子系统广泛应用于嵌入式系统中, 用于控制 LED 灯、读取按钮状态、驱动继电器、读取传感器数据等。凭借其高度的灵活性和易用性, GPIO 子系统使得 Linux 内核能够有效地与各种硬件设备交互, 满足多样化的应用需求。

总的来说, GPIO 子系统是 Linux 内核与硬件交互不可或缺的一部分, 它通过提供一系列的抽象和接口, 简化了 GPIO 的管理和控制。随着技术的发展, GPIO 子系统也在不断进化, 以满足不断增长的性能和功能需求。

8.3 MP257 的 GPIO 说明

8.3.1 文档和资源

想要获取 MP257 系列处理器的 GPIO 详细说明,最直接的方式是参考 ST 官方文档,包括:

1、参考手册(Reference Manual):提供了详细的寄存器级说明,包括 GPIO 模块的寄存器映射以及配置方法等。此文档已放在【正点原子】STM32MP257 开发板(A 盘)-基础资料\07、STM32MP2 参考资料

2、数据手册(Data Sheet):提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板(A 盘)-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板,正点原子还提供了一系列的开发文档,便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发:【正点原子】STM32MP257 开发板(A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南(即本文档)

应用开发:【正点原子】STM32MP257 开发板(A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图:【正点原子】STM32MP257 开发板(A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明:【正点原子】STM32MP257 开发板(A 盘)-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考:【正点原子】STM32MP257 开发板(A 盘)-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

8.3.2 特性说明

MP257 处理器的 GPIO 模块通常具备以下特性:

1. 多功能性:GPIO 引脚在 MP257 处理器上是多功能的,这意味着同一个物理引脚可以根据需要被配置为 GPIO 模式或其他特定功能(如 I2C、SPI、UART 等通信接口)的一部分
2. 可编程性:GPIO 引脚的方向(输入或输出)、状态(高或低)、上下拉电阻配置等都可以通过软件来配置,为不同的应用需求提供灵活性。
3. 中断支持:MP257 的 GPIO 引脚支持配置为中断源,能够在检测到特定事件(如电平变化)时触发处理器中断,这对于实时应用来说非常重要。
4. 低功耗:在低功耗模式下,GPIO 状态可以被保持,有助于降低系统总体功耗,这对于电池供电的嵌入式设备尤其重要。

以下是 MP257 处理器数据手册的里关于 GPIO 的功能框图:

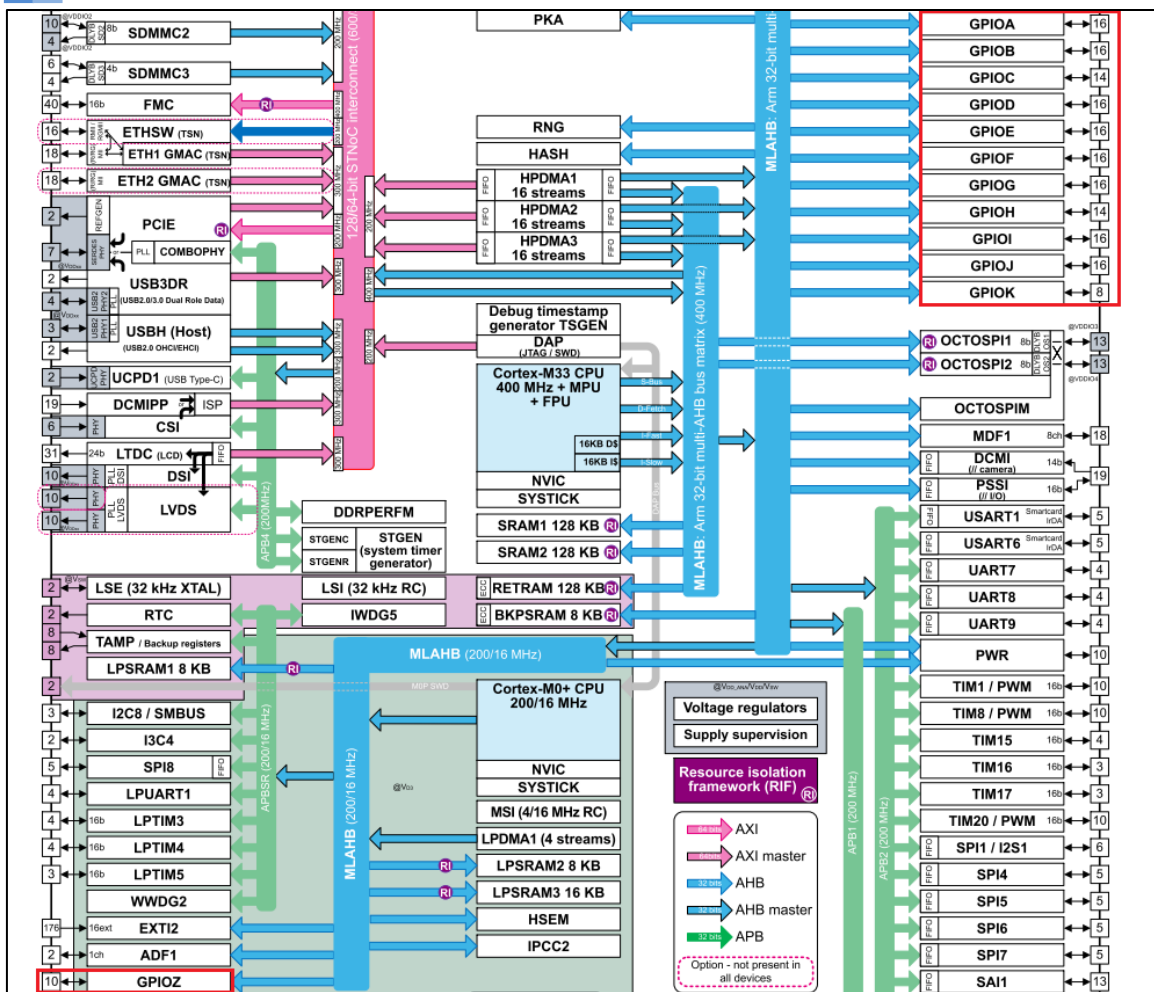


表 8.3.1 MP257 功能框图中的 GPIO

由上表可知,MP257 处理器最多有 12 个 GPIO 模块,分别对应 GPIOA(16 个 IO)、GPIOB(16 个 IO)、GPIOC(14 个 IO)、GPIOD(16 个 IO)、GPIOE(16 个 IO)、GPIOF(16 个 IO)、GPIOG(16 个 IO)、GPIOH(14 个 IO)、GPIOI(16 个 IO)、GPIOJ(16 个 IO)、GPIOK(8 个 IO)、GPIOZ(10 个 IO),共 174 个 GPIO。以下是 MP257 处理器数据手册的关于 GPIO 的说明。

General-purpose input/outputs (GPIO)

Each of the GPIO pins can be configured by software as output (push-pull or open-drain, with or without pull-up or pull-down), as input (with or without pull-up or pull-down), or as peripheral alternate function. Some of the GPIO pins are shared with digital or analog alternate functions. All GPIOs are high-current-capable and have speed selection to better manage internal noise, power consumption and electromagnetic emission.

After reset, all GPIOs are in analog mode to reduce power consumption.

The I/O configuration can be locked if needed by following a specific sequence in order to avoid spurious writing to the I/O registers.

Access to each GPIO configuration bits can be restricted to secure-only and/or privileged-only. These configuration bits can also be allocated to a specific CPU.

表 8.3.2 GPIO 说明

由上表可知 MP257 的每个 GPIO 引脚都可以通过软件配置为输出(推挽或开漏,带或不带上拉或下拉)、输入(带或不带上拉或下拉)或外设备用功能。一些 GPIO 引脚与数字或模拟备用功能共享。所有 GPIO 均支持高电流,并具有速度选择功能,可更好地管理内部噪声、功耗和电磁辐射。

更多特性说明和具体说明, 请参考【正点原子】STM32MP257 开发板(A 盘)-基础资料\07、MP257 参考资料\【参考手册】

8.4.2 实物硬件

开发板上的排针如下图所示:

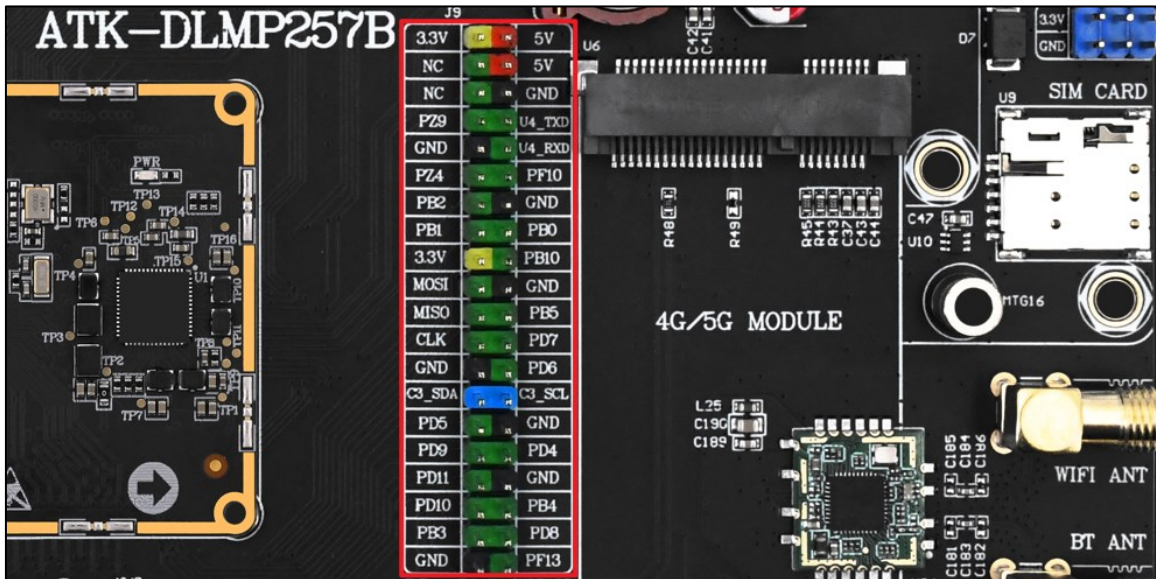


图 8.4.2 开发板上的 GPIO 排针

8.5 内核 GPIO 子系统使用

在 ATK-DLMP257B 内核源码配置文件 `arch/arm64/configs/stm32mp257_atk_defconfig` 中, GPIO 系统配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_GPIOLIB=y           //启用 GPIO 核心库
2 CONFIG_OF_GPIO=y           //允许通过设备树配置 GPIO
3 CONFIG_GPIOLIB_IRQCHIP=y   //允许 GPIO 引脚用于中断请求
4 CONFIG_GPIO_CDEV=y         //启用字符设备 (cdev) 接口来访问 GPIO
5 CONFIG_GPIO_CDEV_V1=y      //使用 GPIO 字符设备接口的版本 V1
6 CONFIG_GPIO_GENERIC=y      //启用通用 GPIO 框架, 提供了标准的 GPIO 实现
```

这些配置反映了一个系统的 GPIO 支持情况, 包括基础架构、访问接口和一些额外的功能。用户可以使用 `menuconfig` 进行配置, 在 `menuconfig` 中搜索上面相关配置即可查看具体说明。

对应的 `menuconfig` 配置如下 (篇幅有限, 截图有所省略):

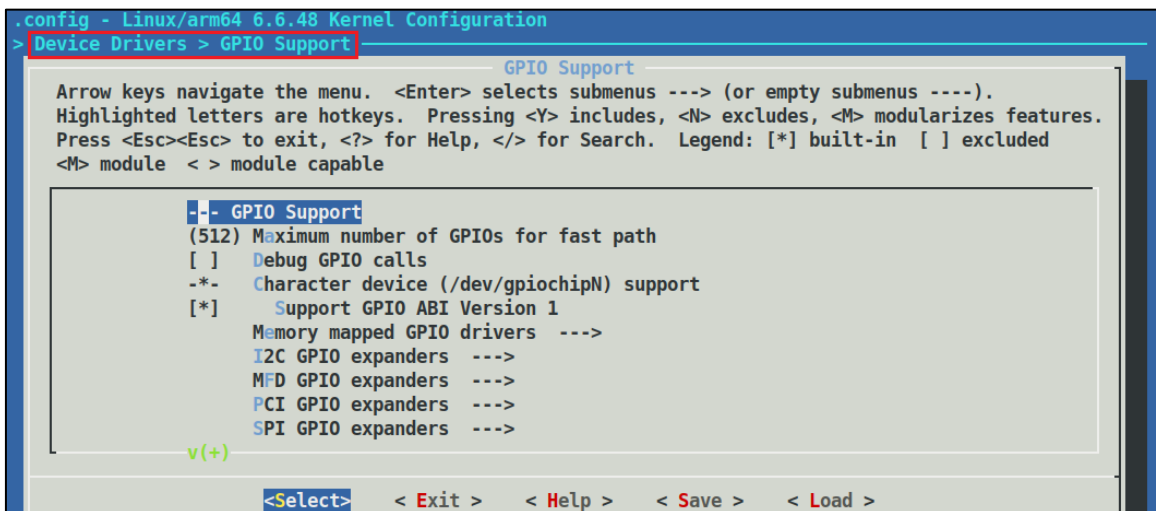


图 8.5-1 GPIO 配置

8.6 设备树文件分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi, 此文件是特定于 MP257 系列芯片的设备树包含文件, 通常不独立使用, 而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置, 而 .dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 GPIO 的描述如下 (以 gpiob 为例):

```
1  gpiob: gpio@44250000 {
2      gpio-controller;
3      #gpio-cells = <2>;
4      interrupt-controller;
5      #interrupt-cells = <2>;
6      reg = <0x10000 0x400>;
7      clocks = <&scmi_clk CK_SCMI_GPIOB>;
8      st,bank-name = "GPIOB";
9      status = "disabled";
10 };
```

以 gpiob 为例, 对上述配置进行具体说明:

第 1 行, 定义一个名为 gpiob 的节点, 地址为 0x44250000, 表示 GPIO 控制器的基地址。

第 2 行, 该节点是一个 GPIO 控制器, 声明此设备可以管理 GPIO 引脚。

第 3 行, 定义 GPIO 控制器的 #gpio-cells 属性, 值为 2。这表示在设备树中使用 GPIO 时, 绑定需要两个参数 (通常是 GPIO 引脚号和标志位)。

第 4 行, 声明该节点是一个中断控制器。

第 5 行, 定义中断控制器的 #interrupt-cells 属性, 值为 2。这表示描述中断时需要两个参数 (通常是中断号和触发方式)。

第 6 行, 定义寄存器的地址空间, 0x10000 是偏移地址, 0x400 是寄存器的大小。

第 7 行, 定义与该设备关联的时钟, &scmi_clk 是时钟控制器的引用, CK_SCMI_GPIOB 是具体的时钟标识符。

第 8 行, 自定义属性, 定义 GPIO 控制器的名称为 "GPIOB"。

第 9 行, 设备的状态为 disabled, 表示当前设备未启用。

这些配置共同确保了 GPIO 控制器在系统中的正确功能, 以及其与其他系统组件 (如中断控制器和时钟管理系统) 的适当交互。

8.7 GPIO 测试工具-libgpiod

libgpiod 是一个相对较新的 Linux 用户空间 GPIO 工具集, 它旨在提供一种简洁的方式来控制和监视系统上的 GPIO 引脚。它是基于 chardev GPIO 接口构建的, 这个接口从 Linux 4.8 版本开始被引入。libgpiod 替代了传统的基于 sysfs 的 GPIO 接口, 提供更高效和灵活的操作方式。在 github 上搜索 libgpiod 即可找到相关源码和使用说明。出厂系统 libgpiod 版本为 v2.0

主要特点

1. 现代接口: 使用基于字符设备的 GPIO 接口, 与旧的基于 sysfs 的方法相比, 这种方式更为高效, 特别是在处理大量 GPIO 时。

2. 工具集:

gpiointfo: 列出所有 gpiochip 信息, 包括每个 GPIO 控制器的所有 GPIO 线。

gpioreset: 设置 GPIO 线的状态（高或低）。

gpioget: 读取 GPIO 线的状态。

gpiomon: 监视 GPIO 线的中断事件。

更多工具集这里不再一一列举。

3. 编程接口: libgpiod 库提供了一套用于控制 GPIO 的 C 语言 API, 方便在脚本或应用程序中直接操作 GPIO。

使用场景

gpiod 工具适用于需要进行 GPIO 控制的各种场景, 如简单的硬件控制任务（例如 LED 控制、按钮输入）或更复杂的系统级硬件交互。由于其简洁和高效, gpiod 特别适合于嵌入式系统和工业应用中。

出厂系统上已经移植好了 gpiod 工具, 如下图所示:

```
root@ATK-DLMP257:~# cd /usr/bin/
root@ATK-DLMP257:/usr/bin#
root@ATK-DLMP257:/usr/bin# ls gpio*
gpiodetect  gpioget  gpioinfo  gpiomon  gpionotify  gpioreset
root@ATK-DLMP257:/usr/bin#
```

图 8.7-1 gpiod 相关工具

8.7.1 gpiodetect

gpiodetect 用于检测并显示系统中所有的 GPIO 控制器及其标签, 常用于了解特定硬件平台上可用 GPIO 资源的情况, 这个工具对于嵌入式系统开发者和硬件调试人员来说非常重要, 因为它允许他们快速了解和评估系统中的 GPIO 配置和可用性。

出厂系统 **gpiodetect** 的使用帮助信息如下:

```
root@ATK-DLMP257:/usr/bin# gpiodetect -h
Usage: gpiodetect [OPTIONS] [chip]...

List GPIO chips, print their labels and number of GPIO lines.

Chips may be identified by number, name, or path.
e.g. '0', 'gpiochip0', and '/dev/gpiochip0' all refer to the same chip.

If no chips are specified then all chips are listed.

Options:
  -h, --help            display this help and exit
  -v, --version          output version information and exit
root@ATK-DLMP257:/usr/bin#
```

图 8.7-2 gpiodetect -h

命令使用说明:

基本用法: 直接运行 **gpiodetect** 将列出所有 GPIO 控制器及其详细信息。

指定控制器: 可以通过控制器编号、名称或路径来指定特定的控制器。例如, 使用 0、gpiochip0 或/dev/gpiochip0 都将引用同一个控制器。

选项:

-h, --help: 显示帮助信息并退出。

-v, --version: 输出版本信息并退出。

出厂系统 **gpiodetect** 基本用法的参考示例如下:

```
root@ATK-DLMP257:/usr/bin# gpiodetect
gpiochip0 [GPIOA] (16 lines)
gpiochip1 [GPIOB] (16 lines)
gpiochip2 [GPIOC] (14 lines)
gpiochip3 [GPIOD] (16 lines)
gpiochip4 [GPIOE] (16 lines)
gpiochip5 [GPIOF] (16 lines)
gpiochip6 [GPIOG] (16 lines)
gpiochip7 [GPIOH] (14 lines)
gpiochip8 [GPIOI] (12 lines)
gpiochip9 [GPIOZ] (10 lines)
root@ATK-DLMP257:/usr/bin#
```

图 8.7-3 gpiodetect 基本用法

可以看到系统上有 10 组 GPIO, 每组 GPIO 有 10 到 16 个线路。

出厂系统 **gpiodetect** 指定控制器用法的参考示例如下:

```
root@ATK-DLMP257:/usr/bin# gpiodetect 0
gpiochip0 [GPIOA] (16 lines)
root@ATK-DLMP257:/usr/bin# gpiodetect /dev/gpiochip0
gpiochip0 [GPIOA] (16 lines)
```

图 8.7-4 gpiodetect 用法-指定控制器

8.7.2 gpioinfo

gpioinfo 使用一个用来打印 GPIO 线路信息的工具，它能提供每条线路的详细信息，如线路名称、状态、配置以及使用者信息。这个工具特别有助于了解特定 GPIO 线路的使用情况和属性。

出厂系统 gpioinfo 的使用帮助信息如下：

```
root@ATK-DLMP257:/usr/bin# gpioinfo
gpiochip0 - 16 lines:
    line 0:      "PA0"          input consumer="kernel"
    line 1:      "PA1"          input
    line 2:      "PA2"          input consumer="kernel"
    line 3:      "PA3"          input consumer="kernel"
    line 4:      "PA4"          input consumer="kernel"
    line 5:      "PA5"          input consumer="kernel"
    line 6:      "PA6"          input consumer="kernel"
    line 7:      "PA7"          input consumer="kernel"
    line 8:      "PA8"          input consumer="kernel"
    line 9:      "PA9"          input consumer="kernel"
    line 10:     "PA10"         input consumer="kernel"
    line 11:     "PA11"         input consumer="kernel"
    line 12:     "PA12"         input
    line 13:     "PA13"         input consumer="kernel"
    line 14:     "PA14"         input consumer="kernel"
    line 15:     "PA15"         input consumer="kernel"
gpiochip1 - 16 lines:
    line 0:      "PB0"          input
    line 1:      "PB1"          output
    line 2:      "PB2"          output
    line 3:      "PB3"          input
```

以下省略

图 8.7-5 gpioinfo -h

命令使用说明：

基本用法：直接运行 gpioinfo 将列出所有可用的 GPIO 线路及其详细信息。

指定线路：用户可以通过名称指定特定的线路，如果提供了 -c 或 --chip 选项，还可以通过偏移量指定。

选项：

- by-name: 即使线路名称可以解析为偏移量，也将其视为名称。
- c, --chip <chip>: 将操作限制在特定的控制器上。
- h, --help: 显示帮助信息并退出。
- s, --strict: 检查所有线路，不假设线路名称是唯一的。
- unquoted: 不为线路或使用者名称添加引号。
- v, --version: 输出版本信息并退出。

控制器识别：用户可以通过编号、名称或路径来指定一个 GPIO 控制器。例如，0、gpiochip0 和 /dev/gpiochip0 都指向同一个控制器。

出厂系统 gpioinfo 基本用法的参考示例如下：

```

root@ATK-DLMP257:/usr/bin# gpioinfo
gpiochip0 - 16 lines:
    line 0:      "PA0"           input consumer="kernel"
    line 1:      "PA1"           input
    line 2:      "PA2"           input consumer="kernel"
    line 3:      "PA3"           input consumer="kernel"
    line 4:      "PA4"           input consumer="kernel"
    line 5:      "PA5"           input consumer="kernel"
    line 6:      "PA6"           input consumer="kernel"
    line 7:      "PA7"           input consumer="kernel"
    line 8:      "PA8"           input consumer="kernel"
    line 9:      "PA9"           input consumer="kernel"
    line 10:     "PA10"          input consumer="kernel"
    line 11:     "PA11"          input consumer="kernel"
    line 12:     "PA12"          input
    line 13:     "PA13"          input consumer="kernel"
    line 14:     "PA14"          input consumer="kernel"
    line 15:     "PA15"          input consumer="kernel"
gpiochip1 - 16 lines:
    line 0:      "PB0"           input
    line 1:      "PB1"           output
    line 2:      "PB2"           output
    line 3:      "PB3"           input

```

以下省略

图 8.7-6 gpioinfo 信息

gpiochip0 包含了 16 条线路，每条线路的状态和用途都有所不同。格式说明：

line <编号>: "<引脚名称>" <方向> consumer="<使用者>"

这里是对部分线路说明：

line0: 引脚 PA0，方向为输入，被内核占用。

line1: 引脚 PA1，方向为输入，未被占用。

出厂系统 gpioinfo 指定线路用法的参考示例如下：

指定 gpio 控制器：

gpioinfo -c gpiochip0

```

root@ATK-DLMP257:/usr/bin# gpioinfo -c gpiochip0
gpiochip0 - 16 lines:
    line 0:      "PA0"           input consumer="kernel"
    line 1:      "PA1"           input
    line 2:      "PA2"           input consumer="kernel"
    line 3:      "PA3"           input consumer="kernel"
    line 4:      "PA4"           input consumer="kernel"
    line 5:      "PA5"           input consumer="kernel"
    line 6:      "PA6"           input consumer="kernel"
    line 7:      "PA7"           input consumer="kernel"
    line 8:      "PA8"           input consumer="kernel"
    line 9:      "PA9"           input consumer="kernel"
    line 10:     "PA10"          input consumer="kernel"
    line 11:     "PA11"          input consumer="kernel"
    line 12:     "PA12"          input
    line 13:     "PA13"          input consumer="kernel"
    line 14:     "PA14"          input consumer="kernel"
    line 15:     "PA15"          input consumer="kernel"
root@ATK-DLMP257:/usr/bin#

```

图 8.7-7 gpioinfo 指定具体 gpio 控制器

指定具体 gpio 线路：

gpioinfo -c gpiochip9 -s 3

gpioinfo -c gpiochip9 -s 3 4 5 6 7 8

```
root@ATK-DLMP257:/usr/bin# gpioinfo -c gpiochip9 -s 3
gpiochip9 3      "PZ3"          output active-low consumer="spi0 CS0"
root@ATK-DLMP257:/usr/bin# gpioinfo -c gpiochip9 -s 3 4 5 6 7 8
gpiochip9 3      "PZ3"          output active-low consumer="spi0 CS0"
gpiochip9 4      "PZ4"          input consumer="kernel"
gpiochip9 5      "PZ5"          output consumer="vbus-5v"
gpiochip9 6      "PZ6"          input consumer="int-n"
gpiochip9 7      "PZ7"          input
gpiochip9 8      "PZ8"          output consumer="DHT11 GPIO"
root@ATK-DLMP257:/usr/bin#
```

图 8.7-8 gpioinfo 指定具体 gpio

8.7.3 gpioget

gpioget 用于读取指定 GPIO 线的状态，例入在需要检测按钮状态或其他输入设备的情况下可以使用此指令。

出厂系统 gpioget 的使用帮助信息如下：

```
root@ATK-DLMP257:/usr/bin# gpioget -h
Usage: gpioget [OPTIONS] <line>...

Read values of GPIO lines.

Lines are specified by name, or optionally by offset if the chip option
is provided.

Options:
  -a, --as-is          leave the line direction unchanged, not forced to input
  -b, --bias <bias>    specify the line bias
                       Possible values: 'pull-down', 'pull-up', 'disabled'.
                       (default is to leave bias unchanged)
  --by-name            treat lines as names even if they would parse as an offset
  -c, --chip <chip>    restrict scope to a particular chip
  -C, --consumer <name> consumer name applied to requested lines (default is 'gpioget')
  -h, --help           display this help and exit
  -l, --active-low     treat the line as active low
  -p, --hold-period <period>
                       wait between requesting the lines and reading the values
  --numeric            display line values as '0' (inactive) or '1' (active)
  -s, --strict         abort if requested line names are not unique
  --unquoted          don't quote line names
  -v, --version        output version information and exit

Chips:
  A GPIO chip may be identified by number, name, or path.
  e.g. '0', 'gpiochip0', and '/dev/gpiochip0' all refer to the same chip.

Periods:
  Periods are taken as milliseconds unless units are specified. e.g. 10us.
  Supported units are 's', 'ms', and 'us'.
root@ATK-DLMP257:/usr/bin#
```

图 8.7-9 gpioget -h

主要选项和用法：

- a, --as-is: 不改变线路的方向设置，不强制设置为输入。
- b, --bias <bias>: 指定线路的偏置。可能的值有：pull-down（下拉）、pull-up（上拉）、disabled（禁用）。默认不改变偏置设置。
- by-name: 即使参数可以解析为偏移量，也将线路名当作名称处理。
- c, --chip <chip>: 限制操作范围到特定的控制器。
- C, --consumer <name>: 应用到请求的线路上的使用者名称（默认为 gpioget）。
- h, --help: 显示帮助信息并退出。
- l, --active-low: 将线路视为低电平有效。
- p, --hold-period <period>: 在请求线路和读取值之间等待一定时间。
- numeric: 以 '0'（非活动）或 '1'（活动）显示线路值。
- s, --strict: 如果请求的线路名称不唯一，则中止操作。
- unquoted: 不为线路名称添加引号。
- v, --version: 输出版本信息并退出。

控制器识别：

可以通过编号、名称或路径来识别 GPIO 控制器，例如 0、gpiochip0 和 /dev/gpiochip0 都指向同一个控制器。

时间周期：

时间周期默认为毫秒，除非指定了单位。支持的单位有 s（秒）、ms（毫秒）和 us（微秒）。

8.7.4 gpiotest

gpiotest 命令是用于设置 GPIO（通用输入输出）线的值的工具。该命令允许用户通过命令行指定 GPIO 线的高低电平状态，非常适合在 Linux 系统中对硬件进行编程和控制。

出厂系统 **gpiotest** 的使用帮助信息如下：

```
root@ATK-DLMP257:/usr/bin# gpiotest -h
Usage: gpiotest [OPTIONS] <line=value>...

Set values of GPIO lines.

Lines are specified by name, or optionally by offset if the chip option
is provided.
Values may be '1' or '0', or equivalently 'active'/'inactive' or 'on'/'off'.

The line output state is maintained until the process exits, but after that
is not guaranteed.

Options:
  --banner           display a banner on successful startup
  -b, --bias <bias> specify the line bias
                    Possible values: 'pull-down', 'pull-up', 'disabled'.
                    (default is to leave bias unchanged)
  --by-name          treat lines as names even if they would parse as an offset
  -c, --chip <chip>  restrict scope to a particular chip
  -C, --consumer <name> consumer name applied to requested lines (default is 'gpiotest')
  -d, --drive <drive> specify the line drive mode
                    Possible values: 'push-pull', 'open-drain', 'open-source'.
                    (default is 'push-pull')
  -h, --help         display this help and exit
  -l, --active-low   treat the line as active low
  -p, --hold-period <period> the minimum time period to hold lines at the requested values
  -s, --strict       abort if requested line names are not unique
  -t, --toggle <period>[,period]... toggle the line(s) after the specified period(s)
                    If the last period is non-zero then the sequence repeats.
  --unquoted         don't quote line names
  -v, --version      output version information and exit
  -z, --daemonize    set values then detach from the controlling terminal

Chips:
  A GPIO chip may be identified by number, name, or path.
  e.g. '0', 'gpiochip0', and '/dev/gpiochip0' all refer to the same chip.

Periods:
  Periods are taken as milliseconds unless units are specified. e.g. 10us.
  Supported units are 's', 'ms', and 'us'.

*Note*
  The state of a GPIO line controlled over the character device reverts to default
  when the last process referencing the file descriptor representing the device file exits.
  This means that it's wrong to run gpiotest, have it exit and expect the line to continue
  being driven high or low. It may happen if given pin is floating but it must be interpreted
  as undefined behavior.
root@ATK-DLMP257:/usr/bin#
```

图 8.7-10 gpiotest -h

主要用法:

gpiotest [OPTIONS] <line=value>...

<line=value>: 指定要设置的 GPIO 线和其值。值可以是 '1' 或 '0', 或等效的 'active'/'inactive' 或 'on'/'off'。

选项说明:

--banner: 成功启动后显示标语。

-b, --bias <bias>: 指定线路偏置，可选值为 'pull-down'（下拉）、'pull-up'（上拉）或 'disabled'（禁用）。默认不改变偏置设置。

--by-name: 即使线路名称可以解析为偏移量，也按名称处理线路。

-c, --chip <chip>: 限定特定控制器的范围。

-C, --consumer <name>: 申请线路时使用的使用者名称，默认为 'gpiotest'。



-d, --drive <drive>: 指定线路驱动模式, 可选值为 'push-pull' (推挽)、'open-drain' (开漏)、'open-source' (开源)。默认为 'push-pull'。

-h, --help: 显示帮助信息并退出。

-i, --interactive: 设置线路后等待额外的设置命令。

-l, --active-low: 将线路视为低电平有效。

-p, --hold-period <period>: 持续保持请求值的最小时间段。

-s, --strict: 如果请求的线路名称不唯一, 则中止。

-t, --toggle <period>[,period]...: 在指定的周期后切换线路。如果最后一个周期非零, 则序列重复。

--unquoted: 不对线路名称加引号。

-v, --version: 输出版本信息并退出。

-z, --daemonize: 设置值后从控制终端分离。

控制器:

GPIO 控制器可以通过编号、名称或路径来识别, 例如 '0'、'gpiochip0' 和 '/dev/gpiochip0' 都指的是同一个控制器。

时间周期:

时间周期默认以毫秒为单位, 除非指定了单位。支持的单位有 's' (秒)、'ms' (毫秒) 和 'us' (微秒)。

注意事项:

通过字符设备控制的 GPIO 线的状态在引用表示设备文件的文件描述符的最后一个进程退出时恢复到默认设置。这意味着如果运行 `gpioset` 后让它退出, 并期望线路继续保持高电平或低电平是不正确的。如果给定引脚是悬空的, 这种情况可能会发生, 但必须将其解释为未定义行为。

8.7.5 gpiomon

gpiomon 是一个用于监控和记录 GPIO（通用输入输出）线路事件的命令行工具，特别适用于需要实时跟踪 GPIO 状态变化的场景。该工具在检测到指定 GPIO 线路上的电平变化（如上升沿或下降沿）时，会输出相关事件信息。

出厂系统 **gpiomon** 的使用帮助信息如下：

```
root@ATK-DLMP257:/usr/bin# gpiomon -h
Usage: gpiomon [OPTIONS] <line>...

Wait for events on GPIO lines and print them to standard output.

Lines are specified by name, or optionally by offset if the chip option
is provided.

Options:
  --banner           display a banner on successful startup
  -b, --bias <bias> specify the line bias
                    Possible values: 'pull-down', 'pull-up', 'disabled'.
                    (default is to leave bias unchanged)
  --by-name          treat lines as names even if they would parse as an offset
  -c, --chip <chip> restrict scope to a particular chip
  -C, --consumer <name> consumer name applied to requested lines (default is 'gpiomon')
  -e, --edges <edges> specify the edges to monitor
                    Possible values: 'falling', 'rising', 'both'.
                    (default is 'both')
  -E, --event-clock <clock> specify the source clock for event timestamps
                    Possible values: 'monotonic', 'realtime', 'hrt'.
                    (default is 'monotonic')
                    By default 'realtime' is formatted as UTC, others as raw u64.
  -h, --help         display this help and exit
  -F, --format <fmt> specify a custom output format
  --idle-timeout <period> exit gracefully if no events occur for the period specified
  -l, --active-low   treat the line as active low, flipping the sense of
                    rising and falling edges
  --localtime        format event timestamps as local time
  -n, --num-events <num> exit after processing num events
  -p, --debounce-period <period> debounce the line(s) with the specified period
  -q, --quiet        don't generate any output
  -s, --strict       abort if requested line names are not unique
  --unquoted         don't quote line or consumer names
  --utc              format event timestamps as UTC (default for 'realtime')
  -v, --version      output version information and exit

Chips:
  A GPIO chip may be identified by number, name, or path.
  e.g. '0', 'gpiochip0', and '/dev/gpiochip0' all refer to the same chip.

Periods:
  Periods are taken as milliseconds unless units are specified. e.g. 10us.
  Supported units are 's', 'ms', and 'us'.

Format specifiers:
  %O  GPIO line offset
  %l  GPIO line name
  %c  GPIO chip name
  %e  numeric edge event type ('1' - rising or '2' - falling)
  %E  edge event type ('rising' or 'falling')
  %S  event timestamp as seconds
  %U  event timestamp as UTC
  %L  event timestamp as local time
```

图 8.7-11 gpiomon -h

主要用法:

gpiomon [OPTIONS] <line>...

<line>: 指定要监控的 GPIO 线路，可以通过名称或在指定控制器时使用偏移量。

选项说明:

--banner: 成功启动后显示标语。

-b, --bias <bias>: 指定线路的电气偏置。可能的值有 'pull-down'（下拉）、'pull-up'（上拉）、'disabled'（禁用）。默认保持偏置不变。

--by-name: 即使线路名称可以解析为偏移量，也按名称处理。



- c, --chip <chip>: 限定特定控制器的范围。
- C, --consumer <name>: 申请线路时使用的使用者名称, 默认为 'gpiomon'。
- e, --edges <edges>: 指定要监控的边缘类型, 可能的值有 'falling' (下降沿)、'rising' (上升沿)、'both' (两者)。默认为 'both'。
- E, --event-clock <clock>: 指定事件时间戳的源时钟。可能的值有 'monotonic'、'realtime'、'hrt'。默认为 'monotonic'。
- h, --help: 显示帮助信息并退出。
- F, --format <fmt>: 指定自定义输出格式。
- l, --active-low: 将线路视为低电平有效, 这会改变上升和下降边缘的意义。
- localtime: 将事件时间戳格式化为当地时间。
- n, --num-events <num>: 处理指定数量的事件后退出。
- p, --debounce-period <period>: 用指定的周期对线路进行去抖处理。
- q, --quiet: 不生成任何输出。
- s, --strict: 如果请求的线路名称不唯一, 则中止。
- unquoted: 不对线路或使用者名称加引号。
- utc: 将事件时间戳格式化为 UTC (默认为 'realtime')。
- v, --version: 输出版本信息并退出。

时间周期:

时间周期默认以毫秒为单位, 除非指定了单位。支持的单位有 's' (秒)、'ms' (毫秒)、'us' (微秒)。

格式说明符:

- %o: GPIO 线路偏移。
- %l: GPIO 线路名称。
- %c: GPIO 控制器名称。
- %e: 数字边缘事件类型 ('1' - 上升或 '2' - 下降)。
- %E: 边缘事件类型 ('rising' - 上升或 'falling' - 下降)。
- %S: 事件时间戳, 以秒为单位。
- %U: 事件时间戳, 以 UTC 格式。
- %L: 事件时间戳, 以当地时间格式。

这些选项使得 gpiomon 在需要跟踪 GPIO 状态变化的嵌入式系统开发和调试中非常有用。

8.7.6 gpionotify

gpionotify 是一个用于监测 GPIO（通用输入输出）线路信息变化的工具，并将相关事件打印到标准输出。这个命令非常适用于需要跟踪 GPIO 线路配置或状态改变的应用场景，如监控线路何时被请求或释放。

出厂系统 gpionotify 的使用帮助信息如下：

```
root@ATK-DLMP257:/usr/bin# gpionotify -h
Usage: gpionotify [OPTIONS] <line>...

Wait for changes to info on GPIO lines and print them to standard output.

Lines are specified by name, or optionally by offset if the chip option
is provided.

Options:
  --banner           display a banner on successful startup
  --by-name          treat lines as names even if they would parse as an offset
  -c, --chip <chip> restrict scope to a particular chip
  -e, --event <event> specify the events to monitor
                     Possible values: 'requested', 'released', 'reconfigured'.
                     (default is all events)
  -h, --help         display this help and exit
  -F, --format <fmt> specify a custom output format
  --idle-timeout <period> exit gracefully if no events occur for the period specified
  --localtime        convert event timestamps to local time
  -n, --num-events <num> exit after processing num events
  -q, --quiet        don't generate any output
  -s, --strict       abort if requested line names are not unique
  --unquoted         don't quote line or consumer names
  --utc              convert event timestamps to UTC
  -v, --version      output version information and exit

Chips:
  A GPIO chip may be identified by number, name, or path.
  e.g. '0', 'gpiochip0', and '/dev/gpiochip0' all refer to the same chip.

Periods:
  Periods are taken as milliseconds unless units are specified. e.g. 10us.
  Supported units are 's', 'ms', and 'us'.

Format specifiers:
  %o  GPIO line offset
  %l  GPIO line name
  %c  GPIO chip name
  %e  numeric info event type ('1' - requested, '2' - released or '3' - reconfigured)
  %E  info event type ('requested', 'released' or 'reconfigured')
  %a  line attributes
  %C  consumer
  %S  event timestamp as seconds
  %U  event timestamp as UTC
  %L  event timestamp as local time
root@ATK-DLMP257:/usr/bin#
```

图 8.7-12 gpionotify -h

主要用法:

gpionotify [OPTIONS] <line>...

<line>: 指定要监控的 GPIO 线路，可以通过名称或在指定控制器时使用偏移量。

选项说明:

- banner: 成功启动后显示标语。
- by-name: 即使线路名称可以解析为偏移量，也按名称处理。
- c, --chip <chip>: 限定特定控制器的范围。
- e, --event <event>: 指定要监控的事件类型。可能的值有 'requested'（请求）、'released'（释放）、'reconfigured'（重新配置）。默认监控所有事件。
- h, --help: 显示帮助信息并退出。
- F, --format <fmt>: 指定自定义输出格式。
- localtime: 将事件时间戳转换为当地时间。
- n, --num-events <num>: 处理指定数量的事件后退出。

- q, --quiet: 不生成任何输出。
- s, --strict: 如果请求的线路名称不唯一, 则中止。
- unquoted: 不对线路或使用者名称加引号。
- utc: 将事件时间戳转换为 UTC。
- v, --version: 输出版本信息并退出。

格式说明符:

- %o: GPIO 线路偏移。
- %l: GPIO 线路名称。
- %c: GPIO 控制器名称。
- %e: 数字信息事件类型 ('1' - 请求, '2' - 释放, '3' - 重新配置)。
- %E: 信息事件类型 ('requested', 'released', 'reconfigured')。
- %a: 线路属性。
- %C: 使用者名称。
- %S: 事件时间戳, 以秒为单位。
- %U: 事件时间戳, 以 UTC 格式。
- %L: 事件时间戳, 以当地时间格式。

这些功能使得 `gpionotify` 成为一个强大的工具, 适合于那些需要实时跟踪和记录 GPIO 线路状态改变的开发者和系统管理员。

8.8 拓展：新增 GPIO 接口配置实验

8.8.1 确认目标 GPIO 信息

完成 7.8 小节后，可以将 PB10 释放出来当做 GPIO 使用，其他的管脚请参考【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\04、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统管脚复用修改手册。

8.8.2 内核 GPIO 子系统使能

出厂内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 默认已使能 GPIO 系统，不需要修改，具体参考 8.5 小节。

8.8.3 源码编译及固件更新

按照 7.8 小节移除掉 PB10 的 PWM 功能，编译内核源码，将设备树更新到开发板的/boot 目录（1GB 版本设备树 stm32mp257d-atk-ddr-1GB.dts，2GB 版本设备树 stm32mp257d-atk-ddr-2GB.dts）。

8.8.4 GPIO 测试

笔者前面将 PB10 管脚配置为 GPIOB_IO10 功能，系统上对应的是 gpiochip1 控制器的 line10

```
gpioinfo -c gpiochip1 -s 10
```

```
root@ATK-DLMP257:~# gpioinfo -c gpiochip1 -s 10
gpiochip1 10      "PB10"
root@ATK-DLMP257:~#
```

图 8.8-1 gpioinfo 信息

GPIO 输出模式测试：

用下面指令将 PB10 设置为高电平，用万用表测量为高电平，说明 GPIO 测试成功。

```
gpioset -c gpiochip1 10=1
```

用下面指令将 PB10 设置为低电平，用万用表测量为低电平，说明 GPIO 测试成功。

```
gpioset -c gpiochip1 10=0
```

用万用表测试 PB10 接口，可以看到测试效果。

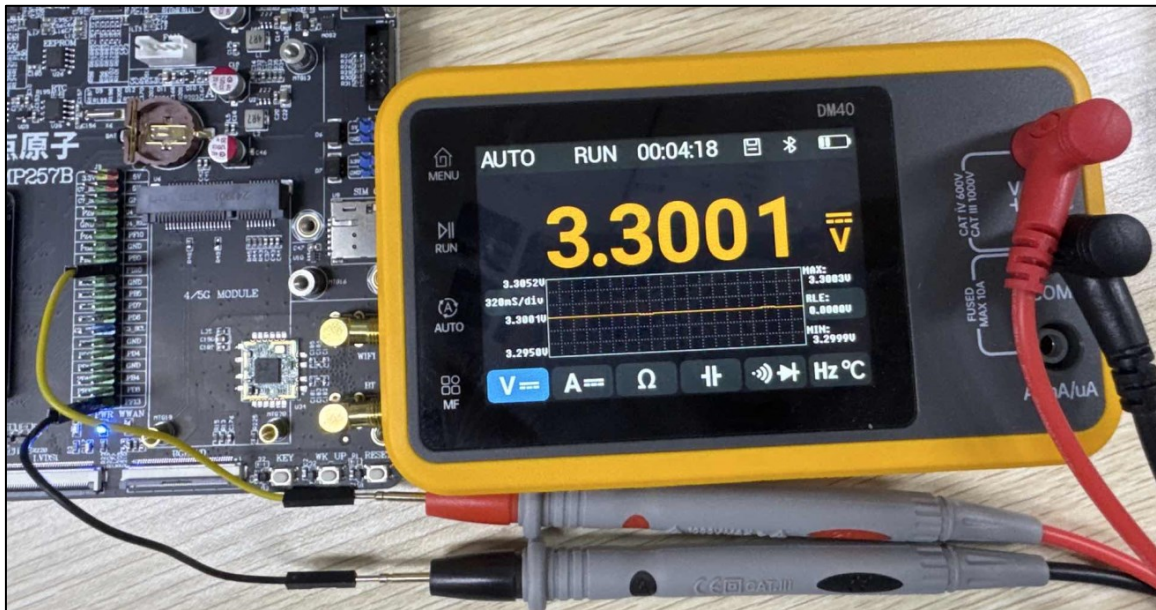


图 8.8-2 GPIO 高电平输出测试

此时再用 `gpioinfo` 指令可以查看 PB10 的状态。

```
gpioinfo -c gpiochip1 -s 10
```

```
root@ATK-DLMP257:~# gpioinfo -c gpiochip1 -s 10
gpiochip1 10      "PB10"          output
root@ATK-DLMP257:~#
```

图 8.8-3 查看 GPIO2_GPIO29 状态

可以看到 PB10 的状态已经从默认 input 变成了 output。

GPIO 输入模式测试:

用下面指令将 PB10 设置为输入模式，并读取 PB10 状态。

```
gpioget -c gpiochip1 -s 10
```

```
gpioinfo -c gpiochip1 -s 10
```

```
root@ATK-DLMP257:~# gpioget -c gpiochip1 -s 10
"10"=inactive
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# gpioinfo -c gpiochip1 -s 10
gpiochip1 10      "PB10"          input
root@ATK-DLMP257:~#
```

图 8.8-4 设置 GPIO 输入模式并读取当前状态

命令的输出 `"10"=inactive` 显示了引脚 10 当前的状态是 inactive。在 GPIO 术语中, inactive 状态通常意味着引脚处于低电平 (0V)，而 active 状态则代表引脚处于高电平。

用杜邦线将 PB10 管脚和开发板上的 3.3V 管脚相接。



图 8.8-5 将 PB10 管脚拉高

再读取输入电平，可以看到当前 GPIO 状态为 active 状态，代表引脚处于高电平。

```
root@ATK-DLMP257:~# gpioget -c gpiochip1 -s 10
"10"=active
root@ATK-DLMP257:~#
```

图 8.8-6 读取 GPIO 当前状态

GPIO 中断测试:

使用 gpiomon 指令可以用于监视指定 GPIO 引脚的电平变化事件。

```
gpiomon -c gpiochip1 -s 10
```

在前面操作中笔者已经将 PB10 接在 3.3V，现在断开 PB10 和 3.3V 的连接，让 PB10 产生从高电平到低电平的电平转变状态，此时打印信息参考如下：

```
root@ATK-DLMP257:~# gpiomon -c gpiochip1 -s 10
1112.460155571 falling gpiochip1 10 "PB10"
1112.462012771 rising gpiochip1 10 "PB10"
1112.499228421 falling gpiochip1 10 "PB10"
1112.502100996 rising gpiochip1 10 "PB10"
1112.518178671 falling gpiochip1 10 "PB10"
1112.523016271 rising gpiochip1 10 "PB10"
1112.537753221 falling gpiochip1 10 "PB10"
```

图 8.8-7 gpiomon 打印信息

每一行输出包括以下几个部分：

时间戳：显示事件发生的时间，单位是秒，相对于 gpiomon 命令开始运行的时间。例如 1112.460155571 表示系统开始运行后大约 1112 秒时记录的事件。

事件类型：

falling 表示电平从高到低的变化，也就是一个下降沿。

rising 表示电平从低到高的变化，也就是一个上升沿。

设备和引脚标识：gpiochip1 10 指出事件来源于 gpiochip1 的第 10 号引脚。

输出中显示引脚 10 的电平变化非常频繁，这是正常现象，因为笔者是用杜邦线来进行临时测试的，存在一定的接触不良现象。如果要提高电平稳定性可以外加去抖动电路或者软件消抖。

第九章 Input 子系统

9.1 Input 子系统简介

嵌入式 Linux 的 Input 子系统是 Linux 内核中用于处理各种输入设备的一个重要部分。它提供了统一的接口和抽象层,使得不同类型的输入设备(如键盘、鼠标、触摸屏、遥控器等)可以通过一致的方式进行管理和使用。

9.1.1 主要特点

嵌入式 Linux 的 Input 子系统具有以下主要特点,使其在处理各种输入设备时表现出色:

1、**设备抽象性**: 提供了统一的接口,使得应用程序能够与各种输入设备进行交互,而无需关注底层硬件细节。

2、**多种输入设备支持**: 支持各种输入设备,如键盘、鼠标、触摸屏等,使得嵌入式系统能够适应不同的用户交互需求。

3、**事件驱动**: 基于事件模型,当用户与输入设备进行交互时,系统会生成相应的事件,应用程序可以通过监听这些事件来响应用户操作。

4、**灵活性和可扩展性**: Input 子系统设计灵活,易于扩展,可以轻松地添加新的输入设备支持或修改现有设备的驱动。

5、**多种协议支持**: 支持多种输入设备的通信协议,如 USB、I2C 等,以满足不同硬件接口的需求。

6、**与用户空间的交互**: 通过设备节点和系统调用等方式,提供了与用户空间程序交互的接口,使得应用程序可以方便地访问输入设备。

9.1.2 组成部分

嵌入式 Linux 的 Input 子系统由以下几个主要组成部分构成:

输入设备驱动程序: 负责与硬件交互,将物理输入设备的信号转换为内核能够理解的数据格式,并将其传递给 Input 子系统。

Input 核心层: 位于内核中,负责管理输入设备驱动程序和处理输入事件。它提供了一个统一的接口,使得各种输入设备驱动程序可以注册和发送事件。

输入设备节点: 在/dev/input 目录下,是用户空间程序与输入设备之间通信的接口。每个输入设备都有一个对应的设备节点,用户空间程序可以通过读取这些设备节点来获取输入事件。

输入事件处理器: 用户空间程序可以通过系统调用或者使用特定的库来注册输入事件处理器,用于监听和处理输入事件。输入事件处理器可以是应用程序、窗口管理器、图形界面库等。

输入事件队列: 内核中维护的输入事件缓冲区,用于存储从输入设备驱动程序发送过来的输入事件。用户空间程序可以通过读取输入设备节点来从输入事件队列中获取输入事件。

这些组成部分共同构成了嵌入式 Linux 的 Input 子系统,实现了对各种输入设备的统一管理和处理。

9.1.3 应用场景

嵌入式 Linux 的 Input 子系统在各种嵌入式系统中具有广泛的应用场景,包括但不限于:

1、工业控制和自动化：嵌入式 Linux 系统可用于控制和监控工厂设备、机器人和自动化系统。Input 子系统可用于接收和处理来自各种传感器和输入设备的数据，如按钮、触摸屏、键盘、鼠标等，实现设备的控制和操作。

2、嵌入式计算设备：在嵌入式计算设备中，如智能家居控制器、智能电视、智能音箱等，Input 子系统可用于处理用户输入，如语音命令、手势识别、遥控器输入等，以实现设备的交互功能。

3、医疗设备：在医疗设备中，如医疗仪器、患者监护设备等，Input 子系统可用于接收和处理医疗传感器的数据，如心率监测器、血压计、体温计等，以及医护人员的输入，如键盘、触摸屏等。

4、交通运输：在交通运输领域，嵌入式 Linux 系统可用于控制和监控交通信号灯、车辆导航系统、自动售票机等设备。Input 子系统可用于接收来自交通传感器和用户的输入，如车辆传感器、触摸屏、按钮等，以实现交通设备的控制和交互功能。

5、嵌入式娱乐设备：在嵌入式娱乐设备中，如游戏机、音频播放器、视频播放器等，Input 子系统可用于处理用户输入，如游戏手柄、遥控器、触摸屏等，以实现设备的交互和控制功能。

总之，嵌入式 Linux 的 Input 子系统在各种嵌入式系统中都发挥着重要作用，为设备的交互和控制提供了基础支持。

9.2 开发板 Input 设备

输入设备类型有很多种，ATK-DLMP257 开发板上有两种典型的 Input 设备，分别是按键和触摸（触摸需要搭配正点原子屏幕），开发板上还预留了 USB 接口，用户可以外接 USB 鼠标或者键盘作为 Input 设备。以下以 KEY 按键为例讲解。

9.2.1 原理图分析

打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257_SCH.pdf，搜索 KEY，可以看到 PH5 管脚默认用做 KEY 功能。

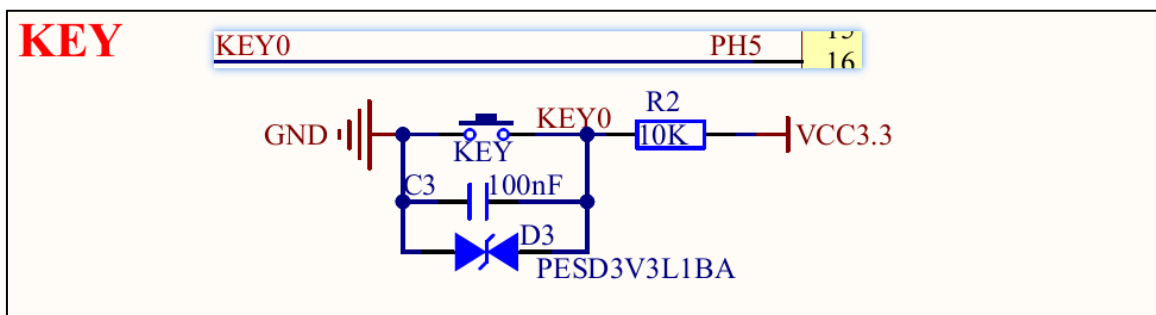


图 9.2-1 KEY 原理图

原理图解析：

按键（KEY）：这是用户实际按压的按钮。按下时，电路中的 KEY 节点会连接到地（GND）。

电阻（R2, 10K Ω ）：这是一个上拉电阻，将 KEY 节点在按键未被按下时上拉至 VCC_3V3（3.3V）。当按键被按下时，KEY 节点被拉低至地电位。

电容（C3, 100nF）：这个电容连接在 KEY 节点和地之间，用于去抖动。当按键被按下或松开时，电容可以滤除噪声信号，确保 KEY 节点的电平变化是干净的。

二极管（D3, PESD3V3LBA）：这是一个 ESD 保护二极管，用于保护电路免受静电放电（ESD）的损害。它连接在 KEY 节点和地之间，当静电放电时，二极管将静电泄放到地。

VCC_3V3：这是电源电压，为整个电路提供 3.3V 的电压。

GND: 这是电路的地连接点。

PH5: 这表示 KEY 节点连接到核心板的管脚, 管脚名为 PH5。这个引脚将读取按键的状态 (高电平表示未按下, 低电平表示按下)。

整个电路的工作原理是, 当按键未被按下时, KEY 节点通过上拉电阻被拉到 3.3V。当按键被按下时, KEY 节点被直接连接到地, 电压降到 0V。MP257 通过读取 KEY 节点的电压状态, 来判断按键是否被按下。电容用于去除按键抖动, ESD 保护二极管用于保护电路免受静电放电的损害。

9.2.2 实物硬件

开发板上 KEY 按键位置如下图所示:

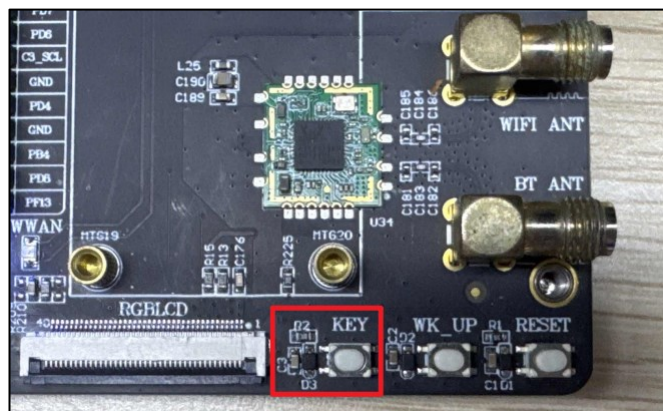


图 9.2-2 开发板 KEY 按键

9.3 内核 Input 子系统使能

在 ATK-DLMP257 内核源码配置文件 `arch/arm64/configs/stm32mp257_atk_defconfig` 中, Input 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_INPUT=y //启用内核的输入子系统
2 CONFIG_INPUT_MATRIXKMAP=y //启用矩阵键盘的键映射支持
3 CONFIG_INPUT_EVDEV=y /*启用 evdev 事件设备接口支持。
                        这是用户空间程序访问输入设备的一种通用接口,
                        支持各种输入设备(如键盘、鼠标、触摸屏等)的事件处理*/
4 CONFIG_INPUT_KEYBOARD=y //启用键盘设备支持
5 CONFIG_INPUT_MOUSE=y //启用鼠标设备支持
6 CONFIG_INPUT_TOUCHSCREEN=y //启用触摸屏设备支持
7 CONFIG_INPUT_MISC=y //启用其他各种杂项输入设备支持
8 CONFIG_SND_JACK_INPUT_DEV=y //启用声音插孔(如耳机插孔)的输入设备支持
9 .....
```

对应的 `menuconfig` 配置如下 (篇幅有限, 截图有所省略):

```
-> Device Drivers
    -> Input device support
        -> *- Generic input layer (needed for keyboard, mouse, ...)
        -> *- Matrix keymap support library
```



```
-> <*> Event interface
-> [*] Keyboards --->
    -> <*> AT keyboard
    -> <*> GPIO Buttons
-> [*] Mice --->
-> [*] Touchscreens --->
    -> <*> Goodix I2C touchscreen
```

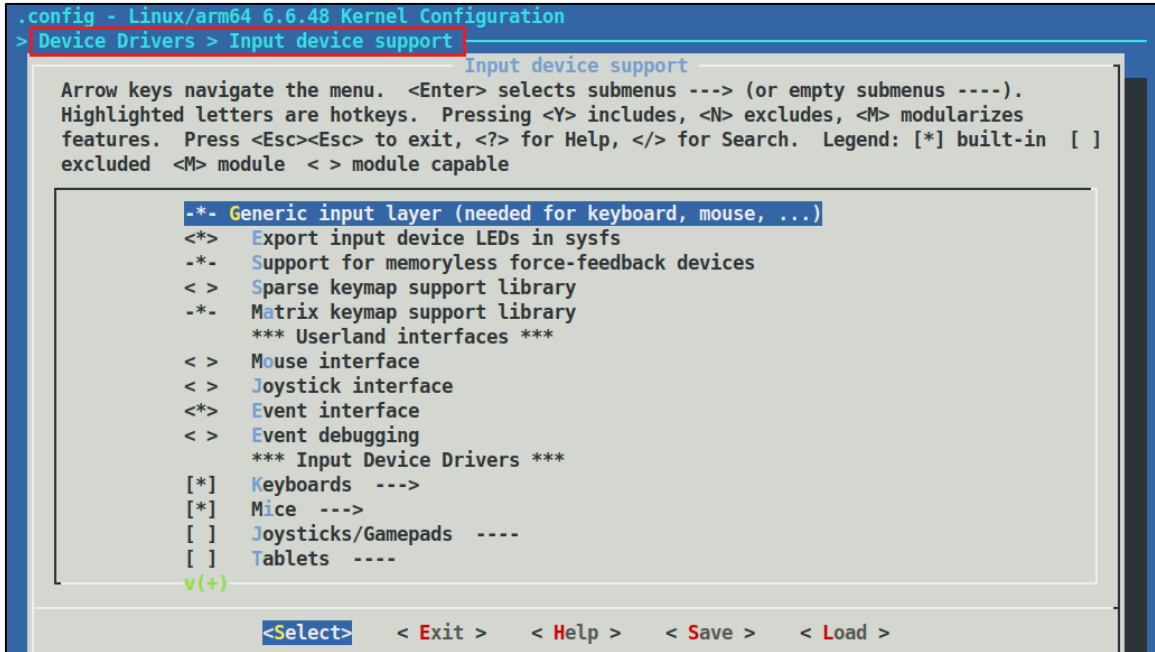


图 9.3-1 Input 配置

9.4 设备树文件分析

打开内核源码 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 文件，找到 key 子节点。

```
1 / {
2     .....
3     gpio-keys {
4         compatible = "gpio-keys";
5
6         button-user {
7             label = "User-Key";
8             linux,code = <BTN_1>;
9             gpios = <&gpioh 5 GPIO_ACTIVE_HIGH>;
10            gpio-key,wakeup;
11            status = "okay";
12        };
13    };
14    .....
15};
```

第 3 行：定义了一个名为 `gpio-keys` 的子节点，用于描述 GPIO 按键的集合。

第 4 行: `compatible` 属性用于指定设备类型, 值为 `"gpio-keys"`, 驱动文件为 `drivers/input/keyboard/gpio_keys.c`。

第 6 行: 定义了一个名为 `button-user` 的子节点, 表示某个具体按键。一般用节点名称标识按键功能, 例如这里的 `button-user` 代表 "用户按键"。

第 7 行: `label` 属性为按键指定一个易于识别的标签, 值为 `"User-Key"`, 此标签可在内核中标识此按键。

第 8 行: `linux,code` 指定按键的 Linux 输入事件代码。`<BTN_1>` 是内核定义的按键事件代码之一, 用于识别按键行为。

第 9 行: `gpios` 属性指定此按键的 GPIO 配置:

`<&gpioh>`: 表示 GPIO 控制器的别名或引用。

5: GPIO 控制器的第 5 号引脚。

`GPIO_ACTIVE_HIGH`: 表示按键的有效电平为高电平。

第 11 行: `status` 属性设置节点状态。`"okay"` 表示此按键节点启用。

9.5 按键 Input 测试方法

在【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南.pdf 的输入设备应用编程章节中, 有关于 KEY 按键的使用说明。这里就不详细展开了。

在出厂文件系统输入如下指令, 这条命令显示了系统中所有输入设备的信息。

```
cat /proc/bus/input/devices
```

开发板不接屏幕的情况下的打印如下:

```
root@ATK-DLMP257:~# cat /proc/bus/input/devices
I: Bus=0019 Vendor=0001 Product=0001 Version=0100
N: Name="gpio-keys"
P: Phys=gpio-keys/input0
S: Sysfs=/devices/platform/gpio-keys/input/input0
U: Uniq=
H: Handlers=event0
B: PROP=0
B: EV=3
B: KEY=2 0 0 0 0
```

图 9.5-1 按键事件

这里以 KEY 按键为例说明:

I: 输入设备的总线类型、供应商、产品和版本信息。这里表示总线类型为 0019 (GPIO 类型), 供应商和产品 ID 均为 0001, 版本为 0100。

N: 设备的名称, 这里为 `"gpio-keys"`。

P: 物理位置, 表示设备的路径。

S: 设备在 `sysfs` 文件系统中的路径。

U: 唯一标识符, 这里为空。

H: 处理程序, 设备的事件处理程序, 这里为 `event0`。

B PROP=0: 设备属性。

B: EV=3: 设备支持的事件类型, 这里为 3 (键盘和同步事件)。

B: KEY=2 0 0 0 0: 设备支持的键码。

笔者这里开发板此时没接 LCD 屏幕, 开发板 KEY 按键对应 `event0`。

如果没有接入 LCD 屏幕, 则测试按键的指令如下 (有接屏幕或者其他 Input 设备的话, 需按实际 event 事件来测试):

```
od -x /dev/input/event0
或者: hexdump /dev/input/event0
```

按下按键 KEY 有如下信息出现, 说明按键正常。如果没有反应, 可以多按几次按键尝试。这两条命令以十六进制格式显示 /dev/input/event0 设备的原始数据。

```
root@ATK-DLMP257:~# od -x /dev/input/event0
00000000 eeb7 65e1 0000 0000 52fb 0006 0000 0000
00000020 0001 0101 0000 0000 eeb7 65e1 0000 0000
00000040 52fb 0006 0000 0000 0000 0000 0000
00000060 eeb7 65e1 0000 0000 e1f3 0008 0000 0000
00000100 0001 0101 0001 0000 eeb7 65e1 0000 0000
00000120 e1f3 0008 0000 0000 0000 0000 0000 0000
^C
root@ATK-DLMP257:~# hexdump /dev/input/event0
00000000 eec4 65e1 0000 0000 9384 0005 0000 0000
00000010 0001 0101 0000 0000 eec4 65e1 0000 0000
00000020 9384 0005 0000 0000 0000 0000 0000
00000030 eec4 65e1 0000 0000 06c6 0008 0000 0000
00000040 0001 0101 0001 0000 eec4 65e1 0000 0000
00000050 06c6 0008 0000 0000 0000 0000 0000 0000
^C
root@ATK-DLMP257:~#
```

图 9.5-2 打印的按键事件

解释原始数据

这些十六进制数据代表输入设备事件的时间戳、事件类型、代码和值。

每个事件数据包含以下部分:

- 时间戳 (秒和微秒)
- 事件类型 (如按键事件、同步事件等)
- 事件代码 (如按键的键码)
- 事件值 (如按键按下或释放)

解析这些数据需要了解 Linux 输入子系统的事件格式。一般来说, 每个事件由 16 字节组成, 其中前 8 字节表示时间戳 (秒和微秒), 接下来的 2 字节表示事件类型, 再接下来的 2 字节表示事件代码, 最后的 4 字节表示事件值。

以 00000000 eeb7 65e1 0000 0000 52fb 0006 0000 0000 为例, 这 16 个字节可以分解为:

- 时间戳 (秒和微秒): eeb7 65e1
eeb7 表示秒数, 65e1 表示微秒数。

- 事件类型: 52fb
这个值表示事件的类型。在这个例子中, 它表示按键事件。

- 事件代码: 0006
这个值表示事件的代码, 即按下或释放的键码。在这个例子中, 它表示某个特定键的键码。

- 事件值: 0000 0000

这个值表示事件的状态或值。在按键事件中, 通常用来表示按键的状态, 比如按下或释放。这里的值可能是按键按下 (非零) 或释放 (零)。

综合起来, 这个事件的解析如下:

时间戳是 eeb7 秒和 65e1 微秒。这是一个按键事件, 对应的按键码为 0006。事件值为 0000 0000, 表示按键已释放。



第一段字符 `0000000` 是 `od -x` 命令或 `hexdump` 命令输出的行号或偏移量。它表示输出的第一行数据的偏移量，通常从 0 开始，以十六进制表示。在解析数据时，它不是实际的数据部分，而是用来标识数据的起始位置。

第十章 ADC 子系统

10.1 ADC 基础知识

1、ADC 的基本概念

ADC (Analog-to-Digital Converter, 模数转换器) 是一种将模拟信号转换为数字信号的设备。ADC 在嵌入式系统、数据采集、通信和音频处理等领域广泛应用。

2、ADC 的工作原理

ADC 通过对连续的模拟信号进行采样, 将其转换为离散的数字信号。这个过程包括三个主要步骤:

- 采样 (Sampling): 以一定频率对模拟信号进行采样, 获取离散的样本点。
- 量化 (Quantization): 将采样值映射到有限的数字值集合中。
- 编码 (Encoding): 将量化后的值转换为二进制数字输出。

3、ADC 的关键参数

分辨率 (Resolution): ADC 能够区分的最小电压变化, 通常以位 (bit) 为单位表示。例如, 10 位 ADC 可以表示 $2^{10}=1024$ 个不同的电压级别。

- 采样率 (Sampling Rate): 每秒钟对模拟信号采样的次数, 通常以样本每秒 (Samples per Second, SPS) 或赫兹 (Hz) 为单位表示。
- 输入范围 (Input Range): ADC 能够处理的模拟电压范围。
- 线性度 (Linearity): ADC 输出与实际输入之间的偏差, 通常用差分非线性 (DNL) 和积分非线性 (INL) 来描述。
- 信噪比 (SNR): 信号与噪声的比率, 表示信号的质量。

4、ADC 的类型

- 逐次逼近型 (SAR) ADC: 通过逐次逼近的方式进行模数转换, 适用于中高分辨率和速度要求的应用。
- 积分型 (Integrating) ADC: 通过积分的方法进行模数转换, 适用于高精度和低速率的应用。
- $\Delta \Sigma$ (Delta-Sigma) ADC: 通过超采样和噪声整形的方法进行模数转换, 适用于高分辨率和低带宽的应用。
- 闪存型 (Flash) ADC: 通过并行比较的方法进行模数转换, 适用于超高速和低分辨率的应用。

5、ADC 的应用

传感器接口: 将传感器输出的模拟信号转换为数字信号, 便于后续处理和分析。

音频采集: 将麦克风输出的模拟音频信号转换为数字信号, 便于存储和处理。

数据采集: 在数据采集系统中, ADC 用于捕获各种模拟信号, 如温度、压力、电压等。

6、在嵌入式系统中的 ADC 操作

在嵌入式系统中, ADC 通常集成在微控制器 (MCU) 中, 可以通过编程进行配置和操作。在嵌入式 Linux 系统中, ADC 设备通常通过 sysfs 接口进行访问。

10.2 ADC 子系统简介

在嵌入式 Linux 系统中, ADC (模数转换器) 子系统用于管理和控制模数转换器硬件, 并提供对模拟输入信号的采样和转换功能。

10.2.1 主要特点

嵌入式 Linux 的 ADC 子系统具有以下主要特点:

- 1、**灵活性:** ADC 子系统提供了灵活的配置选项, 可以根据具体应用需求调整采样率、分辨率等参数, 以满足不同场景下的数据采集要求。
- 2、**高性能:** ADC 子系统在硬件抽象层和驱动程序层面进行了优化, 以实现高效的数据采集和处理能力, 确保系统能够快速、准确地获取模拟输入信号。
- 3、**易用性:** ADC 子系统提供了统一的接口和操作方法, 使得应用程序开发者能够轻松地使用 ADC 设备进行数据采集, 无需关心底层硬件的具体实现细节。
- 4、**可移植性:** ADC 子系统的设计考虑了跨平台兼容性, 可以在不同的硬件平台上进行配置和部署, 确保应用程序的可移植性和跨平台性。

这些特点使得嵌入式 Linux 的 ADC 子系统成为嵌入式系统中重要的模拟信号采集和处理组件, 为各种嵌入式应用提供了强大的支持和功能。

10.2.2 组成部分

嵌入式 Linux 的 ADC 子系统通常由以下几个组成部分构成:

- 1、**驱动程序:** ADC 子系统的核心是驱动程序, 它负责与硬件交互并管理 ADC 设备。驱动程序通常由 SOC (系统芯片) 厂商或开发社区提供, 并针对特定的硬件进行优化。这些驱动程序通常位于 Linux 内核源代码树中的 `drivers` 目录下, 例如 `drivers/iio/adc` 目录。
- 2、**设备树配置:** 设备树描述了硬件平台的结构和组件, 包括 ADC 控制器的配置、引脚连接、中断配置等信息。通过设备树, Linux 内核可以识别和初始化 ADC 控制器, 并将其与相应的驱动程序关联起来。设备树配置通常位于嵌入式 Linux 系统的启动配置文件中, 例如 `dtb` (设备树源文件)。
- 3、**字符设备接口:** ADC 子系统通常通过字符设备接口与用户空间进行通信。在用户空间中, ADC 设备被表示为 `/dev` 下的字符设备文件, 应用程序可以通过读取这些文件来获取 ADC 采样数据。每个 ADC 设备都有一个相应的设备文件, 例如 `/dev/adc0`、`/dev/adc1` 等。
- 4、**用户空间库:** 为了方便应用程序开发者使用 ADC, 通常会提供一些用户空间库或工具, 用于简化 ADC 的配置和数据采集过程。这些库可能提供了一系列函数或命令, 用于初始化 ADC、配置采样参数、进行采样和数据处理等操作。这些库通常位于嵌入式 Linux 系统的软件包管理系统中, 例如使用 `apt` 或 `yum` 安装。

这些组成部分共同构成了嵌入式 Linux 的 ADC 子系统, 提供了对模拟输入信号的采样和转换功能, 并为应用程序提供了统一的接口和操作方法。

10.2.3 应用场景

嵌入式 Linux 的 ADC (模数转换器) 子系统在各种应用场景中被广泛使用, 这些场景通常需要对模拟信号进行采集、处理和转换。以下是一些常见的应用场景:

1、传感器数据采集:

温度传感器: 用于监测环境温度或设备温度, 常见于智能家居、工业控制和医疗设备中。

光传感器: 用于检测光强度或光线变化, 应用于自动亮度调节、环境监测等领域。

压力传感器：用于测量压力变化，广泛应用于气象监测、工业压力控制和汽车工业中。

2、电池管理系统：

电池电压监测：实时监测电池电压，确保电池在安全范围内工作，常用于电动汽车、便携式设备和不间断电源系统（UPS）中。

电池电流监测：监测充放电电流，以优化电池性能和寿命。

3、音频处理：

音频信号采集：将模拟音频信号转换为数字信号，用于音频录制、处理和播放，广泛应用于数字音频设备、录音系统和通信设备中。

4、医疗设备：

生理信号监测：采集生物电信号，如心电图（ECG）、脑电图（EEG）和肌电图（EMG），用于医疗诊断和健康监测设备中。

血糖监测：采集血糖传感器的模拟信号，用于糖尿病患者的血糖监测。

5、工业自动化：

过程控制：采集各种传感器的数据（如温度、湿度、压力、液位），用于工业过程的监测和控制。

设备监测：采集设备运行状态的模拟信号，用于预防性维护和故障诊断。

6、环境监测：

空气质量监测：采集空气中污染物的浓度（如 PM2.5、CO2），用于环境保护和空气质量

管理。

水质监测：采集水质传感器的数据（如 pH 值、电导率），用于水资源管理和污染监测。

7、智能农业：

土壤湿度监测：采集土壤湿度传感器的数据，用于精细农业中的灌溉控制。

气象监测：采集气象传感器的数据（如温度、湿度、风速），用于农业气象服务。

8、消费电子：

运动检测：采集加速度计和陀螺仪的模拟信号，用于运动追踪设备、智能手环和虚拟现实设备中。

触摸屏：采集触摸屏控制器的模拟信号，用于用户输入检测。

这些应用场景展示了嵌入式 Linux 的 ADC 子系统在不同领域中的广泛应用和重要性，通过对模拟信号的高效采集和处理，为各种智能设备和系统提供了基础支持。

10.3 MP257 的 ADC 说明

10.3.1 文档和资源

想要获取 MP257 系列处理器的 ADC 详细说明，最直接的方式是参考 ST 官方文档，包括：

1、参考手册（Reference Manual）：提供了详细的寄存器级说明，包括 ADC 模块的寄存器映射以及配置方法等。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南 (即本文档)

应用开发: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

10.3.2 特性说明

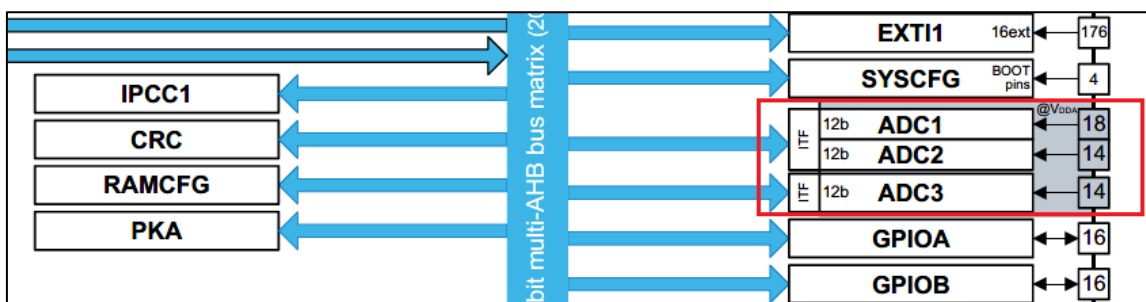


图 10.3-1 MP257 功能框图中的 ADC 部分

MP257 处理器集成了三个 ADC (Analog-to-Digital Converter) 模块, 具有多种功能和特性。根据芯片参考手册可以知道 MP257 ADC 模块的详细特性说明。

41.2 ADC main features

- High-performance features
 - Up to 3 ADCs:
 - ADC1 is connected to 18 external channels + 2 internal channels
 - ADC2 is connected to 14 external channels + 6 internal channels
 - ADC3 is connected to 14 external channels + 6 internal channels
 - 12, 10, 8 or 6-bit configurable resolution
 - ADC conversion time independent from the AHB bus clock frequency
 - Faster conversion time by lowering resolution
 - Management of single-ended or differential inputs (programmable per channels)
 - AHB slave bus interface to allow fast data handling
 - Offset calibration support
 - Channel-wise programmable sampling time
 - Flexible sampling time control
 - Up to 4 injected channels (analog inputs assignment to regular or injected channels is fully configurable)

图 10.3-2 MP257 的 ADC 说明节选

由手册可知 MP257 的 ADC 的功能特性:

- 分辨率: 所有 ADC 均为 12 位, 支持 12、10、8 或 6 位可配置分辨率。

- 最大采样速度：12 位分辨率下，最大 5 Msps。
- 通道数量：
 - ADC1：18 个外部通道 + 2 个内部通道。
 - ADC2：14 个外部通道 + 6 个内部通道。
 - ADC3：14 个外部通道 + 6 个内部通道。
- 双模操作：仅 ADC1 和 ADC2 支持，ADC3 不支持。
- 过采样：支持最高 x1024 倍过采样。
- 数据寄存器：32 位宽度。

更多具体信息请查阅芯片参考手册。

10.4 开发板 ADC 设备

10.4.1 原理图分析

打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257_SCH.pdf，搜索 ADC，可以看到 ADC 相关电路如下。

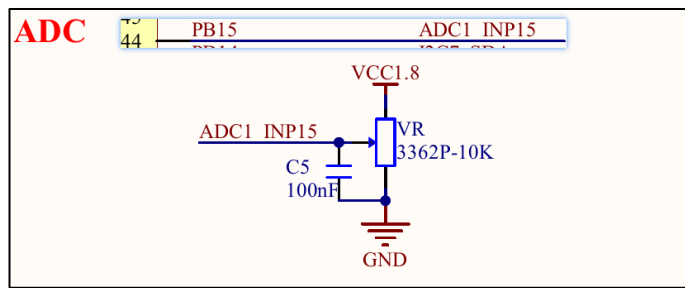


图 10.4-1 开发板原理图 ADC 部分

原理图分析如下：

ADC 输入通道：ADC1_INP15：这是连接到 ADC 的输入引脚。

电阻和电容网络：

- 可调电阻 VR（3362P-10K）：这是一个 10k Ω 的可调电位器，通过调整 VR 可以改变 ADC 输入的电压。连接如下：上端接 1.8V（VCC1.8，作为高电压参考）。下端接地（GND，作为低电压参考）。中间引脚接到 ADC1_INP15，输出可调电压。
- C5：100nF 电容，该电容起到去耦和滤波作用，用于稳定 ADC 输入信号，过滤高频噪声，确保 ADC 测量结果更准确。
- 电源 VCC1.8：提供 1.8V 的参考电压，作为可调电位器的供电电压。
- GND：电路的公共地，提供基准电位。

通过调整可调电阻 VR，可以在 0V 到 1.8V 之间输出一个可变的模拟电压，并将其输入到 ADC1 的 INP15 引脚。电容 C5 用于稳定该输入信号，防止噪声干扰，提高 ADC 转换的精度。最终，ADC1 可以对 INP15 引脚上的电压进行采样并转换成数字值。该电路常用于 ADC 校准或输入电压调节测试，通过手动调整电位器 VR，模拟不同的输入电压值，用于测试 ADC 的采样精度和范围。

10.4.2 实物硬件

开发板上 ADC 相关电路位置如下图所示：

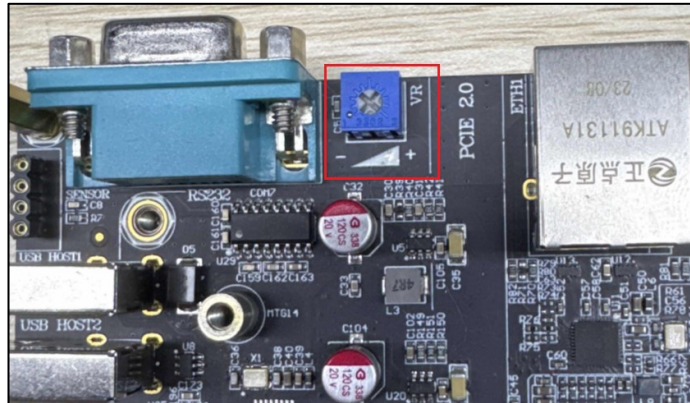


图 10.4-2 开发板 ADC 电路

10.5 内核 ADC 子系统使能

在 ATK-DLMP257 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, ADC 相关配置如下(略有删减, 只列举重点部分):

- 1 CONFIG_STM32_ADC_CORE=m //启用 ST 平台的 ADC 驱动核心程序
- 2 CONFIG_STM32_ADC=m //启用 ST 平台的 ADC 驱动程序
- 3 CONFIG_STM32_MDF_ADC=y //启用 ST 平台的多功能数字滤波器驱动

在 Linux 内核中, ADC (模数转换器) 子系统通常作为 IIO (Industrial I/O) 子系统的一部分, 这里笔者只列举 ADC 相关的配置, 更多的配置需要结合 IIO 章节的配置作为参考。

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

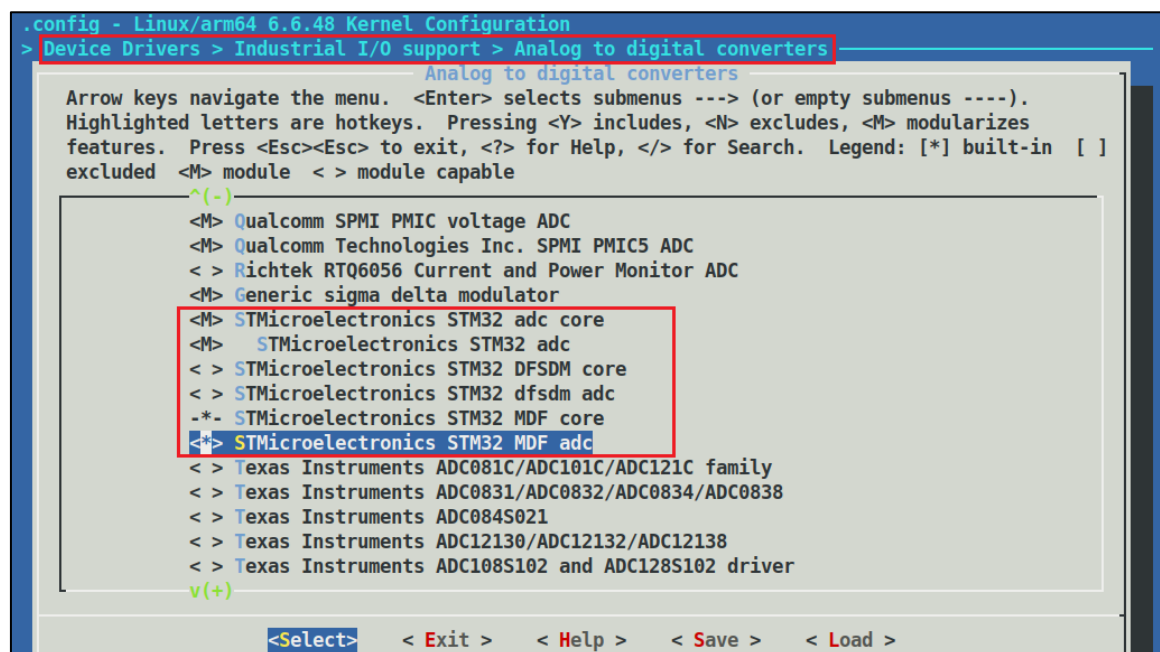


图 10.5-1 ADC 相关配置

10.6 设备树文件分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi, 此文件是特定于 MP2 系列芯片的设备树包含文件, 通常不独立使用, 而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件



共享相同的基础配置，而.dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 ADC 的描述如下：

```

1      adc_12: adc@404e0000 {
2          compatible = "st,stm32mp25-adc-core";
3          reg = <0x404e0000 0x400>;
4          interrupts = <GIC_SPI 87 IRQ_TYPE_LEVEL_HIGH>,
5                      <GIC_SPI 88 IRQ_TYPE_LEVEL_HIGH>;
6          clocks = <&rcc CK_KER_ADC12>;
7          clock-names = "adc";
8          interrupt-controller;
9          #interrupt-cells = <1>;
10         #address-cells = <1>;
11         #size-cells = <0>;
12         access-controllers = <&rifsc 58>;
13         power-domains = <&CLUSTER_PD>;
14         status = "disabled";
15
16         adc1: adc@0 {
17             compatible = "st,stm32mp25-adc";
18             reg = <0x0>;
19             interrupt-parent = <&adc_12>;
20             interrupts = <0>;
21             dmas = <&hpdma 81 0x20 0x12>;
22             dma-names = "rx";
23             #io-channel-cells = <1>;
24             #address-cells = <1>;
25             #size-cells = <0>;
26             nvmem-cells = <&vrefint>;
27             nvmem-cell-names = "vrefint";
28             st,adc-trigger-sel = <0>;
29             status = "disabled";
30             channel@14 {
31                 reg = <14>;
32                 label = "vrefint";
33             };
34         };

```

以上是 ST 的 ADC1 设备树配置，默认没有使能 ADC。compatible="st,stm32mp25-adc-core"，对应的驱动路径为 /drivers/iio/ad/stm32-adc-core.c。以 2GB 版本核心板为例，在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 中，出厂系统默认使能了 ADC，stm32mp257d-atk-ddr-2GB.dts 中关于 ADC 的配置如下：

```

1 &adc_12 {
2     pinctrl-names = "default";
3     pinctrl-0 = <&adc1_in15_pins_a>;

```



```

4  vdda-supply = <&vdda_1v8>;
5  vref-supply = <&vddref_1v8>;
6  status = "okay";
7
8  adc1: adc@0 {
9      #address-cells = <1>;
10     #size-cells = <0>;
11     status = "okay";
12
13     channel@15 {
14         reg = <15>;                                /* ADC1 in15 channel is used */
15         st,min-sample-time-ns = <10000>;          /* 10µs sampling time */
16     };
17 };
18};

```

这段设备树的代码定义了一些硬件配置，具体包括一个固定电压的调节器和 ADC 的电源供应配置：

在根节点下定义固定电压调节器：

第 1 行，`&adc_12`，引用名为 `adc_12` 的节点，表示对主 ADC 控制器节点进行修改或扩展配置。

第 2 行，`pinctrl-names="default"`，定义管脚控制器的名称为 "default"，用于指定默认的引脚复用配置。

第 3 行，`pinctrl-0 = <&adc1_in15_pins_a>`，引用名为 `adc1_in15_pins_a` 的管脚配置，用于将 ADC1 的通道 15 连接到对应引脚。

第 4 行，`vdda-supply = <&vdda_1v8>`，定义 `vdda` 供电来源为 `vdda_1v8`，表示 ADC 使用 1.8V 电源输入。

第 5 行，`vref-supply = <&vddref_1v8>`：定义参考电压 `vref` 的供电来源为 `vddref_1v8`，也是 1.8V 电压输入。

第 6 行，`status = "okay"`：将 `adc_12` 节点状态设置为 "okay"，启用该 ADC 控制器。

第 8 行，`adc1:adc@0`，定义 `adc1` 节点，并挂载在 `adc_12` 控制器下。`adc@0` 表示该 ADC 模块的基地址偏移量为 0

第 9 行，`#address-cells = <1>`，定义子节点的地址单元数量为 1。

第 10 行，`#size-cells = <0>`，定义子节点的大小单元数量为 0，即不需要大小描述。

第 11 行，`status = "okay"`，将 `adc1` 节点状态设置为 "okay"，启用 ADC1。

第 13 行，`channel@15`，定义 `channel@15` 子节点，表示使用 ADC1 的通道 15。

第 14 行，`reg = <15>`，指定该子节点的寄存器编号为 15，即 ADC 通道号为 15。

第 15 行，`st,min-sample-time-ns = <10000>`，定义最小采样时间为 10000ns (10µs)。

此设备树配置启用了 ADC 控制器 和 ADC1 模块，并配置了 ADC1 的通道 15 (ADC1_INP15)。供电电源和参考电压设置为 1.8V。指定了引脚复用配置，采样时间设置为 10 微秒。

10.7 ADC 测试方法

在【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 快速体验手册.pdf 的 ATK-DLMP257B 功能测试章节中, 有关于开发板 ADC 的使用说明。

注意: ADC 的采集电压绝对值最大是 1.8V, 请不要超过 1.8V, 否则可能对芯片造成损坏。
ATK-DLMP257B 底板上有一个可调电位器, 方便用户测试, 位置如下图所示。

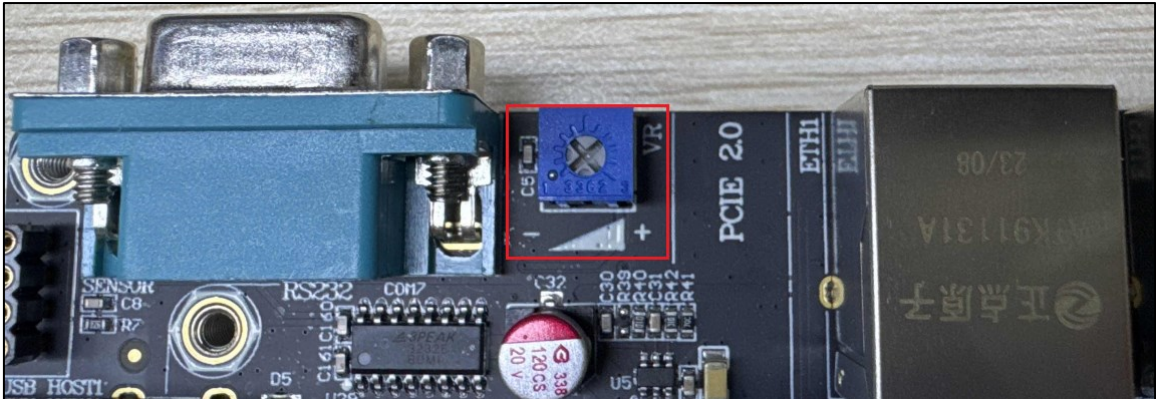


图 10.7-1 开发板 ADC 位置

使用下面的指令来获取 ADC1 采样的数据, 这里读出里出来的值是原始值 raw_value。

```
cat /sys/bus/iio/devices/iio:device0/in_voltage15_raw
```

```
root@ATK-DLMP257:~# cat /sys/bus/iio/devices/iio:device0/in_voltage15_raw
1008
root@ATK-DLMP257:~#
```

图 10.7-2 查看 raw 的值

实际电压值(mV)=in_voltage1_raw * in_voltage_scale。

其中 in_voltage_scale 为 ADC 比例文件(分辨率), 单位为 mV, 可以通过下面的指令获取, 此值跟参考电压有关。

```
cat /sys/bus/iio/devices/iio:device0/in_voltage_scale
```

```
root@ATK-DLMP257:~# cat /sys/bus/iio/devices/iio:device0/in_voltage_scale
0.439453125
root@ATK-DLMP257:~#
```

图 10.7-3 查看 scale 值

那么此时 ADC1 实际电压值为 $1008 * 0.439453125 \approx 442.9\text{mV}$ 。调节电位计 RP_ADC 可以调节 ADC 电压。

第十一章 IIO 子系统

11.1 IIO 子系统简介

工业输入输出（IIO，Industrial Input/Output）子系统是嵌入式 Linux 中用于处理工业传感器和数据采集设备的一个重要子系统。IIO 子系统提供了一个统一的接口来管理和访问各种传感器和数据采集设备，从而简化了驱动程序的编写和应用程序的开发。

11.1.1 主要特点

1、统一接口

IIO 子系统提供了一个标准化的接口，统一管理和访问各种传感器和数据采集设备。这大大简化了驱动程序的开发和应用程序的设计，使得不同设备之间的集成更加容易和高效。

2、多种数据采集模式

IIO 子系统支持多种数据采集模式，包括直接访问模式、缓冲区模式和触发模式。通过这些模式，用户可以根据具体需求选择合适的数据采集方式，确保数据采集的灵活性和高效性。

3、设备属性管理

IIO 子系统利用 sysfs 文件系统暴露设备属性，用户可以通过读取和写入 sysfs 文件来配置和查询设备信息。这种方式使得设备配置和管理变得直观和方便，有助于快速开发和调试。

4、缓冲区支持和触发机制

IIO 子系统提供强大的缓冲区支持和灵活的触发机制。缓冲区支持允许高效地在内核和用户空间之间传输大量数据，适用于高频数据采集应用。而触发机制则可以基于硬件或软件事件触发数据采集，确保数据采集的实时性和精确性。

这些特点使得 IIO 子系统在处理工业传感器和数据采集设备时非常高效和灵活，适用于各种嵌入式应用场景。

11.1.2 组成部分

嵌入式 Linux 的工业输入输出（IIO）子系统由多个组成部分构成，这些部分共同协作以提供各种传感器和数据采集设备的支持。IIO 子系统的主要组成部分包括：

1、IIO 核心

IIO 核心是 IIO 子系统的核心组件，负责管理 IIO 设备、处理设备的通用操作和维护 IIO 设备的状态。它提供了基础设施，用于注册和注销 IIO 设备，并处理通用的设备操作。

2、IIO 设备驱动程序

IIO 设备驱动程序是与具体传感器或数据采集设备相关的驱动程序。每个 IIO 设备都需要一个对应的驱动程序来实现设备的初始化、配置、数据读取和写入等操作。驱动程序通过调用 IIO 核心提供的接口，将设备注册到 IIO 子系统中。

3、IIO 缓冲区

IIO 缓冲区用于存储从设备读取的数据，以便高效地进行批量数据传输。缓冲区机制支持高频数据采集，通过缓冲区，可以避免频繁的设备访问，从而提高系统性能。IIO 缓冲区可以在内核空间和用户空间之间传输数据。

4、IIO 触发

IIO 触发机制用于基于特定事件触发数据采集。触发可以是硬件中断、定时器或软件事件。当触发条件满足时，IIO 设备将采集数据并存储到缓冲区中。这种机制保证了数据采集的实时性和精确性。

5、用户空间接口

用户空间接口是用户应用程序与 IIO 设备交互的桥梁。主要的用户空间接口包括：

sysfs 接口：通过 sysfs 文件系统，用户可以读取和写入设备属性。

chardev 接口：字符设备接口允许用户空间应用程序通过标准的文件操作（如 read、write）与 IIO 设备交互。

ioctl 接口：用于执行特定的设备控制操作。

6、IIO 工具

IIO 子系统还提供了一些用户空间工具，用于方便地与 IIO 设备交互和调试。这些工具包括：

iio_event_monitor：个用于监控 IIO 设备事件的工具。

iio_generic_buffer：用于处理和管理 IIO 设备数据缓冲区的通用工具。

7、事件接口

IIO 子系统支持通过事件接口通知用户空间应用程序设备状态的变化。例如，当新数据可用时，可以通过事件接口通知应用程序，从而实现事件驱动的数据处理。

这些组成部分共同协作，使得 IIO 子系统能够高效地管理和访问各种传感器和数据采集设备，提供灵活的数据采集和处理能力。

11.2 开发板 IIO 设备

大部分 IIO 设备是 ADC 类型设备或传感器，ATK-DLMP257B 开发板上的 ADC 设备使用的就是 IIO 子系统框架。具体参考 ADC 子系统章节。

11.3 内核 IIO 子系统使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中，IIO 相关配置如下(略有删减，只列举重点部分)：

```
1 CONFIG_IIO=y           //启用工业输入输出(IIO)子系统的支持
2 CONFIG_IIO_BUFFER=y     //启用 IIO 缓冲区支持，
                          允许从传感器中读取数据并将其存储在内核中的缓冲区中
3 CONFIG_IIO_KFIFO_BUF=y  //启用 IIO KFIFO 缓冲区支持，
                          KFIFO 是一种特定于 IIO 的高性能环形缓冲区
4 CONFIG_IIO_TRIGGERED_BUFFER=y //启用 IIO 触发缓冲区支持，
                          允许基于事件触发的传感器数据采集
5 CONFIG_IIO_TRIGGER=y    //启用 IIO 触发器支持，
                          允许传感器在特定事件触发时进行数据采集
6 CONFIG_IIO_CONSUMERS_PER_TRIGGER=2 //每个触发器最多可与 2 个传感器相关联
7 .....
```

除此之外，配置文件中还有一系列 ST 等传感器的 IIO 配置。

对应的 menuconfig 配置如下（篇幅有限，截图有所省略）：

```
-> Device
  -> <*> Industrial I/O support
    -> --- Industrial I/O support
    -> -* Enable buffer support within IIO
    -> -* Industrial I/O buffering based on kfifo
    -> -* Industrial I/O triggered buffer support
```

-> *- Enable triggered sampling support

-> (2) Maximum number of consumers per trigger

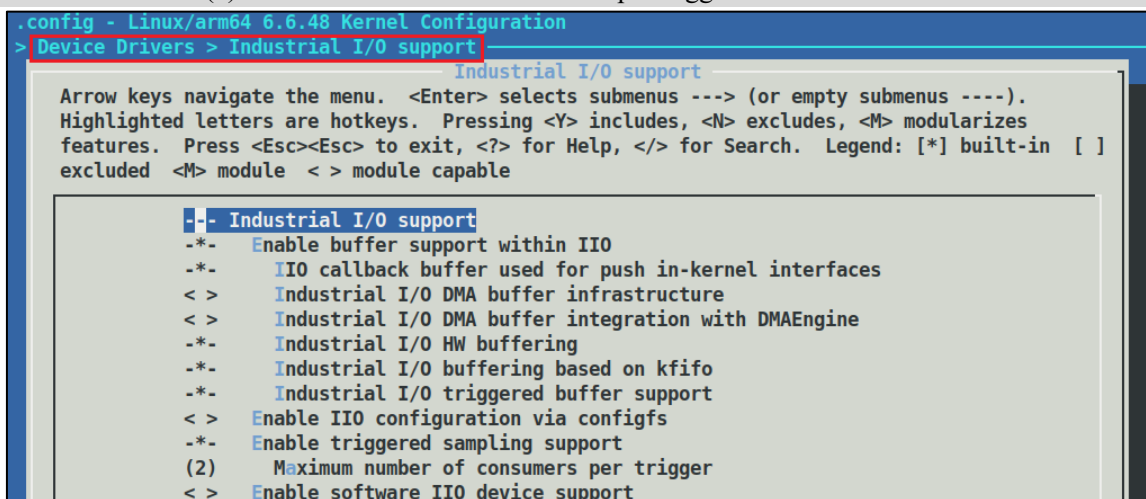


图 11.3-1 IIO 相关配置

第十二章 I2C 子系统

12.1 I2C 基础知识

I2C (Inter-Integrated Circuit) 是一种常用的串行通信协议, 用于在集成电路之间进行短距离通信。I2C 由飞利浦公司 (现为恩智浦半导体) 在 1982 年开发, 主要用于主从模式的多主机控制系统。

基本概念

● 总线结构

I2C 是一种双线制总线, 包含两条信号线: SDA (数据线) 和 SCL (时钟线)。

每个设备都通过这两条线连接, 可以通过唯一的地址进行区分。

● 主从关系

主设备: 控制通信的设备, 生成时钟信号并启动数据传输。

从设备: 被动响应主设备的请求, 不生成时钟信号。

● 信号线

SDA (Serial Data Line):

传输数据的双向线。

开漏结构, 通常需要上拉电阻。

SCL (Serial Clock Line):

传输时钟信号的单向线。

由主设备生成时钟信号, 也需要上拉电阻。

● 数据传输

数据帧格式:

起始条件: 主设备在 SCL 高电平期间将 SDA 线从高拉低。

地址帧: 包含 7 位或 10 位从设备地址和 1 位读/写指令位。

确认位 (ACK/NACK): 接收方每接收一个字节后, 会在第 9 位时钟周期上响应确认 (ACK) 或非确认 (NACK)。

数据帧: 传输的数据字节, 发送或接收。

停止条件: 主设备在 SCL 高电平期间将 SDA 线从低拉高。

● 读写操作

写操作: 主设备发送从设备地址, 并在最后一位写指令为 0。

读操作: 主设备发送从设备地址, 并在最后一位写指令为 1。

特点与优势

简单性: I2C 总线使用两条线, 实现了硬件上的简化。

灵活性: 支持多主多从配置, 设备可以动态加入和离开总线。

低成本: 由于线数少, 系统布线成本和 PCB 面积减少。

可靠性: I2C 有较强的抗干扰能力, 适用于各种嵌入式系统。

限制与注意事项

速度限制: 标准模式为 100kbps, 高速模式为 400kbps, 更高的速率如 Fast-mode Plus(1Mbps) 和 High-speed mode (3.4Mbps) 需要特殊处理。

线长度: 由于 I2C 是并联电阻器终端, 线长度有限, 通常不超过 1 米。

设备数量: 从设备的数量受限于 7 位或 10 位地址空间, 理论上最多支持 127 个设备。

常用 I2C 接口设备

EEPROM: 用于数据存储, 非易失性存储器。

传感器: 温度、湿度、压力等传感器。

实时时钟 (RTC): 提供精确的时间和日期。

LCD 显示器: 用于显示信息。

12.2 I2C 子系统简介

I2C (Inter-Integrated Circuit) 是一种广泛使用的串行通信总线标准, 特别适用于嵌入式系统中的芯片间通信。I2C 子系统在 Linux 内核中有一个全面的实现, 提供了对 I2C 设备和驱动的支持。

12.3 主要特点

1、模块化设计

I2C 子系统采用模块化设计, 将 I2C 总线适配器、设备和驱动程序分离。这样, 开发人员可以独立地开发和调试每个组件, 提高了系统的灵活性和可维护性。

2、设备树支持

设备树 (Device Tree) 提供了一种描述硬件的方法, 使内核能够自动识别和配置 I2C 设备。设备树节点用于定义 I2C 总线和从设备的属性, 使得系统在启动时可以动态配置 I2C 硬件。

3、丰富的 API 支持

I2C 子系统提供了一套丰富的 API, 用于实现 I2C 总线上的数据传输。这些 API 包括简单的读写操作和复杂的 SMBus 协议支持, 使开发人员能够轻松地与 I2C 设备通信。

4、用户空间工具

Linux 提供了一组用户空间工具, 用于检测、访问和调试 I2C 设备。这些工具简化了开发和调试过程, 使开发人员可以快速验证 I2C 总线和设备的功能。

这些特点展示了嵌入式 Linux I2C 子系统的灵活性、易用性和强大的功能支持, 有助于开发人员在各种嵌入式应用中高效地实现 I2C 通信。

12.4 组成部分

嵌入式 Linux 的 I2C 子系统由多个关键组成部分构成, 这些部分协同工作以提供对 I2C 总线和设备的支持。以下是 I2C 子系统的主要组成部分:

1. I2C 适配器 (Adapter)

I2C 适配器代表了 I2C 主控制器硬件, 例如, 处理器内置的 I2C 控制器或外部的 I2C 控制器模块。适配器驱动程序负责与这些硬件进行交互, 并提供对 I2C 总线的访问接口。

i2c_adapter 结构体是内核中表示适配器的主要数据结构。

```
1 struct i2c_adapter {
2     struct module *owner;
3     unsigned int class;
4     const struct i2c_algorithm *algo;
```



```
5 // 其他成员变量
6 };
```

2、I2C 客户端 (Client)

I2C 客户端表示连接到 I2C 总线上的从设备。每个从设备都有一个唯一的 I2C 地址，客户端驱动程序使用该地址与从设备通信。i2c_client 结构体表示一个 I2C 从设备。

```
1 struct i2c_client {
2     struct i2c_adapter *adapter;
3     struct device dev;
4     u16 addr;
5     // 其他成员变量
6 };
```

3、I2C 驱动程序 (Driver)

I2C 驱动程序负责管理和控制 I2C 客户端设备。驱动程序实现了与从设备的通信逻辑，例如数据读写、初始化和配置。i2c_driver 结构体表示一个 I2C 设备驱动程序。

```
1 struct i2c_driver {
2     int (*probe)(struct i2c_client *client,
3                  const struct i2c_device_id *id);
4     int (*remove)(struct i2c_client *client);
5     // 其他成员变量
6 };
```

4、I2C 消息 (Message)

I2C 消息用于表示 I2C 总线上的一次数据传输操作。i2c_msg 结构体用于封装一次读或写操作的信息，包括目标地址、数据缓冲区和传输长度。

```
1 struct i2c_msg {
2     __u16 addr;
3     __u16 flags;
4     __u16 len;
5     __u8 *buf;
6 };
```

5、I2C 算法 (Algorithm)

I2C 算法定义了适配器使用的操作方法集。每个 I2C 适配器驱动程序必须实现一个 I2C 算法，定义如何执行 I2C 操作，如启动、停止、读写数据等。i2c_algorithm 结构体表示这个接口。

```
1 struct i2c_algorithm {
2     int (*master_xfer)(struct i2c_adapter *adap,
3                        struct i2c_msg *msgs, int num);
4     // 其他成员变量
5 };
```

6、设备树支持 (Device Tree Support)

设备树是一种用于描述硬件的文件格式，特别适用于 ARM 等嵌入式平台。I2C 设备通常通过设备树描述，使内核能够自动配置和管理这些设备。

示例设备树片段：

```
1 i2c1: i2c@40120000 {
```



```

2         compatible = "st,stm32mp25-i2c";
3         reg = <0x40120000 0x400>;
4         interrupt-names = "event";
5         interrupts = <GIC_SPI 108 IRQ_TYPE_LEVEL_HIGH>;
6         clocks = <&rcc CK_KER_I2C1>;
7         resets = <&rcc I2C1_R>;
8         #address-cells = <1>;
9         #size-cells = <0>;
10        dmas = <&hpdma 27 0x20 0x00003012>,
11              <&hpdma 28 0x20 0x00003021>;
12        dma-names = "rx", "tx";
13        access-controllers = <&rifsc 41>;
14        power-domains = <&CLUSTER_PD>;
15        status = "disabled";
16    };
    
```

12.5 MP257 的 I2C 说明

12.5.1 文档和资源

想要获取 MP257 系列处理器的 I2C 详细说明，最直接的方式是参考 ST 官方文档，包括：

- 1、参考手册（Reference Manual）：提供了详细的寄存器级说明，包括 I2C 模块的寄存器映射以及配置方法等。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料
- 2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南（即本文档）

应用开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图：【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

12.5.2 特性说明

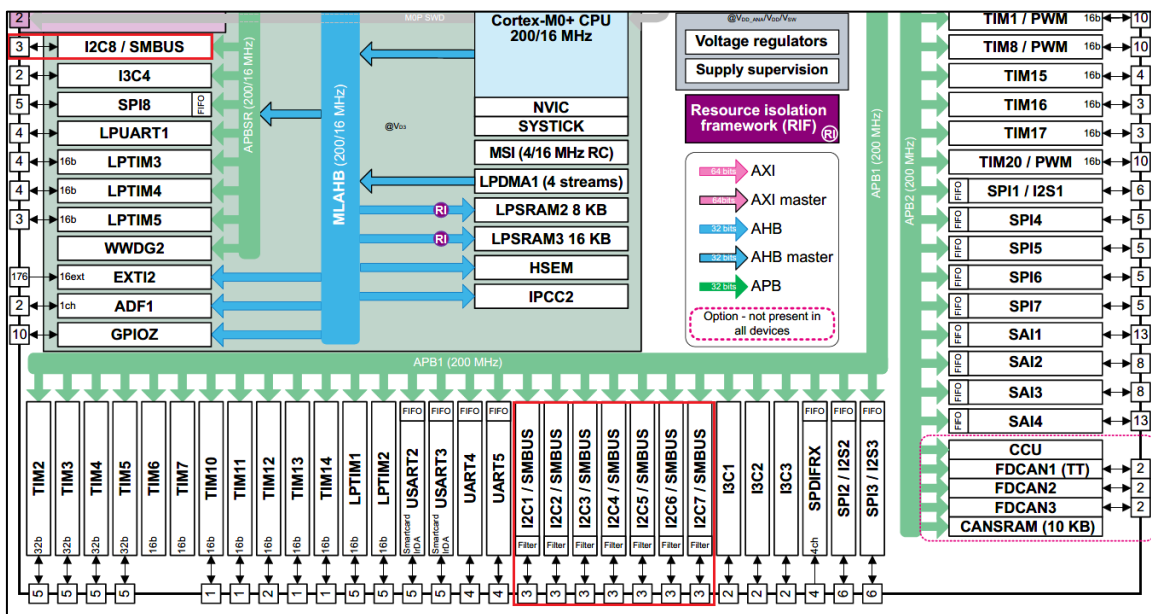


图 12.5-1 MP257 功能框图中的 I2C 部分

MP257 芯片包含 8 个 I2C 模块实例，I2C1~I2C7 属于 APB1 总线，I2C8 属于 APBSR 总线。
芯片参考手册的 I2C 章节如下：

75.3 I2C implementation

This section provides an implementation overview with respect to the I2C instantiation.

Table 733. I2C implementation

I2C features ⁽¹⁾	I2C1	I2C2	I2C3	I2C4	I2C5	I2C6	I2C7	I2C8
7-bit addressing mode	X	X	X	X	X	X	X	X
10-bit addressing mode	X	X	X	X	X	X	X	X
Standard-mode (up to 100 kbit/s)	X	X	X	X	X	X	X	X
Fast-mode (up to 400 kbit/s)	X	X	X	X	X	X	X	X
Fast-mode Plus with 20 mA output drive I/Os (up to 1 Mbit/s)	X	X	X	X	X	X	X	X
Independent clock	X	X	X	X	X	X	X	X
Autonomous in Stop 1/2 mode with wake-up capability	X	X	X	X	X	X	X	X
Autonomous in LP-Stop 1/2, LPLV-Stop 1/2, Standby1 mode with wake-up capability	-/-	-/-	-/-	-/-	-/-	-/-	-/-	X/X/X
SMBus/PMBus	X	X	X	X	X	X	X	X

1. X = supported.

图 12.5-2 芯片参考手册 I2C 章节节选

由此表可知，MP257 所有 I2C 控制器 (I2C1 至 I2C8) 均支持 7-bit 和 10-bit 地址模式、标准模式 (100 kbit/s)、快速模式 (400 kbit/s)、快速模式 Plus (1 Mbit/s) 以及独立时钟和 Stop 1/2 模式下的自主唤醒功能，支持 SMBus/PMBus。此外，仅 I2C8 支持 LP-Stop、LPLV-Stop 和 Standby1 模式下的自主唤醒。

更多的 I2C 说明请查阅芯片参考手册 I2C 章节。

12.6 开发板 I2C 设备

12.6.1 原理图分析

打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 I2C。可以看到 I2C 相关电路如下。

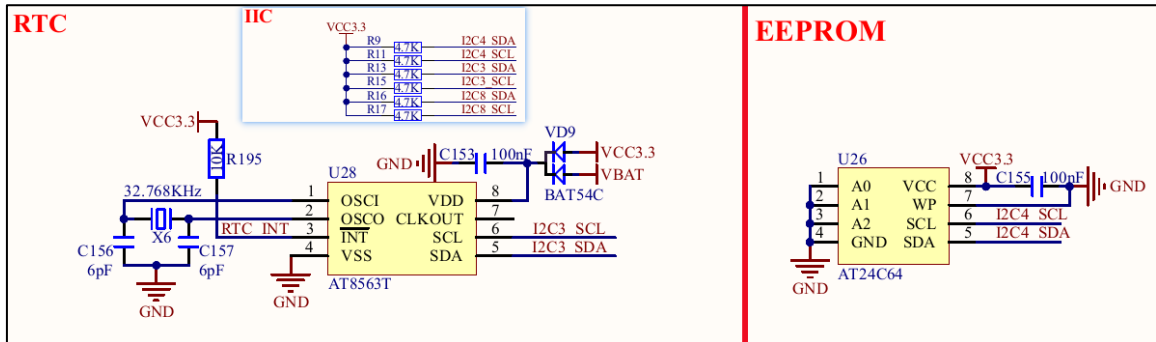


图 12.6-1 开发板 I2C 电路

以上是开发板 I2C 电路节选，I2C4 还挂载了音频芯片和 MIPI CSI，I2C3\4\8 还挂载了屏幕触摸芯片，这里没截图出来，可以自行在底板原理图中查看。本章节笔者只以 I2C 设备 EEPROM 为示例。

EEPROM 器件：AT24C64

地址引脚 (A0, A1, A2): 这些引脚用于设定 EEPROM 的 I2C 地址，可以通过连接到 VCC 或 GND 来选择不同的地址，从而允许在同一 I2C 总线上连接多个 EEPROM。

写保护 (WP): 写保护引脚，当连接到高电平时，EEPROM 处于写保护状态，无法写入数据；当连接到低电平时，允许写操作。

电源 (VCC): 连接到 3.3V 电源。

地 (GND): 连接到地。

串行时钟线 (SCL): I2C 时钟信号输入。

串行数据线 (SDA): I2C 数据输入输出。

在此电路中，AT24C64 的 I2C 地址引脚 (A0, A1, A2) 均连接到 GND，这意味着它的地址是最低的。写保护引脚 WP 也连接到 GND，表示允许写操作。VCC 连接到 3.3V 电源，SCL 和 SDA 分别连接到 I2C 总线的 SCL 和 SDA。

12.6.2 实物硬件

ATK-DLMP257B 开发板上 EEPROM 位置如下：

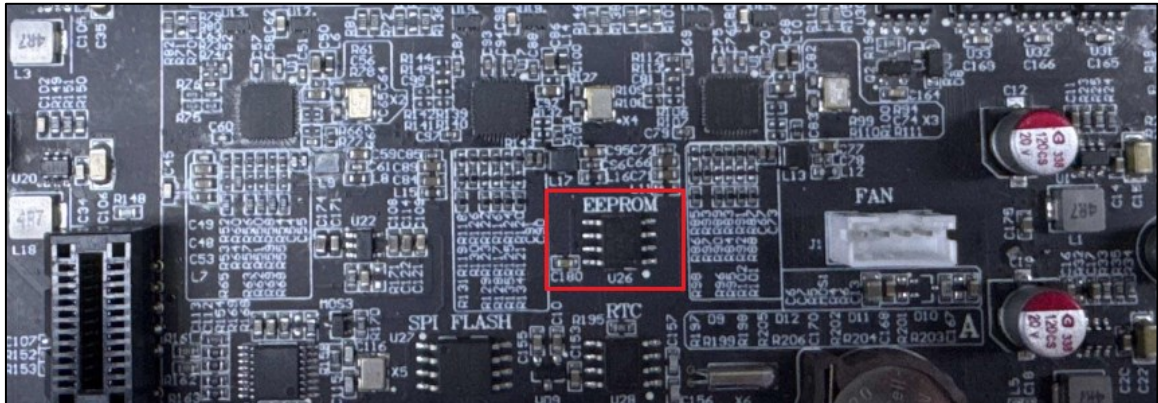


图 12.6-2 开发板 EEPROM 实物

AT24C64 是 Atmel（现为 Microchip Technology）的一个 I2C 接口的 EEPROM 芯片，具有 64Kb（8KB）的存储容量。EEPROM 器件广泛用于嵌入式系统中，用于存储配置数据、校准数据或其他需要在断电后保留的数据，该芯片主要特性：

- 容量：64Kb（8KB）
- 接口：I2C（双线串行接口）
- 工作电压：1.7V 至 5.5V
- 擦写周期：超过 1,000,000 次
- 数据保持时间：超过 100 年

12.7 内核 I2C 子系统使能

在 ATK-DLMP257B 内核源码配置文件 `arch/arm64/configs/stm32mp257_atk_defconfig` 中，I2C 相关配置如下(略有删减，只列举重点部分)：

```
1 CONFIG_I2C=y //启用 I2C 子系统的核心支持
2 CONFIG_I2C_BOARDINFO=y //允许在内核中指定 I2C 设备的板级信息
3 CONFIG_I2C_COMPAT=y /*启用 I2C 的兼容模式支持，
    允许用户空间应用通过/dev 接口访问 I2C 总线*/
4 CONFIG_I2C_HELPER_AUTO=y //自动加载 I2C 辅助模块
5 CONFIG_I2C_GPIO=m //将 GPIO 模拟的 I2C 驱动编译为模块
6 CONFIG_I2C_STM32F7=y //启用 ST 系列处理器的 I2C 支持
7 CONFIG_I2C_SLAVE=y //启用 I2C 从设备模式支持
8 CONFIG_I2C_SLAVE_EEPROM=y //启用 I2C 从设备 EEPROM 模拟支持
```

对应的 menuconfig 配置如下（篇幅有限，截图有所省略）：

```
-> Device Drivers
    -> I2C support --->
        -> *- I2C support
        -> [*] ACPI I2C Operation region support
        -> [*] Enable compatibility bits for old user-space
        -> <*> I2C device interface
        -> {*} I2C bus multiplexing support
        I2C Hardware Bus support --->
            -> <*> STMicroelectronics STM32F7 I2C support
```

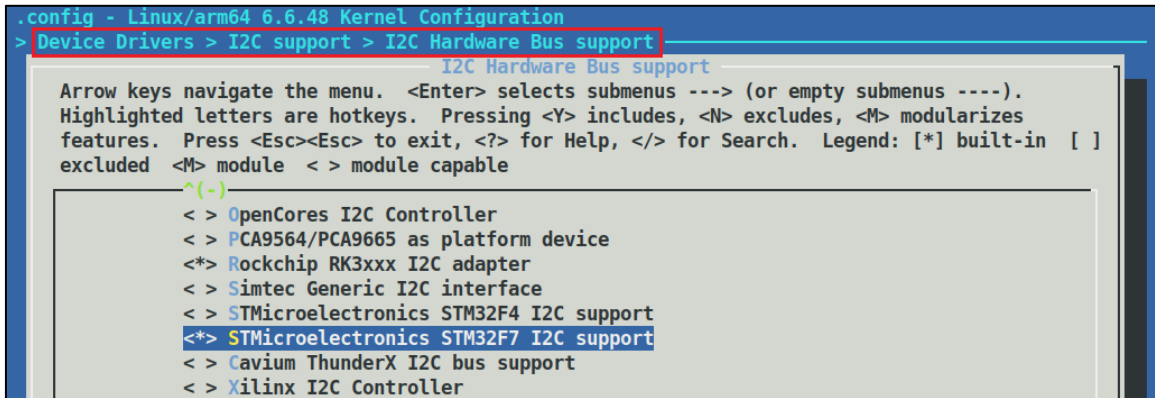


图 12.7-1 I2C 相关配置

12.8 AT24C64 驱动配置

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, AT24C64 相关配置如下:

```
1 CONFIG_EEPROM_AT24=y //启用对 AT24 系列 EEPROM 芯片的支持
```

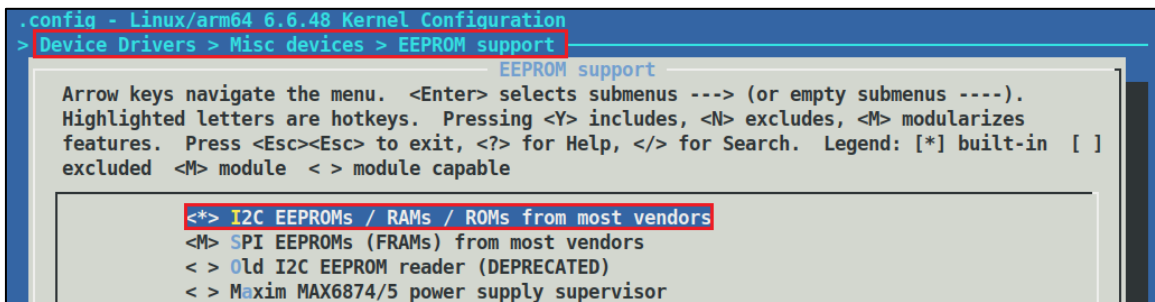


图 12.8-1 CONFIG_EEPROM_AT24 配置

选择配置后, 按下键盘的“?”键可以查看具体配置说明:

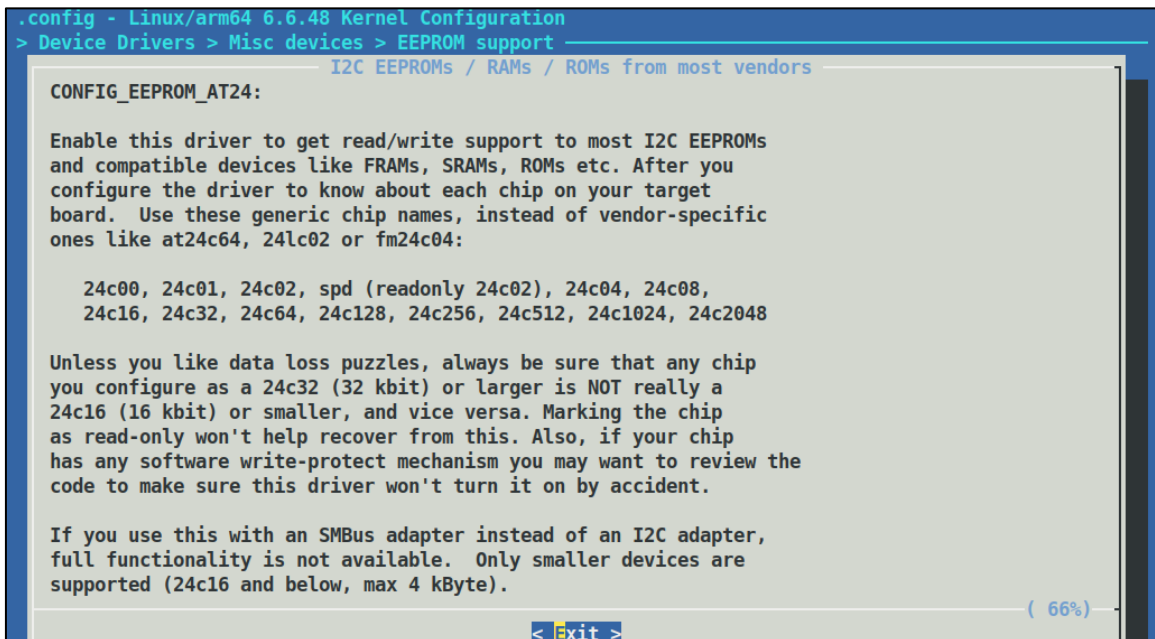


图 12.8-2 CONFIG_EEPROM_AT24 说明

配置项 CONFIG_EEPROM_AT24 说明:

CONFIG_EEPROM_AT24 是 Linux 内核配置选项, 用于启用对大多数 I2C EEPROM 以及类似设备 (如 FRAM、SRAM 和 ROM) 的读写支持。该配置项的详细说明如下:

功能和支持

读写支持: 启用该驱动后, 系统将支持对大多数 I2C EEPROM 和兼容设备 (如 FRAM、SRAM、ROM) 进行读写操作。

兼容设备: 驱动支持的设备包括常见的 I2C EEPROM 型号, 例如 24c00, 24c01, 24c02, 24c04, 24c08, 24c16, 24c32, 24c64, 24c128, 24c256, 24c512, 24c1024, 24c2048 及其他类似设备。

配置设备

配置每个芯片: 在目标板上使用这些通用芯片名称 (如 24c32、24c64 等) 配置每个 EEPROM 芯片, 而不是厂商特定的名称 (如 at24c64, 24lc02, fm24c04 等)。

正确配置容量: 确保为每个芯片配置正确的容量。例如, 不要将一个 16 kbit 的芯片配置为 32 kbit, 反之亦然, 以避免数据丢失。

注意事项

写保护机制: 如果芯片具有软件写保护机制, 建议检查代码以确保驱动程序不会意外启用写保护。

SMBus 适配器限制: 如果使用的是 SMBus 适配器, 而不是 I2C 适配器, 功能会有所限制。仅支持较小的设备 (24c16 及以下, 最大 4 kByte)。

模块支持

模块化: 该驱动程序也可以作为模块构建。如果选择构建为模块, 模块名称为 at24。

依赖关系和选择项

依赖项: 该配置项依赖于 I2C 和 SYSFS 的支持。

选择项: 启用此驱动程序将自动选择 NVMEM、NVMEM_SYSFS、REGMAP 和 REGMAP_I2C 配置选项。

12.9 设备树文件分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi, 此文件是特定于 MP257 系列芯片的设备树包含文件, 通常不独立使用, 而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置, 而 .dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 I2C 的描述如下 (以 I2C4 为例):

```

1      i2c4: i2c@40150000 {
2          compatible = "st,stm32mp25-i2c";
3          reg = <0x40150000 0x400>;
4          interrupt-names = "event";
5          interrupts = <GIC_SPI 168 IRQ_TYPE_LEVEL_HIGH>;
6          clocks = <&rcc CK_KER_I2C4>;
7          resets = <&rcc I2C4_R>;
8          #address-cells = <1>;
9          #size-cells = <0>;
10         dmas = <&hpdma 36 0x20 0x00003012>,
11              <&hpdma 37 0x20 0x00003021>;
12         dma-names = "rx", "tx";
13         access-controllers = <&rifsc 44>;
    
```



```

14         power-domains = <&CLUSTER_PD>;
15         status = "disabled";
16     };
    
```

上面给出的是一个设备树 `stm32mp251.dtsi` 文件的片段, 用于配置 MP257 处理器的 I2C 控制器。这个片段描述了一个 I2C4 控制器及其相关属性。下面是对该设备树片段的详细解析:

第 1 行, `i2c4: i2c@40150000` 定义了 I2C4 控制器的节点, 节点名称为 `i2c4`, 地址为 `0x40150000`。

第 2 行, `compatible = "st,stm32mp25-i2c"` 定义了与设备兼容的驱动程序, 表示该 I2C 控制器兼容 `st,stm32mp25-i2c`, 驱动文件路径 `drivers/i2c/busses/i2c-stm32f7.c`

第 3 行, `reg = <0x40150000 0x400>` 定义了 I2C4 控制器的寄存器地址范围, 起始地址为 `0x40150000`, 长度为 `0x400` 字节。

第 4 行, `interrupt-names = "event"` 定义中断的名称为 `event`, 用于标识中断类型。

第 5 行, `interrupts = <GIC_SPI 168 IRQ_TYPE_LEVEL_HIGH>` 配置中断:

`GIC_SPI`: 表示中断是 SPI (共享外部中断)。

`168`: 中断号为 168。

`IRQ_TYPE_LEVEL_HIGH`: 触发类型为高电平触发。

第 6 行, `clocks = <&rcc CK_KER_I2C4>` 指定时钟源, 由 `rcc` 节点提供, 时钟标识符为 `CK_KER_I2C4`。

第 7 行, `resets = <&rcc I2C4_R>` 指定复位控制, 由 `rcc` 节点管理, 复位标识符为 `I2C4_R`。

第 8 行, `#address-cells = <1>` 表示 I2C 设备节点的地址单元大小为 1。

第 9 行, `#size-cells = <0>` 表示 I2C 设备节点的尺寸单元大小为 0。

第 10 行, `dmass = <&hpdma 36 0x20 0x00003012>, <&hpdma 37 0x20 0x00003021>` 配置 DMA :

`&hpdma`: 指定 DMA 控制器。

`36/37`: 分别表示接收 (rx) 和发送 (tx) 通道编号。

`0x20`: 优先级或其他标志。

`0x00003012 / 0x00003021`: DMA 配置标识符

第 12 行, `dma-names = "rx", "tx"` 分别为接收和发送 DMA 通道指定名称: `rx` 和 `tx`。

第 13 行, `access-controllers = <&rifsc 44>` 指定访问控制器, 由 `rifsc` 节点提供, 标识符为 `44`。

第 14 行, `power-domains = <&CLUSTER_PD>` 指定电源域, 电源管理由 `CLUSTER_PD` 节点负责。

第 15 行, `status = "disabled"` 表示该 I2C 控制器节点当前状态为“禁用”。

以 2GB 版本核心板为例, 在 `arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts` 中, `i2c4` 和 `eeeprom` 相关配置如下:

```

1 &i2c4 {
2     pinctrl-names = "default", "sleep";
3     pinctrl-0 = <&i2c4_pins_a>;
4     pinctrl-1 = <&i2c4_sleep_pins_a>;
5     status = "okay";
6     /delete-property/dmass;
7     /delete-property/dma-names;
8 }
    
```

```

9  eeprom@50 {
10     compatible = "microchip,24c64", "atmel,24c64";
11     pagesize = <32>;
12     reg = <0x50>;
13     vcc-supply = <&vdd_3v3>;
14     status = "okay";
15 };
16};

```

上面给出的是一个设备树 `stm32mp257d-atk-ddr-2GB.dts` 文件的片段，用于配置 MP257 处理器的 I2C 控制器及其相关的 EEPROM 设备。下面是对该设备树片段的详细解析：

第 1 行，`&i2c4` 引用 I2C4 控制器节点并对其进行修改或扩展配置。

第 2 行，`pinctrl-names = "default", "sleep"` 定义引脚控制（`pinctrl`）的名称：

`default`: 表示正常工作状态下的引脚配置。

`sleep`: 表示休眠状态下的引脚配置。

第 3 行，`pinctrl-0 = <&i2c4_pins_a>` 指定正常工作状态下的引脚配置，使用 `i2c4_pins_a`。

第 4 行，`pinctrl-1 = <&i2c4_sleep_pins_a>` 指定休眠状态下的引脚配置，使用 `i2c4_sleep_pins_a`。

第 5 行，`status = "okay"` 将 I2C4 控制器的状态设置为 启用。

第 6 行，`/delete-property/dmas` 删除 `dmas` 属性，表示禁用 DMA 通道。

第 7 行，`/delete-property/dma-names` 删除 `dma-names` 属性，表示不再为 DMA 通道指定名称。

第 9 行，`eeprom@50` 定义 I2C 总线上的一个从设备节点，地址为 `0x50` (EEPROM 节点)。

第 10 行，`compatible = "microchip,24c64", "atmel,24c64"` 指定 EEPROM 设备的兼容性：

`microchip,24c64`: Microchip 的 24c64 EEPROM 芯片。

`atmel,24c64`: Atmel 的 24c64 EEPROM 芯片。

对应的驱动路径 `drivers/misc/eeprom/at24.c`

第 11 行，`pagesize = <32>` 定义 EEPROM 的页面大小为 32 字节。

第 12 行，`reg = <0x50>` 指定 EEPROM 在 I2C 总线上的地址为 `0x50`。

第 13 行，`vcc-supply = <&vdd_3v3>` 指定 EEPROM 的电源供应节点，使用 `vdd_3v3`。

第 14 行，`status = "okay"` 将 EEPROM 节点的状态设置为 启用。

12.10 EEPROM 的使用

出厂系统上已经配置好 AT24C64 驱动并默认编译进内核了，在系统中会生成一个专门的路径 `/sys/bus/i2c/devices/1-0050`。

```

cd /sys/bus/i2c/devices/1-0050
ls

```

```

root@ATK-DLMP257:~# cd /sys/bus/i2c/devices/1-0050
root@ATK-DLMP257:/sys/bus/i2c/devices/1-0050# ls
1-00502  eeprom  name     power    supplier:platform:regulator-vdd-3v3 uevent
driver  modalias of_node subsystem supplier:regulator:regulator.3
root@ATK-DLMP257:/sys/bus/i2c/devices/1-0050#

```

图 12.10-1 `/sys/bus/i2c/devices/1-0050`

目录 `/sys/bus/i2c/devices/1-0050` 包含了与 I2C 设备 1-0050（地址为 `0x50`）的相关信息。以下是每个文件和目录的解释：



- 1-00501: 这是一个符号链接, 指向同一总线下的另一设备(可能是子设备或相关设备)。
- **eeprom**: 这个文件表示 EEPROM 存储空间, 通常可以通过读取和写入这个文件来操作 EEPROM 数据。
- **name**: 这个文件包含设备的名称。
- **power**: 这个目录包含与设备电源管理相关的文件。
- **uevent**: 这个文件包含设备的 uevent 信息, 用于通知用户空间关于设备的事件。
- **driver**: 这是一个符号链接, 指向驱动程序目录, 表示哪个驱动程序正在管理这个设备。
- **modalias**: 这个文件包含设备的模式别名, 用于自动加载合适的驱动程序。
- **of_node**: 这是一个符号链接, 指向设备树节点, 包含设备的设备树信息。
- **subsystem**: 这是一个符号链接, 指向设备所属的子系统(在这里是 I2C)。
- **supplier:platform:regulator-vdd-3v3**: 这是一个符号链接, 指向提供电源的调节器设备。

12.10.1 EEPROM 读写测试

ATK-DLMP257B 开发板板载 EEPROM 芯片, 型号为 AT24C64, 它通过 I2C 总线进行读写访问, 具备 8KB 存储空间。

出厂系统提供一个简单读写测试工具 **eeprom-rw** 来进行 AT24C64 的读写测试, 执行以下指令即可。

```
cd shell/eeprom
./eeprom-rw
```

返回如下信息, 表示读写正常。

```
root@ATK-DLMP257:~# cd shell/eeprom/
root@ATK-DLMP257:~/shell/eeprom# ls
eeprom-rw
root@ATK-DLMP257:~/shell/eeprom# ./eeprom-rw
len = 32
writing...

===write ok===
reading...
Hello,this is an eeprom test!!!!Hello,this is an eeprom test!!!!Hello,this is an eeprom test!!
!!Hello,this is an eeprom test!!!!Hello,this is an eeprom test!!!!Hello,this is an eeprom test
!!!!Hello,this is an eeprom test!!!!Hello,this is an eeprom test!!!!Hello,this is an eeprom te
st!!!!Hello,this is an eeprom test!!!!Hello,this is an eeprom test!!!!Hello,this is an eeprom
```

图 12.10-2 eeprom 读写测试

12.10.2 读取 EEPROM 内容

可以使用 **hexdump** 命令读取 **eeprom** 文件的内容来查看 EEPROM 数据:

```
hexdump -C /sys/bus/i2c/devices/1-0050/eeprom
```

```
root@ATK-DLMP257:~/shell/eeprom# hexdump -C /sys/bus/i2c/devices/1-0050/eeprom
00000000  48 65 6c 6c 6f 2c 74 68  69 73 20 69 73 20 61 6e  |Hello,this is an|
00000010  20 65 65 70 72 6f 6d 20  74 65 73 74 21 21 21 21  | eeprom test!!!!|
00000020  48 65 6c 6c 6f 2c 74 68  69 73 20 69 73 20 61 6e  |Hello,this is an|
00000030  20 65 65 70 72 6f 6d 20  74 65 73 74 21 21 21 21  | eeprom test!!!!|
00000040  48 65 6c 6c 6f 2c 74 68  69 73 20 69 73 20 61 6e  |Hello,this is an|
```

图 12.10-3 hexdump 读取 eeprom 数据

这个输出表示从 EEPROM 的起始地址读取的数据, 具体解析如下:

地址 00000000 到 0000000F:

```
48 65 6C 6C 6F 2C 74 68 69 73 20 69 73 20 61 6E
```

对应的 ASCII 字符: **Hello,this is an**

地址 00000010 到 0000001F:

```
20 65 65 70 72 6F 20 74 65 73 74 21 21 21 21
```

对应的 ASCII 字符: `eeeprom test!!!!`

这说明 EEPROM 中的前 32 字节存储的是字符串 "Hello,this is an eeeprom test!!!!"。后面的以此类推。

12.10.3 清除 EEPROM 中的数据

如果想清除 EEPROM 中的数据, 可以使用以下步骤将所有字节写为 0x00:

```
dd if=/dev/zero bs=1 count=8192 of=/sys/bus/i2c/devices/1-0050/eeprom
sleep 1
```

`sleep 1` 用于等待写入时间, 确保有足够的时间写入数据, 防止数据写入不全。

```
root@ATK-DLMP257:~# dd if=/dev/zero bs=1 count=8192 of=/sys/bus/i2c/devices/1-0050/eeprom
8192+0 records in
8192+0 records out
8192 bytes (8.2 kB, 8.0 KiB) copied, 18.2897 s, 0.4 kB/s
root@ATK-DLMP257:~# sleep 1
root@ATK-DLMP257:~#
```

图 12.10-4 清除 EEPROM 数据

重新查看 EEPROM 数据:

```
hexdump -C /sys/bus/i2c/devices/1-0050/eeprom
```

```
root@ATK-DLMP257:~# hexdump -C /sys/bus/i2c/devices/1-0050/eeprom
00000000  00 00 00 00 00 00 00 00 00 00 00 00 00 00  |.....|
*
00002000
root@ATK-DLMP257:~#
```

图 12.10-5 重新查看 EEPROM 数据

12.10.4 写入数据到 EEPROM

检查写保护状态: 查阅设备手册, 确认写保护引脚 (WP) 的状态。

硬件检查: 确保写保护引脚未被拉高。

确保 EEPROM 可写后, 可以使用 `echo` 或其他命令将数据写入 `eeeprom` 文件。注意数据写入的格式和限制:

```
echo -n "ALIENTEK" > /sys/bus/i2c/devices/1-0050/eeprom
sleep 1
hexdump -C /sys/bus/i2c/devices/1-0050/eeprom
```

`sleep 1` 用于等待写入时间, 确保有足够的时间写入数据, 防止数据写入不全。

```
root@ATK-DLMP257:~# echo -n "ALIENTEK" > /sys/bus/i2c/devices/1-0050/eeprom
root@ATK-DLMP257:~# sleep 1
root@ATK-DLMP257:~# hexdump -C /sys/bus/i2c/devices/1-0050/eeprom
00000000  41 4c 49 45 4e 54 45 4b 00 00 00 00 00 00  |ALIENTEK.....|
00000010  00 00 00 00 00 00 00 00 00 00 00 00 00 00  |.....|
*
00002000
root@ATK-DLMP257:~#
```

图 12.10-6 写入数据到 EEPROM

12.11 I2C 测试工具

在嵌入式 Linux 系统中，有一系列工具可以用来管理和调试 I2C 设备。这些工具大多数都是命令行工具，提供对 I2C 总线和设备的直接访问。其中最常用的工具就是 i2c-tools，这个工具在开发板的出厂系统中也有集成。

i2c-tools 是一个常用的工具集，用于在 Linux 系统中与 I2C 设备进行交互。它提供了几个命令行工具，可以检测、读取和写入 I2C 设备。以下是 i2c-tools 包中的主要工具及其使用说明：

12.11.1 i2cdetect

i2cdetect 用于检测 I2C 总线上连接的所有设备。

直接执行 i2cdetect 可以查看使用说明：

i2cdetect

```
root@ATK-DLMP257:~# i2cdetect
Error: No i2c-bus specified!
Usage: i2cdetect [-y] [-a] [-q|-r] I2CBUS [FIRST LAST]
        i2cdetect -F I2CBUS
        i2cdetect -l
        I2CBUS is an integer or an I2C bus name
        If provided, FIRST and LAST limit the probing range.
root@ATK-DLMP257:~#
```

图 12.11-1 i2cdetect 使用说明

- 用法 1：列出所有 I2C 总线。

示例：

i2cdetect -l

```
root@ATK-DLMP257:~# i2cdetect -l
i2c-0  i2c      STM32F7 I2C(0x0000000040140000)      I2C adapter
i2c-1  i2c      STM32F7 I2C(0x0000000040150000)      I2C adapter
i2c-2  i2c      STM32F7 I2C(0x0000000040160000)      I2C adapter
root@ATK-DLMP257:~#
```

图 12.11-2 列出 I2C 总线

可以看到出厂系统上共有三路 I2C 总线，根据地址可以对应设备树上的 i2c3、i2c4、i2c8。

- 用法 2：扫描指定的 I2C 总线，-y 参数用于跳过用户确认，直接执行命令。

i2cdetect [-y] [-a] [-q|-r] I2CBUS [FIRST LAST]

参数说明

-y: 跳过用户确认，直接执行命令。

-a: 显示所有地址，包括保留地址。

-q 或 -r: 分别使用快速模式或重复开始模式进行检测。

I2CBUS: I2C 总线的编号或名称。例如，1 表示 /dev/i2c-1。

FIRST 和 LAST: 指定检测的地址范围。

示例 1：扫描 I2C4 上的所有地址。

i2cdetect -y 1

这将扫描 I2C4 上的所有地址，并显示连接的设备。这个和前面提到的 aliases 别名是相关的。比如这里开发板 EEPROM 挂载在 I2C4 总线上，那么就要执行 i2cdetect -y 1 进行扫描。

```
root@ATK-DLMP257:~# i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  UU  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  UU  --  --  --  --  --  --  --  --  --  --  --  --  --
20:  --  --  UU  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
50:  UU  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
root@ATK-DLMP257:~#
```

图 12.11-3 i2cdetect -y 1 示例

格式说明

纵列: 表示 I2C 地址的低四位 (0-15)。

横行: 表示 I2C 地址的高四位 (0x00, 0x10, 0x20, ...)。

例如, 0x25 表示的是横行 2 和纵列 5 交叉处的地址 (0x20 + 0x05 = 0x25)。

特殊标记解释

--: 表示该地址没有设备响应。

UU: 表示该地址被内核占用, 通常是因为一个设备驱动程序已绑定到该地址的设备。

UU 的含义

在示例中, 地址 0x50 被标记为 UU, 这表明此地址上的设备正被内核驱动程序占用, 通常是 I2C EEPROM 设备。用户可以通过 sysfs 接口来访问这些设备, 因为它们已经由内核处理。

设备地址解释

0x50: 地址 0x50 被标记为 UU, 表示该地址上的设备已被内核占用。通常, 这是一个 EEPROM 设备, 因为地址 0x50 常用于此类设备。从设备树中可知这个是 AT24C64 芯片。

以此类推, 结合设备树可知, 0x11 地址为 es8388, 0x22 地址为 fusb302, 这两个设备已被内核占用。

- 用法 3: 显示 I2C 总线的功能。

示例:

i2cdetect -F 1

```
root@ATK-DLMP257:~# i2cdetect -F 1
Functionalities implemented by /dev/i2c-1:
I2C                               yes
SMBus Quick Command              yes
SMBus Send Byte                  yes
SMBus Receive Byte               yes
SMBus Write Byte                 yes
SMBus Read Byte                  yes
SMBus Write Word                 yes
SMBus Read Word                  yes
SMBus Process Call               yes
SMBus Block Write                yes
SMBus Block Read                 yes
SMBus Block Process Call         yes
SMBus PEC                        yes
I2C Block Write                  yes
I2C Block Read                   yes
root@ATK-DLMP257:~#
```

图 12.11-4 i2cdetect -F 1 示例

上面成功查看了 /dev/i2c-1 的功能列表。以下是各功能的说明:

I2C: 标准 I2C 通信。

SMBus Quick Command: 快速命令模式。

SMBus Send Byte: 发送一个字节。

SMBus Receive Byte: 接收一个字节。



SMBus Write Byte: 写入一个字节。
 SMBus Read Byte: 读取一个字节。
 SMBus Write Word: 写入一个字。
 SMBus Read Word: 读取一个字。
 SMBus Process Call: 处理调用。
 SMBus Block Write: 块写入。
 SMBus Block Read: 块读取。
 SMBus Block Process Call: 块处理调用（不支持）。
 SMBus PEC: Packet Error Checking, 包错误校验。
 I2C Block Write: I2C 块写入。
 I2C Block Read: I2C 块读取。

12.11.2 i2cdump

i2cdump 用于显示 I2C 设备的寄存器内容。

```
root@ATK-DLMP257:~# i2cdump
Error: No i2c-bus specified!
Usage: i2cdump [-f] [-y] [-r first-last] [-a] I2CBUS ADDRESS [MODE [BANK [BANKREG]]]
I2CBUS is an integer or an I2C bus name
ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
MODE is one of:
  b (byte, default)
  w (word)
  W (word on even register addresses)
  s (SMBus block, deprecated)
  i (I2C block)
  c (consecutive byte)
Append p for SMBus PEC
root@ATK-DLMP257:~#
```

图 12.11-5 i2cdump 用法

用法:

i2cdump [options] I2CBUS ADDRESS [MODE [BANK [BANKREG]]]

- I2CBUS 是 I2C 总线的编号或名称。
- ADDRESS 是设备的地址 (0x08 - 0x77, 或使用 -a 选项时为 0x00 - 0x7f)。
- MODE 是以下模式之一:
 - ◆ b (字节, 默认)
 - ◆ w (字)
 - ◆ W (偶数寄存器地址上的字)
 - ◆ s (SMBus 块, 已弃用)
 - ◆ i (I2C 块)
 - ◆ c (连续字节)
 - ◆ 可附加 p 以使用 SMBus PEC

示例:

i2cdump -y 1 0x50 i

这将使用 i2cdump 读取 I2C 总线 1 上地址为 0x50 的设备, 并以 I2C 块模式读取数据。

```
root@ATK-DLMP257:~# i2cdump -y 1 0x50 i
Error: Could not set address to 0x50: Device or resource busy
root@ATK-DLMP257:~#
```

图 12.11-6 读取结果

可以看到这里显示 0x50 设备（也就是 AT24C64）已经被使用了，因为这个驱动默认已经被编译进出厂系统内核中了，已经可以直接使用的了。i2cdump 适用于驱动开发前期的调试阶段，将内核编译成模块，不加载时就可以使用 i2cdump 工具。

既然讲到这里了，笔者顺便演示下如何将内核中的 AT24C64 驱动编译成驱动模块，不编译进内核，通过手动加载模块的方式添加驱动，这样就能使用 i2cdump 来调试 AT24C64 芯片。

在 12.8 小节中，我们已经知道 AT24C64 驱动编译进内核里了，我们重新打开配置文件并将其修改成编译成模块。

```
source /opt/stm32mp257/5.0.3-snapshot/environment-setup-cortexa35-ostl-linux
make stm32mp257_atk_defconfig
make menuconfig
```

在菜单图形化界面中，按下 “/” 键弹出搜索框，搜索 “CONFIG_EEPROM_AT24”，选择对应的选项（这里是选项 1），就可以直接到达 CONFIG_EEPROM_AT24 的配置，选中配置项并输入 “M” 键将 CONFIG_EEPROM_AT24 构建成内核模块的形式。

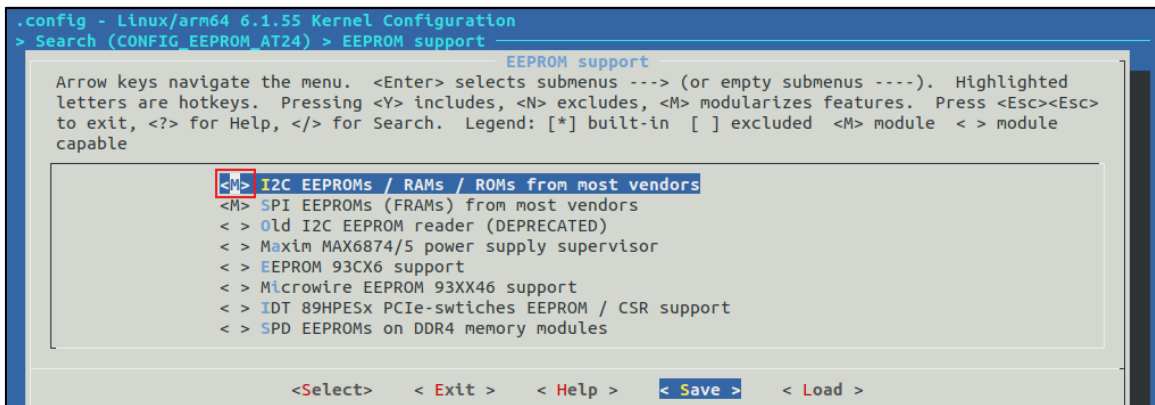


图 12.11-7 编译成内核模块

将修改好的配置文件保存为 ./arch/arm64/configs/stm32mp257_atk_defconfig，将修改好的配置写入其中。此时可以看到 stm32mp257_atk_defconfig 的配置中有 CONFIG_EEPROM_AT24=m。重新编译内核和驱动模块：

```
./build_kernel.sh
```

将编译得到的内核镜像 Image.gz 更新到开发板的/boot 目录，将 at24.ko 拷贝到开发板的 /lib/modules/6.6.48 目录，然后重启开发板。

使用 i2cdetect 工具再次查看 I2C4 总线。

```
i2cdetect -y 1
```

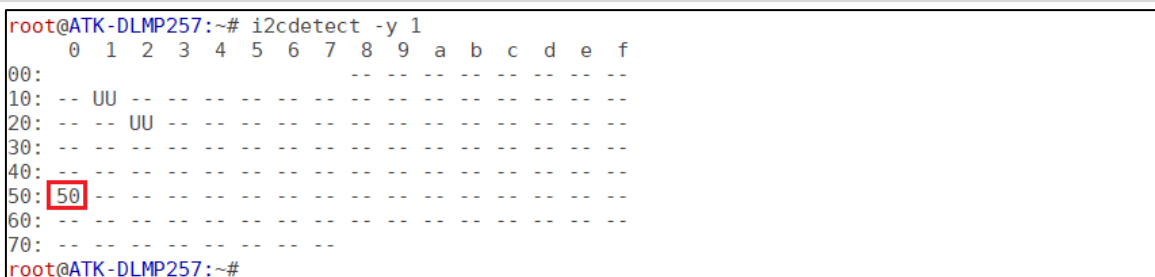


图 12.11-8 I2C4 总线设备情况

可以看到此时 0x50 设备不是显示 UU 标记了，表示没有被编译进内核或者使用。这里我们验证下我们的 at24.ko 驱动的可用性。进入开发板系统/lib/modules/6.6.48 目录下，执行驱动加载指令和测试指令：

```
cd /lib/modules/6.6.48
insmod at24.ko
lsmod | grep at24
i2cdetect -y 1
```

```
root@ATK-DLMP257:~# cd /lib/modules/6.6.48
root@ATK-DLMP257:/lib/modules/6.6.48# insmod at24.ko
t -y 1
[ 97.363197] at24 1-0050: 8192 byte 24c64 EEPROM, writable, 32 bytes/write
root@ATK-DLMP257:/lib/modules/6.6.48# lsmod | grep at24
at24                24576  0
root@ATK-DLMP257:/lib/modules/6.6.48# i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  UU  --  --  --  --  --  --  --  --  --  --  --  --
20:  --  --  UU  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
50:  UU  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
root@ATK-DLMP257:/lib/modules/6.6.48#
```

驱动模块加载信息

查看模块加载情况

再次查看I2C4总线设备情况

图 12.11-9 加载 at24.ko 驱动和验证

可以看到,当我们把 at24.ko 驱动加载到系统后,i2cdetect 显示 0x50 设备的标记变为了 UU,意味着被使用了。

接下来我们就可以卸载 at24.ko 驱动,就可以使用 i2cdump 工具来查看 AT24C64 芯片的寄存器情况了。

```
rmmod at24.ko
i2cdump -y 1 0x50 //初始化
i2cdump -y 1 0x50 c
```

```

root@ATK-DLMP257:/lib/modules/6.6.48# rmmod at24.ko
root@ATK-DLMP257:/lib/modules/6.6.48# i2cdump -y 1 0x50
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f      0123456789abcdef
00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
root@ATK-DLMP257:/lib/modules/6.6.48# i2cdump -y 1 0x50 c
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f      0123456789abcdef
00: 41 4c 49 45 4e 54 45 4b 00 00 00 00 00 00 00 00      ALIENTEK.....
10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....
    
```

图 12.11-10 使用 i2cdump

笔者的 AT24C64 之前存储了一些数据，这个输出显示了从 EEPROM 读取的数据，数据显示的方式是十六进制形式，以及相应的 ASCII 字符。这表明 EEPROM 中存储的内容是一个测试消息：“ALIENTEK”。

我们可以使用 i2cset 往寄存器中写入数据。

12.11.3 i2cset

i2cset 命令用于向 I2C 设备寄存器写入数据，执行 i2cset 可以查看使用说明：

```

root@ATK-DLMP257:/lib/modules/6.6.48# i2cset
Usage: i2cset [-f] [-y] [-m MASK] [-r] [-a] I2CBUS CHIP-ADDRESS DATA-ADDRESS [VALUE] ... [MODE]
I2CBUS is an integer or an I2C bus name
ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
MODE is one of:
  c (byte, no value)
  b (byte data, default)
  w (word data)
  i (I2C block data)
  s (SMBus block data)
Append p for SMBus PEC
root@ATK-DLMP257:/lib/modules/6.6.48#
    
```

图 12.11-11 i2cset 使用说明

参数解释

- I2CBUS: I2C 总线的编号或名称（例如，1）。
- CHIP-ADDRESS: I2C 设备的地址（例如，0x50）。
- DATA-ADDRESS: 要写入数据的地址（EEPROM 内的地址）。
- [VALUE]: 要写入的数据。

- [MODE]: 数据传输模式, 如 c (字节, 无值)、b (字节数据, 默认)、w (字数据)、i (I2C 块数据)、s (SMBus 块数据)。

开发板上 I2C 设备是一个 EEPROM, 使用 i2ctransfer 进行复杂操作, 这里不演示 i2cset。

12.11.4 i2cget

i2cget 命令用于从 I2C 设备寄存器读取数据, 执行 i2cget 可以查看使用说明:

```
root@ATK-DLMP257:/lib/modules/6.6.48# i2cget
Usage: i2cget [-f] [-y] [-a] I2CBUS CHIP-ADDRESS [DATA-ADDRESS [MODE [LENGTH]]]
I2CBUS is an integer or an I2C bus name
ADDRESS is an integer (0x08 - 0x77, or 0x00 - 0x7f if -a is given)
MODE is one of:
  b (read byte data, default)
  w (read word data)
  c (write byte/read byte)
  s (read SMBus block data)
  i (read I2C block data)
  Append p for SMBus PEC
LENGTH is the I2C block data length (between 1 and 32, default 32)
root@ATK-DLMP257:/lib/modules/6.6.48#
```

图 12.11-12 i2cget 使用说明

参数解释

- I2CBUS: I2C 总线号或名称, 通常是一个整数。例如 1 表示 /dev/i2c-1。
- CHIP-ADDRESS: 设备的 I2C 地址, 通常是 0x08 到 0x77 范围内的整数。如果使用 -a 选项, 则范围扩展到 0x00 到 0x7f。
- DATA-ADDRESS: 要读取的数据寄存器地址, 通常是一个整数。
- MODE: 读取模式, 可以是以下几种之一:
 - b: 读取一个字节数据 (默认)
 - w: 读取一个字 (2 字节) 数据
 - c: 写入一个字节并读取一个字节
 - s: 读取 SMBus 块数据
 - i: 读取 I2C 块数据
 - p: 如果使用 SMBus PEC (Packet Error Checking), 则在模式后添加 p
- LENGTH: I2C 块数据长度 (1 到 32 之间), 默认为 32

开发板上 I2C 设备是一个 EEPROM, 使用 i2ctransfer 进行复杂操作, 这里不演示 i2cget。

12.11.5 i2ctransfer

i2ctransfer 命令用于进行复杂的 I2C 读写操作, 可以一次性进行多个读写传输。执行 i2ctransfer 命令可以看到使用方法:

```

root@ATK-DLMP257:/lib/modules/6.6.48# i2ctransfer
Usage: i2ctransfer [-f] [-y] [-v] [-V] [-a] I2CBUS DESC [DATA] [DESC [DATA]]...
I2CBUS is an integer or an I2C bus name
DESC describes the transfer in the form: {r|w}LENGTH[@address]
  1) read/write-flag 2) LENGTH (range 0-65535, or '?')
  3) I2C address (use last one if omitted)
DATA are LENGTH bytes for a write message. They can be shortened by a suffix:
  = (keep value constant until LENGTH)
  + (increase value by 1 until LENGTH)
  - (decrease value by 1 until LENGTH)
  p (use pseudo random generator until LENGTH with value as seed)

Example (bus 0, read 8 byte at offset 0x64 from EEPROM at 0x50):
# i2ctransfer 0 w1@0x50 0x64 r8
Example (same EEPROM, at offset 0x42 write 0xff 0xfe ... 0xf0):
# i2ctransfer 0 w17@0x50 0x42 0xff-
root@ATK-DLMP257:/lib/modules/6.6.48#

```

图 12.11-13 i2ctransfer 使用方法

参数解释

- -f: 强制访问 I2C 设备，即使它被另一个进程使用。
- -y: 自动确认提示，不需要用户交互确认。
- -v: 显示详细的操作信息。
- -V: 显示版本信息。
- -a: 允许使用 10 位地址。

描述符 (DESC)

DESC 描述了传输的类型和长度，其格式为 {r|w}LENGTH[@address]:

- r: 表示读操作。
- w: 表示写操作。
- LENGTH: 表示读或写的字节数（范围 0-65535），可以使用 ? 表示不确定长度。
- @address: 可选的 I2C 地址（如果省略，则使用最后一个地址）。

数据 (DATA)

- DATA 是用于写操作的数据，长度应与描述符中的 LENGTH 匹配。数据可以使用以下缩写形式：
- =: 保持值不变直到指定长度。
- +: 每次递增 1 直到指定长度。
- -: 每次递减 1 直到指定长度。
- p: 使用伪随机生成器直到指定长度。

使用示例：

通过 i2ctransfer 向 I2C 地址 0x50 的设备写入字符串 "HELLO"

```
i2ctransfer -f -y 1 w7@0x50 0x00 0x00 0x48 0x45 0x4c 0x4c 0x4f
```

通过 i2ctransfer 从 I2C 地址 0x50 的设备读取 5 个字节的数据

```
i2ctransfer -f -y 1 w2@0x50 0x00 0x00 r5
```



```

root@ATK-DLMP257:~# i2ctransfer -f -y 1 w7@0x50 0x00 0x00 0x48 0x45 0x4c 0x4c 0x4f
root@ATK-DLMP257:~# i2ctransfer -f -y 1 w2@0x50 0x00 0x00 r5
0x48 0x45 0x4c 0x4c 0x4f
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# i2cdump -y 1 0x50
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....
root@ATK-DLMP257:~# i2cdump -y 1 0x50 c
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 48 45 4c 4c 4f 54 45 4b 00 00 00 00 00 00 00 00    HELL0TEK.....
10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    .....

```

图 12.11-14 i2ctransfer 使用

第十三章 SPI 子系统

13.1 SPI 基础知识

SPI (Serial Peripheral Interface, 串行外设接口) 是一种同步串行通信协议, 常用于微控制器与外围设备之间的通信。它是一种全双工通信协议, 能够实现高速数据传输, 通常用于短距离通信, 如传感器、存储器和显示器等外设的连接。

SPI 的基本概念

- **主从模式 (Master-Slave) :**

主设备 (Master): 通常是微控制器, 控制整个通信过程, 生成时钟信号。

从设备 (Slave): 被动响应主设备的命令, 接收或发送数据。

- **四条主要信号线:**

MOSI (Master Out Slave In): 主设备发送数据, 从设备接收数据。

MISO (Master In Slave Out): 从设备发送数据, 主设备接收数据。

SCLK (Serial Clock): 时钟信号, 由主设备生成, 控制数据传输的同步。

SS (Slave Select): 从设备选择信号, 当该信号为低电平时, 从设备被选中并参与通信。

- **数据传输模式:**

全双工通信: 同时发送和接收数据, MOSI 和 MISO 信号线同时工作。

数据帧格式: 数据以帧的形式传输, 每帧通常为 8 位或 16 位。

- **时钟极性 (CPOL) 和时钟相位 (CPHA) :**

CPOL: 定义时钟信号的空闲状态, 0 表示空闲时钟为低电平, 1 表示空闲时钟为高电平。

CPHA: 定义数据的采样时间, 0 表示第一个时钟边沿进行采样, 1 表示第二个时钟边沿进行采样。

SPI 的优缺点

优点:

高速传输: SPI 能够支持非常高的时钟频率, 通常高于 I2C。

全双工通信: 同时进行数据发送和接收, 提高了通信效率。

简单协议: 实现简单, 无需复杂的握手机制, 硬件开销低。

缺点:

占用引脚多: 每增加一个从设备, 需要增加一个独立的 SS 引脚。

短距离通信: 适用于短距离设备间通信, 不适合长距离传输。

缺乏标准化: 不同设备可能会有不同的 SPI 实现方式, 需要根据具体设备手册进行配置。

SPI 通信过程

初始化: 主设备配置 SPI 参数, 包括时钟频率、数据格式 (CPOL、CPHA)、以及相关引脚。

选择从设备: 主设备将相应的 SS 引脚拉低, 选择要通信的从设备。

数据传输：主设备生成时钟信号，并通过 MOSI 发送数据，同时通过 MISO 接收从设备的数据。

结束传输：数据传输完成后，主设备将 SS 引脚拉高，结束与从设备的通信。

SPI 应用实例

存储器：如闪存（Flash）、EEPROM 等，常通过 SPI 接口进行数据读写。

传感器：如温度传感器、加速度传感器等，通过 SPI 接口传输测量数据。

显示器：如 LCD、OLED 显示器，通过 SPI 接口发送图像数据。

SPI 是一种灵活且高效的通信协议，广泛应用于嵌入式系统中。通过合理配置和使用 SPI，可以实现各种外设与微控制器之间的高速数据交换。

SPI 与其他通信协议的比较

与 I2C 的比较：

速度：SPI 通常比 I2C 速度更快，支持更高的时钟频率。

引脚数：I2C 仅需两根信号线（SCL 和 SDA），而 SPI 需要四根或更多信号线。

复杂度：SPI 协议相对简单，无需地址选择和确认机制，I2C 则需要处理地址冲突和握手信号。

与 UART 的比较：

模式：SPI 是同步通信，UART 是异步通信。

速度：SPI 速度通常高于 UART，支持更高的传输速率。

连接方式：SPI 通常用于短距离通信，UART 可以用于较长距离通信，且支持点对点和多点连接。

13.2 SPI 子系统简介

SPI（Serial Peripheral Interface，串行外围设备接口）是嵌入式系统中常用的一种同步串行通信协议。它由 Motorola（现为 Freescale）公司在上世纪 80 年代开发，用于微控制器与外围设备之间的高速数据传输。SPI 协议广泛应用于 EEPROM、Flash、实时时钟（RTC）、ADC、DAC 和显示控制器等设备中。

13.3 主要特点

嵌入式 Linux SPI 子系统主要有以下几个特点：

1、高效的同步串行通信

SPI 协议是一种同步串行通信协议，具有全双工通信能力，可以同时进行数据的发送和接收，适用于高速数据传输。SPI 子系统通过高效的驱动程序实现了对 SPI 设备的支持，能够在嵌入式系统中实现快速、可靠的数据传输。

2、灵活的设备树支持

嵌入式 Linux SPI 子系统通过设备树（Device Tree）机制描述硬件架构和设备信息，方便系统在启动时自动加载和配置 SPI 设备。设备树支持使得硬件平台的移植和设备的添加变得更加简单和灵活。

3、统一的用户空间接口

SPI 子系统提供了统一的用户空间接口，用户应用程序可以通过标准文件接口（如 `/dev/spidevX.Y`）与 SPI 设备进行通信。用户空间接口的标准化和统一化简化了应用程序与 SPI 设备交互的过程，提高了开发效率。



4、广泛的设备支持

SPI 子系统支持多种类型的 SPI 设备, 如 EEPROM、Flash、实时时钟 (RTC)、ADC、DAC 和显示控制器等。通过 SPI 主机控制器驱动和设备驱动的协作, SPI 子系统能够兼容和支持各种不同的 SPI 从设备, 满足不同应用场景的需求。

这些特点使得嵌入式 Linux SPI 子系统在嵌入式开发中具有重要的地位, 广泛应用于各类嵌入式系统和设备中。

13.4 组成部分

嵌入式 Linux SPI 子系统主要由以下几个部分组成:

1、SPI 核心层

SPI 核心层提供了 SPI 子系统的通用接口和基础功能。它负责 SPI 设备和主机控制器的注册、总线管理、数据传输和设备通信等。核心层位于内核源代码的 `drivers/spi/spi.c` 文件中。

2、SPI 主机控制器驱动

SPI 主机控制器驱动负责管理 SPI 总线上的主机控制器。它实现了与具体硬件平台相关的功能, 允许 SPI 核心层通过统一的接口与不同的硬件进行交互。这些驱动程序通常位于 `drivers/spi` 目录中。

3、SPI 设备驱动

SPI 设备驱动负责与具体的 SPI 从设备进行通信。每个 SPI 从设备都有一个相应的设备驱动程序, 这些驱动程序通过 SPI 核心层与主机控制器驱动进行数据传输。设备驱动通常位于 `drivers/spi` 或与设备类型相关的其他目录中。

4、设备树支持

设备树 (Device Tree) 用于描述硬件架构和设备信息, SPI 子系统利用设备树节点来配置和初始化 SPI 主机和从设备。设备树支持使得系统在启动时能够自动识别和配置 SPI 设备, 通常在设备树源文件 (*.dts) 中进行描述。

5、用户空间接口

SPI 子系统提供了用户空间接口, 允许用户应用程序通过标准文件操作与 SPI 设备进行通信。常用的接口文件包括 `/dev/spidevX.Y`, 用户可以使用 `open`、`read`、`write` 和 `ioctl` 等标准文件操作函数与 SPI 设备进行交互。

13.5 MP257 的 SPI 说明

13.5.1 文档和资源

想要获取 MP257 系列处理器的 SPI 详细说明，最直接的方式是参考 ST 官方文档，包括：

1、参考手册（Reference Manual）：提供了详细的寄存器级说明，包括 SPI 模块的寄存器映射以及配置方法等。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、MP257 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南（即本文档）

应用开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图：【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

13.5.2 特性说明

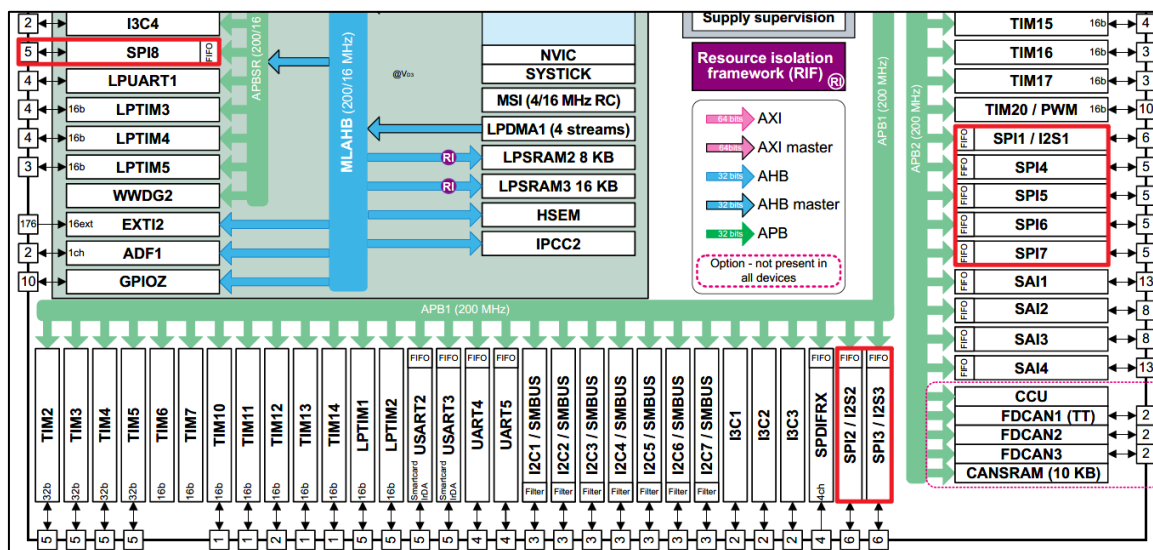


图 13.5-1 MP257 功能框图中的 SPI 部分

MP257 芯片包含八个 SPI（Serial Peripheral Interface）模块实例，分布在不同的功能区域，并具有特定的特性和限制。具体说明需要结合芯片参考手册：

79.3 SPI implementation

The table below describes the SPI implementation. The instances are applied either with a full set or a limited set of features.

Table 792. SPI features

SPI feature	SPI2S1, SPI2S2, SPI2S3 (full feature)	SPI4, SPI5, SPI6, SPI7 (full feature)	SPI8 (limited feature)
Data and CRC size	Configurable from 4 to 32 bits	Configurable from 4 to 16 bits	8/16 bits
CRC computation	CRC polynomial length configurable from 5 to 33 bits	CRC polynomial length configurable from 5 to 17 bits	CRC polynomial length 9/17 bits
Size of FIFOs	16x8 bits	8x8 bits	8x8 bits
Number of data control (TSIZE)	Up to 65536	Up to 65536	Up to 1024, no remaining data counter (CTIZE)
I2S feature	Yes	No	No
Autonomous in Stop modes with wake-up capability	Yes	Yes	Yes
Autonomous in LP-Stop and Standby modes with wake-up capability	No	No	Yes

图 13.5-2 芯片参考手册 SPI 章节节选

这张表格总结了 SPI 实例的功能实现，分为全功能（SPI2S1、SPI2S2、SPI2S3 和 SPI4、SPI5、SPI6、SPI7）和有限功能（SPI8）两类。

SPI2S1 ~ SPI2S3 支持 4 到 32 位的数据与 CRC 大小、可配置 5 到 33 位的 CRC 多项式长度、16x8 位 FIFO、数据控制可达 65536，并具备 I2S 功能和停止模式唤醒，但不支持 LP-Stop 和 Standby 模式唤醒。

SPI4 ~ SPI7 支持 4 到 16 位数据与 CRC 大小、5 到 17 位 CRC 多项式长度、8x8 位 FIFO、数据控制也达 65536, 但不具备 I2S 功能, 支持停止模式唤醒, 不支持 LP-Stop 和 Standby 唤醒。

SPI8 功能受限, 数据与 CRC 固定为 8/16 位、CRC 多项式 9/17 位、8x8 位 FIFO, 数据控制仅达 1024 且无剩余计数器, 支持停止模式和 LP-Stop/Standby 模式唤醒。

更多具体说明请查看芯片参考手册。

13.6 开发板 SPI 设备

13.6.1 原理图分析

打开开发板上【正点原子】STM32MP257 开发板 (A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf, 搜索 SPI。可以看到 SPI 相关电路如下。

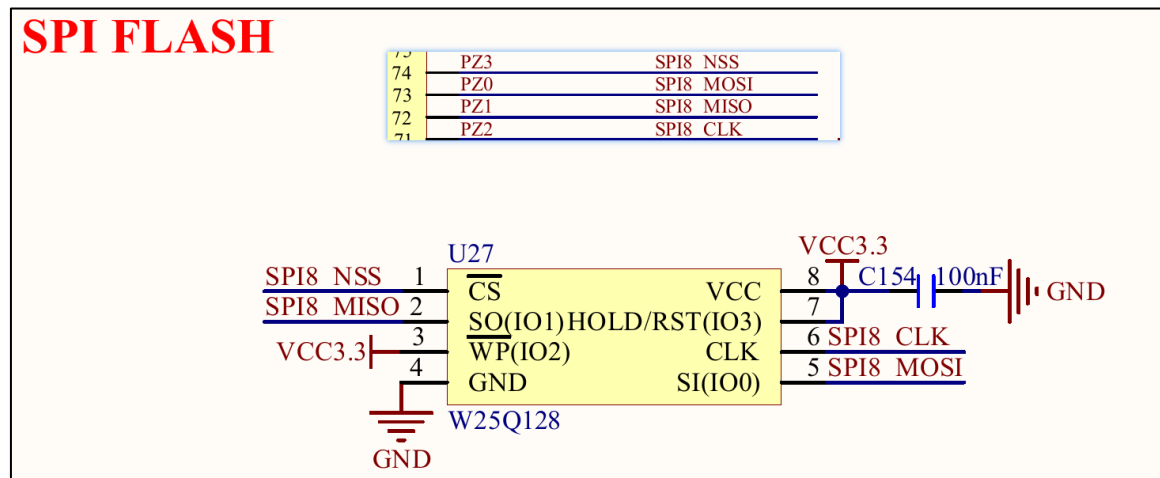


图 13.6-1 开发板 SPI 设备原理图

该原理图展示了一个 SPI Flash 存储器的连接。以下是对该原理图的解析：

主要组件

- W25Q128 SPI Flash 存储器：该芯片是一个串行 Flash 存储器，用于存储数据。
- 连接引脚：
 - ◆ CS (Chip Select): 片选信号，用于选择该 SPI 设备。连接到 SPI8_NSS。
 - ◆ SO (Serial Output, MISO): 数据输出引脚，用于将数据从 Flash 传输到主机。连接到 SPI8_MISO。
 - ◆ SI (Serial Input, MOSI): 数据输入引脚，用于将数据从主机传输到 Flash。连接到 SPI8_MOSI。
 - ◆ WP (Write Protect): 写保护引脚，可以用来保护 Flash 免受写入操作的影响。
 - ◆ HOLD: 持有信号，用于暂停 SPI 通信。连接到 GPIO。
 - ◆ CLK (Clock): 时钟信号，用于同步数据传输。连接到 SPI8_CLK。
 - ◆ VCC: 电源引脚，连接到 3.3V 电源。
 - ◆ GND: 地引脚，连接到地。

电气连接

- 电源连接：
 - ◆ VCC 引脚连接到 3.3V 电源。
 - ◆ GND 引脚连接到地。
- 信号连接：

- ◆ SPI8_MOSI (PZ0)连接到 SI 引脚。
- ◆ SPI8_MISO (PZ1)连接到 SO 引脚。
- ◆ SPI8_CLK (PZ2)连接到 CLK 引脚。
- ◆ SPI8_NSS (PZ3)连接到 CS 引脚。

电容和滤波器

图中 C154 是一个去耦电容，用于稳定电源电压，防止电源噪声对 SPI Flash 的影响。连接在 VCC 和 GND 之间。

13.6.2 实物硬件

ATK-DLMP257B 开发板上 SPI FLASH 位置如下：

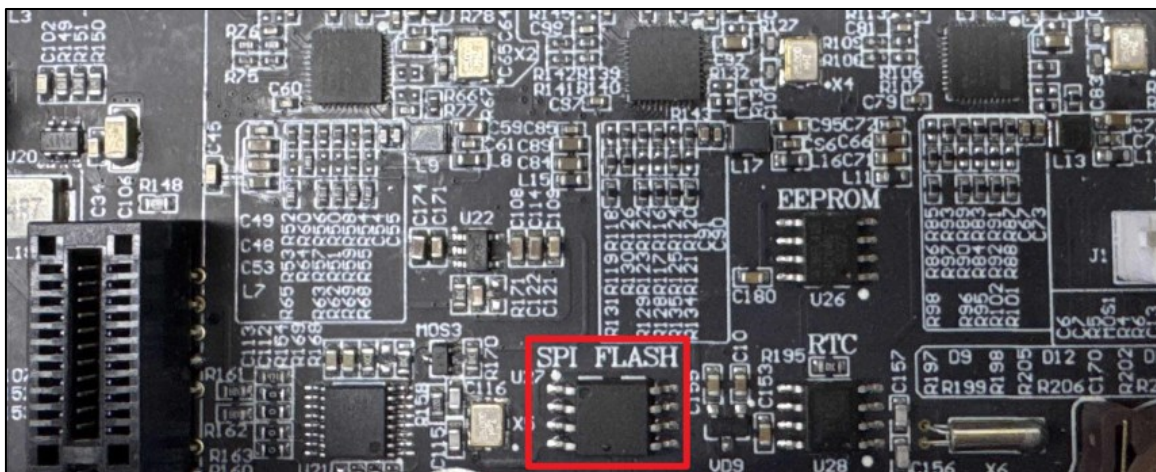


图 13.6-2 开发板 SPI FLASH 实物

13.7 内核 SPI 子系统使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中，SPI 相关配置如下(略有删减，只列举重点部分)：

```
1 CONFIG_SPI=y //启用 SPI 支持
2 CONFIG_SPI_MASTER=y //启用 SPI 主机控制器支持
3 CONFIG_SPI_MEM=y //启用 SPI 存储器接口支持
4 CONFIG_SPI_STM32=m //启用 ST 的 SPI 控制器支持
5 CONFIG_SPI_STM32_OSPI=y //启用 ST 的 Octal SPI 控制器支持
6 CONFIG_SPI_SPIDEV=m //启用用户空间 SPI 设备驱动程序
7 CONFIG_SPI_SLAVE=y //启用 SPI 从设备模式支持
8 CONFIG_SPI_DYNAMIC=y //启用 SPI 动态设备注册支持
```

对应的 menuconfig 配置如下（篇幅有限，截图有所省略）：

```
-> Device Drivers
-> [*] SPI support --->
-> <M> STMicroelectronics STM32 SPI controller
-> <*> STMicroelectronics STM32 OCTO SPI controller
-> <m> User mode SPI device driver support
-> [*] SPI slave protocol handlers
```

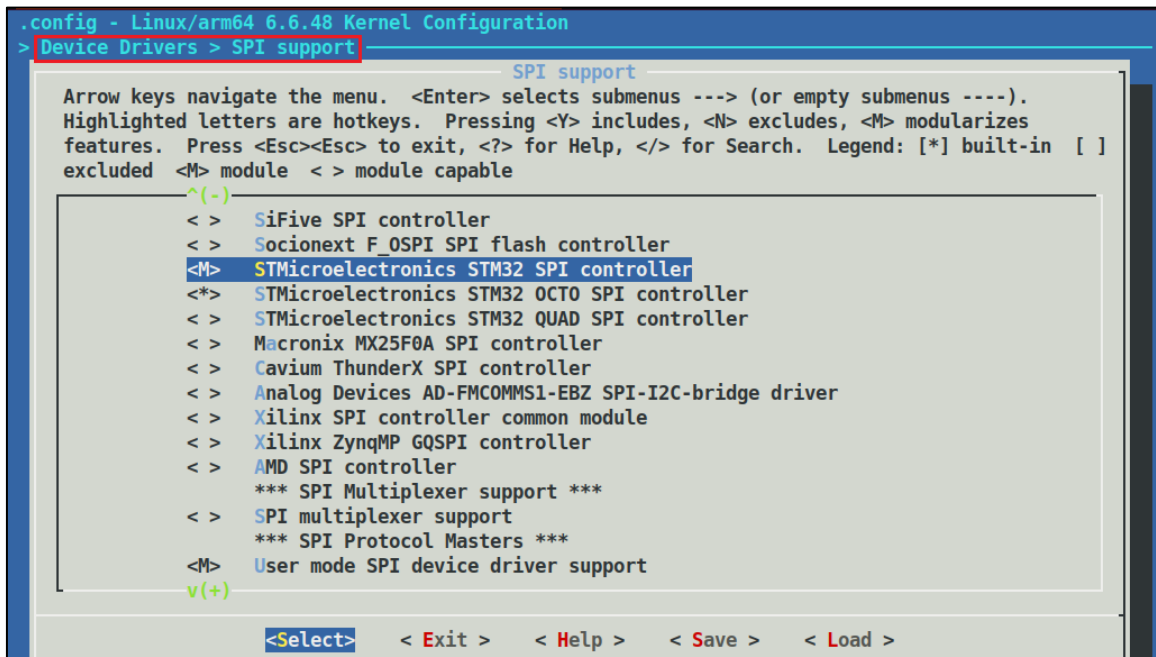


图 13.7-1 SPI 配置

13.8 设备树文件分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi，此文件是特定于 MP257 系列芯片的设备树包含文件，通常不独立使用，而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置，而 .dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 SPI 的描述如下（以 SPI8 为例）

```

1      spi8: spi@46020000 {
2          #address-cells = <1>;
3          #size-cells = <0>;
4          compatible = "st,stm32mp25-spi";
5          reg = <0x46020000 0x400>;
6          interrupts = <GIC_SPI 156 IRQ_TYPE_LEVEL_HIGH>;
7          clocks = <&rcc CK_KER_SPI8>;
8          resets = <&rcc SPI8_R>;
9          dmas = <&hpdma 171 0x20 0x00003012>,
10             <&hpdma 172 0x20 0x00003021>;
11          dma-names = "rx", "tx";
12          access-controllers = <&rifsc 29>;
13          power-domains = <&CLUSTER_PD>;
14          status = "disabled";
15      };

```

第 1 行，spi8: spi@46020000 定义了一个名为 spi8 的 SPI 控制器节点。@46020000 表示该设备的寄存器起始地址为 0x46020000。

第 2 行，#address-cells = <1> 定义子节点的地址单元数量为 1，表示子节点使用一个地址单元描述资源。

第 3 行，#size-cells = <0> 定义子节点的大小单元数量为 0，表示不需要定义资源的大小。

第 4 行, `compatible = "st,stm32mp25-spi"` 说明该设备与 `st,stm32mp25-spi` 兼容, 用于驱动 SPI 控制器的匹配。驱动路径为 `drivers/spi/spi-stm32.c`。

第 5 行, `reg = <0x46020000 0x400>` 定义设备的寄存器空间: 起始地址为 `0x46020000`, 大小为 `0x400` 字节。

第 6 行, `interrupts = <GIC_SPI 156 IRQ_TYPE_LEVEL_HIGH>` 定义中断信息: 使用 GIC (通用中断控制器), 中断号为 156, 类型为 电平触发高电平 (`IRQ_TYPE_LEVEL_HIGH`)。

第 7 行, `clocks = <&rcc CK_KER_SPI8>` 定义该设备使用的时钟资源: 时钟控制来自 `rcc`, 时钟源为 `CK_KER_SPI8`。

第 8 行, `resets = <&rcc SPI8_R>` 定义该设备使用的复位控制: 复位控制来自 `rcc`, 复位线为 `SPI8_R`。

第 9 行, `dmass = <&hpdma 171 0x20 0x00003012>, <&hpdma 172 0x20 0x00003021>` 定义 DMA 通道配置: `&hpdma` 表示 DMA 控制器, 171 和 172 是 RX (接收) 和 TX (发送) 通道号, `0x20` 表示 DMA 请求线, `0x00003012` 和 `0x00003021` 分别为具体的 DMA 配置参数。

第 11 行, `dma-names = "rx", "tx"` 分别为 DMA 通道命名: "rx" 表示接收通道, "tx" 表示发送通道。

第 12 行, `access-controllers = <&rifsc 29>` 定义访问控制器, `rifsc` 资源的索引号为 29。

第 13 行, `power-domains = <&CLUSTER_PD>` 指定该设备的电源域由 `CLUSTER_PD` 管理。

第 14 行, `status = "disabled"` 设备当前状态为 禁用, 表示默认情况下该设备不会启用。

以 2GB 版本核心板为例, 在 `arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts` 中, `spi8` 和 `spiflash` 相关配置如下:

```
1 &spi8 {
2     pinctrl-names = "default", "sleep";
3     pinctrl-0 = <&spi8_pins_a>;
4     pinctrl-1 = <&spi8_sleep_pins_a>;
5     cs-gpios = <&gpioz 3 GPIO_ACTIVE_LOW>;
6     status = "okay";
7
8     spiflash: w25q128@0 {
9         compatible = "jedec,spi-nor";
10        reg = <0>;
11        spi-max-frequency = <50000000>;
12        status = "okay";
13    };
14};
```

第 1 行, `&spi8` 引用之前定义的 `spi8` 节点, 对 `spi8` 设备进行进一步配置。

第 2 行, `pinctrl-names = "default", "sleep"` 定义引脚控制状态的名称: `default` 表示正常工作模式下的引脚配置, `sleep` 表示睡眠模式下的引脚配置。

第 3 行, `pinctrl-0 = <&spi8_pins_a>` 设置 `default` 状态下使用的引脚组为 `spi8_pins_a`。

第 4 行, `pinctrl-1 = <&spi8_sleep_pins_a>` 设置 `sleep` 状态下使用的引脚组为 `spi8_sleep_pins_a`。

第 5 行, `cs-gpios = <&gpioz 3 GPIO_ACTIVE_LOW>` 定义片选信号 (CS) 使用的 GPIO 引脚: 来自 `gpioz` 控制器的第 3 号引脚, 片选信号为 低电平有效 (`GPIO_ACTIVE_LOW`)。

第 6 行, `status = "okay"` 将 spi8 设备的状态设置为 启用, 表示该 SPI 控制器已激活。

第 8 行, `spiflash: w25q128@0` 定义了一个名为 `spiflash` 的子节点, 表示连接到 `spi8` 的外部设备。`w25q128@0` 表示连接的 SPI Flash 芯片的名称, 其中 `@0` 表示该设备的片选地址为 0。

第 9 行, `compatible = "jedec,spi-nor"` 指定该设备兼容标准的 SPI-NOR Flash, 驱动程序将基于 `jedec,spi-nor` 匹配。

第 10 行, `reg = <0>` 指定该设备的片选地址为 0, 对应 `spi8` 控制器的片选信号 `CS0`。

第 11 行, `spi-max-frequency = <50000000>` 定义 SPI 最大频率为 50 MHz, 表示该 Flash 芯片在与 `spi8` 通信时的时钟频率。

第 12 行, `status = "okay"` 将 SPI Flash 设备的状态设置为 启用, 表示该设备可以正常使用。

13.9 SPI 调试工具

13.9.1 spidev_test

`spidev_test` 是一个常用的 SPI 测试工具, 允许用户测试 SPI 通信。用户可以使用它来发送和接收数据, 以确保 SPI 接口和设备工作正常。

```
root@ATK-DLMP257:~# spidev_test --help
spidev_test: unrecognized option '--help'
Usage: spidev_test [-2348CDFHILMNORSZbdilopsv]
general device settings:
  -D --device          device to use (default /dev/spidev1.1)
  -s --speed           max speed (Hz)
  -d --delay           delay (usec)
  -l --loop            loopback
spi mode:
  -H --cpha            clock phase
  -O --cpol            clock polarity
  -F --rx-cpha-flip    flip CPHA on Rx only xfer
number of wires for transmission:
  -2 --dual            dual transfer
  -4 --quad            quad transfer
  -8 --octal           octal transfer
  -3 --3wire           SI/SO signals shared
  -Z --3wire-hiz      high impedance turnaround
data:
  -i --input           input data from a file (e.g. "test.bin")
  -o --output          output data to a file (e.g. "results.bin")
  -p                  Send data (e.g. "1234\xde\xad")
  -S --size            transfer size
  -I --iter            iterations
additional parameters:
  -b --bpw             bits per word
  -L --lsb             least significant bit first
  -C --cs-high         chip select active high
  -N --no-cs           no chip select
  -R --ready           slave pulls low to pause
  -M --mosi-idle-low   leave mosi line low when idle
misc:
  -v --verbose         Verbose (show tx buffer)
```

图 13.9-1 spidev_test 说明

`spidev_test` 提供了多种选项, 用于测试和调试 SPI 设备。以下是每个选项的详细解释和使用示例:

选项解释

- 通用设备设置
 - `-D --device` 指定要使用的 SPI 设备, 默认是 `/dev/spidev1.1`
 - `-s --speed` 设置 SPI 最大传输速度 (以 Hz 为单位)

- -d --delay 每次传输之间的延迟时间（以微秒为单位）
- -l --loop 启用环回模式，将发送数据直接回传到接收端（通常用于测试）
- SPI 模式设置
 - -H --cpha 设置时钟相位（CPHA），用于配置 SPI 模式
 - -O --cpol 设置时钟极性（CPOL），用于配置 SPI 模式
 - -F --rx-cpha-flip 在接收传输时，仅翻转时钟相位（CPHA）
- 数据传输线模式
 - -2 --dual 启用双通道传输（Dual Transfer）
 - -4 --quad 启用四通道传输（Quad Transfer）
 - -8 --octal 启用八通道传输（Octal Transfer）
 - -3 --3wire 启用 3 线模式（SI/SO 信号共享）
 - -Z --3wire-hiz 在 3 线模式下启用高阻抗回转模式
- 数据设置
 - -i --input 从文件（如 test.bin）加载输入数据进行发送
 - -o --output 将接收到的数据保存到文件（如 results.bin）
 - -p 直接发送数据（例如 "1234\xde\xad"），支持十六进制数据
 - -S --size 设置每次传输的数据大小（字节数）
 - -I --iter 设置重复传输的次数
- 附加参数
 - -b --bpw 指定每个数据字的位数（Bits Per Word），如 8 位或 16 位
 - -L --lsb 数据传输中，启用 LSB（最低有效位优先）模式
 - -C --cs-high 启用芯片选择信号（Chip Select, CS）为高电平有效
 - -N --no-cs 禁用芯片选择信号（CS），适用于多从机环境
 - -R --ready 当从机拉低时暂停传输，表示从机未准备好
 - -M --mosi-idle-low 设置空闲状态下 MOSI（主输出从输入）信号为低电平
- 杂项
 - -v --verbose 启用详细模式，显示发送和接收的缓冲区数据（调试用）

13.9.2 spidev_fdx

spidev_fdx 是一个用于与 SPI 设备进行全双工通信的命令行工具。SPI（Serial Peripheral Interface）是一种同步串行通信接口，常用于与传感器、显示屏和存储器等外围设备通信。该工具允许用户与 SPI 设备进行数据传输。

spidev_fdx 的主要功能是：

- 配置 SPI 设备的通信参数，如模式和速度。
- 进行全双工数据传输，即在同一时间发送和接收数据。

```
root@atk-imx93:~# spidev_fdx
usage: spidev_fdx [-h] [-m N] [-r N] /dev/spidevB.D
root@atk-imx93:~#
```

图 13.9-2 spidev_fdx 说明

参数说明：

-h: 显示帮助信息并退出。

-m N: 指定 SPI 模式为整数 N。SPI 模式通常范围是 0 到 3。

-r N: 指定最大速度（Hz）为整数 N。

/dev/spidevB.D: SPI 设备节点，其中 B 和 D 通常是总线和设备编号。

第十四章 MTD 子系统

14.1 MTD 子系统简介

MTD (Memory Technology Device) 子系统是嵌入式 Linux 中的一个重要组件, 主要用于管理和操作闪存等存储设备。MTD 子系统为各种类型的闪存设备 (如 NOR Flash、NAND Flash 等) 提供统一的接口和抽象层, 使得上层应用和文件系统可以不必关心具体的硬件细节。

14.2 主要特点

1、统一接口与多种存储技术支持

MTD 子系统提供了一个统一的接口, 用于操作各种类型的闪存设备 (如 NOR Flash、NAND Flash 等)。这种统一性简化了开发工作, 使应用程序和文件系统能够在不考虑底层硬件差异的情况下访问闪存设备。

2、坏块管理与纠错码 (ECC) 支持

MTD 子系统内置了坏块管理机制, 可以自动检测和标记坏块, 确保数据不会写入不可靠的区域。同时, MTD 子系统支持纠错码 (ECC) 功能, 能够检测和修正存储中的错误位, 提高数据的完整性和可靠性。

3、擦除管理与分区支持

闪存设备的写入操作通常需要先进行擦除, MTD 子系统处理这些擦除操作, 并提供擦除块管理, 保证数据写入的效率和正确性。此外, MTD 子系统允许将闪存设备划分为多个逻辑分区, 每个分区可以单独操作, 方便不同用途 (如引导加载程序、文件系统、应用程序存储等) 的管理和隔离。

4、文件系统与用户空间工具支持

MTD 子系统与多种专为闪存设计的文件系统 (如 JFFS2、YAFFS、UBIFS 等) 兼容, 并提供一系列用户空间工具 (如 mtd-utils 工具包) 帮助用户进行闪存设备的操作和管理。这些工具和文件系统利用 MTD 子系统提供的功能, 实现高效和可靠的存储管理。

14.3 组成部分

嵌入式 Linux 的 MTD 子系统由多个组成部分构成, 每个部分负责不同的功能和任务。以下是 MTD 子系统的主要组成部分:

1、MTD 核心

MTD 核心是 MTD 子系统的核心部分, 负责管理所有的 MTD 设备并提供通用接口。这些接口用于读写、擦除和其他操作, 确保各种闪存设备能够通过统一的方式进行访问和管理。

2、MTD 设备驱动

MTD 设备驱动程序实现了具体的硬件接口。不同类型的闪存设备 (如 NOR Flash、NAND Flash 等) 都有各自的驱动程序, 这些驱动负责处理具体的硬件读写和擦除操作, 并将这些操作抽象为 MTD 核心可以理解的标准接口。

3、MTD 设备文件

MTD 设备文件通常在系统的 /dev 目录下创建设备节点, 例如 /dev/mtd0、/dev/mtd1 等。用户和应用程序可以通过这些设备文件直接访问和操作对应的闪存设备。

4、MTD 用户空间工具

MTD 子系统提供了一些用户空间工具, 这些工具帮助用户进行闪存设备的操作和管理。常见的工具包括:

flashcp: 用于将文件复制到闪存设备。

flash_erase: 用于擦除闪存设备中的特定区域。

nanddump: 用于从 NAND 闪存设备中读取数据。

5、MTD 映射和分区

MTD 子系统支持将闪存设备划分为多个逻辑分区。每个分区可以独立操作，适用于不同的用途（如引导加载程序、文件系统、应用程序存储等）。分区机制提高了存储管理的灵活性和效率。

6、文件系统支持

MTD 子系统与多种专为闪存设计的文件系统兼容，例如：

JFFS2（Journaling Flash File System v2）

YAFFS（Yet Another Flash File System）

UBIFS（Unsorted Block Image File System）

这些文件系统利用 MTD 子系统提供的功能，实现高效和可靠的存储管理。

嵌入式 Linux 的 MTD 子系统通过 MTD 核心、设备驱动、设备文件、用户空间工具、映射和分区以及文件系统支持等组成部分，提供了统一、高效、可靠的闪存设备管理和操作功能。这些组成部分共同协作，使得上层应用和文件系统能够方便地使用各种类型的闪存设备。

14.4 开发板 MTD 设备

14.4.1 原理图分析

开发板上有个 NOR Flash，型号 W25Q128。打开开发板上【正点原子】STM32MP257 开发板(A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 W25Q128。可以看到 W25Q128 相关电路如下。

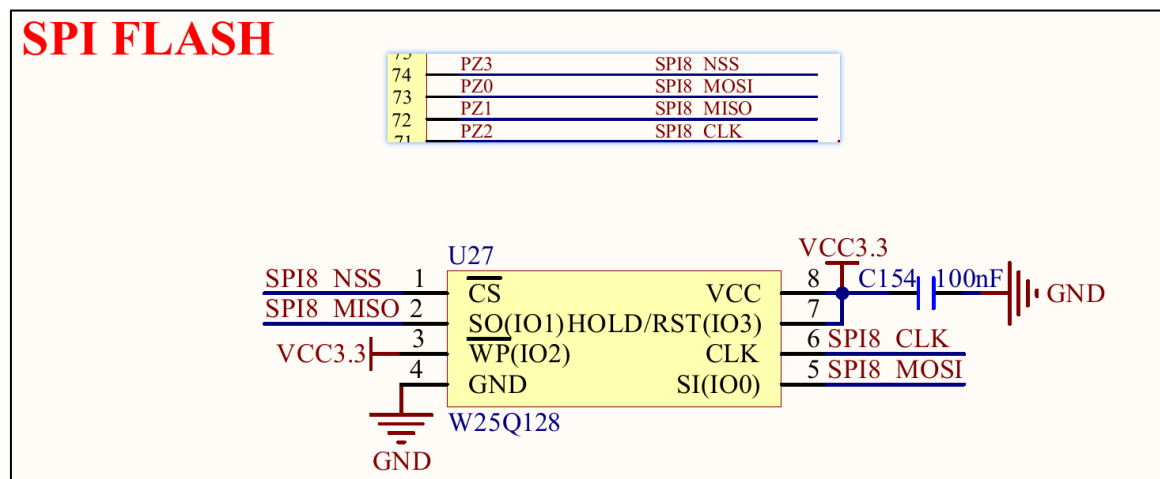


图 14.4-1 开发板 W25Q128 原理图

该原理图展示了一个 SPI Flash 存储器的连接。以下是对该原理图的解析：

主要组件

- W25Q128 SPI Flash 存储器：该芯片是一个串行 Flash 存储器，用于存储数据。
- 连接引脚：
 - ◆ CS (Chip Select): 片选信号，用于选择该 SPI 设备。连接到 SPI8_NSS。

- ◆ SO (Serial Output, MISO): 数据输出引脚, 用于将数据从 Flash 传输到主机。连接到 SPI8_MISO。
- ◆ SI (Serial Input, MOSI): 数据输入引脚, 用于将数据从主机传输到 Flash。连接到 SPI8_MOSI。
- ◆ WP (Write Protect): 写保护引脚, 可以用来保护 Flash 免受写入操作的影响。
- ◆ HOLD: 持有信号, 用于暂停 SPI 通信。连接到 GPIO。
- ◆ CLK (Clock): 时钟信号, 用于同步数据传输。连接到 SPI8_CLK。
- ◆ VCC: 电源引脚, 连接到 3.3V 电源。
- ◆ GND: 地引脚, 连接到地。

电气连接

- 电源连接:
 - ◆ VCC 引脚连接到 3.3V 电源。
 - ◆ GND 引脚连接到地。
- 信号连接:
 - ◆ SPI8_MOSI (PZ0)连接到 SI 引脚。
 - ◆ SPI8_MISO (PZ1)连接到 SO 引脚。
 - ◆ SPI8_CLK (PZ2)连接到 CLK 引脚。
 - ◆ SPI8_NSS (PZ3)连接到 CS 引脚。

电容和滤波器

图中 C154 是一个去耦电容, 用于稳定电源电压, 防止电源噪声对 SPI Flash 的影响。连接在 VCC 和 GND 之间。

14.4.2 实物硬件

ATK-DLMP257B 开发板上 SPI FLASH 位置如下:

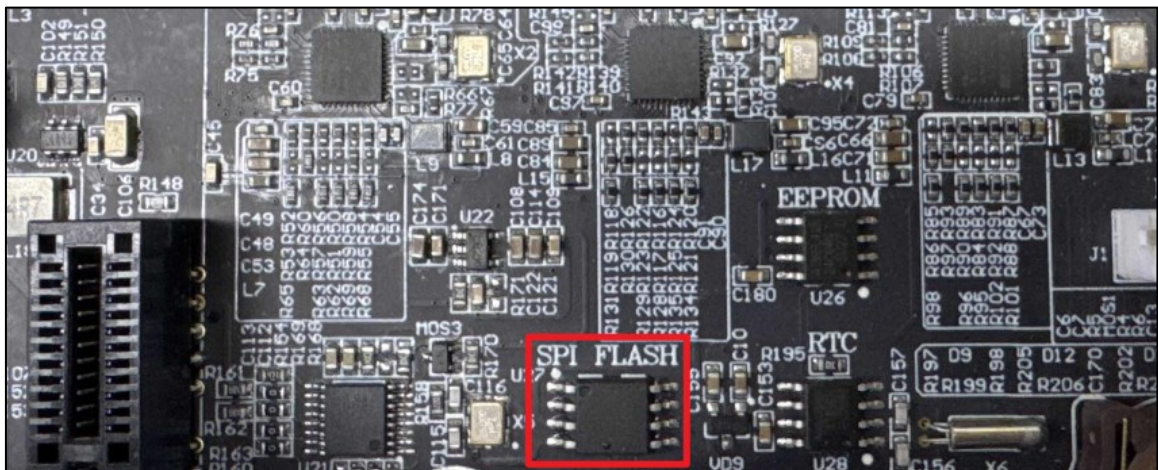


图 14.4-2 开发板 W25Q128 实物

W25Q128 是 Winbond 公司生产的一种高性能的 SPINOR 闪存芯片, 常用于嵌入式系统中存储固件、配置数据等, 具有以下主要特性:

- 容量: 128Mb (16MB)
- 接口: 标准 SPI 接口, 支持双通道 (Dual I/O) 和四通道 (Quad I/O) 模式
- 速度: 最高支持 104MHz 的 SPI 时钟频率
- 擦除块: 4KB (Sector)、32KB、64KB 和全芯片擦除

- 低功耗: 支持低功耗模式
- 存储寿命: 100,000 次擦写周期, 数据保持 20 年

14.5 SPI FLASH 驱动配置

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, SPI NOR FLASH(型号 W25Q128)相关配置如下:

```
1 CONFIG_MTD_SPI_NOR=y //启用对 SPI NOR 闪存的支持
2 CONFIG_MTD_SPI_NOR_SWP_DISABLE_ON_VOLATILE=y /*启用在易失性情况下
                                           禁用 SPI NOR 写保护功能*/
```

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

```
-> Device Drivers
-> <*> Memory Technology Device (MTD) support --->
    -> <*> SPI NOR device support --->
        -> Software write protection at boot (Disable SWP on flashes w/ volatile
protection bits)
```

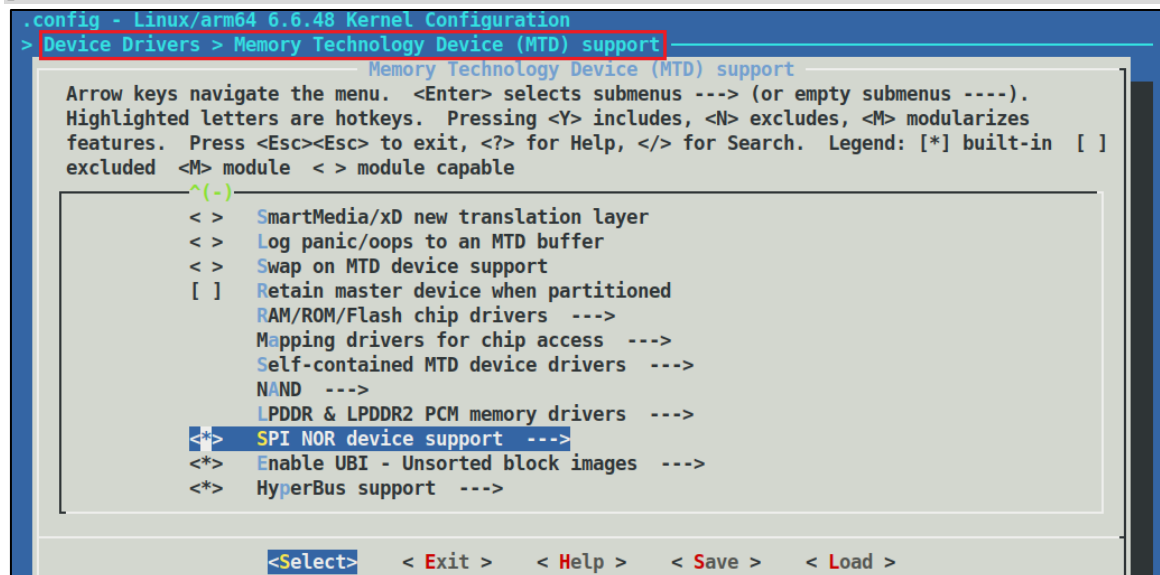


图 14.5-1 SPI NOR 设备配置

选择配置后, 按下键盘的 “?” 键可以查看具体配置说明:

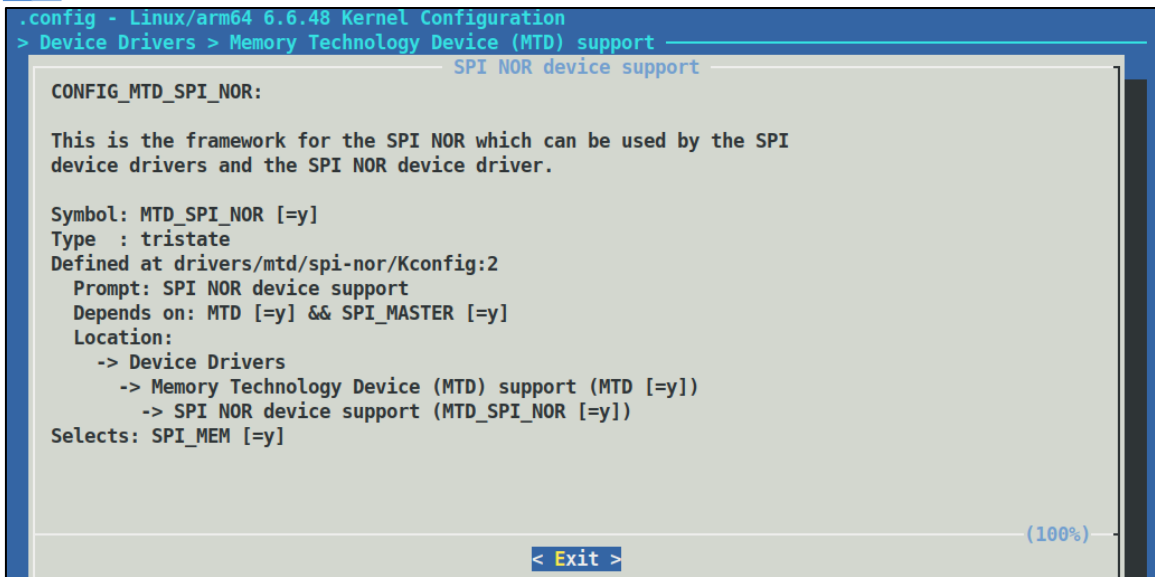


图 14.5-2 配置说明

配置项 CONFIG_MTD_SPI_NOR 说明:

框架: CONFIG_MTD_SPI_NOR 是为 SPINOR 闪存设备提供的框架, 供 SPI 设备驱动程序和 SPI NOR 设备驱动程序使用。

类型: 这是一个三态选项, 意味着它可以设置为 y (编译进内核)、m (作为模块编译) 或 n (不编译)。

位置: 该选项在内核配置菜单中的位置明确显示了它是存储技术设备 (MTD) 支持的一部分。

依赖关系:

MTD (Memory Technology Device): 必须启用, 以支持内存技术设备。

SPI_MASTER: 必须启用, 以支持 SPI 主设备。

选择项:

SPI_MEM: 该选项选择 SPI_MEM, 这意味着启用 CONFIG_MTD_SPI_NOR 也会启用 SPI 存储器接口支持。

功能与应用

主要功能: 该选项启用对 SPINOR 闪存设备的支持。这些设备常用于存储固件、配置数据和其他需要持久化存储的数据。

驱动支持: 启用该选项后, 内核将包含对各种 SPINOR 闪存设备的驱动支持, 这些驱动程序可以与具体的硬件 SPI 控制器驱动程序一起工作, 提供与 SPI NOR 闪存设备的接口。

系统集成: 对于嵌入式系统, 启用 SPI NOR 支持非常重要, 因为许多嵌入式平台使用 SPI NOR 闪存作为主要的存储介质。

14.6 设备树文件分析

以 2GB 版本核心板为例, 在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 中, spi8 和 spiflash 相关配置如下:

```
1 &spi8 {
2     pinctrl-names = "default", "sleep";
3     pinctrl-0 = <&spi8_pins_a>;
4     pinctrl-1 = <&spi8_sleep_pins_a>;
```



```

5  cs-gpios = <&gpioz 3 GPIO_ACTIVE_LOW>;
6  status = "okay";
7
8  spiflash: w25q128@0 {
9      compatible = "jedec,spi-nor";
10     reg = <0>;
11     spi-max-frequency = <50000000>;
12     status = "okay";
13 };
14};

```

第 1 行, `&spi8` 引用之前定义的 `spi8` 节点, 对 `spi8` 设备进行进一步配置。

第 2 行, `pinctrl-names = "default", "sleep"` 定义引脚控制状态的名称: `default` 表示正常工作模式下的引脚配置, `sleep` 表示睡眠模式下的引脚配置。

第 3 行, `pinctrl-0 = <&spi8_pins_a>` 设置 `default` 状态下使用的引脚组为 `spi8_pins_a`。

第 4 行, `pinctrl-1 = <&spi8_sleep_pins_a>` 设置 `sleep` 状态下使用的引脚组为 `spi8_sleep_pins_a`。

第 5 行, `cs-gpios = <&gpioz 3 GPIO_ACTIVE_LOW>` 定义片选信号 (CS) 使用的 GPIO 引脚: 来自 `gpioz` 控制器的第 3 号引脚, 片选信号为 低电平有效 (`GPIO_ACTIVE_LOW`)。

第 6 行, `status = "okay"` 将 `spi8` 设备的状态设置为 启用, 表示该 SPI 控制器已激活。

第 8 行, `spiflash: w25q128@0` 定义了一个名为 `spiflash` 的子节点, 表示连接到 `spi8` 的外部设备。 `w25q128@0` 表示连接的 SPI Flash 芯片的名称, 其中 `@0` 表示该设备的片选地址为 0。

第 9 行, `compatible = "jedec,spi-nor"` 指定该设备兼容标准的 SPI-NOR Flash, 驱动程序将基于 `jedec,spi-nor` 匹配。驱动路径为 `drivers/mtd/spi-nor/core.c`。

第 10 行, `reg = <0>` 指定该设备的片选地址为 0, 对应 `spi8` 控制器的片选信号 `CS0`。

第 11 行, `spi-max-frequency = <50000000>` 定义 SPI 最大频率为 50 MHz, 表示该 Flash 芯片在与 `spi8` 通信时的时钟频率。

第 12 行, `status = "okay"` 将 SPI Flash 设备的状态设置为 启用, 表示该设备可以正常使用。

14.7 MTD 设备作为块设备使用

ATK-DLMP257B 开发板板载 SPINOR FLASH 芯片, 型号为 W25Q128, 它通过 SPI 总线进行读写访问, 具备 16MB 存储空间。

内核启动时会自动加载 W25Q128 驱动, 芯片正常工作时, 内核会驱动加载信息。出厂系统将 SPI Flash 注册成 MTD 设备, 因此我们可以通过 MTD 子系统对其进行文件读写操作。

```
ls /dev/mtd*
```

```

root@ATK-DLMP257:~# ls /dev/mtd*
/dev/mtd0 /dev/mtd0ro /dev/mtdblock0

/dev/mtd:
by-name

```

图 14.7-1 查看 mtd 设备

上面已经列出了系统中的 MTD 设备文件, 包括: `/dev/mtd0`、`/dev/mtd0ro`、`/dev/mtdblock0` 下面是这些设备文件的简要说明及其用途:

/dev/mtd0: 这是一个字符设备文件, 用于直接读写 MTD 设备。适用于需要进行低级别操作的情况。

/dev/mtd0ro: 这是一个只读的字符设备文件, 用于只读访问 MTD 设备。适用于需要从 MTD 设备读取数据但不进行写入操作的情况。

/dev/mtdblock0: 这是一个块设备文件, 允许块级别的读写操作。适用于需要将 MTD 设备作为块设备使用的情况, 例如挂载文件系统。

文件读写测试步骤如下:

0、首次使用, 需要格式化一遍/dev/mtdblock0 分区。注意, 如果不是首次使用, 请不用执行格式化指令, 否则之前执行过的写入文件数据将被清除, 需要重新写入;

```
mkfs.vfat /dev/mtdblock0
```

1、创建自定义挂载目录, 并将/dev/mtdblock0 分区以 VFAT 格式挂载到此目录;

```
mkdir /home/root/w25q128
```

```
mount -t vfat /dev/mtdblock0 /home/root/w25q128
```

2、进入挂载目录, 进行写文件测试;

```
cd /home/root/w25q128
```

```
echo "W25Q128 TEST!" > file.txt
```

```
sync
```

3、读文件, 确认是否写入成功;

```
cat file.txt
```

4、最后退出挂载目录, 并取消/dev/mtdblock0 分区挂载;

```
cd ../
```

```
umount /dev/mtdblock0
```

此时, 写入的文件数据已经存储在 SPIFLASH 芯片中, 可通过再次挂载目录或者重启板子后重新挂载, 来进行文件的读写。

```
root@ATK-DLMP257:~# mkfs.vfat /dev/mtdblock0
mkfs.fat 4.2 (2021-01-31)
root@ATK-DLMP257:~# mkdir /home/root/w25q128
root@ATK-DLMP257:~# mount -t vfat /dev/mtdblock0 /home/root/w25q128
root@ATK-DLMP257:~# cd /home/root/w25q128
root@ATK-DLMP257:~/w25q128# echo "W25Q128 TEST!" > file.txt
root@ATK-DLMP257:~/w25q128# sync
root@ATK-DLMP257:~/w25q128# cat file.txt
W25Q128 TEST!
root@ATK-DLMP257:~/w25q128# cd ../
root@ATK-DLMP257:~# umount /dev/mtdblock0
root@ATK-DLMP257:~#
```

图 14.7-2 SPI FLASH 读写文件测试过程

14.8 MTD 调试工具

14.8.1 mtdinfo

mtdinfo 用于显示 MTD 设备的详细信息, 包括设备大小、分区、擦除块大小等。这个工具对于了解 MTD 设备的具体配置和状态非常有用。

执行 mtdinfo 指令:

```
mtdinfo
```

出厂系统打印如下:

```
root@ATK-DLMP257:~# mtdinfo
Count of MTD devices:      1
Present MTD devices:      mtd0
Sysfs interface supported: yes
root@ATK-DLMP257:~#
```

图 14.8-1 mtdinfo 命令

从 mtdinfo 的输出可以看出，系统中存在一个 MTD 设备（mtd0），并且支持 Sysfs 接口。接下来，我们可以使用一些具体的 mtd_debug 命令来进一步操作和管理这个 MTD 设备。

14.8.2 mtd_debug

mtd_debug 是一个用于调试 MTD 设备的工具。它提供了一些低级操作功能，例如直接读取和写入 MTD 设备的特定区域。这个工具对于开发和调试 MTD 驱动程序非常有用。

执行 mtd_debug 命令查看使用说明：

mtd_debug

出厂系统打印如下：

```
root@ATK-DLMP257:~# mtd_debug
usage: mtd_debug info <device>
       mtd_debug read <device> <offset> <len> <dest-filename>
       mtd_debug write <device> <offset> <len> <source-filename>
       mtd_debug erase <device> <offset> <len>
root@ATK-DLMP257:~#
```

图 14.8-2 mtd_debug 使用说明

根据打印信息可以知道 mtd_debug 命令的使用方法：

- **mtd_debug info <device>**：查看指定 MTD 设备的信息。
- **mtd_debug read <device> <offset> <len> <dest-filename>**：从指定 MTD 设备的特定偏移位置读取数据，并将其保存到指定的文件中。
 - ◆ <device>：MTD 设备文件（例如/dev/mtd0）。
 - ◆ <offset>：从设备的哪个位置开始读取（以字节为单位）。
 - ◆ <len>：读取的字节数。
 - ◆ <dest-filename>：保存读取数据的目标文件名。
- **mtd_debug write <device> <offset> <len> <source-filename>**：将数据从指定的文件写入到 MTD 设备的特定偏移位置。
 - ◆ <device>：MTD 设备文件（例如/dev/mtd0）。
 - ◆ <offset>：从设备的哪个位置开始写入（以字节为单位）。
 - ◆ <len>：写入的字节数。
 - ◆ <source-filename>：包含要写入数据的源文件名。
- **mtd_debug erase <device> <offset> <len>**：擦除 MTD 设备的特定区域。
 - ◆ <device>：MTD 设备文件（例如/dev/mtd0）。
 - ◆ <offset>：从设备的哪个位置开始擦除（以字节为单位）。
 - ◆ <len>：擦除的字节数。

示例：

1、使用 mtd_debug 获取 mtd 设备信息

由前面 mtdinfo 信息我们可以知道 mtd 设备名为/dev/mtd0。

mtd_debug info /dev/mtd0

出厂系统打印如下：

```
root@ATK-DLMP257:~# mtd_debug info /dev/mtd0
mtd.type = MTD_NORFLASH
mtd.flags = MTD_CAP_NORFLASH
mtd.size = 16777216 (16M)
mtd.erasesize = 65536 (64K)
mtd.writesize = 1
mtd.oobsize = 0
regions = 0
root@ATK-DLMP257:~#
```

图 14.8-3 mtd_debug 命令

根据 mtd_debug info /dev/mtd0 的输出，我们可以看到/dev/mtd0 设备的详细信息：

- 类型：MTD_NORFLASH
- 标志：MTD_CAP_NORFLASH
- 大小：16777216 字节（16M）
- 擦除大小：65536 字节（64K）
- 写入大小：1 字节
- OOB 大小：0 字节
- 区域：0

这些信息表明该 MTD 设备是一个 NOR 闪存设备，大小为 16MB，擦除块大小为 64KB，写入粒度为 1 字节，并且没有 OOB（Out-of-Band）数据。

2、擦除 MTD 设备的目标区域

```
mtd_debug erase /dev/mtd0 0 65536
```

```
root@ATK-DLMP257:~# mtd_debug erase /dev/mtd0 0 65536
Erased 65536 bytes from address 0x00000000 in flash
root@ATK-DLMP257:~#
```

图 14.8-4 擦除目标区域数据

3、创建数据，将数据写入 MTD 设备

笔者这里将创建一个 source.bin 文件用于存放数据，这样的好处是便于管理、验证和处理复杂数据。

```
echo "Hello, MTD!" > source.bin
mtd_debug write /dev/mtd0 0 12 source.bin
```

```
root@ATK-DLMP257:~# echo "Hello, MTD!" > source.bin
root@ATK-DLMP257:~# ls
README-CHECK-GPU  source.bin
root@ATK-DLMP257:~# mtd_debug write /dev/mtd0 0 12 source.bin
Copied 12 bytes from source.bin to address 0x00000000 in flash
root@ATK-DLMP257:~#
```

图 14.8-5 使用 mtd_debug write 写入数据

如果不想用文件的形式写入，简单的数据可以直接写入：

```
echo -n "Hello, MTD!" > /dev/mtd0
```

4、从 MTD 设备读取数据并验证

笔者这里将读出的数据保存为 read_back.bin 文件，方便验证和取出数据。

```
mtd_debug read /dev/mtd0 0 12 read_back.bin
cat read_back.bin
```

```
root@ATK-DLMP257:~# mtd_debug read /dev/mtd0 0 12 read_back.bin
Copied 12 bytes from address 0x00000000 in flash to read_back.bin
root@ATK-DLMP257:~# cat read_back.bin
Hello, MTD!
root@ATK-DLMP257:~#
```

图 14.8-6 mtd_debug read 读取数据

如果是使用直接写入的方法, 由于直接写入数据后无法立即通过 mtd_debug read 命令读取, 我们需要使用 dd 指令从设备中读取数据, 笔者将数据保存为 dd_read_back.bin:

```
dd if=/dev/mtd0 bs=1 count=12 of=dd_read_back.bin
cat dd_read_back.bin
```

```
root@ATK-DLMP257:~# dd if=/dev/mtd0 bs=1 count=12 of=dd_read_back.bin
ck.bin
12+0 records in
12+0 records out
12 bytes copied, 0.000822875 s, 14.6 kB/s
root@ATK-DLMP257:~# cat dd_read_back.bin
Hello, MTD!
root@ATK-DLMP257:~#
```

图 14.8-7 使用 dd 指令读取数据

14.8.3 mtdpart

mtdpart 用于显示和管理 MTD 设备的分区信息。它可以显示当前 MTD 设备的分区布局, 并且在某些实现中可能允许对分区进行动态调整。

出厂系统 mtdpart 使用说明如下:

```
root@ATK-DLMP257:~# mtdpart
Usage: mtdpart add [OPTION] <MTD_DEVICE> <PART_NAME> <START> <SIZE>
       mtdpart del [OPTION] <MTD_DEVICE> <PART_NUMBER>
Adds a partition to an MTD device, or remove an existing partition from it.

-h, --help      Display this help and exit
-V, --version    Output version information and exit

<MTD_DEVICE>    MTD device node or 'mtd:<name>'

START location and SIZE of the partition are in bytes. They should align on
eraseblock size. If SIZE is 0 the partition will go to end of MTD device.
root@ATK-DLMP257:~#
```

图 14.8-8 mtdpart 使用说明

根据 mtdpart 工具的使用说明, 我们可以添加或删除 MTD 设备的分区。下面是如何使用这些命令的详细步骤。

1、添加分区

命令格式:

```
mtdpart add [OPTION] <MTD_DEVICE> <PART_NAME> <START> <SIZE>
```

参数说明:

- <MTD_DEVICE>: MTD 设备文件, 例如/dev/mtd0。
- <PART_NAME>: 要添加的分区名称。
- <START>: 分区的起始位置 (以字节为单位), 应对齐到擦除块大小。
- <SIZE>: 分区的大小 (以字节为单位), 也应对齐到擦除块大小。

示例:



通过 `cat /proc/mtd` 指令可以查询 MTD 设备的现有分区, 假设我们要在 `/dev/mtd0` 设备上从起始位置 0 添加一个大小为 1MB 的分区, 分区名称为 `part1`, 可以使用以下命令:

```
cat /proc/mtd
mtdpart add /dev/mtd0 part1 0 1048576
cat /proc/mtd
```

```
root@ATK-DLMP257:~# cat /proc/mtd
td0 part1 0 1048576
cat /proc/mtd
dev:   size   erasesize  name
mtd0: 01000000 00010000 "spi0.0"
root@ATK-DLMP257:~#[92086.772615] 0x00000000000000-0x00000001000000 : "part1"
mtdpart add /dev/mtd0 part1 0 1048576
root@ATK-DLMP257:~# cat /proc/mtd
dev:   size   erasesize  name
mtd0: 01000000 00010000 "spi0.0"
mtd1: 00100000 00010000 "part1"
root@ATK-DLMP257:~#
```

图 14.8-9 使用 `mtdpart add` 添加分区

可以看到新增了一个分区 `/dev/mtd1`。

根据系统提供的 `/proc/mtd` 的输出, 可以看到系统中存在两个 MTD 设备或分区:

- `mtd0`: 表示整个闪存设备 `spi0.0`, 大小为 16MB (01000000 表示 16MB, 单位是字节), 擦除块大小为 64KB (00010000 表示 64KB, 单位是字节)。
- `mtd1`: 表示一个名为 `part1` 的分区, 大小为 1MB (00100000 表示 1MB, 单位是字节), 擦除块大小为 64KB (00010000 表示 64KB, 单位是字节)。

可以使用 `mtdinfo`、`mtd_debug` 等吗, `mtd` 相关指令操作 `/dev/mtd1` 分区。

2、删除分区

命令格式:

```
mtdpart del [OPTION] <MTD_DEVICE> <PART_NUMBER>
```

参数说明:

- `<MTD_DEVICE>`: MTD 设备文件, 例如 `/dev/mtd0`。
- `<PART_NUMBER>`: 要删除的分区号。

示例:

假设我们要删除 `/dev/mtd0` 设备上的第一个分区, 可以使用以下命令:

```
mtdpart del /dev/mtd0 1
cat /proc/mtd
```

```
root@ATK-DLMP257:~# mtdpart del /dev/mtd0 1
c/mtd
root@ATK-DLMP257:~# cat /proc/mtd
dev:   size   erasesize  name
mtd0: 01000000 00010000 "spi0.0"
root@ATK-DLMP257:~#
```

图 14.8-10 使用 `mtdpart del` 删除分区

14.8.4 Sysfs 接口

MTD 子系统支持通过 Sysfs 接口查看和管理 MTD 设备。可以通过 `/sys/class/mtd` 目录查看 MTD 设备的详细信息, 如大小、擦除块大小、分区布局等。

```
cd /sys/class/mtd/mtd0
ls
```

```

root@ATK-DLMP257:~# cd /sys/class/mtd/mtd0
root@ATK-DLMP257:/sys/class/mtd/mtd0# ls
bad_blocks      device          flags           of_node        subpagesize
bbt_blocks      ecc_failures    mtd0            oobavail       subsystem
bitflip_threshold ecc_step_size  mtdblock0       oobsize        type
corrected_bits  ecc_strength    name            power           uevent
dev             erasesize       numeraseregions size            writesize
root@ATK-DLMP257:/sys/class/mtd/mtd0#

```

图 14.8-11 查看/sys/class/mtd/mtd0

在 /sys/class/mtd/mtd0 目录中，可以看到许多文件和目录，它们提供了有关 MTD 设备的信息和状态。以下是每个文件和目录的简要说明：

关键文件和目录

文件和目录	说明
bad_blocks	列出设备中的坏块
bbt_blocks	表示坏块表（Bad Block Table）中的块
bitflip_threshold	位翻转阈值，即允许的最大位翻转数
corrected_bits	显示设备上已纠正的位错误数
dev	显示 MTD 设备的主设备号和次设备号
device	设备的符号链接，指向具体的设备节点
ecc_failures	ECC（纠错码）失败次数
ecc_step_size	ECC 步长大小
ecc_strength	ECC 的纠错能力（可以纠正的最大错误位数）
erasesize	擦除块大小，以字节为单位
flags	MTD 设备的标志，表示设备的属性
mtd0	MTD 设备的符号链接，指向设备节点
mtdblock0	MTD 设备的块设备文件
name	设备名称
numeraseregions	设备的擦除区域数
of_node	指向设备树节点的符号链接
oobavail	可用的 OOB（Out of Band）数据空间大小
oobsize	OOB 数据空间的总大小
power	电源管理相关文件
size	MTD 设备的总大小，以字节为单位
subpagesize	子页面大小
subsystem	符号链接，指向设备所属的子系统
type	MTD 设备的类型，例如 NAND 或 NOR
uevent	用于用户空间事件通知
writesize	写入操作的粒度大小，以字节为单位

使用 cat 指令可以读取对应文件的内容。

第十五章 RTC 子系统

15.1 RTC 子系统简介

嵌入式 Linux 的 RTC (Real-Time Clock) 子系统用于管理和操作实时时钟设备。实时时钟是一个独立于系统主时钟的计时器, 通常使用电池供电, 以确保在系统断电时仍能保持准确的时间。

15.2 主要特点

1、独立时间保持

RTC 设备通常由电池供电, 能够在系统断电的情况下独立保持当前时间。这保证了系统在重新启动后能够恢复准确的时间, 从而避免了时间丢失的问题。

2、时间设置和读取

RTC 子系统提供了简便的接口, 用于设置和读取 RTC 设备的时间。这些接口可以通过命令行工具 (如 `hwclock`) 或程序调用来使用, 方便系统在启动时从 RTC 设备读取时间并同步系统时钟, 确保系统时间的准确性。

3、闹钟和唤醒功能

RTC 设备通常支持闹钟功能, 可以设置一个特定的时间点, 当到达这个时间点时会产生中断。这在嵌入式系统中非常有用, 常用于实现定时任务和唤醒功能。例如, 可以设置系统在特定时间自动唤醒执行任务, 然后再进入低功耗模式。

4、周期性中断

RTC 设备可以配置产生周期性中断, 用于定时操作或定期唤醒系统。这种功能在需要定期执行某些操作的场景中非常有用, 如定期数据采集、状态检查等。

15.3 组成部分

嵌入式 Linux 的 RTC (Real-Time Clock) 子系统由多个组成部分构成, 每个部分负责不同的功能和任务, 以确保系统能够准确、可靠地管理和操作 RTC 设备。以下是 RTC 子系统的主要组成部分:

1、RTC 硬件设备

RTC 硬件设备是实际的实时时钟硬件, 通常由电池供电, 以确保在系统断电的情况下仍能保持时间。常见的 RTC 芯片包括 PCF8563、AT8563T 等。

2、RTC 驱动程序

RTC 驱动程序是内核模块, 负责与 RTC 硬件设备通信。驱动程序实现了对 RTC 设备的控制和管理, 包括时间读取、时间设置、闹钟设置和周期性中断配置等功能。

3、RTC 类设备文件

RTC 类设备文件是用户空间访问 RTC 设备的接口, 通常位于 `/dev` 目录下。常见的 RTC 设备文件包括:

`/dev/rtc`: 默认的 RTC 设备文件。

`/dev/rtc0`: 第一个 RTC 设备。

`/dev/rtc1`: 第二个 RTC 设备, 以此类推。

这些设备文件允许用户空间应用程序通过标准的文件 I/O 操作访问 RTC 设备。

4、RTC 子系统的内核接口

RTC 子系统的内核接口提供了一组标准的系统调用和内核 API，用于 RTC 设备的管理和操作。主要包括以下几个方面：

rtc_class: 内核中的 RTC 类，用于管理所有 RTC 设备。

rtc_device: 表示具体的 RTC 设备实例。

rtc_timer: 用于处理 RTC 定时器和闹钟。

rtc_task: 用于处理 RTC 相关的任务。

5、RTC 用户空间工具

RTC 用户空间工具提供了命令行接口，用于管理和操作 RTC 设备。常用的工具包括：

hwclock: 用于读取和设置 RTC 设备的时间，同步系统时间和 RTC 时间。

rtcwake: 用于设置 RTC 闹钟，以在指定时间唤醒系统。

6、Sysfs 接口

Sysfs 接口提供了一种通过文件系统访问 RTC 设备属性和状态的方法。通常在 /sys/class/rtc/rtcX 目录下可以找到 RTC 设备的相关信息和配置项。

15.4 开发板 RTC 设备

15.4.1 原理图分析

开发板上有个 RTC 芯片，型号 AT8563T。打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 8563。可以看到 AT8563T 相关电路如下。

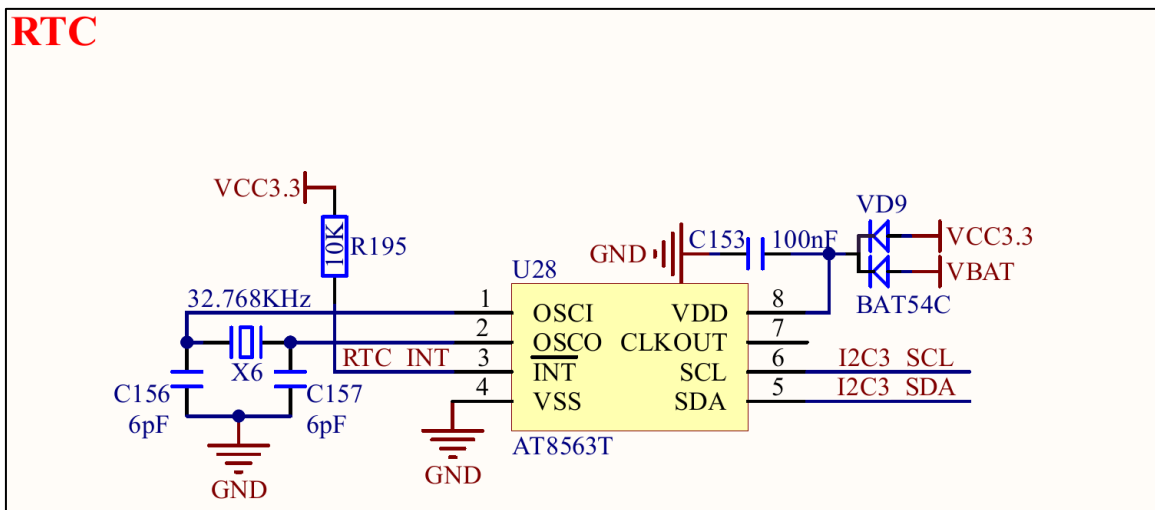


图 15.4-1 开发板 RTC 电路

电路部分解析

AT8563 RTC 芯片（U28）

- 引脚 1（OSCI）和 引脚 2（OSCO）：连接 32.768kHz 晶振（X1），用于提供 RTC 的时钟信号。晶振两端各有一个 6pF 的电容（C156 和 C157）接地，用于晶振的稳定。
- 引脚 3（INT）：中断输出引脚，用于产生中断信号。
- 引脚 4（VSS）：接地。
- 引脚 5（SDA）：I2C 数据线（I2C3_SDA），通过 I2C 总线与主控器通信。
- 引脚 6（SCL）：I2C 时钟线（I2C3_SCL），通过 I2C 总线与主控器通信。
- 引脚 7（CLKOUT）：时钟输出引脚，可以配置为输出时钟信号。此引脚未连接。

- 引脚 8 (VDD): 电源引脚, 连接到 VCC3.3, 通过去耦电容 C153 (100nF) 与地连接, 用于滤波和去耦。

电源管理

- VBAT: 电池供电, 用于在主电源断电时为 RTC 芯片供电。电池 CR1220 通过电容 C153 (100nF) 与地连接, 用于滤波。
- 二极管 (BAT54C): 用于防止主电源和电池供电之间的相互影响, 确保在主电源断电时, 电池能够可靠地为 RTC 供电。

供电路径

- 正常工作时:

主电源 VCC3.3 通过 VDD 引脚为 RTC 芯片供电, 同时通过去耦电容滤波。

- 主电源断电时:

电池 CR1220 通过 BAT54C 二极管和 VBAT 引脚为 RTC 芯片供电, 同时通过去耦电容 C7 滤波。

I2C 通信

SCL (引脚 6) 和 SDA (引脚 5): 连接到主控制器的 I2C 总线, 用于配置和读取 RTC 时间和日期。

总结

该电路设计使用 AT8563 RTC 芯片, 通过 32.768KHz 晶振提供时钟信号, 并通过 I2C 接口与主控制器通信。电路设计包括电源管理部分, 确保在主电源断电时, RTC 芯片能够通过电池继续保持时间。去耦电容用于滤除电源噪声, 保证系统的稳定运行。

15.4.2 实物硬件

ATK-DLMP257B 开发板上 AT8563T 位置如下:

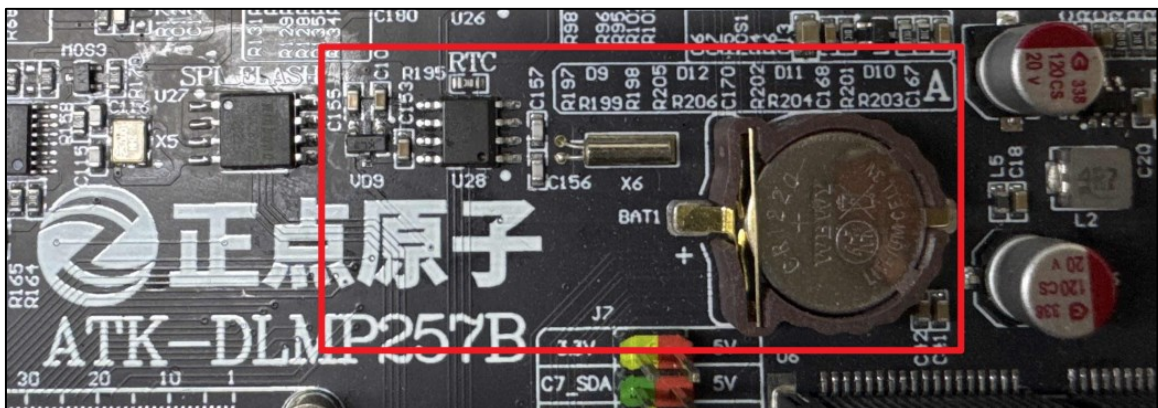


图 15.4-2 开发板 AT8563T 位置

AT8563T 是一个低功耗的实时时钟芯片, 通常用于各种嵌入式应用中, 提供时间和日期的保持功能。其主要特点包括:

低功耗: 适合电池供电的系统, 具有非常低的电流消耗。

时间和日期保持: 能够提供秒、分、小时、日、月和年的保持功能。

闹钟功能: 可以设置闹钟中断, 用于定时唤醒系统。



I2C 接口: 通过 I2C 总线与主机通信, 简单易用。

15.5 内核 RTC 子系统使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, RTC 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_RTC_LIB=y /*启用 RTC 库支持, 这是 RTC 驱动程序和其他相关代码所需的基础*/
2 CONFIG_RTC_CLASS=y /*启用 RTC 类支持, 编译 RTC 设备的核心代码,
   提供通用接口给 RTC 驱动程序*/
3 CONFIG_RTC_HCTOSYS=y /*启用从 RTC 设置系统时间。
   系统启动时, 内核会读取 RTC 时间并将其设置为系统时间*/
4 CONFIG_RTC_HCTOSYS_DEVICE="rtc0" /*指定用于设置系统时间的 RTC 设备。
   rtc0 是第一个 RTC 设备*/
5 CONFIG_RTC_SYSTOHC=y /*启用从系统时间设置 RTC。
   系统关机或重启时, 内核会将系统时间写入 RTC*/
6 CONFIG_RTC_SYSTOHC_DEVICE="rtc0" /*指定用于从系统时间设置硬件时钟的 RTC。
   rtc0 是第一个 RTC 设备*/
7 CONFIG_RTC_INTF_SYSFS=y /*启用 RTC 的 sysfs 接口。
   这允许通过 sysfs 文件系统访问 RTC 设备信息和设置*/
8 CONFIG_RTC_INTF_PROC=y /*启用 RTC 的 /proc 接口。
   这允许通过 /proc 文件系统访问 RTC 设备信息和设置*/
9 CONFIG_RTC_INTF_DEV=y /*启用 RTC 的字符设备接口。
   这允许通过 /dev/rtc* 设备文件访问 RTC 设备*/
10 CONFIG_RTC_I2C_AND_SPI=y /*启用 I2C 和 SPI 接口的 RTC 支持。
   内核将编译支持通过 I2C 和 SPI 总线连接的 RTC 设备的驱动程序*/
```

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

```
-> Device Drivers
  -> [*] Real Time Clock --->
    -> [*] Set system time from RTC on startup and resume
    -> (rtc0) RTC used to set the system time
    -> [*] Set the RTC time based on NTP synchronization
    -> (rtc0) RTC used to synchronize NTP adjustment
    -> [*] RTC non volatile storage support
    -> [*] /sys/class/rtc/rtcN (sysfs)
    -> [*] /proc/driver/rtc (procfs for rtcN)
    -> [*] /dev/rtcN (character devices)
    -> [*] Freescale SNVS RTC support
    -> <*> NXP BBNSM RTC support
    -> <*> NXP i.MX System Controller RTC support
    -> <*> NXP RPMSG RTC support
```

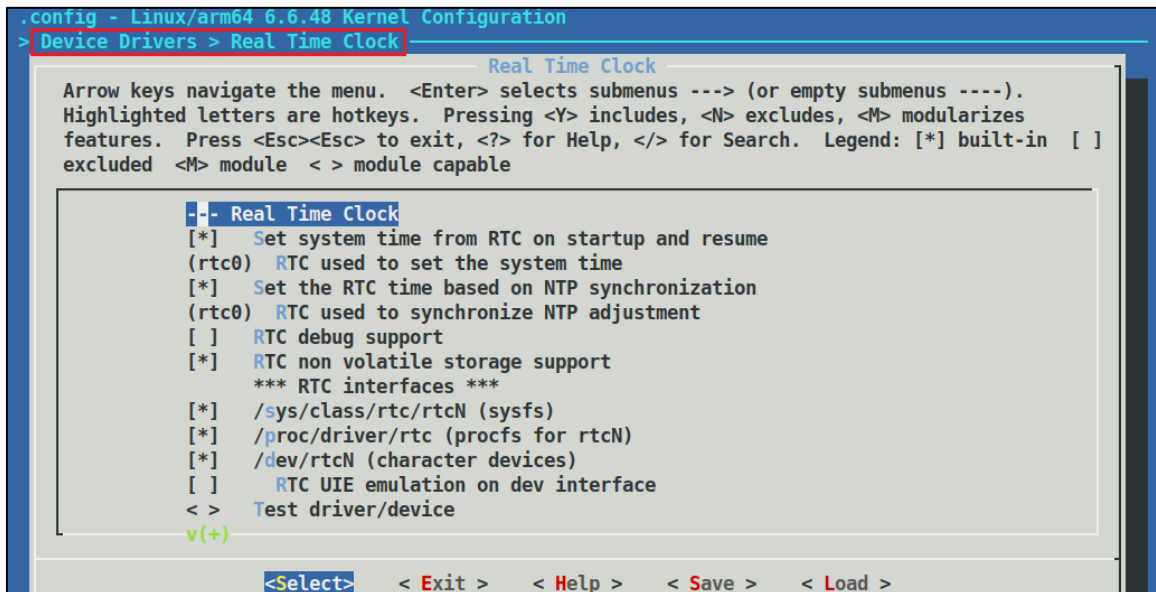


图 15.5-1 RTC 配置节选

15.6 RTC 驱动配置

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, PCF8563(实际配置兼容 AT8563T)相关配置如下:

```
1 CONFIG_RTC_DRV_PCF8563=y //启用 PCF8563 RTC 驱动程序
```

对应的 menuconfig 配置如下 (以下省略没有配置的部分):

```

-> Device Drivers
  -> [*] Real Time Clock --->
    -> <*> Philips PCF8563/Epson RTC8564
    
```

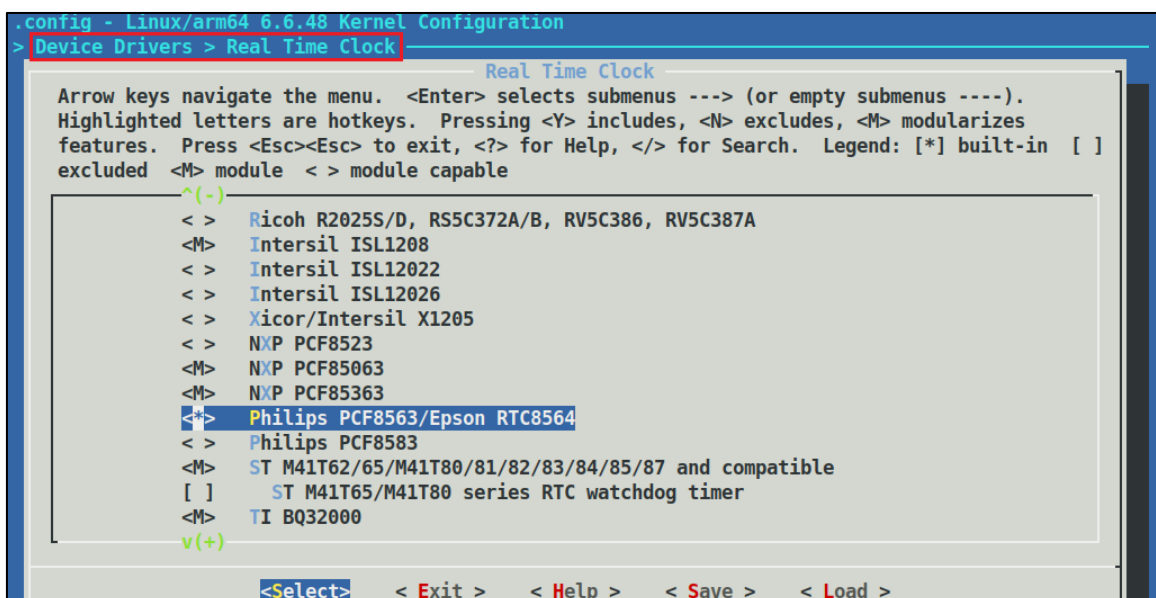


图 15.6-1 PCF8563/AT8563T 配置

选择配置后, 按下键盘的 “?” 键可以查看具体配置说明:

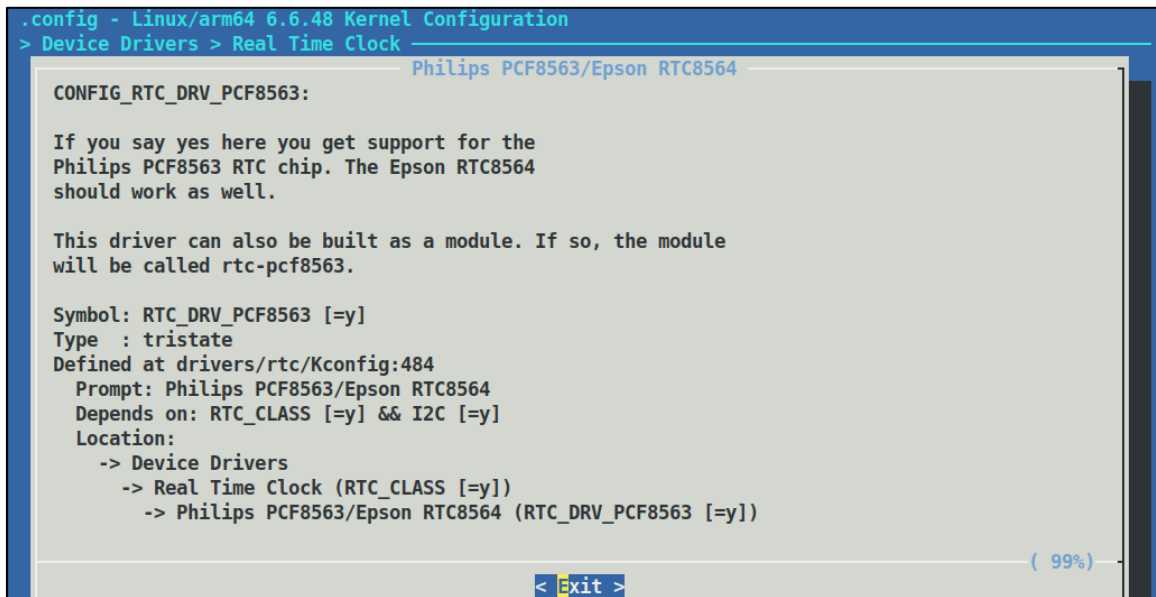


图 15.6-2 配置说明

根据内核配置选项详细信息来看，CONFIG_RTC_DRV_PCF8563 选项用于启用对 Philips PCF8563 和 Epson RTC8564 实时时钟（RTC）芯片的支持。

关于 I2C 的配置，在前面 I2C 章节已有具体配置描述，这里不再复述。

15.7 设备树文件分析

以 2GB 版本核心板为例，在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 中，pcf8563 相关配置如下：

```
1 &i2c3 {
2     pinctrl-names = "default", "sleep";
3     pinctrl-0 = <&i2c3_pins_a>;
4     pinctrl-1 = <&i2c3_sleep_pins_a>;
5     status = "okay";
6     /delete-property/dmas;
7     /delete-property/dma-names;
8
9     pcf8563@51 {
10         compatible = "nxp,pcf8563";
11         reg = <0x51>;
12         status = "okay";
13     };
14};
```

第 1 行，&i2c3 引用之前定义的 i2c3 控制器节点，对 i2c3 进行进一步配置。

第 2 行，pinctrl-names="default","sleep" 定义引脚控制状态的名称：default 表示正常工作模式下的引脚配置，sleep 表示睡眠模式下的引脚配置。

第 3 行，pinctrl-0 = <&i2c3_pins_a> 设置 default 状态下使用的引脚组为 i2c3_pins_a。

第 4 行，pinctrl-1 = <&i2c3_sleep_pins_a> 设置 sleep 状态下使用的引脚组为 i2c3_sleep_pins_a。

第 5 行, `status = "okay"` 将 i2c3 控制器的状态设置为 启用, 表示该 I2C 控制器处于激活状态。

第 6 行, `/delete-property/dmas` 删除 i2c3 控制器中的 dmas 属性, 移除对 DMA 通道的依赖配置。

第 7 行, `/delete-property/dma-names` 删除 i2c3 控制器中的 dma-names 属性, 进一步移除 DMA 相关设置。

第 9 行, `pcf8563@51` 定义一个名为 pcf8563 的子节点, 表示连接到 i2c3 控制器的 I2C 从设备。@51 表示该设备的 I2C 地址为 0x51。

第 10 行, `compatible = "nxp,pcf8563"` 指定该设备的兼容性为 nxp,pcf8563, 表示该设备是 NXP 公司的 PCF8563 RTC (实时时钟) 芯片。驱动文件路径为 `drivers/rtc/rtc-pcf8563.c`

第 11 行, `reg = <0x51>` 设置设备的 I2C 地址为 0x51, 与上一行的 @51 保持一致。

第 12 行, `status = "okay"` 将 pcf8563 设备的状态设置为 启用, 表示该 RTC 芯片可以正常工作。

15.8 RTC 调试工具

15.8.1 注意事项

先关闭 TSN 服务, 再设置时间进行测试;

```
systemctl stop depts.service
date
```

15.8.2 dmesg 命令

dmesg 命令用于查看内核缓冲区中的消息, 包括 RTC 设备的加载信息和错误消息。常见的命令和用法如下:

查看 RTC 相关的内核消息

```
dmesg | grep rtc
```

```
root@ATK-DLMP257:~# dmesg | grep rtc
[ 3.257217] stm32_rtc 46000000.rtc: registered as rtc0
[ 3.264901] stm32_rtc 46000000.rtc: setting system clock to 2000-01-01T00:00:10 UTC (946684810)
[ 3.281731] stm32_rtc 46000000.rtc: Date/Time must be initialized
[ 3.293814] stm32_rtc 46000000.rtc: registered rev:3.4
[ 3.309927] rtc-pcf8563 0-0051: low voltage detected, date/time is not reliable.
[ 3.320546] rtc-pcf8563 0-0051: registered as rtc1
root@ATK-DLMP257:~#
```

图 15.8-1 RTC 相关的内核消息

从 dmesg 输出来看, 系统中已经识别并注册了两个 RTC 设备:

rtc0: 由 stm32_rtc 驱动管理, 地址 46000000.rtc。

rtc1: 由 rtc-pcf8563 驱动管理, 地址 1-0051 (I2C 地址 0x51)。

这里我们只看 rtc1, 这个是 rtc-pcf8563 驱动, rtc0 是 ST 处理器自带的 RTC, 没有掉电保护功能。

此外, rtc-pcf8563 驱动报告检测到了低电压, 这意味着日期和时间可能不可靠。这通常表明 RTC 电池电量不足或没有连接电池。PCF8563/AT8563T 的工作电压范围在 1.0V 到 5.5V 之间, 低电压检测功能在接近工作电压下限时会出现提示。笔者用万用表测试电池为 3.3V, 但仍然有这个提示, 不影响使用。

处理低电压警告 low voltage detected, date/time is not reliable

检查电池：确保 RTC 电池已正确连接并且有足够电量。如果使用的是可更换电池，考虑更换电池。

设置可靠的时间：即使检测到低电压，也可以手动设置 RTC 时间，使其开始可靠计时。

15.8.3 hwclock 工具

hwclock 是一个用于管理硬件时钟（RTC）的工具，在 Linux 系统中非常常用。hwclock 工具用于与计算机的硬件时钟（RTC）进行交互。硬件时钟是一个独立于系统时钟的计时器，通常由电池供电，即使系统断电也能保持时间。hwclock 允许用户读取和设置硬件时钟的时间，并将硬件时钟的时间与系统时钟同步。

使用说明：

hwclock -h

或者 hwclock --help

```
root@ATK-DLMP257:~# hwclock -h
Usage:
hwclock [function] [option...]

Time clocks utility.

Functions:
-r, --show          display the RTC time
--get              display drift corrected RTC time
--set              set the RTC according to --date
-s, --hctosys      set the system time from the RTC
-w, --systohc      set the RTC from the system time
--systz            send timescale configurations to the kernel
-a, --adjust        adjust the RTC to account for systematic drift
--param-get <param> display the RTC parameter
--param-set <param>=<value> set the RTC parameter
--predict          predict the drifted RTC time according to --date

Options:
-u, --utc           the RTC timescale is UTC
-l, --localtime     the RTC timescale is Local
-f, --rtc <file>    use an alternate file to /dev/rtc0
--directisa        use the ISA bus instead of /dev/rtc0 access
--date <time>       date/time input for --set and --predict
--delay <sec>       delay used when set new RTC time
--update-drift      update the RTC drift factor
--noadjfile         do not use /etc/adjtime
--adjfile <file>    use an alternate file to /etc/adjtime
--test             dry run; implies --verbose
-v, --verbose       display more details

-h, --help          display this help
-V, --version       display version

Arguments:
<param> is either a numeric RTC parameter value or one of these aliases:
- features: supported features (0x0)
- correction: time correction (0x1)
- bsm: backup switch mode (0x2)
See Kernel's include/uapi/linux/rtc.h for parameters and values.

<param> and <value> accept hexadecimal values if prefixed with 0x, otherwise decimal.

For more details see hwclock(8).
root@ATK-DLMP257:~#
```

图 15.8-2 hwclock 使用说明

hwclock 工具提供了多种功能和选项，用于管理和操作硬件时钟（RTC）。以下是对各个功能和选项的详细说明。

● 功能

◆ 查看硬件时钟时间：

hwclock -r

或者 hwclock --show

```
root@ATK-DLMP257:~# hwclock -r
2000-01-01 00:01:29.472155+00:00
root@ATK-DLMP257:~#
```

图 15.8-3 显示当前硬件时钟的时间

从输出结果来看, hwclock -r 命令成功读取了硬件时钟的时间, 当前时间是 2000-01-01 00:01:29.472155+00:00。后面的 +00:00 是时间的时区偏移量, 表示当前时间是相对于协调世界时 (UTC) 的偏移量。比如说, +08:00 表示当前时间比 UTC 时间早 8 小时。这通常表示系统时钟设置在中国标准时间 (CST) 或者其他使用 UTC+8 时区的地区。

◆ 将硬件时钟时间设置为指定的日期和时间:

```
hwclock --set --date="2025-01-04 11:00:00"
```

```
root@ATK-DLMP257:~# hwclock -r
2000-01-01 00:01:29.472155+00:00
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# hwclock --set --date="2025-01-04 11:00:00"
root@ATK-DLMP257:~# hwclock -r
2025-01-04 11:00:14.865534+00:00
root@ATK-DLMP257:~#
```

图 15.8-4 设置硬件时钟时间

◆ 将硬件时钟的时间读取并设置为系统时间:

```
hwclock -s
```

或者 hwclock --hctosys

```
root@ATK-DLMP257:~# hwclock -s
root@ATK-DLMP257:~# [ 228.679810] systemd-journald[266]: Time jumped backwards, rotating.
hwclock --hctosys
root@ATK-DLMP257:~#
```

图 15.8-5 从 RTC 设置系统时间

◆ 将系统时间写入 RTC:

```
hwclock -w
```

或者 hwclock --systohc

```
root@ATK-DLMP257:~# hwclock -w
root@ATK-DLMP257:~# hwclock --systohc
root@ATK-DLMP257:~#
```

图 15.8-6 将系统时间写入 RTC

◆ 调整 RTC 以补偿系统漂移:

```
hwclock --adjust
```

调整硬件时钟以补偿系统漂移。这个功能依赖于 /etc/adjtime 文件。

```
root@ATK-DLMP257:~# hwclock --adjust
Needed adjustment is less than one second, so not setting clock.
root@ATK-DLMP257:~#
```

图 15.8-7 调整 RTC 以补偿系统漂移

根据输出信息表明, 硬件时钟的当前时间和系统时间非常接近, 两者之间的差异不到一秒。因此, 时钟不需要进行调整。

◆ 获取 RTC 参数, 显示指定的 RTC 参数值:

```
hwclock --param-get <param>
```

显示指定的 RTC 参数值。

◆ 设置 RTC 参数, 设置指定的 RTC 参数值:

```
hwclock --param-set <param>=<value>
```

◆ 预测漂移后的 RTC 时间:

```
hwclock --predict --date="2025-01-04 11:00:00"
```

```
root@ATK-DLMP257:~# hwclock --predict --date="2025-01-04 11:00:00"
2025-01-04 11:00:00.000000+00:00
root@ATK-DLMP257:~#
```

图 15.8-8 根据指定的日期和时间预测漂移后的 RTC 时间

输出结果 2025-01-04 11:00:00.000000+00:00: 这表示硬件时钟在 2024 年 1 月 4 日 11 点 00 分 00 秒的预期值。

这个命令通常用于检查硬件时钟的精度和未来某一时刻的时间预测，以确保时钟的准确性和可靠性。

- 选项

- ◆ 使用 UTC 时间，以 UTC 格式读取或设置硬件时钟时间：

```
hwclock -u
或者 hwclock --show
```

```
root@ATK-DLMP257:~# hwclock -u
2024-02-27 17:32:43.549357+00:00
root@ATK-DLMP257:~# hwclock --show
2024-02-27 17:32:47.718302+00:00
root@ATK-DLMP257:~#
```

图 15.8-9 使用 UTC 时间

- ◆ 使用本地时间，以本地时间格式读取或设置硬件时钟时间：

```
hwclock -l
hwclock --localtime
```

```
root@ATK-DLMP257:~# hwclock -l
2024-02-27 17:33:17.749540+00:00
root@ATK-DLMP257:~# hwclock --localtime
2024-02-27 17:33:21.724656+00:00
root@ATK-DLMP257:~#
```

图 15.8-10 使用本地时间

hwclock --show 和 hwclock -u: 输出的时间是 2024-02-27 17:32:47.718302+00:00 和 2024-02-27 17:32:43.549357+00:00，显示的是硬件时钟按系统时区转换后的 UTC 时间。

hwclock -l 和 hwclock --localtime: 输出的时间是 2024-02-27 17:33:17.749540+00:00 和 2024-02-27 17:33:21.724656+00:00，这表示硬件时钟当前时间的本地显示。

- 设置时间延迟，设置在设置新 RTC 时间时的延迟：

```
hwclock --delay <sec>
```

- ◆ 进行干运行（不实际更改）：

```
hwclock --test
```

进行干运行，显示将执行的操作但不实际更改任何设置。这个选项通常用于调试。

```
root@ATK-DLMP257:~# hwclock --test
hwclock from util-linux 2.39.3
System Time: 1709055422.271675
Trying to open: /dev/rtc0
Using the rtc interface to the clock.
Last drift adjustment done at 1709055022 seconds after 1969
Last calibration done at 1709055022 seconds after 1969
Hardware clock is on UTC time
Assuming hardware clock is kept in UTC time.
Waiting for clock tick...
...got clock tick
Time read from Hardware Clock: 2024/02/27 17:37:03
Hw clock time : 2024/02/27 17:37:03 = 1709055423 seconds since 1969
Time since last adjustment is 401 seconds
Calculated Hardware Clock drift is 0.000000 seconds
2024-02-27 17:37:02.255465+00:00
Test mode: nothing was changed.
root@ATK-DLMP257:~#
```

图 15.8-11 干运行

hwclock --test 命令的输出：

System Time: 1709055422.271675 这是自 1969 年以来的系统时间（以秒为单位）。这个时间是当前系统时间从 Unix 纪元（1970 年 1 月 1 日）起算的秒数。

Trying to open: /dev/rtc0 这是尝试打开的设备文件，表示正在尝试访问硬件时钟。

Using the rtc interface to the clock. 使用 rtc 接口访问时钟。

Last drift adjustment done at 1709055022 seconds after 1969 最后一次漂移调整是在自 1969 年以来的 1709055022 秒时进行的。

Last calibration done at 1709055022 seconds after 1969 最后一次校准是在自 1969 年以来的 1709055022 秒时进行的。

Hardware clock is on UTC time 硬件时钟是以 UTC 时间为准。

Assuming hardware clock is kept in UTC time. 假设硬件时钟保持在 UTC 时间。

Waiting for clock tick... 等待时钟跳动。

...got clock tick 获取到了时钟跳动。

Time read from Hardware Clock: 2024/02/27 17:37:03 从硬件时钟读取的时间是 2024 年 2 月 27 日 17:37:03。

Hw clock time : 2024/02/27 17:37:03 = 1709055423 seconds since 1969 硬件时钟时间是 2024 年 2 月 27 日 17:37:03，相当于自 1969 年以来的 1709055423 秒。

Time since last adjustment is 401 seconds 自上次调整以来已经过去了 401 秒。

Calculated Hardware Clock drift is 0.000000 seconds 计算出的硬件时钟漂移为 0.000000 秒。

2024-02-27 17:37:02.255465+00:00 当前系统时间为 2024 年 2 月 27 日 17:37:02.255465。

Test mode: nothing was changed. 测试模式：没有进行任何更改。

总结：这段输出显示了硬件时钟与系统时钟之间的同步情况，表明硬件时钟保持在 UTC 时间，且漂移几乎为零。测试模式下，未对任何时钟进行实际调整。

显示更多的详细信息：

hwclock -v
或者 hwclock --verbose

```
root@ATK-DLMP257:~# hwclock -v
hwclock from util-linux 2.39.3
System Time: 1709055625.980939
Trying to open: /dev/rtc0
Using the rtc interface to the clock.
Last drift adjustment done at 1709055022 seconds after 1969
Last calibration done at 1709055022 seconds after 1969
Hardware clock is on UTC time
Assuming hardware clock is kept in UTC time.
Waiting for clock tick...
...got clock tick
Time read from Hardware Clock: 2024/02/27 17:40:26
Hw clock time : 2024/02/27 17:40:26 = 1709055626 seconds since 1969
Time since last adjustment is 604 seconds
Calculated Hardware Clock drift is 0.000000 seconds
2024-02-27 17:40:25.956911+00:00
root@ATK-DLMP257:~#
```

图 15.8-12 显示详细信息

15.8.4 timedatectl

timedatectl 是一个命令行工具，用于查询和更改系统时间和日期设置。它是 systemd 项目的一部分，提供了一个简便的方法来管理系统时钟和时区设置，同时还可以与 NTP（网络时间协议）服务进行交互，以确保系统时间的准确性。

查看 timedatectl 使用说明：

```
root@ATK-DLMP257:~# timedatectl -h
timedatectl [OPTIONS...] COMMAND ...

Query or change system time and date settings.

Commands:
  status          Show current time settings
  show            Show properties of systemd-timesdted
  set-time TIME   Set system time
  set-timezone ZONE Set system time zone
  list-timezones  Show known time zones
  set-local-rtc BOOL Control whether RTC is in local time
  set-ntp BOOL    Enable or disable network time synchronization

systemd-timesyncd Commands:
  timesync-status Show status of systemd-timesyncd
  show-timesync   Show properties of systemd-timesyncd
  ntp-servers INTERFACE SERVER... Set the interface specific NTP servers
  revert INTERFACE Revert the interface specific NTP servers

Options:
  -h --help          Show this help message
  --version          Show package version
  --no-pager         Do not pipe output into a pager
  --no-ask-password Do not prompt for password
  -H --host=[USER@]HOST Operate on remote host
  -M --machine=CONTAINER Operate on local container
  --adjust-system-clock Adjust system clock when changing local RTC mode
  --monitor          Monitor status of systemd-timesyncd
  -p --property=NAME Show only properties by this name
  -a --all           Show all properties, including empty ones
  --value            When showing properties, only print the value

See the timedatectl(1) man page for details.
root@ATK-DLMP257:~#
```

图 15.8-13 timedatectl 使用说明

常见的用法和功能的详细说明:

◆ 查看当前时间设置:

timedatectl status

```
root@ATK-DLMP257:~# timedatectl status
Local time: Tue 2024-02-27 17:41:22 UTC
Universal time: Tue 2024-02-27 17:41:22 UTC
RTC time: Tue 2024-02-27 17:41:22
Time zone: Universal (UTC, +0000)
System clock synchronized: no
NTP service: inactive
RTC in local TZ: no
root@ATK-DLMP257:~#
```

图 15.8-14 当前时间设置

这个命令显示系统的当前时间设置，包括系统时间、本地时间、时区、NTP（网络时间协议）服务状态等。

以下是输出打印信息说明:

Local time: 本地时间设置为 UTC 时间，即 2024 年 2 月 27 日 17:41:22 UTC。

Universal time: 通用时间（UTC），与本地时间一致。

RTC time: 实时时钟（RTC）时间，与本地时间和通用时间一致。

Time zone: 系统时区设置为 UTC（+0000），没有时差。

System clock synchronized: 系统时钟已同步。

NTP service: NTP 服务处于活动状态，这意味着系统时间通过网络时间协议与时间服务器保持同步。

RTC in local TZ: RTC 不使用本地时区，即 RTC 时间为 UTC 时间。

◆ 启用或禁用 NTP（网络时间协议）:

timedatectl set-ntp BOOL

示例: 启用 NTP 服务，使系统时间与网络时间同步

timedatectl set-ntp true

示例：禁用 NTP 服务，以便可以手动设置时间

```
timedatectl set-ntp false
```

```
root@atk-imx93:~# timedatectl set-ntp true
root@atk-imx93:~# timedatectl set-ntp false
root@atk-imx93:~#
```

图 15.8-15 启用或禁用 NTP

◆ 设置系统时间:

```
timedatectl set-time "YYYY-MM-DD HH:MM:SS"
```

示例：将系统时间设置为 2025 年 1 月 4 日 12:14:00，此操作需提前禁用 NTP 服务。

```
timedatectl set-time "2025-01-04 12:15:00"
```

```
root@atk-imx93:~# timedatectl set-ntp false
root@atk-imx93:~#
root@atk-imx93:~# timedatectl set-time "2024-06-01 12:34:56"
root@atk-imx93:~# timedatectl status
          Local time: Sat 2024-06-01 04:35:06 UTC
          Universal time: Sat 2024-06-01 04:35:06 UTC
             RTC time: Sat 2024-06-01 04:35:06
            Time zone: Universal (UTC, +0000)
System clock synchronized: no
              NTP service: inactive
             RTC in local TZ: no
root@atk-imx93:~#
```

图 15.8-16 设置系统时间

◆ 设置系统时区:

示例：将系统时区设置为东八区（中国标准时间）。

```
timedatectl set-timezone Asia/Shanghai
```

```
root@ATK-DLMP257:~# timedatectl set-timezone Asia/Shanghai
root@ATK-DLMP257:~# timedatectl status
          Local time: Wed 2024-02-28 01:44:13 CST
          Universal time: Tue 2024-02-27 17:44:13 UTC
             RTC time: Sat 2025-01-04 12:15:30
            Time zone: Asia/Shanghai (CST, +0800)
System clock synchronized: no
              NTP service: inactive
             RTC in local TZ: no
root@ATK-DLMP257:~#
```

图 15.8-17 将系统时区设置为东八区

◆ 列出所有可用的时区:

```
timedatectl list-timezones
```

以下列出系统中可用的时区，按回车键继续查看，按 Ctrl+C 键取消。

```
root@ATK-DLMP257:~# timedatectl list-timezones
Africa/Abidjan
Africa/Accra
Africa/Addis_Ababa
Africa/Algiers
Africa/Asmara
Africa/Asmera
Africa/Bamako
Africa/Bangui
```

以下省略

图 15.8-18 可用的时区

◆ 设置 RTC（实时时钟）是否使用本地时间:

示例：将 RTC 设置为使用本地时间

```
timedatectl set-local-rtc 1
```

```
root@ATK-DLMP257:~# timedatectl set-local-rtc 1
root@ATK-DLMP257:~# timedatectl status
    Local time: Wed 2024-02-28 01:45:31 CST
    Universal time: Tue 2024-02-27 17:45:31 UTC
    RTC time: Wed 2024-02-28 01:45:31
    Time zone: Asia/Shanghai (CST, +0800)
System clock synchronized: no
    NTP service: inactive
    RTC in local TZ: yes

Warning: The system is configured to read the RTC time in the local time zone.
This mode cannot be fully supported. It will create various problems
with time zone changes and daylight saving time adjustments. The RTC
time is never updated, it relies on external facilities to maintain it.
If at all possible, use RTC in UTC by calling
'timedatectl set-local-rtc 0'.
root@ATK-DLMP257:~#
```

图 15.8-19 将 RTC 设置为使用本地时间

该警告信息指出，将 RTC 设置为本地时间可能会导致以下问题：

时区变化问题：当系统时区发生变化时，本地时间模式下的 RTC 可能会不准确。

夏令时调整问题：在夏令时调整期间，本地时间模式下的 RTC 可能无法正确反映时间变化。

RTC 时间不会自动更新：RTC 时间不会自动更新，需要依赖外部设施（如手动设置）来保持时间准确性。

15.8.5 rtcwake 工具

rtcwake 是一个 Linux 工具，用于管理系统的实时时钟 (RTC) 和系统唤醒事件。它允许用户将系统置于低功耗模式（如待机、休眠或暂停），并在指定的时间自动唤醒系统。这个工具对于节能和定时任务非常有用。

rtcwake 使用说明：

rtcwake -h

```
root@ATK-DLMP257:~# rtcwake -h

Usage:
  rtcwake [options]

Enter a system sleep state until a specified wakeup time.

Options:
  -a, --auto                reads the clock mode from adjust file (default)
  -A, --adjfile <file>     specifies the path to the adjust file
                           the default is /etc/adjtime
  --date <timestamp>       date time of timestamp to wake
  -d, --device <device>    select rtc device (rtc0|rtc1|...)
  -n, --dry-run             does everything, but suspend
  -l, --local               RTC uses local timezone
  --list-modes              list available modes
  -m, --mode <mode>        standby|mem|... sleep mode
  -s, --seconds <seconds>  seconds to sleep
  -t, --time <time_t>       time to wake
  -u, --utc                 RTC uses UTC
  -v, --verbose             verbose messages

  -h, --help                display this help
  -V, --version             display version

For more details see rtcwake(8).
root@ATK-DLMP257:~#
```

图 15.8-20 rtcwake 使用说明

基本用法

rtcwake 命令用于使系统进入某种睡眠状态，并在指定的时间唤醒系统。其基本语法为：

rtcwake [options]

常用选项

- -a, --auto: 从调整文件读取时钟模式（默认行为）。



- -A, --adjfile <file>: 指定调整文件的路径, 默认是 /etc/adjtime。
- --date <timestamp>: 指定唤醒的日期时间戳。
- -d, --device <device>: 选择 RTC 设备 (如 rtc0、rtc1)。
- -n, --dry-run: 执行所有操作, 但不真正进入睡眠状态。
- -l, --local: RTC 使用本地时区。
- --list-modes: 列出可用的睡眠模式。
- -m, --mode <mode>: 指定系统睡眠模式, 如 freeze、mem 等。
- -s, --seconds <seconds>: 指定系统休眠的秒数。
- -t, --time <time_t>: 指定唤醒的绝对时间 (自 1970-01-01 00:00:00 UTC 以来的秒数)。
- -u, --utc: RTC 使用 UTC 时间。
- -v, --verbose: 显示详细信息。
- -h, --help: 显示帮助信息。
- -V, --version: 显示版本信息。

可用模式

可以使用 --list-modes 选项列出系统支持的所有睡眠模式:

```
rtcwake --list-modes
```

```
root@ATK-DLMP257:~# rtcwake --list-modes
freeze mem disk off no on disable show
root@ATK-DLMP257:~#
```

图 15.8-21 系统支持的睡眠模式

各模式的解释

- freeze: 系统进入冻结状态, 所有进程停止, 但系统仍保持电源开启。
- mem: 系统进入内存睡眠模式(Suspend-to-RAM), 系统状态保存在内存中, 功耗较低。
- off: 系统关闭 (需要 ACPI 支持), 完全断电。
- no: 不改变电源状态, 只设置唤醒时间。
- on: 仅唤醒系统, 不改变电源状态。
- disable: 禁用唤醒定时器。
- show: 显示当前的 RTC 唤醒定时器设置。

示例用法

1、系统进入内存睡眠模式并在 10 秒后唤醒:

```
rtcwake -m mem -s 10
```



```

root@ATK-DLMP257:~# rtcwake -m mem -s 10
rtcwake: wakeup from "mem" using /dev/rtc0 at Wed Feb 28 01:47:26 2024
[ 1276.389705] PM: suspend entry (deep)
[ 1276.390304] Filesystems sync: 0.000 seconds
[ 1276.523462] Freezing user space processes
[ 1276.525364] Freezing user space processes completed (elapsed 0.001 seconds)
[ 1276.528923] OOM killer disabled.
[ 1276.532143] Freezing remaining freezable tasks
[ 1276.537878] Freezing remaining freezable tasks completed (elapsed 0.001 seconds)
[ 1276.544036] printk: Suspending console(s) (use no console suspend to debug)
[ 1276.553541] rtk_btusb: btusb_suspend: hdev is not HCI_RUNNING
[ 1276.553545] rtk_btusb: btusb_suspend: suspending...
[ 1276.576784] stm32mp257-pinctrl soc@0:pinctrl@44240000: not freeing pin 57 (PD9) as part of deactivating
group PD9 - it is already used for some other setting
[ 1276.583193] stm32-dwmac 482d0000.eth2 end0: Link is Down
[ 1276.583402] stm32-dwmac 482d0000.eth2 end0: FPE workqueue stop
[ 1276.583920] stm32-dwmac 482c0000.eth1 end1: Link is Down
[ 1276.583977] stm32-dwmac 482c0000.eth1 end1: FPE workqueue stop
[ 1276.604049] Disabling non-boot CPUs ...
[ 1276.604812] psci: CPU1 killed (polled 0 ms)
[ 1276.606450] Enabling non-boot CPUs ...
[ 1276.606671] Detected VIPT I-cache on CPU1
[ 1276.606743] CPU1: Booted secondary processor 0x0000000001 [0x411fd040]
[ 1276.607317] CPU1 is up
[ 1276.620575] dwmac4: Master AXI performs any burst length
[ 1276.620607] stm32-dwmac 482c0000.eth1 end1: No Safety Features support found
[ 1276.620626] stm32-dwmac 482c0000.eth1 end1: IEEE 1588-2008 Advanced Timestamp supported
[ 1276.620873] stm32-dwmac 482c0000.eth1 end1: FPE workqueue start
[ 1276.620920] stm32-dwmac 482c0000.eth1 end1: configuring for fixed/rgmii link mode
[ 1276.620955] stm32-dwmac 482c0000.eth1 end1: Link is Up - 1Gbps/Full - flow control off
[ 1276.811733] sw0p3: Link is DOWN
[ 1277.083494] sw0p2: Link is DOWN
[ 1277.092568] dwmac4: Master AXI performs any burst length
[ 1277.092591] stm32-dwmac 482d0000.eth2 end0: No Safety Features support found
[ 1277.092607] stm32-dwmac 482d0000.eth2 end0: IEEE 1588-2008 Advanced Timestamp supported
[ 1277.092793] stm32-dwmac 482d0000.eth2 end0: FPE workqueue start
[ 1277.092840] stm32-dwmac 482d0000.eth2 end0: configuring for phy/rgmii-id link mode
[ 1277.680669] usb 1-1: reset high-speed USB device number 2 using ehci-platform
[ 1278.152580] usb 1-1.1: reset high-speed USB device number 3 using ehci-platform
[ 1278.853385] start_addr=(0x0), end_addr=(0x20000), buffer_size=(0x20000), smp_number_max=(16384)
[ 1278.893936] rtk_btusb: chip type value: 0x76
[ 1278.895463] rtk_btusb: btusb_probe intf->cur_altsetting->desc.bInterfaceNumber 0
[ 1279.079424] OOM killer enabled.
[ 1279.082496] Restarting tasks ...
[ 1279.085780] rtk_btusb: btusb_open hdev->promisc ==0
[ 1279.088938] done.
[ 1279.092537] rtk_btusb: chip type value: 0x76
[ 1279.097575] random: crng reseeded on system resumption
[ 1279.114303] thermal cooling_device1: GPU Clock Scale to 64/64
[ 1279.114822] rtk_btusb: config filename rtl8733bu_config
[ 1279.119941] rtk_btusb: no bdaddr file /opt/bdaddr
[ 1279.124501] rtk_btusb: Origin cfg len 14
[ 1279.124513] rtk_btusb: New cfg len 14
[ 1279.126027] rtk_btusb: This is not 8723a, use new patch style!
[ 1279.138955] PM: suspend exit
[ 1279.141539] stm32-dwmac 482d0000.eth2 end0: Link is Up - 1Gbps/Full - flow control rx/tx
root@ATK-DLMP257:~# [ 1279.163840] systemd-journald[266]: Time jumped backwards, rotating.
[ 1279.388834] rtk_btusb: chip type value: 0x76

```

进入深度睡眠模式

退出深度睡眠模式

图 15.8-22 进入内存深度睡眠模式 10 秒

系统进入深度睡眠模式，冻结用户空间进程，禁用非启动 CPU，并关闭以太网接口。十秒后，系统从深度睡眠模式唤醒，重新启用非启动 CPU，重新配置以太网接口，并恢复用户空间进程，以太网接口恢复连接，网络功能重新就绪。

2、系统进入冻结状态并在 10 秒后唤醒

```
rtcwake -m freeze -s 10
```

```

root@ATK-DLMP257:~# rtcwake -m freeze -s 10
rtcwake: wakeup from "freeze" using /dev/rtc0 at Wed Feb 28 01:50:01 2024
[1421.043995] PM: suspend entry (s2idle)
[1421.046001] Filesystems sync: 0.001 seconds
[1421.137384] Freezing user space processes
[1421.139280] Freezing user space processes completed (elapsed 0.001 seconds)
[1421.142851] OOM killer disabled.
[1421.146084] Freezing remaining freezable tasks
[1421.151828] Freezing remaining freezable tasks completed (elapsed 0.001 seconds)
[1421.157986] printk: Suspending console(s) (use no console suspend to debug)
[1421.188776] stm32mp257-pinctrl soc@0:pinctrl@44240000: not freeing pin 57 (PD9) as part of deactivating
group PD9 - it is already used for some other setting
[1421.193858] stm32-dwmac 482d0000.eth2 end0: Link is Down
[1421.193962] stm32-dwmac 482d0000.eth2 end0: FPE workqueue stop
[1421.194477] stm32-dwmac 482c0000.eth1 end1: Link is Down
[1421.194530] stm32-dwmac 482c0000.eth1 end1: FPE workqueue stop
[1421.198898] rtk_btusb: btusb_suspend: hdev is not HCI_RUNNING
[1421.198903] rtk_btusb: btusb_suspend: suspending...
[1421.248944] dwmac4: Master AXI performs any burst length
[1421.248977] stm32-dwmac 482c0000.eth1 end1: No Safety Features support found
[1421.248996] stm32-dwmac 482c0000.eth1 end1: IEEE 1588-2008 Advanced Timestamp supported
[1421.249191] stm32-dwmac 482c0000.eth1 end1: FPE workqueue start
[1421.249238] stm32-dwmac 482c0000.eth1 end1: configuring for fixed/rgmii link mode
[1421.249272] stm32-dwmac 482c0000.eth1 end1: Link is Up - 1Gbps/Full - flow control off
[1421.437033] sw0p3: Link is DOWN
[1421.711706] sw0p2: Link is DOWN
[1421.720537] dwmac4: Master AXI performs any burst length
[1421.720558] stm32-dwmac 482d0000.eth2 end0: No Safety Features support found
[1421.720572] stm32-dwmac 482d0000.eth2 end0: IEEE 1588-2008 Advanced Timestamp supported
[1421.720708] stm32-dwmac 482d0000.eth2 end0: FPE workqueue start
[1421.720755] stm32-dwmac 482d0000.eth2 end0: configuring for phy/rgmii-id link mode
[1422.300626] usb 1-1: reset high-speed USB device number 2 using ehci-platform
[1422.772551] usb 1-1.1: reset high-speed USB device number 3 using ehci-platform
[1423.473381] start_addr=(0x0), end_addr=(0x20000), buffer_size=(0x20000), smp_number_max=(16384)
[1423.513933] rtk_btusb: chip type value: 0x76
[1423.515548] rtk_btusb: btusb_probe intf->cur_altsetting->desc.bInterfaceNumber 0
[1423.674552] OOM killer enabled.
[1423.677733] rtk_btusb: btusb_open hdev->promisc ==0
[1423.677733] Restarting tasks ...
[1423.683145] done.
[1423.685965] rtk_btusb: chip type value: 0x76
[1423.696699] random: crng reseeded on system resumption
[1423.697265] thermal cooling device1: GPU Clock Scale to 64/64
[1423.707431] rtk_btusb: config filename rtl8733bu_config
[1423.708448] rtk_btusb: no bdaddr file /opt/bdaddr
[1423.713185] rtk_btusb: Origin cfg len 14
[1423.717094] rtk_btusb: New cfg len 14
[1423.722204] rtk_btusb: This is not 8723a, use new patch style!
[1423.732690] PM: suspend exit
root@ATK-DLMP257:~# [ 1423.770286] stm32-dwmac 482d0000.eth2 end0: Link is Up - 1Gbps/Full - flow control
x/tx
[1423.805880] systemd-journald[266]: Time jumped backwards, rotating.
[1423.968433] rtk_btusb: chip type value: 0x76
[1423.968456] rtk_btusb: btusb_open set HCI UP RUNNING
[1423.972209] rtk_btcoex: Open BTCOEX
[1424.089265] rtk_btcoex: Close BTCOEX
[1424.099736] Bluetooth: MGMT ver 1.22

```

休眠过程

唤醒过程和系统恢复

图 15.8-23 系统冻结状态十秒

系统进入休眠状态（s2idle），文件系统同步完成。用户空间进程和其余可冻结的任务被冻结，OOM（内存不足）killer 被禁用。设备进入休眠模式，包括以太网接口 eth1 和 eth0。十秒后设备恢复，包括重新配置和启动以太网接口 eth1。系统恢复正常操作，重新启用 OOM killer，任务重新启动，随机数生成器重新播种。以太网接口 eth0 恢复并准备就绪。

3、系统关闭并在 10 秒后唤醒

```
rtcwake -m off -s 10
```



```
root@ATK-DLMP257:~# rtcwake -m off -s 10
rtcwake: wakeup from "off" using /dev/rtc0 at Wed Feb 28 01:52:59 2024
root@ATK-DLMP257:~# Stopping Session c2 of User root...
[ OK ] Removed slice Slice /system/modprobe.
[ OK ] Removed slice Slice /system/systemd-coredump.
[ OK ] Stopped target Bluetooth Support.
[ OK ] Stopped target Multi-User System.
[ OK ] Stopped target Login Prompts.
[ OK ] Stopped target Host and Network Name Lookups.
[ OK ] Stopped target RPC Port Mapper.
[ OK ] Stopped target Sound Card.
[ OK ] Stopped target Timer Units.
[ OK ] Stopped Serves stm32mp clocks calibration..
[ OK ] Stopped Daily Cleanup of Temporary Directories.
[ OK ] Stopped target System Time Set.
[ OK ] Stopped target Hardware activated USB gadget.
[ OK ] Closed Load/Save RF Kill Switch Status /dev/rfkill Watch.
Stopping Save/Restore Sound Card State...
Stopping Avahi mDNS/DNS-SD Stack...
Stopping BlueALSA service...
```

系统关机

图 15.8-24 系统关闭并在十秒后唤醒

系统打印 `rtcwake: wakeup from "off" using /dev/rtc0 at Wed Feb 28 01:52:59 2024` 提示在十秒后系统将被唤醒。

```
[ 1591.579003] systemd-shutdown[1]: Powering off.
[ 1591.777282] reboot: Power down
INFO: PSCI Power Domain Map:
INFO: Domain Node : Level 4, parent_node 4294967295, State ON (0x0)
INFO: Domain Node : Level 3, parent_node 0, State ON (0x0)
INFO: Domain Node : Level 2, parent_node 1, State ON (0x0)
INFO: Domain Node : Level 1, parent_node 2, State ON (0x0)
INFO: CPU Node : MPID 0x0, parent_node 3, State ON (0x0)
INFO: CPU Node : MPID 0x1, parent_node 3, State ON (0x0)
WARNING: PSCI system off with other core running.
WARNING: IAC exceptions pending [159:128] = 4000000
WARNING: - 154
NOTICE: CPU: STM32MP257DAK Rev.Y
NOTICE: Model: ALIENTEK STM32MP257 Evaluation Board
INFO: Reset reason (0xd02030):
INFO: System exits from Standby for CA35
INFO: PMIC2 version = 0x11
INFO: PMIC2 product ID = 0x24
INFO: FCONF: Reading TB_FW firmware configuration file from: 0xe011000
INFO: FCONF: Reading firmware configuration information for: stm32mp_fuse
INFO: FCONF: Reading firmware configuration information for: stm32mp_io
```

系统关闭后
等待10秒
系统被唤醒

图 15.8-25 系统被唤醒

4、仅设置唤醒时间，不改变电源状态

此功能不影响系统运行状态，要验证此功能需要结合 `poweroff` 来验证，`poweroff` 指令可以让系统进入休眠状态，通过提前设置好的 `rtcwake` 唤醒时间来定时唤醒系统，这里笔者设置运行 `rtcwake` 指令 20 秒后系统被唤醒。

```
rtcwake -m no -s 20
poweroff
```

```
root@ATK-DLMP257:~# rtcwake -m no -s 20
rtcwake: wakeup using /dev/rtc0 at Wed Feb 28 01:56:02 2024
root@ATK-DLMP257:~# poweroff
root@ATK-DLMP257:~# Stopping Session c2 of User root...
[ OK ] Removed slice Slice /system/modprobe.
[ OK ] Removed slice Slice /system/systemd-coredump.
[ OK ] Stopped target Bluetooth Support.
[ OK ] Stopped target Multi-User System.
[ OK ] Stopped target Login Prompts.
```

图 15.8-26 设置 20 秒唤醒系统，现在执行 `poweroff` 休眠系统

<pre>[35.208296] usb usb1: USB disconnect, device number 1 [35.213341] usb 1-1: USB disconnect, device number 2 [35.219110] ci_hdrc ci_hdrc.1: USB bus 1 deregistered [35.252331] reboot: Power down U-Boot SPL 2023.04-g2a47fb72 (Apr 11 2024 - 14:48:23 +0800) SOC: 0xa1009300 LC: 0x2040010 PMIC: Over Drive Voltage Mode DDR: 3733MTS M33 prepare ok Normal Boot Trying to boot from BOOTROM Boot Stage: Primary boot image offset 0x0, pagesize 0x200, ivt offset 0x0 Load image from 0x47000 by ROM_API NOTICE: BL31: v2.8(release): NOTICE: BL31: Built : 03:29:23, Apr 8 2024 U-Boot 2023.04-g2a47fb72 (Apr 11 2024 - 14:48:23 +0800) CPU: i.MX93(52) rev1.1 1700 MHz (running at 1692 MHz) CPU: Industrial temperature grade (-40C to 105C) at 40C</pre>	系统当前已休眠 到达指定时间后，系统自动唤醒
---	--------------------------------------

图 15.8-27 验证 `rtcwake -m no -s 20` 指令

5、禁用唤醒定时器

如果前面设置了唤醒定时器后，不想让唤醒定时器生效，可以禁用唤醒定时器。

`rtcwake -m disable`

6、显示当前的 RTC 唤醒定时器设置

`rtcwake -m show`

```
root@ATK-DLMP257:~# rtcwake -m show
alarm: off
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# rtcwake -m no -s 20
rtcwake: wakeup using /dev/rtc0 at Wed Feb 28 03:58:21 2024
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# rtcwake -m show
alarm: on Wed Feb 28 03:58:22 2024
root@ATK-DLMP257:~#
```

图 15.8-28 显示当前的 RTC 唤醒定时器设置

禁用 RTC 唤醒定时器后或者没设置 RTC 唤醒定时器，当前 RTC 唤醒定时器状态为 off 关闭状态。在设置完 RTC 唤醒定时器后，RTC 唤醒定时器生效，使用 `rtcwake -m show` 指令可以看到当前 RTC 唤醒定时器的具体状态。

第十六章 UART 子系统

16.1 UART 子系统简介

嵌入式 Linux 的 UART (Universal Asynchronous Receiver/Transmitter) 子系统是嵌入式系统中非常重要的部分, 它负责串行通信的管理和控制。UART 子系统在嵌入式设备中用于与其他设备 (如传感器、外设、计算机等) 进行数据交换。

UART 的基本概念:

UART 是一种串行通信协议, 用于异步数据传输。它可以将并行数据转换为串行数据, 并通过串口发送。

波特率: 数据传输速率, 通常以比特每秒 (bps) 表示。

数据位、停止位和校验位: 控制数据传输的格式。

16.2 主要特点

嵌入式 Linux 的 UART 子系统有几个显著的特点, 这些特点使得它在嵌入式系统中广泛应用于处理低速、串行的数据通信。以下是 UART 子系统的主要特点:

1、低成本和简单性:

- UART 硬件通常非常简单, 这意味着成本较低, 适合用在成本敏感的嵌入式项目中。
- 实现 UART 通信不需要复杂的硬件支持, 如时钟信号同步, 因为它是异步通信。

2、广泛的设备兼容性:

- UART 是一种极为普遍的通信标准, 被多种微控制器、处理器及其他电子组件支持。
- 这种通用性确保了它可以与各种传感器、模块和外部系统 (如调试设备、PC 等) 轻松集成。

3、直接的内核支持和标准接口:

- Linux 内核提供了对 UART 的广泛支持, 包括标准的设备驱动程序和 tty 接口, 这使得从用户空间访问 UART 设备变得非常简单。
- 开发者可以通过标准的系统调用, 如 open、read、write 和 close 来操作 UART 设备, 无需复杂的配置。

4、可扩展的串行通信选项:

- UART 子系统支持多种配置选项, 包括不同的波特率、停止位、奇偶校验等, 使其能够灵活地适应各种通信需求。
- 通过使用设备树 (Device Tree), 可以灵活地配置和管理硬件资源, 例如指定中断和 GPIO 映射, 以支持更多的串行端口。

这些特点使得 UART 在嵌入式系统中成为实现设备间低速数据交换的理想选择, 尤其是在资源受限或成本敏感的应用场景中。

16.3 组成部分

嵌入式 Linux 的 UART 子系统由多个部分组成, 这些部分共同作用, 确保串行通信的有效性和可靠性。以下是 UART 子系统的主要组成部分:

1、硬件部分

- UART 控制器: UART 硬件模块, 负责串行数据的发送和接收。典型的 UART 控制器具有发送 (TX) 和接收 (RX) 引脚。
- 时钟和电源管理: 提供给 UART 控制器的时钟信号和电源, 确保其正常工作。



- 引脚复用 (Pin Multiplexing): 在许多嵌入式系统中, UART 引脚可能与其他功能共享, 需要通过引脚复用来配置。

2、设备树 (Device Tree)

- 设备树源文件 (.dts): 描述硬件平台的配置文件, 包括 UART 控制器的内存映射、中断号和其他硬件参数。
- 设备树编译文件 (.dtb): 编译后的设备树二进制文件, 由内核在启动时加载。

3、驱动程序

- 内核 UART 驱动程序: 位于 Linux 内核中的驱动代码, 负责初始化和管理工作。典型的 UART 驱动程序包括:
 - 低级驱动程序: 直接与 UART 硬件交互, 进行数据传输和接收的基本操作。
 - 中断处理程序: 处理 UART 控制器发出的中断信号, 管理数据的缓冲和传输。

4、tty 子系统

- tty 层: Linux 内核中的字符设备子系统, 负责管理终端设备和串行通信接口。tty 层为用户空间应用程序提供标准的字符设备接口。
- tty 驱动程序: UART 驱动程序通常实现为 tty 驱动, 使得 UART 设备可以作为 tty 设备被访问, 如/dev/ttyS0。

5、用户空间工具和库

- 设备节点: 位于/dev 目录下的设备文件, 如/dev/ttyS0, 用户空间应用程序通过这些设备文件与 UART 硬件进行交互。
- 配置和测试工具:
 - stty: 用于设置和显示串行设备的参数。
 - minicom: 一个串行终端仿真程序, 用于调试和测试 UART 设备。

6、调试和诊断

- 日志和调试信息: 内核日志 (通过 dmesg 查看) 和调试信息有助于识别和解决 UART 通信中的问题。
- 调试接口: 许多 UART 驱动程序提供调试接口, 通过这些接口可以获取驱动程序的运行状态和统计信息。

16.4 MP257 的 UART 说明

16.4.1 文档和资源

想要获取 MP257 系列处理器的 UART 详细说明, 最直接的方式是参考 ST 官方文档, 包括:

- 1、参考手册 (Reference Manual): 提供了详细的寄存器级说明, 包括 UART 模块的寄存器映射以及配置方法等。此文档已放在【正点原子】STM32MP257 开发板 (A 盘)-基础资料\07、STM32MP2 参考资料
- 2、数据手册 (Data Sheet): 提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板 (A 盘)-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板, 正点原子还提供了一系列的开发文档, 便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南 (即本文档)

应用开发: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

16.4.2 特性说明

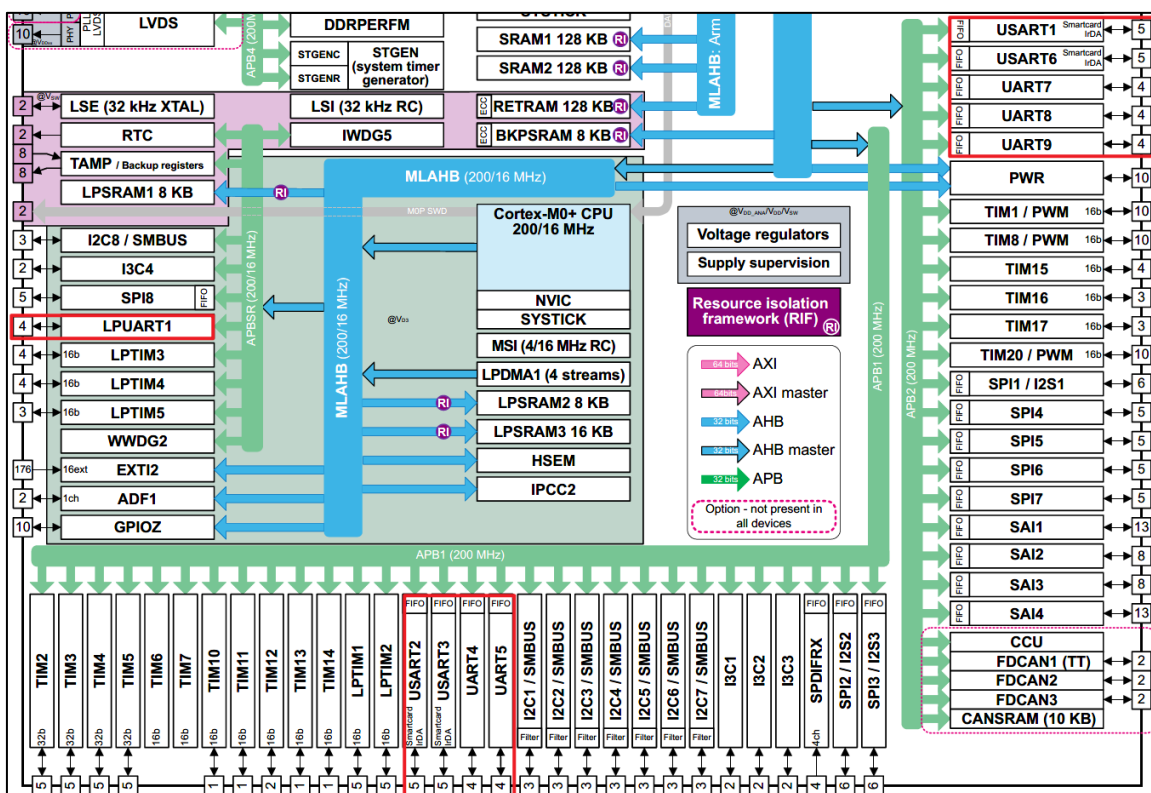


图 16.4-1 MP257 功能框图中的 UART 部分

MP257 最多支持 $5 \times \text{UART} + 4 \times \text{USART}$ (12.5 Mbit/s, ISO7816 interface, LIN, IrDA, SPI) + $1 \times \text{LPUART}$, 具体特性可以在芯片参考手册中查到:

Instance	STM32MP25xx
USART1	Full
USART2	Full
USART3	Full
USART6	Full
UART4	Basic
UART5	Basic
UART7	Basic
UART8	Basic
UART9	Basic
LPUART1	Low-power

图 16.4-2 MP257 的 UART 示例

Modes/features ⁽¹⁾	Full feature	Basic feature	Low-power feature
Hardware flow control for modem	X	X	X
Continuous communication using DMA	X	X	X
Multiprocessor communication	X	X	X
Synchronous mode (master/slave)	X	-	-
Smartcard mode	X	-	-
Single-wire half-duplex communication	X	X	X
IrDA SIR ENDEC block	X	X	-
LIN mode	X	X	-
Dual-clock domain	X	X	X
Receiver timeout interrupt	X	X	-
Modbus communication	X	X	-
Auto baud rate detection	X	X	-
Driver enable	X	X	X
USART data length	7, 8, and 9 bits		
Tx/Rx FIFO	X	X	X
Tx/Rx FIFO size (bytes)	64		8
Wake-up from low-power mode	X ⁽²⁾	X ⁽²⁾	X ⁽²⁾⁽³⁾
Autonomous mode	X	X	X

1. "X" = supported, "-" = not supported.

2. Wake-up supported from Stop 1, Stop 2, Low-power stop 1, and Low-power stop 2 modes.

3. Wake-up supported from LPLV-Stop 1, LPLV-Stop 2, and Standby 1 modes.

图 16.4-3 MP257 的 USART/LPUART 特性

该表格展示了三种 USART 特性组合（Full feature, Basic feature, Low-power feature）支持的功能对比。Full feature 组合支持所有功能，如硬件流控、DMA、同步模式、单线半双工、LIN 模式等。Basic feature 保留了主要通信功能，但缺乏如同步模式、双时钟域、LIN 模式等特性。而 Low-power feature 以节能为主，仅支持基础功能（如 DMA、单线模式）和低功耗唤醒能力，FIFO 缓冲区较小（8 字节）。

更多具体说明请查看芯片参考手册。

16.5 开发板 UART 设备

16.5.1 原理图分析

开发板上有个 USB 转 TTL 芯片，型号 CH342F。打开开发板上【正点原子】STM32MP257 开发板(A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 UART。可以看到 UART 相关电路如下。

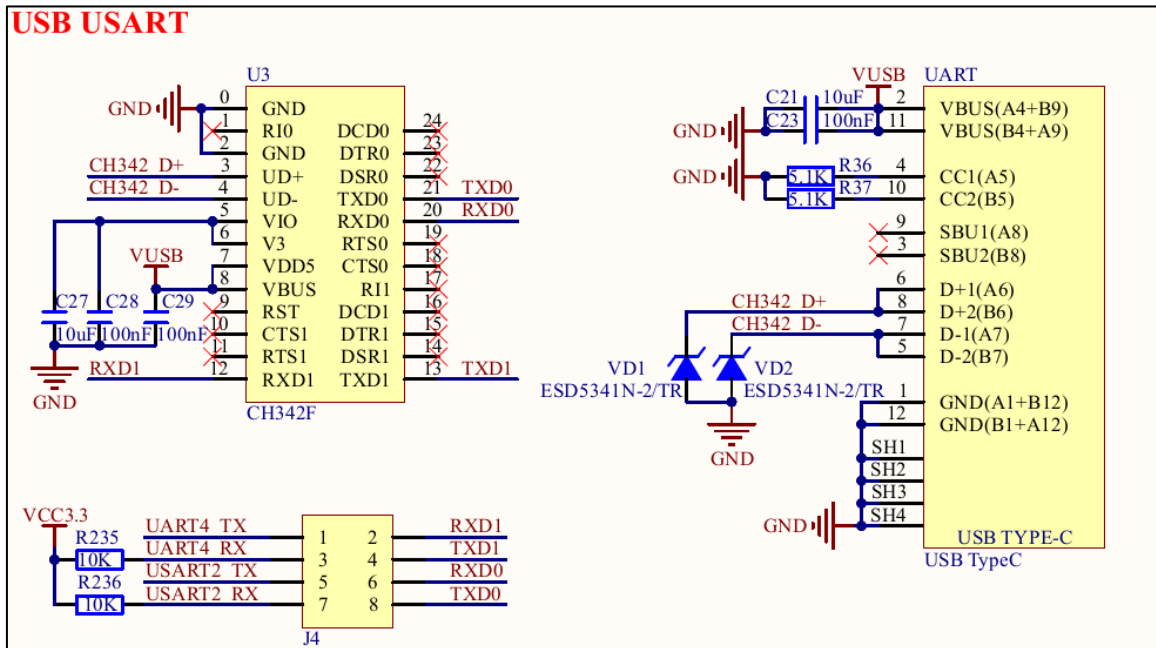


图 16.5-1 开发板 UART 电路

原理图分析：

1、CH342F USB-UART 转换器 (U3)

- 功能: CH342F 是一个 USB 到 UART 的转换芯片, 它将 USB 信号转换为 UART 信号, 用于串行通信。
- 关键引脚:

VBUS、D+、D-: 连接到 USB 端口, 用于传输 USB 信号。

TXD0、RXD0: UART 发送和接收引脚, 用于与 Cortex-A 核心的通信 (USART2)。

TXD1、RXD1: UART 发送和接收引脚, 用于与 Cortex-M 核心的通信 (UART4)。

2、ESD 保护电路 (VD1、VD2、ESD341N-2/TR)

- 功能: 保护 USB 数据线 (D+、D-) 免受静电放电 (ESD) 损害。
- 关键元件:
 - ESD341N-2/TR: 双向 ESD 保护二极管。
 - VD1、VD2: 额外的 ESD 保护二极管, 进一步增强保护。

3、USB Type-C 连接器

- 功能: 提供 USB Type-C 接口, 用于电源输入和数据通信。
- 关键引脚:
 - VBUS: 电源线, 提供 5V 电源。
 - CC1、CC2: 通道配置引脚, 用于确定 USB Type-C 插头的方向和供电角色。
 - SBU1、SBU2: 辅助信号线, 用于某些高级功能 (如音频传输)。
 - D+、D-: USB 数据线。

4、其他元件

- 电容和电阻：用于滤波、限流和稳定电路工作。
- 调试接口 (DEBUG_3V3)：提供 3.3V 电压，用于调试和测试目的。

电路连接和工作原理

1. USB 信号通过 Type-C 接口进入电路，通过 CH342F 转换为 UART 信号。
2. UART 信号被送到 Cortex-A 和 Cortex-M 核心，分别处理 USART2 和 UART4 的通信。
3. ESD 保护电路确保 USB 信号线免受静电损坏，提高系统的可靠性。

这个电路设计的目的是通过 USB 接口实现与嵌入式系统核心的串行通信，同时提供必要的电源和保护措施，确保系统的稳定运行。

16.5.2 实物硬件

开发板上 CH342F 芯片和 USB_TTL 接口位置如下图所示：

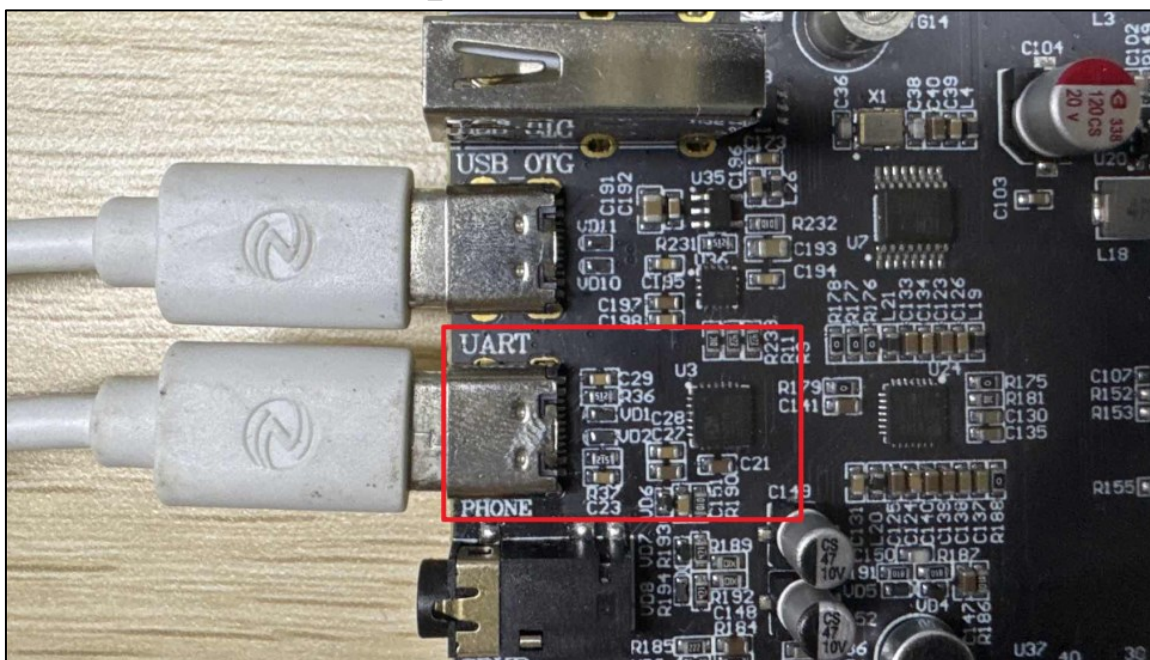


图 16.5-2 开发板 CH342F 和 USB_TTL 位置

开发板 USB_TTL 接口是 Type-C 接口类型，可以使用配套的 Type-C 线接到电脑的 USB 口，CH342F 芯片是一个 USB 到 URT 的转换芯片，它将 USB 信号转换为 UART 信号，用于串行通信。

16.6 内核 UART 子系统使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, 串口 SERIAL 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_SERIAL_8250=y //启用标准的 8250 系列 UART 控制器支持
2 CONFIG_SERIAL_OF_PLATFORM=y //启用平台设备树中的串行控制器支持
3 CONFIG_SERIAL_CORE=y //启用 Linux 内核的串行子系统核心功能
4 CONFIG_SERIAL_CORE_CONSOLE=y //启用将串行端口用作控制台的支持
5 CONFIG_SERIAL_STM32=y //启用 STM32 的 UART/USART 串行驱动程序
6 CONFIG_SERIAL_STM32_CONSOLE=y //将 STM32 的串行端口作为系统的默认控制台
7 CONFIG_SERIAL_DEV_BUS=y //启用了串行设备总线的支持*/
8 CONFIG_SERIAL_DEV_CTRL_TTYPORT=y /*启用了控制台 ttyport 的支持,
    允许用户在控制台中连接到多个 tty 端口*/
```

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

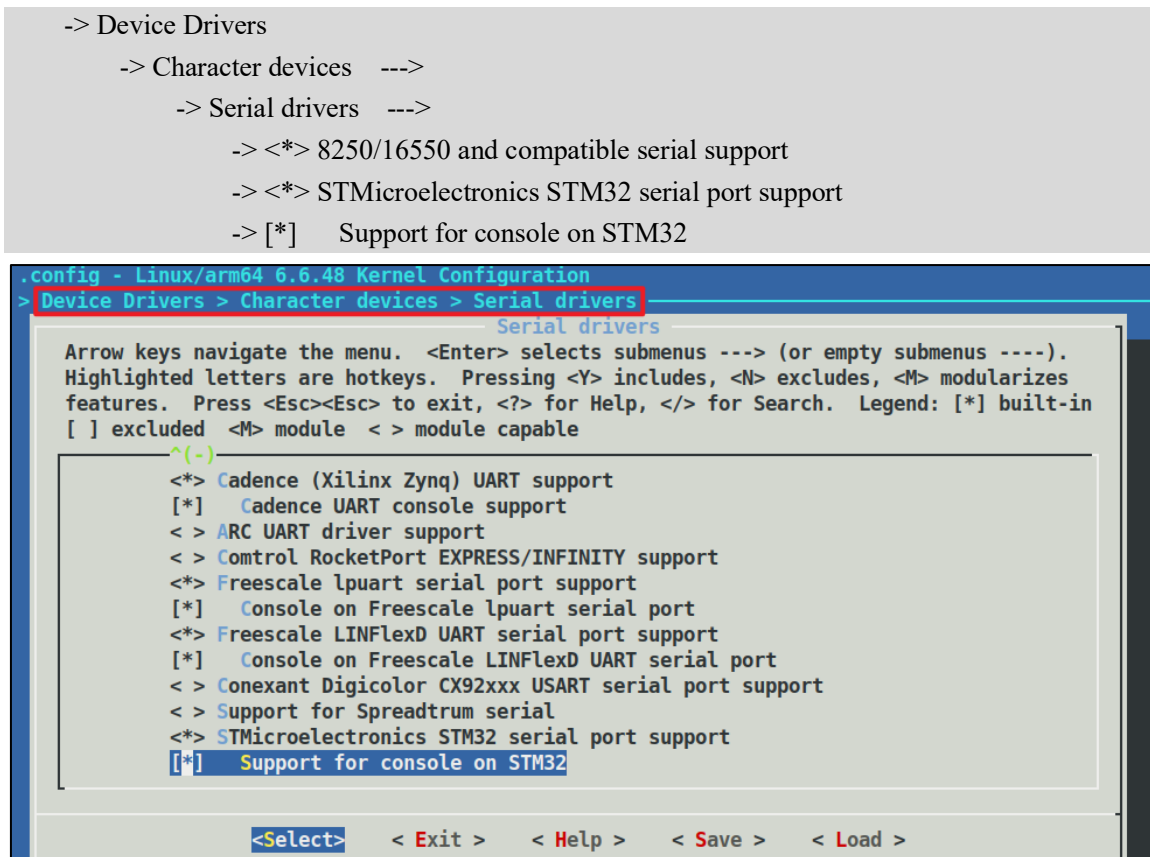


图 16.6-1 serial 配置节选

16.7 CH342F 驱动配置

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, CH342F(实际 CH341 驱动支持 CH342F)相关配置如下:

```
1 CONFIG_USB_SERIAL=m //启用 USB 串行设备支持
2 CONFIG_USB_SERIAL_CH341=m /*启用对 CH341 和 CH342 USB 串行设备的支持*/
3 CONFIG_USB_U_SERIAL=m //启用 USB 连接的串行设备的支持
```

```
4 CONFIG_USB_F_SERIAL=m /*启用 USB Function 类的串行设备支持,
                        允许系统将 USB 设备配置为串行通信功能*/
5 CONFIG_TYPEC=m        //启用 USB Type-C 支持
6 CONFIG_TYPEC_TCPM=m    //启用 USB Type-C 控制器端口管理器 (TCPM) 的支持
```

对应的 menuconfig 配置如下（篇幅有限，截图有所省略）：

```
-> Device Drivers
    -> Character devices --->
        -> [*] USB support --->
            -> <*> USB Serial Converter support --->
                -> <*> USB Winchiphead CH341 Single Port Serial Driver
```

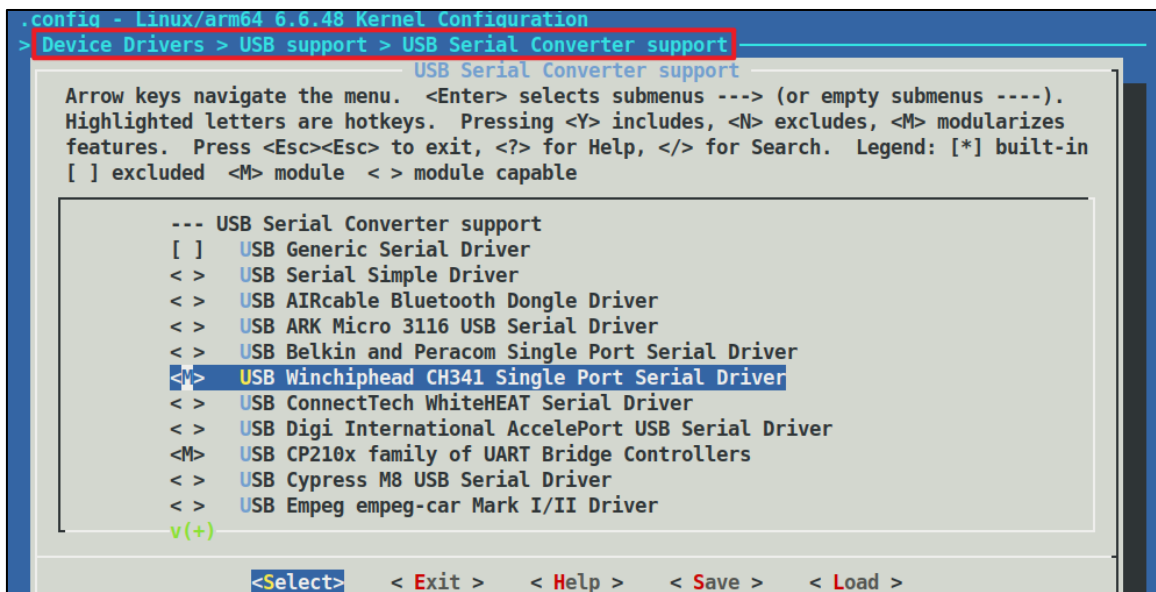


图 16.7-1 CH341 和 CH342 配置

16.8 设备树文件分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi，此文件是特定于 STM32MP2 系列芯片的设备树包含文件，通常不独立使用，而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置，而 .dtsi 则作为它们的公共部分。以 usart2 为例，stm32mp251.dtsi 中关于 UART/USART 的描述如下：

```
1      usart2: serial@400e0000 {
2          compatible = "st,stm32h7-uart";
3          reg = <0x400e0000 0x400>;
4          interrupts-extended = <&exti1 27 IRQ_TYPE_LEVEL_HIGH>;
5          clocks = <&rcc CK_KER_USART2>;
6          dmas = <&hpdma 11 0x20 0x10012>,
7                <&hpdma 12 0x20 0x3021>;
8          dma-names = "rx", "tx";
9          access-controllers = <&rifsc 32>;
10         power-domains = <&CLUSTER_PD>;
11         wakeup-source;
```

```
12         status = "disabled";
13     };
```

第 1 行, `usart2: serial@400e0000` 定义一个设备节点, 命名为 `usart2`。`serial@400e0000` 表示这是一个串行设备, 其寄存器基地址为 `0x400e0000`

第 2 行, `compatible` 用于指定该设备与哪些驱动程序兼容。`"st,stm32h7-uart"` 表示这个设备是 STM32H7 系列芯片的 UART 外设, 说明 MP257 的 UART 沿用的是这个配置。驱动文件路径为 `drivers/tty/serial/stm32-usart.c`

第 3 行, `reg` 定义了设备的寄存器地址和大小。`<0x400e0000 0x400>` 表示基地址是 `0x400e0000`, 寄存器的地址空间大小为 `0x400` 字节。

第 4 行, `interrupts-extended = <&exti1 27 IRQ_TYPE_LEVEL_HIGH>` 定义该设备使用的中断。`<&exti1 27 IRQ_TYPE_LEVEL_HIGH>` 表示中断控制器是 `exti1`, 中断号为 27, 中断类型为电平触发高电平。

第 5 行, `clocks = <&rcc CK_KER_USART2>` 定义该设备使用的时钟。`<&rcc CK_KER_USART2>` 表示时钟由 `rcc` 控制器管理, 时钟信号为 `CK_KER_USART2`。

第 6~7 行, 定义该设备使用的 DMA 通道。`<&hpdma 11 0x20 0x10012>` 表示接收 (RX) 使用 `hpdma` 控制器的第 11 通道。`<&hpdma 12 0x20 0x3021>` 表示发送 (TX) 使用 `hpdma` 控制器的第 12 通道。

第 8 行, `dma-names = "rx", "tx"` 给 DMA 通道命名为 `rx` 和 `tx`, 分别对应接收和发送。

第 9 行, `access-controllers = <&rifsc 32>` 指定该设备使用的访问控制器。`<&rifsc 32>` 表示访问控制器是 `rifsc`, 相关配置为 32。

第 10 行, `power-domains = <&CLUSTER_PD>` 定义设备所属的电源域。`<&CLUSTER_PD>` 表示该设备依赖于 `CLUSTER_PD` 电源域。

第 11 行, 指定该设备支持唤醒功能。`wakeup-source` 表示设备可以从低功耗模式唤醒系统。

第 12 行, 设置设备的初始状态。`"disabled"` 表示该设备默认处于禁用状态。

以 2GB 版本核心板为例, 在 `arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts` 中, `usart2` 相关配置如下:

```
1 / {
2     .....
3     aliases {
4         .....
5         serial0 = &usart2;
6         .....
7     };
8     .....
9     &usart2 {
10         pinctrl-names = "default", "idle", "sleep";
11         pinctrl-0 = <&usart2_pins_a>;
12         pinctrl-1 = <&usart2_idle_pins_a>;
13         pinctrl-2 = <&usart2_sleep_pins_a>;
14         /delete-property/dmas;
15         /delete-property/dma-names;
16         status = "okay";
```



```
17 };
18 .....
19 };
```

第 1 行, 表示设备树的根节点, 所有设备节点都在其下定义。

第 3 行, 定义 aliases 节点, 用于为设备节点创建别名, 便于引用。

第 5 行, 将 usart2 节点设置为别名 serial0, 用于表示系统的第一个串口设备, 这个是系统调试串口。

第 9 行, 引用 usart2 节点并对其进行补充配置。

第 10 行, 为 usart2 节点定义引脚控制(pinctrl)状态名称: "default": 正常工作状态。"idle": 空闲状态。"sleep": 睡眠状态。

第 11 行, 指定 usart2 节点在 "default" 状态时的引脚设置, 引用 usart2_pins_a。

第 12 行, 指定 usart2 节点在 "idle" 状态时的引脚设置, 引用 usart2_idle_pins_a。

第 13 行, 指定 usart2 节点在 "sleep" 状态时的引脚设置, 引用 usart2_sleep_pins_a。

第 14 行, 删除之前 usart2 节点中定义的 dmas 属性。

第 15 行, 删除之前 usart2 节点中定义的 dma-names 属性。

第 16 行, 将 usart2 节点的状态设置为 "okay", 表示启用该设备。

16.9 系统接口

16.9.1 /dev/ttySTM*

在嵌入式 Linux 系统中, /dev/ttySTM* 设备节点通常表示串口设备。

```
root@ATK-DLMP257:~# ls /dev/ttySTM*
/dev/ttySTM0 /dev/ttySTM1 /dev/ttySTM2
root@ATK-DLMP257:~#
```

图 16.9-1 开发板出厂系统串口设备

16.9.2 /sys/class/tty/ttySTM*

从 /sys/class/tty/ttySTM*路径下列出的信息可以进一步确认和了解这些串口设备的详细属性。

```
root@ATK-DLMP257:~# ls /sys/class/tty/ttySTM*
/sys/class/tty/ttySTM0:
close_delay  custom_divisor  flags            iomem_reg_shift  port            type            xmit_fifo_size
closing_wait dev              io_type          irq              power           uartclk         uevent
console      device          iomem_base       line             subsystem       uevent
/sys/class/tty/ttySTM1:
close_delay  custom_divisor  flags            iomem_reg_shift  port            type            xmit_fifo_size
closing_wait dev              io_type          irq              power           uartclk         uevent
console      device          iomem_base       line             subsystem       uevent
/sys/class/tty/ttySTM2:
close_delay  custom_divisor  flags            iomem_reg_shift  port            type            xmit_fifo_size
closing_wait dev              io_type          irq              power           uartclk         uevent
console      device          iomem_base       line             subsystem       uevent
root@ATK-DLMP257:~#
```

图 16.9-2 开发板出厂系统串口设备属性

下面是这些属性的含义:

关键属性解析

- close_delay 和 closing_wait:

这些文件控制串口关闭时的延迟设置, 单位为毫秒。

- dev:



这个文件包含设备的主次设备号。例如，241:0 表示主设备号为 241，次设备号为 0。

- device:

指向该串口设备对应的硬件设备目录。

- flags:

显示串口设备的标志状态。

- irq:

显示分配给该串口设备的中断请求号（IRQ）。

- port:

显示串口设备的 I/O 端口地址。

- uartclk:

显示 UART 时钟频率。

- type:

显示串口类型。

确认具体硬件设备

- 查看设备的主次设备号。例如:

```
cat /sys/class/tty/ttySTM0/dev
```

```
root@ATK-DLMP257:~# cat /sys/class/tty/ttySTM0/dev
241:0
root@ATK-DLMP257:~#
```

图 16.9-3 查看设备的主次设备号

输出类似 241:0，表示主设备号和次设备号。

- 查看串口设备的 IRQ 号

```
cat /sys/class/tty/ttySTM0/irq
```

```
root@ATK-DLMP257:~# cat /sys/class/tty/ttySTM0/irq
80
root@ATK-DLMP257:~#
```

图 16.9-4 查看设备中断号

- 可以通过查看 device 文件夹进一步确认串口设备对应的具体硬件设备。例如:

```
ls -l /sys/class/tty/ttySTM0/device
```

这个命令会展示串口设备 ttySTM0 关联的硬件设备信息，包括设备树路径或平台设备信息。

```
root@ATK-DLMP257:~# ls -l /sys/class/tty/ttySTM0/device
lrwxrwxrwx 1 root root 0 Feb 28 04:03 /sys/class/tty/ttySTM0/device -> ../../../../400e0000.serial
root@ATK-DLMP257:~#
```

图 16.9-5 查看串口设备关联的硬件设备

从结果来看，/sys/class/tty/ttySTM0/device 链接到 ../../../../400e0000.serial，这表示 ttySTM0 对应的硬件设备路径是 /sys/devices/platform/soc@0/42080000.bus/400e0000.serial。可以通过这个路径进一步了解硬件设备的详细信息。

16.9.3 /sys/devices/platform/xxx

/sys/class/tty/ttySTM0/device 链接到 ../../../../400e0000.serial，这表示 ttySTM0 对应的硬件设备路径是 /sys/devices/platform/soc@0/42080000.bus/400e0000.serial。

- 查看设备属性:

```
ls -l /sys/devices/platform/soc@0/42080000.bus/400e0000.serial
```

```
root@ATK-DLMP257:~# ls -l /sys/devices/platform/soc@0/42080000.bus/400e0000.serial/
total 0
drwxr-xr-x 4 root root    0 Feb 28 04:09 400e0000.serial:0
lrwxrwxrwx 1 root root    0 Feb 28 04:09 driver -> ../../../../bus/platform/drivers/stm32-usart
-rw-r--r-- 1 root root 4.0K Feb 28 04:09 driver_override
-r--r--r-- 1 root root 4.0K Feb 28 04:09 modalias
lrwxrwxrwx 1 root root    0 Feb 28 04:09 of_node -> ../../../../firmware/devicetree/base/soc@0/bus@42080000/serial@400e0000
drwxr-xr-x 2 root root    0 Feb 28 04:09 power
lrwxrwxrwx 1 root root    0 Feb 28 04:09 subsystem -> ../../../../bus/platform
lrwxrwxrwx 1 root root    0 Feb 28 04:09 supplier:platform:42080000.bus -> ../../../../virtual/devlink/platform:42080000.bus--platform:400e0000.serial
lrwxrwxrwx 1 root root    0 Feb 28 04:09 supplier:platform:44200000.clock-controller -> ../../../../virtual/devlink/platform:44200000.clock-controller--platform:400e0000.serial
lrwxrwxrwx 1 root root    0 Feb 28 04:09 supplier:platform:44220000.interrupt-controller -> ../../../../virtual/devlink/platform:44220000.interrupt-controller--platform:400e0000.serial
lrwxrwxrwx 1 root root    0 Feb 28 04:09 supplier:platform:psci -> ../../../../virtual/devlink/platform:psci--platform:400e0000.serial
lrwxrwxrwx 1 root root    0 Feb 28 04:09 supplier:platform:soc@0:pinctrl@44240000 -> ../../../../virtual/devlink/platform:soc@0:pinctrl@44240000--platform:400e0000.serial
drwxr-xr-x 3 root root    0 Feb 28 2024 tty
-rw-r--r-- 1 root root 4.0K Feb 28 04:09 uevent
root@ATK-DLMP257:~#
```

图 16.9-6 查看设备属性

从列出的文件和目录可以获得 400e0000.serial 设备的详细信息。这些文件和目录帮助进一步了解设备的属性和配置。

重要文件和目录说明

driver: 这是一个符号链接，指向串口设备的驱动程序目录。在这里，它指向 stm32-usart 驱动程序，表示这个设备使用该驱动程序。

driver_override: 这个文件允许你覆盖默认的设备驱动程序。

modalias: 这个文件包含设备的模块别名，可以用于加载合适的内核模块。

of_node: 这是一个符号链接，指向设备树中的对应节点。在这里，它指向 firmware/devicetree/base/soc@0/bus@42080000/serial@400e0000，这是该串口设备在设备树中的位置。

power: 这个目录包含与设备电源管理相关的文件。

subsystem: 这是一个符号链接，指向设备所属的子系统目录。

supplier:platform:42080000.bus 和 **supplier:platform:44200000.clock-controller:** 这些符号链接指向设备的供应商信息，分别为引脚控制器和时钟控制器。

tty: 这个目录包含与串口相关的具体设备节点。

uevent: 这个文件包含用于用户空间的设备事件信息。

● 设备节点信息

可以通过查看 of_node 目录获取设备树中该节点的更多信息：

```
ls -l /sys/firmware/devicetree/base/soc@0/bus@42080000/serial@400e0000
```

```
root@ATK-DLMP257:~# ls -l /sys/firmware/devicetree/base/soc@0/bus@42080000/serial@400e0000/
total 0
-r--r--r-- 1 root root 8 Feb 28 04:14 access-controllers
-r--r--r-- 1 root root 8 Feb 28 04:14 clocks
-r--r--r-- 1 root root 16 Feb 28 04:14 compatible
-r--r--r-- 1 root root 12 Feb 28 04:14 interrupts-extended
-r--r--r-- 1 root root 7 Feb 28 04:14 name
-r--r--r-- 1 root root 4 Feb 28 04:14 phandle
-r--r--r-- 1 root root 4 Feb 28 04:14 pinctrl-0
-r--r--r-- 1 root root 4 Feb 28 04:14 pinctrl-1
-r--r--r-- 1 root root 4 Feb 28 04:14 pinctrl-2
-r--r--r-- 1 root root 19 Feb 28 04:14 pinctrl-names
-r--r--r-- 1 root root 4 Feb 28 04:14 power-domains
-r--r--r-- 1 root root 8 Feb 28 04:14 reg
-r--r--r-- 1 root root 5 Feb 28 04:14 status
-r--r--r-- 1 root root 0 Feb 28 04:14 wakeup-source
root@ATK-DLMP257:~#
```

图 16.9-7 查看设备树节点信息

16.10 USART 调试工具

16.10.1 dmesg

Linux 内核通过串口输出日志消息, 可以使用 `dmesg` 命令来查看这些消息。这对于调试引导过程中的问题非常有用, 也可以用于检查内核模块加载和设备初始化过程中的消息。`grep` 是一个 Linux/Unix 下的命令行工具, 用于在文本文件中搜索指定的字符串模式, 并将包含匹配的行输出到标准输出 (通常是终端)。

示例:

`dmesg | grep usart`

```
root@ATK-DLMP257:~# dmesg | grep usart
[ 2.622061] stm32-usart 40330000.serial: Wakeup might not be available: No autonomous clock
[ 2.628977] 40330000.serial: ttySTM1 at MMIO 0x40330000 (irq = 62, base_baud = 1000000) is a stm32-usar
t
[ 2.638611] stm32-usart 40370000.serial: Wakeup might not be available: No autonomous clock
[ 2.646301] 40370000.serial: ttySTM2 at MMIO 0x40370000 (irq = 63, base_baud = 4000000) is a stm32-usar
t
[ 3.541483] stm32-usart 400e0000.serial: Wakeup might not be available: No autonomous clock
[ 3.552541] stm32-usart 400e0000.serial: interrupt mode for rx (no dma)
[ 3.563526] stm32-usart 400e0000.serial: interrupt mode for tx (no dma)
[ 3.578944] 400e0000.serial: ttySTM0 at MMIO 0x400e0000 (irq = 80, base_baud = 4000000) is a stm32-usar
t
root@ATK-DLMP257:~#
```

图 16.10-1 dmesg

在给定的示例中, `dmesg | grep usart` 命令将 `dmesg` 命令的输出通过管道传递给 `grep` 命令进行搜索, 搜索字符串模式为 "usart", 然后输出包含 "usart" 字符串的行。

16.10.2 stty

`stty` 是一个用于设置和显示终端设备参数的命令行工具。它可以用于配置串行设备 (如串口) 的参数, 也可以用于配置终端窗口 (如终端的大小和行为) 的参数。

`stty`

```
root@ATK-DLMP257:~# stty
speed 115200 baud; line = 0;
-brkint ixoff iutf8
-iexten
root@ATK-DLMP257:~#
```

图 16.10-2 stty 指令

`stty` 命令的输出结果显示了当前终端设备 (通常是串口) 的参数设置。这些设置的含义:

- `speed 115200 baud`: 串口的波特率被设置为 115200 比特每秒, 这是数据传输速率的单位, 用于测量串口通信的速度。
- `line=0`: 该参数设置指示终端行为。在这种情况下, `line` 设置为 0, 表示终端行为可能未进行设置或默认设置。
- `-brkint ixoff iutf8`: 这部分显示了一些终端特性的设置:
- `-brkint`: 禁用了 BREAK 字符 (Ctrl+C) 的作用, 即 BREAK 字符不会引起终端驱动程序的中断操作。
- `ixoff`: 禁用了输入流控制, 即禁用了通过发送 XOFF 和 XON 控制串口的输入流的功能。
- `iutf8`: 启用了 UTF-8 输入字符编码。
- `-iexten`: 禁用了输入扩展特性。这意味着输入字符的扩展功能被禁用。

这些设置描述了当前终端设备的行为和特性, 这些特性将影响终端设备与其他设备 (如计算机或外部设备) 之间的通信和交互。

stty 指令帮助信息:

stty --help

```

root@ATK-DLMP257:~# stty --help
Usage: stty [-F DEVICE | --file=DEVICE] [SETTING]...
or: stty [-F DEVICE | --file=DEVICE] [-a|--all]
or: stty [-F DEVICE | --file=DEVICE] [-g|--save]
Print or change terminal characteristics.

Mandatory arguments to long options are mandatory for short options too.
-a, --all          print all current settings in human-readable form
-g, --save         print all current settings in a stty-readable form
-F, --file=DEVICE  open and use the specified DEVICE instead of stdin
--help            display this help and exit
--version          output version information and exit

Optional - before SETTING indicates negation. An * marks non-POSIX
settings. The underlying system defines which settings are available.

Special characters:
* discard CHAR    CHAR will toggle discarding of output
eof CHAR         CHAR will send an end of file (terminate the input)
eol CHAR         CHAR will end the line
* eol2 CHAR       alternate CHAR for ending the line
erase CHAR       CHAR will erase the last character typed
intr CHAR        CHAR will send an interrupt signal
kill CHAR        CHAR will erase the current line
* lnext CHAR      CHAR will enter the next character quoted
quit CHAR        CHAR will send a quit signal
* rprnt CHAR      CHAR will redraw the current line
start CHAR       CHAR will restart the output after stopping it
stop CHAR        CHAR will stop the output
susp CHAR        CHAR will send a terminal stop signal
* swtch CHAR      CHAR will switch to a different shell layer
* werase CHAR     CHAR will erase the last word typed

```

以下省略

图 16.10-3 stty --help 节选

这是 stty 命令的帮助信息，其中列出了该命令的用法和可用选项，以及各个选项的含义和作用，内容过长，截图省略后面部分的说明。以下是一些常用选项的解释：

- -a, --all: 打印当前终端设备的所有参数设置，以人类可读的形式显示。
- -g, --save: 打印当前终端设备的所有参数设置，以 stty 可读的格式显示。
- -F, --file=DEVICE: 指定要操作的终端设备文件，而不是标准输入。默认情况下，stty 会操作标准输入的终端设备。
- speed: 设置或显示终端的波特率。
- csN: 设置字符大小，其中 N 可以是 5 到 8 之间的一个整数，表示字符位数。
- rows 和 cols: 告诉内核终端的行数和列数。
- ispeed 和 ospeed: 分别设置输入和输出的波特率。
- -icanon: 禁用规范模式，即在输入一行数据后立即交给程序处理，而不是等待换行符。
- -echo: 关闭回显，即将不将用户的输入显示在终端上。
- -ixoff 和 -ixon: 分别启用和禁用 XON/XOFF 流控制。
- -parenb: 禁用奇偶校验，不生成奇偶校验位。
- -iutf8: 假定输入字符是 UTF-8 编码。

这些选项和参数可以用于设置和显示终端设备的各种参数，以满足特定的需求和场景。

配置串口示例:

```
stty -F /dev/ttySTM0 ispeed 115200 ospeed 115200 cs8
```

这条命令设置了串口 /dev/ttySTM0 的输入速度 (ispeed) 和输出速度 (ospeed) 为 115200 比特每秒，并将字符大小 (cs) 设置为 8 位。这些参数设置通常用于配置串口的基本通信参数，以确保串口与其他设备正确通信。

具体来说:

ispeed 115200: 设置输入速度 (波特率) 为 115200 比特每秒，表示串口接收数据的速率。

ospeed 115200: 设置输出速度 (波特率) 为 115200 比特每秒，表示串口发送数据的速率。

cs8: 将字符大小设置为 8 位, 表示每个字符由 8 位组成, 这是常见的串口通信配置之一, 表示每个字符有 8 个数据位。

通过这些设置, 串口 /dev/ttySTM0 将以 115200 波特率进行双向通信, 并使用每个字符 8 位的数据位。这通常是在串口通信中常见的标准设置, 但具体设置需根据实际需求和串口设备的要求进行调整。

16.10.3 minicom

minicom 是一个基于文本的串口通信程序, 广泛用于在 Linux 和其他类 Unix 操作系统中与串口设备进行通信, 用于设置、控制和监视串行端口。由于 /dev/ttySTM0 是调试串口, 不方便使用 minicom 演示, minicom 的使用请参考 RS485 章节。

第十七章 RS232/RS485 子系统

17.1 RS485 子系统简介

RS232 是一种 串行通信接口标准, 由 EIA (美国电子工业协会) 在 1960 年制定, 广泛用于计算机和通信设备之间的短距离通信, 适用于短距离、低速点对点通信场景, 如调试接口。

RS485 是一种 工业串行通信标准, 由 EIA 在 1983 年制定, 适合长距离、多节点通信, 常见于工业控制系统, 适用于长距离、高抗干扰性、多设备连接场景, 如工业总线。

17.2 主要特点

嵌入式 Linux 的 RS232、RS485 子系统主要有以下几个特点:

特性	RS232	RS485
传输方式	点对点(一个发送端,一个接收端)	多点通信 (支持一个主机与多个从机)
信号电平	$\pm 3V$ 到 $\pm 15V$ (典型值 $\pm 12V$)	差分信号(+5V 和 0V, 或 $\pm 2V$ 及以上)
通信距离	最长约 15 米 (依赖于波特率)	最长约 1200 米(波特率低于 100 kbps 时)
抗干扰性	较弱	强, 适用于工业环境
全/半双工	全双工(独立的发送和接收通道)	半双工(同一对线进行发送和接收需换方向)
应用场景	短距离通信, 如调试、控制设备	长距离、多节点工业总线通信(Modbus 等)

17.3 组成部分

嵌入式 Linux 的 RS232、RS485 子系统由多个组成部分构成, 这些部分共同协作, 实现 RS232、RS485 通信的配置和操作。以下是 RS232、RS485 子系统的主要组成部分:

1、硬件支持

- 串行接口 (UART)

嵌入式系统中的 UART (通用异步收发传输器) 接口是实现 RS232、RS485 通信的基础。UART 接口负责将数据转换为串行格式并通过物理连接进行传输。

- RS232、RS485 收发器

RS232 收发器 (如 MAX232)、RS485 收发器 (如 MAX485) 用于将 UART 的 TTL 电平信号转换为 RS232、RS485 的差分信号。收发器还负责控制数据的发送和接收, 确保正确的信号传输。

2、内核驱动

- 串行驱动程序

Linux 内核中的串行驱动程序 (如 8250 系列驱动) 提供对 UART 接口的支持。这些驱动程序负责初始化串行端口, 配置波特率、数据位、停止位和校验位等参数, 并处理数据的发送和接收。RS232 通常通过 UART 驱动支持。RS485 也是 UART 驱动, 但需要额外的 RS485 支持, 通过 `ioctl` 配置控制方向。

- RS485 特性支持

内核驱动程序需要支持 RS485 特性, 通过 `ioctl` 系统调用配置 RS485 模式。典型的配置包括启用 RS485 模式、设置 RTS (请求发送) 延迟等。

3、配置接口

- `ioctl` 系统调用

ioctl 系统调用用于配置串行端口的 RS232、RS485 特性。用户可以通过该接口设置和获取 RS232、RS485 相关参数，如 RTS 控制、延迟时间等。

4、用户空间工具和库

- **termios 库**

termios 库提供了一组函数用于配置和控制串行端口参数，包括波特率、数据位、停止位和流控制等。开发人员可以使用 termios 库配置串行端口以支持 RS232、RS485 通信。

通信工具

minicom、picocom 等用户空间工具用于与 RS232、RS485 设备进行交互和调试。它们提供了便捷的界面，方便用户配置串行端口和发送接收数据。

5、应用程序

应用程序通过上述用户空间工具和库与 RS232、RS485 设备进行通信。这些应用程序可以是专用的调试工具、设备配置工具或实际的业务逻辑实现。

总结来说，嵌入式 Linux 的 RS232、RS485 子系统由硬件支持、内核驱动、配置接口、用户空间工具和库以及应用程序组成。这些组成部分共同协作，实现对 RS232、RS485 通信的全面支持。

17.4 开发板 RS232 设备

打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 RS232。可以看到 RS232 相关电路如下。

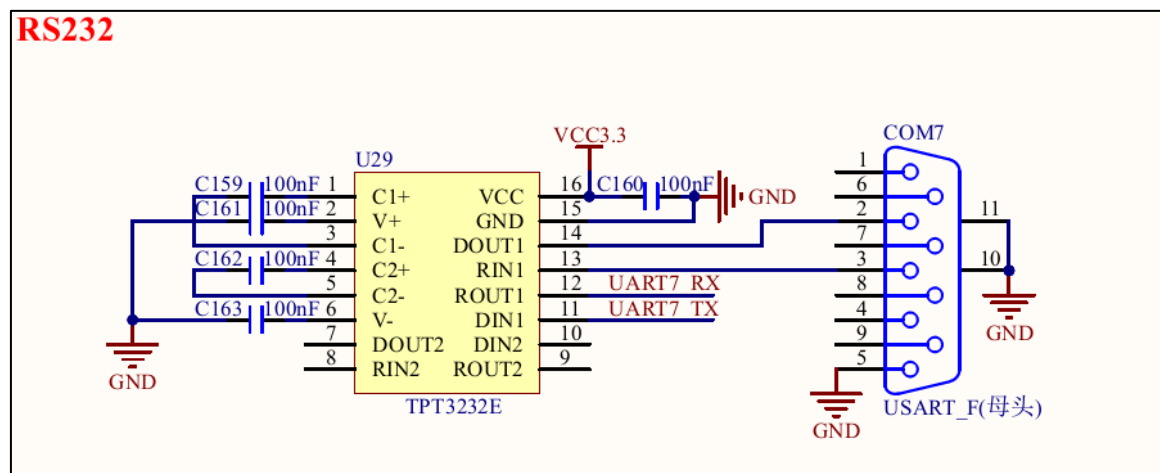


图 17.4-1 开发板 RS232 原理图

以下是对电路的详细解析：

1、RS232 转换芯片

- **芯片名称：TPT2322E**
 - 用于将 UART 的 TTL（低电平）信号转换为 RS232（高电平）信号。
 - 提供双通道（两个发送和两个接收）。
- **主要引脚说明：**
 - C1+ / C1- / C2+ / C2-：用于内部电荷泵电容。通过外部电容（如 C159-C163）提供电压转换所需的电荷。
 - VCC / GND：电源和地，供电电压为 3.3V。
 - DIN1 / DIN2：输入 TTL 信号（来自 UART 的 TX 引脚）。
 - DOUT1 / DOUT2：输出 RS232 信号（连接到 DB9 接口的 RX 引脚）。

- RIN1 / RIN2: 输入 RS232 信号 (来自 DB9 接口的 TX 引脚)。
- ROUT1 / ROUT2: 输出 TTL 信号 (连接到 UART 的 RX 引脚)。

2、UART 与 RS232 的信号连接

- UART_TX:
 - 从 MCU (UART7) 发送的 TTL 信号, 接入芯片的 DIN1 引脚。
 - 转换为 RS232 信号后, 从 DOUT1 引脚输出, 连接到 DB9 接口的 2 脚 (RS232_RX)。
- UART_RX:
 - 从 DB9 接口的 3 脚 (RS232_TX) 输入的 RS232 信号, 接入芯片的 RIN1 引脚。
 - 转换为 TTL 信号后, 从 ROUT1 引脚输出, 连接到 MCU (UART7) 的 RX 引脚。

3、电容

- 作用:
 - C159-C163: 电荷泵电容, 支持 RS232 电平的转换。
 - C160: 电源去耦电容, 用于滤波电源噪声, 确保芯片稳定运行。
- 电容值:
 - 电荷泵电容: 通常为 100nF (如图中标注)。
 - 去耦电容: 100nF (如图中标注)。

4、DB9 接口 (COM7)

- 引脚说明:
 - 引脚 2: RX (接收信号), 连接芯片的 DOUT1。
 - 引脚 3: TX (发送信号), 连接芯片的 RIN1。
 - 引脚 5: GND (地), 连接系统地。
 - 其他引脚: 未使用。

5、电源部分

- 供电电压: VCC 为 3.3V, 适用于芯片 TPT2322E。
- 电源地: GND 与整个系统共享。

6、电路工作原理

- TTL 到 RS232 转换:
 - MCU 通过 UART7 发送的 TTL 电平信号 (3.3V) 进入芯片的 DIN1。
 - TPT2322E 将信号转换为 RS232 电平信号 ($\pm 3V \sim \pm 15V$), 从 DOUT1 输出到 DB9 接口的 RX (引脚 2)。
- RS232 到 TTL 转换:
 - 从 DB9 接口的 TX (引脚 3) 接收的 RS232 电平信号进入芯片的 RIN1。
 - TPT2322E 将信号转换为 TTL 电平信号, 从 ROUT1 输出到 MCU 的 UART7 RX。
- 电荷泵工作:

电荷泵电容 (C159-C163) 通过芯片内部电路生成 RS232 所需的正负电压。

这是一种典型的 RS232 转 TTL 电路, 基于 TPT2322E 芯片实现。其功能是将嵌入式设备的 UART (TTL) 信号转换为 RS232 信号, 以便与传统的串口设备 (如调试终端、PC 等) 通信。通过 UART 的 RX 和 TX 引脚与芯片配合, 再加上电容网络和 DB9 接口, 完成整个信号转换和传输。

17.5 开发板 RS485 设备

17.5.1 原理图分析

打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 RS485。可以看到 RS485 相关电路如下。

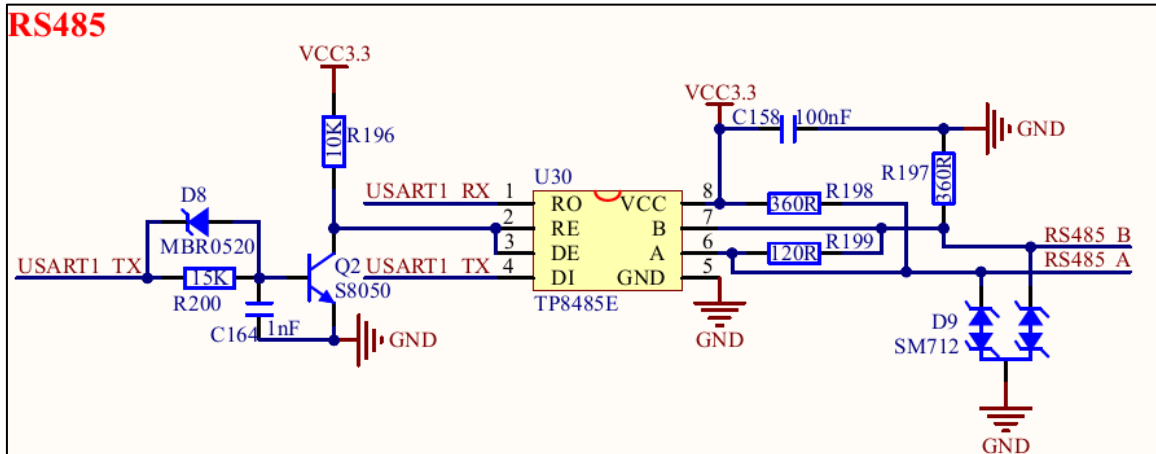


图 17.5-1 开发板 RS485 原理图

以下是对电路的详细解析：

1、USART1 TX 发送端电路

- R200 (15K) 和 C164 (1nF)：用于保护和滤波，限制高频干扰进入发送路径。
- Q2 (S8050)：NPN 三极管，用作逻辑电平控制。当 USART1 TX 为高电平时，Q2 导通，DE 和 DI 信号被发送到 RS485 芯片的输入端。反之，Q2 截止，USART1 TX 信号被隔离。
- D8 (MBR0520)：用于保护电路，防止反向电流损坏元器件。

2、USART1 RX 接收端电路

- RO (TP485E 芯片的 1 脚)：输出 RS485 芯片接收到的数据，连接到 USART1 RX 引脚。数据从 RS485 总线 (RS485 A 和 RS485 B) 通过差分接收，经过内部放大和转换后输出到 USART1 RX。

3、RS485 芯片 (TP485E) 功能模块

- RO (1 脚)：接收输出，用于将 RS485 总线的数据发送到 RX 引脚。
- RE (2 脚)：接收使能，低电平使能接收功能。
- DE (3 脚)：发送使能，高电平使能发送功能。
- DI (4 脚)：发送输入，从 TX 引脚接收数据。
- A (6 脚) 和 B (7 脚)：RS485 差分总线接口。
- VCC (8 脚)：电源，连接 3.3V。
- GND (5 脚)：接地。

4、差分信号与保护电路

- R199 (120R)：连接在 A 和 B 差分信号线之间，作为终端匹配电阻，防止信号反射。
- R198 (360R) 和 R197 (560R)：差分电阻，用于限制差分信号的电流大小，保护 RS485 芯片。
- D9 (SM712)：TVS (二极管阵列) 保护电路，防止静电和瞬态电压对差分线造成损害。

5、电源部分

- C158 (100nF): 电源旁路电容, 用于滤除电源上的高频噪声, 保证芯片供电稳定。
- R196 (10K): 上拉电阻, 确保 RE 或 DE 的初始状态。

6、RS485 总线接口

- RS485 A 和 RS485 B: 差分总线接口, 用于连接 RS485 总线, 支持多设备通信。A 和 B 之间的信号电压差决定逻辑状态。

17.5.2 实物硬件

开发板上 RS485 接口位置如下图所示:

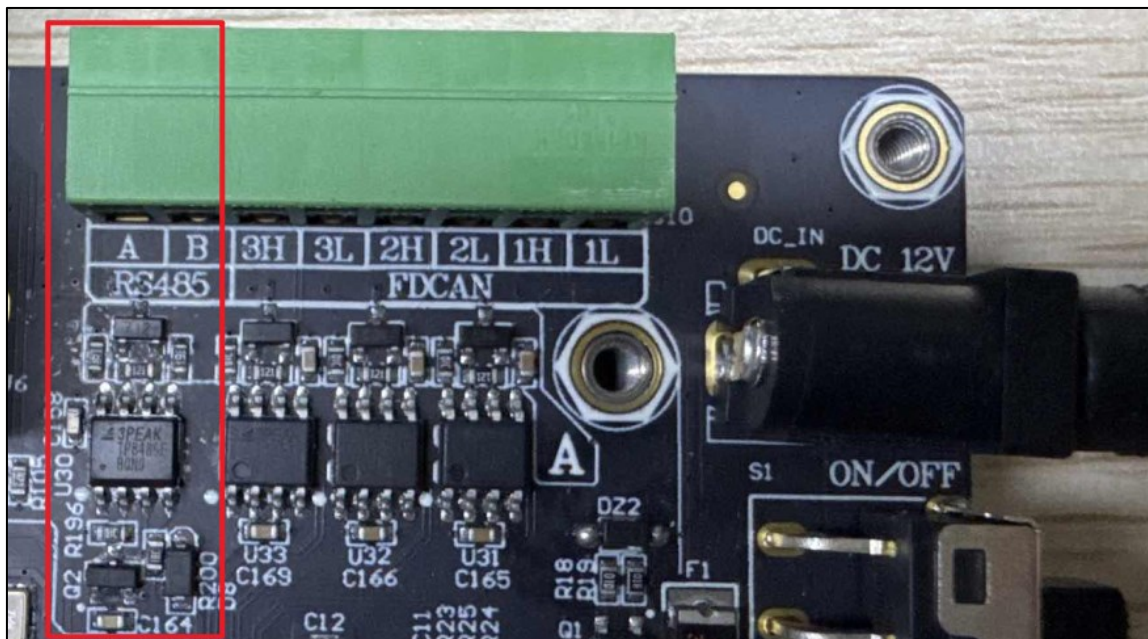


图 17.5-2 开发板 RS485 接口硬件实物

17.6 内核 RS23、RS485 子系统使能

参考 16.6 小节。

17.7 系统接口

在嵌入式 Linux 系统中, /dev/ttySTM* 设备节点通常表示串口设备。

```
root@ATK-DLMP257:~# ls /dev/ttySTM*
/dev/ttySTM0 /dev/ttySTM1 /dev/ttySTM2
root@ATK-DLMP257:~#
```

图 17.7-1 开发板出厂系统串口设备

串口设备对应外设信息如下表所示:

串口设备	开发板外设
/dev/ttySTM0	USB_TTL
/dev/ttySTM1	RS485
/dev/ttySTM2	RS232

17.8 RS232、RS485 调试工具

17.8.1 stty

stty 是一个用于设置和显示终端设备参数的命令行工具。它可以用于配置串行设备（如串口）的参数，也可以用于配置终端窗口（如终端的大小和行为）的参数。具体使用方法请参考 16.10.2 小节。

17.8.2 minicom

minicom 是一个基于文本的串口通信程序，广泛用于在 Linux 和其他类 Unix 操作系统中与串口设备进行通信，用于设置、控制和监视串行端口。

主要功能

- 串口配置：
设置串口的波特率、数据位、停止位、校验位等参数。
- 数据传输：
发送和接收数据。可以用于调试串口设备、下载和上传文件等。
- 脚本支持：
支持脚本功能，允许自动化串口通信任务。
- 日志功能：
记录通信数据到文件，便于后续分析。

配置和使用 minicom

在开发板中输入“minicom -s”，打开 minicom 配置界面。

minicom -s



图 17.8-1 minicom 配置界面

然后选中“Serial port setup”

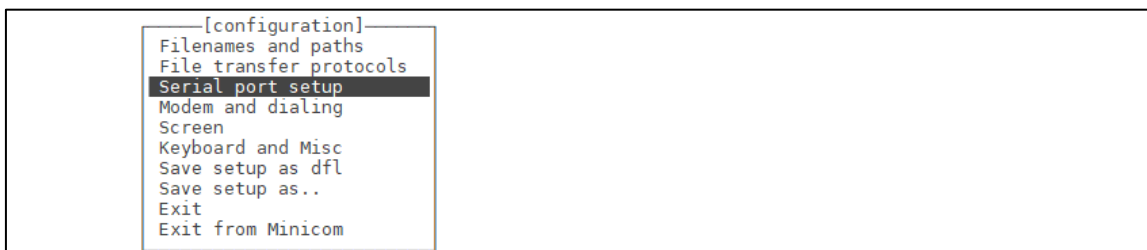


图 17.8-2 光标选中“Serial port setup”

选中后按回车键进入配置界面，默认界面如下：

```
A - Serial Device      : /dev/ttyS1
B - Lockfile Location  : /var/lock
C - Callin Program    :
D - Callout Program    :
E - Bps/Par/Bits       : 115200 8N1
F - Hardware Flow Control : No
G - Software Flow Control : No
H - RS485 Enable       : No
I - RS485 Rts On Send  : No
J - RS485 Rts After Send : No
K - RS485 Rx During Tx : No
L - RS485 Terminate Bus : No
M - RS485 Delay Rts Before: 0
N - RS485 Delay Rts After : 0

Change which setting? |
```

图 17.8-3 Serial port setup 默认界面

1、修改串口设备

按下 A 键以修改串口设备，输入设备节点，比如 RS485 对应/dev/ttySTM1，RS232 对应/dev/ttySTM2，然后按 Enter 键确认。

```
A - Serial Device      : /dev/ttySTM1
B - Lockfile Location  : /var/lock
C - Callin Program    :
D - Callout Program    :
E - Bps/Par/Bits       : 115200 8N1
F - Hardware Flow Control : No
G - Software Flow Control : No
H - RS485 Enable       : No
I - RS485 Rts On Send  : No
J - RS485 Rts After Send : No
K - RS485 Rx During Tx : No
L - RS485 Terminate Bus : No
M - RS485 Delay Rts Before: 0
N - RS485 Delay Rts After : 0

Change which setting? |
```

图 17.8-4 修改串口设备

2、修改波特率、校验位和数据位

按下 E 键修改波特率、校验位和数据位。设置为 115200 8N1(默认是这个就不需要改):

```
[Comm Parameters]
A - Serial De          Current: 115200 8N1
B - Lockfile Loc
C - Callin Pro         Speed      Parity    Data
D - Callout Pro        A: <next>   L: None   S: 5
E - Bps/Par/B          B: <prev>   M: Even   T: 6
F - Hardware Flo       C: 9600     N: Odd    U: 7
G - Software Flo       D: 38400    O: Mark   V: 8
H - RS485 En          E: 115200   P: Space
I - RS485 Rts
J - RS485 Rts A        Stopbits
K - RS485 Rx Du        W: 1        Q: 8-N-1
L - RS485 Termi        X: 2        R: 7-E-1
M - RS485 Delay
N - RS485 Delay

Choice, or <Enter> to exit? |

Change which
```

图 17.8-5 修改串口配置

3、硬件流控制和软件流控制

一般情况硬件流控制和软件流控制均设置为 No，有流控需求则根据自己需求设置:

```
F - Hardware Flow Control : No
G - Software Flow Control : No
```

图 17.8-6 硬件/软件流控

4、RS485 配置

RS485 Enable (H): 启用 RS485 模式。如果使用 RS485 通信协议, 这个选项必须设置为 Yes。

RS485 Rts On Send (I): 当数据发送时, RTS (Request to Send) 信号将被拉高。这是控制 RS485 总线方向的常见做法。

RS485 Rts After Send (J): 如果设置为 No, 在数据发送后, RTS 信号不会保持拉高状态。如果设置为 Yes, RTS 信号将在发送完成后仍然保持高电平, 这在某些情况下可能会导致总线冲突。

RS485 Rx During Tx (K): 设置为 No, 在发送数据时是否接收数据。通常情况下, RS485 半双工通信在发送数据时不接收数据, 因此设置为 No。

RS485 Terminate Bus (L): 是否在总线上启用终端电阻。终端电阻用于匹配总线的特性阻抗, 减少信号反射。如果 RS485 总线的两端都需要终端电阻, 可以设置为 Yes。

RS485 Delay Rts Before (M): 发送数据之前 RTS 信号拉高的延迟时间, 单位为毫秒。通常设置为 0, 即没有延迟。

RS485 Delay Rts After (N): 在数据发送完成后 RTS 信号拉低的延迟时间, 单位为毫秒。通常设置为 0, 即没有延迟。

按下对应的按键即可配置对应的功能, 根据项目需求的 RS485 功能选择配置即可。

```
A - Serial Device      : /dev/ttySTM1
B - Lockfile Location  : /var/lock
C - Callin Program    :
D - Callout Program   :
E - Bps/Par/Bits       : 115200 8N1
F - Hardware Flow Control : No
G - Software Flow Control : No
H - RS485 Enable       : Yes
I - RS485 Rts On Send  : No
J - RS485 Rts After Send : No
K - RS485 Rx During Tx : No
L - RS485 Terminate Bus : No
M - RS485 Delay Rts Before : 0
N - RS485 Delay Rts After : 0

Change which setting? |
```

图 17.8-7 RS485 配置

最终确认完全部的配置后, 按回车键确认设置, 即可保存。

选中 Exit 后, 按下回车键即可退出设置界面。

```
[configuration]
Filenames and paths
File transfer protocols
Serial port setup
Modem and dialing
Screen
Keyboard and Misc
Save setup as dfl
Save setup as..
Exit
Exit from Minicom
```

图 17.8-8 退出设置

此时会进入 minicom 使用界面, 如下图所示:

```
Welcome to minicom 2.9

OPTIONS: I18n
Compiled on Sep 22 2023, 21:10:41.
Port /dev/ttySTM1, 04:31:52

Press CTRL-A Z for help on special keys

[ 9367.414888] stm32-usart 40330000.serial: ttySTM1 (1): invalid RTS setting, using RTS_ON_SEND in
```

图 17.8-9 minicom 使用界面

按下 Ctrl 和 A 键可以打开操作提示，如下图所示。

```
Welcome to minicom 2.9

OPTIONS: I18n
Compiled on Sep 22 2023, 21:10:41.
Port /dev/ttySTML, 04:31:52

Press CTRL-A Z for help on special keys

[ 9367.414888] stm32-usart 40330000.serial: ttySTML (1): invalid RTS setting, using RTS_ON_SEND in

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.9 | VT102 | Online 0:0 | ttySTML
```

图 17.8-10minicom 提示

此时再按下 Z 键，即可打开 minicom 的命令菜单。

```
Welcome to minicom
OPTIONS: I18n
Compiled on Sep 22 2023, 21:10:41.
Port /dev/ttySTML,
Press CTRL-A Z for
[ 9367.414

Minicom Command Summary
Commands can be called by CTRL-A <key>

Main Functions                                Other Functions
Dialing directory..D    run script (Go)...G    Clear Screen.....C
Send files.....S        Receive files.....R    cOnfigure Minicom..O
comm Parameters...P    Add linefeed.....A    Suspend minicom...J
Capture on/off....L    Hangup.....H          eXit and reset....X
send break.....F        initialize Modem...M    Quit with no reset.Q
Terminal settings..T    run Kermit.....K        Cursor key mode...I
lineWrap on/off...W    local Echo on/off..E    Help screen.....Z
Paste file.....Y        Timestamp toggle...N    scroll Back.....B
Add Carriage Ret...U

Select function or press Enter for none.|
```

图 17.8-11 minicom 命令菜单

minicom 命令菜单选项

- D - Dialing directory: 用于拨号目录，可以设置拨号连接的电话号码。
- G - run script (Go): 运行脚本。你可以使用脚本自动化串口操作。
- C - Clear Screen: 清除屏幕上的所有内容。
- S - Send files: 发送文件。用于通过串口发送文件。
- R - Receive files: 接收文件。用于通过串口接收文件。
- O - cOnfigure Minicom: 进入配置菜单。用于调整 minicom 的配置。
- P - comm Parameters: 设置通信参数，如波特率、数据位、停止位和校验位。
- A - Add linefeed: 添加换行符。
- J - Suspend minicom: 挂起 minicom。暂时停止 minicom，而不退出。
- L - Capture on/off: 打开或关闭捕获功能。用于记录通信数据到文件。
- H - Hangup: 挂断当前连接。
- X - eXit and reset: 退出 minicom 并重置串口。
- Q - Quit with no reset: 退出 minicom 而不重置串口。
- F - send break: 发送 Break 信号。
- M - initialize Modem: 初始化调制解调器。
- T - Terminal settings: 终端设置。
- K - run Kermit: 运行 Kermit 协议，用于文件传输。
- I - Cursor key mode: 切换光标键模式。
- W - lineWrap on/off: 打开或关闭自动换行功能。
- E - local Echo on/off: 打开或关闭本地回显功能。



- Z - Help screen: 显示帮助屏幕。
- Y - Paste file: 粘贴文件内容。
- N - Timestamp toggle: 切换时间戳显示。
- B - scroll Back: 向后滚动屏幕内容。
- U - Add Carriage Return: 添加回车符。

常用操作

- **发送和接收文件**

发送文件: 按 S 选择发送文件功能。按照提示选择文件并开始发送。

接收文件: 按 R 选择接收文件功能。按照提示配置接收文件的位置和方式。

- **配置 minicom**

进入配置菜单: 按 O 进入配置菜单, 调整 Minicom 的各种设置。

设置通信参数: 按 P 设置波特率、数据位、停止位和校验位等参数。

- **捕获数据**

打开捕获功能: 按 L 打开数据捕获功能, 开始记录通信数据到文件。

关闭捕获功能: 再次按 L 关闭数据捕获功能。

- **退出 minicom**

退出并重置串口: 按 X 退出 minicom 并重置串口。

退出但不重置串口: 按 Q 退出 minicom 而不重置串口。

17.9 具体使用示例

17.9.1 RS232

ATK-DLMP257B 开发板板载一路 RS232, 测试前需要准备一根 USB 转 RS232 公头串口线, 将 USB 端接入电脑, RS232 公头接入开发板 RS232 接口。



图 17.9-1 USB 转 RS232 串口线 (公头)

在计算机的设备管理器查看端口号。编者的端口号有三个, 两个是开发板 USB 调试串口的, 另一个就是 RS232 的端口号了。

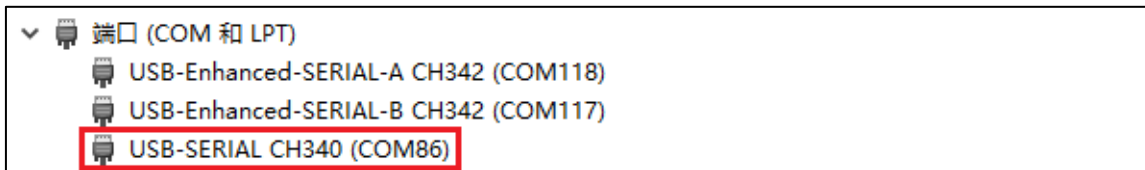


图 17.9-2 设备管理器查看端口号

可以看到 USB-SERIAL CH340(COM86)为开发板底板的 RS232 串口。下面使用 minicom 串口调试工具来进行数据收发测试。

进入 minicom 配置界面，参考之前的设置来配置/dev/ttySTM2，在出厂系统中，RS232 串口对应的设备为/dev/ttySTM2，所以我们下面配置为/dev/ttySTM2，115200，8N1，无硬件流控。

操作方式为，在下面界面按“A”键即可修改“Serial Device”选项内容，我们改为“/dev/ttySTM2”，按 Enter 确认；再按“F”键即可关闭硬件流控功能，即将默认的“Yes”改为“No”。按“E”键可调整波特率，但它默认为“115200 8N1”，所以我们不用调整。

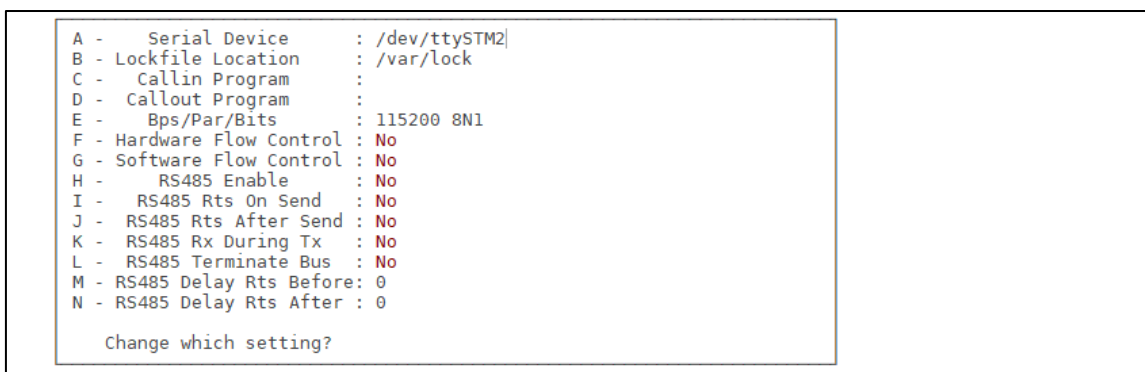


图 17.9-3 RS232 串口配置界面

最后按 Enter 确认最终配置，即退出当前配置界面。以下界面选择“Exit”退出配置界面，即可进入数据收发窗口。

在进行数据收发之前，先打开 minicom 数据回显功能，显示将要发送的数据和接收的数据。按 Ctrl+A，再按“Z”即可打开 minicom 命令界面，按“E”键打开回显功能，返回数据收发窗口，即可通过 RS232 串口和 PC 电脑端口进行数据收发。

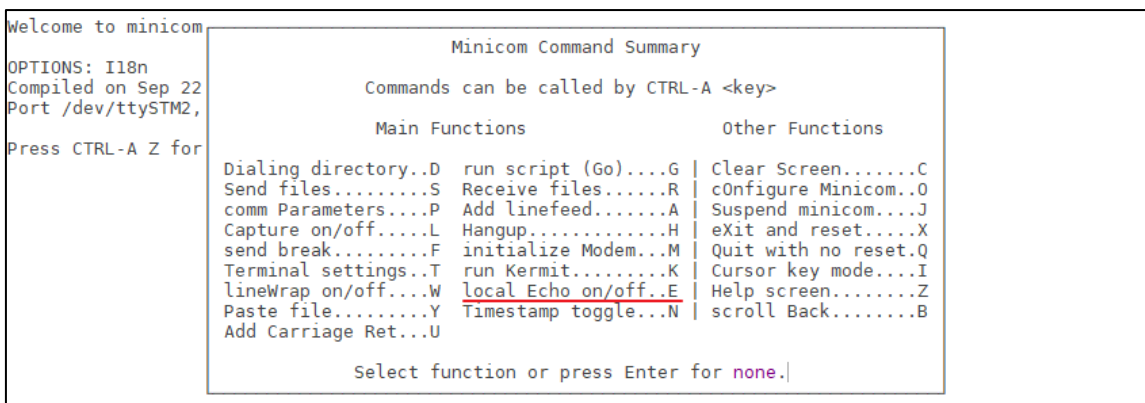


图 17.9-4 按“E”打开回显功能

电脑端使用 XCOM 软件打开 USB-SERIAL CH340(COM86)串口，波特率 115200，8N1，即可通过 RS232 串口和开发板进行数据收发。

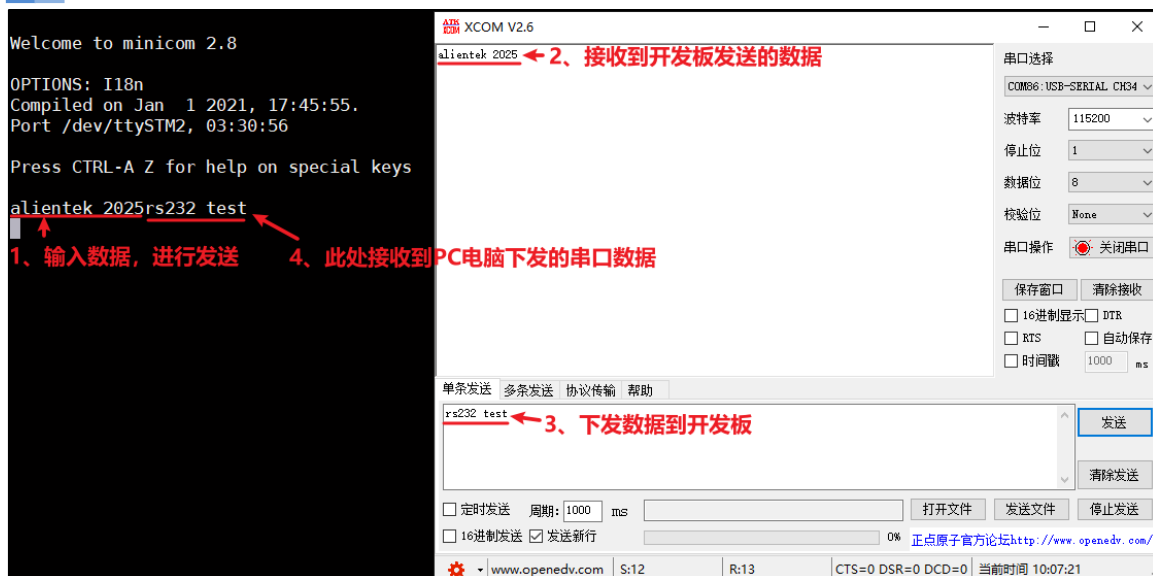


图 17.9-5 RS232 数据收发

测试完成后，按 Ctrl+A，再按“Z”打开 minicom 命令界面，按“X”键退出 minicom 工具，按 Enter 键确认退出，完成测试过程。

17.9.2 RS485

ATK-DLMP257B 开发板板载 1 路 RS485，测试前需要准备正点原子 USB 转换器来完成测试，如下图所示：

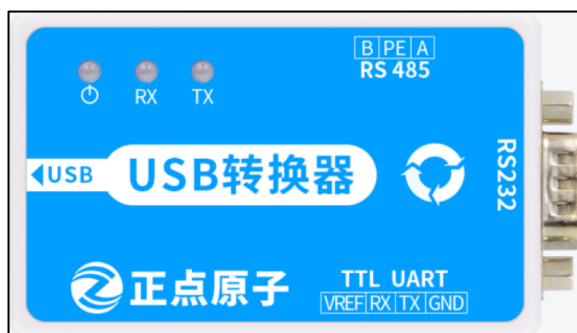


图 17.9-6 正点原子多合一 USB 转换器模块

将开发板的 RS485 A 端通过杜邦线连接到正点原子 USB 转换器模块的 RS485 A 端，B 端连接到 B 端。使用了正点原子的 USB 转换器模块，会在 PC(电脑端)看到一个串口。



图 17.9-7 RS485 接口连接

在计算机的设备管理器查看端口号。编者的端口号有三个，两个是开发板 USB 调试串口的，另一个就是 RS485 的端口号了。

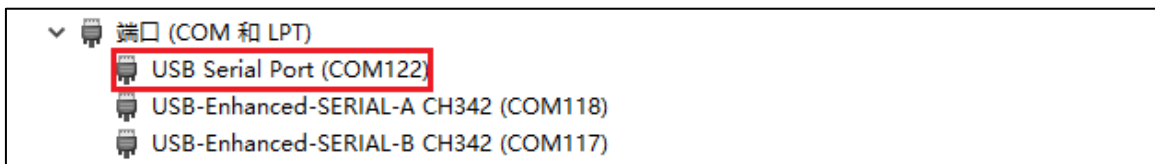


图 17.9-8 设备管理器查看端口号

可以看到 USB-SERIAL Port(COM122)为开发板底板的 RS485 串口。下面使用 minicom 串口调试工具来进行数据收发测试。

进入 minicom 配置界面，参考之前的设置来配置/dev/ttySTM1，在出厂系统中，RS485 串口对应的设备为/dev/ttySTM1，所以我们下面配置为/dev/ttySTM1，115200，8N1，无硬件流控。最后按 Enter 确认最终配置，即退出当前配置界面。以下界面选择“Exit”退出配置界面，即可进入数据收发窗口。

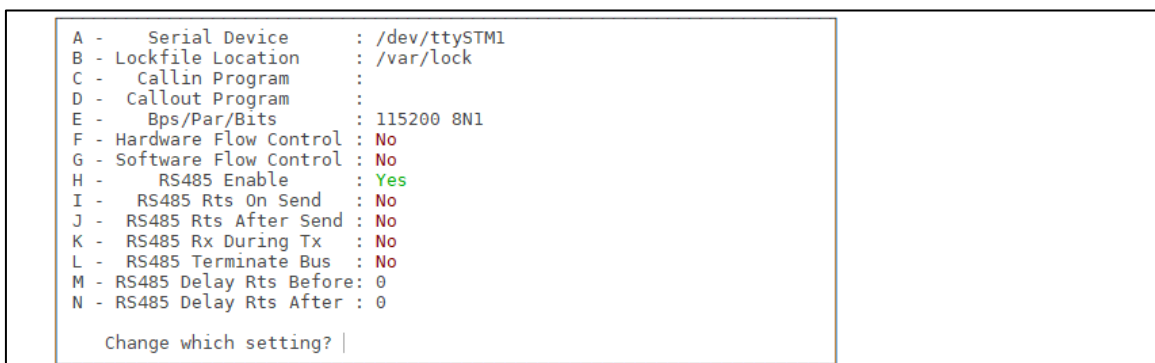


图 17.9-9 RS485 配置

在进行数据收发之前，先打开 minicom 数据回显功能，显示将要发送的数据和接收的数据。按 Ctrl+A，再按“Z”即可打开 minicom 命令界面，按“E”键打开回显功能，返回数据收发窗口，即可通过 RS232 串口和 PC 电脑端口进行数据收发。

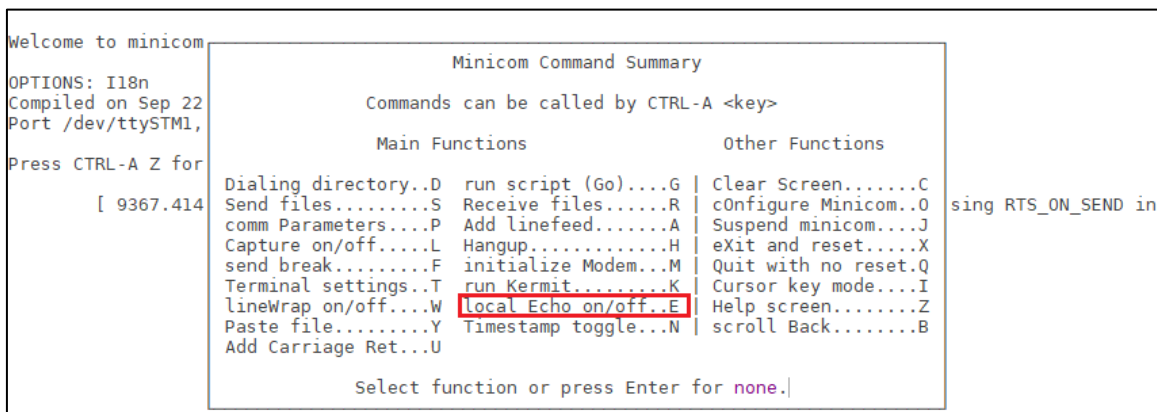


图 17.9-10 按“E”打开回显功能

电脑端使用 XCOM 软件打开 USB-SERIAL Port(COM112)串口，波特率 115200，8N1，即可通过 RS485 串口和开发板进行数据收发。

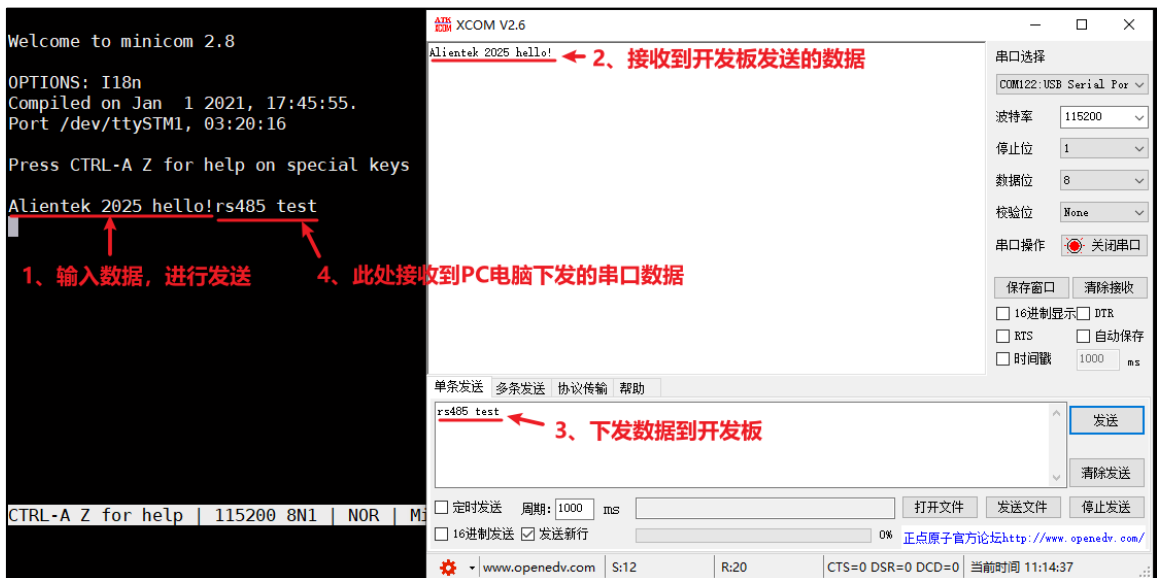


图 17.9-11 RS485 收发

测试完成后，按 Ctrl+A，再按“Z”打开 minicom 命令界面，按“X”键退出 minicom 工具，按 Enter 键确认退出，完成测试过程。

在调用 minicom 的路径下，可以看到 minicom.cap 和 minicom.log 文件。

minicom.cap 文件用于捕获并记录从串口接收到的数据。这是一个二进制捕获文件，记录所有从串口设备接收的原始数据。当用户需要详细的二进制数据捕获时，例如调试底层通信协议或分析数据包时，可以使用 minicom.cap 文件。

minicom.log 文件用于记录 minicom 会话的日志信息。它记录的是会话期间的所有文本输出，包括用户输入和设备响应等信息。当用户需要记录会话日志以便日后查看和分析，例如调试程序输出或记录会话历史时，可以使用 minicom.log 文件。

通过这些步骤，可以有效地使用 minicom 进行串口通信和调试。

第十八章 CAN 子系统

18.1 CAN 子系统简介

Linux 的 CAN (Controller Area Network) 子系统是一个提供 CAN 总线支持的内核子系统。CAN 是一种广泛应用于汽车、工业自动化和嵌入式系统中的网络协议，主要用于微控制器和设备之间的通信。Linux 提供了一个强大的 CAN 子系统，使开发者能够轻松地在 Linux 环境中使用 CAN 功能。

18.2 主要特点

嵌入式 Linux 的 CAN 子系统具有以下主要特点：

- 基于 SocketCAN 的统一接口：简化应用开发，增强代码可移植性。
- 灵活的协议支持：满足不同应用场景的需求。
- 丰富的硬件驱动支持：支持多样化的硬件平台。
- 强大的用户空间工具：简化开发和调试过程。
- 动态配置和管理：提供灵活的配置和管理能力。

这些特点使得 Linux 的 CAN 子系统在嵌入式系统中具有广泛的应用前景和实用价值。

18.3 组成部分

嵌入式 Linux 的 CAN 子系统由多个组件组成，每个组件在 CAN 通信中扮演着不同的角色。以下是主要组成部分的详细介绍：

1、SocketCAN 框架

SocketCAN 是 Linux 内核中用于 CAN 总线通信的框架。它提供了一组基于套接字的 API，使 CAN 通信类似于网络编程。

特点：

- 基于 Berkeley 套接字接口。
- 支持标准 CAN 协议和扩展 CAN 协议。
- 提供原始 CAN 帧访问和高层次协议支持。

2、CAN 协议模块

SocketCAN 支持多种协议模块，每个模块负责特定类型的 CAN 通信。

主要协议模块：

- CAN_RAW：用于原始 CAN 帧的发送和接收。
- CAN_BCM (Broadcast Manager)：支持定时消息传输和接收，用于周期性消息。
- CAN_GW (Gateway/Router)：支持 CAN 帧的路由和网关功能。
- CAN_ISOTP：支持 ISO-TP (ISO 15765-2) 协议，适用于长消息的分片和重组。

3、CAN 网络设备驱动

这些驱动程序负责与具体的 CAN 硬件控制器进行通信。不同的硬件控制器有不同的驱动程序。

常见驱动：

- flexcan：用于 NXP FlexCAN 控制器。
- can_dev：通用 CAN 网络设备驱动，提供基本的 CAN 设备接口。

4、用户空间工具

一组用于测试和调试 CAN 网络的工具，这些工具大多位于 `can-utils` 软件包中。

常见工具：

- `candump`: 用于监听和记录 CAN 总线上的消息。
- `cansend`: 用于发送 CAN 帧。
- `cangen`: 用于生成并发送随机或特定模式的 CAN 帧。
- `canplayer`: 用于回放之前记录的 CAN 数据。

5、配置和管理工具

用于配置和管理 CAN 接口的工具。

`iproute2`: `ip` 命令提供了配置 CAN 网络接口的功能，例如设置波特率、启动和停止接口：

```
ip link set can0 up type can bitrate 500000
ip link set can0 down
```

6、调试和开发支持

内核日志和调试工具可以帮助开发者调试 CAN 驱动程序和应用程序。

内核日志：使用 `dmesg` 查看内核日志，获取 CAN 驱动程序的调试信息。

调试工具：使用 GDB 或其他调试工具调试用户空间 CAN 应用程序。

18.4 MP257 的 CAN 说明

18.4.1 文档和资源

想要获取 MP257 系列处理器的 CAN 详细说明，最直接的方式是参考 ST 官方文档，包括：

1、参考手册（Reference Manual）：提供了详细的寄存器级说明，包括 CAN 模块的寄存器映射以及配置方法等。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南（即本文档）

应用开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图：【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

18.4.2 特性说明

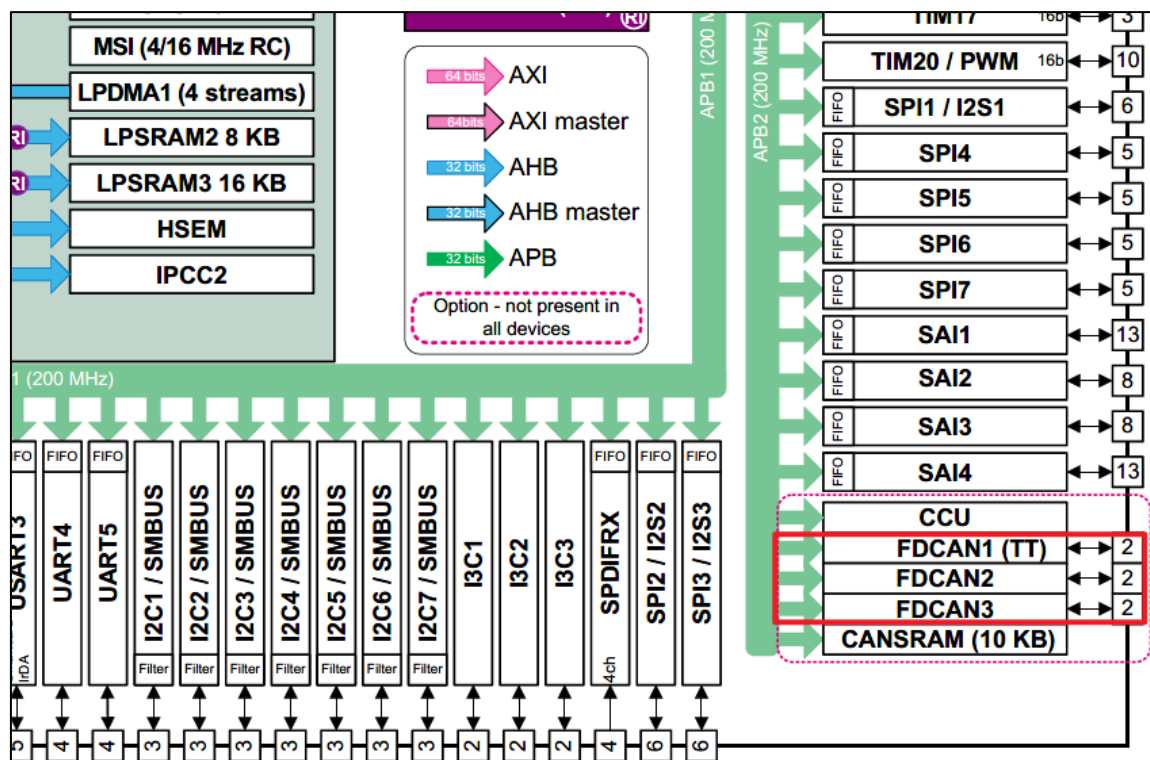


图 18.4-1 MP257 功能框图中的 CAN 部分

CAN-FD (Controller Area Network with Flexible Data rate) 模块是一种通信控制器, 实施 CAN 协议, 并符合 ISO11898-1 和 CAN 2.0B 协议规范。

关键特性

- CAN 协议支持:

ISO11898-1: CAN-FD 模块遵循 ISO11898-1 标准, 这是一种用于车辆和工业自动化中的通信协议标准。

CAN 2.0B: CAN-FD 模块完全兼容 CAN 2.0B 协议规范, 支持 11 位和 29 位扩展标识符。

- 灵活的数据速率:

CAN-FD 引入了灵活的数据速率, 在同一消息中可以有不同的速率部分。

数据阶段的位速率可以比仲裁阶段更高, 提高了数据传输的效率。

- 更大的数据负载:

CAN-FD 允许在一个数据帧中传输更大的数据负载, 从传统 CAN 的 8 字节增加到最多 64 字节。这种改进提高了数据传输效率, 减少了传输长消息所需的帧数。

更多的 CAN 说明请查阅芯片参考手册 CAN 章节。

18.5 开发板 CAN 设备

18.5.1 原理图分析

打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 CAN。可以看到 CAN 相关电路如下。

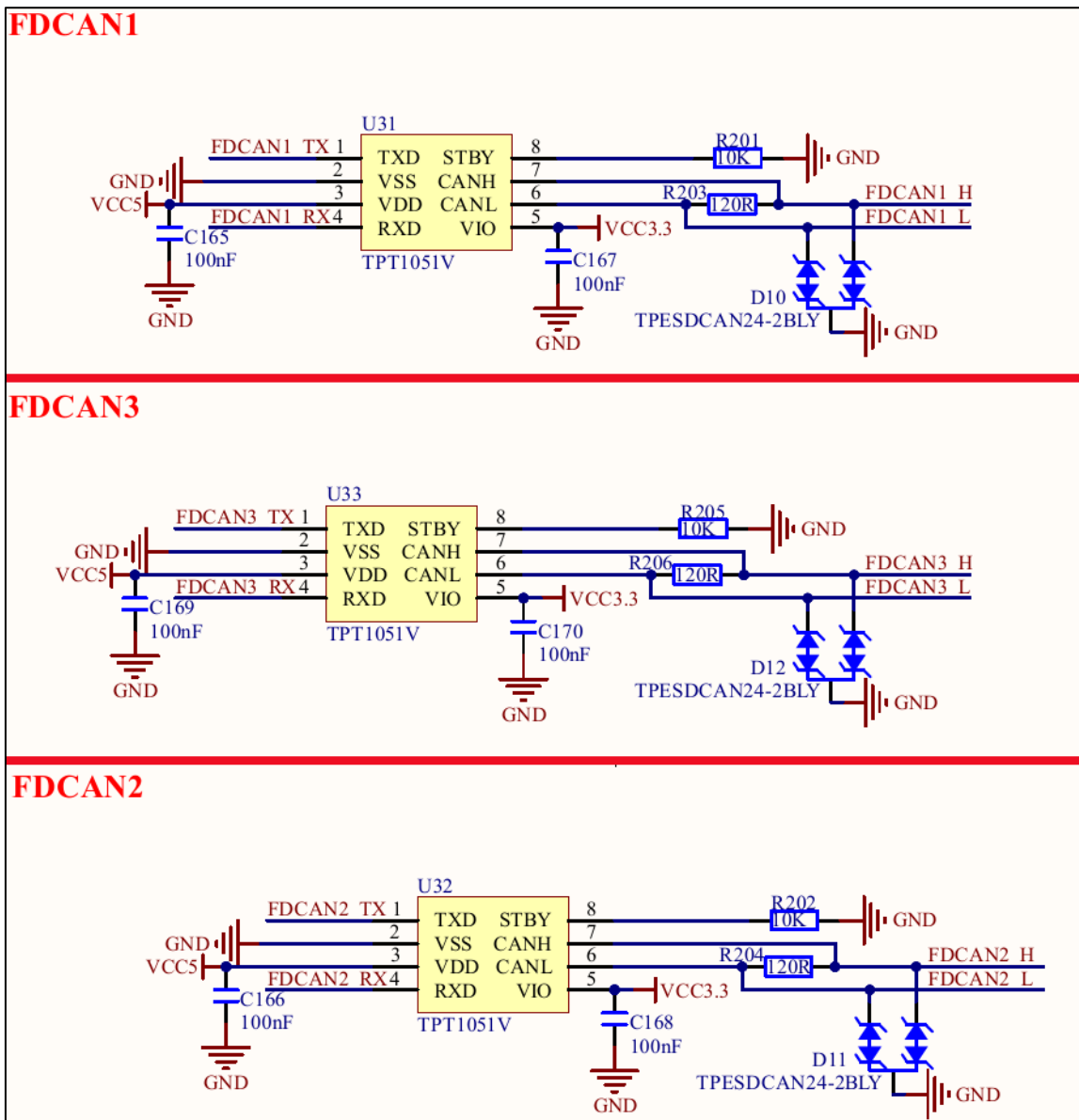


图 18.5-1 开发板 CAN 原理图

FDCAN 模块解析

● 发送与接收通道:

TXD (1 脚):接收来自 MCU 的发送数据信号 (FDCAN_x_TX)。将该信号发送到 CAN 总线。

RXD (4 脚):接收来自 CAN 总线的差分信号。将信号转换后传输给 MCU 的接收通道 (FDCAN_x_RX)。

● 供电与稳压

VCC5 为 TPT1051V 芯片提供工作电压 (VDD)。

VCC3.3 提供 CAN 总线的参考电压 (VIO)，用于逻辑接口匹配。

C165、C166、C167、C168、C169、C170 (100nF):去耦电容,用于滤除电源噪声,保证芯片供电稳定。

● 差分总线接口

CANH 和 CANL (7 脚和 6 脚):CANH(差分高电平)和 CANL(差分低电平)构成 CAN 总线差分信号对,用于双向通信。

R203、R204、R206 (120R):终端匹配电阻,连接在 CANH 和 CANL 之间,匹配总线阻抗,防止信号反射。

R201、R202、R205 (10K):上拉电阻,用于稳定信号逻辑电平。

● 保护电路

D10、D11、D12 (TPESDCAN24-2BLY):TVS 二极管,用于保护 CANH 和 CANL 信号线,防止静电 (ESD) 或瞬态高压对电路的损害。

● 芯片功能

STBY (2 脚):芯片的待机控制脚。通过逻辑信号控制芯片是否进入低功耗模式。

VSS (3 脚):芯片接地引脚。

18.5.2 实物硬件

开发板上的 CAN 接口和 CAN 芯片位置如下图所示:

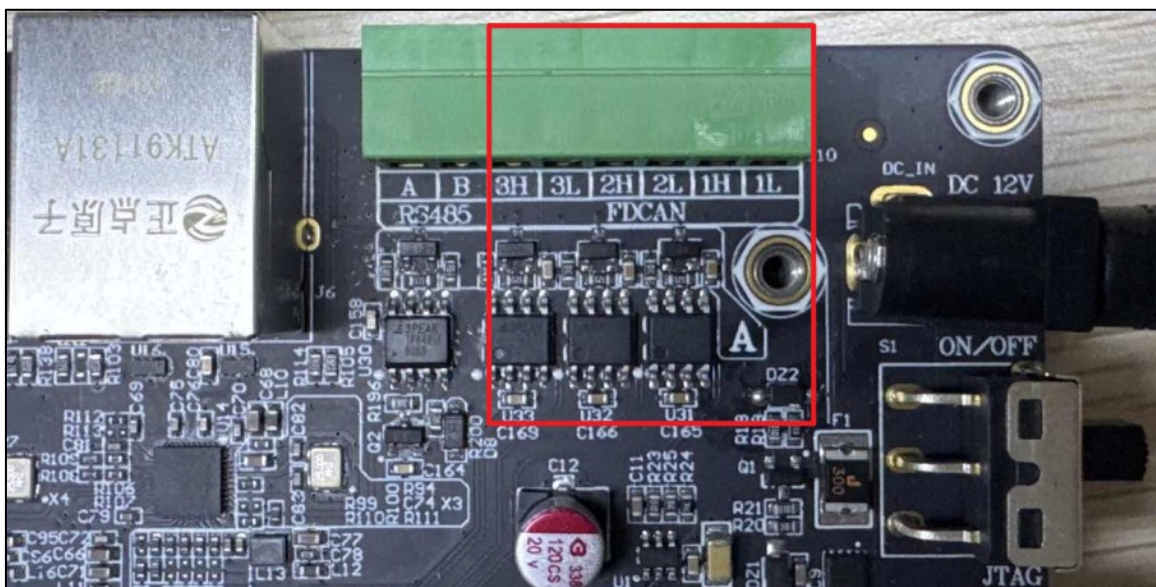


图 18.5-2 开发板 CAN 接口硬件实物

18.6 内核 CAN 子系统使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, CAN 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_CAN=m //启用 CAN 总线支持
```



```

2 CONFIG_CAN_RAW=m //启用原始 CAN 协议支持, 允许用户空间应用直接读写 CAN 帧
3 CONFIG_CAN_BCM=m //启用 CAN 广播管理器协议, 支持定时消息传输和接收
4 CONFIG_CAN_GW=m //启用 CAN 网关/路由功能, 允许在不同的 CAN 网络之间转发帧
5 CONFIG_CAN_DEV=m //启用 CAN 设备驱动框架, 提供通用的 CAN 网络设备接口
6 CONFIG_CAN_NETLINK=y //启用通过 Netlink 进行 CAN 配置和管理的功能
7 CONFIG_CAN_CALC_BITTIMING=y /*启用 CAN 位时序计算功能,
                                帮助用户选择合适的 CAN 控制器时钟参数*/
8 CONFIG_CAN_RX_OFFLOAD=y //启用 CAN 接收卸载功能, 提高高负载 CAN 网络的接收性能
9

```

对应的 menuconfig 配置如下（篇幅有限，截图有所省略）：

```

-> Device Drivers
  -> [*] Networking support --->
    -> <M> CAN bus subsystem support --->
      -> <M> Raw CAN Protocol (raw access with CAN-ID filtering)
      -> <M> Broadcast Manager CAN Protocol (with content filtering)
      -> <M> CAN Gateway/Router (with netlink configuration)
    -> Network device support (NETDEVICES [=y])
      -> <M> CAN Device Drivers --->
        -> [*] CAN device drivers with Netlink support
        -> [*] CAN bit-timing calculation

```

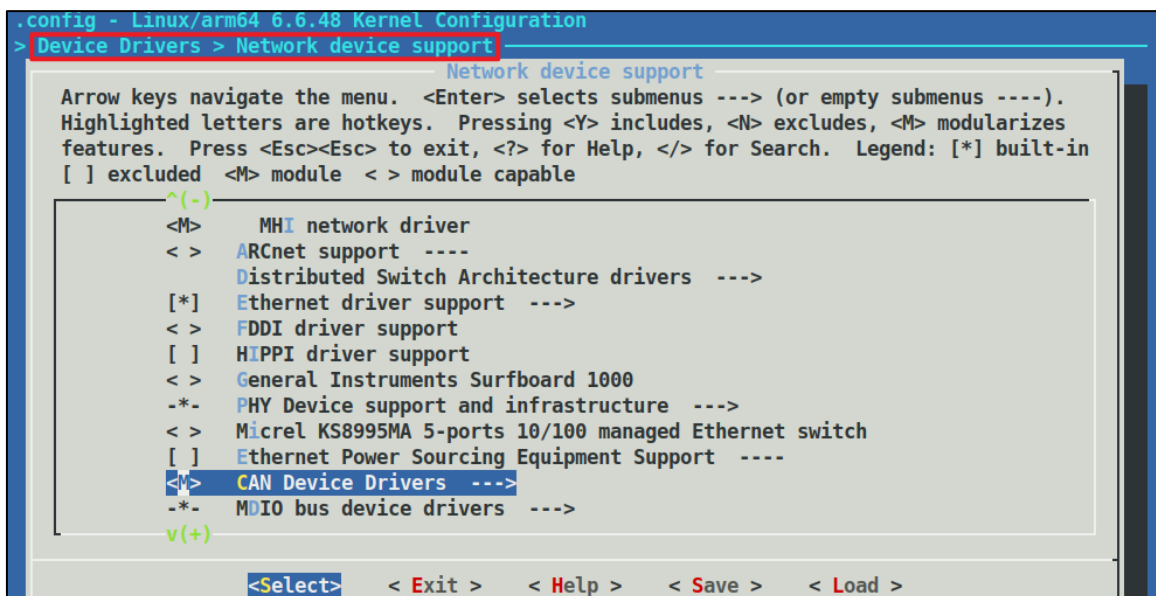


图 18.6-1 CAN 配置节选

18.7 设备树分析

打开 arch/arm64/boot/dts/st/stm32mp253.dtsi，此文件是特定于 STM32MP2 系列芯片的设备树包含文件，通常不独立使用，而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置，而 .dtsi 则作为它们的公共部分。stm32mp253.dtsi 中关于 CAN 的描述如下：

```

1 &rifsc {

```



```
2  m_can1: can@402d0000 {
3      compatible = "bosch,m_can";
4      reg = <0x402d0000 0x400>, <0x40310000 0xd50>;
5      reg-names = "m_can", "message_ram";
6      interrupts = <GIC_SPI 91 IRQ_TYPE_LEVEL_HIGH>,
7                  <GIC_SPI 94 IRQ_TYPE_LEVEL_HIGH>;
8      interrupt-names = "int0", "int1";
9      clocks = <&rcc CK_BUS_FDCAN>, <&rcc CK_KER_FDCAN>;
10     clock-names = "hclk", "cclk";
11     bosch,mram-cfg = <0x0 0 0 32 0 0 2 2>;
12     access-controllers = <&rifsc 56>;
13     power-domains = <&CLUSTER_PD>;
14     status = "disabled";
15 };
16
17 m_can2: can@402e0000 {
18     compatible = "bosch,m_can";
19     reg = <0x402e0000 0x400>, <0x40310000 0x1aa0>;
20     reg-names = "m_can", "message_ram";
21     interrupts = <GIC_SPI 92 IRQ_TYPE_LEVEL_HIGH>,
22                 <GIC_SPI 95 IRQ_TYPE_LEVEL_HIGH>;
23     interrupt-names = "int0", "int1";
24     clocks = <&rcc CK_BUS_FDCAN>, <&rcc CK_KER_FDCAN>;
25     clock-names = "hclk", "cclk";
26     bosch,mram-cfg = <0xd50 0 0 32 0 0 2 2>;
27     access-controllers = <&rifsc 56>;
28     power-domains = <&CLUSTER_PD>;
29     status = "disabled";
30 };
31
32 m_can3: can@402f0000 {
33     compatible = "bosch,m_can";
34     reg = <0x402f0000 0x400>, <0x40310000 0x2800>;
35     reg-names = "m_can", "message_ram";
36     interrupts = <GIC_SPI 93 IRQ_TYPE_LEVEL_HIGH>,
37                 <GIC_SPI 96 IRQ_TYPE_LEVEL_HIGH>;
38     interrupt-names = "int0", "int1";
39     clocks = <&rcc CK_BUS_FDCAN>, <&rcc CK_KER_FDCAN>;
40     clock-names = "hclk", "cclk";
41     bosch,mram-cfg = <0x1aa0 0 0 32 0 0 2 2>;
42     access-controllers = <&rifsc 56>;
43     power-domains = <&CLUSTER_PD>;
44     status = "disabled";
```

```
45 };
```

```
46 };
```

设备树解析:

第 1 行, 表示该节点的父节点为 `rifsc`, 这是一个片上资源的控制模块, 该节点定义了多个 CAN 控制器。

第一个 CAN 控制器 (`m_can1`):

第 2 行, `m_can1: can@402d0000` 定义第一个 CAN 控制器的节点, 节点名称为 `m_can1`, 其寄存器地址为 `0x402d0000`。

第 3 行, `compatible = "bosch,m_can"` 表示该 CAN 控制器的硬件兼容类型是博世公司的 `m_can` 控制器。驱动文件路径为 `drivers/net/can/m_can/m_can_platform.c`

第 4 行, `0x402d0000 0x400` 表示 CAN 控制器的寄存器基地址为 `0x402d0000`, 大小为 `0x400` 字节。`0x40310000 0xd50` 表示消息 RAM 的基地址为 `0x40310000`, 大小为 `0xd50` 字节。

第 5 行, `reg-names = "m_can", "message_ram"` 给寄存器块命名, 其中: "m_can" 表示控制器寄存器。"message_ram" 表示消息 RAM。

第 6~8 行, `interrupts` 和 `interrupt-names` 定义中断信号: `GIC_SPI91 IRQ_TYPE_LEVEL_HIGH`: 主中断号为 91, 电平触发, 高电平有效。`GIC_SPI94 IRQ_TYPE_LEVEL_HIGH`: 第二个中断号为 94, 功能类似。中断名称分别为 "int0" 和 "int1"。

第 9~10 行, `clocks`: 表示 CAN 控制器的时钟源: `<&rcc CK_BUS_FDCAN>`: 总线时钟。`<&rcc CK_KER_FDCAN>`: 核心时钟。`clock-names`: 时钟的名称为 "hclk" 和 "cclk"。

第 11 行, `bosch,mram-cfg = <0x0 0 0 32 0 0 2 2>` 表示消息 RAM 的配置参数, 具体含义需参考博世 `m_can` 的文档。

第 12 行, `access-controllers = <&rifsc 56>` 定义该节点的访问控制器, 编号为 56。

第 13 行, `power-domains = <&CLUSTER_PD>` 定义该 CAN 控制器属于 `CLUSTER_PD` 电源域。

第 14 行, `status = "disabled"` 表示该设备默认状态为禁用。

第二个 CAN 控制器 (m_can2):

第 17 行, `m_can2: can@402e0000` 定义第二个 CAN 控制器, 地址为 `0x402e0000`。

第 18~29 行, 与 `m_can1` 节点类似, 主要差异:

第 19 行: 消息 RAM 地址为 `0x40310000`, 大小为 `0x1aa0`。

第 26 行: 消息 RAM 配置参数为 `<0xd50 0 0 32 0 0 2 2>`。

中断号: 主中断号为 92, 第二中断号为 95。

第三个 CAN 控制器 (m_can3):

第 32 行: `m_can3: can@402f0000` 定义第三个 CAN 控制器, 地址为 `0x402f0000`。

第 33~44 行: 与 `m_can1` 节点类似, 主要差异:

第 34 行: 消息 RAM 地址为 `0x40310000`, 大小为 `0x2800`。

第 41 行: 消息 RAM 配置参数为 `<0x1aa0 0 0 32 0 0 2 2>`。

中断号: 主中断号为 93, 第二中断号为 96。

以 2GB 版本核心板为例, 在 `arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts` 中, CAN 相关配置如下:

```
1 &m_can1 {
```

```

2   pinctrl-names = "default", "sleep";
3   pinctrl-0 = <&m_can1_pins_a>;
4   pinctrl-1 = <&m_can1_sleep_pins_a>;
5   status = "okay";
6 };
7
8 &m_can2 {
9   pinctrl-names = "default", "sleep";
10  pinctrl-0 = <&m_can2_pins_a>;
11  pinctrl-1 = <&m_can2_sleep_pins_a>;
12  status = "okay";
13 };
14
15 &m_can3 {
16  pinctrl-names = "default", "sleep";
17  pinctrl-0 = <&m_can3_pins_a>;
18  pinctrl-1 = <&m_can3_sleep_pins_a>;
19  status = "okay";
20 };
    
```

第 1 行, `&m_can1` 指向 `m_can1` 节点的引用, 定义与 CAN1 相关的设备树配置。

第 2 行, `pinctrl-names = "default", "sleep"` 定义引脚控制的名称, 包括两种状态: 默认状态 (default) 和睡眠状态 (sleep)。

第 3 行, `pinctrl-0 = <&m_can1_pins_a>` 将 `m_can1_pins_a` 定义为默认状态 (default) 下使用的引脚组。

第 4 行, `pinctrl-1 = <&m_can1_sleep_pins_a>` 将 `m_can1_sleep_pins_a` 定义为睡眠状态 (sleep) 下使用的引脚组。

第 5 行, `status = "okay"` 将 CAN1 节点设置为启用状态。

第 8 行, `&m_can2` 指向 `m_can2` 节点的引用, 定义与 CAN2 相关的设备树配置。

第 9 行, `pinctrl-names = "default", "sleep"` 定义引脚控制的名称, 包括默认状态和睡眠状态。

第 10 行, `pinctrl-0 = <&m_can2_pins_a>` 将 `m_can2_pins_a` 定义为默认状态下使用的引脚组。

第 11 行, `pinctrl-1 = <&m_can2_sleep_pins_a>` 将 `m_can2_sleep_pins_a` 定义为睡眠状态下使用的引脚组。

第 12 行, `status = "okay"` 将 CAN2 节点设置为启用状态。

第 15 行, `&m_can3` 指向 `m_can3` 节点的引用, 定义与 CAN3 相关的设备树配置。

第 16 行, `pinctrl-names = "default", "sleep"` 定义引脚控制的名称, 包括默认状态和睡眠状态。

第 17 行, `pinctrl-0 = <&m_can3_pins_a>` 将 `m_can3_pins_a` 定义为默认状态下使用的引脚组。

第 18 行, `pinctrl-1 = <&m_can3_sleep_pins_a>` 将 `m_can3_sleep_pins_a` 定义为睡眠状态下使用的引脚组。

第 19 行, `status = "okay"` 将 CAN3 节点设置为启用状态。

18.8 系统接口

ATK-DLMP257B 开发板上有三路 CAN 接口, 执行下面指令可以查看开发板所有的网络设备:

ifconfig -a

```
root@ATK-DLMP257:~# ifconfig -a
can0      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:67

can1      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:68

can2      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:69
```

图 18.8-1 开发板网络设备

可以看到 can0、can1 和 can2 三个 CAN 设备，系统中对应的 CAN 设备如下：

外设接口	系统接口
CANFD1	can0
CANFD2	can1
CANFD3	can2

18.9 CAN 调试工具

18.9.1 使能 CAN 接口

在 Linux 用户空间中，使用 CAN 工具可以方便地进行 CAN 总线通信的测试和调试。下面是开发板出厂系统中的一些 CAN 工具和使用示例。这里我们将开发板的 CANFD1 接口和 CANFD2 接口互相连接，将 CAN1 的 H 端接在 CAN2 的 H 端，将 CAN1 的 L 端接在 CAN2 的 L 端，用于测试。CAN3 测试也是同理，接在 CAN1 对应的端口上也可以测试。

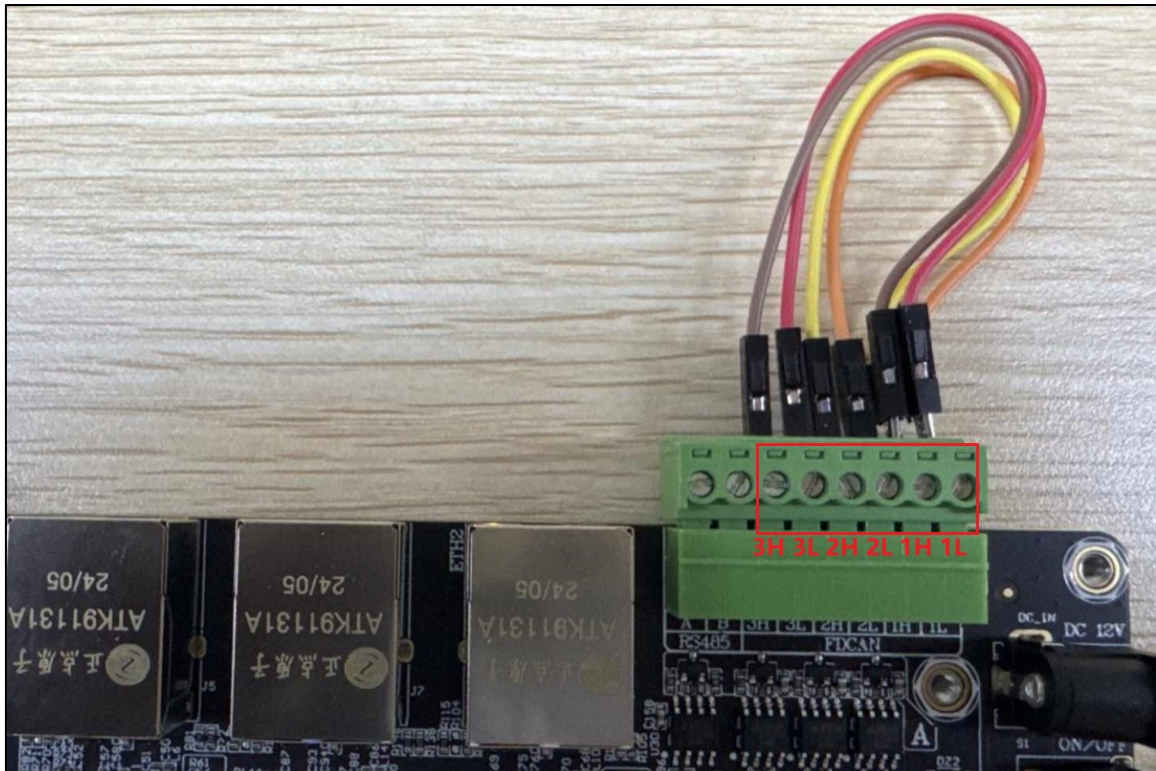


图 18.9-1 开发板 CAN 接口连接

由于需要开启两个终端才能看具体的收发信息，开发板板载的一路USB_TTL 已经不够满足，因此我们需要通过SSH 登陆来查看，一路用于操作can0发送，一路用于查看can1接收。

开发板SSH 登陆请查看【正点原子】ATK-DLMP257B快速体验手册。

执行下面指令可以开启并配置 can0 和 can1:

```
ip link set can0 up type can bitrate 1000000 dbitrate 5000000 fd on
```

```
ip link set can1 up type can bitrate 1000000 dbitrate 5000000 fd on
```

此处设置波特率为 1000 kBit/s，数据波特率为 5000 kBit/s 进行测试。

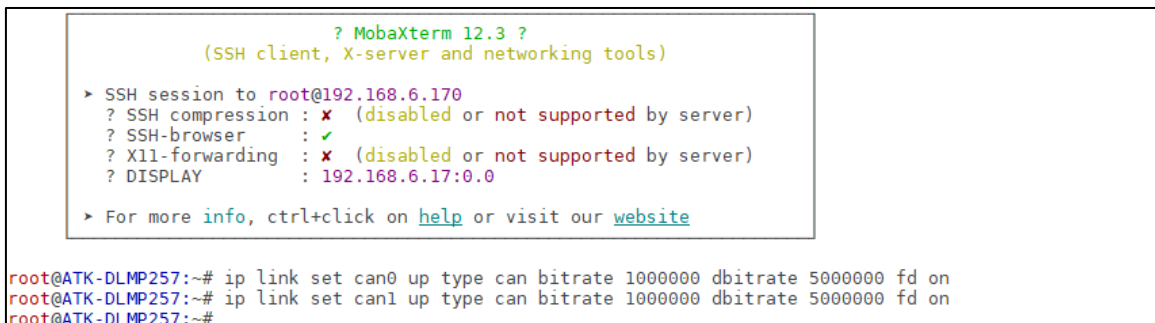


图 18.9-2 开启 can0 和 can1

命令分解和解释

ip link:

- ip 是 Linux 的网络管理工具，用于显示和修改网络接口的属性。
- link 子命令用于管理网络接口的属性。

set can0 up:

- set: 指定要修改接口的属性。
- can0: 指明要配置的 CAN 接口。



- up: 启用（激活）该接口。

type can:

- type can: 指定接口类型为 CAN（控制器局域网）。

bitrate 1000000:

- bitrate: 设定 CAN 标准帧的位速率。
- 1000000: 设定 CAN 标准帧的位速率为 1 Mbps（1000000 bits per second）。

dbitrate 5000000:

- dbitrate: 设定 CAN FD 帧的数据段位速率。
- 5000000: 设定 CAN FD 帧的数据段位速率为 5 Mbps（5000000 bits per second）。

fd on:

- fd on: 启用 CAN FD（Flexible Data-Rate）模式。CAN FD 是 CAN 的扩展，允许更高的数据速率和更大的数据帧。

整体作用

该命令配置并启动了 can0 接口，使其运行在 CAN FD 模式下，设定标准 CAN 帧的位速率为 1 Mbps，CAN FD 数据段的位速率为 5 Mbps。以下是这些配置的详细作用：

CAN 接口 can0: 指定要配置的网络接口。

标准 CAN 帧位速率 1 Mbps: 设置基本 CAN 帧的传输速度。标准 CAN 帧通常用于较低速率的通信。

CAN FD 帧的数据段位速率 5 Mbps: 设置 CAN FD 帧的数据段传输速度。CAN FD 允许更高的传输速率，使其适用于需要更高数据吞吐量的应用。

启用 CAN FD 模式: CAN FD 允许在数据帧中传输更多的数据，同时可以使用更高的传输速率。

18.9.2 candump

candump 是一个强大的工具，用于监控并记录 CAN 总线上的流量。执行 candump 指令可以看到使用说明。

candump

```
root@ATK-DLMP257:~# candump
candump - dump CAN bus traffic.

Usage: candump [options] <CAN interface>+
       (use CTRL-C to terminate candump)

Options:
  -t <type>      (timestamp: (a)bsolute/(d)elta/(z)ero/(A)bsolute w date)
                  (read hardware timestamps instead of system timestamps)
  -H             (increment color mode level)
  -c             (binary output - may exceed 80 chars/line)
  -i             (enable additional ASCII output)
  -S             (swap byte order in printed CAN data[] - marked with ``')
  -s <level>      (silent mode - 0: off (default) 1: animation 2: silent)
  -l             (log CAN-frames into file. Sets '-s 2' by default)
  -f <fname>      (log CAN-frames into file <fname>. Sets '-s 2' by default)
  -L             (use log file format on stdout)
  -n <count>      (terminate after reception of <count> CAN frames)
  -r <size>       (set socket receive buffer to <size>)
  -D             (Don't exit if a "detected" can device goes down)
  -d             (monitor dropped CAN frames)
  -e             (dump CAN error frames in human-readable format)
  -8             (display raw DLC values in {} for Classical CAN)
  -x             (print extra message infos, rx/tx brs esi)
  -T <msecs>      (terminate after <msecs> if no frames were received)

Up to 16 CAN interfaces with optional filter sets can be specified
on the commandline in the form: <ifname>[,filter]*

Filters:
  Comma separated filters can be specified for each given CAN interface:
  <can_id>:<can_mask>
    (matches when <received_can_id> & mask == can_id & mask)
  <can_id>~<can_mask>
    (matches when <received_can_id> & mask != can_id & mask)
  #<error_mask>
    (set error frame filter, see include/linux/can/error.h)
  [j|J]
    (join the given CAN filters - logical AND semantic)

CAN IDs, masks and data content are given and expected in hexadecimal values.
When the can_id is 8 digits long the CAN_EFF_FLAG is set for 29 bit EFF format.
Without any given filter all data frames are received ('0:0' default filter).
```

图 18.9-3 candump 使用说明

下面是详细使用说明和参数说明。

candump 用于监控 CAN 总线流量。使用格式为：

candump [options] <CAN interface>+

- <CAN interface>+ 表示一个或多个 CAN 接口，例如 can0, can1 等。
- 使用 CTRL-C 终止 candump。

选项 (Options)

-t <type> (timestamp: (a)bsolute/(d)elta/(z)ero/(A)bsolute w date)

-t <type>: 选择时间戳类型：

- a: 绝对时间戳。
- d: 相对时间戳（与前一帧的时间差）。
- z: 零基时间戳（从 0 开始计时）。
- A: 绝对时间戳（包含日期）。

-H: 使用硬件时间戳而不是系统时间戳。

-c: 增加颜色模式的级别，可以多次使用以增加级别。

-i: 二进制输出，可能超过 80 个字符每行。

-a: 启用附加的 ASCII 输出。



- S: 交换打印的 CAN 数据字节顺序, 用 `` 标记。
- s <level>: 静默模式:
 - 0: 关闭 (默认)。
 - 1: 动画模式。
 - 2: 完全静默模式。
- l: 将 CAN 帧记录到文件中, 默认设置 -s 2 (静默模式)。
- f <fname>: 将 CAN 帧记录到指定文件 <fname> 中, 默认设置 -s 2。
- L: 在标准输出上使用日志文件格式。
- n <count>: 接收指定数量的 CAN 帧后终止。
- r <size>: 设置套接字接收缓冲区大小为 <size>。
- D: 如果检测到 CAN 设备关闭, 不退出程序。
- d: 监控丢弃的 CAN 帧。
- e: 以人类可读的格式转储 CAN 错误帧
- 8: 显示传统 CAN 的原始 DLC 值。
- x: 打印附加消息信息, 如接收/发送 BRS (位速率开关) 和 ESI (错误状态指示)。
- T <msecs>: 如果在指定的毫秒内没有接收到帧, 则终止程序。

过滤器 (Filters)

- 可以为每个 CAN 接口指定逗号分隔的过滤器:
- <can_id>:<can_mask>: 当 $\text{received_can_id} \& \text{mask} == \text{can_id} \& \text{mask}$ 时匹配。
- <can_id>~<can_mask>: 当 $\text{received_can_id} \& \text{mask} != \text{can_id} \& \text{mask}$ 时匹配。
- #<error_mask>: 设置错误帧过滤器, 具体参见 include/linux/can/error.h。
- [j]J: 加入指定的 CAN 过滤器 (逻辑与语义)。

CAN ID、掩码和数据内容 (CAN IDs, masks and data content)

- CAN IDs、掩码和数据内容应使用十六进制值表示。
- 当 CAN ID 为 8 位时, 设置 CAN_EFF_FLAG 以表示 29 位扩展帧格式 (EFF)。
- 如果没有指定过滤器, 则接收所有数据帧 ('0:0' 默认过滤器)

示例: 使用 `candump -ta can1` 命令, 通过在 CAN 帧前面添加绝对时间戳的方式监控 can1 接口上的 CAN 流量, 使得时间敏感问题的调试和日志记录更加方便。这是一种非常有效的方法来跟踪和分析 CAN 网络中的实时通信。

```
candump -ta can1
```

18.9.3 cansend

打开一个新的 SSH 终端，并在 SSH 终端中使用 `cansend` 指令从 `can0` 发送数据。

`cansend` 是一个用于通过 `CAN_RAW` 套接字发送 CAN 帧的工具，执行 `cansend` 指令可以看到使用说明。

```
root@ATK-DLMP257:~# cansend
cansend - send CAN-frames via CAN_RAW sockets.

Usage: cansend <device> <can_frame>.

<can_frame>:
<can_id>#{data}          for Classical CAN 2.0 data frames
<can_id>#R{len}          for Classical CAN 2.0 data frames
<can_id>#{data}_{dlc}    for Classical CAN 2.0 data frames
<can_id>#R{len}_{dlc}    for Classical CAN 2.0 data frames
<can_id>##<flags>{data}  for CAN FD frames

<can_id>:
3 (SFF) or 8 (EFF) hex chars
{data}:
0..8 (0..64 CAN FD) ASCII hex-values (optionally separated by '.')
{len}:
an optional 0..8 value as RTR frames can contain a valid dlc field
{dlc}:
an optional 9..F data length code value when payload length is 8
<flags>:
a single ASCII Hex value (0 .. F) which defines canfd_frame.flags

Examples:
SA1#11.2233.44556677.88 / 123#DEADBEEF / SAA# / 123##1 / 213##311223344 /
1F334455#1122334455667788 B / 123#R / 00000123#R3 / 333#R8 E
```

图 18.9-4 cansend 使用说明

下面是详细使用说明和参数说明。

`cansend <device> <can_frame>`

`cansend`: 工具名称，用于发送 CAN 帧。

使用格式为：

`cansend <device> <can_frame>`

`<device>`: CAN 接口设备名称，例如 `can0`, `can1` 等。

`<can_frame>`: 要发送的 CAN 帧。

CAN 帧格式说明

- `<can_id>#{data}`: 发送带有数据的 CAN 帧。
- `<can_id>#R{len}`: 发送远程请求帧 (RTR)。
- `<can_id>#{data}_{dlc}`: 发送带有数据和数据长度码 (DLC) 的 CAN 帧。
- `<can_id>#R{len}_{dlc}`: 发送带有远程请求和数据长度码 (DLC) 的 CAN 帧。

CAN FD 帧

- `<can_id>##<flags>{data}`: 发送带有标志和数据的 CAN FD 帧。

CAN ID 和数据说明

- `<can_id>`: CAN 帧的标识符，可以是 3 位 (标准帧格式, SFF) 或 8 位 (扩展帧格式, EFF) 十六进制字符。
- `{data}`: CAN 帧的数据部分，可以是 0 到 8 个 ASCII 十六进制值 (Classical CAN)，或 0 到 64 个 ASCII 十六进制值 (CAN FD)，可以选择性地用 `!` 分隔。
- `{len}`: 可选的 0 到 8 值，用于远程请求帧的有效 DLC 字段。
- `_{dlc}`: 可选的数据长度码 (DLC)，用于当有效载荷长度为 8 时，可取值 9 到 F。
- `<flags>`: 单个 ASCII 十六进制值 (0 到 F)，定义 CAN FD 帧的标志。

示例：使用 `cansend can0` 命令，通过 `can0` 接口发送一个 CANFD 帧。该帧的 CAN ID 为 123，数据字段总长度为 64 字节。数据字段被分隔为多个十六进制数值，并包含在 CAN FD 帧的负载中。

在 SSH 端执行下面这条指令，让 `can0` 使用 `cansend` 命令发送数据，下面为一行完整指令，请分行复制并组合！

```
cansend can0
123##300.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.f
ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff
```

参数说明：

- `cansend can0`

使用 `cansend` 工具通过 `can0` 接口发送 CAN 帧。

- `123##`

123: CAN ID 为 123。

- `##`: 双井号 `##` 表示使用 CAN FD 帧格式。

- `300`

3: CANFD 帧的标志字段，通常表示 BRS (Bit Rate Switch) 和 ESI (Error State Indicator) 标志。在这个例子中，3 表示 BRS 和 ESI 都为 0。

00: 保留位。

- `11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.f
ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff.00.11.22.33.44.55.66.77.88.99.aa.bb.cc.dd
.ee.ff`

数据字段，表示发送的数据。数据总长度为 64 字节，其中每个字节以十六进制表示，用 . 分隔。

从 `can0` 发送数据后，在 `can1` 接收数据如下：

```
? MobaXterm 12.3 ?
(SSH client, X-server and networking tools)

> SSH session to root@192.168.6.170
? SSH compression : x (disabled or not supported by server)
? SSH-browser      : x
? X11-forwarding   : x (disabled or not supported by server)
? DISPLAY          : 192.168.6.17:0.0

> For more info, ctrl+click on help or visit our website

Last login: Wed Feb 28 06:21:26 2024 from 192.168.6.17
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# candump -ta can1
(1709072797.911111) can1 123 [64] 00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF 00 11 22 33 44 55 6
6 77 88 99 AA BB CC DD EE FF 00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF 00 11 22 33 44 55 66
AA BB CC DD EE FF
```

图 18.9-5 can1 接收到的数据

在 CAN 网络中，发送和接收的数据内容实际上是相同的，但在显示时，某些工具可能会将十六进制数显示为大写或小写。这只是显示方式的不同，实际发送和接收的数据并没有变化，

`cansend` 工具显示的数据部分采用的是小写的十六进制格式。例如：
11.22.33.44.55.66.77.88.99.aa.bb.cc.dd.ee.ff。

`candump` 工具显示的数据部分采用的是大写的十六进制格式。例如：00 11 22 33 44 55 66
77 88 99 AA BB CC DD EE FF。

显示数据的大小写不影响实际的 CAN 帧内容。这只是 cansend 和 candump 工具在显示格式上的差异，实际发送和接收的数据是一致的。

18.9.4 cangen

cangen 工具用于生成和发送随机的 CAN 帧，通常用于测试 CAN 网络的性能和行为。我们可以在 SSH 端指定 cangen 发送数据到 can1，执行 cangen 指令可以查看使用说明：

```
root@ATK-DLMP257:~# cangen
cangen - CAN frames generator.

Usage: cangen [options] <CAN interface>
Options:
  -g <ms>          (gap in milli seconds - default: 200 ms)
  -a              (use absolute time for gap)
  -t              (use SO_TXTIME)
  --start <ns>    (start time (UTC nanoseconds))
  --mark <id>    (set SO_MARK to <id>, default 1)
  -e              (generate extended frame mode (EFF) CAN frames)
  -f              (generate CAN FD CAN frames)
  -b              (generate CAN FD CAN frames with bitrate switch (BRS))
  -E              (generate CAN FD CAN frames with error state (ESI))
  -R              (generate RTR frames)
  -8              (allow DLC values greater than 8 for Classic CAN frames)
  -m              (mix -e -f -b -E -R frames)
  -I <mode>       (CAN ID generation mode - see below)
  -L <mode>       (CAN data length code (dlc) generation mode - see below)
  -D <mode>       (CAN data (payload) generation mode - see below)
  -p <timeout>    (poll on -ENOBUFS to write frames with <timeout> ms)
  -n <count>      (terminate after <count> CAN frames - default infinite)
  -i              (ignore -ENOBUFS return values on write() syscalls)
  -x              (disable local loopback of generated CAN frames)
  -c <count>      (number of messages to send in burst, default 1)
  -v              (increment verbose level for printing sent CAN frames)

Generation modes:
'r' => random values (default)
'e' => random values, even ID
'o' => random values, odd ID
'i' => increment values
<value> => fixed value (in hexadecimal for -I and -D)
=> nibbles written as 'x' are randomized (only -D)
```

图 18.9-6 cangen 使用说明

下面是 cangen 的详细使用说明和参数解析。

cangen: 工具名称，用于生成和发送 CAN 帧。

Usage: 使用格式为：

cangen [options] <CAN interface>

<CAN interface>: 指定要发送 CAN 帧的 CAN 接口，例如 can0、can1 等。

选项 (Options)

- **-g<ms>:** 设置每个帧之间的间隔时间，默认是 200 毫秒。可以使用小数点，例如 -g 4.73。
- **-a:** 使用绝对时间间隔。
- **-t:** 使用 SO_TXTIME 选项。
- **--start <ns>:** 设置开始时间（UTC 纳秒）。
- **--mark <id>:** 设置 SO_MARK 标志，默认值为 1。
- **-e:** 生成扩展帧格式（EFF）的 CAN 帧。
- **-f:** 生成 CAN FD 帧。
- **-b:** 生成带位速率开关（BRS）的 CAN FD 帧。
- **-E:** 生成带错误状态（ESI）的 CAN FD 帧。
- **-R:** 生成远程请求帧（RTR）。
- **-8:** 允许经典 CAN 帧的 DLC 值大于 8。
- **-m:** 混合生成扩展帧、CAN FD 帧、带位速率开关和错误状态的帧以及远程请求帧。
- **-I<mode>:** 设置 CAN ID 的生成模式。
- **-L<mode>:** 设置 CAN 数据长度码（DLC）的生成模式。
- **-D<mode>:** 设置 CAN 数据（有效载荷）的生成模式。

- -p <timeout>: 在 -ENOBUFFS 错误时轮询以 <timeout> 毫秒为超时写入帧。
- -n <count>: 在发送 <count> 个 CAN 帧后终止, 默认为无限制。
- -i: 忽略 write() 系统调用中的 -ENOBUFFS 返回值。
- -x: 禁用生成的 CAN 帧的本地回环。
- -c <count>: 一次突发发送的消息数量, 默认是 1。
- -v: 增加详细级别以打印发送的 CAN 帧。

生成模式 (Generation modes)

- 'r': 随机值 (默认)。
- 'e': 随机值, 偶数 ID。
- 'o': 随机值, 奇数 ID。
- 'i': 递增值。
- <value>: 固定值 (-I 和 -D 使用十六进制)。写作 'x' 的半字节是随机化的 (仅适用于 -D)。

使用示例:

- 基本使用:

打开串口端, 执行 candump 指令打开 can1 查看接收。

candump -ta can1

```
root@ATK-DLMP257:~# candump -ta can1
```

图 18.9-7 can1 准备接收数据

打开 SSH 端, 生成并发送随机 CAN 帧到 can1 接口:

cangen can1

```
root@ATK-DLMP257:~# cangen can1
```

图 18.9-8 发送随机 CAN 帧到 can1

此时查看串口终端, 可以看到 can1 接收到随机 CAN 帧数据。测试完成可以关闭查看 can1。

```
root@ATK-DLMP257:~# candump -ta can1
(1709072908.156417) can1 247 [6] B7 DA 6D 51 DD 59
(1709072908.356541) can1 5A7 [7] DF 7C 62 1C D4 50 77
(1709072908.556675) can1 112 [8] 08 43 4E 4A D2 BD 1F 7F
(1709072908.756781) can1 053 [8] 77 9D 03 7C 53 AA 5D 18
(1709072908.956855) can1 2A5 [0]
(1709072909.157032) can1 120 [8] D3 69 C5 58 18 1E 09 0B
(1709072909.357055) can1 199 [1] E8
(1709072909.557174) can1 0B7 [0]
(1709072909.757317) can1 4B7 [6] DC 94 A4 46 5E 1A
(1709072909.957444) can1 6BD [8] 32 6B EE 0F D0 3F C0 07
(1709072910.157547) can1 114 [8] A2 FD DF 06 67 39 16 6D
(1709072910.357658) can1 624 [8] BA E3 73 05 C9 18 9E 0A
```

图 18.9-9 can1 接收到的随机 CAN 帧数据

- 控制发送频率和数据长度

可以通过选项控制发送频率和数据长度。例如, 每隔 10 毫秒发送一个帧, 数据长度为 8:

cangen can1 -g 10 -L 8

```
root@ATK-DLMP257:~# candump -ta can1
(1709073063.361699) can1 367 [8] B0 87 54 3D 1B 06 86 36
(1709073063.371814) can1 444 [8] 22 11 53 69 62 42 7D 73
(1709073063.381921) can1 2E4 [8] 41 16 46 28 B2 8F E0 68
(1709073063.392021) can1 0DA [8] 4E 4A A4 5B EC 74 55 6E
(1709073063.402133) can1 6FF [8] 83 17 CF 42 09 3A E6 0F
(1709073063.412234) can1 6F5 [8] B8 A6 EE 6C 76 0A 34 62
(1709073063.422349) can1 7C1 [8] F9 0E 2F 6F CC 28 B5 17
(1709073063.432447) can1 63D [8] 81 B4 63 2E F1 CA 46 71
```

图 18.9-10 can1 接收到的 CAN 数据

- 生成特定 ID 的随机数据

生成并发送特定 CAN ID 的随机数据帧。例如，CAN ID 为 123，数据长度为 8：

```
cangen can1 -I 123 -L 8
```

```
root@ATK-DLMP257:~# candump -ta can1
(1709073141.787866) can1 123 [8] 1A F8 26 19 65 29 E0 05
(1709073141.987951) can1 123 [8] F9 08 9B 50 27 3E AB 61
(1709073142.188051) can1 123 [8] ED 96 75 3C D8 0E 5A 79
(1709073142.282352) can1 123 [8] BB C4 1A 5E AF 76 98 24
```

图 18.9-11 can1 接收到的 CAN 数据

- 使用随机数据模式

使用随机数据模式，可以生成随机数据帧。例如，每隔 100 毫秒发送一个随机数据帧，CAN ID 为 200：

```
cangen can1 -g 100 -I 200 -D r
```

```
root@ATK-DLMP257:~# candump -ta can1
(1709073171.350540) can1 200 [1] A7
(1709073171.450718) can1 200 [8] 69 DE 32 72 4E 1A 34 0C
(1709073171.550786) can1 200 [1] 42
(1709073171.650962) can1 200 [8] 15 54 4E 3E F6 C4 91 1D
(1709073171.751083) can1 200 [8] D3 EF 9F 0F 5C 16 DB 19
(1709073171.851187) can1 200 [8] 29 77 6E 4C FA 99 64 77
(1709073171.951262) can1 200 [6] 90 5D 76 43 CF 65
(1709073172.051381) can1 200 [8] F0 24 76 77 57 46 AD 43
(1709073172.151478) can1 200 [8] 43 65 D0 27 C3 6E 03 3E
(1709073172.251581) can1 200 [8] B8 3B C4 65 D2 2C 2A 10
(1709073172.263696) can1 200 [8] 69 00 C9 0B 79 E5 C6 00
```

图 18.9-12 can1 接收到的 CAN 数据

18.9.5 canplayer

canplayer 工具用于回放之前记录的 CAN 日志，通常用于重现某些特定的 CAN 总线活动。执行下面指令可以查看使用说明：

```
canplayer -h

root@ATK-DLMP257:~# canplayer -h
canplayer: invalid option -- 'h'
canplayer - replay a compact CAN frame logfile to CAN devices.

Usage: canplayer <options> [interface assignment]*

Options:
  -I <infile>    (default stdin)
  -l <num>        (process input file <num> times)
                  (Use 'i' for infinite loop - default: 1)
  -t             (ignore timestamps: send frames immediately)
  -i             (interactive - wait for ENTER key to process next frame)
  -n <count>     (terminate after processing <count> CAN frames)
  -g <ms>        (gap in milli seconds - default: 1 ms)
  -s <s>         (skip gaps in timestamps > 's' seconds)
  -x             (disable local loopback of sent CAN frames)
  -v             (verbose: print sent CAN frames)

Interface assignment:
  0..n assignments like <write-if>=<log-if>

  e.g. vcan2=can0 (send frames received from can0 on vcan2)
  extra hook: stdout=can0 (print logfile line marked with can0 on stdout)
  No assignments => send frames to the interface(s) they had been received from

Lines in the logfile not beginning with '(' (start of timestamp) are ignored.

root@ATK-DLMP257:~#
```

图 18.9-13 canplayer 使用说明

下面是 canplayer 的详细使用说明和参数解析。

canplayer: 工具名称，用于回放 CAN 日志。

Usage: 使用格式为：

canplayer <options> [interface assignment]*

- <options>: 各种选项，用于控制回放行为。
- [interface assignment]*: 接口分配，用于将日志文件中的帧发送到特定的 CAN 接口。

选项 (Options)

- -I <infile>: 指定输入文件，默认是标准输入 (stdin)。
- -l <num>: 处理输入文件的次数，默认为 1 次。使用 i 表示无限循环。
- -t: 忽略时间戳，立即发送帧。
- -i: 交互模式，按 ENTER 键处理下一帧。
- -n <count>: 在处理指定数量的 CAN 帧后终止。
- -g <ms>: 设置帧之间的间隔时间，默认为 1 毫秒。
- -s <s>: 跳过时间戳间隔超过指定秒数的帧。
- -x: 禁用发送 CAN 帧的本地回环。
- -v: 详细模式，打印发送的 CAN 帧。

接口分配 (Interface assignment)

<write-if>=<log-if>

- 例如，can1=can0 表示将从 can0 接收到的帧发送到 can1。
- 额外的挂钩: stdout=can1 表示将标记为 can1 的日志文件行打印到标准输出。
- 如果没有分配，帧将发送到它们被接收到的接口。

日志文件格式



日志文件中不以（开头的行（时间戳的开始）将被忽略。

生成 candump.log 示例:

1、使用 candump 工具将 CAN 总线上的数据记录到文件中。

```
candump can1 -l > candump.log
```

- candump: 工具名称, 用于监听和记录 CAN 总线上的数据。
- can1: 要监听的 CAN 接口。
- -l: 启用日志记录模式, 默认情况下将数据记录到 candump.log 文件中。
- >: 将输出重定向到 candump.log 文件中。

```
root@ATK-DLMP257:~# candump can1 -l > candump.log
Disabled standard output while logging.
Enabling Logfile 'candump-2024-02-28_063413.log'
```

图 18.9-14 记录接收到的 CAN 数据到 candump.log 文件

2、打开 SSH 终端, 并在 SSH 终端中使用 cansend 指令从 can0 发送数据。

```
cansend can0 123#01.01.01.01.01.01.01.01
cansend can0 123#02.02.02.02.02.02.02.02
cansend can0 123#03.03.03.03.03.03.03.03
cansend can0 123#04.04.04.04.04.04.04.04
cansend can0 123#05.05.05.05.05.05.05.05
cansend can0 123#06.06.06.06.06.06.06.06
cansend can0 123#07.07.07.07.07.07.07.07
cansend can0 123#08.08.08.08.08.08.08.08
```

```
root@ATK-DLMP257:~# cansend can0 123#01.01.01.01.01.01.01.01
cansend can0 123#06.06.06.06.06.06.06.06
cansend can0 123#07.07.07.07.07.07.07.07
cansend can0 123#08.08.08.08.08.08.08.08
root@ATK-DLMP257:~# cansend can0 123#02.02.02.02.02.02.02.02
root@ATK-DLMP257:~# cansend can0 123#03.03.03.03.03.03.03.03
root@ATK-DLMP257:~# cansend can0 123#04.04.04.04.04.04.04.04
root@ATK-DLMP257:~# cansend can0 123#05.05.05.05.05.05.05.05
root@ATK-DLMP257:~# cansend can0 123#06.06.06.06.06.06.06.06
root@ATK-DLMP257:~# cansend can0 123#07.07.07.07.07.07.07.07
root@ATK-DLMP257:~# cansend can0 123#08.08.08.08.08.08.08.08
root@ATK-DLMP257:~#
```

图 18.9-15 cansend 发送数据

3、can0 发送完数据后, can1 这边取消接收, 执行 Ctrl+C 可以取消接收。此时可以看到系统中生成了 candump.log 文件。

```
root@ATK-DLMP257:~# candump can1 -l > candump.log
Disabled standard output while logging.
Enabling Logfile 'candump-2024-02-28_063413.log'
^Croot@ATK-DLMP257:~# ls
README-CHECK-GPU candump-2024-02-28_063413.log candump.log shell
root@ATK-DLMP257:~#
```

图 18.9-16 生成 candump.log 文件

图中 candump-2024-02-28_063413.log 就是实际存放 CAN 数据的文件, 具有时间标识, 以自己实际的系统时间为准。这里 candump.log 文件为空白文件。我们后面要使用 candump-2024-02-28_063413.log 文件, 均简称为 candump.log 文件。

```
root@ATK-DLMP257:~# cat candump.log
root@ATK-DLMP257:~# cat candump-2024-02-28_063413.log
(1709073282.189146) can1 123#0101010101010101
(1709073282.193202) can1 123#0202020202020202
(1709073282.197958) can1 123#0303030303030303
(1709073282.201892) can1 123#0404040404040404
(1709073282.205594) can1 123#0505050505050505
(1709073282.209679) can1 123#0606060606060606
(1709073282.213623) can1 123#0707070707070707
(1709073282.217762) can1 123#0808080808080808
root@ATK-DLMP257:~#
```

图 18.9-17 candump 文件数据

接下来我们看下 canplayer 的使用示例:

- 基本使用

使用 cat 指令查看 candump.log 内容:

```
root@ATK-DLMP257:~# cat candump-2024-02-28_063413.log
(1709073282.189146) can1 123#0101010101010101
(1709073282.193202) can1 123#0202020202020202
(1709073282.197958) can1 123#0303030303030303
(1709073282.201892) can1 123#0404040404040404
(1709073282.205594) can1 123#0505050505050505
(1709073282.209679) can1 123#0606060606060606
(1709073282.213623) can1 123#0707070707070707
(1709073282.217762) can1 123#0808080808080808
root@ATK-DLMP257:~#
```

图 18.9-18 candump.log 内容

确保日志文件的每一行格式如下:

(时间戳) 接口名 CAN ID#数据

这里根据接口名可以知道接口为 can1, 在 SSH 端使用 candump 指令打开 can1 准备查看接收数据。

candump can1

```
root@ATK-DLMP257:~# candump can1
```

图 18.9-19 candump 指令打开指定接口

在串口端执行 canplayer 指令回放一个名为 candump.log 的日志文件:

canplayer -I candump-2024-02-28_063413.log

```
root@ATK-DLMP257:~# canplayer -I candump-2024-02-28_063413.log
root@ATK-DLMP257:~# |
```

图 18.9-20 canplayer 指令回放日志文件

此时在 SSH 端的 candump 可以看到对应的 CAN 数据回放, 是 candump.log 里的数据。执行 Ctrl + C 按键可以关闭 candump。

```
root@ATK-DLMP257:~# candump can1
can1 123 [8] 01 01 01 01 01 01 01 01
can1 123 [8] 02 02 02 02 02 02 02 02
can1 123 [8] 03 03 03 03 03 03 03 03
can1 123 [8] 04 04 04 04 04 04 04 04
can1 123 [8] 05 05 05 05 05 05 05 05
can1 123 [8] 06 06 06 06 06 06 06 06
can1 123 [8] 07 07 07 07 07 07 07 07
can1 123 [8] 08 08 08 08 08 08 08 08
```

图 18.9-21 查看回放的 CAN 数据

- 循环回放

无限循环处理输入文件:

canplayer -I candump-2024-02-28_063413.log -l i

将参数 i 换成对应的循环次数，就是有限循环处理输入文件，比如将参数 i 换成 3 就是处理输入文件 3 次。

```
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# canplayer -I candump-2024-02-28_063413.log -l 3
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# candump can1
can1 123 [8] 01 01 01 01 01 01 01 01
can1 123 [8] 02 02 02 02 02 02 02 02
can1 123 [8] 03 03 03 03 03 03 03 03
can1 123 [8] 04 04 04 04 04 04 04 04
can1 123 [8] 05 05 05 05 05 05 05 05
can1 123 [8] 06 06 06 06 06 06 06 06
can1 123 [8] 07 07 07 07 07 07 07 07
can1 123 [8] 08 08 08 08 08 08 08 08
can1 123 [8] 01 01 01 01 01 01 01 01
can1 123 [8] 02 02 02 02 02 02 02 02
can1 123 [8] 03 03 03 03 03 03 03 03
can1 123 [8] 04 04 04 04 04 04 04 04
can1 123 [8] 05 05 05 05 05 05 05 05
can1 123 [8] 06 06 06 06 06 06 06 06
can1 123 [8] 07 07 07 07 07 07 07 07
can1 123 [8] 08 08 08 08 08 08 08 08
can1 123 [8] 01 01 01 01 01 01 01 01
can1 123 [8] 02 02 02 02 02 02 02 02
can1 123 [8] 03 03 03 03 03 03 03 03
can1 123 [8] 04 04 04 04 04 04 04 04
can1 123 [8] 05 05 05 05 05 05 05 05
can1 123 [8] 06 06 06 06 06 06 06 06
can1 123 [8] 07 07 07 07 07 07 07 07
can1 123 [8] 08 08 08 08 08 08 08 08
```

图 18.9-22 循环处理输入文件

- 忽略时间戳立即发送帧

忽略时间戳，立即发送帧：

canplayer -I candump-2024-02-28_063413.log -t

```
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# canplayer -I candump-2024-02-28_063413.log -t
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# candump can1
can1 123 [8] 01 01 01 01 01 01 01 01
can1 123 [8] 02 02 02 02 02 02 02 02
can1 123 [8] 03 03 03 03 03 03 03 03
can1 123 [8] 04 04 04 04 04 04 04 04
can1 123 [8] 05 05 05 05 05 05 05 05
can1 123 [8] 06 06 06 06 06 06 06 06
can1 123 [8] 07 07 07 07 07 07 07 07
can1 123 [8] 08 08 08 08 08 08 08 08
```

图 18.9-23 立刻发送帧

- 交互模式

交互模式下按 ENTER 键处理下一帧：

canplayer -I candump-2024-02-28_063413.log -i

```
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# canplayer -I candump-2024-02-28_063413.log -i
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# canplayer -I candump-2024-02-28_063413.log -i
interactive mode: press ENTER to process next CAN frame ...
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# candump can1
can1 123 [8] 01 01 01 01 01 01 01 01
```

图 18.9-24 交互模式

- 处理特定数量的 CAN 帧

例如处理 5 个 CAN 帧后终止：

canplayer -I candump-2024-02-28_063413.log -n 5

```
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# candump can1
can1 123 [8] 01 01 01 01 01 01 01 01
can1 123 [8] 02 02 02 02 02 02 02 02
can1 123 [8] 03 03 03 03 03 03 03 03
can1 123 [8] 04 04 04 04 04 04 04 04
can1 123 [8] 05 05 05 05 05 05 05 05
root@ATK-DLMP257:~# canplayer -I candump-2024-02-28_063413.log -n 5
root@ATK-DLMP257:~#
```

图 18.9-25 处理特定数量的 CAN 帧

- 设置帧间隔

设置帧之间的间隔时间为 5 毫秒:

`canplayer -I candump-2024-02-28_063413.log -g 5`

```
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# candump can1
can1 123 [8] 01 01 01 01 01 01 01 01
can1 123 [8] 02 02 02 02 02 02 02 02
can1 123 [8] 03 03 03 03 03 03 03 03
can1 123 [8] 04 04 04 04 04 04 04 04
can1 123 [8] 05 05 05 05 05 05 05 05
can1 123 [8] 06 06 06 06 06 06 06 06
can1 123 [8] 07 07 07 07 07 07 07 07
can1 123 [8] 08 08 08 08 08 08 08 08
root@ATK-DLMP257:~# canplayer -I candump-2024-02-28_063413.log -g 5
root@ATK-DLMP257:~#
```

图 18.9-26 设置帧间隔

- 详细模式

详细模式下打印发送的 CAN 帧:

`canplayer -I candump-2024-02-28_063413.log -v`

```
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# candump can1
can1 123 [8] 01 01 01 01 01 01 01 01
can1 123 [8] 02 02 02 02 02 02 02 02
can1 123 [8] 03 03 03 03 03 03 03 03
can1 123 [8] 04 04 04 04 04 04 04 04
can1 123 [8] 05 05 05 05 05 05 05 05
can1 123 [8] 06 06 06 06 06 06 06 06
can1 123 [8] 07 07 07 07 07 07 07 07
can1 123 [8] 08 08 08 08 08 08 08 08
root@ATK-DLMP257:~# canplayer -I candump-2024-02-28_063413.log -v
can1 (can1) 123 [8] 01 01 01 01 01 01 01 01
can1 (can1) 123 [8] 02 02 02 02 02 02 02 02
can1 (can1) 123 [8] 03 03 03 03 03 03 03 03
can1 (can1) 123 [8] 04 04 04 04 04 04 04 04
can1 (can1) 123 [8] 05 05 05 05 05 05 05 05
can1 (can1) 123 [8] 06 06 06 06 06 06 06 06
can1 (can1) 123 [8] 07 07 07 07 07 07 07 07
can1 (can1) 123 [8] 08 08 08 08 08 08 08 08
root@ATK-DLMP257:~#
```

图 18.9-27 详细模式

通过这些选项和示例, 用户可以灵活地使用 `canplayer` 工具回放 CAN 日志, 测试和调试 CAN 网络中的各种场景。

第十九章 MMC 子系统

19.1 MMC 子系统简介

嵌入式 Linux 的 MMC 子系统是管理和访问多媒体卡（MMC）和安全数字卡（SD 卡）的核心组件。这个子系统提供了与 MMC/SD 存储设备交互的接口，支持各种卡类型和操作，包括数据读写、卡识别和初始化等。

19.2 主要特点

1、支持多种卡类型

- 多媒体卡（MMC）：支持标准的 MMC 卡，适用于各种嵌入式存储应用。
- 安全数字卡（SD）：支持 SD、SDHC（大容量 SD）、SDXC（扩展容量 SD）等卡类型，广泛用于消费电子和嵌入式系统。
- 嵌入式 MMC（eMMC）：支持内嵌存储解决方案，适用于智能手机、平板电脑和其他嵌入式设备。
- SDIO（Secure Digital Input Output）卡：支持带有输入/输出功能的 SD 卡，如 Wi-Fi、蓝牙模块。

2、卡识别和初始化

- 自动识别：在卡插入时，自动识别卡类型、容量和其他参数。
- 初始化：完成卡的初始化，包括读取 CID（卡标识符）、CSD（卡特定数据）等信息，以确定卡的能力和配置。

3、数据传输支持

- 多种数据传输模式：
- 单块传输：每次传输一个数据块。
- 多块传输：一次传输多个数据块，提高数据传输效率。
- 高效的数据传输：支持高速模式（High-Speed）和超高速模式（Ultra High-Speed），实现快速数据读写。

4、低级别命令和响应处理

- 标准命令支持：支持 MMC/SD 协议中的各种标准命令，如读写命令、状态命令等。
- 高级命令支持：支持高级命令和特性，如高速模式切换、擦除命令等。
- 响应处理：处理卡响应，包括成功响应、错误响应和状态响应。

5、电源管理和低功耗特性

- 低功耗模式：支持卡的低功耗模式，包括睡眠模式和休眠模式，减少电力消耗。
- 电源控制：管理卡的电源状态，包括卡的上电、断电和重置操作。

6、卡状态和事件处理

- 卡状态监控：实时监控卡的状态变化，如插入、移除、错误等。
- 事件通知：在卡状态变化时，通过中断或轮询机制通知系统和用户，确保及时响应。

7、文件系统支持

- 标准文件系统：支持在卡上使用常见的文件系统，如 FAT32、exFAT、EXT4 等。
- 块设备接口：通过块设备接口访问卡，支持标准的块设备操作，如读取、写入、格式化等。

8、热插拔支持

- 即插即用：支持卡的热插拔操作，用户可以在系统运行过程中随时插入或移除卡，系统能够自动检测并处理卡的插拔事件。
- 数据保护：在卡插拔时，确保数据的完整性和系统的稳定性，防止数据丢失或损坏。

9、错误检测和恢复

- 错误检测：能够检测各种传输错误、卡错误和协议错误，并采取相应的处理措施。
- 错误恢复：支持错误恢复机制，如重传、重试和卡重置，确保数据传输的可靠性和系统的稳定性。

嵌入式 Linux 的 MMC 子系统通过支持多种卡类型、提供高效的数据传输、管理卡的电源状态和处理卡状态变化等特性，为嵌入式系统中的存储解决方案提供了强大的支持。其低功耗特性、热插拔支持和错误恢复能力，使得它在各种嵌入式应用中得到广泛应用和信赖。

19.3 组成部分

嵌入式 Linux 的 MMC (MultiMediaCard) 子系统由多个关键组成部分构成，每个部分都负责处理特定的功能和任务，以确保对 MMC/SD 卡的有效管理和操作。以下是 MMC 子系统的主要组成部分及其功能：

1、主控制器驱动 (Host Controller Driver)

功能：负责与硬件主控制器交互，发送命令和数据，接收响应。控制数据传输的低级操作，如时钟配置、数据线控制和中断处理。

示例：

`sdhci.c`：SD Host Controller Interface 驱动，用于支持通用的 SDHCI 主控制器。

2、核心层 (Core Layer)

功能：提供公共功能和接口，如卡识别、命令处理和错误处理。负责管理多个主控制器和卡设备的协调工作，确保系统对所有 MMC/SD 设备的一致性操作。

示例：

`mmc_core.c`：核心层实现，包括卡初始化、数据传输和命令处理等功能。

3、卡驱动 (Card Driver)

功能：针对不同类型的卡（如 MMC、SD、SDIO 等）提供特定的处理逻辑。处理卡的初始化、配置和特定功能，例如 SDIO 卡的 I/O 操作。

示例：

`mmc_block.c`：块设备驱动，处理文件系统操作，如读写块设备。

`sdio.c`：用于处理 SDIO 设备的特定驱动逻辑。



4、文件系统支持 (File System Support)

功能: 支持在 MMC/SD 卡上使用标准文件系统, 如 FAT32、exFAT、EXT4 等。提供块设备接口, 通过通用的块设备操作访问卡上的数据。

示例:

vfat: FAT 文件系统支持。

ext4: EXT4 文件系统支持。

5、电源管理 (Power Management)

功能: 管理卡的电源状态, 包括上电、断电和重置操作。支持卡的低功耗模式, 如睡眠模式和休眠模式, 以减少电力消耗。

示例:

在驱动代码中实现的电源管理函数, 如 `mmc_power_up` 和 `mmc_power_off`。

6、热插拔支持 (Hotplug Support)

功能: 支持卡的热插拔操作, 用户可以在系统运行过程中随时插入或移除卡。处理卡插拔事件, 确保系统及时响应和数据完整性。

示例:

使用热插拔机制, 如 `udev` 守护进程, 通过内核事件通知用户空间的插拔操作。

7、错误检测和恢复 (Error Detection and Recovery)

功能: 负责检测传输错误、卡错误和协议错误, 并采取相应的处理措施。提供错误恢复机制, 如重传、重试和卡重置, 确保数据传输的可靠性。

示例:

在驱动代码中实现的错误处理函数, 如 `mmc_handle_error`。

8、中断处理 (Interrupt Handling)

功能: 处理来自主控制器的中断, 管理数据传输和命令完成的通知。确保高效的中断响应, 减少数据传输延迟。

示例:

中断处理函数在驱动代码中, 如 `sdhci_irq`。

9、数据传输引擎 (Data Transfer Engine)

功能: 管理数据的读写操作, 支持单块传输和多块传输模式。提供高速数据传输支持, 提高系统的整体性能。

示例:

在驱动中实现的传输函数, 如 `mmc_blk_rw_rq`。

嵌入式 Linux 的 MMC 子系统由主控制器驱动、核心层、卡驱动、文件系统支持、电源管理、热插拔支持、错误检测和恢复、中断处理和数据传输引擎等多个部分组成。这些部分紧密协作, 确保对 MMC/SD 卡的高效管理和操作, 满足各种嵌入式应用的需求。通过合理配置和使用这些组件, 系统可以实现可靠的存储和数据传输解决方案。

19.4 MP257 的 MMC 说明

SDMMC 是 Secure Digital MultiMediaCard 的缩写, 它结合了 SD 卡和 MMC 卡的技术, 用于描述一种支持 SD 和 MMC 协议的通用接口。在嵌入式系统 (例如 STM32 芯片或其他 SoC) 中, SDMMC 通常指硬件模块或外设, 它支持通过 SD 和 MMC 协议与存储卡进行通信。

19.4.1 文档和资源

想要获取 MP257 系列处理器的 SDMMC 详细说明, 最直接的方式是参考 ST 官方文档, 包括:

1、参考手册 (Reference Manual): 提供了详细的寄存器级说明, 包括 SDMMC 模块的寄存器映射以及配置方法等。此文档已放在【正点原子】STM32MP257 开发板 (A 盘)-基础资料\07、STM32MP2 参考资料

2、数据手册 (Data Sheet): 提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板 (A 盘)-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板, 正点原子还提供了一系列的开发文档, 便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南 (即本文档)

应用开发: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

19.4.2 特性说明

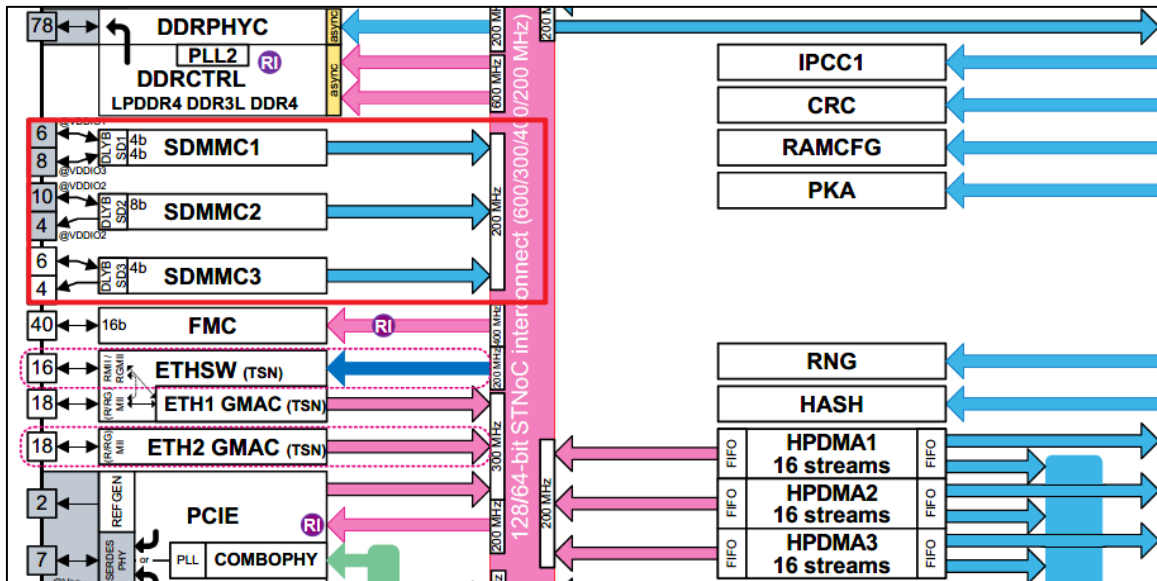


图 19.4-1 MP257 功能框图中的 SDMMC 部分

以下是 MP257 芯片收据手册中关于 SDMMC 的说明:

3.56 Secure digital input/output MultiMediaCard interface (SDMMC1/2/3)

Three SDMMCs provide an interface between the AHB bus and SD memory cards, SDIO and eMMC devices. SDMMC features include the following:

- Full compliance with MultiMediaCard System Specification Version 5.1

Card support for three different databus modes: 1-bit (default), 4-bit and 8-bit (HS200 speed limited by maximum allowed I/O speed, HS400 is not supported).

- Full compatibility with previous versions of MultiMediaCards (backward compatibility)
- Full compliance with SD memory card specifications version 6.0 (SDR104 SDMMC_CK speed limited to maximum allowed I/O speed, SPI and UHS-II modes not supported)
- Full compliance with SDIO card specification version 4.0

Card support for two different databus modes: 1-bit (default) and 4-bit (SDR104 SDMMC_CK speed limited to maximum allowed I/O speed, SPI and UHS-II modes not supported)

- Data transfer up to 208 Mbyte/s for the 8-bit mode (depending on the maximum allowed I/O speed)
- Data and command output enable signals to control external bidirectional drivers
- The SDMMC host interface embeds a dedicated DMA controller that allows high-speed transfers between the interface and the SRAM.
- IDMA linked list support

Each SDMMC is coupled with a delay block (DLYBSD) that supports an external data frequency above 100 MHz.

图 19.4-2 SDMMC 说明

MP257 的三个 SDMMC 接口提供了 AHB 总线与 SD 存储卡、SDIO 卡及 eMMC 设备之间的接口, 特性说明如下:

规范支持:

完全兼容 eMMC 规范 5.1: 支持三种数据总线模式 (1-bit (默认)、4-bit 和 8-bit 模式); 支持 HS200 模式 (速率受 I/O 速度限制), 但不支持 HS400 模式。

完全兼容 旧版 eMMC 卡 (向后兼容性)。

完全符合 SD 卡规范 6.0: 支持 SDR104 模式 (速率受 I/O 速度限制), 不支持 SPI 模式和 UHS-II 模式。

完全符合 SDIO 卡规范 4.0: 支持 1-bit (默认) 和 4-bit 数据总线模式; 支持 SDR104 模式, 不支持 SPI 模式和 UHS-II 模式。

性能特点:



在 8-bit 模式下, 数据传输速率可达 208 Mbyte/s (受限于最大允许 I/O 速度)。

提供数据和命令输出使能信号, 用于控制外部双向驱动器。

内嵌 DMA 控制器: 支持在接口与 SRAM 之间进行高速数据传输。

支持 基于链表的 IDMA (内部直接存储器访问)。

其他特性:

每个 SDMMC 接口配备一个延迟模块(DLYBSD), 支持超过 100 MHz 的外部数据频率。

更多关于 MP257 的 SDMMC 资料请查看芯片参考手册。

19.5 开发板 TF Card 接口

开发板上有只有 EMMC、TF Card 接口（即 micro SD card 接口）两种 SDMMC 设备，本小节我们主要围绕 MMC 系统，故以 TF card 接口为例讲解。

19.5.1 原理图分析

打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 TF。可以看到 TF Card 相关电路如下。

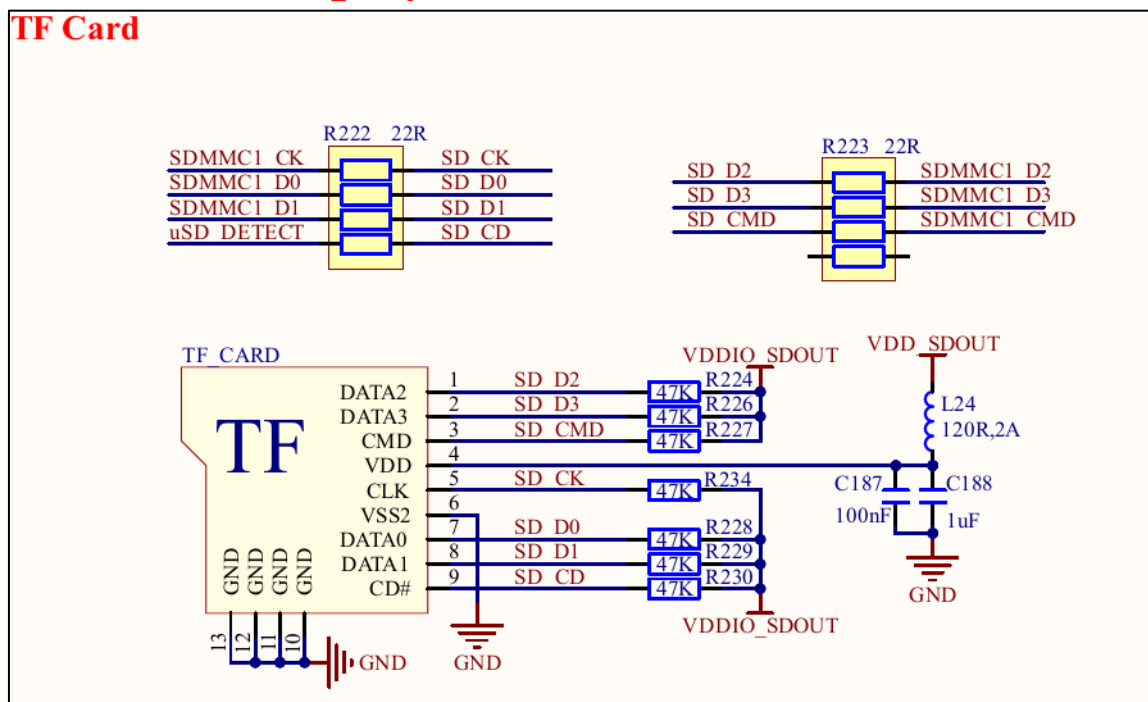


图 19.5-1 开发板 TF Card 电路

以下是对该原理图的详细解析：

主要组件

● TF 卡插槽：

插槽用于插入 TF 卡。

各引脚定义如下：

- DATA2: 数据线 2
- DATA3: 数据线 3
- CMD: 命令线
- VDD: 电源
- CLK: 时钟线
- VSS: 接地
- DATA0: 数据线 0
- DATA1: 数据线 1
- CD: 卡检测

● 电源连接：

VDD 连接到 VSD_3V3，这是 3.3V 电源供电。

VSS 连接到地 (GND)。

- 数据线和命令线:

数据线 DATA0 到 DATA3、命令线 CMD 和时钟线 CLK 都连接到对应的电阻和滤波电容, 以确保信号的完整性和减少噪声。

数据线、命令线和时钟线通过电阻连接到 SDMMC1 接口, 分别是 SDMMC1_D0 到 SDMMC1_D3、SDMMC1_CMD 和 SDMMC1_CLK。

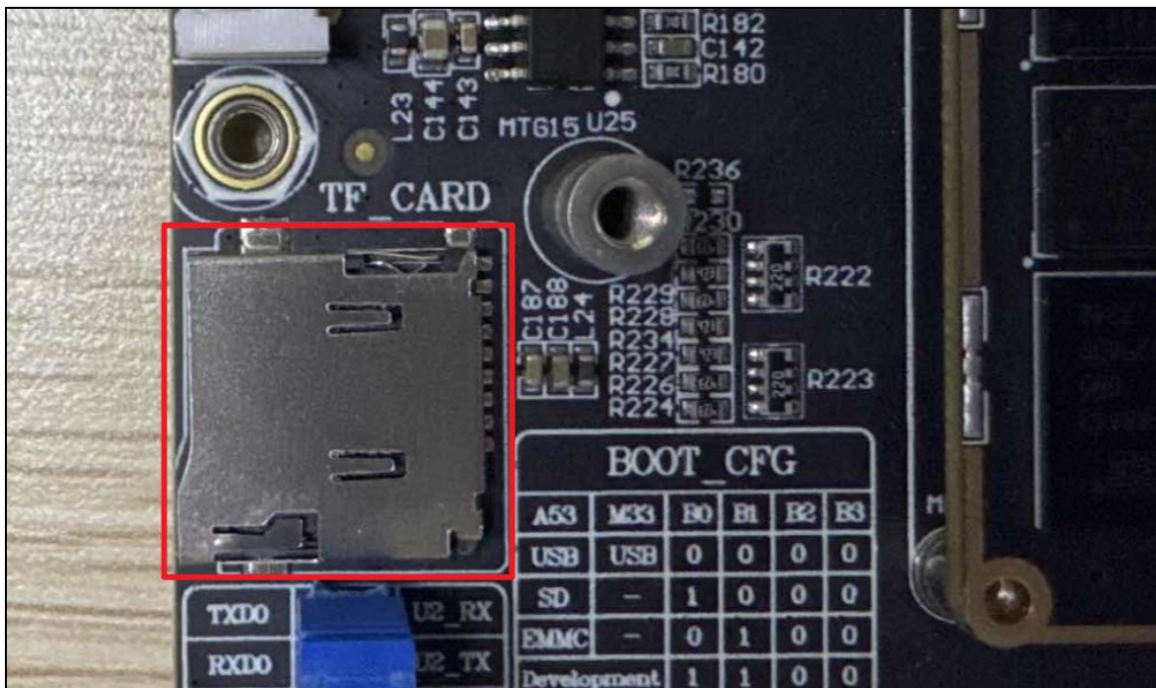
- 卡检测电路:

CD 引脚用于卡检测, 连接到 SD_CD。

这个 TF 卡接口电路设计提供了稳定和可靠的连接, 包括必要的滤波和保护电路, 以确保信号完整性和设备的长寿命。通过合理的电阻、电容和电感配置, 该设计能够有效地减少噪声和干扰, 确保 TF 卡的正常工作。

19.5.2 硬件实物

开发板上的 TF Card 接口位置如下图所示:



在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, MMC 相关配置如下(略有删减, 只列举重点部分):

- 1 CONFIG_MMC=y //启用对 MMC/SD/SDIO 卡的支持
- 2 CONFIG_MMC_BLOCK=y /*启用 MMC 块设备层, 提供 MMC 卡的块设备接口, 使其可以作为常规存储设备进行访问*/
- 3 CONFIG_MMC_BLOCK_MINORS=32 /*设置每个主设备号的次设备号数量为 32。这决定了系统可以处理的 MMC 设备的最大数量*/
- 4 CONFIG_MMC_ARMMMC=y /*启用 ARM PrimeCell MultiMediaCard 接口 (MMCI) 驱动程序的支持*/
- 5 CONFIG_MMC_STM32_SDMMC=y //启用对 STM32 微控制器上的 SD/MMC 控制器的支持。
- 6 CONFIG_MMC_SDHCI=y /*启用对 SD 主机控制器接口 (SDHCI) 的支持, 这是一个 SD 卡控制器的标准接口。*/
- 7 CONFIG_MMC_SDHCI_IO_ACCESSORS=y /*启用 SDHCI 的 I/O 访问器。它为 SDHCI 控制器所需的 I/O 操作提供抽象层*/
- 8 CONFIG_MMC_SDHCI_ACPI=y /*启用 SDHCI 的 ACPI (高级配置和电源接口) 支持。ACPI 用于提供硬件抽象并管理设备的电源*/
- 9 CONFIG_MMC_SDHCI_PLTFM=y /*启用 SDHCI 平台设备支持, 允许 SDHCI 控制器作为平台设备使用*/

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

-> Device Drivers

-> <*> MMC/SD/SDIO card support --->

-> --- MMC/SD/SDIO card support

-> <*> MMC block device driver

-> (32) Number of minors per block device

-> <*> ARM AMBA Multimedia Card Interface support

-> [*] STMicroelectronics STM32 SDMMC Controller

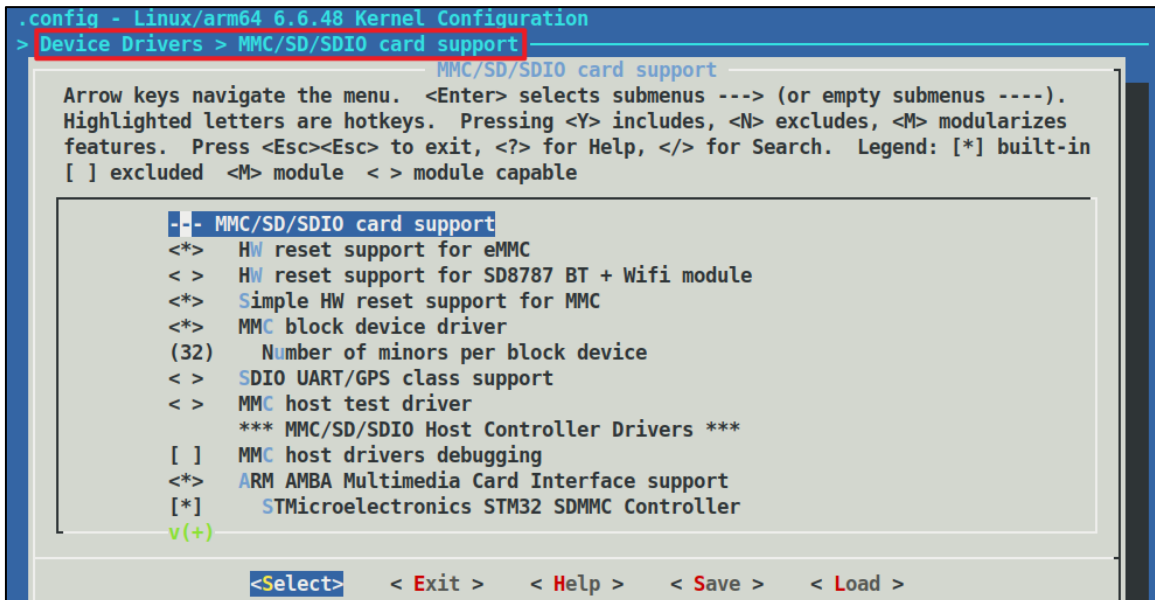


图 19.6-1 MMC 配置节选

19.7 设备树分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi, 此文件是特定于 STM32MP2 系列芯片的设备树包含文件, 通常不独立使用, 而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置, 而 .dtsi 则作为它们的公共部分。以 sdmmc1 为例, stm32mp251.dtsi 中关于 sdmmc1 的描述如下:

```
1 sdmmc1: mmc@48220000 {
2     compatible = "st,stm32mp25-sdmmc2", "arm,pl18x", "arm,primecell";
3     arm,primecell-periphid = <0x00353180>;
4     reg = <0x48220000 0x400>, <0x44230400 0x8>;
5     interrupts = <GIC_SPI 123 IRQ_TYPE_LEVEL_HIGH>;
6     clocks = <&rcc CK_KER_SDMMC1>;
7     clock-names = "apb_pclk";
8     resets = <&rcc SDMMC1_R>;
9     cap-sd-highspeed;
10    cap-mmc-highspeed;
11    max-frequency = <120000000>;
12    access-controllers = <&rifsc 76>;
13    power-domains = <&CLUSTER_PD>;
14    status = "disabled";
```


15 };

第 1 行, `sdmmc1: mmc@48220000` 定义了一个名为 `sdmmc1` 的节点, 表示一个 SDMMC 控制器, 地址为 `0x48220000`。

第 2 行, `compatible = "st,stm32mp25-sdmmc2", "arm,pl18x", "arm,primecell"` 该设备兼容以下三个设备模型:

`st,stm32mp25-sdmmc2`: 用于 STM32MP25 系列 SDMMC 控制器。

`arm,pl18x`: ARM 的 PL18x 系列控制器。

`arm,primecell`: ARM 的 PrimeCell 外设架构。

第 3 行, `arm,primecell-periphid = <0x00353180>` 表示该外设的 PrimeCell 唯一标识符 (Peripheral ID) 为 `0x00353180`。

第 4 行, 定义两个寄存器区域: 地址 `0x48220000`, 大小 `0x400`。地址 `0x44230400`, 大小 `0x8`。

第 5 行, `interrupts = <GIC_SPI 123 IRQ_TYPE_LEVEL_HIGH>` 定义了一个中断信号: 使用 GIC 的 SPI 123 中断号, 类型为 高电平触发。

第 6 行, `clocks = <&rcc CK_KER_SDMMC1>` 定义了该控制器使用的时钟信号: 来自 `rcc`, 时钟名称为 `CK_KER_SDMMC1`。

第 7 行, `clock-names = "apb_pclk"` 指定时钟的名称为 `apb_pclk`, 表示 APB 总线的外围设备时钟。

第 8 行, `resets = <&rcc SDMMC1_R>` 定义了复位控制器: 使用 `rcc` 的 `SDMMC1_R` 信号。

第 9 行, `cap-sd-highspeed` 支持 SD 卡的高速模式 (High-Speed Mode)。

第 10 行, `cap-mmc-highspeed` 支持 MMC 卡的高速模式 (High-Speed Mode)。

第 11 行, `max-frequency = <120000000>` SDMMC 控制器支持的最大工作频率为 120 MHz。

第 12 行, `access-controllers = <&rifsc 76>` 定义了一个访问控制器, 编号为 76, 来源于 `rifsc`。

第 13 行, `power-domains = <&CLUSTER_PD>` 该设备属于 `CLUSTER_PD` 电源域管理。

第 14 行, `status = "disabled"` 默认状态为 禁用 (disabled)。要启用该设备, 需要将其状态改为 "okay"。

以 2GB 版本核心板为例, 在 `arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts` 中, `sdmmc1` 相关配置如下:

```
1 &sdmmc1 {
2     pinctrl-names = "default", "opendrain", "sleep";
3     pinctrl-0 = <&sdmmc1_b4_pins_a>;
4     pinctrl-1 = <&sdmmc1_b4_od_pins_a>;
5     pinctrl-2 = <&sdmmc1_b4_sleep_pins_a>;
6     cd-gpios = <&gpioi 8 (GPIO_ACTIVE_LOW | GPIO_PULL_UP)>;
7     disable-wp;
8     st,neg-edge;
9     bus-width = <4>;
10    vmmc-supply = <&scmi_vdd_sdcard>;
11    vqmmc-supply = <&scmi_vddiol>;
12    sd-uhs-sdr12;
13    sd-uhs-sdr25;
14    sd-uhs-sdr50;
```

```
15     sd-uhs-ddr50;
16     sd-uhs-sdr104;
17     status = "okay";
18 };
```

第 1 行, `&sdmmc1` 引用了之前定义的 `sdmmc1` 节点, 开始对其配置。

第 2 行, `pinctrl-names = "default", "opendrain", "sleep"` 为设备定义了 3 个引脚控制模式: "default": 默认模式。"opendrain": 开漏模式。"sleep": 休眠模式。

第 3 行, `pinctrl-0 = <&sdmmc1_b4_pins_a>` 将默认模式下的引脚配置设置为 `sdmmc1_b4_pins_a`。

第 4 行, `pinctrl-1 = <&sdmmc1_b4_od_pins_a>` 将开漏模式下的引脚配置设置为 `sdmmc1_b4_od_pins_a`。

第 5 行, `pinctrl-2 = <&sdmmc1_b4_sleep_pins_a>` 将休眠模式下的引脚配置设置为 `sdmmc1_b4_sleep_pins_a`。

第 6 行, `cd-gpios = <&gpioi8 (GPIO_ACTIVE_LOW|GPIO_PULL_UP)>` 定义卡检测 (Card Detect) 引脚: 使用 GPIOI 控制器的第 8 个引脚。配置为低电平有效 (GPIO_ACTIVE_LOW)。配置了上拉电阻 (GPIO_PULL_UP)。

第 7 行, `disable-wp` 禁用写保护功能。

第 8 行, `st,neg-edge` 配置控制器使用 负边沿 进行数据采样。

第 9 行, `bus-width = <4>` 设置总线宽度为 4 位。

第 10 行, `vmmc-supply = <&scmi_vdd_sdcard>` 定义主电源 (VMMC) 的电压供应: 来源于 `scmi_vdd_sdcard`。

第 11 行, `vqmmc-supply = <&scmi_vddio1>` 定义信号电平 (VQMMC) 的电压供应: 来源于 `scmi_vddio1`。

第 12 行, `sd-uhs-sdr12` 启用 UHS-I 的 SDR12 模式 (支持 12.5 MB/s 的传输速度)。

第 13 行, `sd-uhs-sdr25` 启用 UHS-I 的 SDR25 模式 (支持 25 MB/s 的传输速度)。

第 14 行, `sd-uhs-sdr50` 启用 UHS-I 的 SDR50 模式 (支持 50 MB/s 的传输速度)。

第 15 行, `sd-uhs-ddr50` 启用 UHS-I 的 DDR50 模式 (支持双数据速率, 50 MB/s 的传输速度)。

第 16 行, `sd-uhs-sdr104` 启用 UHS-I 的 SDR104 模式 (支持 104 MB/s 的传输速度)。

第 17 行, `status = "okay"` 将设备的状态设置为 启用 (okay)。

19.8 系统接口

将一张 TF 卡接入到开发板的 TF 卡槽中, 系统会提醒接入信息, 参考如下:

```
root@ATK-DLMP257:~# [19223.214151] mmc0: new ultra high speed SDR104 SDHC card at address 5048
[19223.220822] mmcblk0: mmc0:5048 SD32G 28.9 GiB
[19223.222630] mmcblk0: p1
```

图 19.8-1 TF 卡接入时的信息

系统成功检测并初始化了一个 32GB 的 SDHC 卡, 并将其识别为块设备 `/dev/mmcblk1`, 该卡包含一个分区 `/dev/mmcblk1p1`, 具体如下:

`mmc1: new ultra high speed SDR104 SDHC card at address 5048`

描述: 在地址 5048 检测到一个新的超高速 SDR104 SDHC 卡。SDR104 是 UHS-I (超高速一代) 的一种模式, 支持高达 104 MB/s 的传输速率。

`mmcblk0: mmc0:5048 SD32G 28.9 GiB`

描述: 识别到的 SD 卡型号为 SD32G, 容量为 28.9 GiB (千兆二进制字节)。mmcblk0 是为该卡分配的块设备名称。

mmcblk0: p1

描述: 该 SD 卡包含一个分区, 分区名称为 p1。这意味着/dev/mmcblk1p0 是该分区的设备节点, 可以挂载和访问数据。

这里笔者的卡系统接口位于/sys/bus/mmc/devices/mmc0:5048 路径, 内容如下:

```
root@ATK-DLMP257:~# cd /sys/bus/mmc/devices/mmc0:5048/
root@ATK-DLMP257:/sys/bus/mmc/devices/mmc0:5048# ls
block  date      erase_size  manfid  oemid          rca  removable  serial  type
cid    driver    fwrev      name    power          scr  subsystem  uevent
csd     dsr       hwrev      ocr     preferred_erase_size  scr  subsystem
```

图 19.8-2 mmc1 目录

目录和文件解释

- block/: 包含与块设备相关的文件和目录。
- erase_size: 指示擦除块的大小。
- oemid: 原始设备制造商 ID。
- ssr: SD 卡的 SSR (Status Register) 状态寄存器。
- cid: 卡标识寄存器 (CID)。
- fwrev: 固件版本。
- power/: 电源管理相关信息。
- subsystem/: 子系统信息。
- csd: 卡特定数据寄存器 (CSD)。
- hwrev: 硬件版本。
- preferred_erase_size: 首选擦除块大小。
- type: 设备类型。
- date: 制造日期。
- manfid: 制造商 ID。
- rca: 相对地址。
- uevent: 用于用户空间事件通知。
- driver/: 驱动程序相关信息。
- name: 设备名称。
- scr: SD 配置寄存器 (SCR)。
- dsr: 驱动器阶段寄存器 (DSR)。
- ocr: 操作条件寄存器 (OCR)。
- serial: 序列号。

第二十章 Display 子系统

20.1 Display 子系统简介

嵌入式 Linux 的显示子系统通常包括以下几个主要组件：显示驱动、帧缓冲设备（Framebuffer）、显卡驱动、图形用户接口（GUI）库以及显示服务器等。这些组件协同工作，提供从硬件图形渲染到高级图形用户界面的全套功能。

20.2 主要特点

嵌入式 Linux 的显示子系统具有以下几个主要特点：

1、轻量级设计

嵌入式显示子系统设计得尽可能轻量级，以减少资源占用。这对于资源有限的嵌入式设备尤其重要，确保系统能够高效运行。

2、模块化结构

显示子系统由独立的组件组成，如显示驱动、帧缓冲、图形库和显示服务器。这种模块化设计使系统更易于定制和优化，以满足不同应用场景的需求。

3、多种显示接口支持

嵌入式 Linux 支持多种显示接口，包括 HDMI、LVDS、MIPI-DSI 和 VGA。这种多样性使系统能够适配各种显示屏和输出设备。

4、硬件加速支持

许多嵌入式系统的显示子系统支持硬件加速图形处理，通过 GPU 驱动和图形库（如 OpenGL ES）实现。这显著提升了图形渲染性能，适用于需要高性能图形处理的应用场景。

嵌入式 Linux 显示子系统通过其轻量级、模块化、多接口支持和硬件加速等特点，适应了嵌入式设备对高效、灵活和低功耗显示解决方案的需求。这些特点使嵌入式 Linux 成为构建复杂图形界面和高性能显示系统的理想选择。

20.3 组成部分

1、显示驱动 (Display Driver)

显示驱动程序是直接与显示硬件交互的内核模块。它们负责初始化显示设备、设置分辨率、管理帧缓冲区等。显示驱动通常依赖于设备树（Device Tree）中的硬件描述信息。

2、帧缓冲设备 (Framebuffer)

Framebuffer 是一种内核机制，它允许应用程序直接访问显示设备的显存（视频内存）。Framebuffer 设备通常在 `/dev/fb0` 等文件中表示。应用程序可以通过这些设备文件直接读写图像数据。

3、显卡驱动 (Graphics Driver)

显卡驱动程序（如 DRM/KMS 驱动）提供更高级的图形渲染功能。DRM（Direct Rendering Manager）和 KMS（Kernel Mode Setting）是 Linux 内核子系统，用于管理图形硬件并提供直接渲染支持。

4、图形用户接口库 (GUI Libraries)

嵌入式系统中常用的 GUI 库包括 Qt、GTK、DirectFB 等。这些库提供了创建和管理窗口、控件、事件处理等功能，简化了图形用户界面的开发。

5、显示服务器 (Display Server)

显示服务器（如 X.Org、Wayland）管理窗口系统和图形渲染，处理来自多个应用程序的图形请求，并将它们合成到屏幕上。Wayland 是现代显示服务器协议，比 X.Org 更适合嵌入式系统。

嵌入式 Linux 的显示子系统涉及多个层次的组件，从底层的显示驱动和帧缓冲设备，到高级的图形库和显示服务器。设备树用于描述硬件资源，驱动程序实现硬件的具体控制，图形库和显示服务器则提供应用程序开发接口。通过这些组件的协同工作，可以构建功能强大的嵌入式图形界面。

20.4 MP257 的 Display 说明

20.4.1 文档和资源

想要获取 MP257 系列处理器的 Display 详细说明，最直接的方式是参考 ST 官方文档，包括：

1、参考手册（Reference Manual）：此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南（即本文档）

应用开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图：【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

20.4.2 特性说明

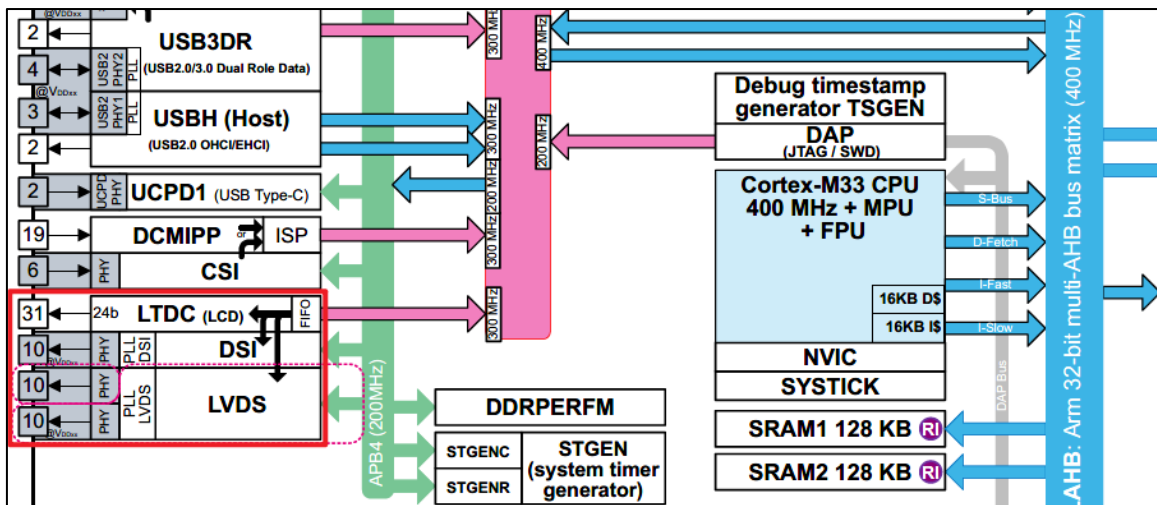


图 20.4-1 MP257 的 Display 部分

MP257 芯片支持 RGBLCD、MIPI DSI 和 LVDS 三种显示接口，在 ATK-CLMP257B 核心板上引出了这几组管脚，用户可以自行设计底板对应的接口。在 ATK-DLMP257B 开发板上设计了这三种接口，方便用户搭配正点原子屏幕进行使用。

根据芯片数据手册可知，MP257 的 **MIPI DSI** 有以下特性：

MIPI DSI（显示串行接口）主控制器是符合 MIPI Alliance 标准 的数字核心，负责实现 MIPI DSI 协议中的所有功能。它通过与 MIPI D-PHY 的接口，支持与兼容 DSI 的显示器进行通信。

主要功能：

协议支持：

- 完全兼容 MIPI Alliance 标准。
- 支持 MIPI DSI 规范中定义的所有 显示命令集（DCS）。
- 数据通道 0 支持双向通信和 逃逸模式。
- 支持 非连续时钟模式，以实现额外的节能效果。
- 支持超低功耗模式（PLL 禁用时可用）。
- 支持 ECC 和校验和。
- 支持 传输结束数据包（EoTp）。
- 提供故障恢复机制。

接口配置：

- 支持 AMBA APB 接口 用于控制，以及通用和 DCS 命令。
- 支持通过 LTDC 的视频模式接口。
- 通过 LTDC 支持自适应命令模式接口。
- 各模式下可独立编程 虚拟通道 ID。

视频模式接口功能：

- LTDC 接口支持颜色编码映射到 16 位、18 位和 24 位接口。
- 可编程所有 LTDC 接口信号的极性。

自适应接口功能：



- 支持通过 `memory_write_start` (WMS) 和 `memory_write_continue` (WMC) 命令发送大量数据。
- LTDC 接口支持颜色编码映射到 16 位、18 位和 24 位接口。

其他功能:

- 内置 视频模式图案生成器。
- 支持最高 4 条数据通道, 每条通道速率最高可达 2.5 Gbps。
- 支持分辨率最高 QXGA (2048 × 1536) @ 60 fps。

根据芯片数据手册可知, MP257 的 LVDS 有以下特性:

LVDS 支持以下高层次功能:**协议支持:**

- 支持 FPD-Link-I 和 OpenLDI (v0.95) 协议。

操作模式:

- 单链路 (Single-link) 或 双链路 (Dual-link) 操作。
- 支持 单显示屏 或 双显示屏 (内容同步输出)。

灵活的位图映射:

- 支持 JEIDA 和 VESA 格式。
- 支持 RGB888 或 RGB666 输出。

数据传输能力:**数据通道:**

- 最多支持 2 个链路, 每个链路 4 条数据通道。
- 每条通道速率最高为 1.1 Gbit/s。

不同协议的比特率:

- FPD-Link: 每通道 784 Mbit/s (每链路 112 Mpixel/s, 双链路 224 Mpixel/s)。
- OpenLDI: 每通道 1100 Mbit/s (每链路 157 Mpixel/s, 双链路 314 Mpixel/s)。

分辨率支持:**双链路:**

最高支持 QXGA (2048 × 1536) @ 60 fps。

单链路:

最高支持 WSXGA+ (1680 × 1050) @ 60 fps。

在 OpenLDI 协议下支持 1080p60。

根据芯片数据手册可知, MP257 的 RGBLCD 有以下特性:

主要功能:

- 支持 显示合成 和 旋转功能。
- 显示层: 提供 3 个显示层, 每层配备独立的 FIFO。
- 输入像素格式: 支持灵活的像素格式, 包括 YUV420 全平面格式。
- 安全层: 具有安全层功能, 保护对缓冲区和配置寄存器的访问。
- 输出旋转: 支持 90 度 和 270 度 旋转。
- 镜像功能: 支持水平和垂直镜像。
- 图像处理: 支持颜色查找表 (LUT)、色键、伽马校正 和 输出抖动处理。

LTDC 并行接口:

- 并行 RGB 输出: 提供 24 位并行数字 RGB, 可直接连接多种 LCD 和 TFT 显示面板。



- LCD_BL (Pin 34): 背光控制信号, 用于打开或关闭 LCD 的背光。

I2C 接口 (触摸屏相关):

- I2C3_SCL (Pin 38): I2C 时钟信号, 用于与触摸屏控制器通信。
- I2C3_SDA (Pin 36): I2C 数据信号。
- RGB_TP_RST (Pin 35): 触摸屏复位信号, 通过 R209 (10k Ω) 上拉到 VCC3.3。
- RGB_TP_INT (Pin 39): 触摸屏中断信号, 通过 R207 (DNP) 和 R210 (NC) 设计为可选接入。

其他设计要点:

- DNP 和 NC: 指 “Do Not Populate” (不焊接) 和 “Not Connected” (未连接), 说明这些元件为备用设计, 不实际使用。
- 电阻 R209 和 R210: R209 (10k Ω) 上拉电阻, 确保复位信号稳定。R210 未连接, 备用设计。

20.6 开发板 MIPI DSI 接口

20.6.1 原理图分析

开发板上有一个 MIPI DSI 接口。打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 DSI。可以看到 MIPI DSI 相关电路如下。

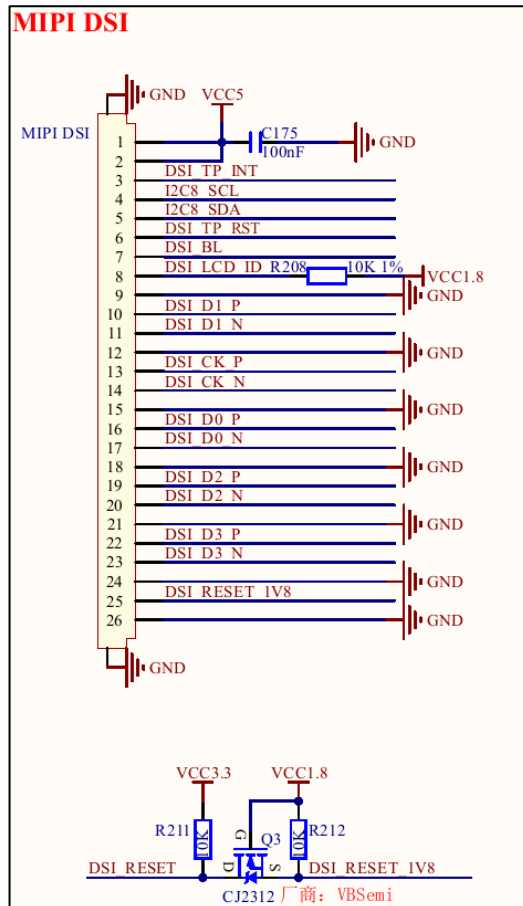


图 20.6-1 MIPI DSI 电路

原理图显示了一个 MIPI-DSI 接口的连接和相关的驱动电路。以下是对该原理图的分析：

MIPI-DSI 接口引脚：

- VCC_5V: 5V 电源输入。
- DSI_TP_INT: MIPI 触摸屏中断引脚。
- I2C8_SCL 和 I2C8_SDA: 触摸芯片 I2C 总线的时钟和数据线。
- DSI_TP_RST: MIPI 触摸屏触摸芯片复位引脚。
- DSI_BL: MIPI 屏幕背光引脚。
- DSI_LCD_ID: 通过一个 10K 的上拉电阻连接到 VCC_1.8，用于支持正点原子多种 MIPI 屏类型识别。
- DSI_D1_N 和 DSI_D1_P: MIPI-DSI 数据通道 1 的负和正差分对。
- DSI_CLK_N 和 DSI_CLK_P: MIPI-DSI 时钟通道的负和正差分对。
- GND: 接地。
- DSI_D0_N 和 DSI_D0_P: MIPI-DSI 数据通道 0 的负和正差分对。
- DSI_D3_N 和 DSI_D3_P: MIPI-DSI 数据通道 3 的负和正差分对。

- DSI_D2_N 和 DSI_D2_P: MIPI-DSI 数据通道 2 的负和正差分对。
- DSI_D0_N 和 DSI_D0_P: 重复的 MIPI-DSI 数据通道 0 的负和正差分对。
- DSI_RESET_1V8: MIPI 触摸屏复位引脚, 引脚电平是 1.8V, 由 DST_RESET 引脚电平转换而来。

DST_RESET 信号电平转换 DSI_RESET_1V8 电路:

- R211 和 R212: 两个上拉电阻, 分别连接到 VCC3.3 (3.3V) 和 VCC1.8 (1.8V) 电源, 用于为电路提供上拉电压。
- Q3: 一个 N 沟道 MOSFET (三极管), 型号为 CJ2312, 用作电平转换器。
- G (Gate, 栅极): 连接到 VCC1.8 (通过 R212)。
- S (Source, 源极): 连接到较低电压信号端 DSI_RESET_1V8。
- D (Drain, 漏极): 连接到较高电压信号端 DSI_RESET。
- VCC3.3 和 VCC1.8: 分别为 3.3V 和 1.8V 的电源电压。
- 信号线: DSI_RESET: 3.3V 电压的复位信号。DSI_RESET_1V8: 1.8V 电压复位信号。

电路原理:

- 该电路用于将 DSI_RESET 信号的电压从 3.3V 转换为 1.8V, 实现不同电压域的兼容。MOSFET Q3 的栅极(G)始终保持 1.8V 电平(由 VCC1.8 和 R212 提供上拉)。
- 当 DSI_RESET 为高电平 (3.3V) 时: 漏极 (D) 和源极 (S) 之间的电压差足够大, MOSFET 处于截止状态。DSI_RESET_1V8 通过 R212 被拉高到 1.8V。
- 当 DSI_RESET 为低电平 (0V) 时: 漏极 (D) 和源极 (S) 之间的电压差减小, MOSFET 导通。DSI_RESET_1V8 被拉低到 0V, 与 DSI_RESET 同步。
- 上拉电阻作用: R211: 当 MOSFET 截止时, 将 DSI_RESET 拉到 3.3V。R212: 当 MOSFET 截止时, 将 DSI_RESET_1V8 拉到 1.8V。
- 电平转换: 通过控制 MOSFET 的导通与截止, DSI_RESET (3.3V) 与 DSI_RESET_1V8 (1.8V) 之间实现了安全的电平转换。

ATK-DLMP257B 开发板 MIPI_DSI 接口已适配正点原子 5.5 寸 720*1280、5.5 寸 1080*1920、10.1 寸 800*1280 MIPI LCD 电容触摸屏, 支持 I2C 接口触摸。图中 DSI_TP_INT 引脚为屏幕触摸芯片的中断引脚, DSI_TP_RST 引脚为屏幕触摸芯片的上电复位使能引脚, DSI_LCD_ID 引脚为 ADC 采集电压引脚来识别屏幕 ID, I2C8_SCL 和 I2C8_SDA 为屏幕触摸芯片的 I2C 通信接口, 用于配置触摸芯片, DSI_BL 引脚为屏幕的背光引脚, 支持 PWM 调光。

由于此接口需要适配正点原子不同分辨率的 MIPI 屏幕, 故本电路使用 ADC_LCD_ID 引脚进行 ADC 电压采集, 通过屏幕自身不同分压电阻配置情况, 来区分不同分辨率的屏幕, 以此实现屏幕 ID 识别功能。(用户可查阅正点原子不同分辨率的 MIPI 屏幕原理图, 以此了解)

需注意此引脚在开发板接口处默认上拉 10K 电阻 (精度 1%) 至 1.8V, 当开发板不接任何屏幕时, 此 ADC 引脚电压采样值默认为 1.8V 左右。此设计会与正点原子 MIPI 屏幕自身的分压电阻形成并联, 进而影响屏幕 ID 设计电平, 故屏幕 ID 电平的实际 ADC 电压采样值以本电路设计为准。用户设计底板时, 建议参考本电路设计。

20.6.2 实物硬件

开发板上的 MIPI DSI 接口位置如下图所示:

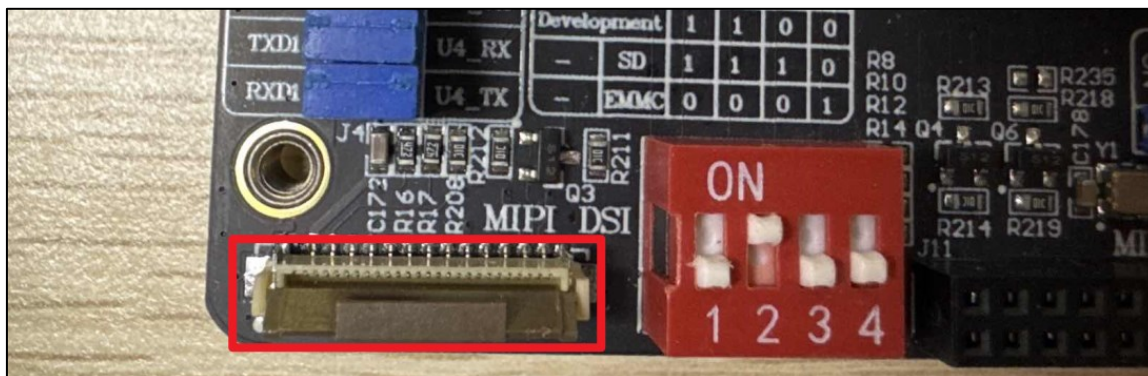


图 20.6-2 MIPI DSI 接口

MIPI DSI 接口兼容正点原子 5.5 寸 MIPI 720x1280 触摸屏、正点原子 5.5 寸 MIPI 1080x1920 触摸屏、正点原子 10.1 寸 MIPI 800x1280 触摸屏。



图 20.6-3 正点原子 5.5 寸 MIPI 720x1280 触摸屏



图 20.6-4 正点原子 5.5 寸 MIPI 1080x1920 触摸屏



图 20.6-5 正点原子 10.1 寸 MIPI 800x1280 触摸屏

20.7 开发板 LVDS 接口

20.7.1 原理图分析

开发板上有两个 LVDS 接口。打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 LVDS。可以看到 LVDS 相关电路如下。

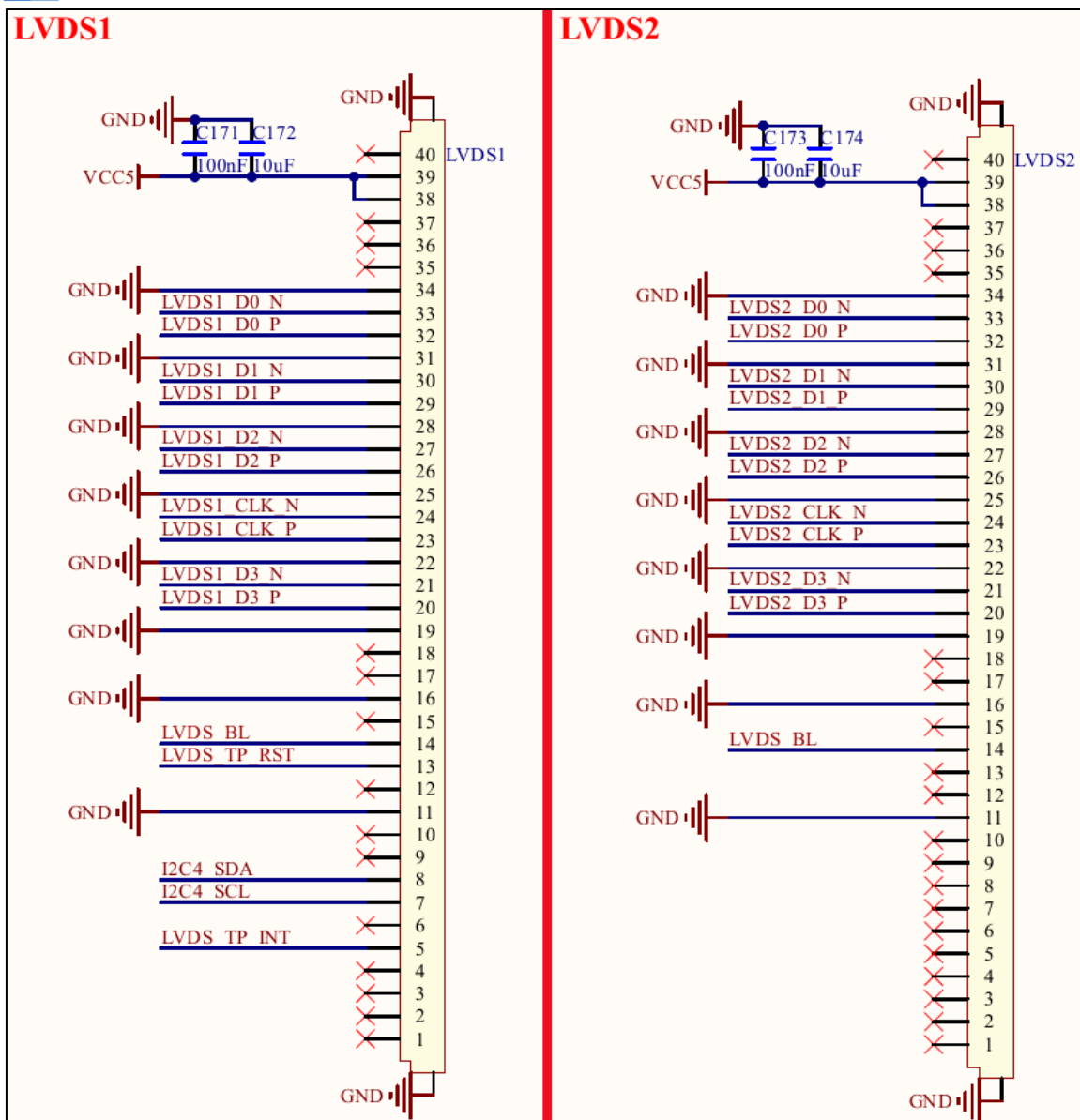


图 20.7-1 开发板 LVDS 电路原理图

ATK-DLMP257B 开发板的 LVDS1 接口已适配一款正点原子 10.1 寸 1280*800 LVDS LCD 电容触摸屏，支持 I2C 接口触摸。图中 LVDS_TP_INT 引脚为屏幕触摸芯片的中断引脚，LVDS_TP_RST 引脚为屏幕触摸芯片的上电复位使能引脚，I2C4_SCL 和 I2C4_SDA 为屏幕触摸芯片的 I2C 通信接口，用于驱动程序配置触摸芯片，LVDS_BL 引脚为屏幕的背光引脚，支持 PWM 调光。

开发板上的 LVDS2 接口没引出触摸相关功能引脚，不能单独使用，和 LVDS1 接口结合使用可以实现 LVDS 双屏同显，单屏使用时请连接 LVDS1 接口。

20.7.2 实物硬件

开发板上的 LVDS 接口位置如下图所示：

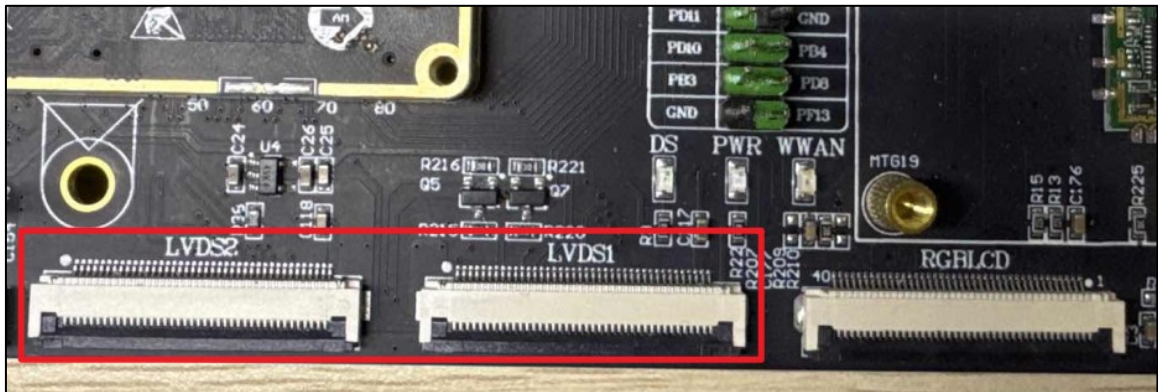


图 20.7-2 LVDS 接口

LVDS 接口兼容正点原子 10.1 寸 LVDS 1280x800 触摸屏。



图 20.7-3 正点原子 10.1 寸 LVDS 1280x800 触摸屏

20.8 内核显示功能使能

20.8.1 RGBLCD 使能

在 ATK-DLMP257B 内核源码配置文件 `arch/arm64/configs/stm32mp257_atk_defconfig` 中，RGBLCD（DSI）相关配置如下(略有删减，只列举重点部分)：

```
1 CONFIG_DRM=y /*启用 Direct Rendering Manager (DRM) 。
    作用：DRM 是 Linux 内核中的一个子系统，用于管理图形显示设备。
    它提供了一个通用的 API，使用户空间程序可以与图形硬件进行交互。*/
2 CONFIG_DRM_STM=y //开启对 STM 平台显示子系统的 DRM 驱动支持。
```

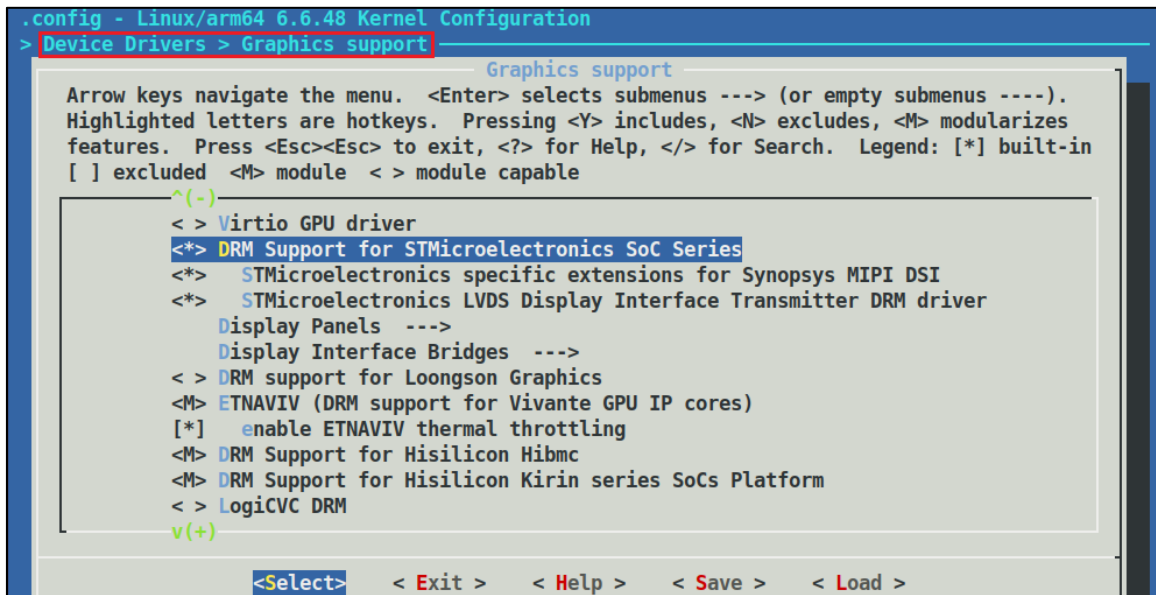


图 20.8-1 DSI 配置节选

20.8.2 MIPI DSI 使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中，DSI 相关配置如下(略有删减，只列举重点部分)：

- 1 CONFIG_DRM=y /*启用 Direct Rendering Manager (DRM) 。
作用：DRM 是 Linux 内核中的一个子系统，用于管理图形显示设备。
它提供了一个通用的 API，使用户空间程序可以与图形硬件进行交互。*/
- 2 CONFIG_DRM_MIPI_DSI=y /*启用 DRM 子系统内的 MIPI DSI 支持。
该选项启用对 MIPI DSI 的支持，这是一个用于在移动设备中连接显示屏的标准接口。*/
- 3 CONFIG_DRM_DW_MIPI_DSI=y //启用 DesignWare MIPI DSI 驱动程序。
- 4 CONFIG_DRM_STM=y //开启对 STM 平台显示子系统的 DRM 驱动支持。
- 5 CONFIG_DRM_STM_DSI=y //启用 STM 平台中对 MIPI DSI 的支持

对应的 menuconfig 配置如下（篇幅有限，截图有所省略）：

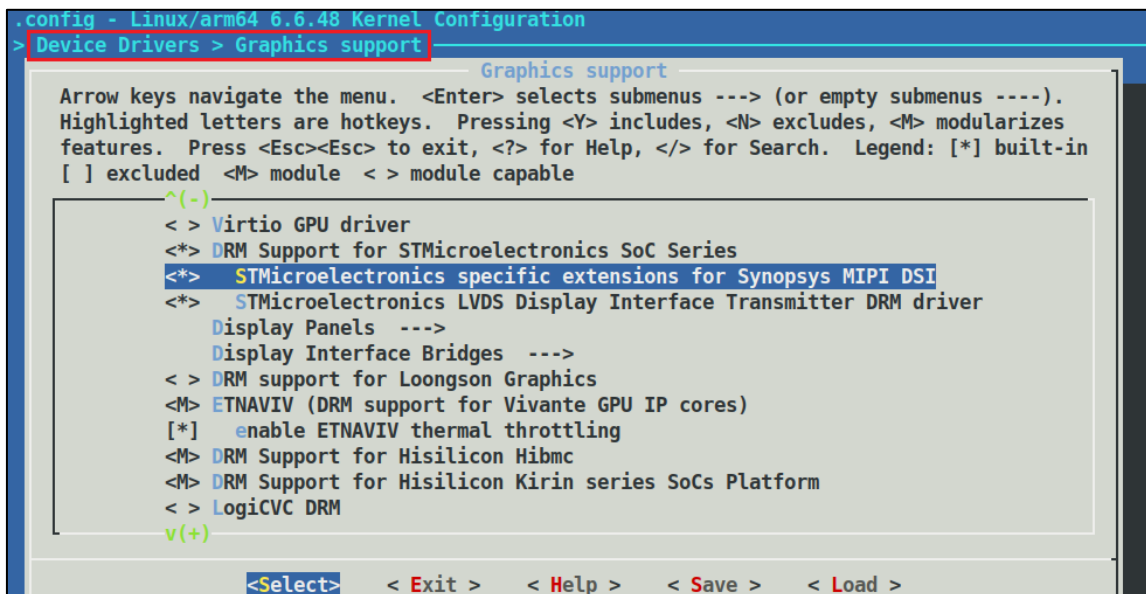


图 20.8-2 MIPI DSI 配置节选

20.8.3 LVDS 使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, LVDS 相关配置如下(略有删减, 只列举重点部分, DRM 和 LCDIF 相关的在前面 DSI 部分已列举):

- 1 CONFIG_DRM_PANEL_LVDS=y //启用将 LVDS 面板驱动程序
- 2 CONFIG_DRM_STM_LVDS=y //启用 STM 平台中对 LVDS 的支持。

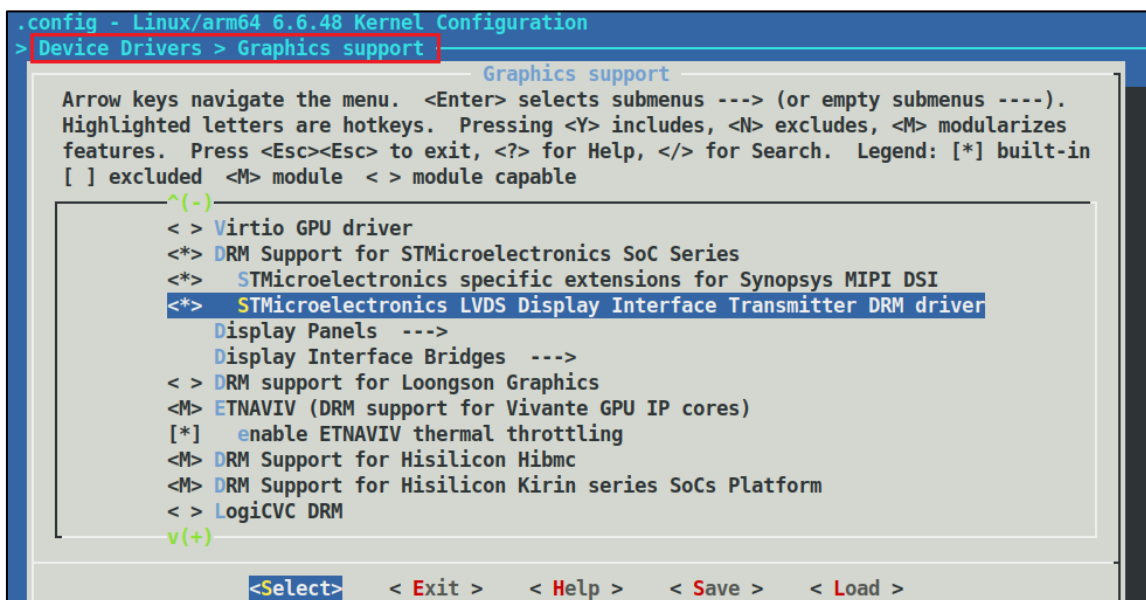


图 20.8-3 LVDS 配置

20.9 设备树分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi, 此文件是特定于 STM32MP2 系列芯片的设备树包含文件, 通常不独立使用, 而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts

文件共享相同的基础配置,而.dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 display 的描述如下:

```

1  ltcd: display-controller@48010000 {
2      compatible = "st,stm32mp25-ltcd";
3      reg = <0x48010000 0x400>;
4      st,syscon = <&syscfg>;
5      interrupts = <GIC_SPI 158 IRQ_TYPE_LEVEL_HIGH>,
6                  <GIC_SPI 159 IRQ_TYPE_LEVEL_HIGH>;
7      clocks = <&rcc CK_BUS_LTDC>, <&rcc CK_KER_LTDC>;
8      clock-names = "bus", "lcd";
9      resets = <&rcc LTDC_R>;
10     power-domains = <&CLUSTER_PD>;
11     status = "disabled";
12     access-controllers = <&rifsc 80>;
13     access-controller-names = "cmn";
14
15     l112 {
16         access-controllers = <&rifsc 119>;
17         access-controller-names = "l112";
18     };
19     l3 {
20         access-controllers = <&rifsc 120>;
21         access-controller-names = "l3";
22     };
23     rot {
24         access-controllers = <&rifsc 121>;
25         access-controller-names = "rot";
26     };
27 };

```

第 1 行, `ltcd: display-controller@48010000` 定义名为 `ltcd` 的显示控制器设备节点,其地址为 `0x48010000`。

第 2 行, `compatible = "st,stm32mp25-ltcd"` 声明该设备与 `st,stm32mp25-ltcd` 驱动兼容。

第 3 行, `reg = <0x48010000 0x400>` 该设备的寄存器地址范围为起始地址 `0x48010000`,大小为 `0x400` 字节。

第 4 行, `st,syscon = <&syscfg>` 引用系统配置节点 `syscfg`。

第 5~6 行, `interrupts = <GIC_SPI 158 IRQ_TYPE_LEVEL_HIGH>` 定义设备的中断号为 `158` 和 `159`,类型为电平触发高电平。

第 7~8 行, `clocks = <&rcc CK_BUS_LTDC>, <&rcc CK_KER_LTDC>` 定义所需的时钟信号,分别为总线时钟 `CK_BUS_LTDC` 和 LCD 时钟 `CK_KER_LTDC`。

第 9 行, `clock-names = "bus", "lcd"` 对应的时钟名称分别为 `"bus"` 和 `"lcd"`。

第 10 行, `resets = <&rcc LTDC_R>` 定义设备复位控制器的复位线为 `LTDC_R`。

第 11 行, `power-domains = <&CLUSTER_PD>` 设备属于 `CLUSTER_PD` 电源域。

第 12 行, `status = "disabled"` 当前设备状态为禁用。



第 13 行, `access-controllers = <&rifsc 80>` 定义访问控制器, 引用 rifsc 中的 ID 为 80 的控制器。

第 14 行, `access-controller-names = "cmn"` 访问控制器名称为 "cmn"。

第 15 行, `1112` 定义子节点 1112, 用于描述 Layer 1 和 Layer 2 的访问控制。

第 16 行, `access-controllers = <&rifsc 119>` Layer 1 和 Layer 2 的访问控制器引用 rifsc 中 ID 为 119 的控制器。

第 17 行, `access-controller-names = "1112"` Layer 1 和 Layer 2 的访问控制器名称为 "1112"。

第 20 行, `access-controllers = <&rifsc 120>` Layer 3 的访问控制器引用 rifsc 中 ID 为 120 的控制器。

第 21 行, `access-controller-names = "l3"` Layer 3 的访问控制器名称为 "l3"。

第 23 行, `rot` 定义子节点 rot, 用于描述旋转功能的访问控制。

第 24 行, `access-controllers = <&rifsc 121>` 旋转功能的访问控制器引用 rifsc 中 ID 为 121 的控制器。

第 25 行, `access-controller-names = "rot"` 旋转功能的访问控制器名称为 "rot"

20.9.1 RGB 设备树

正点原子 ATK-DLMP257B 开发板 RGBLCD 接口支持正点原子的 4.3 寸 RGB 800x480 屏幕、7 寸 RGB 800x480 屏幕、和 7 寸 RGB 1024x600 屏幕。

以 2GB 版本核心板为例, 设备树为 `arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB-rgb.dts`。

```

1  #include "stm32mp257d-atk-ddr-2GB.dts"
2  / {
3  /*
4      rgb_lcd_id select id:
5          atk_rgb_lcd_7_800x480 = 1,
6          atk_rgb_lcd_7_1024x600 = 2,
7          atk_rgb_lcd_4.3_800x480 = 4,
8          atk_rgb_lcd_10.1_1280x800 = 5,
9  */
10     rgb_lcd_id {
11         rgb_select_id = <2>;    //default, atk_rgb_lcd_7_1024x600
12     };
13
14     panel_rgb: panel-rgb {
15         compatible = "panel-dpi";
16         pinctrl-names = "default", "sleep";
17         pinctrl-0 = <&lt;tdc_pins_a>;
18         pinctrl-1 = <&lt;tdc_sleep_pins_a>;
19         backlight = <&panel_rgb_backlight>;
20         power-supply = <&vdd_5v>;
21         status = "okay";
22
23         data-mapping = "rgb565";
24

```



```
25 //atk_rgb_lcd_7_800x480
26 panel-timing-1 {
27     clock-frequency = <33300000>;
28     hactive = <800>;
29     vactive = <480>;
30     hback-porch = <46>;
31     hfront-porch = <210>;
32     hsync-len = <1>;
33     vback-porch = <23>;
34     vfront-porch = <22>;
35     vsync-len = <1>;
36 };
37
38 //atk_rgb_lcd_7_1024x600
39 panel-timing-2 {
40     clock-frequency = <51200000>;
41     hactive = <1024>;
42     vactive = <600>;
43     hback-porch = <140>;
44     hfront-porch = <160>;
45     hsync-len = <20>;
46     vback-porch = <20>;
47     vfront-porch = <12>;
48     vsync-len = <3>;
49 };
50
51 //atk_rgb_lcd_4.3_800x480
52 panel-timing-4 {
53     clock-frequency = <31000000>;
54     hactive = <800>;
55     vactive = <480>;
56     hback-porch = <88>;
57     hfront-porch = <40>;
58     hsync-len = <48>;
59     vback-porch = <32>;
60     vfront-porch = <13>;
61     vsync-len = <3>;
62 };
63
64 //atk_rgb_lcd_10.1_1280x800
65 panel-timing-5 {
66     clock-frequency = <71100000>;
67     hactive = <1280>;
```



```

68         vactive = <800>;
69         hback-porch = <80>;
70         hfront-porch = <70>;
71         hsync-len = <10>;
72         vback-porch = <10>;
73         vfront-porch = <10>;
74         vsync-len = <3>;
75     };
76
77     port {
78         rgb_panel_in: endpoint {
79             remote-endpoint = <&lt;dc_ep0_out>>;
80         };
81     };
82 };
83
84 panel_rgb_backlight: panel-rgb-backlight {
85     compatible = "pwm-backlight";
86     pwms = <&pwm1 3 5000000 0>;
87     brightness-levels = <
88         0   1   2   3   4   5   6   7   8   9
89         10  11  12  13  14  15  16  17  18  19
90         20  21  22  23  24  25  26  27  28  29
91         30  31  32  33  34  35  36  37  38  39
92         40  41  42  43  44  45  46  47  48  49
93         50  51  52  53  54  55  56  57  58  59
94         60  61  62  63  64  65  66  67  68  69
95         70  71  72  73  74  75  76  77  78  79
96         80  81  82  83  84  85  86  87  88  89
97         90  91  92  93  94  95  96  97  98  99
98         100 101 102 103 104 105 106 107 108 109
99         110 111 112 113 114 115 116 117 118 119
100        120 121 122 123 124 125 126 127 128 129
101        130 131 132 133 134 135 136 137 138 139
102        140 141 142 143 144 145 146 147 148 149
103        150 151 152 153 154 155 156 157 158 159
104        160 161 162 163 164 165 166 167 168 169
105        170 171 172 173 174 175 176 177 178 179
106        180 181 182 183 184 185 186 187 188 189
107        190 191 192 193 194 195 196 197 198 199
108        200 201 202 203 204 205 206 207 208 209
109        210 211 212 213 214 215 216 217 218 219
110        220 221 222 223 224 225 226 227 228 229

```



```
111          230 231 232 233 234 235 236 237 238 239
112          240 241 242 243 244 245 246 247 248 249
113          250 251 252 253 254 255>;
114      power-supply = <&vdd_3v3>;
115      default-brightness-level = <255>;
116      status = "okay";
117  };
118 };
119
120 &timers1 {
121     status = "okay";
122     /* spare dmas for other usage */
123     /delete-property/dmas;
124     /delete-property/dma-names;
125
126     pwm1: pwm {
127         pinctrl-0 = <&pwm1_pins_a>;
128         pinctrl-1 = <&pwm1_sleep_pins_a>;
129         pinctrl-names = "default", "sleep";
130         #pwm-cells = <2>;
131         status = "okay";
132     };
133 };
134
135 &i2c3 {
136     goodix_ts@14 {
137         compatible = "goodix,gt9271", "goodix,gt911";
138         reg = <0x14>;
139         pinctrl-names = "default";
140         interrupt-parent = <&gpiof>;
141         interrupts = <11 IRQ_TYPE_EDGE_RISING>;
142         irq-gpios = <&gpiof 11 GPIO_ACTIVE_HIGH>;
143         reset-gpios = <&gpiof 13 GPIO_ACTIVE_HIGH>;
144         status = "okay";
145     };
146 };
147
148 &ltcdc {
149     rotation-memory = <&ltcdc_sec_rotation>;
150     status = "okay";
151
152     port {
153         #address-cells = <1>;
```



```

154     #size-cells = <0>;
155
156     ltdc_ep0_out: endpoint@0 {
157         reg = <0>;
158         remote-endpoint = <&rgb_panel_in>;
159     };
160 };
161 };

```

第 1 行, 引入主板相关的设备树文件 stm32mp257d-atk-ddr-2GB.dts (DDR 2GB 配置)。

第 3~8 行, 定义了 rgb_lcd_id 对应的 LCD RGB 屏幕的型号:

- atk_rgb_lcd_7_800x480 = 1
- atk_rgb_lcd_7_1024x600 = 2
- atk_rgb_lcd_4.3_800x480 = 4
- atk_rgb_lcd_10.1_1280x800 = 5

第 10~12 行, rgb_select_id = <2> 默认选择 7 英寸 1024x600 的 LCD 屏幕。

第 13~75 行, LCD RGB 面板配置, panel_rgb 节点

- compatible = "alientek,panel-dpi" 兼容 正点原子 的 LCD RGB 屏幕。
- pinctrl-names = "default", "sleep" 配置两种引脚状态: 默认和休眠模式。
- pinctrl-0 = <<dc_pins_a> 默认状态引脚配置。
- pinctrl-1 = <<dc_sleep_pins_a> 休眠模式引脚配置。
- backlight = <&panel_rgb_backlight> 关联背光配置。
- status = "okay" 启用节点。
- data-mapping = "rgb565", 数据映射为 RGB565 格式。
- panel-timing-1 到 panel-timing-5 分别是对应的 LCD 屏幕时序参数。

第 77~85 行, port 和 endpoint 节点, remote-endpoint = <<dc_ep0_out> 配置与 LTDC 的端点连接。

第 84~116 行, panel_rgb_backlight 节点

- compatible = "pwm-backlight" 背光使用 PWM 驱动。
- pwms = <&pwm1 3 5000000 0> 使用定时器 1 的通道 3, 周期为 5ms。
- brightness-levels 配置亮度等级从 0 到 255。
- power-supply = <&vdd_3v3> 背光电源为 3.3V。
- default-brightness-level = <255> 默认亮度为 255 (最大)。
- status = "okay" 启用节点。

第 120~133 行, 定时器 1 和 PWM 配置

timers1 节点: status = "okay" 启用定时器 1。删除 dmas 和 dma-names 属性, 保留 DMA 通道供其他用途。

pwm1 节点:

- pinctrl-0 = <&pwm1_pins_a> 默认状态下的 PWM 引脚配置。
- pinctrl-1 = <&pwm1_sleep_pins_a> 休眠状态下的 PWM 引脚配置。
- #pwm-cells = <2> PWM 配置单元: 周期和占空比。
- status = "okay" 启用 PWM 节点。

第 135~145 行。I2C3 触摸屏配置, goodix_ts 节点



- compatible = "goodix,gt9271", "goodix,gt911" 兼容 Goodix 的 GT9271 和 GT911 触摸屏。
- reg = <0x14> I2C 地址为 0x14。
- interrupt-parent = <&gpiof> 中断来源为 GPIOF。
- interrupts = <11 IRQ_TYPE_EDGE_RISING> GPIOF 的 11 号引脚触发上升沿中断。
- irq-gpios = <&gpiof 11 GPIO_ACTIVE_HIGH> 定义中断 GPIO 配置。
- reset-gpios = <&gpiof 13 GPIO_ACTIVE_HIGH> 定义复位 GPIO 配置。
- status = "okay" 启用触摸屏。

第 148~151 行, LTDC 显示控制器配置

ltdc 节点:

- rotation-memory = <<dc_sec_rotation> 使用指定的旋转内存。
- status = "okay" 启用 LTDC。

port 和 endpoint 节点:

- 定义 LTDC 的端口和端点配置。
- ltdc_ep0_out: endpoint@0 { ... } 配置端点 0, 连接到 rgb_panel_in。

20.9.2 MIPI 屏幕设备树

正点原子 ATK-DLMP257B 开发板 MIPI DSI 接口支持正点原子的 5.5 寸 MIPI 720x1280 屏幕、5.5 寸 MIPI 1080x1920 屏幕、和 10.1 寸 MIPI 800x1280 屏幕。

以 2GB 版本核心板为例, 设备树为 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB-mipi.dts。

```

1  #include "stm32mp257d-atk-ddr-2GB.dts"
2  / {
3      /*
4          dsi_lcd_id select id:
5              atk_no_mipi = 1,
6              atk_mipi_dsi_5x5_720x1280 = 2,
7              atk_mipi_dsi_5x5_1080x1920 = 3,
8              atk_mipi_dsi_10x1_800x1280 = 4,
9      */
10     dsi_lcd_id {
11         dsi_select_id = <2>;
12     };
13
14     panel_dsi_backlight: panel-dsi-backlight {
15         compatible = "pwm-backlight";
16         pwms = <&pwm16 0 5000000 0>;
17         brightness-levels = <
18             0   1   2   3   4   5   6   7   8   9
19             10  11  12  13  14  15  16  17  18  19
20             20  21  22  23  24  25  26  27  28  29
21             30  31  32  33  34  35  36  37  38  39
22             40  41  42  43  44  45  46  47  48  49
23             50  51  52  53  54  55  56  57  58  59

```



```

24      60 61 62 63 64 65 66 67 68 69
25      70 71 72 73 74 75 76 77 78 79
26      80 81 82 83 84 85 86 87 88 89
27      90 91 92 93 94 95 96 97 98 99
28      100 101 102 103 104 105 106 107 108 109
29      110 111 112 113 114 115 116 117 118 119
30      120 121 122 123 124 125 126 127 128 129
31      130 131 132 133 134 135 136 137 138 139
32      140 141 142 143 144 145 146 147 148 149
33      150 151 152 153 154 155 156 157 158 159
34      160 161 162 163 164 165 166 167 168 169
35      170 171 172 173 174 175 176 177 178 179
36      180 181 182 183 184 185 186 187 188 189
37      190 191 192 193 194 195 196 197 198 199
38      200 201 202 203 204 205 206 207 208 209
39      210 211 212 213 214 215 216 217 218 219
40      220 221 222 223 224 225 226 227 228 229
41      230 231 232 233 234 235 236 237 238 239
42      240 241 242 243 244 245 246 247 248 249
43      250 251 252 253 254 255>;
44      power-supply = <&vdd_3v3>;
45      default-brightness-level = <255>;
46      status = "okay";
47  };
48 };
49
50 &timers16 {
51     status = "okay";
52     /* spare dmas for other usage */
53     /delete-property/dmas;
54     /delete-property/dma-names;
55
56     pwm16: pwm {
57         pinctrl-0 = <&pwm16_pins_a>;
58         pinctrl-1 = <&pwm16_sleep_pins_a>;
59         pinctrl-names = "default", "sleep";
60         #pwm-cells = <2>;
61         status = "okay";
62     };
63 };
64
65 &i2c8 {
66     goodix_ts@14 {

```



```
67     compatible = "goodix,gt9271", "goodix,gt911";
68     reg = <0x14>;
69     pinctrl-names = "default";
70     interrupt-parent = <&gpio0>;
71     interrupts = <2 IRQ_TYPE_EDGE_RISING>;
72     irq-gpios = <&gpio0 2 GPIO_ACTIVE_HIGH>;
73     reset-gpios = <&gpio0 1 GPIO_ACTIVE_HIGH>;
74     status = "okay";
75 };
76 };
77
78 &lt;ldc {
79     rotation-memory = <&ldc_sec_rotation>;
80     status = "okay";
81
82     port {
83         #address-cells = <1>;
84         #size-cells = <0>;
85
86         ldc_ep0_out: endpoint@0 {
87             reg = <0>;
88             remote-endpoint = <&dsim_in>;
89         };
90     };
91 };
92
93 &dsim {
94     vdd-supply = <&scmi_vddcore>;
95     vdda18-supply = <&scmi_v1v8>;
96     status = "okay";
97     #address-cells = <1>;
98     #size-cells = <0>;
99
100     ports {
101         #address-cells = <1>;
102         #size-cells = <0>;
103
104         port@0 {
105             reg = <0>;
106             dsim_in: endpoint {
107                 };
108         };
109     };
110 }
```



```

110     port@1 {
111         reg = <1>;
112         dsi_out: endpoint {
113             };
114     };
115 };
116
117 panel_dsi: panel@0 {
118     compatible = "alientek,mipi-lcd";
119     reg = <0>;
120     dsi-lanes = <4>;
121     reset-gpio = <&gpioi 11 GPIO_ACTIVE_LOW>;
122     power-supply = <&vdd_5v>;
123     backlight = <&panel_dsi_backlight>;
124     status = "okay";
125
126     port {
127         dsi_panel_in: endpoint {
128             remote-endpoint = <&dsi_out>;
129         };
130     };
131 };
132 };
133
134 &dsi_in {
135     remote-endpoint = <&lt;dc_ep0_out>;
136 };
137
138 &dsi_out {
139     remote-endpoint = <&dsi_panel_in>;
140 };

```

第 1 行, 引入外部文件 `stm32mp257d-atk-ddr-2GB.dts`, 为设备树提供基础配置。

第 2 行, 根节点部分, 定义了全局的设备属性。

第 3~9 行, 定义 LCD 的选择 ID:

- `atk_no_mipi = 1`: 不使用 MIPI 屏幕;
- `atk_mipi_dsi_5x5_720x1280 = 2`: 使用分辨率 720x1280 的 5.5 寸 MIPI 屏幕;
- `atk_mipi_dsi_5x5_1080x1920 = 3`: 使用分辨率 1080x1920 的 5.5 寸 MIPI 屏幕;
- `atk_mipi_dsi_10x1_800x1280 = 4`: 使用分辨率 800x1280 的 10.1 寸 MIPI 屏幕。

第 11 行, `dsi_select_id = <2>` 设置默认的画面为 ID 为 2 的画面 (720x1280 分辨率)。

第 14~46 行, `panel_dsi_backlight` 定义 DSI 面板的背光节点。

第 15 行, `compatible = "pwm-backlight"` 该节点兼容 `pwm-backlight` 驱动程序, 用于控制背光的 PWM 驱动。

第 16 行, `pwms=<&pwm16 0 5000000 0>` 配置背光使用 `pwm16`, 周期为 5000000 纳秒, 极性为 0 (默认)。

第 17~43 行, `brightness-levels = <0 ... 255>` 定义了 0-255 的背光亮度级别。

第 44 行, `power-supply = <&vdd_3v3>` 背光供电电源为 `vdd_3v3`。

第 45 行, `default-brightness-level = <255>` 默认亮度级别设置为最大值 255。

第 46 行, `status = "okay"` 表示该背光节点启用。

第 50 行, `&timers16` 定义定时器节点 `timers16` 的配置。

第 51 行, `status = "okay"` 启用该定时器节点。

第 53~54 行, `/delete-property/dmas; /delete-property/dma-names` 删除 `dmas` 和 `dma-names` 属性, 释放 DMA 通道供其他用途。

第 56~61 行, `pwm16: pwm` 定义 PWM 子节点, 用于控制背光的 PWM 信号:

- `pinctrl-0 = <&pwm16_pins_a>`; 设置默认引脚状态
- `pinctrl-1 = <&pwm16_sleep_pins_a>`; 设置休眠时的引脚状态
- `pinctrl-names = "default", "sleep"`; 定义两种状态的名称

第 65 行, `&i2c8` 定义触摸屏控制器 `goodix_ts` 的配置

第 66 行, `goodix_ts@14` 定义触摸屏设备地址为 0x14。

第 67 行, `compatible = "goodix,gt9271", "goodix,gt911"` 兼容 Goodix 的两款触摸屏芯片 (gt9271 和 gt911)。

第 68 行, `reg = <0x14>` 设置 I2C 从设备地址为 0x14。

第 70~72 行, `interrupt-parent` 和 `interrupts` 设置中断引脚为 `GPIOB2`, 触发类型为上升沿。

第 73 行, `reset-gpios = <&gpio1 1 GPIO_ACTIVE_HIGH>` 定义复位引脚为 `GPIOB1`, 高电平有效。

第 74 行, `status = "okay"` 启用触摸屏设备。

第 78 行, `<ldc` 定义 LCD-TFT 控制器 `ldc` 的配置。

第 79 行, `rotation-memory = <&ldc_sec_rotation>` 指定旋转内存区域。

第 80 行, `status = "okay"` 启用该节点。

第 82 行, `port` 定义输出端口配置。

第 86~88 行, `ldc_ep0_out` 定义一个输出端点, 连接到 `&dsi_in`。

第 93 行, `&dsi` 定义 MIPI DSI 控制器的配置。

第 94~95 行, `vdd-supply` 和 `vdda18-supply` 配置电源为 `scmi_vddcore` 和 `scmi_v1v8`。

第 96 行, `status = "okay"` 启用该节点。

第 100~114 行, 定义两个端口:

- `port@0` 输入端点 `dsi_in`
- `port@1` 输出端点 `dsi_out`

第 117 行, `panel_dsi` 定义面板节点。

第 118 行, `compatible = "alientek,mipi-lcd"`, 兼容正点原子的 MIPI LCD 屏幕驱动。

第 120 行, `dsi-lanes = <4>` 使用 4 条数据通道。

第 121 行, `reset-gpio = <&gpio1 11 GPIO_ACTIVE_LOW>` 定义复位引脚为 `GPIOI11`, 低电平有效。

第 122 行, `power-supply = <&vdd_5v>` 使用 5V 电源供电。

第 123 行, `backlight = <&panel_dsi_backlight>` 背光控制连接到前面定义的 `panel_dsi_backlight`

第 126~130 行, `port` 定义输入端点 `dsi_panel_in`, 连接到 `&dsi_out`。

第 134~140 行, &dsi_in 和 &dsi_out 将 DSI 的输入端点连接到 ltdc_ep0_out, 输出端点连接到 dsi_panel_in。

这段设备树配置了 STM32MP257 平台上用于显示和触摸的 MIPI DSI 显示屏和触摸屏控制器, 包括 LCD-TFT 控制器 (LTDC)、DSI 控制器、背光控制、触摸屏中断、以及输入输出端点的映射。

20.9.3 LVDS 设备树

我们再看下 LVDS 的配置, 以 stm32mp257d-atk-ddr-2GB-lvds-1xSingleLink.dts 为例:

```

1  #include "stm32mp257d-atk-ddr-2GB.dts"
2  / {
3      panel_lvds: panel-lvds {
4          compatible = "panel-lvds";
5          backlight = <&panel_lvds_backlight>;
6          status = "okay";
7
8          width-mm = <217>;
9          height-mm = <136>;
10         data-mapping = "vesa-24";
11
12         panel-timing {
13             clock-frequency = <72000000>;
14             hactive = <1280>;
15             vactive = <800>;
16             hfront-porch = <72>;
17             hback-porch = <88>;
18             hsync-len = <3>;
19             vfront-porch = <15>;
20             vback-porch = <23>;
21             vsync-len = <3>;
22         };
23
24         ports {
25             #address-cells = <1>;
26             #size-cells = <0>;
27
28             port@0 {
29                 reg = <0>;
30                 lvds_panel_in0: endpoint {
31                     remote-endpoint = <&lvds_out0>;
32                 };
33             };
34
35             port@1 {

```



```

36         reg = <1>;
37         lvds_panel_in1: endpoint {
38             remote-endpoint = <&lvds_out1>;
39         };
40     };
41 };
42 };
43
44 panel_lvds_backlight: panel-lvds-backlight {
45     compatible = "pwm-backlight";
46     pwms = <&pwm20 2 5000000 0>;
47     brightness-levels = <
48         0   1   2   3   4   5   6   7   8   9
49         10  11  12  13  14  15  16  17  18  19
50         20  21  22  23  24  25  26  27  28  29
51         30  31  32  33  34  35  36  37  38  39
52         40  41  42  43  44  45  46  47  48  49
53         50  51  52  53  54  55  56  57  58  59
54         60  61  62  63  64  65  66  67  68  69
55         70  71  72  73  74  75  76  77  78  79
56         80  81  82  83  84  85  86  87  88  89
57         90  91  92  93  94  95  96  97  98  99
58         100 101 102 103 104 105 106 107 108 109
59         110 111 112 113 114 115 116 117 118 119
60         120 121 122 123 124 125 126 127 128 129
61         130 131 132 133 134 135 136 137 138 139
62         140 141 142 143 144 145 146 147 148 149
63         150 151 152 153 154 155 156 157 158 159
64         160 161 162 163 164 165 166 167 168 169
65         170 171 172 173 174 175 176 177 178 179
66         180 181 182 183 184 185 186 187 188 189
67         190 191 192 193 194 195 196 197 198 199
68         200 201 202 203 204 205 206 207 208 209
69         210 211 212 213 214 215 216 217 218 219
70         220 221 222 223 224 225 226 227 228 229
71         230 231 232 233 234 235 236 237 238 239
72         240 241 242 243 244 245 246 247 248 249
73         250 251 252 253 254 255>;
74     power-supply = <&vdd_3v3>;
75     default-brightness-level = <255>;
76     status = "okay";
77 };
78 };

```




```
79
80 &timers20 {
81     status = "okay";
82     /* spare dmas for other usage */
83     /delete-property/dmas;
84     /delete-property/dma-names;
85
86     pwm20: pwm {
87         pinctrl-0 = <&pwm20_pins_a>;
88         pinctrl-1 = <&pwm20_sleep_pins_a>;
89         pinctrl-names = "default", "sleep";
90         #pwm-cells = <2>;
91         status = "okay";
92     };
93 };
94
95 &i2c4 {
96     goodix_ts@14 {
97         compatible = "goodix,gt9271", "goodix,gt911";
98         reg = <0x14>;
99         pinctrl-names = "default";
100        interrupt-parent = <&gpio_b>;
101        interrupts = <5 IRQ_TYPE_EDGE_RISING>;
102        irq-gpios = <&gpio_b 5 GPIO_ACTIVE_HIGH>;
103        reset-gpios = <&gpio_b 4 GPIO_ACTIVE_HIGH>;
104        status = "okay";
105    };
106 };
107
108 &lt;ldc {
109     default-on;
110     rotation-memory = <&ldc_sec_rotation>;
111     status = "okay";
112
113     port {
114         #address-cells = <1>;
115         #size-cells = <0>;
116
117         ldc_ep0_out: endpoint@0 {
118             reg = <0>;
119             remote-endpoint = <&lvds_in>;
120         };
121     };
```



```

122 };
123
124 &lvds {
125     default-on;
126     vdd-supply = <&scmi_vddcore>;
127     vdda18-supply = <&scmi_v1v8>;
128     status = "okay";
129
130     ports {
131         #address-cells = <1>;
132         #size-cells = <0>;
133
134         port@0 {
135             reg = <0>;
136             lvds_in: endpoint {
137                 remote-endpoint = <&ltltdc_ep0_out>;
138             };
139         };
140
141         port@1 {
142             reg = <1>;
143             lvds_out0: endpoint {
144                 remote-endpoint = <&lvds_panel_in0>;
145             };
146         };
147
148         port@2 {
149             reg = <2>;
150             lvds_out1: endpoint {
151                 remote-endpoint = <&lvds_panel_in1>;
152             };
153         };
154     };
155 };

```

第 1 行, 引用基础的 stm32mp257d-atk-ddr-2GB.dts 的设备树定义。

第 3 行, panel_lvds: panel-lvds 定义 LVDS 面板的节点, 标签为 panel_lvds。

第 4 行, compatible = "panel-lvds" 表明该节点与 "panel-lvds" 设备兼容。

第 5 行, backlight=<&panel_lvds_backlight> 指定该面板使用 panel_lvds_backlight 作为背光控制。

第 6 行, status = "okay" 启用该节点。

第 7~8 行, width-mm = <217>; height-mm = <136> 面板的物理宽度和高度, 单位为毫米。

第 9 行, data-mapping = "vesa-24" 定义数据映射模式为 VESA-24。

第 10~20 行, panel-timing 定义面板的显示时序参数:



- clock-frequency = <72000000>; 面板的时钟频率为 72 MHz。
- hactive = <1280>; 水平有效像素数为 1280。
- vactive = <800>; 垂直有效像素数为 800。
- hfront-porch = <72>; 水平前沿时序为 72。
- hback-porch = <88>; 水平后沿时序为 88。
- hsync-len = <3>; 水平同步信号长度为 3。
- vfront-porch = <15>; 垂直前沿时序为 15。
- vback-porch = <23>; 垂直后沿时序为 23。
- vsync-len = <3>; 垂直同步信号长度为 3。

第 21 行, ports 定义端口的开始。

第 22~23 行, #address-cells = <1>; #size-cells = <0> 配置地址单元数为 1, 大小单元数为 0。

第 24~28 行, port@0 定义端口 0。

- reg = <0> 端口编号为 0。
- lvds_panel_in0: endpoint { remote-endpoint = <&lvds_out0>; } 连接到 lvds_out0

第 29~33 行, 定义端口 1 (port@1)

- reg = <1>; 端口编号为 1。
- lvds_panel_in1: endpoint { remote-endpoint = <&lvds_out1>; } 连接到 lvds_out1。

第 35~66 行, 背光节点 panel_lvds_backlight 的配置

- panel_lvds_backlight: panel-lvds-backlight 定义 LVDS 面板的背光控制节点。
- compatible = "pwm-backlight" 声明与 "pwm-backlight" 设备兼容。
- pwms = <&pwm20 2 5000000 0> 使用定时器 20 的通道 2 生成 PWM 信号。PWM 周期为 5ms。
- brightness-levels = 0 ... 255 定义背光亮度等级从 0 到 255。每行表示一组亮度值。
- power-supply = <&vdd_3v3> 定义背光的供电电源为 vdd_3v3。
- default-brightness-level = <255> 默认亮度为 255 (最大值)。
- status = "okay" 启用该节点。

第 70~84 行, PWM 定时器 20 的配置。

- &timers20 定义定时器 20 的节点。
- status = "okay" 启用定时器 20。
- /delete-property/dmas; /delete-property/dma-names 删除 dmas 和 dma-names 属性, 释放 DMA 通道以供其他用途。
- pwm20: pwm 定义 PWM 子节点, 标签为 pwm20。
- pinctrl-0 = <&pwm20_pins_a> 指定 PWM 的默认引脚控制组。
- pinctrl-1 = <&pwm20_sleep_pins_a> 指定 PWM 的低功耗模式引脚控制组。
- pinctrl-names = "default", "sleep" 定义引脚控制组名称。
- #pwm-cells = <2> 声明 PWM 参数格式 (周期和占空比)。
- status = "okay" 启用 PWM。

第 85~94 行, I2C4 节点的触摸屏配置

- &i2c4 定义 I2C4 节点。
- goodix_ts@14 定义 Goodix 触摸屏的子节点, I²C 地址为 0x14。
- compatible = "goodix,gt9271", "goodix,gt911" 声明与 Goodix 的 GT9271 和 GT911 触摸屏兼容。
- reg = <0x14> I2C 地址为 0x14。



- `pinctrl-names = "default"` 使用默认引脚配置。
- `interrupt-parent = <&gpio>` 中断信号来源于 GPIOB。
- `interrupts = <5 IRQ_TYPE_EDGE_RISING>` GPIOB 的第 5 引脚触发上升沿中断。
- `irq-gpios = <&gpio 5 GPIO_ACTIVE_HIGH>` 定义中断 GPIO 配置。
- `reset-gpios = <&gpio 4 GPIO_ACTIVE_HIGH>` 定义复位 GPIO 配置。
- `status = "okay"` 启用该节点。

第 97~110 行, LTDC 显示控制器配置

- `<ldc` 定义 LTDC (LCD-TFT 显示控制器) 的节点。
- `default-on` 默认开启显示控制器。
- `rotation-memory = <&ldc_sec_rotation>` 使用指定的旋转内存。
- `status = "okay"` 启用 LTDC。
- `port` 定义 LTDC 的端口配置。
- `#address-cells = <1>; #size-cells = <0>` 配置地址单元数为 1, 大小单元数为 0。
- `ldc_ep0_out: endpoint@0` 定义端口 0 的端点
- `reg = <0>` 端点编号为 0。
- `remote-endpoint = <&lvds_in>` 连接到 LVDS 的输入端点。

第 111~131 行, LVDS 配置

- `&lvds` 定义 LVDS 控制器节点。
- `default-on` 默认开启 LVDS。
- `vdd-supply = <&scmi_vddcore>; vdda18-supply = <&scmi_v1v8>` 定义 LVDS 的供电电源: 核心电压和 1.8V 电压。
- `status = "okay"` 启用 LVDS。
- `ports` 定义 LVDS 的端口配置。
- `#address-cells = <1>; #size-cells = <0>` 配置地址单元数为 1, 大小单元数为 0。
- `port@0` 定义端口 0 的输入端点
- `reg = <0>` 端口编号为 0。
- `lvds_in: endpoint { remote-endpoint = <&ldc_ep0_out>; }` 连接到 LTDC 的输出端点。
- `port@1` 定义端口 1 的输出端点
- `reg = <1>` 端口编号为 1。
- `lvds_out0: endpoint { remote-endpoint = <&lvds_panel_in0>; }` 连接到 LVDS 面板的输入端点 0。
- `port@2` 定义端口 2 的输出端点
- `reg = <2>` 端口编号为 2。
- `lvds_out1: endpoint { remote-endpoint = <&lvds_panel_in1>; }` 连接到 LVDS 面板的输入端点 1。

20.10 Display 工具

20.10.1 fbset

`fbset` 是一个用于显示和设置帧缓冲设备参数的工具。它可以查询和调整帧缓冲设备 (如 LCD 显示屏) 的当前模式、分辨率和其他属性。在嵌入式 Linux 系统中, `fbset` 常用于配置和调试显示设备。

以正点原子 MIPI 720x1280 屏幕为例, 系统打印如下:

fbset

```
root@ATK-DLMP257:~# fbset
mode "720x1280-0"
# D: 0.000 MHz, H: 0.000 kHz, V: 0.000 Hz
geometry 720 1280 720 1280 16
timings 0 0 0 0 0 0
accel false
rgba 5/11,6/5,5/0,0/0
endmode
root@ATK-DLMP257:~#
```

图 20.10-1 正点原子 720P 屏幕的 fbset 信息

打印信息解析如下:

- mode "720x1280"

这是显示模式的名称,通常根据分辨率命名。这里表示当前帧缓冲的分辨率为 720x1280 像素。

- geometry 720 1280 720 1280 16

geometry 参数描述了帧缓冲区的几何信息:

720: 显示的宽度(像素)。

1280: 显示的高度(像素)。

720: 帧缓冲区的可见宽度(像素),通常与显示宽度相同。

1280: 帧缓冲区的可见高度(像素),通常与显示高度相同。

16: 每个像素的颜色深度(位)。这里是 16 位,表示每个像素用 16 位存储。

- timings 0 0 0 0 0 0

timings 参数用于描述显示的时序参数(如同步脉冲、前后肩等)。这些值通常由显示硬件自动设置,因此在许多情况下,它们可能显示为零。

- accel true

该字段表示是否启用了硬件加速。true 表示硬件加速已启用,这通常意味着显示控制器支持某些加速操作,如快速位块传输或矢量图形操作。

- rgba 5/11,6/5,5/0,0/0

rgba 参数定义了像素格式,即每个颜色通道的位数和顺序:

5/11: 红色通道占用 5 位,从第 11 位开始。

6/5: 绿色通道占用 6 位,从第 5 位开始。

5/0: 蓝色通道占用 5 位,从第 0 位开始。

0/0: Alpha 通道不使用。

这表明当前的像素格式是 16 位真彩色,即 RGB565 格式。

20.10.2 modetest

modetest 是一个来自 libdrm (Direct Rendering Manager) 工具包的命令行工具,用于测试和显示 DRM/KMS (Kernel Mode Setting) 的相关信息。它能够列出显示硬件的编码器(Encoders)、连接器(Connectors)、显示控制器(CRTCs)、平面(Planes)和帧缓冲(Frame Buffers)等信息。该工具对于调试和配置显示设备非常有用,尤其是在开发驱动程序或调试显示输出时。

以正点原子 MIPI 720x1280 屏幕为例,系统打印如下:

```
modetest
```

```

root@ATK-DLMP257:~# modetest
trying to open device 'i915'...failed
trying to open device 'amdgpu'...failed
trying to open device 'radeon'...failed
trying to open device 'nouveau'...failed
trying to open device 'vmwgfx'...failed
trying to open device 'omapdrm'...failed
trying to open device 'exynos'...failed
trying to open device 'tilcdc'...failed
trying to open device 'msm'...failed
trying to open device 'sti'...failed
trying to open device 'tegra'...failed
trying to open device 'imx-drm'...failed
trying to open device 'rockchip'...failed
trying to open device 'atmel-hlcdc'...failed
trying to open device 'fsl-dcu-drm'...failed
trying to open device 'vc4'...failed
trying to open device 'virtio_gpu'...failed
trying to open device 'mediatek'...failed
trying to open device 'meson'...failed
trying to open device 'plll1'...failed
trying to open device 'stm'...done
Encoders:
id      crtc   type    possible crtcs  possible clones
31      41     DSI     0x00000001     0x00000001

```

图 20.10-2 正点原子 720P 屏幕的 modetest 信息-Encoders 部分

打印信息解析如下:

失败部分

- "trying to open device '...': failed"

含义: modetest 工具尝试访问常见的 DRM (Direct Rendering Manager) 显示设备驱动程序, 但未能成功。

原因:

系统没有加载这些设备对应的 DRM 驱动程序。

所有列出的设备 (如 i915、amdgpu 等) 都不匹配当前的硬件。

- "trying to open device 'stm': done"

含义: 最后成功打开了名为 stm 的设备。

解释: 当前硬件平台的 DRM 驱动为 stm (可能指代 STM32 系列处理器的 DRM 驱动)。

成功部分

- Encoders:

表示系统中的编码器信息, 用于将显示内容映射到显示屏。

当前系统只有一个编码器: id: 31, crtc: 41, type: DSI, possible crtcs: 0x00000001

类型: DSI (Display Serial Interface)。

关联 CRTC: ID 为 41 的显示控制器。

可能的 CRTC 映射: 0x00000001, 说明只支持一个 CRTC。


```
Connectors:
id      encoder status      name      size (mm)  modes  encoders
32      31      connected  DSI-1     0x0      1      31
modes:
  index name refresh (Hz) hdisp hss hse htot vdisp vss vse vtot
#0 720x1280 59.98 720 748 758 774 1280 1294 1302 1314 61000 flags: nhsync, nvsync; type: preferred, driver
props:
  1 EDID:
    flags: immutable blob
    blobs:
      value:
  2 DPMS:
    flags: enum
    enums: On=0 Standby=1 Suspend=2 Off=3
    value: 0
  5 link-status:
    flags: enum
    enums: Good=0 Bad=1
    value: 0
  6 non-desktop:
    flags: immutable range
    values: 0 1
    value: 0
  4 TILE:
    flags: immutable blob
    blobs:
      value:
  33 dithering:
    flags: enum
    enums: Off=0 On=1
    value: 0
```

图 20.10-3 正点原子 720P 屏幕的 modetest 信息-Connectors 部分

● Connectors

系统中列出的显示连接器。

当前系统只有一个连接器: id: 32, encoder: 31, status: connected, name: DSI-1, size: 0x0, modes: 1, encoders: 31

连接状态: 已连接。

名称: DSI-1。

分辨率模式: 只有一个模式, 720x1280 @ 59.98 Hz。

```
CRTCs:
id      fb      pos      size
41      58      (0,0)    (720x1280)
#0 720x1280 59.98 720 748 758 774 1280 1294 1302 1314 61000 flags: nhsync, nvsync; type: preferred, driver
props:
  24 VRR_ENABLED:
    flags: range
    values: 0 1
    value: 0
  28 GAMMA_LUT:
    flags: blob
    blobs:
      value:
  29 GAMMA_LUT_SIZE:
    flags: immutable range
    values: 0 4294967295
    value: 256
```

图 20.10-4 正点原子 720P 屏幕的 modetest 信息- CRTC s 部分

● CRTC s (Display Controllers)

显示控制器负责将帧缓冲区内容传递到显示设备。

当前系统的 CRTC 信息: id: 41, fb: 58, pos: (0,0), size: (720x1280)

帧缓冲区 ID: 58。显示区域位置与尺寸: (0,0), (720x1280)。

```
Planes:
id      crtc   fb      CRTC x,y      x,y      gamma size      possible crtcs
34      41      58      0,0          0,0          0          0x00000001
formats: AR24 XR24 AB24 XB24 RA24 RX24 BA24 BX24 RG16 BG16 RG24 BG24 AR15 XR15 AR12 XR12 C8 YUYV YVYU UY
VY VYUY
props:
  8 type:
    flags: immutable enum
    enums: Overlay=0 Primary=1 Cursor=2
    value: 1
  30 IN_FORMATS:
    flags: immutable blob
    blobs:
      value:
        01000000000000001500000018000000
        01000000700000004152323458523234
        41423234584232345241323452583234
        42413234425832345247313642473136
        52473234424732344152313558523135
        41523132585231324338202059555956
        59565955555956595659555900000000
        ffff1f000000000000000000000000
        0000000000000000
      in_formats blob decoded:
        AR24: LINEAR(0x0)
        XR24: LINEAR(0x0)
        AB24: LINEAR(0x0)
        XB24: LINEAR(0x0)
        RA24: LINEAR(0x0)
        RX24: LINEAR(0x0)
        BA24: LINEAR(0x0)
        BX24: LINEAR(0x0)
        RG16: LINEAR(0x0)
        BG16: LINEAR(0x0)
        RG24: LINEAR(0x0)
        BG24: LINEAR(0x0)
        AR15: LINEAR(0x0)
        XR15: LINEAR(0x0)
        AR12: LINEAR(0x0)
        XR12: LINEAR(0x0)
        C8: LINEAR(0x0)
        YUYV: LINEAR(0x0)
        YVYU: LINEAR(0x0)
        UYVY: LINEAR(0x0)
        VYUY: LINEAR(0x0)
  36 COLOR_ENCODING:
    flags: enum
    enums: ITU-R BT.601 YCbCr=0 ITU-R BT.709 YCbCr=1
    value: 0
  37 COLOR_RANGE:
    flags: enum
    enums: YCbCr limited range=0 YCbCr full range=1
    value: 0
```

图 20.10-5 正点原子 720P 屏幕的 modetest 信息-Planes 部分

● Planes:

平面是图形硬件中的一个组成部分，用于控制图像的显示。

系统中列出了 3 个平面，其中：

第一个平面 (ID: 34) 为主平面，支持格式包括：AR24, XR24, AB24, XB24, RG16, BG16, YUYV, NV12 等。

其他平面支持类似格式，但不一定启用。

这些信息为系统中的显示设备提供了详细的硬件和软件配置数据，有助于调试和优化显示输出。

20.10.3 drmdevice

drmdevice 是一个用于显示和检查系统中可用的 DRM (Direct Rendering Manager) 设备的工具。DRM 是 Linux 内核的一部分，用于管理图形处理单元 (GPU) 的资源 and 提供直接渲染功能。drmdevice 工具可以帮助开发者和系统管理员获取与 DRM 设备相关的信息，比如设备的节点、总线类型、平台信息和兼容性等。

以正点原子 MIPI 720x1280 屏幕为例，系统打印如下：

drmdevice

```

root@ATK-DLMP257:~# drmdevice
--- Checking the number of DRM device available ---
--- Devices reported 1 ---
--- Retrieving devices information (PCI device revision is ignored) ---
device[0]
+--> available_nodes 0x01
+--> nodes
| +--> nodes[0] /dev/dri/card0
+--> bustype 0002
| +--> platform
|   +--> fullname /soc@0/bus@42080000/display-controller@48010000
+--> deviceinfo
+--> platform
+--> compatible
    st,stm32mp25-ltdc

--- Opening device node /dev/dri/card0 ---
--- Retrieving device info, for node /dev/dri/card0 ---
device[0]
+--> available_nodes 0x01
+--> nodes
| +--> nodes[0] /dev/dri/card0
+--> bustype 0002
| +--> platform
|   +--> fullname /soc@0/bus@42080000/display-controller@48010000
+--> deviceinfo
+--> platform
+--> compatible
    st,stm32mp25-ltdc

root@ATK-DLMP257:~#

```

图 20.10-6 正点原子 720P 屏幕的 drmdevice 信息

- Checking the number of DRM device available:

这是检查系统中有多少个 DRM 设备的步骤。

输出显示 "Devices reported 1", 表示系统中发现了一个 DRM 设备。

- Retrieving devices information:

开始检索每个设备的详细信息。

- device[0]:

表示第一个设备的详细信息。这里只有一个设备, 所以它被标识为 device[0]。

- available_nodes 0x01:

这表示设备的可用节点。0x01 是一个位掩码, 表示存在一个可用节点。

- nodes:

nodes[0] /dev/dri/card0:

表示该设备的设备文件节点是 /dev/dri/card0。这是一个典型的 DRM 设备文件, 用于与图形硬件进行交互。

- bustype 0002:

这表示设备的总线类型。0002 通常表示平台设备, 而不是 PCI 或其他类型的设备。

- platform:

fullname /soc@0/bus@42080000/display-controller@48010000:

这是设备的完整平台名称, 表明它是一个显示子系统。

- deviceinfo:

platform:

st,stm32mp25-ltdc:

表示设备的兼容性信息。st,stm32mp25-ltdc 表示该设备是由 ST 公司制造的 MP25 系列处理器的显示子系统。

- Opening device node /dev/dri/card0:

这是尝试打开 /dev/dri/card0 设备节点的操作, 用于进一步查询设备信息。



- Retrieving device info, for node /dev/dri/card0:

再次显示设备的详细信息，这与之前的信息基本一致，确认该设备的各种属性和兼容性。

总结来说，这段输出表明系统中有一个 DRM 设备，具体是基于 STMP25 显示子系统的平台设备，设备节点是 /dev/dri/card0。这类信息对于开发者在系统中配置和使用显示硬件非常有用。

第二十一章 Ethernet 子系统

21.1 Ethernet 子系统简介

嵌入式 Linux 的以太网子系统 (Ethernet Subsystem) 是用于处理网络通信的关键组件。它主要涉及管理网络接口、处理数据包传输和接收、支持各种网络协议等。

21.2 组成部分

1、网络接口 (Network Interfaces) :

管理物理和虚拟网络接口, 如以太网卡、Wi-Fi 等。

支持多种网络设备驱动程序, 确保硬件兼容性。

2、网络协议栈 (Network Protocol Stack) :

提供 TCP/IP 协议栈, 支持标准网络通信协议 (如 ARP、ICMP、UDP、TCP 等)。

处理数据包的封装和解封装, 确保数据正确传输。

3、网络配置工具 (Network Configuration Tools) :

提供命令行工具 (如 ifconfig、ip、ethtool 等) 用于配置和管理网络接口。

支持动态 IP 配置 (DHCP) 和静态 IP 配置。

4、网络服务 (Network Services) :

提供基本的网络服务, 如 DHCP 客户端/服务器、DNS 解析、NTP 时间同步等。

支持高级网络服务和功能, 如 VPN、网桥、虚拟局域网 (VLAN) 等。

21.3 主要功能

1、网络接口管理:

配置和管理多种网络接口, 支持网络接口的启用、禁用和参数调整。

2、数据包处理:

高效处理数据包的发送和接收, 确保低延迟和高吞吐量。

处理数据包的路由和转发, 支持多种路由协议。

3、网络协议支持:

支持多种标准网络协议, 确保嵌入式设备与其他设备的互操作性。

提供可靠的传输控制和流量控制, 确保数据传输的稳定性。

4、网络安全:

提供基本的网络安全功能, 如防火墙 (iptables)、网络地址转换 (NAT) 等。

支持安全的通信协议, 如 TLS/SSL 等。

5、服务质量 (QoS) :

提供服务质量管理, 确保关键数据流的优先级和带宽分配。

支持流量整形和带宽管理, 优化网络资源使用。

21.4 MP257 的 Ethernet 说明

21.4.1 文档和资源

想要获取 MP257 系列处理器的 Ethernet 详细说明，最直接的方式是参考 ST 官方文档，包括：

1、参考手册（Reference Manual）：此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南（即本文档）

应用开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图：【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

21.4.2 特性说明

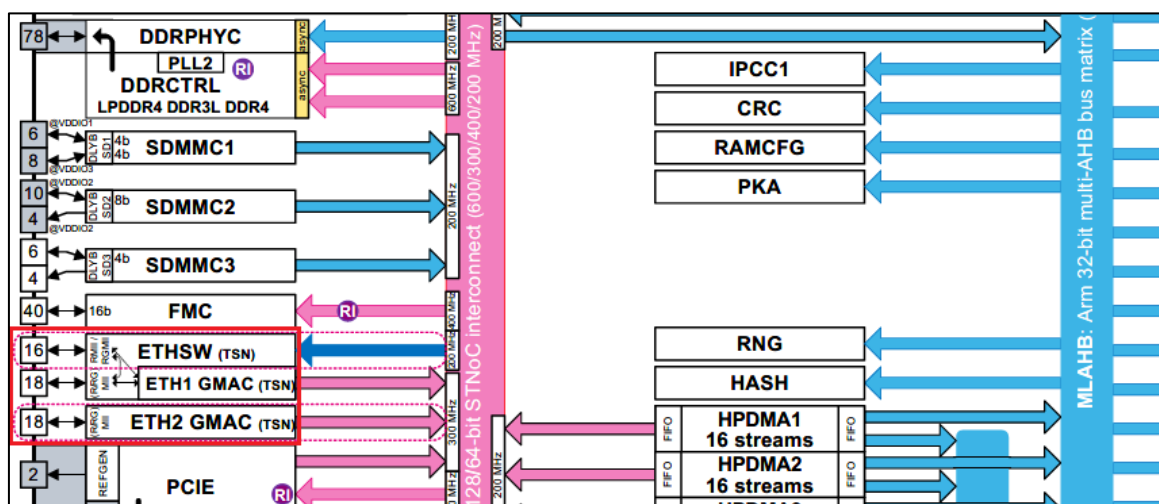


图 21.4-1 MP257 的 Ethernet 部分

STM32MP257 集成了两个独立的千兆以太网 MAC 控制器（ETH1/2），支持 IEEE 802.3-2008 标准。每个控制器通过标准的媒体独立接口（RGMII、MII 或 RMII）连接外部 PHY，支

持 10Mbps、100Mbps 和 1000Mbps 的传输速率，可实现全双工和半双工操作，并支持标准或 Jumbo 帧。控制器提供两个独立的收发队列，支持基于 PCP 代码的队列管理，具备低功耗特性（如 EEE 节能以太网和 LAN 唤醒帧检测），并支持 IEEE 1588-2008（PTP）和 IEEE 802.1AS 的时序与同步功能。支持时间敏感网络（TSN）特性，包括 IEEE 802.1Qav、802.1Qbv 和 802.1Qbu 协议。此外，还提供自动 CRC 生成与校验、帧预处理、MAC 地址过滤、VLAN 标签过滤、多播/广播规则、TCP/IP 卸载功能（如校验和计算与检测、TCP 分段卸载）等多种高级功能，适用于高性能网络通信场景。

STM32MP257 设备集成了一个三端口千兆以太网交换机 (ETHSW)，支持时间敏感网络 (TSN) 及相关标准，并在两个外部端口之间提供名为 ACM 的切换加速功能。交换机的两个端口通过 RGMII 和 RMII 接口对外连接，第三个端口内部连接到 ETH1 控制器。交换机支持完全旁路模式，在此模式下，ETH1 可以直接连接到一个外部端口。

更多关于 MP257 的 Ethernet 资料请查看芯片参考手册。

21.5 开发板 Ethernet 接口

21.5.1 原理图分析

开发板上有三个以太网接口。打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 ETH。可以看到相关电路如下。

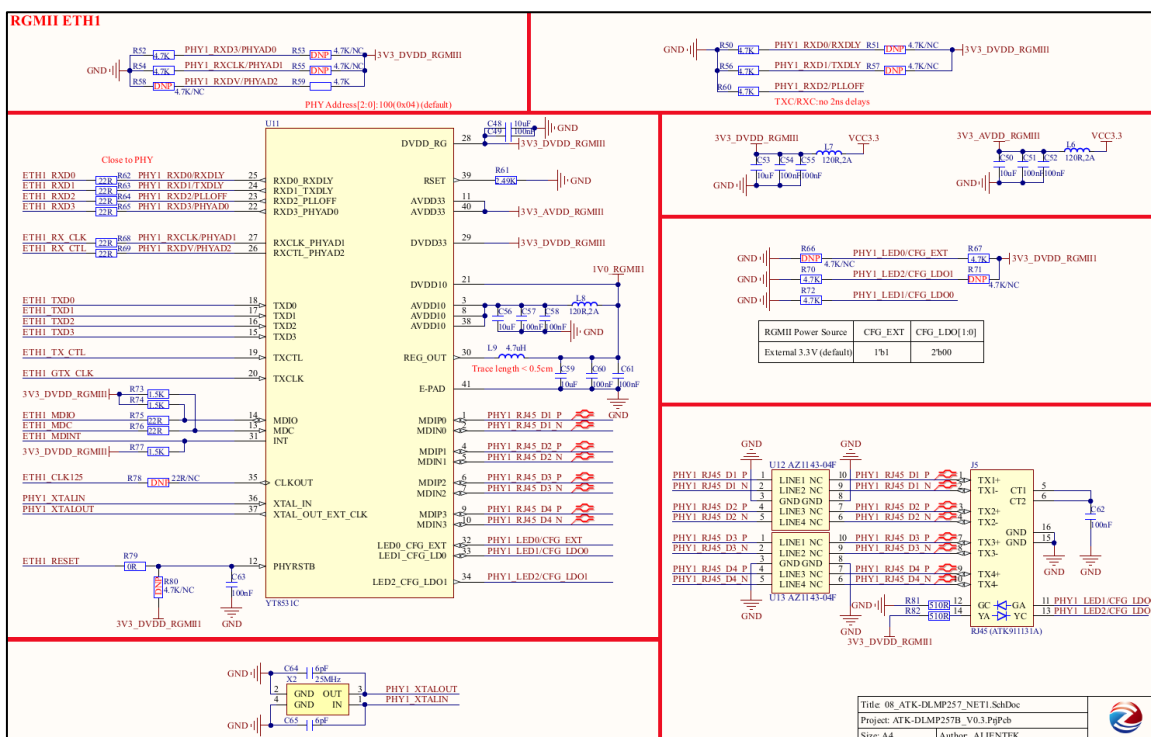


图 21.5-1 开发板 ETH1 电路

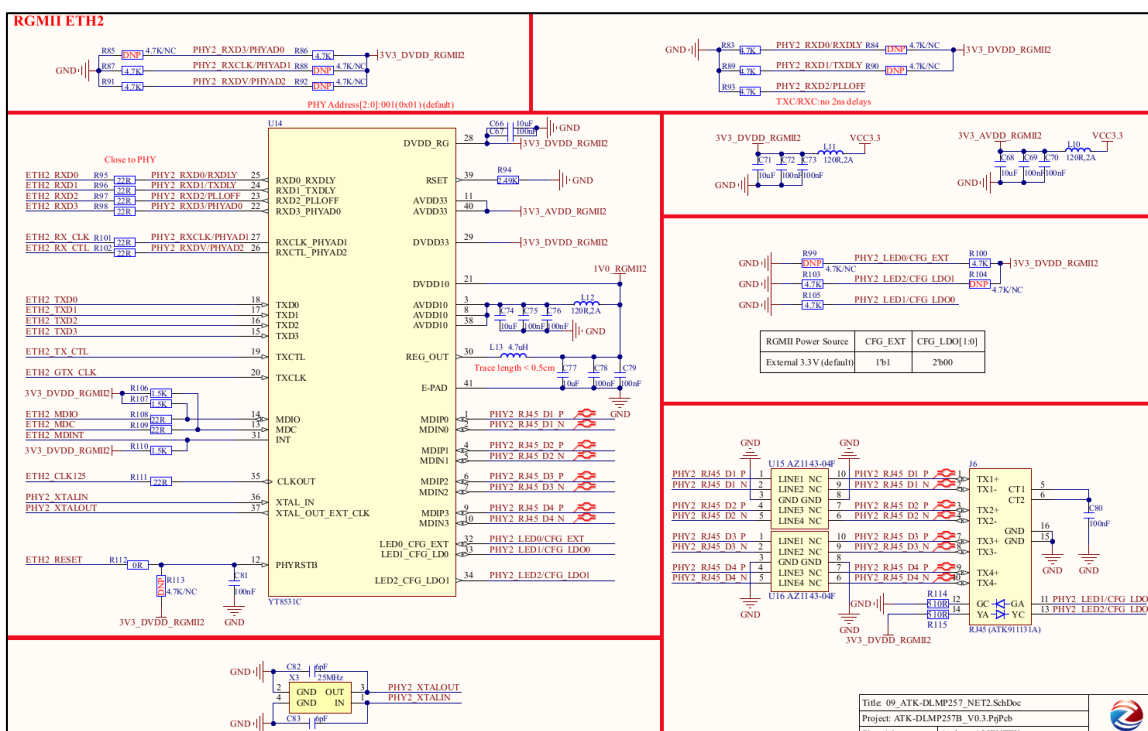


图 21.5-2 ETH2 电路

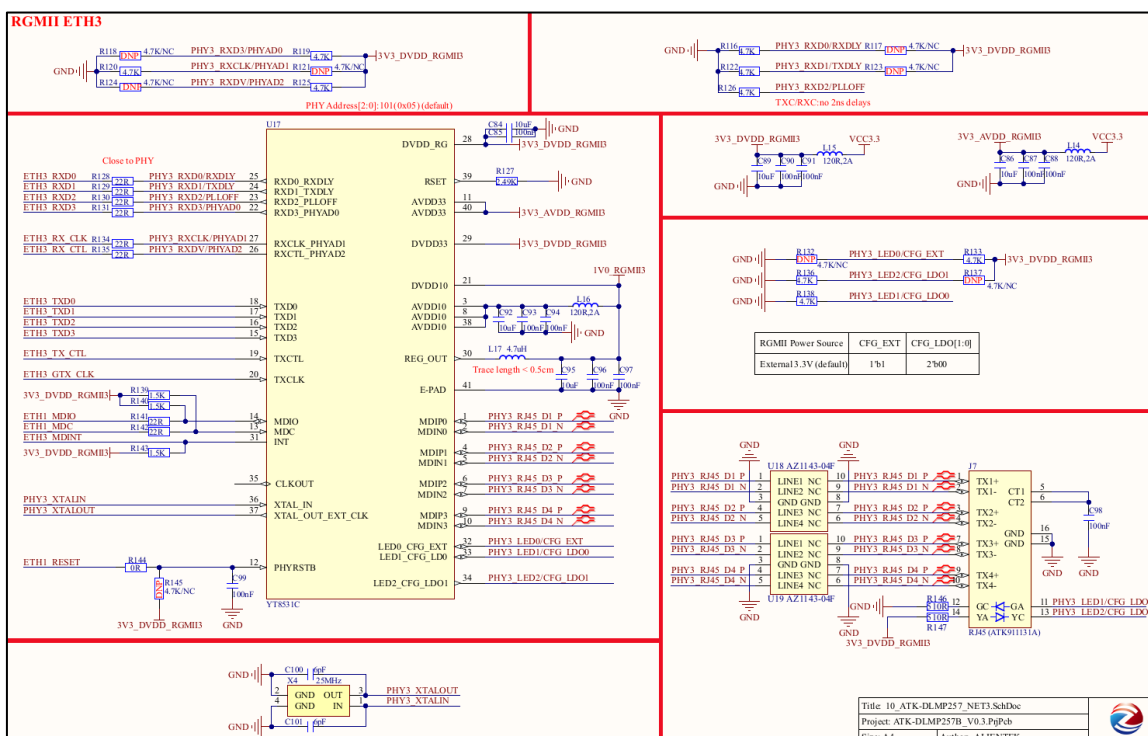


图 21.5-3 ETH3 电路

这三个网口的原理图类似，这里以 ETH1 为例进行说明：

1、RGMII PHY 接口

左侧标注为 RGMII ETH1 的部分:



- ETH1_TXD[0-3], ETH1_RXD[0-3]: 分别为发送和接收数据线, 连接到处理器的 RGMII 接口。
- ETH1_TX_CLK, ETH1_RX_CLK: 分别为发送时钟和接收时钟, 用于同步数据传输。
- ETH1_GTX_CLK: 发送千兆数据时使用的时钟信号。
- ETH1_TX_CTL/ETH1_RX_CTL: 分别为发送和接收控制信号, 标志数据帧的起止。

2、电源供电

电路中使用了以下电压:

- 3.3V_DVDD_RGMII: 供电给 RGMII 相关部分和 PHY 芯片。
- 1V0_RGMII: 通过电感 L9 和去耦电容生成, 用于供电 PHY 芯片的低电压核心。
- 各电源引脚 (如 AVDD33/DVDD10) 均配有滤波电容 (如 C20、C61), 用于稳定电源。

3、PHY 芯片模块

黄色框内为 PHY 芯片:

- MDIO/MDC: 管理数据接口, 用于通过主机 (处理器) 配置和读取 PHY 寄存器。
- RESET: 复位引脚, 配合外部 RC 电路实现复位控制。
- LED_CFG: 状态指示引脚, 通过外接 LED 显示网络链路状态和通信活动。
- XTAL_IN/XTAL_OUT: 连接外部晶振 (如 25MHz) 提供主时钟。
- TXD/RXD 引脚直接连接到处理器, 用于数据收发。

4、时钟电路

右下部分:

- 使用一个 25MHz 的晶振 (X2) 提供 PHY 芯片的时钟。
- 晶振两端的电容 (C64 和 C65) 用于频率稳定。
- 如果需要外部提供时钟, 可以通过 PHY 的时钟输入 (XTAL_IN) 实现。

5、RJ45 连接器与变压器

右侧底部为网络接口:

- RJ45 (J5) 连接到外部以太网设备。
- 使用磁性变压器 (U12, U13) 实现信号隔离和抗干扰。
- 差分信号线 (如 TX+, TX-) 通过变压器连接到 RJ45。
- 共模扼流圈 (如 L8) 进一步减少电磁干扰 (EMI)。

6、信号调试和配置

PHY 地址选择: 通过电阻 (如 R53, R54) 配置 PHY 地址, 默认地址为 0x04。

延时匹配:

- 对接收和发送信号的引脚 (如 RXD、TXD) 增加电阻 (如 R58, R59), 调整信号的传输延迟。
- 根据标注, TX 和 RX 信号具有 2ns 延迟。

7、状态指示灯

PHY 芯片的 LED_CFG_LDO0 和 LED2_CFG_LDO1 引脚连接到 RJ45 的 LED, 用于显示网络状态:

- 链路状态 (LINK)。
- 数据传输活动 (ACTIVITY)。

21.5.2 硬件实物

开发板上的 Ethernet 接口位置如下图所示:

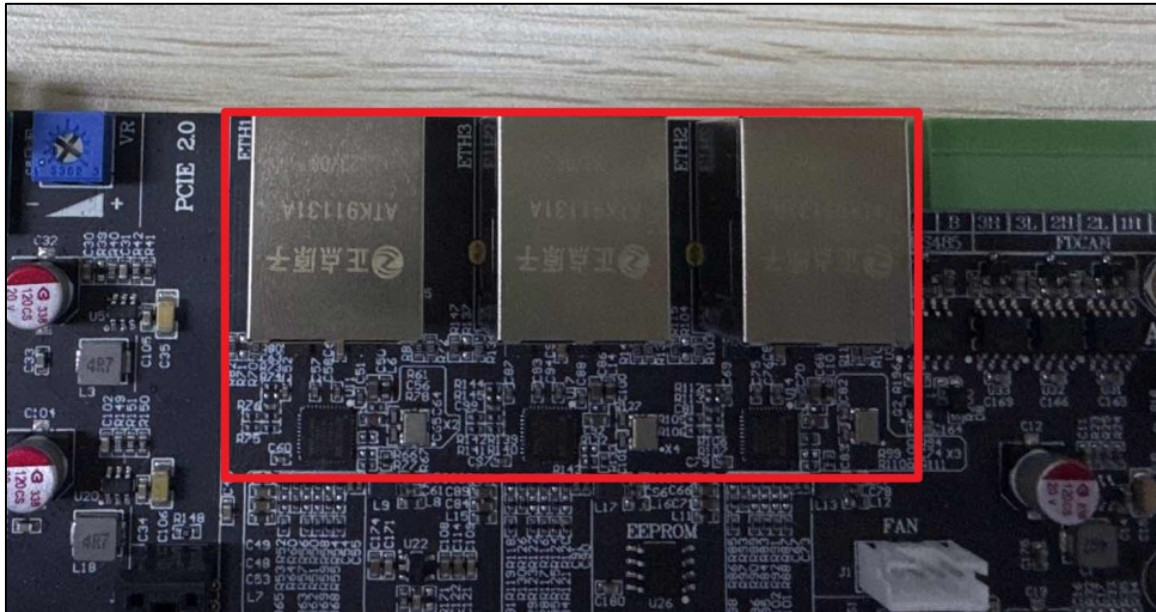


图 21.5-4 开发板 Ethernet 接口

21.6 内核 Ethernet 使能

在 ATK-DLMP257B 内核源码配置文件 `arch/arm64/configs/stm32mp257_atk_defconfig` 中, Ethernet 接口相关配置如下(略有删减, 只列举重点部分):

21.6.1 网络功能使能

在 ATK-DLMP257B 内核源码配置文件 `arch/arm64/configs/stm32mp257_atk_defconfig` 中, 网络相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_PACKET=y //Packet Socket 允许用户空间程序直接发送和接收链路层数据包
2 CONFIG_INET=y //启用 IPv4 协议栈支持。
3 CONFIG_IP_PNP=y //启用 IP 地址的自动配置 (IP PnP)。
4 CONFIG_IP_PNP_DHCP=y //启用 DHCP (动态主机配置协议) 支持, 用于 IP 自动配置。
5 CONFIG_IP_PNP_BOOTP=y //启用 Bootstrap Protocol 支持, 用于 IP 自动配置。
6 CONFIG_IPV6=m //启用 IPv6 协议栈支持。
7 CONFIG_DNS_RESOLVER=y //启用 内核 DNS 解析支持。
```

对应的 `menuconfig` 配置如下 (篇幅有限, 截图有所省略):

```
[*] Networking support --->
    [*] Networking options --->
        [*] Packet socket
        [*] TCP/IP networking
            [*] IP: kernel level autoconfiguration
                [*] IP: DHCP support
                [*] IP: BOOTP support
                [*] IP: RARP support
            [*] INET: socket monitoring interface
            [*] The IPv6 protocol
            [*] DNS Resolver support
```

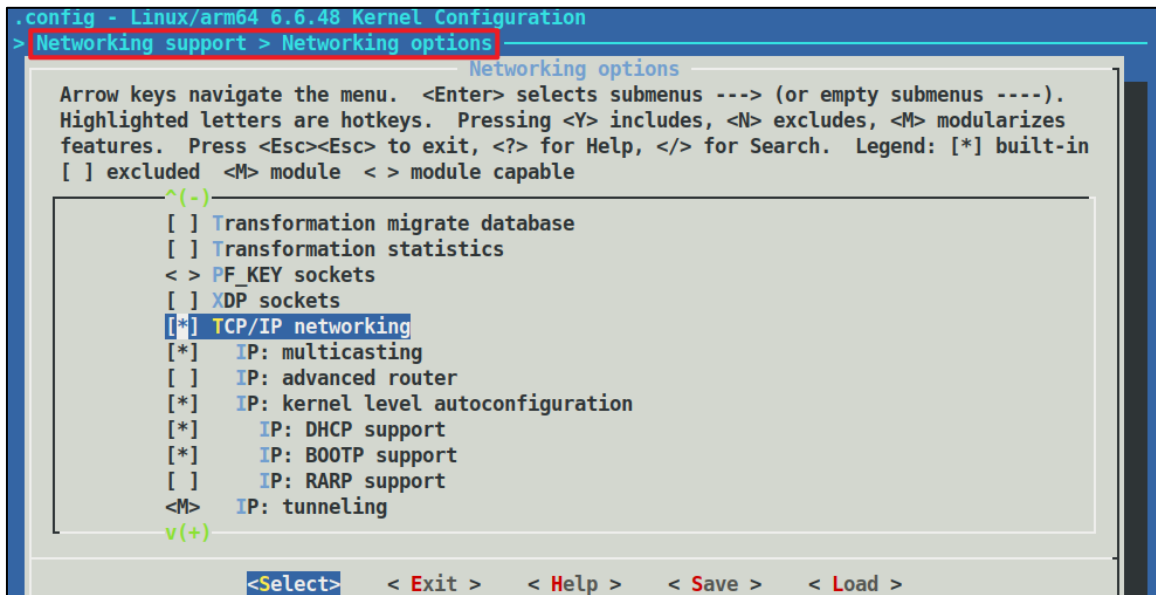


图 21.6-1 网络配置节选

21.6.2 通用 PHY 使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, 通用 PHY 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_GENERIC_PHY=y //启用通用 PHY 框架支持
```

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

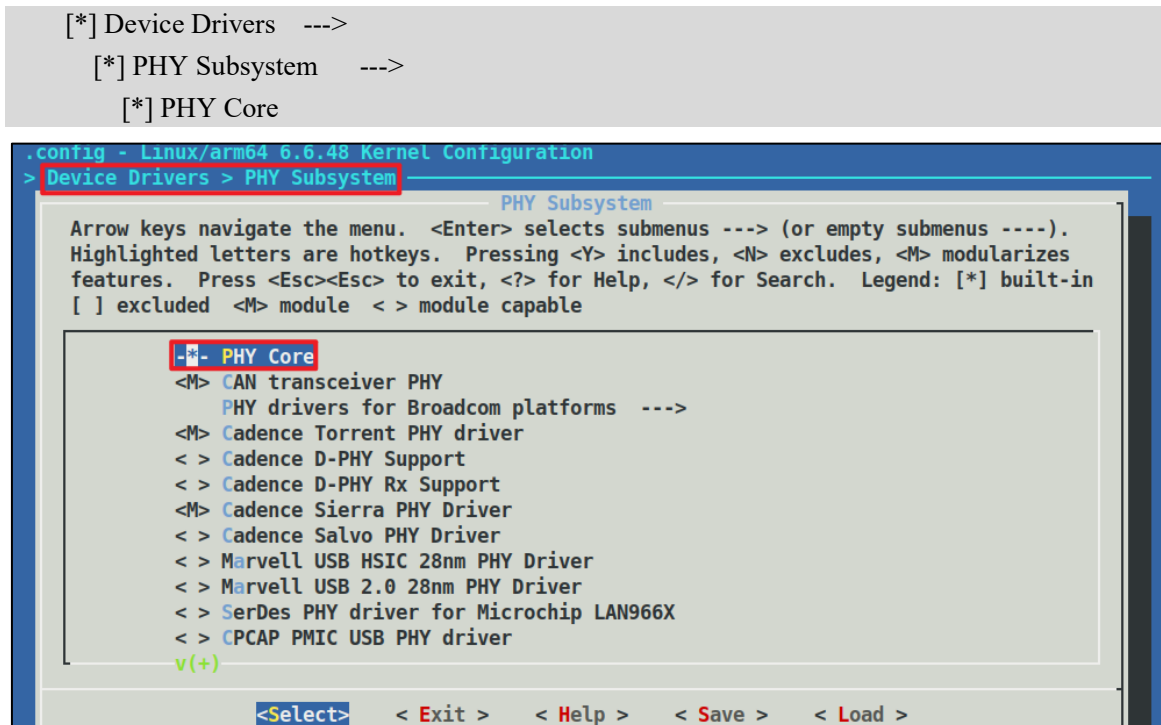


图 21.6-2 通用 PHY 配置

21.6.3 STM32 DWMAC 使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, 通用 STM32 DWMAC 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_NET_VENDOR_STMICRO=y //启用对 STMicroelectronics 网络设备的支持
2 CONFIG_STMMAC_ETH=y // 启用 Synopsys DWC Ethernet MAC 驱动。
3 CONFIG_STMMAC_PLATFORM=y //启用 STMMAC 平台驱动支持。
4 CONFIG_DWMAC_GENERIC=y //启用 通用 DWMAC 驱动 支持。
5 CONFIG_DWMAC_STM32=y //启用 STM32 专用 DWMAC 驱动 支持。
```

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

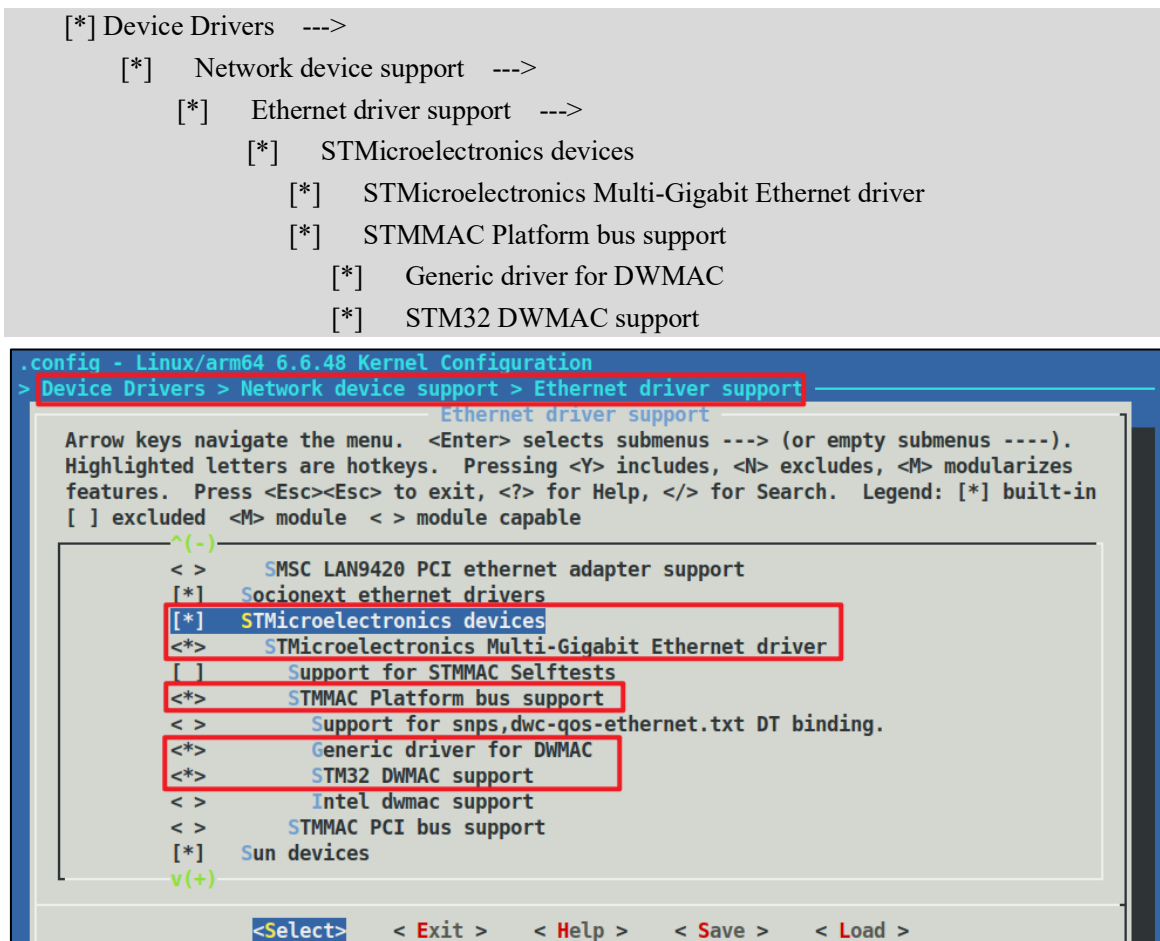


图 21.6-3 STM32 DWMAC 配置

21.7 设备树分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi, 此文件是特定于 STM32MP2 系列芯片的设备树包含文件, 通常不独立使用, 而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置, 而 .dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 eth1 描述如下 (eth2 在 stm32mp253.dtsi、交换机 switch0 在 stm32mp257.dtsi):

```
1 eth1: eth1@482c0000 {
2     compatible = "st,stm32mp25-dwmac", "snps,dwmac-5.10a";
3     reg = <0x482c0000 0x4000>;
```




```

4     reg-names = "stmmaceth";
5     interrupts-extended = <&intc GIC_SPI 130 IRQ_TYPE_LEVEL_HIGH>,
6                           <&exti1 68 IRQ_TYPE_LEVEL_HIGH>;
7     interrupt-names = "macirq",
8                       "eth_wake_irq";
9     clock-names = "stmmaceth",
10                  "mac-clk-tx",
11                  "mac-clk-rx",
12                  "ptp_ref",
13                  "ethstp",
14                  "eth-ck";
15     clocks = <&rcc CK_ETH1_MAC>,
16             <&rcc CK_ETH1_TX>,
17             <&rcc CK_ETH1_RX>,
18             <&rcc CK_KER_ETH1PTP>,
19             <&rcc CK_ETH1_STP>,
20             <&rcc CK_KER_ETH1>;
21     st,syscon = <&syscfg 0x3000 0xffffffff>;
22     snps,mixed-burst;
23     snps,pbl = <2>;
24     snps,txqos = <7>;
25     snps,rxqos = <7>;
26     snps,axi-config = <&stmmac_axi_config_1>;
27     snps,tso;
28     access-controllers = <&rifsc 60>;
29     power-domains = <&CLUSTER_PD>;
30     wakeup-source;
31     status = "disabled";
32     snps,mtl-rx-config = <&mtl_rx_setup_1>;
33     snps,mtl-tx-config = <&mtl_tx_setup_1>;
34
35     stmmac_axi_config_1: stmmac-axi-config {
36         snps,wr_osr_lmt = <0x7>;
37         snps,rd_osr_lmt = <0x7>;
38         snps,blen = <0 0 0 0 16 8 4>;
39     };
40
41     mtl_rx_setup_1: rx-queues-config {
42         snps,rx-queues-to-use = <2>;
43         queue0 {};
44         queue1 {};
45     };
46

```



```

47     mtl_tx_setup_1: tx-queues-config {
48         snps,tx-queues-to-use = <4>;
49         queue0 {};
50         queue1 {};
51         queue2 {};
52         queue3 {};
53     };
54 };

```

第 1 行, eth1: eth1@482c0000 定义一个名为 eth1 的设备节点, 其物理地址为 0x482c0000, 表示第二个以太网控制器。

第 2 行, compatible = "st,stm32mp25-dwmac", "snps,dwmac-5.10a" 表示此设备兼容的驱动类型: "st,stm32mp25-dwmac" 是专门针对 STM32MP25 的 DWMAC 驱动。

"snps,dwmac-5.10a" 是 Synopsys DesignWare MAC 控制器的版本标识。

第 3 行, reg = <0x482c0000 0x4000> 定义设备寄存器的物理地址范围: 起始地址: 0x482c0000。大小: 0x4000 字节 (16 KB)。

第 4 行, reg-names = "stmmaceth" 为寄存器块命名, 名称为 "stmmaceth"。

第 5~6 行, 定义两个中断:

GIC_SPI 130 表示全局中断控制器 (GIC) 的 SPI 130 中断, 为高电平触发。

exti1 68 表示 EXTI 控制器的第 68 个中断, 为高电平触发。

第 7~8 行, 为中断命名:

"macirq" 对应 MAC 中断。

"eth_wake_irq" 对应以太网唤醒中断。

第 9~14 行, 定义时钟的名称列表:

"stmmaceth": 主 MAC 时钟。

"mac-clk-tx": 传输时钟。

"mac-clk-rx": 接收时钟。

"ptp_ref": 精确时间协议 (PTP) 参考时钟。

"ethstp": 以太网停止信号时钟。

"eth-ck": 以太网主时钟。

第 15~20 行, 定义实际使用的时钟信号, 引用 RCC (时钟控制器) 的时钟节点:

CK_ETH1_MAC: MAC 时钟。

CK_ETH1_TX: 传输时钟。

CK_ETH1_RX: 接收时钟。

CK_KER_ETH1PTP: PTP 时钟。

CK_ETH1_STP: 以太网停止信号时钟。

CK_KER_ETH1: 内核时钟。

第 21 行, st,syscon = <&syscfg 0x3000 0xffffffff> 指定与系统控制器 (SYSCFG) 的关联: 偏移地址为 0x3000。掩码为 0xffffffff。

第 22 行, snps,mixed-burst 启用混合突发模式 (Mixed Burst Mode), 用于优化 AXI 总线性能。

第 23~25 行, 定义 Synopsys DesignWare MAC 的配置参数:

snps,pbl = <2>: 设置可编程突发长度 (PBL) 为 2。

snps,txqos = <7>: 设置传输队列的 QoS 等级为 7。



snps,rxqos = <7>: 设置接收队列的 QoS 等级为 7。

第 26 行, snps,axi-config = <&stmmac_axi_config_1> 指定 AXI 总线的配置, 引用后续定义的 stmmac_axi_config_1 节点。

第 27 行, snps,tso 启用分段任务卸载 (TSO, TCP Segmentation Offload)。

第 28 行, 定义访问控制器的节点, 引用 rifsc, ID 为 60。

第 29 行, 指定该设备的电源域, 引用 CLUSTER_PD。

第 30 行, 声明此设备支持唤醒功能。

第 31 行, 设置该设备节点的默认状态为 禁用。

第 32~33 行, 指定 MAC 传输层 (MTL) 的配置:

接收队列配置引用 mtl_rx_setup_1。

传输队列配置引用 mtl_tx_setup_1。

第 35~39 行, 定义 AXI 总线的详细配置:

snps,wr_osr_lmt = <0x7>: 写操作的最大突发长度为 7。

snps,rd_osr_lmt = <0x7>: 读操作的最大突发长度为 7。

snps,blen = <0 0 0 0 16 8 4>: 定义突发长度列表。

第 41~45 行, 定义接收队列配置: 启用 2 个接收队列 (queue0 和 queue1)。

第 47~53 行, 定义传输队列配置: 启用 4 个传输队列 (queue0、queue1、queue2 和 queue3)。

这个设备树文件定义了 STM32MP25 平台上 eth1 以太网控制器的硬件配置, 包括其地址、时钟、中断、AXI 总线、接收和传输队列等内容。

以 2GB 版本核心板为例, 在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 中, eth 相关配置如下:

```

1 / {
2     .....
3     aliases {
4         ethernet0 = &eth2;
5         ethernet1 = &eth1;
6         .....
7     };
8     &eth1 {
9         status = "okay";
10        pinctrl-0 = <&eth1_mdio_pins_a>;
11        pinctrl-names = "default";
12        phy-mode = "rgmii";
13        st,eth-clk-sel;
14        snps,ext-systemtime;
15        fixed_link: fixed-link {
16            speed = <1000>;
17            full-duplex;
18        };
19
20        mdio1 {
21            #address-cells = <1>;

```



```

22     #size-cells = <0>;
23     compatible = "snps,dwmac-mdio";
24     phy1_eth1: ethernet-phy@4 {
25         compatible = "ethernet-phy-id4f51.e91b",
26             "ethernet-phy-ieee802.3-c22";
27         reg = <4>;
28     };
29     phy2_eth1: ethernet-phy@5 {
30         compatible = "ethernet-phy-id4f51.e91b",
31             "ethernet-phy-ieee802.3-c22";
32         reset-gpios = <&gpiof 3 GPIO_ACTIVE_LOW>;
33         reset-assert-us = <10000>;
34         reset-deassert-us = <80000>;
35         reg = <5>;
36     };
37 };
38 };
39
40 &eth2 {
41     status = "okay";
42     pinctrl-0 = <&eth2_rgmii_pins_a>;
43     pinctrl-1 = <&eth2_rgmii_sleep_pins_a>;
44     pinctrl-names = "default", "sleep";
45     phy-mode = "rgmii-id";
46     max-speed = <1000>;
47     phy-handle = <&phy1_eth2>;
48     st,eth-ptp-from-rcc;
49
50     mdio1 {
51         #address-cells = <1>;
52         #size-cells = <0>;
53         compatible = "snps,dwmac-mdio";
54         phy1_eth2: ethernet-phy@1 {
55             compatible = "ethernet-phy-id4f51.e91b";
56             reset-gpios = <&gpiof 4 GPIO_ACTIVE_LOW>;
57             reset-assert-us = <10000>;
58             reset-deassert-us = <80000>;
59             reg = <1>;
60         };
61     };
62 };

```

第 3~7 行, 定义别名 (aliases), 用于简化设备节点的引用:

- ethernet0 对应 eth2 (第二个以太网控制器)。



- ethernet1 对应 eth1（第一个以太网控制器）。

第 8 行，定义第一个以太网控制器节点，标识为 eth1。

第 9 行，将该设备节点的状态设置为启用（okay）。

第 10 行，pinctrl-0 = <ð1_mdio_pins_a> 关联第一个以太网控制器的 MDIO 引脚配置，引用 eth1_mdio_pins_a。

第 11 行，pinctrl-names = "default" 定义引脚配置的模式名称为 "default"。

第 12 行，phy-mode = "rgmii" 设置 PHY 的工作模式为 RGMII（Reduced Gigabit Media Independent Interface）。

第 13 行，st,eth-clk-sel STM32 专用属性，启用以太网时钟选择功能。

第 14 行，snps,ext-systime 启用外部系统时间支持（通常用于 PTP 时间同步）。

第 15~18 行，定义固定链路配置：

- speed = <1000>：链路速度为 1000 Mbps。
- full-duplex：启用全双工模式。

此配置表示该以太网端口使用固定链路，而不是通过 MDIO 动态配置。

第 20 行，定义一个 MDIO 总线节点，标识为 mdio1。

第 21~22 行，指定 MDIO 总线的地址和大小单元：

- #address-cells = <1>：地址单元大小为 1。
- #size-cells = <0>：大小单元为 0（MDIO 总线不需要额外大小信息）。

第 23 行，指定 MDIO 总线的兼容性为 snps,dwmac-mdio，表示这是一个 Synopsys DesignWare MAC 的 MDIO 控制器。

第 24~28 行，定义第一个 PHY 节点：

- 标识为 phy1_eth1，地址为 4（@4）。
- 兼容性为 "ethernet-phy-id4f51.e91b" 和 "ethernet-phy-ieee802.3-c22"（表示这是 IEEE 802.3 标准兼容的以太网 PHY）。
- reg = <4>：PHY 在 MDIO 总线上的地址为 4。

第 29~35 行，定义第二个 PHY 节点：

- 标识为 phy2_eth1，地址为 5（@5）。
- 兼容性与 phy1_eth1 相同。
- reset-gpios = <&gpiof3 GPIO_ACTIVE_LOW>：定义复位引脚为 GPIOF 的第 3 个引脚，低电平有效。
- reset-assert-us = <10000>：复位持续时间为 10,000 微秒。
- reset-deassert-us = <80000>：复位释放后的延迟时间为 80,000 微秒。
- reg = <5>：PHY 在 MDIO 总线上的地址为 5。

第 40 行，ð2 定义第二个以太网控制器节点，标识为 eth2。

第 41 行，status = "okay" 将该设备节点的状态设置为启用（okay）。

第 42~43 行，定义两个引脚配置：

- pinctrl-0 关联正常工作模式下的引脚配置（eth2_rgmii_pins_a）。
- pinctrl-1 关联低功耗模式下的引脚配置（eth2_rgmii_sleep_pins_a）。

第 44 行，定义引脚配置的模式名称：

- "default" 对应正常工作模式。
- "sleep" 对应低功耗模式。

第 45 行，phy-mode = "rgmii-id" 设置 PHY 的工作模式为 RGMII-ID（带内部延迟的 RGMII）。

第 46 行, max-speed = <1000> 定义最大速度为 1000 Mbps (千兆以太网)。

第 47 行, phy-handle = <&phy1_eth2> 关联 PHY, 引用 phy1_eth2。

第 48 行, st,eth-ptp-from-rcc STM32 专用属性, 表示以太网 PTP 时钟从 RCC 获取。

第 50 行, 定义第二个 MDIO 总线节点, 标识为 mdio1。

第 51 行, 指定 MDIO 总线的地址和大小单元:

- #address-cells = <1>: 地址单元大小为 1。
- #size-cells = <0>: 大小单元为 0 (MDIO 总线不需要额外大小信息)。

第 53 行, compatible = "snps,dwmac-mdio" 定义兼容性, 与第一个 MDIO 总线相同。

第 54 行, 定义 eth2 的 PHY:

- 标识为 phy1_eth2, 地址为 1 (@1)。
- 定义复位引脚、时间以及兼容性, 与 phy2_eth1 类似, 但地址不同 (reg = <1>)。

总结: eth1 和 eth2 是两个以太网控制器节点。eth1 使用固定链路, 定义了两个 PHY (phy1_eth1 和 phy2_eth1)。eth2 使用动态 PHY, 关联了 phy1_eth2。两个节点的时钟、引脚和复位配置均明确定义, 支持高性能以太网通信。

21.8 ETH 工具

21.8.1 dmesg

Linux 内核通过串口输出日志消息, 可以使用 dmesg 命令来查看这些消息。这对于调试引导过程中的问题非常有用, 也可以用于检查内核模块加载和设备初始化过程中的消息。grep 是一个 Linux/Unix 下的命令行工具, 用于在文本文件中搜索指定的字符串模式, 并将包含匹配的行输出到标准输出 (通常是终端)。

示例:

dmesg | grep end0

```
root@ATK-DLMP257:~# dmesg | grep end0
[ 5.337744] stm32-dwmac 482d0000.eth2 end0: renamed from eth1
[ 14.339353] stm32-dwmac 482d0000.eth2 end0: Register MEM_TYPE_PAGE_POOL RxQ-0
[ 14.342504] stm32-dwmac 482d0000.eth2 end0: Register MEM_TYPE_PAGE_POOL RxQ-1
[ 14.629363] stm32-dwmac 482d0000.eth2 end0: PHY [stmmac-0:01] driver [YT8531 Gigabit Ethernet] (irq=POLL)
[ 14.642771] stm32-dwmac 482d0000.eth2 end0: No Safety Features support found
[ 14.649913] stm32-dwmac 482d0000.eth2 end0: IEEE 1588-2008 Advanced Timestamp supported
[ 14.659358] stm32-dwmac 482d0000.eth2 end0: registered PTP clock
[ 14.666035] stm32-dwmac 482d0000.eth2 end0: FPE workqueue start
[ 14.673144] stm32-dwmac 482d0000.eth2 end0: configuring for phy/rgmii-id link mode
[ 17.750085] stm32-dwmac 482d0000.eth2 end0: Link is Up - 1Gbps/Full - flow control rx/tx
root@ATK-DLMP257:~#
```

图 21.8-1 eth0 的打印信息

根据 dmesg 输出的信息, 可知:

stm32-dwmac 482d0000.eth2 end0: renamed from eth1

设备 stm32-dwmac 的网络接口被从 eth1 重命名为 end0。

stm32-dwmac 482d0000.eth2 end0: Register MEM_TYPE_PAGE_POOL RxQ-0

stm32-dwmac 482d0000.eth2 end0: Register MEM_TYPE_PAGE_POOL RxQ-1

网络接口的接收队列 (Rx Queue) 注册了内存池 (PAGE_POOL) 用于优化内存管理。这可以提高接收数据包的效率和吞吐量。

stm32-dwmac 482d0000.eth2 end0: PHY [stmmac-0:01] driver [YT8531 Gigabit Ethernet] (irq=POLL)

以太网物理层 (PHY) 使用的是 YT8531 Gigabit Ethernet 驱动。该 PHY 驱动工作在中断模式 (irq=POLL)，表示设备支持轮询方式的中断处理。

stm32-dwmac 482d0000.eth2 end0: No Safety Features support found

当前以太网设备未发现支持的安全功能。

stm32-dwmac 482d0000.eth2 end0: IEEE 1588-2008 Advanced Timestamp supported

stm32-dwmac 482d0000.eth2 end0: registered PTP clock

支持 IEEE 1588-2008 标准的高级时间戳功能，用于高精度时间同步。注册了 PTP 时钟设备，这对工业自动化或时间敏感型网络 (如 TSN) 的应用非常重要。

stm32-dwmac 482d0000.eth2 end0: FPE workqueue start

启动了帧抢占 (Frame Preemption) 功能的工作队列。FPE 是 TSN 的一部分，用于支持低优先级帧的中断发送，从而确保高优先级帧快速传输。

stm32-dwmac 482d0000.eth2 end0: configuring for phy/rgmii-id link mode

PHY 配置为 RGMII-ID (RGMII with Internal Delay) 模式，表示物理接口支持带内延时的 RGMII。

stm32-dwmac 482d0000.eth2 end0: Link is Up - 1Gbps/Full - flow control rx/tx

链路状态正常，连接速率为 1Gbps，全双工模式，并启用了流量控制 (接收和发送均支持)。表明网络接口已成功初始化，并准备好进行数据通信。

21.8.2 ifconfig

ifconfig (interface configuration) 指令是用于配置网络接口和查看网络接口状态的工具。它通常用于显示或修改网络接口的配置参数，例如 IP 地址、子网掩码、广播地址等。

开发板网口 ETH2 通过千兆网线接到千兆路由器接口后，会自动获取 IP 地址，此时执行 ifconfig 指令可以看到对应的网络接口信息，参考如下：

ifconfig

```
root@ATK-DLMP257:~#
root@ATK-DLMP257:~#
root@ATK-DLMP257:~# ifconfig
br0    Link encap:Ethernet  HWaddr 42:19:56:3A:F6:15
        inet6 addr: fe80::4019:56ff:fe3a:f615/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:0  errors:0  dropped:0  overruns:0  frame:0
        TX packets:17  errors:0  dropped:0  overruns:0  carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:0 (0.0 B)  TX bytes:1984 (1.9 KiB)

end0    Link encap:Ethernet  HWaddr B2:DB:3F:7A:F5:9E
        inet addr:192.168.6.170  Bcast:192.168.6.255  Mask:255.255.255.0
        inet6 addr: fe80::b0db:3fff:fe7a:f59e/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:40643  errors:0  dropped:0  overruns:0  frame:0
        TX packets:410  errors:0  dropped:0  overruns:0  carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:2824500 (2.6 MiB)  TX bytes:85471 (83.4 KiB)
        Interrupt:68
```

以下省略

图 21.8-2 ifconfig 信息

- flags: 接口的当前状态，常见的标志有 UP (接口启用)、BROADCAST (支持广播)、RUNNING (接口正在运行)、MULTICAST (支持多播) 等。
- mtu: 最大传输单元，表示一次可以传输的最大数据包大小。
- inet: 接口的 IPv4 地址。
- netmask: 子网掩码，用于确定网络部分和主机部分。



- broadcast: 广播地址，用于发送广播数据包。
- inet6: 接口的 IPv6 地址。
- ether: 接口的 MAC 地址（物理地址）。
- txqueuelen: 传输队列长度。
- RX packets/bytes: 接收的数据包数量和字节数。
- RX errors: 接收过程中出现的错误数量。
- TX packets/bytes: 发送的数据包数量和字节数。
- TX errors: 发送过程中出现的错误数量。
- device interrupt: 设备中断号。

查看 ifconfig 指令的使用方法:

```
ifconfig --help
```

```
root@ATK-DLMP257:~# ifconfig --help
BusyBox v1.36.1 () multi-call binary.

Usage: ifconfig [-a] [IFACE] [ADDRESS]
root@ATK-DLMP257:~#
```

图 21.8-3 ifconfig 使用说明

基本用法

```
ifconfig [-a] [IFACE] [ADDRESS]
```

- -a: 显示所有接口（包括那些没有分配地址的接口）。如果没有提供 IFACE，此选项会列出系统中所有接口。
- [IFACE]: 指定要操作的网络接口名称（例如 end0 或 lo）。如果未指定，则显示所有已启用接口的信息。
- [ADDRESS]: 设置指定网络接口的 IPv4 地址。

示例:

1、查看所有接口的配置

```
ifconfig -a
```

2、配置 IP 地址

```
ifconfig end0 192.168.1.100
```

3、启动和关闭网络接口

```
ifconfig end0 up
ifconfig end0 down
```

21.8.3 udhcp

udhcp 是一个用于嵌入式系统的非常小的 DHCP 客户端。它的主要功能是向 DHCP 服务器请求网络配置参数，如 IP 地址、子网掩码和网关等。在使用指令之前，确保开发板网口网线连接到 DHCP 服务器，比如路由器接口。

查看 udhcp 使用方法:

```
udhcp -h
```

```

root@ATK-DLMP257:~# udhcpc -h
udhcpc: invalid option -- 'h'
BusyBox v1.36.1 () multi-call binary.

Usage: udhcpc [-fbqRB] [-a[MSEC]] [-t N] [-T SEC] [-A SEC|-n]
        [-i IFACE] [-s PROG] [-p PIDFILE]
        [-oC] [-r IP] [-V VENDOR] [-F NAME] [-x OPT:VAL]... [-O OPT]...
root@ATK-DLMP257:~#
    
```

图 21.8-4 udhcp 使用说明

基本用法:

udhcpc [选项]

选项解释

1、通用选项

- -i IFACE

指定要请求 DHCP 的网络接口（如 end0）。

示例: `udhcpc -i end0`

- -s PROG

指定脚本 PROG，用于处理 DHCP 的事件（如租约获取、续订或释放）。

默认脚本通常位于 /usr/share/udhcpc/default.script。

- -p PIDFILE

指定 PID 文件路径，用于保存 udhcpc 的进程 ID。

示例: `udhcpc -p /var/run/udhcpc.pid`

- -r IP

请求特定的 IP 地址。

示例: `udhcpc -i end0 -r 192.168.1.100`

- -V VENDOR

设置 Vendor Class Identifier。

- -F NAME

设置主机名，用于 DHCP 请求。

- -x OPT:VAL

发送特定的 DHCP 选项和值。

示例: `udhcpc -x hostname:mydevice`

- -O OPT

请求特定的 DHCP 选项。

示例: `-O dns`（请求 DNS 服务器）。

2、行为控制选项

- -f

前台运行（不作为后台进程）。

示例: 调试时非常有用: `udhcpc -i end0 -f`

- -q

安静模式，尽量减少日志输出。

- -R

禁用默认的路由设置。

- -B

后台运行。



- -o

如果获取不到租约，也退出而不运行默认脚本。

3、重试和超时相关

- -t N

设置尝试获取 DHCP 的次数（默认：3）。

示例: `udhcpc -i end0 -t 5`

- -T SEC

设置初次重试超时（单位：秒，默认：3 秒）。

示例: `udhcpc -i end0 -T 10`

- -A SEC

设置选择超时时间（单位：秒）。

示例: `udhcpc -i end0 -A 15`

- -n

如果无法获取租约，则直接退出。

- -a [MSEC]

自动更新租约，指定时间间隔（以毫秒为单位）。

4、调试相关

- -C

禁用调用默认脚本。

示例：获取 IP 地址。

`udhcpc -i end0`

```
root@ATK-DLMP257:~# udhcpc -i end0
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'end0.udhcpc'. Using 'end0' (ifindex=3).
udhcpc: broadcasting discover
udhcpc: broadcasting select for 192.168.6.170, server 192.168.6.1
udhcpc: lease of 192.168.6.170 obtained from 192.168.6.1, lease time 86400
/etc/udhcpc.d/50default: Adding DNS 192.168.6.1
Dropped protocol specifier '.udhcpc' from 'end0.udhcpc'. Using 'end0' (ifindex=3).
root@ATK-DLMP257:~#
```

图 21.8-5 udhcpc 获取 IP 地址

从输出信息可以看到，udhcpc 成功获取了一个 DHCP 租约。以下是输出的详细解释：

`udhcpc: started, v1.36.1`

这表示 udhcpc 客户端启动，使用的是 v1.36.1 版本。

`Dropped protocol specifier '.udhcpc' from 'end0.udhcpc'. Using 'end0' (ifindex=3).`

这行表示 udhcpc 从接口名称中删除了协议指定符，使用了 'end0' 接口，ifindex 表示接口索引。

`udhcpc: broadcasting discover`

这表示 udhcpc 正在广播 DHCP DISCOVER 包，以请求 DHCP 服务器分配一个 IP 地址。

`udhcpc: broadcasting select for 192.168.6.170, server 192.168.6.1`

这表示发送 DHCP REQUEST，udhcpc 选择了 192.168.6.170 作为请求的 IP 地址，并向 192.168.6.1 服务器发送了 DHCP REQUEST 包。

`udhcpc: lease of 192.168.6.170 obtained from 192.168.6.1, lease time 86400`

这表示成功获取租约, 从 192.168.6.1 服务器成功获取了 192.168.6.170 的租约, 租约时间为 86400 秒 (24 小时)。

```
/etc/udhcpd.d/50default: Adding DNS 192.168.6.1
```

这表示更新 DNS 配置, 这是 udhcpd 客户端脚本的一部分, 它将 192.168.6.1 添加到 DNS 配置中。

以上信息表示系统成功使用 udhcpd 客户端通过 DHCP 服务器获取到了 IP 地址 192.168.6.170, 并更新了 DNS 设置。如果需要进一步配置, 可以根据需求调整 DHCP 客户端的脚本或选项。

21.8.4 ip

ip 指令是一个强大的网络配置和管理工具, 提供了比 ifconfig 更多的功能。它是现代 Linux 系统中管理网络接口、路由、隧道、邻居对象等的推荐工具。

1、查看所有网络接口的状态

```
ip addr show
```

或者简写为 ip a

```
root@ATK-DLMP257:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_ll
        valid_lft forever preferred_lft forever
2: end1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 36:49:c2:37:9b:55 brd ff:ff:ff:ff:ff:ff
    altnam ethernet1
    altnam ethernet1.eth1
    inet6 fe80::3449:c2ff:fe37:9b55/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: end0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether b2:db:3f:7a:f5:9e brd ff:ff:ff:ff:ff:ff
    altnam ethernet0
    altnam ethernet0.eth2
    inet 192.168.6.170/24 brd 192.168.6.255 scope global end0
        valid_lft forever preferred_lft forever
    inet6 fe80::b0db:3fff:fe7a:f59e/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
4: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
5: can1: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
6: can2: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
7: wlan0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether f0:a8:82:c1:ff:98 brd ff:ff:ff:ff:ff:ff
8: wlan1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether f2:a8:82:c1:ff:98 brd ff:ff:ff:ff:ff:ff
9: sw0p1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master br0 state UNKNOWN group default
    qlen 1000
    link/ether 1e:e5:30:9f:3d:00 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::1ce5:30ff:fe9f:3d00/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
10: sw0p2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master br0 state DOWN group default
    qlen 1000
    link/ether 1e:e5:30:9f:3d:01 brd ff:ff:ff:ff:ff:ff
11: sw0p3: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel master br0 state DOWN group default
    qlen 1000
    link/ether 1e:e5:30:9f:3d:02 brd ff:ff:ff:ff:ff:ff
12: sw0ep: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 36:49:c2:37:9b:55 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.10/32 scope global sw0ep
        valid_lft forever preferred_lft forever
    inet6 fe80::3449:c2ff:fe37:9b55/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
13: br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 42:19:56:3a:f6:15 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::4019:56ff:fe3a:f615/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

图 21.8-6 ip addr show 指令

**网络接口信息解析:**

活跃接口 (已启用, 处于 "UP" 状态):

- lo (回环接口):

IPv4: 127.0.0.1/8

IPv6: ::1/128

- end0 (以太网接口):

MAC 地址: b2:db:3f:7a:f5:9e

IPv4: 192.168.6.170/24

子网掩码: 255.255.255.0

广播地址: 192.168.6.255

IPv6: 链路本地地址: fe80::b0db:3fff:fe7a:f59e/64

- end1 (以太网接口):

MAC 地址: 36:49:c2:37:9b:55

IPv6: 链路本地地址: fe80::3449:c2ff:fe37:9b55/64

- sw0p1 (交换机端口 1):

IPv6: 链路本地地址: fe80::1ce5:30ff:fe9f:3d00/64

- sw0ep (交换机嵌入式端口):

MAC 地址: 36:49:c2:37:9b:55

IPv4: 192.168.0.10/32

(注意 /32 表示子网掩码为 255.255.255.255, 仅指定一个 IP)

IPv6: 链路本地地址: fe80::3449:c2ff:fe37:9b55/64

- br0 (桥接接口):

MAC 地址: 42:19:56:3a:f6:15

IPv6: 链路本地地址: fe80::4019:56ff:fe3a:f615/64

未启用接口 (处于 "DOWN" 状态):

- wlan0 和 wlan1 (Wi-Fi 接口):

MAC 地址:

wlan0: f0:a8:82:c1:ff:98

wlan1: f2:a8:82:c1:ff:98

- can0、can1 和 can2 (CAN 总线接口):

均未启用。

- sw0p2 和 sw0p3 (交换机端口 2 和 3):

没有物理连接 (状态 NO-CARRIER)。

2、查看特定网络接口的状态

示例: 查看 end0 接口的状态:

```
ip addr show dev end0
```



```

root@ATK-DLMP257:~# ip addr show dev end0
3: end0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether b2:db:3f:7a:f5:9e brd ff:ff:ff:ff:ff:ff
    altname ethernet0
    altname ethernet0.eth2
    inet 192.168.6.170/24 brd 192.168.6.255 scope global end0
        valid_lft forever preferred_lft forever
    inet6 fe80::b0db:3fff:fe7a:f59e/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@ATK-DLMP257:~#

```

图 21.8-7 查看 end0 接口的状态

3、配置网络接口的 IP 地址

通过 ip addr add 指令可以设置开发板的 IP 地址，格式为：

ip addr add IP 地址/子网掩码 dev 设备接口

示例：给 end0 接口配置 IP 地址（192.168.6.115，根据自己的实际网段设置），子网掩码为 24（即 255.255.255.0，根据自己的实际网段设置）

ip addr add 192.168.6.115/24 dev end0

ip addr show dev end0

```

root@ATK-DLMP257:~# ip addr add 192.168.6.115/24 dev end0
root@ATK-DLMP257:~# ip addr show dev end0
3: end0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether b2:db:3f:7a:f5:9e brd ff:ff:ff:ff:ff:ff
    altname ethernet0
    altname ethernet0.eth2
    inet 192.168.6.170/24 brd 192.168.6.255 scope global end0
        valid_lft forever preferred_lft forever
    inet 192.168.6.115/24 scope global secondary end0
        valid_lft forever preferred_lft forever
    inet6 fe80::b0db:3fff:fe7a:f59e/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@ATK-DLMP257:~#

```

图 21.8-8 ip addr add 添加 IP 地址

笔者的 end0 接路由器默认获取了一个 IP 地址，使用 ip addr add 指令手动配置了一个别的 IP 地址。此外，默认还配置了一个链路本地的 IPv6 地址，这使得 end0 接口可以在同一局域网内使用两个不同的 IPv4 地址进行通信。一般情况下一个网口对应一个 IP 地址，我们使用一个网口即可。

4、删除网络接口的 IP 地址

通过 ip addr del 指令可以删除开发板的 IP 地址，格式为：

ip addr del IP 地址/子网掩码 dev 设备接口

示例：删除 end0 接口配置 IP 地址（192.168.6.115，根据自己的实际网段设置），子网掩码为 24（即 255.255.255.0，根据自己的实际网段设置），即删除我们上步设置的 IP 地址，这里笔者顺便也删除掉自动获取的 IP 地址。

ip addr del 192.168.6.115/24 dev end0

ip addr show dev end0

```

root@ATK-DLMP257:~# ip addr del 192.168.6.115/24 dev end0
root@ATK-DLMP257:~# ip addr show dev end0
3: end0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether b2:db:3f:7a:f5:9e brd ff:ff:ff:ff:ff:ff
    altname ethernet0
    altname ethernet0.eth2
    inet 192.168.6.170/24 brd 192.168.6.255 scope global end0
        valid_lft forever preferred_lft forever
    inet6 fe80::b0db:3fff:fe7a:f59e/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@ATK-DLMP257:~#

```

图 21.8-9 ip addr show

5、禁用或启用网络接口



禁用网络接口:

```
ip link set end0 down
```

启用网络接口:

```
ip link set end0 up
```

6、查看路由表

ip route 是路由表相关配置指令, show 参数用于打印路由表信息。

```
ip route show
```

或者简写为 ip r

```
root@ATK-DLMP257:~# ip route show
default via 192.168.6.1 dev end0 proto dhcp src 192.168.6.170 metric 10
192.168.0.0/24 dev sw0ep scope link
192.168.6.0/24 dev end0 proto kernel scope link src 192.168.6.170 metric 10
192.168.6.1 dev end0 proto dhcp scope link src 192.168.6.170 metric 10
root@ATK-DLMP257:~#
```

图 21.8-10 查看路由表

路由表解析:

默认路由 (default):

- 网关: 192.168.6.1
- 出接口: end0
- 源 IP: 192.168.6.170
- 路由协议: dhcp (通过 DHCP 获取)
- 路由优先级 (metric): 10

解释: 所有非本地网络的流量 (例如访问互联网) 将通过接口 end0 发送到网关 192.168.6.1

网络 192.168.0.0/24:

- 出接口: sw0ep
- 作用域: link (链路本地, 仅适用于该接口上的直接通信)

解释: 该路由表示网络 192.168.0.0/24 的设备可以通过 sw0ep 接口直接访问 (无需网关)。

网络 192.168.6.0/24:

- 出接口: end0
- 路由协议: kernel (由内核自动添加)
- 源 IP: 192.168.6.170
- 路由优先级: 10

解释: 该路由表示网络 192.168.6.0/24 的设备可以通过 end0 接口直接访问。

网关 192.168.6.1:

- 出接口: end0
- 路由协议: dhcp (通过 DHCP 获取)
- 源 IP: 192.168.6.170
- 路由优先级: 10

解释: 这是一个专门用于访问网关 192.168.6.1 的路由, 确保设备可以直接与网关通信。

7、删除默认网关

由于笔者使用的是路由器网线连接开发板, 默认 DHCP 会重新添加默认路由。这时, 可以尝试以下命令尝试删除所有的默认路由:

```
ip route flush dev end0
```

```
ip route show
```

```
root@ATK-DLMP257:~# ip route flush dev end0
root@ATK-DLMP257:~# ip route show
192.168.0.0/24 dev sw0ep scope link
root@ATK-DLMP257:~#
```

图 21.8-11 强制删除 end0 所有的默认路由

如果不是 DHCP 分配的网络, 是手动添加的静态 IP 地址网络, 则可以使用 `ip route del default` 指令来删除对应的网关, 参考示例如下:

```
ip route del default via 192.168.6.1 dev end0
ip route del default via 192.168.6.1 dev end0 proto dhcp src 192.168.6.198 metric 10
```

8、添加默认网关

在手动添加的静态 IP 地址网络中使用 `ip route add default` 指令可以添加默认网关, 示例如下:

```
ip route add default via 192.168.6.1
```

9、启用网络

`ifconfig` 可以启用网络, `ip` 指令也可以, 启用网络的指令参考如下:

```
ip link set end0 up
```

第二十二章 Camera 子系统

22.1 Camera 子系统简介

嵌入式 Linux 的 Camera 子系统是用于管理和控制摄像头设备的框架。它通常用于嵌入式系统中，如智能手机、平板电脑、智能家居设备和工业自动化系统。这个子系统的主要功能包括摄像头驱动程序的开发和集成、图像数据的捕获和处理，以及与应用程序之间的数据传输。

22.2 组成部分

1、Video4Linux (V4L2):

V4L2 是一个用于捕获、解码和处理视频的框架。它提供了标准的 API，用于与摄像头硬件进行交互。V4L2 支持多种视频格式和功能，包括视频流的捕获、处理和编码。

2、摄像头驱动程序:

驱动程序是嵌入式系统中不可或缺的一部分，用于控制硬件设备。摄像头驱动程序负责与实际的摄像头硬件进行通信，控制摄像头的各项参数，如分辨率、帧率和曝光时间。

3、Media Controller:

Media Controller 是用于处理复杂多媒体设备的子系统。它可以管理多个摄像头、编码器、解码器等设备，并协调它们之间的交互。

4、用户空间工具和库:

用户空间工具和库，例如 libv4l2，提供了更高级别的 API，简化了应用程序开发。应用程序可以通过这些工具和库访问摄像头数据，而不需要直接与内核驱动程序交互。

22.3 MP257 的 Camera 说明

22.3.1 文档和资源

想要获取 MP257 系列处理器的 Camera 详细说明，最直接的方式是参考 ST 官方文档，包括：

1、参考手册（Reference Manual）：此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南（即本文档）

应用开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考: 【正点原子】STM32MP257 开发板 (A 盘)-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

22.3.2 特性说明

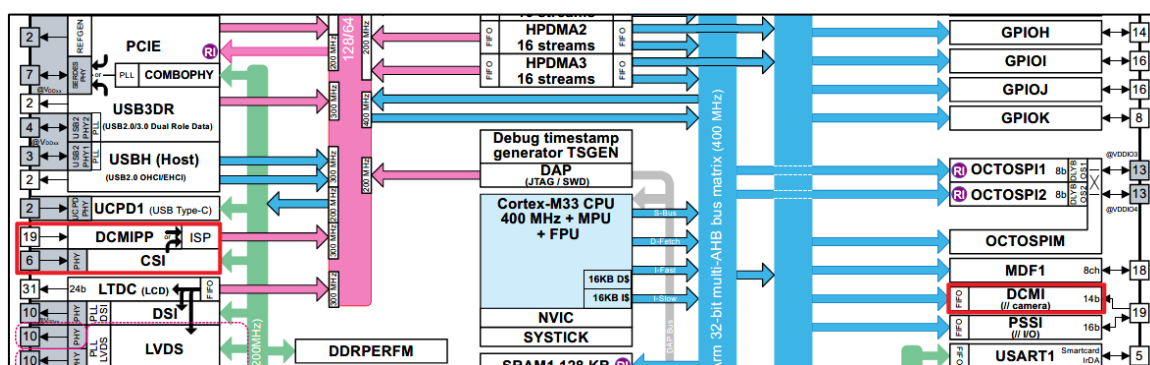


图 22.3-1 MP257 的 Camera 部分

MP257 的 camera 有 DCMI 和 DCMIPP 两种，相关说明如下：

DCMIPP (数字相机接口与像素处理)

输入接口

- 并行输入接口：

支持 16 位 @ 120 MHz，最高支持 2 百万像素传感器 @ 30 fps。

支持像素格式：RGB565、RGB888、YUV422、RawBayer/Mono (8/10/12/14 位)。

- CSI-2 输入接口：

支持高达 200 Mpixel/s，最高支持 5 百万像素传感器 @ 30 fps，兼容 MIPI CSI-2 v1.3 标准。

像素格式：MIPI CSI-2 v1.3 标准的所有格式 (RGB565、RGB888、YUV422、RawBayer)。

功能：支持交错包 (Interleaved packets) 和 4 个虚拟通道。

图像处理特性

- 数据流选择和帧控制
- 字节到像素的转换
- 统计信息移除
- 坏点修复：自动检测并修正传感器阵列中的坏点。
- 像素抽取 (Decimation)：每 1/2/4/8 像素采样一次。
- 集成图像处理：适配低成本摄像头模块 (无嵌入式 ISP)，包括：
 - 色彩转换以适配传感器并调节照明。
 - 对比度增强。
 - RawBayer 到 RGB 的转换 (去马赛克处理)。
 - 曝光控制。
 - 统计信息提取。



- 管道 0 的抽取支持 (pipe dump): 对数据进行简单转储, 无处理。

并行处理管道

- Pipe0 (数据转储): 用于直接数据转储, 无需处理。
- Pipe1 (主用管道): 支持降采样、色彩转换、解码、多平面和图像处理。
- Pipe2 (辅助管道): 支持降采样、色彩转换、解码和共平面处理。

图像缩小 (降采样)

- 采用盒式滤波 (Box-filtering), 支持任意小数比率, 最大支持 8x8 降采样 (Pipe1 和 Pipe2 支持)。

图像处理功能

- 伽马转换 (Gamma conversion)
- RGB 到 YUV 的色彩转换

输出像素格式

- Pipe0: 支持所有原始数据输出, 包括 Y/Rb (8/10/12/14 位统计数据) 和比特流。
- Pipe2: 支持 RGB888、RGB565、YUV422-1、Y8、ARGB 和 RGBA (仅限共平面)。
- Pipe1: 支持 Pipe2 格式, 额外支持 YUV422-2、YUV420-2 和 YUV420-3 (支持多平面)。

数据总线

- AXI Master 接口: 提供高速数据传输支持。

DCMIPP (数字相机接口与像素处理模块)

输入接口

并行接口:

- 支持 16 位数据, 最高 120 MHz 频率。
- 可支持 2 百万像素传感器 @30 帧/秒。
- 像素格式: RGB565、RGB888、YUV422、RawBayer/Mono (8/10/12/14 位)。

CSI-2 接口:

- 支持 200 Mpixels/s, 最高支持 5 百万像素传感器 @30 帧/秒。
- 支持 MIPI CSI-2 v1.3 标准, 像素格式包括: RGB565、RGB888、YUV422、RawBayer。
- 功能特性: 支持交错数据包 (Interleaved packets) 和 4 个虚拟通道。

图像处理特性

- 数据流选择与帧控制。
- 字节到像素的格式转换。
- 统计信息移除: 移除冗余统计数据。
- 坏点修复: 自动检测并修复传感器阵列中的坏像素。
- 像素抽取 (Decimation): 支持 1/2/4/8 的像素采样。
- 集成图像处理功能, 适配低成本摄像头模块 (无嵌入式 ISP):
- 色彩转换, 适配传感器及调整光照。
- 对比度增强。
- RawBayer 到 RGB 的转换 (去马赛克)。
- 曝光控制。
- 提取统计数据。
- 管道 0 像素抽取支持: 直接转储数据, 无需处理。

多管道并行处理



- Pipe0（数据转储）：用于直接输出原始数据，不进行图像处理。
- Pipe1（主处理管道）：支持降采样、色彩转换、解码、多平面输出和图像处理。
- Pipe2（辅助处理管道）：支持降采样、色彩转换、解码和共平面输出。

图像降采样（Downsize）

- 使用盒式滤波（Box-filtering）。
- 支持任意小数比率，最大支持 8x8 降采样（Pipe1 和 Pipe2）。

高级图像处理功能

- 伽马校正（Gamma Conversion）。
- RGB 到 YUV 的色彩空间转换。

输出像素格式

- Pipe0：原始数据（如 Y/Rb 统计数据：8/10/12/14 位）或比特流。
- Pipe2：支持 RGB888、RGB565、YUV422-1、Y8、ARGB 和 RGBA（仅共平面模式）。
- Pipe1：包括 Pipe2 支持的格式，以及 YUV422-2、YUV420-2 和 YUV420-3（支持多平面模式）。

数据传输接口

- 支持 AXI 主机接口，用于高效数据传输。

更多关于 MP257 的 Camera 资料请查看芯片参考手册。

22.4 开发板 Camera 接口

22.4.1 原理图分析

开发板上有一个 MIPI CSI 接口，我们后续也主要围绕此接口讲解（也就是 DCMIPP）。打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 CSI。可以看到相关电路如下。

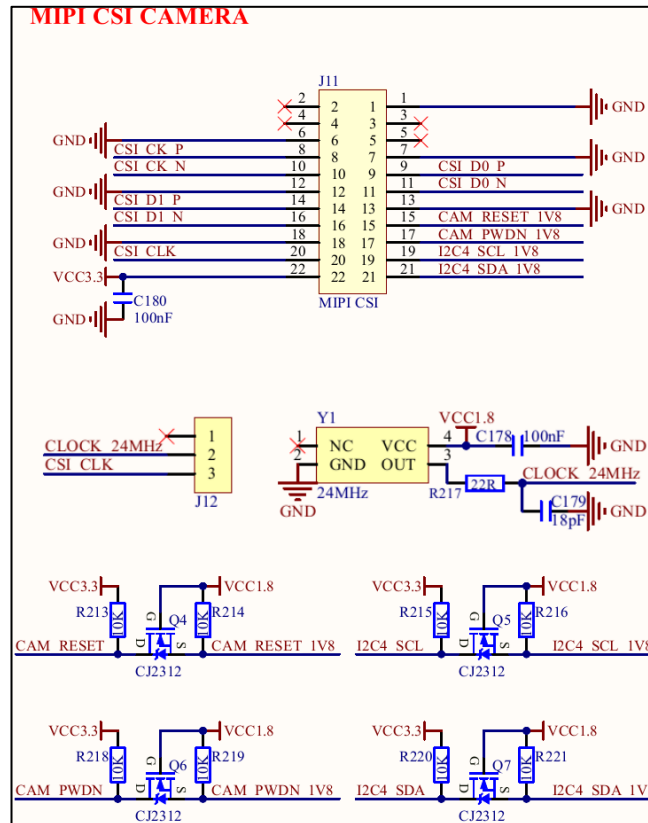


图 22.4-1 开发板 MIPI CSI 电路

这张原理图展示了一个 MIPI-CSI（摄像头串行接口）摄像头模块的连接。以下是关键组件及其功能的解析：

MIPI CSI 接口部分

数据信号：

- CSI_D0_P / CSI_D0_N：MIPI CSI 通道 0 的数据正负差分信号。
- CSI_D1_P / CSI_D1_N：MIPI CSI 通道 1 的数据正负差分信号。

时钟信号：

- CSI_CLK_P / CSI_CLK_N：MIPI CSI 接口的时钟正负差分信号。

供电：

- VCC3.3 为电源供电，旁路电容 C180（100nF）用于电源滤波。

地：

- 多个 GND 引脚提供接地连接，确保信号完整性。

时钟电路部分

24MHz 时钟晶振（Y1）：

- 提供 24MHz 时钟信号输出给摄像头模块。
- 晶振 Y1 通过 J12 接口的 CSI_CLK 引脚连接到摄像头。
- 旁路电容 C178（100nF）用于电源去耦，C179（18pF）用于稳定晶振信号。
- 电阻 R217（22Ω）用于匹配时钟信号的阻抗。

摄像头控制信号

复位信号（CAM_RESET）：

- 使用 MOS 管 Q4 完成从 3.3V 到 1.8V 的电平转换。
- 电阻 R213 和 R214 分别用于上拉和电流限制。

掉电控制信号 (CAM_PWDN) :

- 使用 MOS 管 Q6 完成从 3.3V 到 1.8V 的电平转换。
- 电阻 R218 和 R219 分别用于上拉和电流限制。

I2C 接口

信号:

- I2C4_SCL (时钟信号) 通过 MOS 管 Q5 完成从 3.3V 到 1.8V 的电平转换。
- I2C4_SDA (数据信号) 通过 MOS 管 Q7 完成从 3.3V 到 1.8V 的电平转换。

元件:

- 电阻 R215 和 R216 (I2C 时钟线), R220 和 R221 (I2C 数据线) 用于上拉。

用途:

- 用于摄像头模块的初始化和寄存器配置。

处理器 MP257 支持 2-lane MIPI CSI-2 摄像头输入信号, 包含 2 对 RX 数据线和 1 对 RX 时钟线。此接口已适配支持正点原子 ATK-MCIMX335 摄像头模块, 其中 CSI_RST_1V8、CSI_PWDN_1V8、I2C4_SCL_1V8 和 I2C4_SDA_1V8 四根信号引脚均为 1.8V IO, 如果用户设计底板选用 3.3V IO, 则需要进行电平转换, 原理图中已给出 3.3V 转 1.8V 示例。

22.4.2 硬件实物

开发板上的 MIPI CSI 接口位置如下图所示:

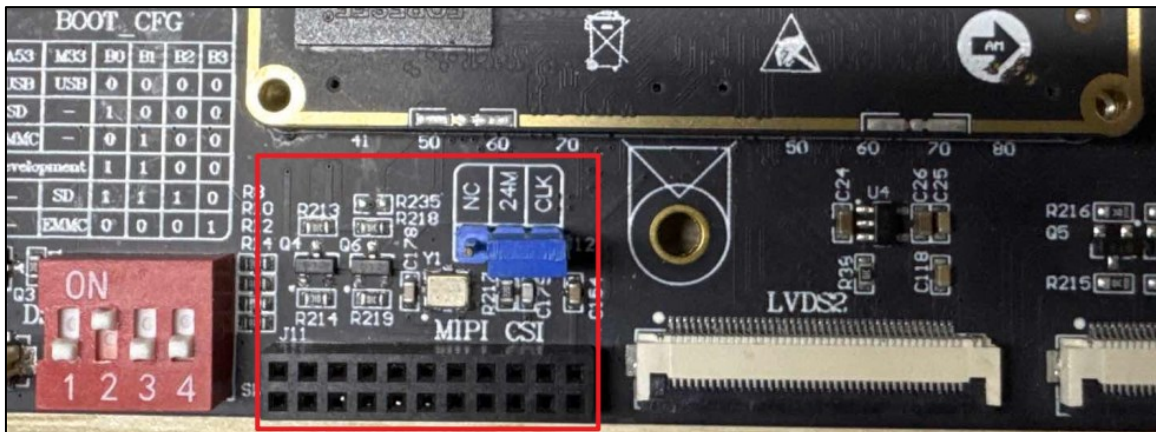


图 22.4-2 开发板 MIPI CSI 接口

此接口兼容正点原子 IMX335 模块 (型号 ATK-MCIMX335), 支持 500 万像素。



图 22.4-3 正点原子 IMX335 摄像头

接法示意: 摄像头镜头往开发板外侧, 直接插到 MIPI CSI 插口处。插摄像头时需要注意看底板丝印, 按引脚编号对应插上, 即镜头朝向板子外侧。24M 跳线帽接到 CLK 端, 连接完成后如下图所示:

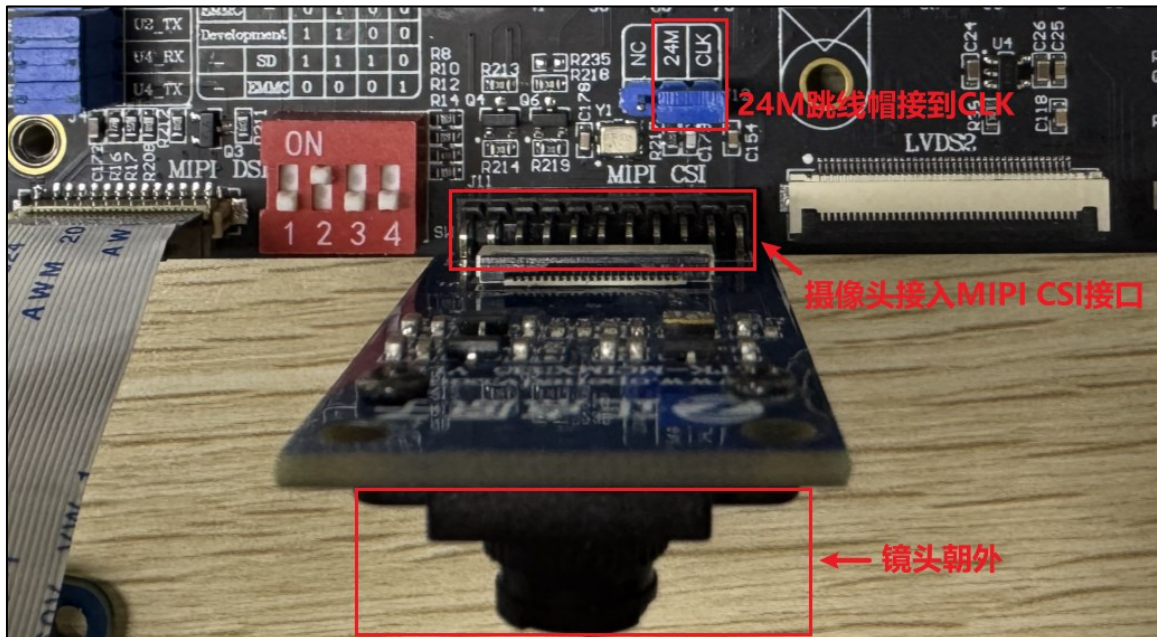


图 22.4-4 IMX335 连接开发板

默认摄像头镜头有个保护膜贴纸, 如果影响了使用可以撕下贴纸。

22.5 内核 Camera 相关使能

22.5.1 DCMIPP 驱动使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, DCMIPP 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_MEDIA_SUPPORT=m //启用内核媒体子系统
2 CONFIG_MEDIA_PLATFORM_SUPPORT=y //启用平台设备的媒体支持
3 CONFIG_MEDIA_PLATFORM_DRIVERS=y //启用媒体平台驱动支持
4 CONFIG_V4L_PLATFORM_DRIVERS=y //启用 Video4Linux (V4L) 平台驱动
5 CONFIG_VIDEO_DEV=m //启用视频设备支持
6 CONFIG_ARCH_STM32=y //指定目标平台为 STM32 系列
7 CONFIG_VIDEO_STM32_DCMIPP=m //启用 STM32 DCMIPP 驱动
```

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

```
Device Drivers --->
  <M> Multimedia support --->
    Media drivers --->
      [*] Media platform devices --->
        <M> STM32 Digital Camera Memory Interface Pixel Processor (DCMIPP) support
```

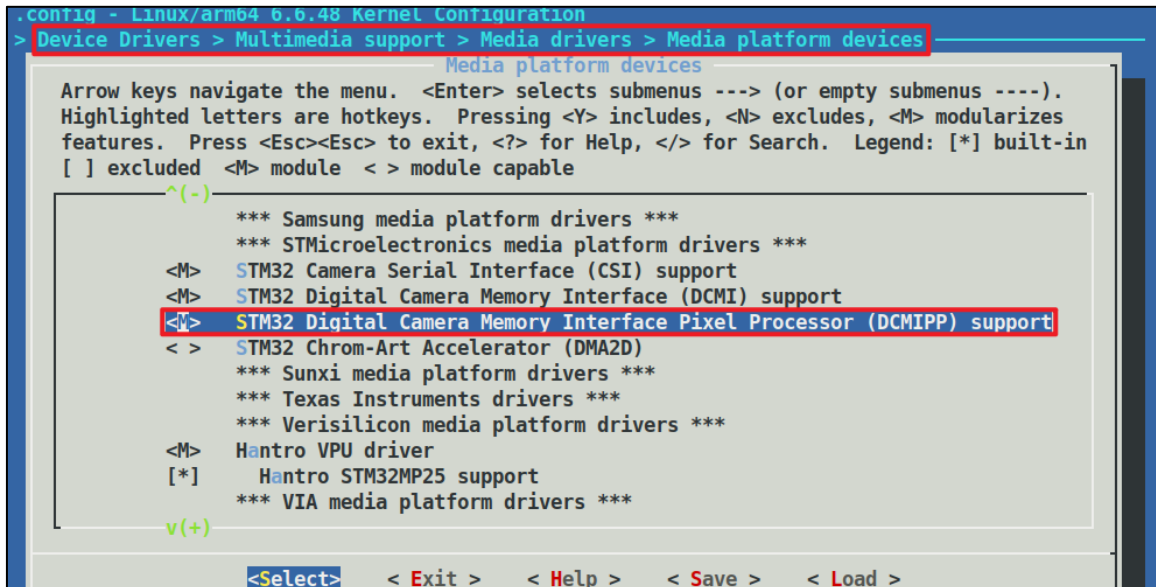


图 22.5-1 DCMIPP 配置节选

22.5.2 IMX335 驱动使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, IMX335 相关配置如下(略有删减, 只列举重点部分):

- 1 CONFIG_MEDIA_SUPPORT=m //启用 Linux 内核媒体子系统
- 2 CONFIG_VIDEO_DEV=m //启用视频设备支持
- 3 CONFIG_VIDEO_CAMERA_SENSOR=y //启用摄像头传感器
- 4 CONFIG_OF_GPIO=y //启用设备树 (Device Tree) 中的 GPIO 支持
- 5 CONFIG_VIDEO_IMX335=m //启用 IMX335 摄像头传感器的驱动

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

```
Device Drivers --->
  <M> Multimedia support --->
    Media ancillary drivers --->
      [*] Camera sensor devices --->
        <M> Sony IMX335 sensor support
```

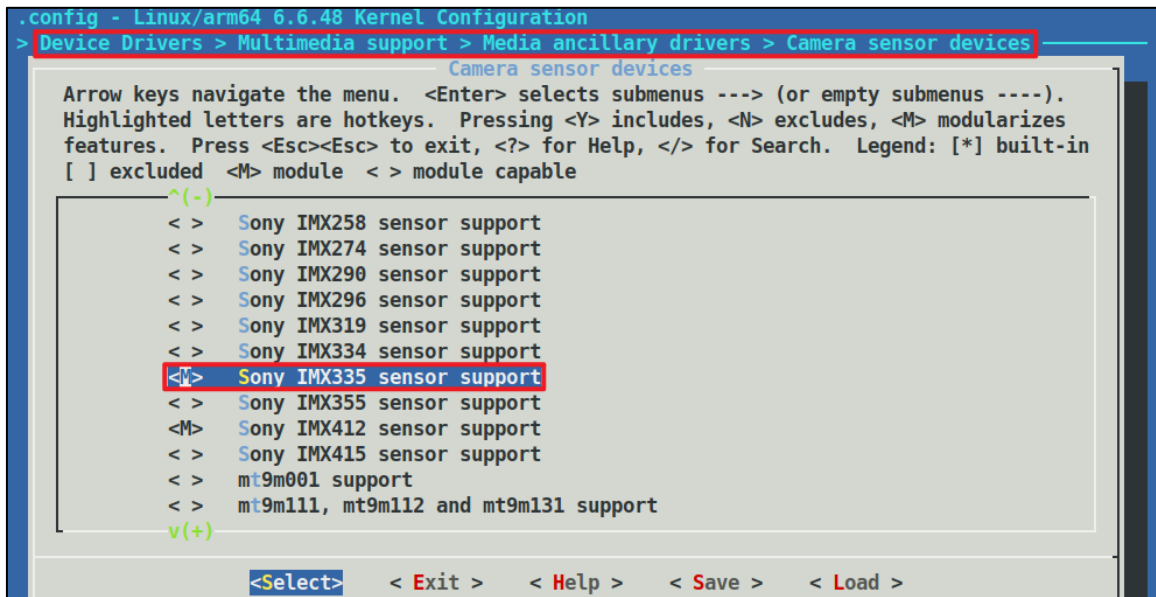



图 22.5-2 IMX335 驱动配置

22.6 设备树分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi，此文件是特定于 STM32MP2 系列芯片的设备树包含文件，通常不独立使用，而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置，而 .dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 csi 和 dcmipp 描述如下：

```

1  csi: csi@48020000 {
2      compatible = "st,stm32mp25-csi";
3      reg = <0x48020000 0x2000>;
4      interrupts = <GIC_SPI 142 IRQ_TYPE_LEVEL_HIGH>;
5      resets = <&rcc CSI_R>;
6      clocks = <&rcc CK_KER_CSI>, <&rcc CK_KER_CSITXESC>,
7              <&rcc CK_KER_CSIPHY>;
8      clock-names = "pclk", "txesc", "csi2phy";
9      access-controllers = <&rifsc 86>;
10     status = "disabled";
11 };
12
13 dcmipp: dcmipp@48030000 {
14     compatible = "st,stm32mp25-dcmipp";
15     reg = <0x48030000 0x1000>;
16     interrupts = <GIC_SPI 198 IRQ_TYPE_LEVEL_HIGH>;
17     resets = <&rcc DCMIPP_R>;
18     clocks = <&rcc CK_BUS_DCMIPP>, <&rcc CK_KER_CSI>;
19     clock-names = "kclk", "mclk";
20     access-controllers = <&rifsc 87>;
21     power-domains = <&CLUSTER_PD>;

```



```
22     status = "disabled";
23 };
```

设备树解析:

CSI 节点:

第 1 行, 定义设备节点, 节点名称为 `csi`, 地址为 `csi@48020000`。这是一个支持 MIPI CSI-2 接口的硬件模块。

第 2 行, `compatible = "st,stm32mp25-csi"` 兼容字符串, 表明该硬件模块的驱动程序应识别为 `stm32mp25-csi`。

第 3 行, `reg = <0x48020000 0x2000>` 指定设备的内存地址和大小。此设备的基地址为 `0x48020000`, 大小为 `0x2000` 字节。

第 4 行, `interrupts = <GIC_SPI 142 IRQ_TYPE_LEVEL_HIGH>` 定义该设备使用的中断号。使用全局中断控制器 (GIC) 的 SPI 编号 142, 类型为高电平触发。

第 5 行, `resets = <&rcc CSI_R>` 定义与复位控制器 (rcc) 的关联, 复位信号为 `CSI_R`。

第 6~8 行, 定义设备使用的时钟:

- `<&rcc CK_KER_CSI>`: 内核时钟 (`pclk`)。
- `<&rcc CK_KER_CSITXESC>`: 传输时钟 (`txesc`)。
- `<&rcc CK_KER_CSIPHY>`: MIPI PHY 时钟 (`csi2phy`)。

第 9 行, `clock-names = "pclk", "txesc", "csi2phy"` 为每个时钟定义名称, 便于驱动程序使用。

第 10 行, `access-controllers = <&rifsc 86>` 定义访问控制器, 标号为 86。

第 11 行, `status = "disabled"` 指定设备状态为禁用, 需启用时修改为 `"okay"`。

`csi` 节点: 定义了 MIPI CSI-2 接口硬件, 处理摄像头数据输入。

DCMIPP 节点:

第 13 行, 定义设备节点, 节点名称为 `dcmipp`, 地址为 `dcmipp@48030000`。这是一个数字摄像头接口模块, 用于像素处理。

第 14 行, `compatible = "st,stm32mp25-dcmipp"` 兼容字符串, 表明该硬件模块的驱动程序应识别为 `stm32mp25-dcmipp`。

第 15 行, `reg = <0x48030000 0x1000>` 指定设备的内存地址和大小。此设备的基地址为 `0x48030000`, 大小为 `0x1000` 字节。

第 16 行, `interrupts = <GIC_SPI 198 IRQ_TYPE_LEVEL_HIGH>` 定义该设备使用的中断号。使用全局中断控制器 (GIC) 的 SPI 编号 198, 类型为高电平触发。

第 17 行, `resets = <&rcc DCMIPP_R>` 定义与复位控制器 (rcc) 的关联, 复位信号为 `DCMIPP_R`。

第 18~19 行, 定义设备使用的时钟:

- `<&rcc CK_BUS_DCMIPP>`: 总线时钟 (`kclk`)。
- `<&rcc CK_KER_CSI>`: 内核时钟 (`mclk`)。

第 20 行, `clock-names = "kclk", "mclk"` 为每个时钟定义名称, 便于驱动程序使用。

第 21 行, `access-controllers = <&rifsc 87>` 定义访问控制器, 标号为 87。

第 22 行, `power-domains = <&CLUSTER_PD>` 定义设备所属的电源域 (`CLUSTER_PD`), 用于动态电源管理。

第 23 行, `status = "disabled"` 指定设备状态为禁用, 需启用时修改为 `"okay"`。



以 2GB 版本核心板为例, 在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 中, Camera 相关配置如下:

```
1 &i2c4 {
2     imx335: imx335@1a {
3         compatible = "sony,imx335";
4         reg = <0x1a>;
5         clocks = <&clk_ext_camera>;
6         avdd-supply = <&imx335_2v9>;
7         ovdd-supply = <&imx335_1v8>;
8         dvdd-supply = <&imx335_1v2>;
9         reset-gpios = <&gpio4 4 (GPIO_ACTIVE_HIGH | GPIO_PUSH_PULL)>;
10        powerdown-gpios = <&gpio4 3 (GPIO_ACTIVE_HIGH | GPIO_PUSH_PULL)>;
11        status = "okay";
12
13        port {
14            imx335_ep: endpoint {
15                remote-endpoint = <&csi_sink>;
16                clock-lanes = <0>;
17                data-lanes = <1 2>;
18                link-frequencies = /bits/ 64 <594000000>;
19            };
20        };
21    };
22 };
23
24 &csi {
25     vdd-supply = <&scmi_vddcore>;
26     vdda18-supply = <&scmi_v1v8>;
27     status = "okay";
28     ports {
29         #address-cells = <1>;
30         #size-cells = <0>;
31         port@0 {
32             reg = <0>;
33             csi_sink: endpoint {
34                 remote-endpoint = <&imx335_ep>;
35                 data-lanes = <0 1>;
36                 bus-type = <4>;
37             };
38         };
39
40         port@1 {
41             reg = <1>;
```



```

42     csi_source: endpoint {
43         remote-endpoint = <&dcmipp_0>;
44     };
45 };
46 };
47 };
48
49 &dcmipp {
50     status = "okay";
51     port {
52         dcmipp_0: endpoint {
53             remote-endpoint = <&csi_source>;
54             bus-type = <4>;
55         };
56     };
57 };

```

I2C4 节点:

第 1 行, 引用设备树中的 i2c4 节点, 定义与摄像头传感器连接的 I2C 总线。

第 2 行, 创建子节点 imx335:imx335@1a, 表示连接在 I2C 地址为 0x1a 的摄像头传感器。

第 3 行, 指定摄像头传感器的兼容字符串, 表示设备是 Sony IMX335。

第 4 行, 定义传感器的 I2C 地址为 0x1a。

第 5 行, clocks = <&clk_ext_camera> 定义传感器的外部时钟, 连接到 clk_ext_camera。

第 6~8 行, 定义传感器的电源:

- avdd-supply: 模拟电压 (2.9V)。
- ovdd-supply: I/O 电压 (1.8V)。
- dvdd-supply: 数字核心电压 (1.2V)。

第 9~10 行, 定义与 GPIO 控制相关的引脚:

- reset-gpios: 复位引脚, 连接 GPIOG 的第 4 引脚。
- powerdown-gpios: 电源关闭引脚, 连接 GPIOG 的第 3 引脚。
- 两者都为高电平有效, 推挽输出。

第 11 行, status = "okay" 表示传感器启用。

第 13~14 行, 定义传感器的 MIPI 数据接口端口。

- port: 定义传感器输出的端口。
- endpoint: 定义传感器的 MIPI 数据连接细节。

第 15 行, remote-endpoint = <&csi_sink> 表示此端点连接到 CSI 模块的 csi_sink 端点。

第 16~17 行, 定义 MIPI 数据通道配置:

- clock-lanes = <0>: 时钟通道为通道 0。
- data-lanes = <1 2>: 数据通道为 1 和 2。

第 18 行, link-frequencies = /bits/ 64 <594000000> 定义 MIPI 接口链路频率为 594 MHz。

CSI 节点:

第 24 行, 引用 csi 节点, 定义 MIPI CSI 接口。

第 25~26 行, 定义 CSI 模块使用的电源:

- vdd-supply: 核心电源 (scmi_vddcore)。

- vdda18-supply: 1.8V 电源 (scmi_v1v8)。

第 27 行, status = "okay" 启用 CSI 模块。

第 28 行, 定义 CSI 模块的端口和连接。

第 29~30 行, #address-cells = <1>; 和 #size-cells = <0> 定义端口地址单元长度为 1, 大小单元长度为 0。

第 31 行, 定义 port@0 作为 CSI 的输入端口。

第 32 行, reg = <0> 端口编号为 0。

第 33 行, remote-endpoint = <&imx335_ep> 此端口连接到 IMX335 的 endpoint。

第 35~36 行, 定义数据通道信息和总线类型:

- data-lanes = <0 1>;: 时钟通道为 0, 数据通道为 1。
- bus-type = <4>;: 总线类型为 CSI-2。

第 40 行, 定义 port@1 作为 CSI 的输出端口。

第 42~43 行, remote-endpoint = <&dcmipp_0> 此端口连接到 DCMIPP 的 endpoint。

DCMIPP 节点:

第 49 行, 引用 dcmipp 节点, 定义数字摄像头接口的像素处理模块。

第 50 行, status = "okay" 启用 DCMIPP 模块。

第 51 行, 定义 DCMIPP 的数据端口。

第 52 行, dcmipp_0: endpoint 定义端点名称为 dcmipp_0。

第 53 行, remote-endpoint = <&csi_source> 表示此端点连接到 CSI 模块的 csi_source。

第 54 行, bus-type = <4> 定义总线类型为 CSI-2。

22.7 Camera 工具

22.7.1 dmesg

Linux 内核通过串口输出日志消息, 可以使用 dmesg 命令来查看这些消息。这对于调试引导过程中的问题非常有用, 也可以用于检查内核模块加载和设备初始化过程中的消息。grep 是一个 Linux/Unix 下的命令行工具, 用于在文本文件中搜索指定的字符串模式, 并将包含匹配的行输出到标准输出 (通常是终端)。

示例:

dmesg | grep imx335

```
root@ATK-DLMP257:~# dmesg | grep imx335
[ 0.284200] platform 48020000.csi2host: Fixed dependency cycle(s) with /soc@0/rifsc@42080000/i2c@40150000/imx335@1a
[ 3.505450] platform 48020000.csi2host: Fixed dependency cycle(s) with /soc@0/rifsc@42080000/i2c@40150000/imx335@1a
[ 3.596476] imx335 1-001a: Consider updating driver imx335 to match on endpoints
root@ATK-DLMP257:~#
```

图 22.7-1 imx335 信息

若摄像头设备已被正确创建, 则会在/dev 目录下生成设备节点, 可以执行下面指令查看:

ls /dev/video*

```
root@ATK-DLMP257:~# ls /dev/video*
/dev/video0 /dev/video1 /dev/video2 /dev/video3 /dev/video4 /dev/video5 /dev/video6
root@ATK-DLMP257:~#
```

图 22.7-2 摄像头设备节点

22.7.2 v4l2-sysfs-path

v4l2-sysfs-path 是一个工具，用于查找 V4L2 设备的 sysfs 路径。sysfs 是 Linux 内核中一个用于暴露设备和驱动程序信息的虚拟文件系统。

开发板执行下面指令：

v4l2-sysfs-path

```
root@ATK-DLMP257:~# v4l2-sysfs-path
Video device: video0
  video: video1
  v4l subdevice: v4l-subdev0
Video device: video1
  video: video2
  v4l subdevice: v4l-subdev0
Video device: video2
  video: video3
  v4l subdevice: v4l-subdev0
Video device: video3
  video: video4
  v4l subdevice: v4l-subdev0
Video device: video4
  v4l subdevice: v4l-subdev0
Video device: video5
Video device: video6
Alsa playback device(s): hw:0,1
root@ATK-DLMP257:~#
```

图 22.7-3 v4l2-sysfs-path 打印信息

v4l2-sysfs-path 输出了有关视频设备和音频设备的信息。以下是解释：

Video device: video0

表示已识别并注册的 V4L2 视频设备为 /dev/video0，以此类推。

Alsa playback device(s): hw:0,0

表示已识别并注册的 ALSA 播放设备，设备号为 hw:0,0。

22.7.3 v4l2-compliance

v4l2-compliance 是一个验证工具，用于测试 V4L2 驱动程序是否符合 V4L2 API 的规范。它通过一系列测试来检查驱动程序的实现是否正确和兼容。

开发板执行下面指令：

v4l2-compliance

输出信息参考如下（内容过长，以下截图分段处理）：

```
root@ATK-DLMP257:~# v4l2-compliance
v4l2-compliance 1.26.1-5142, 64 bits, 64-bit time_t
v4l2-compliance SHA: 4aee01a02792 2023-12-12 21:40:38

Compliance test for dcmipp device /dev/video0:

Driver Info:
  Driver name      : dcmipp
  Card type        : stm32_dcmipp
  Bus info         : platform:48030000.dcmipp
  Driver version   : 6.6.48
  Capabilities     : 0xa200001
    Video Capture
    I/O MC
    Streaming
    Extended Pix Format
    Device Capabilities
  Device Caps      : 0x24200001
    Video Capture
    I/O MC
    Streaming
    Extended Pix Format
```

图 22.7-4 驱动信息

驱动名称: dcmipp

驱动卡类型: stm32_dcmipp, 表明是针对 STM32 的 DCMIPP (Digital Camera Memory Interface Pixel Processor)

支持特性:

视频捕获 (Video Capture)

媒体控制接口 (I/O MC)

数据流 (Streaming)

扩展像素格式 (Extended Pix Format)

```
Media Driver Info:
  Driver name      : dcmipp
  Model           : DCMIPP MDEV
  Serial          :
  Bus info        : platform:48030000.dcmipp
  Media version    : 6.6.48
  Hardware revision: 0x00000030 (48)
  Driver version   : 6.6.48
```

图 22.7-5 媒体驱动信息

Driver name: 媒体驱动程序的名称是 mxc-md。

Model: 模型是 FSL Capture Media Device。

Bus info: 总线信息是 platform:42800000.bus:camera。

Media version: 媒体版本是 6.6.48。

Hardware revision: 硬件版本是 0x00000000 (0)。

Driver version: 驱动程序的版本是 6.6.48。

```
Interface Info:
  ID      : 0x0300000b
  Type    : V4L Video
```

图 22.7-6 接口信息

ID: 接口的 ID 是 0x0300000b。

Type: 接口类型是 V4L Video。

```
Entity Info:
  ID      : 0x00000009 (9)
  Name    : dcmipp_dump_capture
  Function: V4L2 I/O
  Pad 0x0100000a : 0: Sink
  Link 0x0200002e: from remote pad 0x10000008 of entity 'dcmipp_dump_postproc' (Video Scaler): Data
, Enabled, Immutable
```

图 22.7-7 实体信息

/dev/video0 是 dcmipp_dump_capture 实体的一部分, 功能是 V4L2 输入/输出。

其数据来自 dcmipp_dump_postproc 实体, 通过数据链路连接。

表明 /dev/video0 是一个捕获视频流的设备节点。

后面的是测试项目, 包括输入输出配置、控制接口、格式支持等, 视频输入端 (Input 0) 支持格式枚举、分辨率和帧间隔设置。

关于 v4l2-compliance 的更多使用方法可以查看使用说明, 指令如下:

```
v4l2-compliance -h
```


22.7.4 v4l2-ctl

v4l2-ctl 是一个命令行工具,用于控制和查询 V4L2 设备的属性,可以使用它来列出设备、获取和设置设备参数、查询格式等。

执行 v4l2-ctl 指令可以直接查看使用方法:

v4l2-ctl

```
root@ATK-DLMP257:~# v4l2-ctl
General/Common options:
--all                display all information available
-C, --get-ctrl <ctrl>[,<ctrl>...]
                    get the value of the controls [VIDIOC_G_EXT_CTRLS]
-c, --set-ctrl <ctrl>=<val>[,<ctrl>=<val>...]
                    set the value of the controls [VIDIOC_S_EXT_CTRLS]
-D, --info           show driver info [VIDIOC_QUERYCAP]
-d, --device <dev>  use device <dev> instead of /dev/video0
                    if <dev> starts with a digit, then /dev/video<dev> is used
                    Otherwise if -z was specified earlier, then <dev> is the entity name
                    or interface ID (if prefixed with 0x) as found in the topology of the
                    media device with the bus info string as specified by the -z option.
-e, --out-device <dev> use device <dev> for output streams instead of the
                    default device as set with --device
                    if <dev> starts with a digit, then /dev/video<dev> is used
                    Otherwise if -z was specified earlier, then <dev> is the entity name
                    or interface ID (if prefixed with 0x) as found in the topology of the
                    media device with the bus info string as specified by the -z option.
-E, --export-device <dev> use device <dev> for exporting DMA buffers
                    if <dev> starts with a digit, then /dev/video<dev> is used
                    Otherwise if -z was specified earlier, then <dev> is the entity name
                    or interface ID (if prefixed with 0x) as found in the topology of the
                    media device with the bus info string as specified by the -z option.
-z, --media-bus-info <bus-info>
                    find the media device with the given bus info string. If set, then
                    -d, -e and -E options can use the entity name or interface ID to refer
                    to the device nodes.
-h, --help           display this help message
--help-all          all options
--help-io            input/output options
--help-meta          metadata format options
--help-misc          miscellaneous options
--help-overlay       overlay format options
--help-sdr           SDR format options
--help-selection     crop/selection options
--help-stds          standards and other video timings options
--help-streaming     streaming options
--help-subdev        sub-device options
--help-tuner         tuner/modulator options
--help-vbi           VBI format options
--help-vidcap        video capture format options
--help-vidout        vidout output format options
--help-edid          edid handling options
-k, --concise        be more concise if possible.
-l, --list-ctrls     display all controls and their values [VIDIOC_QUERYCTRL]
-L, --list-ctrls-menus
                    display all controls and their menus [VIDIOC_QUERYMENU]
-r, --subset <ctrl>[,<offset>,<size>]+
                    the subset of the N-dimensional array to get/set for control <ctrl>,
                    for every dimension an (<offset>, <size>) tuple is given.
-w, --wrapper        use the libv4l2 wrapper library.
--list-devices       list all v4l devices. If -z was given, then list just the
                    devices of the media device with the bus info string as
                    specified by the -z option.
--log-status         log the board status in the kernel log [VIDIOC_LOG_STATUS]
--get-priority       query the current access priority [VIDIOC_G_PRIORITY]
--set-priority <prio>
                    set the new access priority [VIDIOC_S_PRIORITY]
                    <prio> is 1 (background), 2 (interactive) or 3 (record)
                    only set the result code, do not print any messages
--silent            only set the result code, do not print any messages
--sleep <secs>      sleep <secs>, call QUERYCAP and close the file handle
--verbose           turn on verbose ioctl status reporting
--version           show version information
```

图 22.7-8 v4l2-ctl 使用说明

选项解析:

- --all

显示所有可用信息。

- -C, --get-ctrl <ctrl>[,<ctrl>...]



获取控制项的值 [VIDIOC_G_EXT_CTRL]。

- `-c, --set-ctrl <ctrl>=<val>[,<ctrl>=<val>...]`

设置控制项的值 [VIDIOC_S_EXT_CTRL]。

- `-D, --info`

显示驱动信息 [VIDIOC_QUERYCAP]。

- `-d, --device <dev>`

使用设备 `<dev>` 而不是 `/dev/video0`。如果 `<dev>` 以数字开头, 则使用 `/dev/video<dev>`。

否则, 如果之前指定了 `-z`, 则 `<dev>` 是实体名称或接口 ID (如果前缀为 `0x`), 可以在媒体设备的拓扑中找到。

- `-e, --out-device <dev>`

使用设备 `<dev>` 进行输出流, 而不是默认的输入设备。如果 `<dev>` 以数字开头, 则使用 `/dev/video<dev>`。否则, 如果之前指定了 `-z`, 则 `<dev>` 是实体名称或接口 ID (如果前缀为 `0x`), 可以在媒体设备的拓扑中找到。

- `-E, --export-device <dev>`

使用设备 `<dev>` 导出 DMA 缓冲区。如果 `<dev>` 以数字开头, 则使用 `/dev/video<dev>`。

否则, 如果之前指定了 `-z`, 则 `<dev>` 是实体名称或接口 ID (如果前缀为 `0x`), 可以在媒体设备的拓扑中找到。

- `-z, --media-bus-info <bus-info>`

查找具有给定总线信息字符串的媒体设备。如果设置了该选项, 则 `-d`、`-e` 和 `-E` 选项可以使用实体名称或接口 ID 来引用设备节点。

- `-h, --help`

显示此帮助信息。

- `--help-all`

显示所有选项的帮助信息。

- `--help-io`

显示输入/输出选项的帮助信息。

- `--help-meta`

显示元数据格式选项的帮助信息。

- `--help-misc`

显示杂项选项的帮助信息。

- `--help-overlay`

显示覆盖格式选项的帮助信息。

- `--help-sdr`

显示 SDR 格式选项的帮助信息。

- `--help-selection`

显示裁剪/选择选项的帮助信息。

- `--help-stds`

显示标准和其他视频定时选项的帮助信息。

- `--help-streaming`

显示流媒体选项的帮助信息。

- `--help-subdev`

显示子设备选项的帮助信息。

- `--help-tuner`



显示调谐器/调制器选项的帮助信息。

- --help-vbi

显示 VBI 格式选项的帮助信息。

- --help-vidcap

显示视频捕获格式选项的帮助信息。

- --help-vidout

显示视频输出格式选项的帮助信息。

- --help-edid

显示 EDID 处理选项的帮助信息。

- -k, --concise

尽可能简洁。

- -l, --list-ctrls

显示所有控制项及其值 [VIDIOC_QUERYCTRL]。

- -L, --list-ctrls-menus

显示所有控制项及其菜单 [VIDIOC_QUERYMENU]。

- -r, --subset <ctrl>[,<offset>,<size>]+

获取/设置控制项 <ctrl> 的 N 维数组的子集。对于每个维度，给出一个 (<offset>, <size>) 元组。

- -w, --wrapper

使用 libv4l2 包装库。

- --list-devices

列出所有 V4L 设备。如果给出了 -z，则只列出总线信息字符串指定的媒体设备的设备。

- --log-status

在内核日志中记录板状态 [VIDIOC_LOG_STATUS]。

- --get-priority

查询当前访问优先级 [VIDIOC_G_PRIORITY]。

- --set-priority <prio>

设置新的访问优先级 [VIDIOC_S_PRIORITY]。<prio> 为 1(后台), 2(交互)或 3(记录)。

- --silent

只设置结果代码，不打印任何消息。

- --sleep <secs>

休眠 <secs> 秒，调用 QUERYCAP 并关闭文件句柄。

- --verbose

打开详细的 ioctl 状态报告。

- --version

显示版本信息。

示例：列出所有设备

```
v4l2-ctl --list-devices
```

```
root@ATK-DLMP257:~# v4l2-ctl --list-devices
stm32_dcmipp (platform:48030000.dcmipp):
/dev/video0
/dev/video1
/dev/video2
/dev/video3
/dev/video4
/dev/media0

st,stm32mp25-vdec-dec (platform:480d0000.vdec):
/dev/video5
/dev/media1

st,stm32mp25-venc-enc (platform:480e0000.venc):
/dev/video6
/dev/media2
```

图 22.7-9 v4l2-ctl --list-devices

笔者开发板接了 MIPI 屏幕和 IMX335 摄像头，分析设备列表如下：

1、stm32_dcmipp (DCMIPP Video Pipeline):

- 设备节点：
/dev/video0 到 /dev/video4 是视频捕获设备节点。
/dev/media0 是媒体设备节点，主要用于配置媒体控制器拓扑。
- 这些设备可能与摄像头（如 IMX335 传感器）连接，负责视频捕获和后处理。

2、st,stm32mp25-vdec-dec (Video Decoder):

- 设备节点：
/dev/video5 是视频解码器节点。
/dev/media1 是其对应的媒体设备节点。
- 表明系统中包含硬件视频解码单元，可能用于播放压缩视频流（如 H.264、H.265）。

3、st,stm32mp25-venc-enc (Video Encoder):

- 设备节点：
/dev/video6 是视频编码器节点。
/dev/media2 是其对应的媒体设备节点。
- 表明系统中包含硬件视频编码单元，可能用于将捕获的 RAW 视频流编码为压缩格式（如 H.264、H.265）。

示例：查看/dev/video1 所有可用信息。

```
v4l2-ctl -d /dev/video1 --all
```

```

root@ATK-DLMP257:~# v4l2-ctl -d /dev/video1 --all
Driver Info:
    Driver name      : dcmipp
    Card type        : stm32_dcmipp
    Bus info         : platform:48030000.dcmipp
    Driver version    : 6.6.48
    Capabilities     : 0xa4200001
        Video Capture
        I/O MC
        Streaming
        Extended Pix Format
        Device Capabilities
    Device Caps      : 0x24200001
        Video Capture
        I/O MC
        Streaming
        Extended Pix Format
Media Driver Info:
    Driver name      : dcmipp
    Model           : DCMIPP MDEV
    Serial          :
    Bus info        : platform:48030000.dcmipp
    Media version    : 6.6.48
    Hardware revision: 0x00000030 (48)
    Driver version   : 6.6.48
Interface Info:
    ID              : 0x03000017
    Type            : V4L Video
Entity Info:
    ID              : 0x00000015 (21)
    Name            : dcmipp_main_capture
    Function        : V4L2 I/O
    Pad 0x01000016  : 0: Sink
    Link 0x02000036: from remote pad 0x1000014 of entity 'dcmipp_main_postproc' (Video Pixel Formatt
er): Data, Enabled, Immutable
Priority: 2
Video input : 0 (dcmipp_main_capture: ok)
Format Video Capture:
    Width/Height     : 640/480
    Pixel Format      : 'RGBP' (16-bit RGB 5-6-5)
    Field            : None
    Bytes per Line    : 1280
    Size Image       : 614400
    Colorspace       : Rec. 709
    Transfer Function : Default (maps to Rec. 709)
    YCbCr/HSV Encoding: Default (maps to Rec. 709)
    Quantization     : Default (maps to Full Range)
    Flags            :
root@ATK-DLMP257:~#

```

图 22.7-10 v4l2-ctl -d /dev/video1 -all

驱动信息:

- 驱动名称: dcmipp, 这是设备使用的驱动。
- 卡片类型: stm32_dcmipp, 设备的名称。
- 总线信息: platform:48030000.dcmipp, 表示设备位于平台总线上的某个地址。
- 驱动版本: 6.6.48, 驱动的版本号。
- 能力:
- Video Capture: 支持视频捕获。
- I/O MC: 支持多通道 I/O 操作。
- Streaming: 支持视频流传输。
- Extended Pix Format: 支持扩展的像素格式。
- Device Capabilities: 设备具备的功能与驱动能力相同。

媒体驱动信息:

- 驱动名称: dcmipp。
- 模型: DCMIPP MDEV, 驱动的设备模型。
- 硬件版本: 0x00000030 (48), 硬件的版本号。
- 驱动版本: 6.6.48, 驱动版本。

接口信息:



- ID: 0x03000017, 接口的唯一标识符。
- 类型: V4L Video, 表示这是一个视频接口。

实体信息:

- 实体 ID: 0x00000015 (21), 表示与实体相关的标识符。
- 实体名称: dcmipp_main_capture, 实体的名称, 表示视频捕获的功能。
- 功能: V4L2 I/O, 表示这是一个 I/O 功能实体。
- Pad 信息:
- Pad ID: 0x01000016, 表示一个 Pad (连接点)。
- 方向: Sink (数据输入)。
- 连接信息: 这个 Pad 连接到 dcmipp_main_postproc 实体的 Pad (处理后的图像数据)。

视频输入:

- 视频输入: 0 (dcmipp_main_capture: ok), 表示视频输入通道 0 正在工作。

视频捕获格式:

- 分辨率: 640x480, 捕获的视频分辨率为 640x480。
- 像素格式: RGBP (16-bit RGB 5-6-5), 表示使用 RGB 色彩模式, 每个像素为 16 位。
- 场: None, 表示没有场, 说明是逐行扫描而不是隔行扫描。
- 每行字节数: 1280 字节, 每一行图像数据占用的字节数。
- 图像大小: 614400 字节, 表示整个图像的数据大小。
- 色彩空间: Rec. 709, 采用 Rec. 709 色彩空间标准。
- 传输函数: 默认 (对应 Rec. 709)。
- YCbCr/HSV 编码: 默认 (对应 Rec. 709)。
- 量化方式: 默认 (对应全范围)。

示例: 显示支持的格式

```
v4l2-ctl --list-formats-ext -d /dev/video1
```



```
root@ATK-DLMP257:~# v4l2-ctl --list-formats-ext -d /dev/video1
ioctl: VIDIOC_ENUM_FMT
Type: Video Capture

[0]: 'RGBP' (16-bit RGB 5-6-5)
    Size: Continuous 16x16 - 4096x2160
[1]: 'YUYV' (YUYV 4:2:2)
    Size: Continuous 16x16 - 4096x2160
[2]: 'YVYU' (YVYU 4:2:2)
    Size: Continuous 16x16 - 4096x2160
[3]: 'UYVY' (UYVY 4:2:2)
    Size: Continuous 16x16 - 4096x2160
[4]: 'VYUY' (VYUY 4:2:2)
    Size: Continuous 16x16 - 4096x2160
[5]: 'GREY' (8-bit Greyscale)
    Size: Continuous 16x16 - 4096x2160
[6]: 'RGB3' (24-bit RGB 8-8-8)
    Size: Continuous 16x16 - 4096x2160
[7]: 'BGR3' (24-bit BGR 8-8-8)
    Size: Continuous 16x16 - 4096x2160
[8]: 'BA24' (32-bit ARGB 8-8-8-8)
    Size: Continuous 16x16 - 4096x2160
[9]: 'AR24' (32-bit BGRA 8-8-8-8)
    Size: Continuous 16x16 - 4096x2160
[10]: 'AB24' (32-bit RGBA 8-8-8-8)
    Size: Continuous 16x16 - 4096x2160
[11]: 'RA24' (32-bit ABGR 8-8-8-8)
    Size: Continuous 16x16 - 4096x2160
[12]: 'NV12' (Y/UV 4:2:0)
    Size: Continuous 16x16 - 4096x2160
[13]: 'NV21' (Y/VU 4:2:0)
    Size: Continuous 16x16 - 4096x2160
[14]: 'NV16' (Y/UV 4:2:2)
    Size: Continuous 16x16 - 4096x2160
[15]: 'NV61' (Y/VU 4:2:2)
    Size: Continuous 16x16 - 4096x2160
[16]: 'YU12' (Planar YUV 4:2:0)
    Size: Continuous 16x16 - 4096x2160
[17]: 'YV12' (Planar YVU 4:2:0)
    Size: Continuous 16x16 - 4096x2160
root@ATK-DLMP257:~#
```

图 22.7-11 查看/dev/video1 支持的格式

此命令显示了 /dev/video0 设备支持的多种视频捕获格式和分辨率。不同的格式代表了视频数据的不同编码方式，每种编码方式支持不同的分辨率组合。选择适合的格式和分辨率取决于具体的应用需求，例如画质、处理速度和存储要求等。

22.7.5 gst-launch-1.0

gst-launch-1.0 是 GStreamer 多媒体框架中的一个命令行工具，用于构建和运行媒体处理流水线。GStreamer 是一个开源的多媒体框架，支持音频、视频、录制、播放、流式传输和处理等各种媒体操作。

gst-launch-1.0 命令允许用户通过命令行指定一系列的 GStreamer 元素，这些元素通过管道连接起来，形成一个完整的媒体处理流水线。这个命令通常用于快速测试、调试和原型开发。

执行下面指令查看 gst-launch-1.0 使用说明：

```
gst-launch-1.0 -h
```

```

root@ATK-DLMP257:~# gst-launch-1.0 -h
Usage:
  gst-launch-1.0 [OPTION?] PIPELINE-DESCRIPTION

Help Options:
  -h, --help                Show help options
  --help-all               Show all help options
  --help-gst               Show GStreamer Options

Application Options:
  -t, --tags                Output tags (also known as metadata)
  -c, --toc                Output TOC (chapters and editions)
  -v, --verbose            Output status information and property notifications
  -q, --quiet              Do not print any progress information
  -m, --messages           Output messages
  -X, --exclude=PROPERTY-NAME Do not output status information for the specified property if verbose
                           output is enabled (can be used multiple times)
  -f, --no-fault           Do not install a fault handler
  -e, --eos-on-shutdown    Force EOS on sources before shutting the pipeline down
  --version               Print version information and exit
  --no-position           Do not print current position of pipeline. If this option is unspecified,
                           the position will be printed when stdout is a TTY. To enable printing position when stdout is not a TTY,
                           use "force-position" option
  --force-position         Allow printing current position of pipeline even if stdout is not a TTY. This option has no effect if the "no-position" option is specified

root@ATK-DLMP257:~#

```

图 22.7-12 gst-launch-1.0 使用说明

常用选项

- -h, --help: 显示帮助选项
- --help-all: 显示所有帮助选项
- --help-gst: 显示 GStreamer 特有选项
- -t, --tags: 输出标签（也称为元数据）
- -c, --toc: 输出 TOC（章节和编辑）
- -v, --verbose: 输出状态信息和属性通知
- -q, --quiet: 不打印任何进度信息
- -m, --messages: 输出消息
- -X, --exclude=PROPERTY-NAME: 在启用详细输出时不输出指定属性的信息（可以多次使用）
- -f, --no-fault: 不安装故障处理程序
- -e, --eos-on-shutdown: 在关闭管道之前强制 EOS（End of Stream）
- --version: 打印版本信息并退出
- --no-position: 不打印管道的当前位置。如果未指定此选项，则当 stdout 是 TTY 时会打印位置。要在 stdout 不是 TTY 时启用位置打印，请使用“force-position”选项
- --force-position: 即使 stdout 不是 TTY 也允许打印管道的当前位置。如果指定了“no-position”选项，此选项无效

第二十三章 Audio 子系统

23.1 Audio 子系统简介

嵌入式 Linux 的音频子系统（Audio Subsystem）是一个复杂而强大的系统，用于管理和控制音频硬件。它提供了必要的接口和驱动程序，支持音频数据的捕获、回放、混音等操作。

23.2 组成部分

嵌入式 Linux 的音频子系统由多个部分组成，每个部分都有其特定的功能和角色，以协同工作来实现音频管理和控制。以下是嵌入式 Linux 音频系统的主要组成部分：

1、ALSA（Advanced Linux Sound Architecture）

ALSA 核心

- PCM（Pulse Code Modulation）接口：用于处理音频数据的捕获和回放。
- 控制接口：用于设置和获取音频设备的参数。
- Mixer 接口：用于音量控制和音频信号的混合。

用户空间库

- libasound：提供高级 API，使应用程序可以方便地进行音频操作。

2、音频驱动程序

平台驱动（Platform Driver）

- 负责与特定硬件平台相关的音频设备初始化和控制。

Codec 驱动（Codec Driver）

- 控制音频编解码器，处理音频信号的转换和处理。

机器驱动（Machine Driver）

- 描述平台和 Codec 之间的连接和交互。

3、设备树（Device Tree）

- 描述硬件布局：用于描述系统中音频硬件的配置和连接。
- 配置参数：提供驱动程序初始化所需的配置参数。

4、音频框架

ASoC（ALSA System on Chip）

- 专为嵌入式系统设计的音频框架，简化了复杂音频硬件的配置。

5、用户空间应用

命令行工具

- aplay/arecord：用于音频数据的播放和录制。
- alsamixer：基于命令行的音量控制和混音工具。
- amixer：命令行音频控制工具。

6、Use Case Manager（UCM）

- 管理使用场景：提供不同音频使用场景的灵活配置和优化。



7、调试和开发工具

- 调试工具：如 `alsa-info.sh` 用于收集系统音频信息，`alsabat` 用于音频环回测试。

通过以上各组成部分，嵌入式 Linux 音频子系统能够提供灵活、强大且高效的音频管理功能，支持多种音频硬件和复杂的音频应用需求。

23.3 MP257 的 Audio 说明

23.3.1 文档和资源

想要获取 MP257 系列处理器的 Camera 详细说明，最直接的方式是参考 ST 官方文档，包括：

1、参考手册（Reference Manual）：此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

2、数据手册（Data Sheet）：提供了关于 MP257 系列处理器的电气特性、引脚分配图和其他硬件级细节。此文档已放在【正点原子】STM32MP257 开发板（A 盘）-基础资料\07、STM32MP2 参考资料

对于 ATK-DLMP257B 开发板，正点原子还提供了一系列的开发文档，便于提高用户开发效率。开发文档包含且不仅限于驱动开发、应用开发、原理图、系统管脚修改等。

BSP 开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux 驱动开发指南（即本文档）

应用开发：【正点原子】STM32MP257 开发板（A 盘）-基础资料\09、文档教程\【正点原子】ATK-DLMP257B 嵌入式 Linux C 应用编程指南

开发板底板原理图：【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf

核心板管脚功能说明：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\02、核心板管脚定义\【正点原子】ATK-DLMP257B 核心板接口数据手册

核心板 IO 功能修改参考：【正点原子】STM32MP257 开发板（A 盘）-基础资料\06、硬件资料\05、核心板资料\03、核心板使用参考手册\【正点原子】ATK-DLMP257B 出厂系统外设复用手册

更多文档请下载最新开发板网盘资料进行查阅。

23.3.2 特性说明

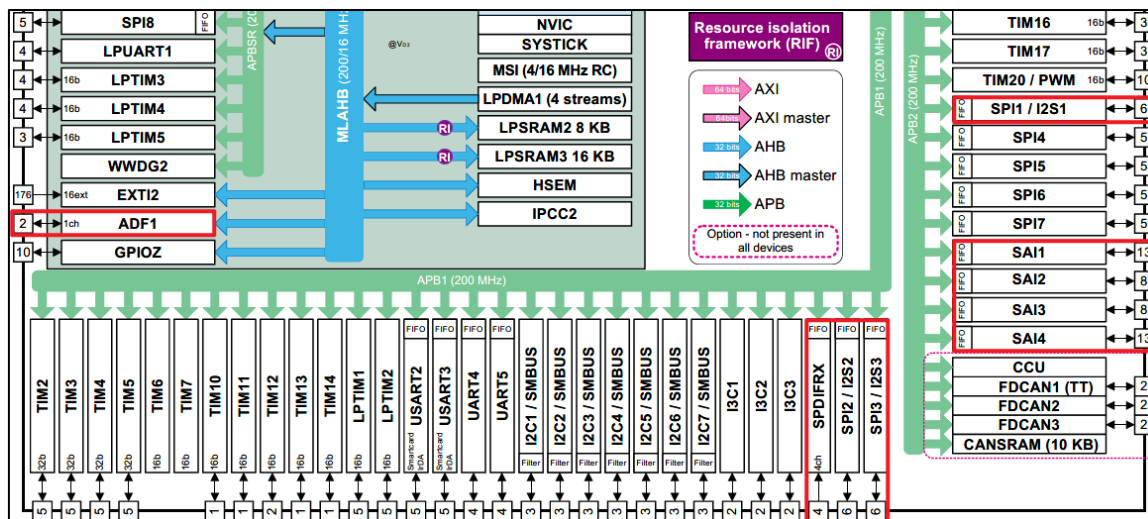


图 23.3-1 MP257 的 Audio 部分

MP257 的 Audio 包括三个 I2S、四个 SAI、一个 ADF1 和一个 SPDIFRX，它们各有用途，通常根据应用场景和硬件资源需求进行选择 and 组合使用。

- I2S 是最基础的音频接口，功能简单，适合基本的音频通信需求。
- SAI 是对 I2S 的扩展，支持更多协议和通道，适用于高端音频场景。
- ADF1 专注于处理 PDM 麦克风信号，主要用于语音识别或录音场景。
- SPDIFRX 专门处理 SPDIF 音频流，常用于音频接收和解码的消费电子设备。

特性/接口	I2S	SAI	ADF1	SPDIFRX
主要用途	音频设备间数据传输	多协议音频接口	PDM 麦克风信号处理	SPDIF 格式音频流接收
支持协议	I2S、PCM、左右对齐	I2S、TDM、AC97	PDM (转换为 PCM)	SPDIF
支持通道	单个或少量音频通道	多通道	单声道或双声道麦克风	单声道或多声道(如 AC-3)
硬件资源	简单，适合基础音频	复杂，支持高级音频	专用于麦克风信号	专用硬件用于 SPDIF
适用场景	嵌入式音频通信	高端音频设备和音频流	数字麦克风音频前处理	家庭影院、消费电子
灵活性	低	高	专用功能，灵活性较低	专用功能，灵活性较低
数据类型	PCM 数据	PCM、压缩数据	PDM -> PCM 转换	PCM 或压缩数据
DMA 支持	支持	支持	支持	支持

更多关于 MP257 的 Audio 资料请查看芯片参考手册。

23.4 开发板 Audio 接口

23.4.1 原理图分析

开发板上有一个音频芯片 ES8388。打开开发板上【正点原子】STM32MP257 开发板(A 盘)-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 SAI。可以看到相关电路如下。

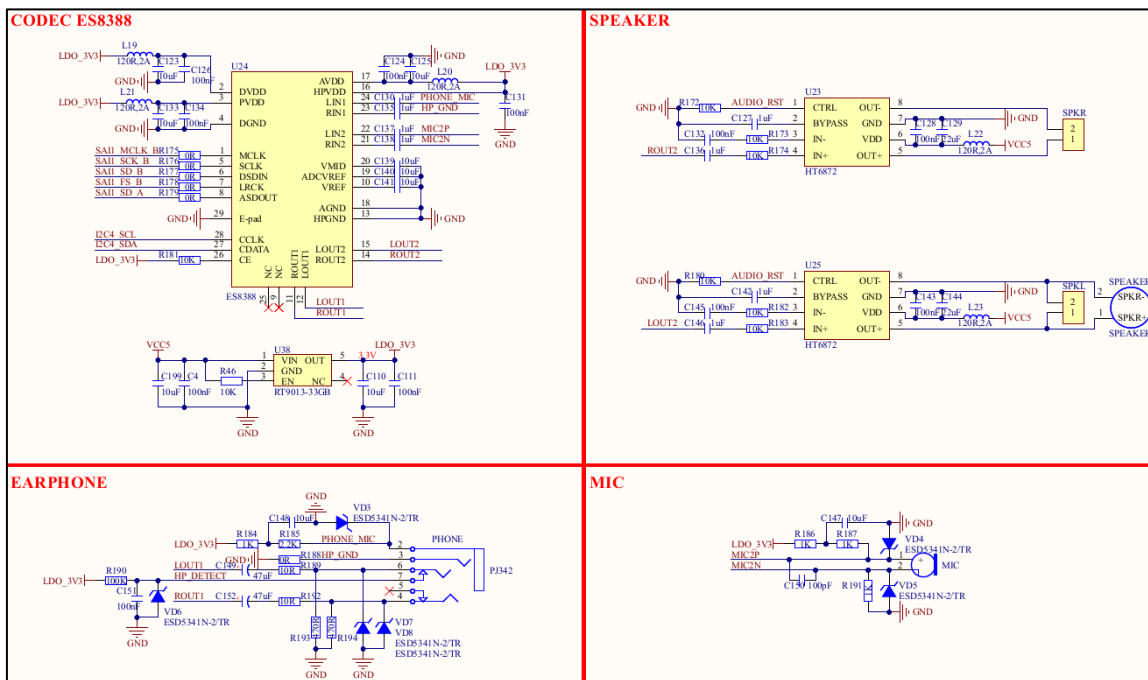


图 23.4-1 开发板音频电路原理图

音频相关原理图共分为 CODEC ES8388、SPEAKER、EARPHONE 和 MIC 四部分：

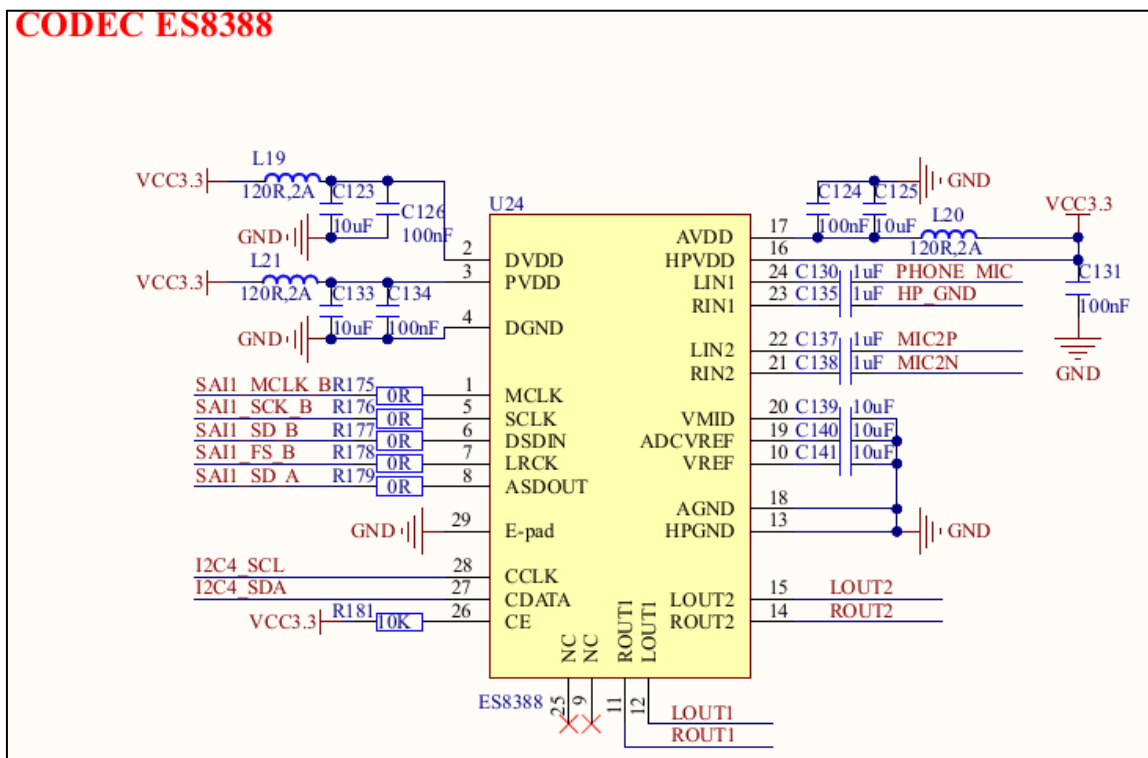


图 23.4-2 ES8388 部分原理图

1、电源部分

- DVDD、PVDD（引脚 2、3）：连接到 VCC3.3（3.3V 供电），通过电感 L19 和 L21 以及去耦电容 C123/C126 和 C133/C134 进行滤波，确保电源的稳定性。



- AVDD、HPVDD（引脚 16、17）：同样连接到 VCC3.3，通过电感 L20 和去耦电容 C124/C125 提供模拟电源。
- GND（引脚 4、29 及其他）：为芯片提供数字和模拟地线。

2、音频输入/输出接口

- 输入接口（LINE 和 MIC）：
 - LINE IN（引脚 23 和 21）：分别为 LIN1 和 LIN2，接入外部音频输入信号，通过 C135 和 C137 电容耦合。
 - MIC IN（引脚 22 和 21）：分别为 MIC2P 和 MIC2N，连接外部麦克风输入信号，通过 C137 和 C138 进行耦合，并附加了 1 μ F 电容以稳定信号。
 - HP GND（引脚 18）：耳机的地参考信号。
- 输出接口（ROUT 和 LOUT）：
 - ROUT 和 LOUT（引脚 14 和 15）：分别为右声道和左声道的音频输出，通过 0 Ω 电阻连接到负载或后续电路。

2、I2C 控制接口

- I2C 时钟和数据线（引脚 28 和 27）：
 - SCL（I2C 时钟）和 SDA（I2C 数据）通过 R181 上拉到 VCC3.3，用于配置 ES8388 寄存器。
- I2S 音频数据接口（引脚 5-8）：
 - MCLK（引脚 5）：主时钟输入。
 - SCLK（引脚 6）：I2S 比特时钟（也称为 BCLK）。
 - LRCK（引脚 8）：帧时钟，决定左右声道的数据时序。
 - DSDIN（引脚 7）：I2S 音频输入信号。

其他功能引脚

- VMID（引脚 19）：参考电压输出，提供 0.5*AVDD 的基准电压，外接去耦电容 C139。
- VREF（引脚 20）：模拟基准电压输出，外接去耦电容 C140。
- CE（引脚 26）：芯片使能控制信号，通过软件控制启动或关闭芯片。

器件说明

- L19/L21/L20：用于电源滤波的电感，阻隔高频噪声。
- C123/C126/C133/C134/C124/C125 等电容：去耦电容，用于滤除电源纹波，稳定电压。
- R175-R179、R181：用于连接 I2S 和 I2C 信号的串联电阻或上拉电阻。

功能总结

- 电源滤波电路：确保芯片的稳定供电。
- 音频输入/输出：支持耳机、麦克风和外部音频信号的输入输出。
- I2S 接口：传输音频数据，用于与主控芯片（如 MCU 或 SoC）通信。
- I2C 接口：通过 I2C 协议配置和控制 ES8388 的功能。

这张原理图展示了 ES8388 编解码芯片与外部电路的典型连接，适合用作音频输入/输出模块。

SPEAKER

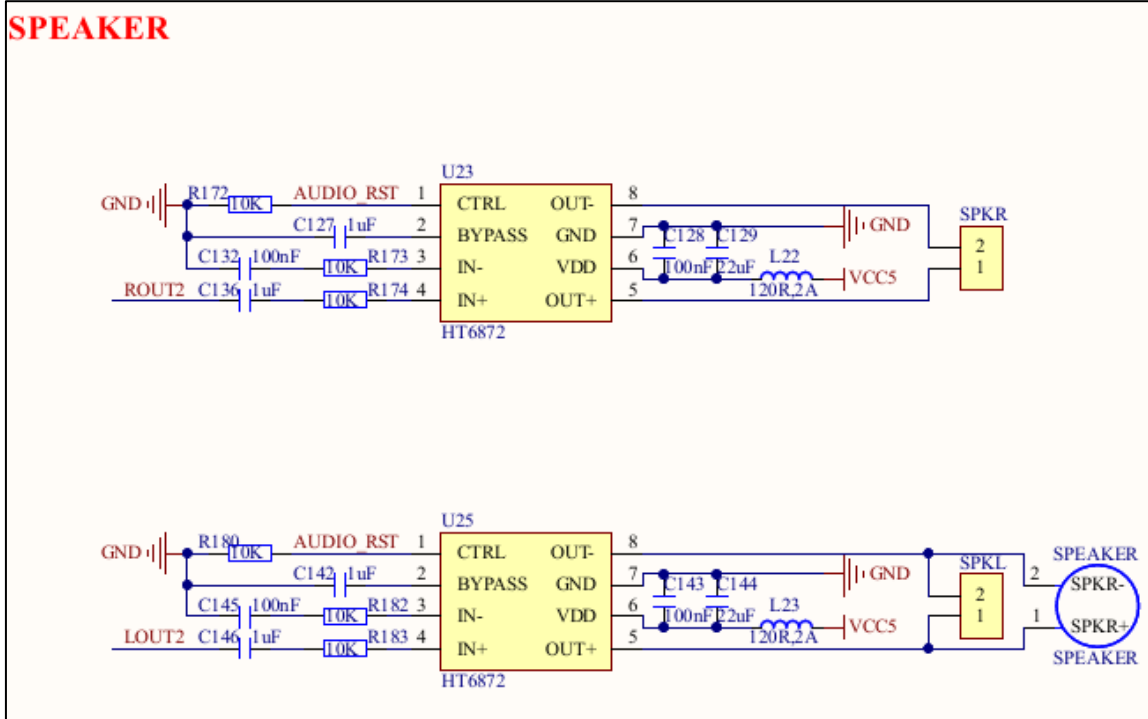


图 23.4-3 SPEAKER 部分原理图

该电路使用两颗 HT6872 音频功率放大器芯片,用于驱动左声道(SPKL)和右声道(SPKR)的扬声器输出,分别对应上半部分和下半部分电路。

1、电源部分

● VCC5 (5V 供电):

- 两颗芯片的 VDD (引脚 6) 通过电感 L22/L23 进行滤波,并搭配电容 C128/C129 和 C143/C144 (100nF+22 μF 组合) 去耦,以确保电源的稳定性,减少噪声干扰。
- 电源滤波网络由电感 (L22/L23) 和电容 (C128/C129、C143/C144) 组成,阻隔高频干扰。

- GND (地线): 为芯片提供电源地参考,所有地线在 PCB 布局中应尽量保持短路径以降低电阻。

2、信号输入部分

● 输入信号 (IN+、IN-):

- 上半部分 (右声道): 输入信号来自 ROUT2 (右声道输出),通过电容 C136 (1 μF) 进行交流耦合,再经过分压电阻网络 (R173、R174, 10k Ω) 进入 HT6872 的 IN+和 IN-引脚 (引脚 4 和 3)。C132 (100nF) 用于滤除高频干扰。
- 下半部分 (左声道): 输入信号来自 LOUT2 (左声道输出),通过 C146 (1 μF) 进行耦合,后续与分压电阻网络 (R182、R183, 10k Ω) 连接到芯片的 IN+和 IN-引脚。C145 (100nF) 同样用于高频信号滤波。

- AUDIO_RST 信号: 上拉到 VCC,通过 R172 和 R180 (10k Ω),用于控制音频电路的复位。

3、功率放大部分

● HT6872 (功放芯片):

- 上半部分电路使用 U23 驱动右声道扬声器 (SPKR),下半部分电路使用 U25 驱动左声道扬声器 (SPKL)。

- OUT+和 OUT-（引脚 5 和 7）为功放输出，驱动扬声器的正负端。
- BYPASS 引脚（引脚 2）接地，用于调节输入电压偏置，减少直流偏移。
- CTRL（引脚 1）：控制引脚，可通过外部逻辑控制功放状态（未具体接出）。

4、扬声器接口

- SPKR（右声道扬声器）：由 U23 的 OUT+（引脚 5）和 OUT-（引脚 7）驱动，直接连接扬声器正负极。
- SPKL（左声道扬声器）：由 U25 的 OUT+（引脚 5）和 OUT-（引脚 7）驱动，同样连接扬声器正负极。

5、滤波和保护

- L22 和 L23（120 Ω , 2.2A）：电感用于降低电源纹波及高频噪声对功放的影响。
- C128/C129/C143/C144（100nF+22 μ F）：去耦电容，为芯片提供低阻抗的电源路径。
- 耦合电容（C136/C146, 1 μ F）：用于交流信号的传输，同时隔离直流分量。

6、功能总结

- 信号输入耦合：通过 1 μ F 电容（C136、C146）将音频信号耦合至 HT6872，同时分压电阻（10k Ω ）用于调整输入信号幅度。
- 功率放大：HT6872 为 D 类音频功放，分别驱动左右声道扬声器，功率输出通过 OUT+ 和 OUT- 端子完成。
- 滤波与稳定：电源滤波电路（L22/L23+C128/C129 等）为功放提供干净的电源输入。
- 扬声器驱动：左右声道的扬声器（SPKL 和 SPKR）通过对应的功放芯片直接驱动，支持立体声输出。

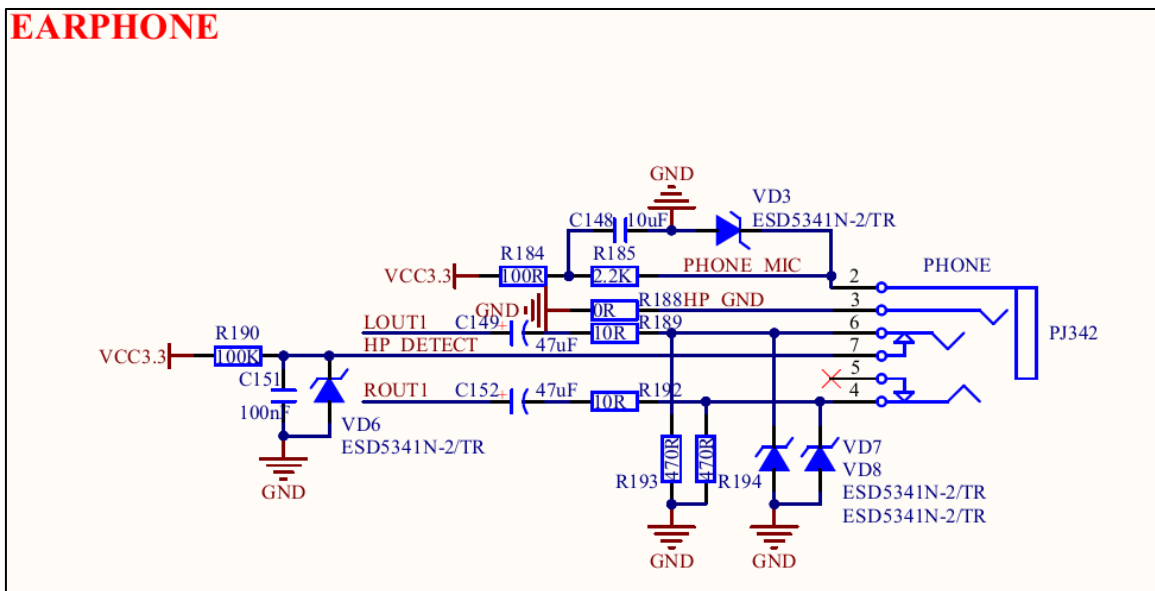


图 23.4-4 EARPHONE 部分原理图

该电路的核心功能是为耳机提供音频信号（左右声道）和麦克风接口，同时实现耳机插入检测，保障信号完整性和静电保护。

1、电源部分

- VCC3.3：为耳机部分的信号提供供电电压。



- 电阻 R190 (100k Ω) 提供耳机相关的下拉功能, 用于信号参考或检测。
- C149 (0.1 μ F) 用于电源去耦, 滤除高频干扰。

2、左声道 (LOUT1)

- 路径分析:
 - LOUT1 是左声道音频信号的输出, 来自音频编解码器(如 ES8388)的输出引脚。
 - 通过耦合电容 C151 (100nF), 用以隔直通交, 防止直流分量影响耳机。
 - 左声道信号连接到耳机插孔 (PJ342) 的 Pin 6。

3、右声道 (ROUT1)

- 路径分析:
 - ROUT1 是右声道音频信号的输出, 同样来自音频编解码器。
 - 通过耦合电容 C152 (47 μ F) 耦合到耳机插孔的 Pin 5。
 - 电容值较大 (47 μ F), 通常用于低频信号的传输。

4、麦克风 (PHONE_MIC)

- 路径分析:
 - PHONE_MIC 信号通过保护二极管 VD3 (ESD5341N-2/TR) 实现静电防护。
 - 信号线上还串联了下拉电阻 R185 (2.2k Ω), 用于偏置麦克风信号, 避免浮空或不稳定。
 - 最终连接到耳机插孔的 Pin 2, 提供麦克风输入功能。

5、耳机插入检测 (HP_DETECT)

- 实现方式:
 - HP_DETECT 信号通过插孔 (PJ342) 的 Pin 7 实现检测功能。
 - 插入耳机后, Pin 7 的电平状态会发生变化, 通过 R188 (0 Ω) 和 R189 (10 Ω) 连接到控制器的检测引脚, 用于识别耳机插入状态。

6、静电保护

- 保护元件: VD6、VD7、VD8 (ESD5341N-2/TR) 分别用于左右声道、麦克风和检测信号的静电保护。这些二极管能有效地吸收静电放电能量, 保护后端的敏感电路。

7、耳机插孔 (PJ342)

- 插孔支持三段式耳机 (左右声道+麦克风)。
- 引脚功能:
 - Pin 6: 左声道音频信号。
 - Pin 5: 右声道音频信号。
 - Pin 2: 麦克风输入信号。
 - Pin 7: 耳机插入检测。
 - Pin 4: 接地 (GND)。

MIC

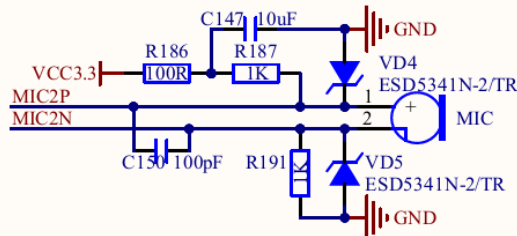


图 23.4-5 MIC 部分原理图

该电路用于支持差分麦克风的输入功能，同时具备抗静电保护的特性。

1、电源部分

- VCC3.3: 为麦克风电路提供供电电压。
 - 电阻 R186 (100Ω) 用于限制电流，保护麦克风模块。
 - 耦合电容 C147 (10μF) 用于稳定电源，滤除电源纹波和高频噪声，保障电源稳定性。

2、差分输入 (MIC2P、MIC2N)

- MIC2P 和 MIC2N: 分别是麦克风的正向输入和负向输入信号。
 - C150 (100pF): 差分输入信号的去耦电容，用于滤除高频干扰。
 - R191: 该位置是一个未标注阻值的电阻，可能用于阻抗匹配或偏置功能。

3、静电保护 (VD4 和 VD5)

- 保护元件: VD4 和 VD5 均为 ESD5341N-2/TR 静电保护二极管，分别用于正向和负向信号线的保护。这些二极管有效地吸收静电放电能量，避免静电对后端电路造成损害。
- 连接方式: 二极管的正极接信号输入，负极接地，能够有效保护 MIC2P 和 MIC2N 信号线。

4、信号输出 (MIC)

- 麦克风信号经过差分处理后，输出到 MIC 引脚。
- R187 (1kΩ): 用于为麦克风电路提供偏置电流，稳定麦克风的工作状态。

23.4.2 硬件实物

开发板上的 Audio 相关的电路位置如下图所示:

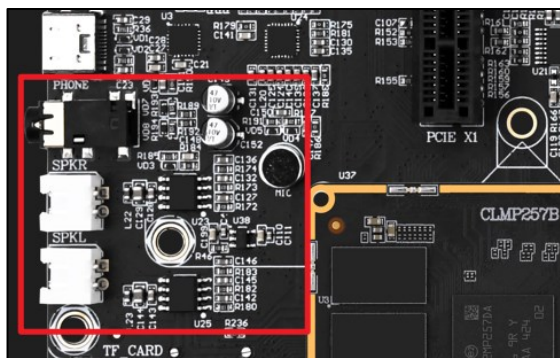


图 23.4-6 Audio 电路实物

开发板板载 1 个耳机接口。耳机座规格为 PJ-342 单柱带 T，该接口可以插 4 段式 3.5mm 的耳机，支持录音与放音。放音时，支持耳机热插拔；录音时，支持耳机录音和板载麦克风 MIC 两种方式进行录音。

开发板板载 2 路扬声器外接接口，其中一路扬声器接口 SPKR 已在开发板接入一个小扬声器，方便用户进行音频播放测试。

开发板板载 1 个驻极体麦克风，即录音输入口（MIC），可以用来实现录音功能。

开发板背面板载 1 个外置扬声器，扬声器型号为 GSPK2307P-8R1W，功率为 8Ω、1W，供用户进行音频播放测试。当音频芯片 ES8388 播放音频时，开发板支持从扬声器播放及耳机播放。在不插入耳机时，开发板自动从外置扬声器进行播放；当插入耳机时，会自动切换到耳机播放音频，此时扬声器自动停止播放。该设计符合用户常规使用体验。

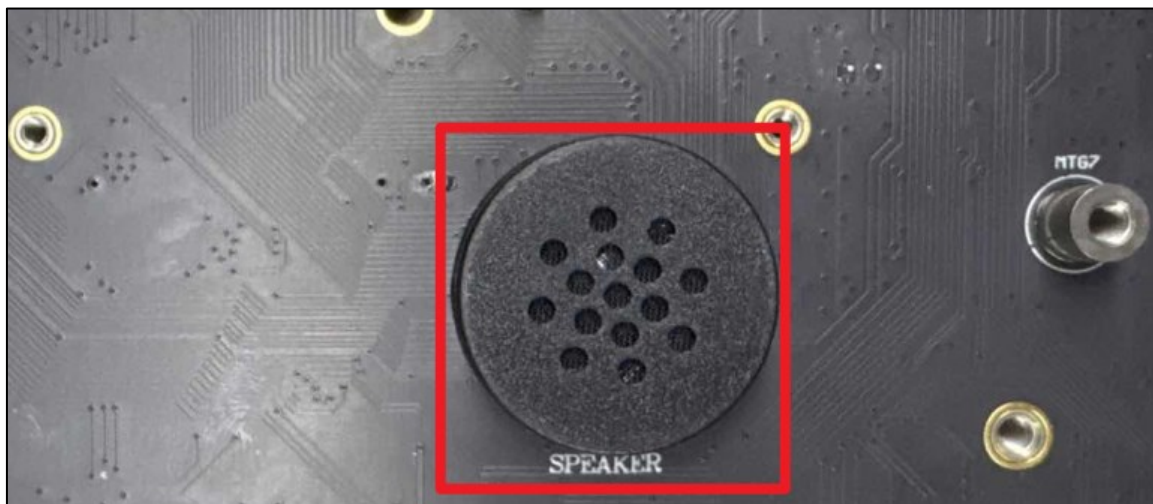


图 23.4-7 开发板背面扬声器

顺芯 ES8388 是一款高性能、低功耗、低成本的立体声音频编解码器，该芯片内部集成了 24 位高性能 ADC 和 DAC，由双通道 ADC、双通道 DAC、麦克风放大器、耳机放大器、数字音效以及模拟混音和增益功能组成，支持 8KHZ~96KHz 采样率，可通过 I2C 接口进行配置。

顺芯 ES8388 编解码芯片内部没有功放功能，故需要外置功放芯片才能进行扬声器放音。开发板采用 HT6872 单声道 D 类音频功率放大器，来驱动外部扬声器。开发板板载了一个 8Ω 1W 扬声器（在背面），通过功放芯片 HT6872 连接到 ES8388 的 ROUT2 通道，提供给用户进行音频测试，同时还对外提供 SPKR 及 SPKL 两个扬声器连接端子（XH2.54 标准接口），方便用户进行扬声器拓展连接。

除了扬声器可以放音, 开发板还支持耳机放音。当 ES8388 播放音频时, 在不插入耳机情况下, 开发板自动从外置扬声器进行播放; 当插入耳机时, 会自动切换到耳机播放, 此时扬声器自动停止播放。该设计符合用户常规使用体验。开发板板载耳机接口 PHONE 采用一个四段式 3.5mm 耳机接口 PJ342, 支持放音、录音及热插拔检测。用户在设计此接口时, 需注意所选耳机座物料规格是否支持 4 段式。也可以按项目需要进行设计。

录音有两种选择, 一种是耳机自带的 MIC, 一种是开发板板载的驻极体麦克风, 同一时刻只能二选一。

23.5 内核 Audio 相关使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig 中, Audio 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_SOUND=y //启用通用声音子系统, 允许内核支持音频功能
2 CONFIG_SND=y //启用高级 Linux 声音架构 ALSA, 为音频设备提供内核支持
3 CONFIG_ARCH_STM32=y //指定使用 STM32 架构, 内核会加载与 STM32 相关驱动和支持库
4 CONFIG_OF=y //开启设备树支持
5 CONFIG_COMMON_CLK=y //启用通用时钟框架, 提供对时钟管理的支持
6 CONFIG_SND_SOC=y //启用 ALSA 的声音开放式组件
7 CONFIG_SND_SOC_STM32_SAI=y //启用 STM32 SAI 驱动支持
8 CONFIG_SND_SOC_STM32_I2S=y //启用 STM32 I2S 驱动支持
9 CONFIG_SND_SOC_STM32_SPDIFRX=y //启用 STM32 SPDIFRX 支持
10 CONFIG_STM32_MDF_ADC=y //启用 STM32 的多功能数字滤波器
11 CONFIG_SND_SOC_STM32_MDF=y //启用 MDF 音频子系统支持
12 CONFIG_SND_SOC_DMIC=y //启用数字麦克风支持
13 CONFIG_SND_SIMPLE_CARD_UTILS=y //启用简单音频卡工具
14 CONFIG_SND_AUDIO_GRAPH_CARD=y //启用音频图声卡支持
15 CONFIG_IIO_BUFFER_CB=y //启用 IIO (工业 I/O) 缓冲区回调支持
16 CONFIG_IIO_BUFFER_HW_CONSUMER=y //启用 IIO 硬件缓冲区消费者支持
17 CONFIG_IIO_BACKEND=y //启用 IIO 后端支持
18 CONFIG_STM32_MDF_CORE=y //启用 STM32 的多功能数字滤波器核心
```

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

```
[*] Device Drivers
  [*] Sound card support
    [*] Advanced Linux Sound Architecture
      [*] ALSA for SoC audio support
        STMicroelectronics STM32 SOC audio support
          <*> STM32 SAI interface (Serial Audio Interface) support
          <*> STM32 I2S interface (SPI/I2S block) support
          <*> STM32 S/PDIF receiver (SPDIFRX) support
          <*> SoC Audio support for STM32 MDF
        CODEC drivers
          [ ] ...
          [*] ASoC Audio Graph sound card support
```

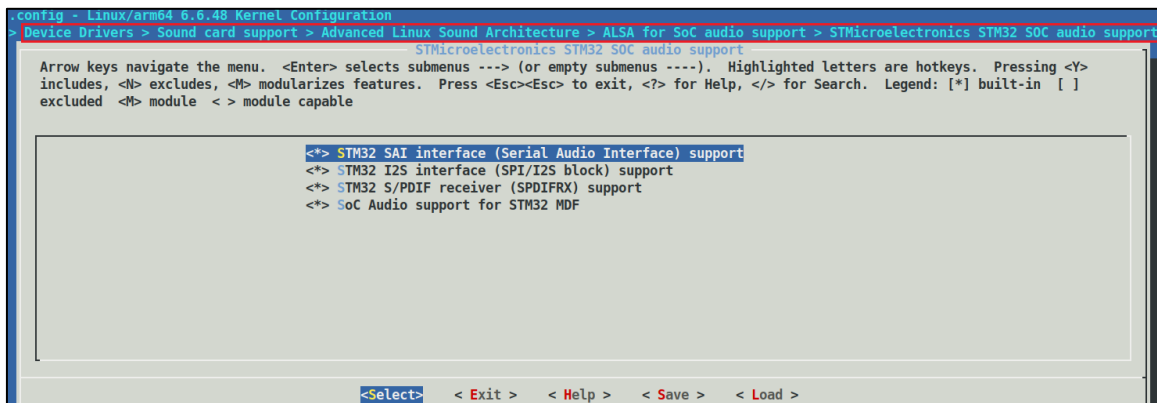


图 23.5-1 Audio 配置节选

23.6 设备树分析

打开 arch/arm64/boot/dts/st/stm32mp251.dtsi, 此文件是特定于 STM32MP2 系列芯片的设备树包含文件, 通常不独立使用, 而是被包含在其他 .dts 设备树文件中。这样的结构允许多个 .dts 文件共享相同的基础配置, 而 .dtsi 则作为它们的公共部分。stm32mp251.dtsi 中关于 Audio 描述如下:

```

1  sai1: sai@40290000 {
2      compatible = "st,stm32mp25-sai";
3      reg = <0x40290000 0x4>, <0x4029a3f0 0x10>;
4      ranges = <0 0x40290000 0x400>;
5      #address-cells = <1>;
6      #size-cells = <1>;
7      clocks = <&rcc CK_BUS_SAI1>;
8      clock-names = "pclk";
9      interrupts = <GIC_SPI 157 IRQ_TYPE_LEVEL_HIGH>;
10     resets = <&rcc SAI1_R>;
11     access-controllers = <&rifsc 49>;
12     status = "disabled";
13
14     saila: audio-controller@40290004 {
15         compatible = "st,stm32-sai-sub-a";
16         reg = <0x4 0x20>;
17         #sound-dai-cells = <0>;
18         clocks = <&rcc CK_KER_SAI1>;
19         clock-names = "sai_ck";
20         dmas = <&hpdma 73 0x43 0x21>;
21         dma-names = "tx";
22         power-domains = <&CLUSTER_PD>;
23         status = "disabled";
24     };
25
26     sailb: audio-controller@40290024 {

```



```

27     compatible = "st,stm32-sai-sub-b";
28     reg = <0x24 0x20>;
29     #sound-dai-cells = <0>;
30     clocks = <&rcc CK_KER_SAI1>;
31     clock-names = "sai_ck";
32     dmas = <&hpdma 74 0x43 0x12>;
33     dma-names = "rx";
34     power-domains = <&CLUSTER_PD>;
35     status = "disabled";
36 };
37 };

```

第 1 行, sai:sai@40290000 定义了设备节点, 名称为 sai1, 设备地址为 0x40290000, 对应的是 STM32MP25 的 SAI (串行音频接口) 硬件模块。

第 2 行, compatible = "st,stm32mp25-sai" 表示该设备节点与 st,stm32mp25-sai 设备驱动程序兼容, 驱动程序将识别并加载适配该设备。

第 3 行, reg = <0x40290000 0x4>, <0x4029a3f0 0x10> 定义了该设备的寄存器范围:

- 第一个寄存器块的起始地址为 0x40290000, 大小为 0x4 字节。
- 第二个寄存器块的起始地址为 0x4029a3f0, 大小为 0x10 字节。

第 4 行, ranges = <0 0x40290000 0x400> 定义了地址空间映射, SAI 的地址范围从 0x40290000 开始, 大小为 0x400 字节。

第 5 行, #address-cells = <1> 定义子节点中地址字段占用的单元数为 1。

第 6 行, #size-cells = <1> 定义子节点中大小字段占用的单元数为 1。

第 7 行, clocks = <&rcc CK_BUS_SAI1> 定义了时钟源, SAI1 使用 CK_BUS_SAI1 作为其总线时钟。

第 8 行, clock-names = "pclk" 为时钟定义名称 pclk (外设时钟)。

第 9 行, interrupts = <GIC_SPI 157 IRQ_TYPE_LEVEL_HIGH> 定义了中断配置: 使用 GIC 的 SPI 中断线号 157, 触发类型为高电平中断。

第 10 行, resets = <&rcc SAI1_R> 定义了复位控制, SAI1_R 是该 SAI 设备的复位信号。

第 11 行, access-controllers = <&rifsc 49> 指定访问控制器, rifsc 表示资源隔离单元, 其 ID 为 49。

第 12 行, status = "disabled" 当前节点状态为禁用, 需要启用时改为 "okay"。

第 14 行, sai1a: audio-controller@40290004 定义了 SAI1 的子节点 A, 地址偏移为 0x4。

第 15 行, compatible = "st,stm32-sai-sub-a" 子节点 A 与 st,stm32-sai-sub-a 驱动程序兼容。

第 16 行, reg = <0x4 0x20> 子节点 A 的寄存器范围为从偏移地址 0x4 开始, 大小为 0x20 字节。

第 17 行, #sound-dai-cells = <0> 定义音频接口的 Dai (Digital Audio Interface) 配置无参数。

第 18 行, clocks = <&rcc CK_KER_SAI1> 指定子节点 A 的时钟为 CK_KER_SAI1。

第 19 行, clock-names = "sai_ck" 定义时钟名称为 sai_ck。

第 20 行, dmas = <&hpdma 73 0x43 0x21> 配置 DMA 控制: 使用 hpdma 控制器, 通道为 73, 0x43 和 0x21 为具体的 DMA 配置信息。

第 21 行, dma-names = "tx" 定义 DMA 的名称为 tx (传输)。

第 22 行, power-domains = <&CLUSTER_PD> 指定电源域, CLUSTER_PD 为电源管理单元。

第 23 行, status = "disabled" 子节点 A 状态为禁用。



第 26 行, sailb: audio-controller@40290024 定义了 SAI1 的子节点 B, 地址偏移为 0x24。

第 27 行, compatible = "st,stm32-sai-sub-b" 子节点 B 与 st,stm32-sai-sub-b 驱动程序兼容。

第 28 行, reg = <0x24 0x20>子节点 B 的寄存器范围为从偏移地址 0x24 开始, 大小为 0x20 字节。

第 29 行, #sound-dai-cells = <0> 定义音频接口的 Dai 配置无参数。

第 30 行, clocks = <&rcc CK_KER_SAI1> 指定子节点 B 的时钟为 CK_KER_SAI1。

第 31 行, clock-names = "sai_ck" 定义时钟名称为 sai_ck。

第 32 行, dmas = <&hpdma 74 0x43 0x12> 配置 DMA 控制: 使用 hpdma 控制器, 通道为 74, 0x43 和 0x12 为具体的 DMA 配置信息。

第 33 行, dma-names = "rx" 定义 DMA 的名称为 rx (接收)。

第 34 行, power-domains = <&CLUSTER_PD> 指定电源域, CLUSTER_PD 为电源管理单元。

第 35 行, status = "disabled" 子节点 B 状态为禁用。

以 2GB 版本核心板为例, 在 arch/arm64/boot/dts/st/stm32mp257d-atk-ddr-2GB.dts 中, Audio 相关配置如下:

```
1  &i2c4 {
2      es8388: es8388@11 {
3          compatible = "alientek,es8388";
4          reg = <0x11>;
5          #sound-dai-cells = <0>;
6          spk-con-gpios = <&gpio12 GPIO_ACTIVE_LOW>;
7          hp-det-gpios = <&gpio13 GPIO_ACTIVE_LOW>;
8          clocks = <&sailb>;
9          clock-names = "MCLK";
10         DVDD-supply = <&vdd_3v3>;
11         PVDD-supply = <&vdd_3v3>;
12         AVDD-supply = <&vdd_3v3>;
13         HPVDD-supply = <&vdd_3v3>;
14         status = "okay";
15
16         port {
17             #address-cells = <1>;
18             #size-cells = <0>;
19
20             es8388_tx_endpoint: endpoint@0 {
21                 reg = <0>;
22                 remote-endpoint = <&sailb_endpoint>;
23                 frame-master;
24                 bitclock-master;
25             };
26
27             es8388_rx_endpoint: endpoint@1 {
28                 reg = <1>;
```



```
29         remote-endpoint = <&saila_endpoint>;
30         frame-master;
31         bitclock-master;
32     };
33 };
34 };
35 };
36
37 &sail {
38     clocks = <&rcc CK_BUS_SAI1>;
39     clock-names = "pclk";
40     pinctrl-names = "default", "sleep";
41     pinctrl-0 = <&saila_pins_a>, <&sailb_pins_b>;
42     pinctrl-1 = <&saila_sleep_pins_a>, <&sailb_sleep_pins_b>;
43     status = "okay";
44
45     saila: audio-controller@40290004 {
46         dma-names = "rx";
47         clocks = <&rcc CK_KER_SAI1>, <&sailb>;
48         clock-names = "sai_ck", "MCLK";
49         dmas = <&hpdma 73 0x43 0x12>;
50         st, sync = <&sailb 2>;
51         status = "okay";
52
53         saila_port: port {
54             saila_endpoint: endpoint {
55                 remote-endpoint = <&es8388_rx_endpoint>;
56                 dai-format = "i2s";
57                 mclk-fs = <256>;
58                 dai-tdm-slot-num = <2>;
59                 dai-tdm-slot-width = <32>;
60             };
61         };
62     };
63
64     sailb: audio-controller@40290024 {
65         #clock-cells = <0>;
66         dma-names = "tx";
67         clock-names = "sai_ck";
68         dmas = <&hpdma 74 0x43 0x21>;
69         status = "okay";
70
71         sailb_port: port {
```

```

72     sailb_endpoint: endpoint {
73         remote-endpoint = <&es8388_tx_endpoint>;
74         dai-format = "i2s";
75         mclk-fs = <256>;
76         dai-tdm-slot-num = <2>;
77         dai-tdm-slot-width = <32>;
78     };
79 };
80 };
81 };

```

第 1 行, &i2c4 表示对 I2C4 设备节点的扩展, 用于在 I2C 总线上挂载子设备。

第 2 行, es8388: es8388@11 定义了挂载在 I2C4 上的设备节点, 名称为 es8388, I2C 地址为 0x11。

第 3 行, compatible="alientek,es8388" 指定设备兼容属性, 与驱动程序匹配。alientek,es8388 表示支持的 ES8388 音频编解码器。

第 4 行, reg = <0x11> 定义 I2C 设备地址为 0x11。

第 5 行, #sound-dai-cells = <0> 定义 Dai 接口所需的参数数量为 0。

第 6 行, spk-con-gpios = <&gpio12 GPIO_ACTIVE_LOW> 定义了扬声器控制的 GPIO, 使用 GPIO12, 低电平有效。

第 7 行, hp-det-gpios = <&gpio13 GPIO_ACTIVE_LOW> 定义了耳机检测的 GPIO, 使用 GPIO13, 低电平有效。

第 8 行, clocks = <&sailb> 指定音频编解码器使用的时钟来自 sailb 节点。

第 9 行, clock-names = "MCLK" 为时钟命名为 MCLK (主时钟)。

第 10~13 行, 电源供应定义:

DVDD-supply = <&vdd_3v3>; DVDD 供电来自 vdd_3v3 电源节点。

PVDD-supply = <&vdd_3v3>; PVDD 供电来自 vdd_3v3 电源节点。

AVDD-supply = <&vdd_3v3>; AVDD 供电来自 vdd_3v3 电源节点。

HPVDD-supply = <&vdd_3v3>; HPVDD 供电来自 vdd_3v3 电源节点。

第 14 行, status = "okay" 启用该节点。

第 16 行, port 定义 ES8388 的音频端口, 用于与音频控制器连接。

第 17~18 行, #address-cells 和 #size-cells 定义子节点中地址和大小字段的格式: 地址占 1 单元, 大小占 0 单元。

第 20 行, es8388_tx_endpoint: endpoint@0 定义 ES8388 的发送端点 (TX), 子节点标识为 endpoint@0。

第 21 行, reg = <0> 该端点的编号为 0。

第 22 行, remote-endpoint = <&sailb_endpoint> 指定该端点与 sailb_endpoint 远程端点相连。

第 23~24 行, 主控配置:

- frame-master; 定义 ES8388 为帧主控。
- bitclock-master; 定义 ES8388 为位时钟主控。

第 27 行, es8388_rx_endpoint: endpoint@1 定义 ES8388 的接收端点 (RX), 子节点标识为 endpoint@1。

第 28 行, reg = <1>; 该端点的编号为 1。



第 29 行, `remote-endpoint = <&saila_endpoint>`; 指定该端点与 `saila_endpoint` 远程端点相连。

第 30~31 行, 主控配置:

- `frame-master`; 定义 ES8388 为帧主控。
- `bitclock-master`; 定义 ES8388 为位时钟主控。

第 37 行, `&sail` 对 SAI1 设备节点的扩展。

第 38~39 行, 时钟配置:

- `clocks = <&rcc CK_BUS_SAI1>`; 指定 SAI1 总线时钟为 `CK_BUS_SAI1`。
- `clock-names = "pclk"`; 时钟名称为 `pclk`。

第 40~42 行, 引脚控制:

- `pinctrl-names = "default", "sleep"`; 定义两种引脚控制状态: `default` 和 `sleep`。
- `pinctrl-0 = <&saila_pins_a>, <&sailb_pins_b>`; 默认状态下使用的引脚配置。
- `pinctrl-1 = <&saila_sleep_pins_a>, <&sailb_sleep_pins_b>`; 休眠状态下使用的引脚配置。

第 43 行, `status = "okay"`; 启用 SAI1 节点。

第 45 行, `saila: audio-controller@40290004` 定义 SAI1 的子节点 A, 地址偏移为 `0x4`。

第 46~48 行, DMA 与时钟配置:

- `dma-names = "rx"`; 定义 DMA 名称为 `rx` (接收)。
- `clocks = <&rcc CK_KER_SAI1>, <&sailb>`; 指定两个时钟源: `CK_KER_SAI1` 和 `sailb`。
- `clock-names = "sai_ck", "MCLK"`; 时钟名称分别为 `sai_ck` 和 `MCLK`。

第 49 行, `dmass = <&hpdma 73 0x43 0x12>`; 配置 DMA 控制器, 使用 `hpdma` 的通道 73, DMA 配置为 `0x43` 和 `0x12`。

第 50 行, `st, sync = <&sailb 2>`; 指定与 `sailb` 子节点的同步模式。

第 51~63 行, 端口配置:

- `saila_port: port` 定义子节点 A 的音频端口。
- `saila_endpoint: endpoint` 定义端点, 连接到 `es8388_rx_endpoint`。
- 音频参数配置:
 - `dai-format = "i2s"`; 定义音频格式为 I2S。
 - `mclk-fs = <256>`; 主时钟与采样率的比值为 256。
 - `dai-tdm-slot-num = <2>`; TDM 时隙数为 2。
 - `dai-tdm-slot-width = <32>`; 每个 TDM 时隙宽度为 32 位。

第 64 行, `sailb: audio-controller@40290024` 定义 SAI1 的子节点 B, 地址偏移为 `0x24`。

第 65 行, `#clock-cells = <0>` 定义时钟单元数量为 0。

第 66~67 行, DMA 与时钟配置:

- `dma-names = "tx"`; 定义 DMA 名称为 `tx` (发送)。
- `clock-names = "sai_ck"`; 时钟名称为 `sai_ck`。

第 68 行, `dmass = <&hpdma 74 0x43 0x21>`; 配置 DMA 控制器, 使用 `hpdma` 的通道 74, DMA 配置为 `0x43` 和 `0x21`。

第 71~80 行, 端口配置:

- `sailb_port: port` 定义子节点 B 的音频端口。
- `sailb_endpoint: endpoint` 定义端点, 连接到 `es8388_tx_endpoint`。
- 音频参数配置:
 - `dai-format = "i2s"`; 定义音频格式为 I2S。

- `mclk-fs = <256>;`: 主时钟与采样率的比值为 256。
- `dai-tdm-slot-num = <2>;`: TDM 时隙数为 2。
- `dai-tdm-slot-width = <32>;`: 每个 TDM 时隙宽度为 32 位。

23.7 ALSA 工具

23.7.1 alsamixer

`alsamixer` 是 ALSA 的混音器控制工具，可以在命令行中提供一个图形化界面，用于调整音量和配置音频设备。下面是如何使用 `alsamixer` 的一些基本示例：

启动 alsamixer

在终端中运行以下命令来启动 `alsamixer`：

```
alsamixer
```

此时会看到一个带有音量控制条的图形化界面，通过键盘可以进行导航和调整。



图 23.7-1 alsamixer 界面

基本键盘操作

- 方向键左/右：选择不同的控制条（例如音量、麦克风增益等）。
- 方向键上/下：调整选定控制条的音量。

- m 键: 切换静音/取消静音。
- F1: 显示帮助, 可能会有键位冲突, 按 “?” 键或者 “H” 也可以调出。
- F2: 显示系统信息, 可能会有键位冲突, 按 “/” 键也可以调出。
- F3: 显示播放控制, 可能会有键位冲突。
- F4: 显示录音控制, 可能会有键位冲突。
- F5: 显示所有控制, 可能会有键位冲突。
- F6: 显示用于选择声卡, 可能会有键位冲突, 按 “S” 键也可以调出
- Esc: 退出 alsamixer。

```

----- Help -----
Esc      Exit
F1 ? H   Help
F2 /     System information
F3       Show playback controls
F4       Show capture controls
F5       Show all controls
Tab      Toggle view mode (F3/F4/F5)
F6 S     Select sound card
L        Redraw screen

Left     Move to the previous control
Right    Move to the next control

Up/Down  Change volume
+ -      Change volume
Page Up/Dn Change volume in big steps
End      Set volume to 0%
0-9      Set volume to 0%-90%
Q W E    Increase left/both/right volumes
Z X C    Decrease left/both/right volumes
B        Balance left and right volumes

M        Toggle mute
< >     Toggle left/right mute

Space    Toggle capture
; '      Toggle left/right capture

Authors:
  Tim Janik
  Jaroslav Kysela <perex@perex.cz>
  Clemens Ladisch <clemens@ladisch.de>
    
```

图 23.7-2 帮助信息

23.7.2 amixer

amixer 是 Linux 系统中的一个命令行工具, 用于配置和控制高级 Linux 声音架构 (ALSA) 的音频设备。它能够显示和修改混音器控件, 包括音量、音频通道的选择、静音控制以及高级音频设置。

开发板执行指令:

```
amixer
```

```

root@ATK-DLMP257:~# amixer
Simple mixer control 'PCM',0
  Capabilities: volume
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 192
  Front Left: 152 [79%] [-20.00dB]
  Front Right: 152 [79%] [-20.00dB]
Simple mixer control 'Mono Mux',0
  Capabilities: enum
  Items: 'Stereo' 'Mono (Left)' 'Mono (Right)'
  Item0: 'Stereo'
Simple mixer control 'Playback De-emphasis',0
  Capabilities: enum
  Items: 'None' '32Khz' '44.1Khz' '48Khz'
  Item0: 'None'
Simple mixer control 'Capture Digital',0
  Capabilities: volume
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 192
  Front Left: 192 [100%] [0.00dB]
  Front Right: 192 [100%] [0.00dB]
Simple mixer control 'Capture Mute',0
  Capabilities: pswitch pswitch-joined
  Playback channels: Mono
  Mono: Playback [off]
Simple mixer control 'Capture Polarity',0
  Capabilities: enum
  Items: 'Normal' 'L Invert' 'R Invert' 'L + R Invert'
  Item0: 'Normal'

```

以下省略

图 23.7-3 amixer 输出信息

从 amixer 输出来看，这是一段嵌入式系统中音频设备的混音器配置。具体信息包括音频通道的配置、增益控制、捕捉和播放的音量设置等。这些配置控制了音频系统的不同部分，如立体声、单声道切换，3D 效果模式，以及自动电平控制 (ALC) 等功能。

主要混音器控制项解析

● PCM

控制项：音量

左右声道：前左 (Front Left) 和前右 (Front Right)

范围：0 到 192

当前设置：152 (79%, -20.00dB)

● Mono Mux

控制项：音频输入模式选择

可选项：Stereo (立体声)、Mono (Left) (单声道左声道)、Mono (Right) (单声道右声道)

当前设置：Stereo

● Playback De-emphasis

控制项：播放去加重 (常用于特定采样率的去加重滤波)

可选项：None (无)、32Khz、44.1Khz、48Khz

当前设置：None

● Capture Digital

控制项：捕捉 (录音) 音量

范围：0 到 192

左右声道：前左 (Front Left) 和前右 (Front Right)

当前设置：192 (100%, 0.00dB)



- Capture Mute

控制项: 捕捉静音开关

当前设置: off (关闭)

- 3D Mode

控制项: 3D 音效模式

可选项: No 3D、Level 1 到 Level 7

当前设置: No 3D

- ALC Capture 系列控制项

控制项: 自动电平控制 (ALC), 用于动态调节录音增益

包含: 攻击时间、衰减时间、最大/最小增益、噪声门限等设置

当前主要设置:

ALC 功能: Off (关闭)

最大 PGA: 7 (100%)

目标电平: 11 (73%)

在开发板的 /home/root/shell/audio 目录下, 有一个 `atk_audio.sh` 脚本, 这个脚本功能多样, 主要用于初始化和配置音频设备, 并执行录音和播放测试。它包括了设备初始化、模式切换、麦克风配置等多个步骤。脚本解析如下:

1、脚本变量定义

```
#!/bin/bash

INIT_FLAG="/home/root/shell/audio/.initialized_audio_device"
RECORD_FILE="./record.wav"
PLAY_FILE="./sunshine.mp3"
```

- INIT_FLAG: 初始化标志文件, 用于判断音频设备是否已初始化。
- RECORD_FILE: 录音保存的文件路径。
- PLAY_FILE: 播放音频文件的路径。

2、检查命令是否存在

```
# 检查命令是否存在

check_command() {
    command -v "$1" > /dev/null 2>&1
}
```

`check_command` 函数用于检测指定命令是否存在。它通过 `command -v` 判断命令是否在系统中可用, 并返回结果。

3、音频设备初始化

```
# 初始化音频设备

init_audio_device() {
    init_audio_device() {
        amixer cset name='PCM Volume' 192 > /dev/null 2>&1
```



```

amixer cset name='Mono Mux' 'Stereo' > /dev/null 2>&1
amixer cset name='Playback De-emphasis' 2 > /dev/null 2>&1
amixer cset name='Capture Digital Volume' 192 > /dev/null 2>&1
amixer cset name='Capture Mute' 'on' > /dev/null 2>&1
amixer cset name='Capture Polarity' 'Normal' > /dev/null 2>&1
amixer cset name='3D Mode' 'No 3D' > /dev/null 2>&1
amixer cset name='ALC Capture Attack Time' 5 > /dev/null 2>&1
amixer cset name='ALC Capture Decay Time' 2 > /dev/null 2>&1
amixer cset name='ALC Capture Function' 'Stereo' > /dev/null 2>&1
amixer cset name='ALC Capture Hold Time' 2 > /dev/null 2>&1
amixer cset name='ALC Capture Max PGA' 3 > /dev/null 2>&1
amixer cset name='ALC Capture Min PGA' 6 > /dev/null 2>&1
amixer cset name='ALC Capture NG Switch' 'on' > /dev/null 2>&1
amixer cset name='ALC Capture NG Threshold' 9 > /dev/null 2>&1
amixer cset name='ALC Capture NG Type' 'Mute ADC Output' > /dev/null
2>&1
amixer cset name='ALC Capture Target Volume' 15 > /dev/null 2>&1
amixer cset name='ALC Capture ZC Switch' 'on' > /dev/null 2>&1
amixer cset name='Left Channel Capture Volume' 100% > /dev/null 2>&1
amixer cset name='Right Channel Capture Volume' 100% > /dev/null
2>&1
amixer cset name='Left Mixer Left Bypass Volume' 100% > /dev/null
2>&1
amixer cset name='Right Mixer Right Bypass Volume' 100% > /dev/null
2>&1
amixer cset name='Output 1 Playback Volume' 100% > /dev/null 2>&1
amixer cset name='Output 2 Playback Volume' 100% > /dev/null 2>&1
amixer cset name='ZC Timeout Switch' 'on' > /dev/null 2>&1
amixer cset name='Left PGA Mux' 'DifferentialL' > /dev/null 2>&1
amixer cset name='Right PGA Mux' 'DifferentialR' > /dev/null 2>&1
}
}

```

- `amixer cset name='PCM Volume' 192 > /dev/null 2>&1`

描述: 设置 PCM (Pulse Code Modulation) 音量。

作用: PCM 是音频信号经过采样和量化后的数字表示。这里的设置值为 192, 具体取值范围根据音频设备的设计而定。

- `amixer cset name='Mono Mux' 'Stereo' > /dev/null 2>&1`

描述: 设置单声道 (Mono) 的多路选择器为立体声 (Stereo)。

作用: 使得单声道信号通过立体声输出, 适用于一些特定的音频设备配置。

- `amixer cset name='Playback De-emphasis' 2 > /dev/null 2>&1`

描述: 设置播放去加重 (De-emphasis)。

作用: 去加重是用于补偿在录制过程中引入的高频部分的增强, 通常用于处理旧的音频格式。设置值 2 对应于特定的去加重曲线。



- `amixer cset name='Capture Digital Volume' 192 > /dev/null 2>&1`
描述: 设置录音时的数字音量。
作用: 控制录音的增益 (gain), 避免录音音量过小或过大。
- `amixer cset name='Capture Mute' 'on' > /dev/null 2>&1`
描述: 静音录音输入。
作用: 用于暂时禁止录音输入, 可以在不需要录音时使用。
- `amixer cset name='Capture Polarity' 'Normal' > /dev/null 2>&1`
描述: 设置录音极性为正常。
作用: 控制录音信号的相位, Normal 意味着使用默认相位。
- `amixer cset name='3D Mode' 'No 3D' > /dev/null 2>&1`
描述: 关闭 3D 模式。
作用: 3D 模式通常用于增强立体声效果, 在这里被关闭。
- `amixer cset name='ALC Capture Attack Time' 5 > /dev/null 2>&1`
描述: 设置自动增益控制 (ALC) 的捕获攻击时间为 5。
作用: 控制自动增益开始生效的速度。值越小, 响应越快。
- `amixer cset name='ALC Capture Decay Time' 2 > /dev/null 2>&1`
描述: 设置自动增益控制的衰减时间为 2。
作用: 控制增益减小的速度。值越小, 增益衰减得越快。
- `amixer cset name='ALC Capture Function' 'Stereo' > /dev/null 2>&1`
描述: 将自动增益控制设置为立体声模式。
作用: ALC 在立体声录制时同时作用于两个声道。
- `amixer cset name='ALC Capture Hold Time' 2 > /dev/null 2>&1`
描述: 设置自动增益保持时间为 2。
作用: 控制增益在多长时间保持不变。
- `amixer cset name='ALC Capture Max PGA' 3 > /dev/null 2>&1`
描述: 设置自动增益控制的最大 PGA (Programmable Gain Amplifier) 增益为 3。
作用: 限制自动增益的上限, 避免录音增益过大导致失真。
- `amixer cset name='ALC Capture Min PGA' 6 > /dev/null 2>&1`
描述: 设置自动增益控制的最小 PGA 增益为 6。
作用: 限制增益的下限, 防止录音音量过小。
- `amixer cset name='ALC Capture NG Switch' 'on' > /dev/null 2>&1`
描述: 打开噪声门限 (Noise Gate)。
作用: 当输入信号低于某一阈值时, 自动静音录音, 减少背景噪声。
- `amixer cset name='ALC Capture NG Threshold' 9 > /dev/null 2>&1`
描述: 设置噪声门限值为 9。
作用: 控制噪声门限的触发灵敏度, 值越高, 门限越严格。
- `amixer cset name='ALC Capture NG Type' 'Mute ADC Output' > /dev/null 2>&1`
描述: 噪声门限触发时静音 ADC (模数转换器) 输出。
作用: 在噪声门限被触发时停止录音信号的输出。
- `amixer cset name='ALC Capture Target Volume' 15 > /dev/null 2>&1`
描述: 设置自动增益控制的目标音量为 15。
作用: 设定 ALC 调整后的理想音量。
- `amixer cset name='ALC Capture ZC Switch' 'on' > /dev/null 2>&1`



描述: 打开零交叉检测 (Zero Crossing Detection)。

作用: 零交叉检测可以减少录音时的爆音, 提供更平滑的音频捕捉。

- `amixer cset name='Left Channel Capture Volume' 100% > /dev/null 2>&1`
- `amixer cset name='Right Channel Capture Volume' 100% > /dev/null 2>&1`

描述: 设置左/右声道的录音音量为 100%。

作用: 确保立体声录音时左右声道音量一致, 且达到最大增益。

- `amixer cset name='Left Mixer Left Bypass Volume' 100% > /dev/null 2>&1`
- `amixer cset name='Right Mixer Right Bypass Volume' 100% > /dev/null 2>&1`

描述: 设置左右通道的旁路音量为 100%。

作用: 确保音频信号能够全量通过旁路。

- `amixer cset name='Output 1 Playback Volume' 100% > /dev/null 2>&1`
- `amixer cset name='Output 2 Playback Volume' 100% > /dev/null 2>&1`

描述: 设置输出通道 1 和 2 的播放音量为 100%。

作用: 确保播放音量达到最大。

- `amixer cset name='ZC Timeout Switch' 'on' > /dev/null 2>&1`

描述: 打开零交叉超时开关。

作用: 在检测不到零交叉时启用超时功能。

- `amixer cset name='Left PGA Mux' 'DifferentialL' > /dev/null 2>&1`
- `amixer cset name='Right PGA Mux' 'DifferentialR' > /dev/null 2>&1`

描述: 配置左/右声道为差分输入。

作用: 差分输入有助于减少噪声干扰, 提升音质。

4、麦克风初始化

```
function init_board_mic() {
    # 初始化板载麦克风 c() {
    check_command amixer && {
        amixer cset name='Differential Mux' 'Line 2' > /dev/null 2>&1
        amixer cset name='Left Line Mux' 'Line 2L' > /dev/null 2>&1
        amixer cset name='Right Line Mux' 'Line 2R' > /dev/null 2>&1
    }
}

function init_headphone_mic() {
    # 初始化耳机麦克风 _mic() {
    check_command amixer && { {
        amixer cset name='Differential Mux' 'Line 1' > /dev/null 2>&1
        amixer cset name='Left Line Mux' 'Line 1L' > /dev/null 2>&1
        amixer cset name='Right Line Mux' 'NC' > /dev/null 2>&1      2>&1
    }
}
```

`init_board_mic` 和 `init_headphone_mic` 函数分别配置板载麦克风和耳机麦克风的音频通道。

5、清理退出

```
function cleanup() {
    # 清理并退出
    printf "清理并退出...\n"
    stty sane # 还原终端状态
    exit 0
}
```

- cleanup 函数用于在用户中断（如按下 Ctrl+C）时执行清理操作，并恢复终端的正常状态。

6、模式切换

```
function switch_mode() {
    # 录音/播音模式切换
    case $1 in
        1)
            # 进入录音模式
            l 2>&1
            check_command amixer && {
                amixer cset name='Left Mixer Left Bypass Switch' 'on' >
/dev/null 2>&1
                amixer cset name='Right Mixer Right Bypass Switch' 'on' >
/dev/null 2>&1
                amixer cset name='Left Mixer Left Playback Switch' 'off' >
/dev/null 2>&1
                amixer cset name='Right Mixer Right Playback Switch'
'off' > /dev/null 2>&1
            }
            ;;
        2)
            # 进入播音模式
            l 2>&1
            check_command amixer && {
                amixer cset name='Left Mixer Left Bypass Switch' 'off' >
/dev/null 2>&1
                amixer cset name='Right Mixer Right Bypass Switch' 'off' >
/dev/null 2>&1
                amixer cset name='Left Mixer Left Playback Switch' 'on' >
/dev/null 2>&1
                amixer cset name='Right Mixer Right Playback Switch'
'on' > /dev/null 2>&1
            }
            ;;
        3)
            # 关闭录音和播音模式
            null 2>&1
            check_command amixer && {
```



```

        amixer cset name='Left Mixer Left Bypass Switch' 'off' >
/dev/null 2>&1
        amixer cset name='Right Mixer Right Bypass Switch' 'off' >
/dev/null 2>&1
        amixer cset name='Left Mixer Left Playback Switch' 'off' >
/dev/null 2>&1
        amixer cset name='Right Mixer Right Playback Switch'
'off' > /dev/null 2>&1
        amixer cset name='Left Line Mux' 'NC' > /dev/null 2>&1
        amixer cset name='Right Line Mux' 'NC' > /dev/null 2>&1
    }
    ;;
esac
}

```

switch_mode 函数根据参数设置音频模式:

- 1: 录音模式, 启用旁路开关, 禁用播放开关。
- 2: 播音模式, 启用播放开关, 禁用旁路开关。
- 3: 关闭所有通道, 并将输入设置为 NC (未连接)。

7、应用配置

```

function apply_config() {
    echo "可选麦克风测试项.:::"
    echo "1. 耳机麦克风"
    echo "2. 板载麦克风"
    fi
done

echo "应用麦克风配置项: " "$choice"
case $choice in
    1)
        switch_mode 1
        init_headphone_mic
        ;;
    2)
        switch_mode 1
        init_board_mic
        ;;
    *)
        echo "无效选项"
        ;;
esac
}

```

apply_config 函数用于让用户选择麦克风类型, 并根据选择初始化麦克风配置。



8、捕获 Ctrl+C 信号

捕获 Ctrl+C 信号，并调用 cleanup 函数

```
trap cleanup SIGINT
```

- 脚本捕获 SIGINT 信号（通常是 Ctrl+C），当捕获到信号时，执行 cleanup 函数进行清理操作。

9、测试逻辑

```
while true; do
    echo "可选 ES8388 测试项目:"
    echo "1. 录音测试"
    echo "2. 播音测试"

    while true; do
        read -r -p "输入您需要测试的项目编号（1 或 2）: " command
        case $command in
            1)
                apply_config
                echo "开始录音..."
                check_command arecord && arecord -f cd -d 5 "$RECORD_FILE"
                echo "结束录音..."
                switch_mode 2
                echo "播放录音..."
                check_command aplay && aplay "$RECORD_FILE"
                echo "结束播放..."
                switch_mode 3
                break
                ;;
            2)
                switch_mode 2
                echo "开始播音，按 Ctrl+C 可退出播音"
                gst-play-1.0 --audiosink="alsasink" "$PLAY_FILE"
                echo ""
                switch_mode 3
                break
                ;;
            *)
                cleanup
                ;;
        esac
    done

    break
```

主循环中，用户可以选择执行录音测试或播音测试：

录音测试：用户选择麦克风类型后，录制 5 秒音频并播放录音。

播音测试：播放预定义的音频文件。

- 每次测试后都会进行模式切换并还原音频设置。

23.8 音频测试

ATK-DLMP257B 开发板板载一颗高性能音频编解码芯片 ES8388。板载麦克风可录音，底板背面接了一个小功率喇叭，方便用户进行音频测试。

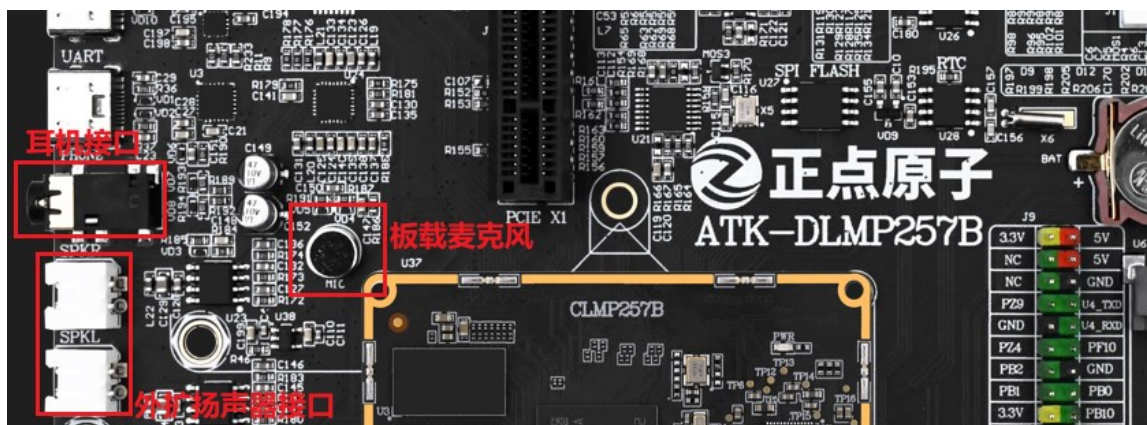


图 23.8-1 开发板正面音频部分

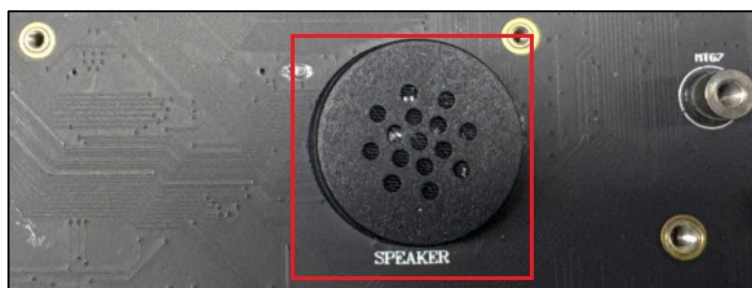


图 23.8-2 开发板背面音频部分

23.8.1 播放音频测试

开发板出厂系统里有音频配置和测试文件，按照如下指令执行音频测试脚本。

```
cd shell/audio
./atk_audiosh
```

第一次运行该脚本时，会打印音频设备初始化相关操作，后续执行此脚本时不再打印初始化相关信息。按 `ctrl+c` 组合键可以退出脚本。

初始化完音频设备后，输入数字 2 并确认即可播放音频测试，播放信息如下。期间板载扬声器会播放音频，如果使用的是耳机则会在耳机端播放。


```

root@ATK-DLMP257:~# cd shell/audio/
root@ATK-DLMP257:~/shell/audio# ls
atk-audio.sh  sunshine.mp3
root@ATK-DLMP257:~/shell/audio# ./atk-audio.sh
可选ES8388测试项目:
1. 录音测试
2. 播音测试
输入您需要测试的项目编号 (1或2): 2
开始播音, 按 Ctrl+C 可退出播音
Press 'k' to see a list of keyboard shortcuts.
Now playing /home/root/shell/audio/sunshine.mp3
Redistribute latency...
Redistribute latency...
清理并退出...:04:29.7
root@ATK-DLMP257:~/shell/audio#

```

输入数字2并按回车键确认

音频播放信息

按ctrl+c组合键后可退出并打印此信息

图 23.8-3 音频播放测试

23.8.2 录音测试

开发板出厂系统里有音频配置和测试文件, 按照如下指令执行音频测试脚本。

```

cd shell/audio
./atk_audio.sh

```

第一次运行该脚本时, 会打印音频设备初始化相关操作, 后续执行此脚本时不再打印初始化相关信息。按 ctrl+c 组合键可以退出脚本。

初始化完音频设备后, 输入数字 1 确认后, 下一步选择麦克风测试项目, 如果是使用带麦克风的耳机接在开发板 PHONE 接口则使用 1. 耳机麦克风; 如果是没接耳机, 直接使用开发板自带的板载麦克风 MIC, 则使用 2. 板载麦克风。这里笔者使用的是开发板自带的板载麦克风 MIC, 这里选择第二项。

```

root@ATK-DLMP257:~/shell/audio# ./atk-audio.sh
可选ES8388测试项目:
1. 录音测试
2. 播音测试
输入您需要测试的项目编号 (1或2): 1
可选麦克风测试项目:
1. 耳机麦克风
2. 板载麦克风
请输入您的选择: 2
应用麦克风配置项: 2
开始录音...
Recording WAVE './record.wav' : Signed 16 bit Little Endian, Rate 44100 Hz, Stereo
结束录音...
播放录音...
Playing WAVE './record.wav' : Signed 16 bit Little Endian, Rate 44100 Hz, Stereo
结束播放...
root@ATK-DLMP257:~/shell/audio#

```

输入数字1并按回车键确认

输入数字2并按回车键确认, 使用板载麦克风

自动录音, 请靠近板载麦克风进行说话

录音结束, 自动播放已录制的音频文件

图 23.8-4 使用板载麦克风配置

录音完成后会在当前目录下生成 record.wav 文件, 此文件就是笔者刚刚录音生成的音频文件, 可以使用 aplay 再次播放录制好的音频文件。

```

aplay record.wav

```

```
root@ATK-DLMP257:~/shell/audio# ls
atk-audio.sh  record.wav  sunshine.mp3
root@ATK-DLMP257:~/shell/audio# aplay record.wav
Playing WAVE 'record.wav' : Signed 16 bit Little Endian, Rate 44100 Hz, Stereo
root@ATK-DLMP257:~/shell/audio#
```

图 23.8-5 aplay 播放录制好的音频文件

第二十四章 WiFi 子系统

24.1 WiFi 子系统简介

嵌入式 Linux 的 WiFi 子系统主要负责无线网络的管理和通信。它由多个组件和框架组成, 提供驱动程序支持、网络配置管理、以及用户空间和内核空间之间的通信接口

24.2 组成部分

嵌入式 Linux 的 WiFi 子系统由多个关键组件组成, 每个组件都在 WiFi 通信和管理过程中扮演着重要角色。以下是这些组成部分的详细介绍:

1、无线驱动程序 (Wireless Drivers)

mac80211:这是 Linux 内核中的一个通用的 WiFi 协议栈, 支持大多数现代 WiFi 芯片。它处理与无线通信相关的各种任务, 例如帧管理、数据传输、以及协议实现。

cfg80211:这是 mac80211 的配置接口层, 它为用户空间提供一个标准化的 API, 用于配置和管理 WiFi 设备。cfg80211 与 mac80211 协作, 处理扫描、连接、断开等操作。

硬件特定驱动程序:这些驱动程序直接与 WiFi 芯片硬件交互, 通常由芯片制造商提供, 并使用 mac80211/cfg80211 框架来实现硬件功能。

2、内核接口

nl80211:这是一个 netlink 接口, 提供用户空间与内核 WiFi 子系统 (通过 cfg80211 和 mac80211) 的交互。通过 nl80211, 用户空间程序可以请求执行诸如网络扫描、连接到网络、设置无线电信道等操作。

3、用户空间工具

wpa_supplicant:这是一个广泛使用的用户空间守护程序, 用于处理 WiFi 认证和加密 (如 WPA/WPA2)。它通过 nl80211 与内核交互, 实现网络连接和安全协议管理。

iw:这是一个命令行工具, 使用 nl80211 接口, 允许用户直接配置和管理 WiFi 接口, 执行扫描、连接和网络状态监控等任务。

4、网络管理框架

NetworkManager:这是一个常见的网络管理工具, 自动管理有线和无线网络连接, 适合桌面和嵌入式系统。它能够处理复杂的网络配置和连接切换。

systemd-networkd:这是 systemd 的一部分, 用于网络配置和管理, 支持静态 IP 配置和基本的 WiFi 连接管理, 适合资源受限的嵌入式系统。

5、射频开关 (RFKILL)

rftkill:这是一个内核子系统, 用于管理无线设备的射频功能。它允许系统或用户关闭无线电发射以节省电力或遵守法规。

6、硬件抽象层

固件 (Firmware):WiFi 芯片通常需要加载特定的固件来启用完整功能, 这些固件在启动时加载并运行于设备中, 管理低级硬件操作。

驱动程序抽象:Linux 通过 mac80211 和 cfg80211 提供一个通用的接口, 使得不同制造商的硬件能够在相同的框架下运作, 简化了驱动开发和集成。

7、性能与功耗管理

嵌入式 WiFi 设备需要在有限的资源下优化性能和功耗。内核和驱动程序会使用各种技术, 如省电模式 (Power Save Mode, PSM) 和动态频率调整, 来减少电力消耗并延长电池寿命。

8、网络配置与管理

DHCP Client/Server:用于动态分配 IP 地址，通常在设备连接到无线网络时使用。

dnsmasq:这是一个轻量级的 DNS、DHCP、TFTP 服务，适用于嵌入式系统，用于提供本地网络服务。

这些组成部分共同协作，确保嵌入式 Linux 系统能够可靠且高效地管理 WiFi 连接，从硬件通信到用户空间的网络配置和管理，涵盖了整个网络栈。

24.3 开发板 WiFi 说明

24.3.1 原理图分析

开发板上有一个 WiFi&BT 芯片。打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 WIFI。可以看到相关电路如下。

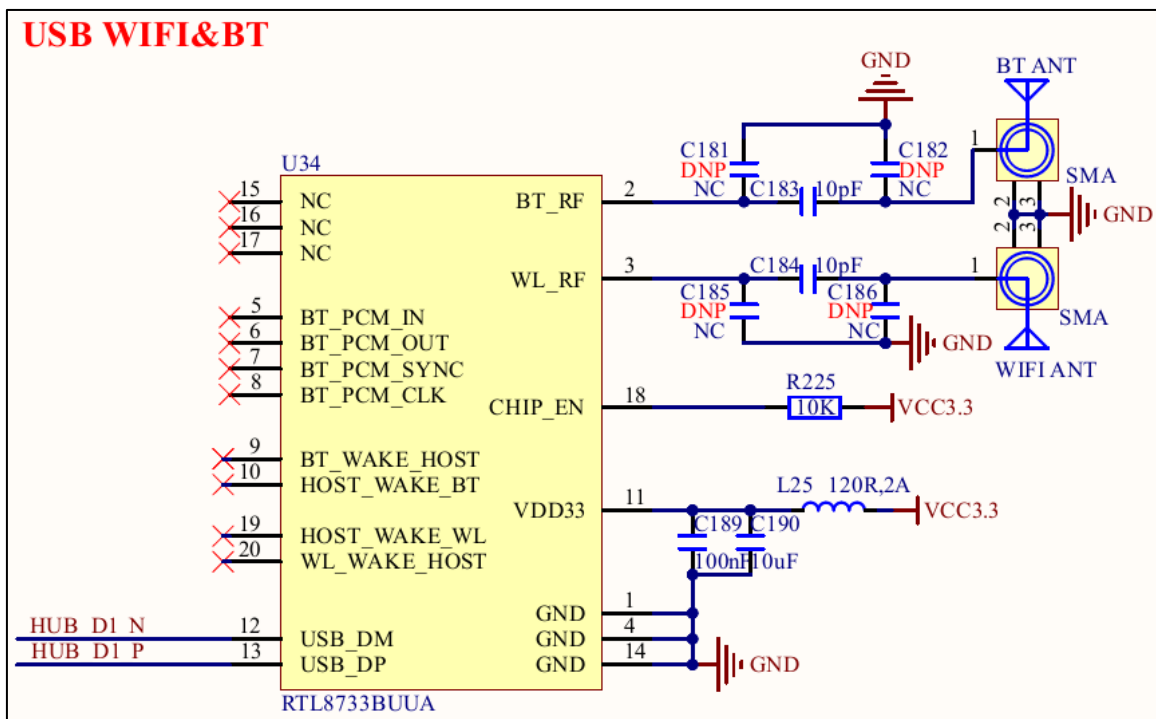


图 24.3-1 开发板 WIFI 电路

这个原理图展示了一个 USB WiFi 和蓝牙模块的电路连接，该模块是 Realtek 的 RTL8733BU，支持通过 USB 进行通信。以下是对该电路的解析：

1、电源部分

VDD33(Pin 11): 连接到 3.3V 电源，通过滤波电容 (C189: 100nF 和 C190: 10uF) 去除电源噪声。为模块内部电路提供工作电源。

电源电感 (L25): 规格为 120R, 2A, 起到电流限流和电源噪声抑制作用。

2、USB 接口

USB_DM (Pin 12) 和 USB_DP (Pin 13): 连接到 USB 主机，分别是 USB 的 D- 和 D+ 信号线。用于 Wi-Fi 和蓝牙模块与主机之间的数据传输。

接地引脚 (Pin 1, 4, 14): 提供模块的电气接地，确保信号完整性。

3、天线接口

BT_RF (Pin 15) 和 WL_RF (Pin 16): 分别连接蓝牙和 Wi-Fi 的射频天线。

通过旁路电容 (C181、C183、C184、C186, 均为 10pF, 部分未安装) 进行高频滤波。

天线匹配网络: 天线通过 SMA 接头连接到模块。可能需要调节匹配电路以满足实际射频性能需求。

4、控制接口

CHIP_EN (Pin 18): 模块的使能引脚, 拉高至 VCC3.3 以激活模块。外接一个 10K 的上拉电阻 (R225)。

BT_WAKE_HOST (Pin 9) 和 HOST_WAKE_BT (Pin 10): 蓝牙唤醒引脚, 模块与主机之间进行唤醒信号通信。

HOST_WAKE_WL (Pin 20) 和 WL_WAKE_HOST (Pin 19): Wi-Fi 唤醒引脚, 与主机通信控制 Wi-Fi 功能。

24.3.2 硬件实物

开发板上的 WiFi 相关的电路位置如下图所示:

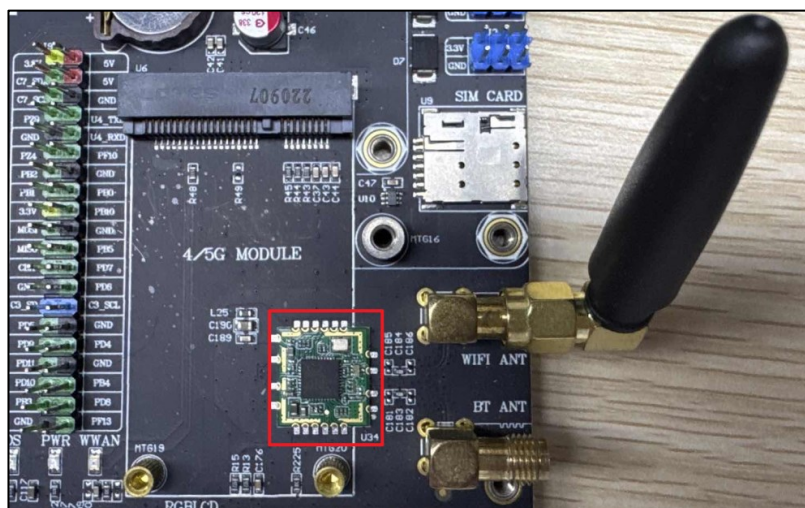


图 24.3-2 开发板 WiFi 模块

在使用 wifi 之前, 需要接好对应的天线。

24.4 内核 WiFi 相关使能

24.4.1 无线框架使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig, WiFi 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_PACKET=y //允许用户空间程序直接访问网络层帧
2 CONFIG_INET=y //启用 IPv4 协议栈支持
3 CONFIG_IP_PNP=y //允许在系统启动时通过网络获取 IP 地址
4 CONFIG_IP_PNP_DHCP=y //启用通过 DHCP 协议获取网络配置
5 CONFIG_IP_PNP_BOOTP=y //启用通过 BOOTP 协议获取网络配置
6 CONFIG_INET_DIAG=m //启用套接字状态的诊断功能, 用于监控和调试网络连接。
7 CONFIG_IPV6=m //启用 IPv6 协议栈支持
8 CONFIG_DNS_RESOLVER=y //启用内核 DNS 解析功能
9 CONFIG_CFG80211=m //启用 cfg80211 无线配置 API 支持
```

10 CONFIG_MAC80211=m //启用 mac80211 模块, 为基于软件的无线协议栈提供核心支持

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

```
[*] Networking support --->
    [*] Networking options --->
        [*] Packet socket
        [*] TCP/IP networking
            [*] IP: kernel level autoconfiguration
            [*] IP: DHCP support
            [*] IP: BOOTP support
        [M] INET: socket monitoring interface
        [M] The IPv6 protocol
        [*] DNS Resolver support
    [*] Wireless --->
        [M]   cfg80211 - wireless configuration API
        [M]   Generic IEEE 802.11 Networking Stack (mac80211)
```

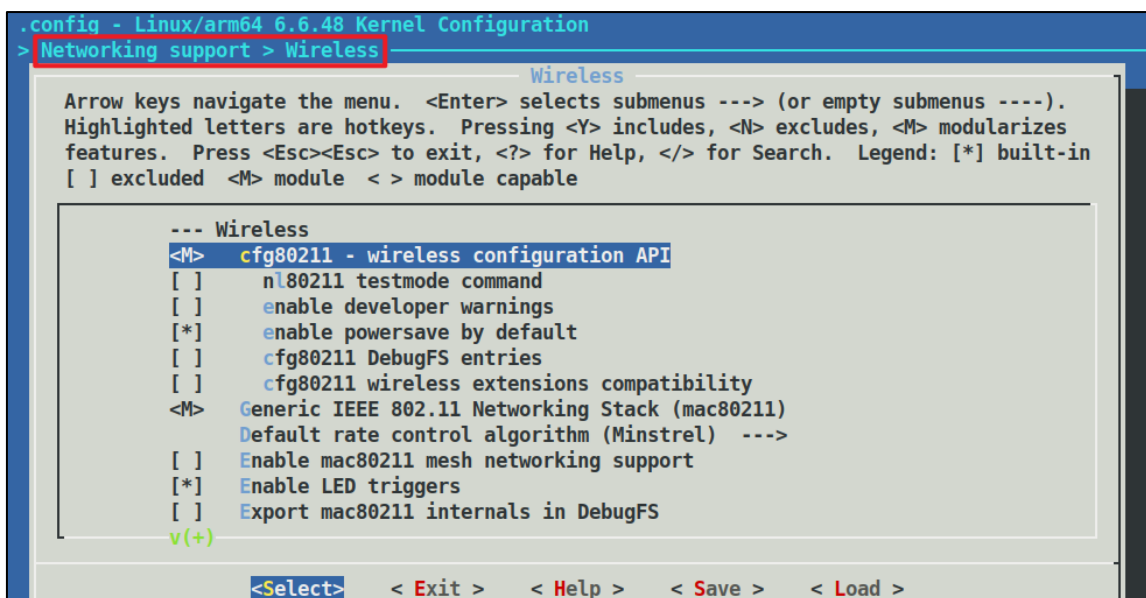


图 24.4-1 cfg80211 部分配置

24.4.2 WIFI 驱动使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig, WiFi 相关配置如下(略有删减, 只列举重点部分):

1 CONFIG_RTL8733BU=m //启用 Realtek RTL8733BU 无线网卡驱动支持

对应的 menuconfig 配置如下 (篇幅有限, 截图有所省略):

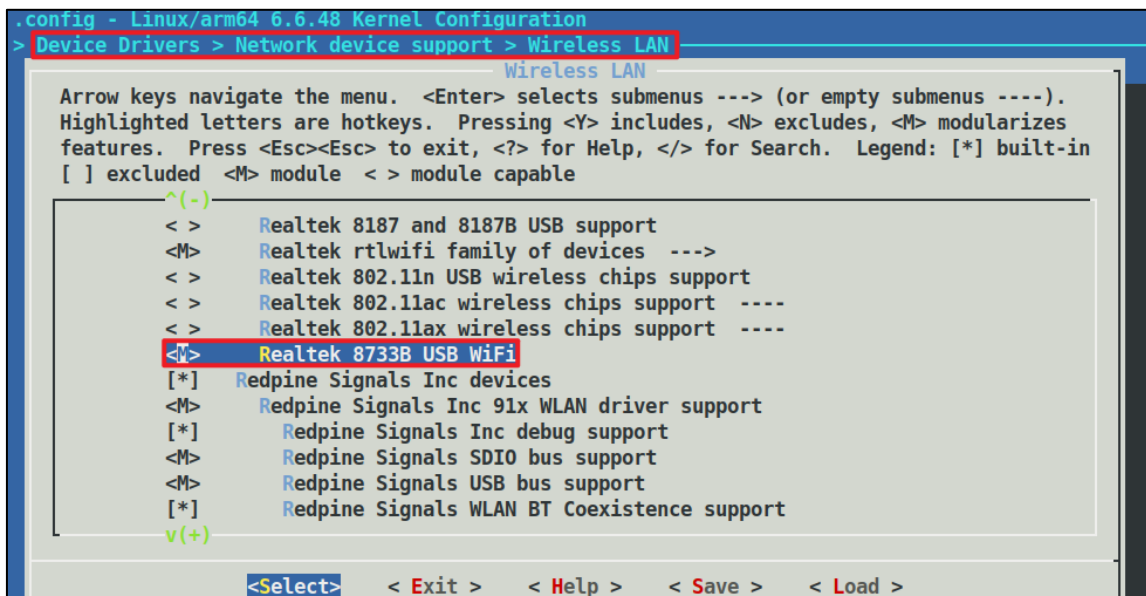


图 24.4-2 RTL8733BU 驱动模块

对应的驱动源码在出厂内核源码 `drivers/net/wireless/realtek/rtl8733bu` 目录,此驱动使用的是 USB 接口协议,开发板出厂设备树上已添加 USB 节点,就不需要再其他额外配置了。因此本章节没有设备树讲解,参考出厂系统设备树的 `usb` 相关配置即可。

24.5 WiFi 工具

24.5.1 connmanctl

`connmanctl` 是 ConnMan (Connection Manager) 的命令行界面,用于在 Linux 系统上管理网络连接。ConnMan 是一个网络连接管理器,专门为嵌入式设备设计,支持有线网络、Wi-Fi、蓝牙、蜂窝网络等。

开发板执行下面指令:

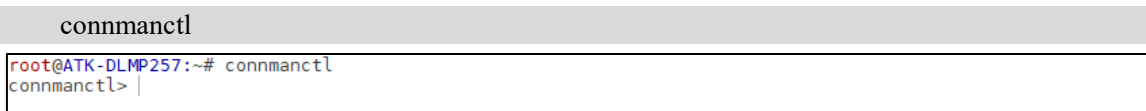


图 24.5-1 connmanctl 指令

此时会进入 `connmanctl` 命令行终端,在终端中可以执行 `connmanctl` 相关指令操作,我们可以执行 `help` 指令查看 `connmanctl` 命令行终端支持的指令:

```
connmanctl> help
state                               Shows if the system is online or offline
technologies                        Display technologies
clock                               Get System Clock Properties
enable <technology>|offline         Enables given technology or offline mode
disable <technology>|offline        Disables given technology or offline mode
tether <technology> on|off          Enable, disable tethering, set SSID and passphrase
                                     wifi [on|off] <ssid> <passphrase> [<freq>]
for wifi
tethering_clients                   Display tethering clients
services [<service>]                Display services
--properties [<service>]            (obsolete)
peers [peer]                        Display peers
scan <technology>                   Scans for new services for given technology
connect <service/peer>              Connect a given service or peer
disconnect <service/peer>           Disconnect a given service or peer
move-before <service> <target service> Move <service> before <target service>
move-after <service> <target service> Move <service> after <target service>
config <service>                    Set service configuration options
--nameservers <dns1> [<dns2>] [<dns3>]
--timeservers <ntp1> [<ntp2>] [...]
--domains <domain1> [<domain2>] [...]
--mdns yes|no
--ipv6 off|auto [enable|disable|preferred]
--proxy manual <address> <prefixlength> <gateway>
       direct|auto <URL>|manual <URL1> [<URL2>] [...]
       [exclude <exclude1> [<exclude2>] [...]]
--autoconnect yes|no
--ipv4 off|dhcp|manual <address> <netmask> <gateway>
--remove Remove service
monitor [off]                       Monitor signals from interfaces
--services [off]                    Monitor only services
--tech [off]                        Monitor only technologies
--manager [off]                    Monitor only manager interface
--vpnmanager [off]                 Monitor only VPN manager interface
--vpnconnection [off]              Monitor only VPN connections
agent on|off                         Agent mode
vpnconnections [<connection>]       Display VPN connections
vpnagent on|off                     VPN Agent mode
session on|off|connect|disconnect|config Enable or disable a session
--bearers <technology1> [<technology2> [...]]
--type local|internet|any
--ifname [<interface_name>]
--srciprule yes|no
--ctxid <context_identifier>
peer_service register|unregister <specs> <master>
where specs are:
  bjr_query <query> bjr_response <response>
  upnp_service <service> upnp_version <version>
  wfd_ies <ies>
(Un)Register a Peer Service
help                                Show help
exit                                Exit
quit                                Quit
connmanctl> |
```

图 24.5-2 connmanctl 支持的指令

一些关键命令的详细解释:

- state: 显示系统当前的联网状态（在线或离线）。
- technologies: 列出系统中可用的网络技术（如 Wi-Fi、Ethernet 等）。
- enable 和 disable: 启用或禁用指定的网络技术或将系统设置为离线模式。例如：
 - enable wifi 启用 Wi-Fi
 - disable wifi 禁用 Wi-Fi
- tether: 用于启用或禁用网络共享(热点), 以及设置 Wi-Fi 热点的 SSID 和密码。例如：
 - tether wifi on myssid mypassword 启用 Wi-Fi 热点并设置 SSID 为 myssid, 密码为 mypassword。
- services: 列出当前可用的网络服务（例如，已扫描到的 Wi-Fi 网络）
- connect 和 disconnect: 连接或断开指定的网络服务。例如：
 - connect wifi_service 连接到指定的 Wi-Fi 服务
 - disconnect wifi_service 断开与指定 Wi-Fi 服务的连接
- config: 配置指定服务的网络参数，如 DNS、NTP 服务器、IPv4 和 IPv6 设置等。例如：

- config wifi_service --ipv4 manual 192.168.1.10 255.255.255.0 192.168.1.1 手动配置指定 Wi-Fi 服务的 IPv4 地址
- scan: 扫描指定网络技术的新服务。例如
 - scan wifi 扫描可用的 Wi-Fi 网络
- monitor: 监控来自接口的信号, 例如网络服务和技术的变化
- vpnconnections: 显示当前的 VPN 连接

WiFi连接示例: 进行WiFi测试之前, 请确保开发板上安装好天线, 否则无法连接WiFi。进入connmanctl命令行后, 输入下面命令进行为WiFi 测试。

```
enable wifi
scan wifi
agent on
services

connmanctl> enable wifi
connmanctl> [ 5178.833403] start_addr=(0x0), end_addr=(0x20000), buffer_size=(0x20000), smp_number_max=(16384)
Enabled wifi
connmanctl> scan wifi
Scan completed for wifi
connmanctl> agent on
Agent registered
connmanctl> services
ALIENTEK-linux      wifi_f2a882c1ff98_414c49454e54454b2d6c696e7578_managed_psk
ALIENTEK-WX_5G      wifi_f2a882c1ff98_hidden_managed_psk
RICH901-5G          wifi_f2a882c1ff98_414c49454e54454b2d57585f3547_managed_psk
ALIENTEK-WX          wifi_f2a882c1ff98_524943483930312d3547_managed_psk
ChinaNet-ymyk       wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk
ChinaNet-ymyk       wifi_f2a882c1ff98_4368696e614e65742d796d796b_managed_psk
connmanctl>
```

图 24.5-3 开启 WiFi 并扫描 WiFi

enable wifi

作用: 启用 Wi-Fi 技术。执行此命令后, 系统会启动 Wi-Fi 硬件模块, 使其能够扫描和连接无线网络。

scan wifi

作用: 扫描可用的 Wi-Fi 网络。此命令会搜索附近所有的 Wi-Fi 网络, 并在列表中显示它们的服务标识符 (SSID)。

agent on

作用: 启用代理模式。在启用代理后, 系统会允许用户处理一些需要交互的操作, 例如连接 Wi-Fi 时需要输入密码的情况。

services

作用: 列出当前检测到的服务 (如 Wi-Fi 网络)。显示的结果包括每个网络的服务标识符 (例如 SSID) 和加密类型。

wifi6_5g: 这是一个 5GHz 的 Wi-Fi 网络, 使用 PSK (预共享密钥) 进行管理。

ALIENTEK-YY: 一个名为 ALIENTEK-YY 的 Wi-Fi 网络, 也使用 PSK 进行管理。

wifi_488f4c348da4_hidden_managed_psk: 这是一个隐藏的 Wi-Fi 网络, SSID 没有广播, 因此显示为隐藏网络。

这些命令的组合用于启用 Wi-Fi, 扫描可用的网络, 并配置代理以便在连接过程中处理输入 (如输入 Wi-Fi 密码)。当你看到可用的网络列表后, 可以选择连接到其中的一个网络。

从扫描到的WIFI 热点列表中, 使用命令connect XXXX 进行连接, 再输入密码按回车确认。比如示例要连接ALIENTEK-YY, 运行结果如下:

```
connect wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk\\此为示例, 需修改有 Passphrase?提示后输入 WiFi 密码
```

exit

```
connmanctl> connect wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk
Agent RequestInput wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk
  Passphrase = [ Type=psk, Requirement=mandatory, Alternates=[ WPS ] ]
  WPS = [ Type=wpspin, Requirement=alternate ]
Passphrase?
Connected wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk
connmanctl> exit
root@ATK-DLMP257:~#
```

图 24.5-4 连接 WiFi

connect wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk

作用：连接到指定的 Wi-Fi 网络，该网络的标识符为 wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk。这个网络的 SSID 是 ALIENTEK-WX，并且使用预共享密钥（PSK）进行加密管理。

Agent RequestInput

作用：代理请求输入连接该 Wi-Fi 网络所需的信息。在这种情况下，系统要求输入 Passphrase（密码短语），这是连接 Wi-Fi 所必需的。

Passphrase? 输入 WiFi 密码

作用：输入 WiFi 密码短语，用于连接该 Wi-Fi 网络。

Connected wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk

作用：表明连接成功，设备已连接到 SSID 为 ALIENTEK-WX 的 Wi-Fi 网络。

exit

作用：退出 connmanctl 交互界面。

通过这些步骤，系统已经成功启用 Wi-Fi 模块，扫描网络，选择并连接到了指定的 Wi-Fi 网络。

退出 connmanctl 交互界面后，可以执行 ifconfig 指令查看 WiFi 获取到的 IP 地址。

```
wlan1  Link encap:Ethernet HWaddr F2:A8:82:C1:FF:98
        inet addr:192.168.7.29 Bcast:192.168.7.255 Mask:255.255.255.0
        inet6 addr: fe80::f0a8:82ff:fecl:ff98/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
        RX packets:167 errors:0 dropped:0 overruns:0 frame:0
        TX packets:95 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:21019 (20.5 KiB) TX bytes:17271 (16.8 KiB)
```

图 24.5-5 查看获取到的 IP 地址

ping 百度测试连通性，也可以 ping 网关来测试 WiFi 的连通性。

ping www.baidu.com -I wlan1

```
root@ATK-DLMP257:~# ping www.baidu.com -I wlan1
PING www.a.shifen.com (183.2.172.42) from 192.168.7.29 wlan1: 56(84) bytes of data.
64 bytes from 183.2.172.42: icmp_seq=1 ttl=51 time=9.27 ms
64 bytes from 183.2.172.42: icmp_seq=2 ttl=51 time=9.80 ms
64 bytes from 183.2.172.42: icmp_seq=3 ttl=51 time=11.1 ms
64 bytes from 183.2.172.42: icmp_seq=4 ttl=51 time=18.7 ms
^C
--- www.a.shifen.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 9.266/12.203/18.698/3.805 ms
root@ATK-DLMP257:~#
```

图 24.5-6 ping 百度测试

如果接入了网线或者其他网络设备，可以通过删除原本默认的网关地址，再使用 route 指令添加 WiFi 网关。具体可以看【正点原子】STM32MP257 开发板（A 盘）-基础资料\10、用户手册\【正点原子】ATK-DLMP257B 快速体验手册

如果要删除掉已保存的 WiFi，在 connmanctl 命令行中执行下面操作：

显示已保存的服务（包括 Wi-Fi 网络）：

services

```
connmanctl> services
*AO ALIENTEK-WX      wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk
ALIENTEK-linux      wifi_f2a882c1ff98_414c49454e54454b2d6c696e7578_managed_psk
RICH901              wifi_f2a882c1ff98_52494348393031_managed_psk
```

图 24.5-7 已保存的服务

在 connmanctl 中, *AO: 当前所选的网络连接标记, 表示该网络已连接并且活跃 (* 表示当前连接, A 表示已激活, O 表示该服务是“在线”的)。

config wifi_<identifier> --remove

替换 <identifier> 为要删除的 Wi-Fi 网络标识符, 这里笔者的 Wi-Fi 网络标识符是 wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk

```
connmanctl> config wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk --remove
connmanctl> services
ALIENTEK-linux      wifi_f2a882c1ff98_414c49454e54454b2d6c696e7578_managed_psk
                    wifi_f2a882c1ff98_hidden_managed_psk
RICH901             wifi_f2a882c1ff98_52494348393031_managed_psk
ALIENTEK-WX         wifi_f2a882c1ff98_414c49454e54454b2d5758_managed_psk
DIRECT-HCDESKTOP-7A885MVmsJN wifi_f2a882c1ff98_4449524543542d48434445534b544f502d37413838354d566d734a4
e_managed_psk
ChinaNet-ymyk       wifi_f2a882c1ff98_4368696e614e65742d796d796b_managed_psk
ATK-HUAWEI          wifi_f2a882c1ff98_41544b2d485541574549_managed_psk
ALIENTEK-7机房      wifi_f2a882c1ff98_414c49454e54454b2d37e69cb6e688bf_managed_psk
ALIENTEK-XHYS       wifi_f2a882c1ff98_414c49454e54454b2d58485953_managed_psk
ALIENTEK-SYS        wifi_f2a882c1ff98_414c49454e54454b2d535953_managed_psk
YinNengHanJin_Guest wifi_f2a882c1ff98_59696e4e656e6748616e4a696e5f4775657374_managed_psk
qm1506              wifi_f2a882c1ff98_716d31353036_managed_psk
ALIENTEK-YF-SYS     wifi_f2a882c1ff98_414c49454e54454b2d59462d535953_managed_psk
C21                  wifi_f2a882c1ff98_433231_managed_psk
YinNengHuanJin      wifi_f2a882c1ff98_59696e4e656e674875616e4a696e_managed_psk
606-2                wifi_f2a882c1ff98_3630362d32_managed_psk
ALIENTEK-5f办公     wifi_f2a882c1ff98_414c49454e54454b2d3566e58a9ee585ac_managed_psk
RICH901-5G          wifi_f2a882c1ff98_524943483930312d3547_managed_psk
ALIENTEK-WX_5G       wifi_f2a882c1ff98_414c49454e54454b2d57585f3547_managed_psk
863egsw             wifi_f2a882c1ff98_38363365677377_managed_psk
ALIENTEK-SYS_5G      wifi_f2a882c1ff98_414c49454e54454b2d5359535f3547_managed_psk
connmanctl>
```

图 24.5-8 删除 WiFi

可以看到删除完后, 之前保存的 WiFi 不显示*AO 标识了, 此时 WiFi 也会断开。

24.5.2 iw

`iw` 是一个用于管理和配置 Linux 系统中的无线网络设备的命令行工具。它主要用于与内核的 `cfg80211` 和 `mac80211` 子系统交互，以执行各种无线网络任务。执行下面指令可以查看使用说明：

```
iw
root@ATK-DLMP257:~# iw
Usage: iw [options] command
Options:
  --debug          enable netlink debugging
  --version        show version (6.7)
Commands:
  dev <devname> ap stop
  dev <devname> ap start
  phy <phyname> coalesce show
  phy <phyname> coalesce disable
  phy <phyname> coalesce enable <config-file>
  dev <devname> auth <SSID> <bssid> <type:open|shared> <freq in MHz> [key 0:abcde d:1:6162636465]
  dev <devname> connect [-w] <SSID> [<freq in MHz>] [<bssid>] [auth open|shared] [key 0:abcde d:1:61
62636465] [mfp:req/opt/no]
  dev <devname> disconnect
  dev <devname> cqm rssi <threshold|off> [<hysteresis>]
```

篇幅过长，以下省略

图 24.5-9 iw 工具使用说明

使用示例：开启无线网络接口

```
ip link set wlan1 up
```

这里的 `wlan1` 是你的无线网络接口名称。可以通过 `iw dev` 或 `ip link` 命令来查看接口名称。

使用示例：查看无线设备信息

```
iw dev wlan1 info
```

```
root@ATK-DLMP257:~# ip link set wlan1 up
root@ATK-DLMP257:~# iw dev wlan1 info
Interface wlan1
  ifindex 8
  wdev 0x2
  addr f2:a8:82:c1:ff:98
  type managed
  wiphy 0
  txpower -100.00 dBm
root@ATK-DLMP257:~#
```

图 24.5-10 查看 WiFi 设备信息

作用：显示无线设备 `wlan1` 的详细信息，包括接口类型、MAC 地址、支持的频段等。

设备信息解析：

- `Interface wlan1`：这是接口的名称，通常在 Linux 系统上，Wi-Fi 接口的名称是 `wlan1`。
- `ifindex 8`：这是网络接口的索引编号，系统内部用来标识这个网络接口。
- `wdev 0x2`：这是无线设备的唯一标识符（wdev ID），以十六进制表示。
- `addr f2:a8:82:c1:ff:98`：这是该接口的 MAC 地址，用于网络通信中的设备标识。
- `type managed`：这个字段表示接口的类型。`managed` 类型意味着接口正在运行在客户端模式（也称为站点模式），用于连接到接入点（AP）。
- `wiphy 0`：这是无线物理设备的标识符，在系统中用于管理无线硬件设备。
- `txpower -100.00 dBm`：表示该接口的发射功率为 -100.00 dBm（分贝毫瓦），用于确定无线信号的强度。

使用示例：扫描可用无线网络

iw dev wlan1 scan

```
root@ATK-DLMP257:~# iw dev wlan1 scan
BSS 02:d0:29:4f:50:c3(on wlan1)
    TSF: 830717556 usec (0d, 00:13:50)
    freq: 2412.0
    beacon interval: 100 TUs
    capability: ESS Privacy ShortPreamble ShortSlotTime APSD RadioMeasure (0x1c31)
    signal: -45.00 dBm
    last seen: 0 ms ago
    Information elements from Probe Response frame:
    SSID: ALIENTEK-linux
    Supported rates: 1.0* 2.0* 5.5* 11.0* 6.0 9.0 12.0 18.0
    DS Parameter set: channel 1
    ERP: Use Protection
    Extended supported rates: 24.0 36.0 48.0 54.0
    HT capabilities:
        Capabilities: 0x106e
            HT20/HT40
            SM Power Save disabled
            RX HT20 SGI
            RX HT40 SGI
            No RX STBC
            Max AMSDU length: 3839 bytes
            DSSS/CCK HT40
        Maximum RX AMPDU length 65535 bytes (exponent: 0x003)
        Minimum RX AMPDU time spacing: 4 usec (0x05)
        HT RX MCS rate indexes supported: 0-15
        HT TX MCS rate indexes are undefined
    HT operation:
        * primary channel: 1
        * secondary channel offset: no secondary
        * STA channel width: 20 MHz
        * RIFS: 0
        * HT protection: nonmember
        * non-GF present: 0
        * OBSS non-GF present: 1
        * dual beacon: 0
        * dual CTS protection: 0
        * STBC beacon: 0
        * L-SIG TXOP Prot: 0
        * PCO active: 0
        * PCO phase: 0
```

以下省略

图 24.5-11 扫描无线网络

作用：扫描当前范围内的可用无线网络，并列出所有检测到的 SSID（网络名称）。

信息解析：

扫描到的接入点是 ALIENTEK-linux，它在 2.4 GHz 频段（频道 1）工作，信号强度非常强（-45 dBm）。该接入点支持 802.11n 高吞吐量模式，能够提供更高的数据传输速率（如 54 Mbps），并支持 20 MHz 和 40 MHz 带宽。

扫描所有可用的 Wi-Fi 网络，并显示其 SSID（网络名称），可以执行下面指令：

iw dev wlan1 scan | grep SSID

```
root@ATK-DLMP257:~# iw dev wlan1 scan | grep SSID
SSID: ALIENTEK-linux
SSID: RICH901
SSID: ALIENTEK-linux
SSID: RICH901-5G
SSID: ALIENTEK-WX_5G
SSID: ATK-HUAWEI
SSID: ALIENTEK-WX
SSID: YinNengHuanJin
    * UTF-8 SSID
SSID: ChinaNet-ymyk
SSID: ALIENTEK-5f\xe5\x8a\x9e\xe5\x85\xac
SSID: kaweiya
SSID: 606-2
SSID: GZ-GNG
SSID: DIRECT-HCDESKTOP-7A885MVmsJN
SSID: qml1506
    * UTF-8 SSID
SSID: ALIENTEK-SYS
SSID: ALIENTEK-7\xe5\x8a\x9e\xe5\x85\xac
SSID: YinNengHanJin_Guest
    * UTF-8 SSID
SSID: ChinaNet-ymyk-5G
SSID: ALIENTEK-XHYS
root@ATK-DLMP257:~#
```

图 24.5-12 扫描 WiFi 并显示网络名称

iw 命令也可以连接 WiFi，如果连接到 WPA/WPA2 加密的 Wi-Fi 网络，建议使用 wpa_supplicant 配置文件来管理连接，因为 iw 命令本身不支持 WPA/WPA2 加密的网络连接。

使用 connmanctl 命令行连接上 WiFi 后，可以使用 iw 相关指令查看当前网络连接状态：

iw dev wlan1 link

```
root@ATK-DLMP257:~# iw dev wlan1 link
Connected to 84:76:37:1d:8b:74 (on wlan1)
    SSID: ALIENTEK-WX
    freq: 2452.0
    signal: -65 dBm
    tx bitrate: 72.2 MBit/s
root@ATK-DLMP257:~#
```

图 24.5-13 当前 WiFi 连接状态

输出信息解析

- Connected to 84:76:37:1d:8b:74 (on wlan1): 表示当前的无线接口 wlan1 已经连接到 MAC 地址为 84:76:37:1d:8b:74 的接入点 (AP)。
- SSID: ALIENTEK-WX: 连接的网络名称 (SSID)。
- freq: 2452: 表示连接的频率为 2452 MHz，对应的是 2.4 GHz 频段的第 7 频道。
- signal: -65 dBm: 表示接收到的信号强度为 -65 dBm，这个值越接近 0 表示信号越强。一般来说，-65 dBm 是中等信号强度。
- tx bitrate: 72.2 MBit/s: 表示当前的数据传输速率为 72.2 Mbit/s。

第二十五章 Bluetooth 子系统

25.1 Bluetooth 子系统简介

嵌入式 Linux 的 Bluetooth 子系统是用于管理和操作蓝牙设备的一整套软件框架和驱动程序。这个子系统主要由蓝牙协议栈、驱动程序、用户空间工具和 API 组成，用于实现蓝牙通信和设备管理。

25.2 组成部分

1、蓝牙协议栈（Bluetooth Protocol Stack）

- HCI（Host Controller Interface）：
提供主机与蓝牙控制器之间的通信接口。
在内核中实现，管理底层硬件与协议栈的交互。
- L2CAP（Logical Link Control and Adaptation Protocol）：
逻辑链路控制和适配协议，负责数据包的封装和适配。
支持面向连接和无连接的服务。
- RFCOMM：
提供串行端口仿真功能，常用于模拟 RS-232 串行通信。
- SCO（Synchronous Connection-Oriented）：
用于传输同步音频数据，如语音通信。
- GATT（Generic Attribute Profile）：
在低功耗蓝牙（BLE）中用于描述设备间的数据交换格式，支持发现和操作远程设备的服务。

2、蓝牙驱动程序

- HCI 驱动程序：
负责控制和管理与蓝牙硬件设备（如 USB、UART、SPI 等接口的蓝牙模块）的通信。
提供了对蓝牙控制器的初始化、配置、传输数据等功能。
- 蓝牙控制器驱动（Controller Drivers）：
特定于硬件的驱动程序，用于支持各种蓝牙芯片或模块。
例如，Broadcom、Qualcomm 等芯片的蓝牙驱动程序。

3、内核支持

- 蓝牙核心模块（Bluetooth Core Module）：
包括蓝牙协议栈在内核中的实现，如 HCI、L2CAP 等。
提供基本的蓝牙功能支持，包括设备发现、连接管理、数据传输等。
- 特定协议的内核模块：
支持各种蓝牙协议的内核模块，如 RFCOMM、SCO、A2DP（用于音频传输）等。

4、用户空间工具和 API

- BlueZ：
Linux 系统中的蓝牙协议栈实现，主要运行在用户空间。
提供了与内核蓝牙模块的接口，以及管理和操作蓝牙设备的工具和库。
- bluetoothd：
BlueZ 的守护进程，负责设备管理、服务发现、连接管理等核心功能。
- bluetoothctl：

交互式命令行工具，用于管理蓝牙设备，包括设备发现、配对、连接等。

- hcitool 和 hciconfig:

用于配置和管理 HCI 接口的工具，提供蓝牙设备的低级配置和控制。

- D-Bus API:

提供了蓝牙服务与应用程序之间的通信接口，使得用户空间应用可以通过 D-Bus 与蓝牙设备交互。

5、蓝牙管理和应用层

- 蓝牙设备发现和配对:

通过协议栈和工具实现设备发现、配对和连接管理。

- 服务发现和配置:

使用 GATT 和其他配置文件管理和操作蓝牙设备提供的服务。

6、应用示例

- 音频传输 (A2DP): 通过 A2DP 协议实现音频流的无线传输。
- 串行通信 (RFCOMM): 通过 RFCOMM 实现模拟串行端口通信。
- BLE 传感器连接: 通过 GATT 协议连接低功耗传感器，并读取数据。

嵌入式 Linux 的蓝牙子系统是一个复杂的集成系统，涵盖了从底层硬件驱动到高级应用的各个层次，使得开发者能够灵活实现各种蓝牙应用。

25.3 开发板 Bluetooth 说明

25.3.1 原理图分析

开发板上有一个 WiFi&BT 芯片。打开开发板上【正点原子】STM32MP257 开发板 (A 盘) -基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 BT。可以看到相关电路如下。

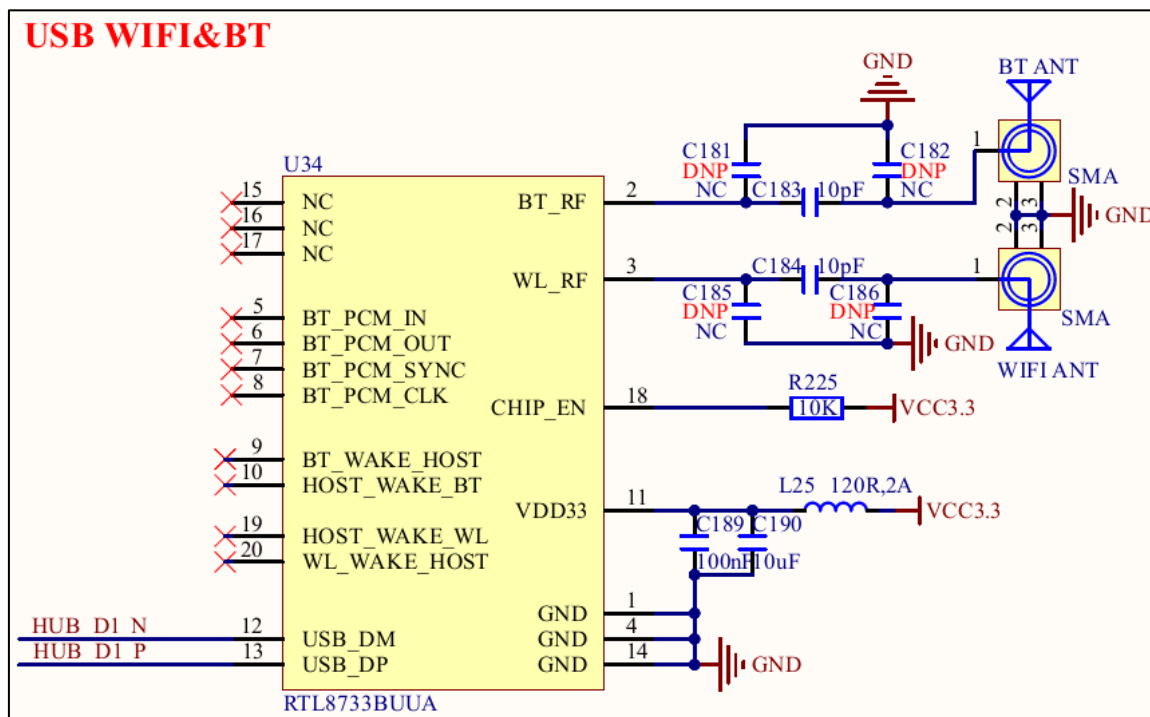


图 25.3-1 开发板 Bluetooth 电路

这个原理图展示了一个 USB WiFi 和蓝牙模块的电路连接,该模块是 Realtek 的 RTL8733BU,支持通过 USB 进行通信。以下是对该电路的解析:

1、电源部分

VDD33 (Pin 11): 连接到 3.3V 电源,通过滤波电容 (C189: 100nF 和 C190: 10uF) 去除电源噪声。为模块内部电路提供工作电源。

电源电感 (L25): 规格为 120R, 2A, 起到电流限流和电源噪声抑制作用。

2、USB 接口

USB_DM (Pin 12) 和 USB_DP (Pin 13): 连接到 USB 主机,分别是 USB 的 D- 和 D+ 信号线。用于 Wi-Fi 和蓝牙模块与主机之间的数据传输。

接地引脚 (Pin 1, 4, 14): 提供模块的电气接地,确保信号完整性。

3、天线接口

BT_RF (Pin 15) 和 WL_RF (Pin 16): 分别连接蓝牙和 Wi-Fi 的射频天线。

通过旁路电容 (C181、C183、C184、C186, 均为 10pF, 部分未安装) 进行高频滤波。

天线匹配网络: 天线通过 SMA 接头连接到模块。可能需要调节匹配电路以满足实际射频性能需求。

4、控制接口

CHIP_EN (Pin 18): 模块的使能引脚,拉高至 VCC3.3 以激活模块。外接一个 10K 的上拉电阻 (R225)。

BT_WAKE_HOST (Pin 9) 和 HOST_WAKE_BT (Pin 10): 蓝牙唤醒引脚,模块与主机之间进行唤醒信号通信。

HOST_WAKE_WL (Pin 20) 和 WL_WAKE_HOST (Pin 19): Wi-Fi 唤醒引脚,与主机通信控制 Wi-Fi 功能。

25.3.2 硬件实物

开发板上的 Bluetooth 相关的电路位置如下图所示:

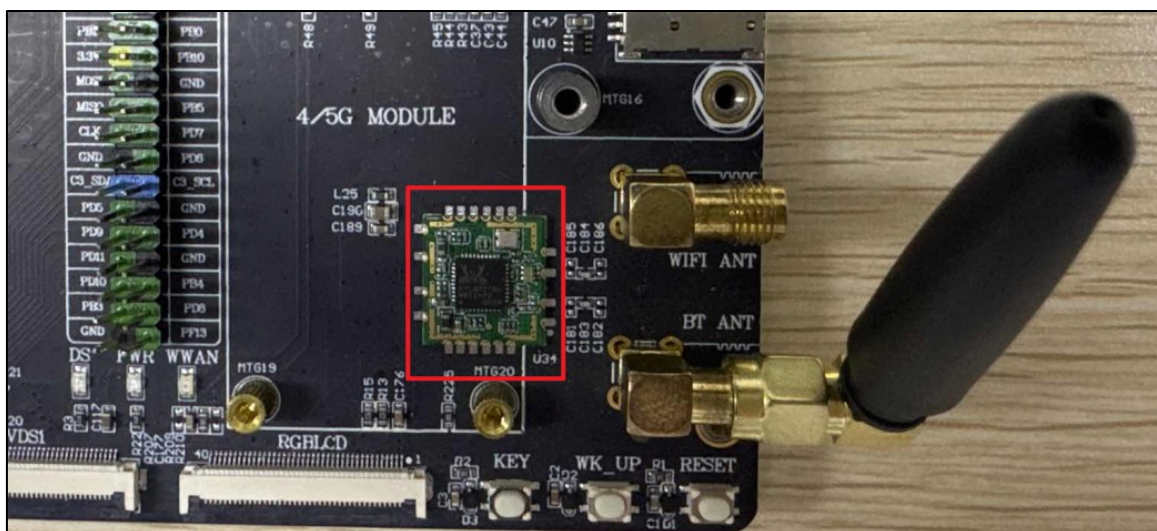


图 25.3-2 开发板 Bluetooth 电路

25.4 内核 Bluetooth 相关使能

25.4.1 蓝牙框架使能

在 ATK-DLMP257B 内核源码配置文件 arch/arm64/configs/stm32mp257_atk_defconfig, Bluetooth 相关配置如下(略有删减, 只列举重点部分):

```
1 CONFIG_NET=y //启用内核的网络子系统支持
2 CONFIG_BT=m //启用蓝牙子系统支持, 并将其编译为可加载模块
3 CONFIG_USB=y //启用内核对 USB 子系统的支持
4 CONFIG_BT_BREDR=y //启用 BR/EDR 蓝牙协议栈支持
5 CONFIG_BT_LE=y //启用 BLE 蓝牙协议支持
6 CONFIG_BT_RTK_HCIBTUSB=m //启用 Realtek HCIBTUSB 蓝牙驱动支持
```

对应的 menuconfig 配置如下(篇幅有限, 截图有所省略):

```
[*] Networking support --->
    [*] Bluetooth subsystem support
        [*] Bluetooth Classic (BR/EDR) features
        [*] Bluetooth Low Energy (LE) features
        [*] Bluetooth device drivers --->
            [*] <M> Realtek HCI USB driver
```

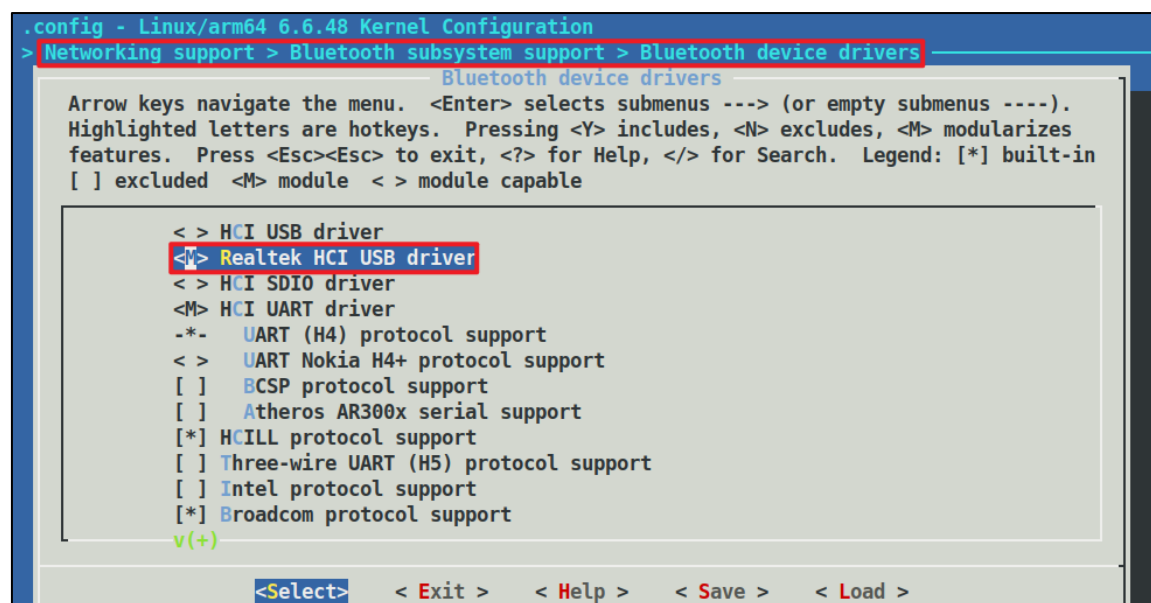


图 25.4-1 Bluetooth 部分配置

对应的驱动源码在出厂内核源码 drivers/net/wireless/realtek/rtl8733bu 目录, 此驱动使用的是 USB 接口协议, 开发板出厂设备树上已添加 USB 节点, 就不需要再其他额外配置了。因此本章节没有设备树讲解, 参考出厂系统设备树的 usb 相关配置即可。

25.5 板载蓝牙测试

25.5.1 蓝牙配对

蓝牙配对需要用到 bluetoothd 服务, bluetoothd 作为系统服务在后台运行, 负责管理蓝牙适配器和处理蓝牙设备的连接、配对和通信。它是 Linux 系统中与蓝牙相关的核心组件之一。

bluetoothd 存放在/usr/libexec/bluetooth 目录下，默认系统运行时已开启，如果需要手动开启可以执行下面命令在后台开启服务。

```
/usr/libexec/bluetooth/bluetoothd -C &
```

输入下面指令，进入 bluetoothctl 交互模式。

```
bluetoothctl
```

依次执行下面指令。

power on	#打开电源
agent on	#开启代理
discoverable on	#开启可被检测
scan on	#开启扫描设备
devices	#当前已扫描搭到的设备

```
root@ATK-DLMP257:~# bluetoothctl
Agent registered
[CHG] Controller F0:A8:82:3A:7B:8B Pairable: yes
[bluetooth]# power on
[bluetooth]# [ 55.016487] rtk_btusb: btusb_open hdev->promisc ==0
[ 55.016819] rtk_btusb: chip type value: 0x76
[ 55.033964] rtk_btusb: chip type value: 0x76
[ 55.033986] rtk_btusb: btusb_open set HCI UP RUNNING
[ 55.037651] rtk_btcoex: Open BTCOEX
[CHG] Controller F0:A8:82:3A:7B:8B Class: 0x000c0000
Changing power on succeeded
[CHG] Controller F0:A8:82:3A:7B:8B Powered: yes
[bluetooth]# agent on
Agent is already registered
[bluetooth]# discoverable on
Changing discoverable on succeeded
[CHG] Controller F0:A8:82:3A:7B:8B Discoverable: yes
[bluetooth]# scan on
[bluetooth]# [ 73.191960] rtk_btcoex: hci (periodic)inq start
Discovery started
[CHG] Controller F0:A8:82:3A:7B:8B Discovering: yes
[NEW] Device 49:9D:A9:BF:CD:1C 49-9D-A9-BF-CD-1C
[NEW] Device D4:6C:27:39:75:52 D4-6C-27-39-75-52
```

图 25.5-1 配置并运行蓝牙

如下图，笔者扫描到自己的安卓手机蓝牙 MAC 地址，确认后就可以进行配对了。不同手机系统可能对蓝牙有不同的拦截或者数据保护策略（比如 iOS 系统），请确保手机蓝牙不被拦截。

```
[bluetooth]# devices
Device 49:9D:A9:BF:CD:1C 49-9D-A9-BF-CD-1C
Device D4:6C:27:39:75:52 D4-6C-27-39-75-52
Device A6:D7:3C:42:4A:D3 L3250 Series
Device 48:DC:13:D9:B7:8E 48-DC-13-D9-B7-8E
Device 64:68:76:9D:A5:71 EDIFIER ZB BLE
Device 61:F5:8B:BF:36:84 61-F5-8B-BF-36-84
Device 0C:83:9A:00:06:EB Mate 40 Pro
Device FC:E8:00:F8:72:2C EDIFIER BLE
Device 53:D1:F4:3E:92:54 53-D1-F4-3E-92-54
Device 58:92:EF:3F:94:C0 58-92-EF-3F-94-C0
Device 43:F1:E7:23:D3:16 43-F1-E7-23-D3-16
Device 60:98:34:D7:BC:47 60-98-34-D7-BC-47
Device 60:73:09:28:C2:DD 60-73-09-28-C2-DD
Device 44:F3:28:3E:A7:A7 44-F3-28-3E-A7-A7
Device 5C:6B:D9:61:C7:E2 5C-6B-D9-61-C7-E2
Device 6E:F8:5F:DD:BD:12 6E-F8-5F-DD-BD-12
Device 00:13:EF:A0:04:F1 DX3 Pro
Device 5F:31:3F:31:78:9D lock 5F313F31789D
Device 54:FA:56:06:FA:6A 54-FA-56-06-FA-6A
Device 5D:5B:93:BA:F2:96 5D-5B-93-BA-F2-96
Device A8:96:09:4A:25:D7 RTK BT 4.1
Device 72:D2:AD:09:D1:B0 72-D2-AD-09-D1-B0
Device F4:39:A6:74:68: 小陈的手机
Device 90:F0:52:BC:8D:7B MEIZU 20
```

图 25.5-2 扫描到蓝牙设备

输入下面指令进行配对。

pair XX:XX:XX:XX:XX:XX # 请填写你自己的蓝牙 MAC

```
[bluetooth]# pair F4:39:A6:74:68:
Attempting to pair with F4:39:A6:74:68:
Failed to pair: org.bluez.Error.AlreadyExists
[CHG] Device A8:96:09:4A:25:D7 RSSI: -102
[CHG] Device FC:E8:00:F8:72:2C RSSI: -82
[CHG] Device F4:39:A6:74:68: RSSI: -50
[CHG] Device 5A:9E:1F:D5:BD:3F ManufacturerData Key: 0x004c
[CHG] Device 5A:9E:1F:D5:BD:3F ManufacturerData Value:
10 06 71 1d 6f d9 c2 38 ..q.o..8
[CHG] Device 78:59:D7:D8:82:74 RSSI: -98
[CHG] Device 6C:0F:07:EE:E4:9C RSSI: -76
[CHG] Device FC:E8:00:F8:72:2C RSSI: -90
[CHG] Device 6C:0F:07:EE:E4:9C RSSI: -90
[NEW] Device 5E:E8:C5:AA:53:D8 5E-E8-C5-AA-53-D8
[CHG] Device 70:0F:56:2A:99:7E RSSI: -94
[CHG] Device A8:96:09:4A:25:D7 RSSI: -94
[bluetooth]# [ 293.676778] rtk_btcoex: hci accept conn req
[ 293.846719] rtk_btcoex: connected, handle 0001, status 0x00
[ 293.846763] rtk_btcoex: Page success
[CHG] Device F4:39:A6:74:68: Connected: yes ← 输入yes
[CHG] Device 4C:0C:51:D0:C3:02 RSSI: -84
[CHG] Device 70:0F:56:2A:99:7E RSSI: -70
[小陈的手机]# [ 294.527719] rtk_btcoex: io capability request
[NEW] Device 5C:91:80:39:60:E2 5C-91-80-39-60-E2
Request confirmation
[agent] Confirm passkey 561728 (yes/no): [CHG] Device 7E:60:25:45:1B:85 RSSI: -98
[agent] Authorize service 0000110d-0000-1000-8000-00805f9b34fb (yes/no): yes ← 输入yes
Authorize service
[agent] Authorize service 0000110e-0000-1000-8000-00805f9b34fb (yes/no): [NEW] Endpoi
nt /org/bluez/hci0/dev_F4_39_A6_74_68_/sep1 如果后续有提示, 继续输入yes
[agent] Authorize service 0000110e-0000-1000-8000-00805f9b34fb (yes/no): [NEW] Endpoi
```

图 25.5-3 确认蓝牙配对请求

同时, 手机端也选择配对。



图 25.5-4 手机端蓝牙配对请求
匹配成功后，我们就可以在手机端手动连接开发板蓝牙了。



图 25.5-5 手机端连接开发板蓝牙

同理，开发板端也会提示蓝牙连接，输入 yes 确认连接。

```
[bluetooth]# [ 972.650812] rtk_btcoex: hci accept conn req
[ 972.842721] rtk_btcoex: connected, handle 0002, status 0x00
[ 972.842766] rtk_btcoex: Page success
[CHG] Device F4:39:A6:74:68: Connected: yes
[CHG] Device 6A:41:34:CC:1A:B0 RSSI: -88
[CHG] Device 6A:41:34:CC:1A:B0 ManufacturerData Key: 0x004c
[CHG] Device 6A:41:34:CC:1A:B0 ManufacturerData Value:
10 07 2c 1f e8 24 9b 3e 48 ...$.>H
[小陈的手机]# [ 973.400127] rtk_btcoex: list_allocate_add: Add a2dp signal channel
Authorize service
[agent] Authorize service 0000110d-0000-1000-8000-00805f9b34fb (yes/no): [CHG] Device
7C:DB:0F:91:F9:94 RSSI: -84
[agent] Authorize service 0000110e-0000-1000-8000-00805f9b34fb (yes/no): yes
[NEW] Player /org/bluez/hci0/dev_F4_39_A6_74_68_/player0 [default]
```

图 25.5-6 开发板确认连接手机蓝牙

等到连接到自己的蓝牙设备，输入 scan off 停止扫描，然后输入 exit 退出交互模式。

```
scan off
exit
```

```
[小陈的手机]# scan off
[小陈的手机]# [ 1208.760411] rtk_btcoex: hci (periodic)inq cancel/exit
Discovery stopped
[CHG] Device D0:62:2C:47:C2:D2 RSSI is nil
[DEL] Device D0:62:2C:47:C2:D2 Xiaomi Smart Band 8 C2D2
[CHG] Device 73:10:F2:95:07:84 TxPower is nil
[CHG] Device 73:10:F2:95:07:84 RSSI is nil
[CHG] Device 00:13:EF:A0:04:F1 TxPower is nil
[CHG] Device 00:13:EF:A0:04:F1 RSSI is nil
[CHG] Controller F0:A8:82:3A:7B:8B Discovering: no
[DEL] Device 5A:95:0B:25:02:64 N41BT-0264
[DEL] Device 63:22:FE:4C:08:C8 63-22-FE-4C-08-C8
[DEL] Device 0C:83:9A:00:06:EB Mate 40 Pro
[小陈的手机]# exit
root@ATK-DLMP257:~#
```

图 25.5-7 退出蓝牙工具

使用 l2ping 工具测试蓝牙连接。此工具需要目标设备蓝牙支持 L2CAP 回显功能且开启相关的蓝牙权限，在适中距离（一般是十米内）且无明显物理障碍物和电磁干扰源。笔者使用的目标设备是安卓手机。

l2ping XX:XX:XX:XX:XX:XX # 请填写你自己的蓝牙 MAC

```
root@ATK-DLMP257:~# l2ping F4:39:A6:74:68:
Ping: F4:39:A6:74:68: from F0:A8:82:3A:7B:8B (data size 44) ...
0 bytes from F4:39:A6:74:68: id 0 time 67.06ms
0 bytes from F4:39:A6:74:68: id 1 time 24.83ms
0 bytes from F4:39:A6:74:68: id 2 time 39.88ms
0 bytes from F4:39:A6:74:68: id 3 time 53.70ms
0 bytes from F4:39:A6:74:68: id 4 time 26.02ms
0 bytes from F4:39:A6:74:68: id 5 time 18.57ms
0 bytes from F4:39:A6:74:68: id 6 time 21.10ms
0 bytes from F4:39:A6:74:68: id 7 time 21.14ms
0 bytes from F4:39:A6:74:68: id 8 time 11.11ms
0 bytes from F4:39:A6:74:68: id 9 time 24.82ms
0 bytes from F4:39:A6:74:68: id 10 time 24.84ms
0 bytes from F4:39:A6:74:68: id 11 time 24.84ms
0 bytes from F4:39:A6:74:68: id 12 time 7.35ms
0 bytes from F4:39:A6:74:68: id 13 time 139.87ms
0 bytes from F4:39:A6:74:68: id 14 time 114.88ms
^C15 sent, 15 received, 0% loss
root@ATK-DLMP257:~#
```

图 25.5-8 蓝牙 l2ping 测试

第二十六章 4G/5G 子系统

26.1 4G/5G 子系统简介

嵌入式 Linux 的 4G/5G 子系统涉及多个组件和协议，用于支持 4G（LTE）通信。这套系统主要处理 4G/5G 调制解调器的连接、数据传输以及网络管理等任务。

26.2 组成部分

嵌入式 Linux 的 4G/5G 子系统主要包括以下组成部分：

1、硬件接口与驱动层

硬件接口：4G/5G 模块通常通过 USB、UART、PCIe 或 SPI 接口连接到嵌入式主机。其中，USB 接口最常见。

内核驱动：Linux 内核提供了多种驱动程序来支持不同类型的 4G 模块，如：

usbserial：通用 USB 串口驱动，适用于支持 AT 命令的调制解调器。

qmi_wwan：用于支持 Qualcomm 模块的 QMI 协议驱动。

cdc_mbim：用于支持基于 MBIM 协议的模块。

2、通信协议层

AT 命令集：许多 4G 模块支持 AT 命令（例如标准的 3GPP TS 27.007 命令），用于配置模块、拨号和管理连接。

QMI（Qualcomm MSM Interface）协议：这是 Qualcomm 调制解调器使用的一种协议，广泛用于嵌入式设备。libqmi 库和 qmicli 工具提供了对 QMI 设备的管理支持。

MBIM（Mobile Broadband Interface Model）协议：一种通用的移动宽带接口协议，支持多厂商的模块。Linux 中 libmbim 库和 mbimcli 工具支持该协议。

3、用户空间管理工具

ModemManager：一个通用的调制解调器管理守护进程，支持多种 4G 模块。它提供 DBus 接口，允许其他应用程序查询和控制 4G 连接。

NetworkManager：通常与 ModemManager 协作，自动化 4G 连接的配置、建立和管理。

PPP（Point-to-Point Protocol）工具：对于一些旧款或低端模块，PPP 仍用于数据传输。pppd 是常用的工具，负责建立 PPP 连接。

4、网络配置与路由管理

网络接口：当 4G 模块连接成功后，系统通常会生成一个网络接口（如 wwan0 或 ppp0），供数据传输使用。

IP 配置与路由：通过 ip 命令管理接口的 IP 地址、路由和 DNS 配置。例如：

ip addr：查看和配置 IP 地址。

ip route：查看和配置路由。

5、调试与监控工具

日志与诊断：使用 dmesg 查看系统日志，检查模块驱动加载状态和连接问题。

mmcli：ModemManager 的命令行工具，支持查询模块状态、发送命令和调试。

qmicli 与 mbimcli：分别用于管理 QMI 和 MBIM 协议的模块，提供设备信息查询、连接配置等功能。

6、连接与数据传输管理

连接建立与管理：使用 ModemManager 或命令行工具（如 qmicli、mbimcli、AT 命令）进行连接配置和管理。

数据传输与监控：一旦建立连接，4G/5G 模块通过网络接口进行数据传输，系统可通过常用网络工具如 ping、wget 等测试和监控网络连接。

7、辅助功能

SIM 卡管理：4G/5G 模块通常需要支持 SIM 卡管理（如 PIN 码验证、网络选择等），可以通过 mmcli、AT 命令等实现。

信号质量与连接状态监控：可以查询 4G/5G 信号强度、网络类型（LTE、HSPA 等）和连接状态，确保网络稳定性。

嵌入式 Linux 的 4G/5G 子系统由硬件驱动、通信协议、用户空间管理工具、网络配置和调试工具组成。这些组件协同工作，为嵌入式设备提供稳定的 4G/5G 连接和网络服务。

26.3 开发板 4G/5G 接口说明

26.3.1 原理图说明

开发板上有一个 Mini PCIE 接口，使用的是 USB 协议，可以用于连接移远 EC20 或者高新兴 ME3630 这两种 4G 模块，5G 模块测试过广和通的 FG132。开发板上的 Mini PCIE 接口支持以上三种模块，原路图中以 4G 命名进行简写。打开开发板上【正点原子】STM32MP257 开发板（A 盘）-基础资料\02、开发板原理图\底板原理图\ATK-DLMP257B_SCH.pdf，搜索 4G。可以看到相关电路如下。

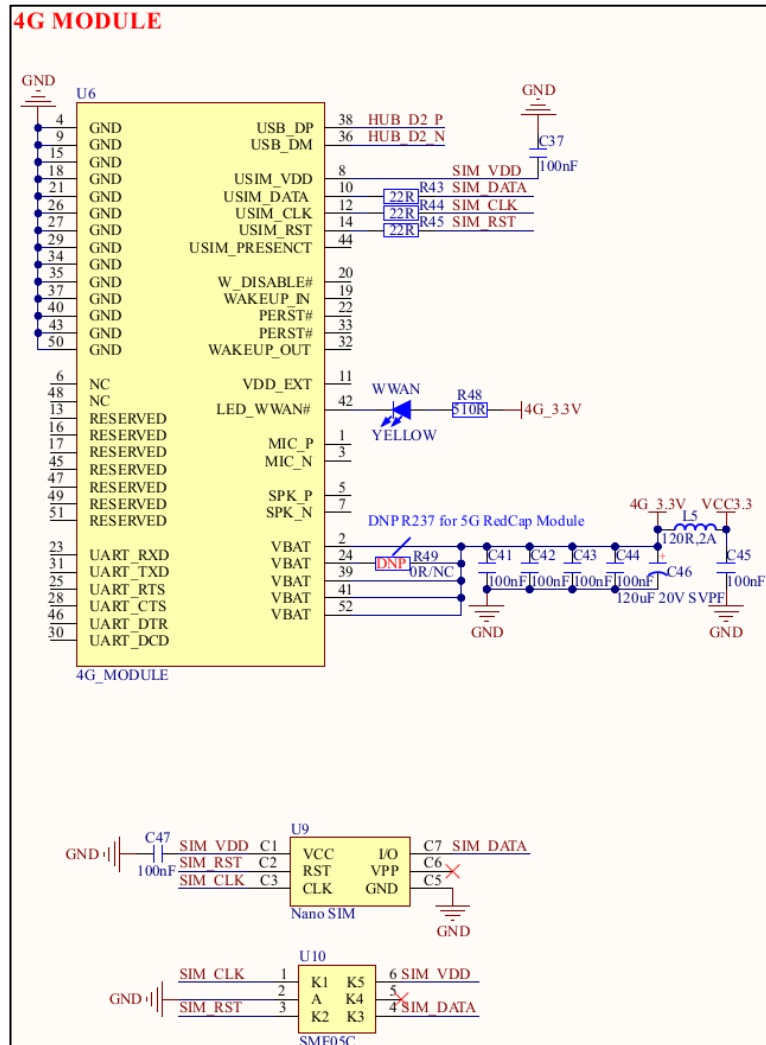


图 26.3-1 4G 模块原理图

原理图解析：

1、4G 模块接口 (U6)

USB 接口：图中的 USB_DP 和 USB_DM 用于 4G 模块与主机之间的数据通信。这是典型的 USB 2.0 接口信号线。

SIM 接口：包括 SIM_VDD、SIM_DATA、SIM_CLK、SIM_RST 等信号线，用于连接 SIM 卡。SIM 卡接口由电容、上拉电阻和去耦电容进行保护和稳定信号。

LED 指示：LED_WWAN#控制 4G 模块的工作状态指示灯。

2、SIM 卡接口 (U9)

该部分电路是一个 Nano SIM 卡座的连接。信号包括 SIM_VDD、SIM_DATA、SIM_CLK、SIM_RST 等，用于和 4G 模块的 SIM 接口进行交互。

图中还包括去耦电容和上拉电阻，以确保 SIM 卡接口的信号完整性和稳定性。

3、电源部分

4G 模块的主要电源输入是 4G_3.3V，通过多个去耦电容进行滤波，以确保电源的稳定性。

4、其他接口

UART 接口引脚（如 UART_RXD、UART_TXD 等）用于调试和控制，常见于嵌入式系统中。其他保留接口未被使用或未连接。

5、R49 电阻选择

原理图中提到了 R49，默认不焊接时可以用于 5G RedCap 模块。

6、LED 指示电路

LED_WWAN#引脚连接到 4G 模块的工作状态指示灯，通过 R48 限流电阻驱动工作状态指示灯，用于显示网络状态。

该原理图展示了一个典型的 4G 模块电路，包括 USB 数据接口、SIM 卡接口、电源和指示灯等设计。每个部分都通过适当的电容、电阻等元件来保证信号的稳定和可靠性。这种设计常见于嵌入式系统中，用于提供移动通信功能。

26.3.2 硬件实物

开发板上的 4G 相关的电路位置如下图所示：

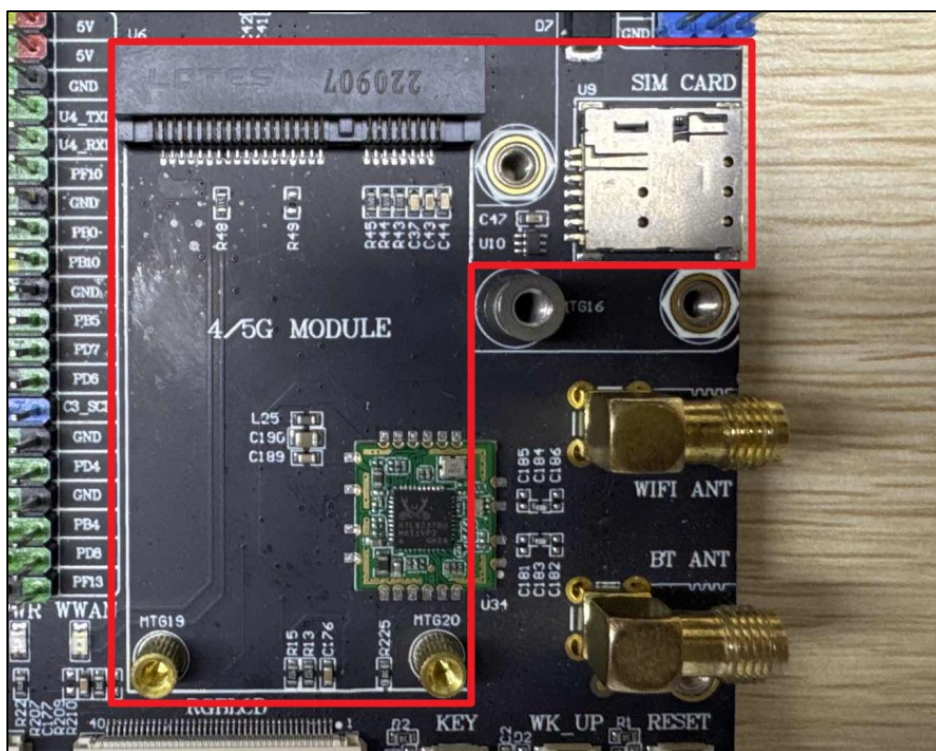


图 26.3-2 开发板 4G 接口和电路

ATK-DLMP257B 开发板默认添加了 4G 模块 EC20 和 ME3630 的驱动以及 5G 模块 FG132 驱动，EC20 模块在正点原子淘宝店铺可以购买，如下图所示：



图 26.3-3 EC20 模块

其他两个模块请自行淘宝搜索对应的模块店铺进行购买。

26.4 内核 4G/5G 相关使能

1、USB 串口驱动

许多 4G/5G 模块通过 USB 接口与主机通信，需要启用 USB 串口驱动：

```
CONFIG_USB_SERIAL=y
CONFIG_USB_SERIAL_OPTION=m # 常见的 4G 模块驱动，例如 Qualcomm 等
CONFIG_USB_SERIAL_WWAN=m
```

2、WDM 与 QMI 协议支持

许多现代 4G/5G 模块支持 QMI 协议，需要启用以下配置：

```
# QMI 协议驱动
CONFIG_USB_NET_QMI_WWAN=m

#启用 USB Wireless Device Management (WDM) 驱动
CONFIG_USB_WDM=m
```

3、PPP 支持

对于使用 PPP 协议的 4G/5G 模块，需要启用 PPP 驱动：

```
CONFIG_PPP=y
CONFIG_PPP_ASYNC=y
CONFIG_PPP_SYNC_TTY=y
CONFIG_PPP_DEFLATE=y
CONFIG_PPP_MPPE=y
CONFIG_PPP_FILTER=y
```

4、通用网络接口支持

4G/5G 模块通常通过网络接口与系统交互，需要启用通用网络驱动：

```
CONFIG_NET=y
CONFIG_NETDEVICES=y
CONFIG_WIRELESS=y
```

5、GPIO 控制与其他电源管理选项

有些 4G/5G 模块使用 GPIO 引脚控制电源或复位信号，这需要启用 GPIO 和电源管理相关的配置：

```
CONFIG_POWER_SUPPLY=y
```

EC20、ME3630 模块对应的驱动源码在出厂内核源码 drivers/usb/serial/usb_wwan.c，FG132 模块对应的驱动源码在出厂内核源码 drivers/net/usb/qmi_wwan.q.c，配置文件 drivers/usb/serial/option.c 和 drivers/net/usb/qmi_wwan.c，此驱动使用的是 USB 接口协议，开发板出厂设备树上已添加 USB 节点，就不需要再其他额外配置了。因此本章节没有设备树讲解，参考出厂系统设备树的 usb 相关配置即可。

26.5 4G 模块 EC20 测试

正点原子 ATK-DLMP257B 底板上预留 4G 模块接口（MINI PCIE 接口），支持正点原子店铺在售的 ME3630-W、EC20 等 4G 模块的安装。

准备 EC20 模块，请在正点原子店铺购买，注意购买时需要买天线，单单模块是不能正常工作的！（备注：EC20 有许多类型模块，目前测试过的是 EC20-CE 模块，其中 EC20-CE 系列又有多种模块，不同的模块功能不一样，比如支持的运营商不一样，详细请咨询卖家），其他 EC20 系列请自行测试，理论上驱动一样，有需求找移远技术支持）。

将 EC20 模块插到 4G 模块接口处，拧上螺丝。保证 4G 模块与座子接口吻合连接。请使用原装天线，把天线连接到 4G 模块的 MAIN 接口处。

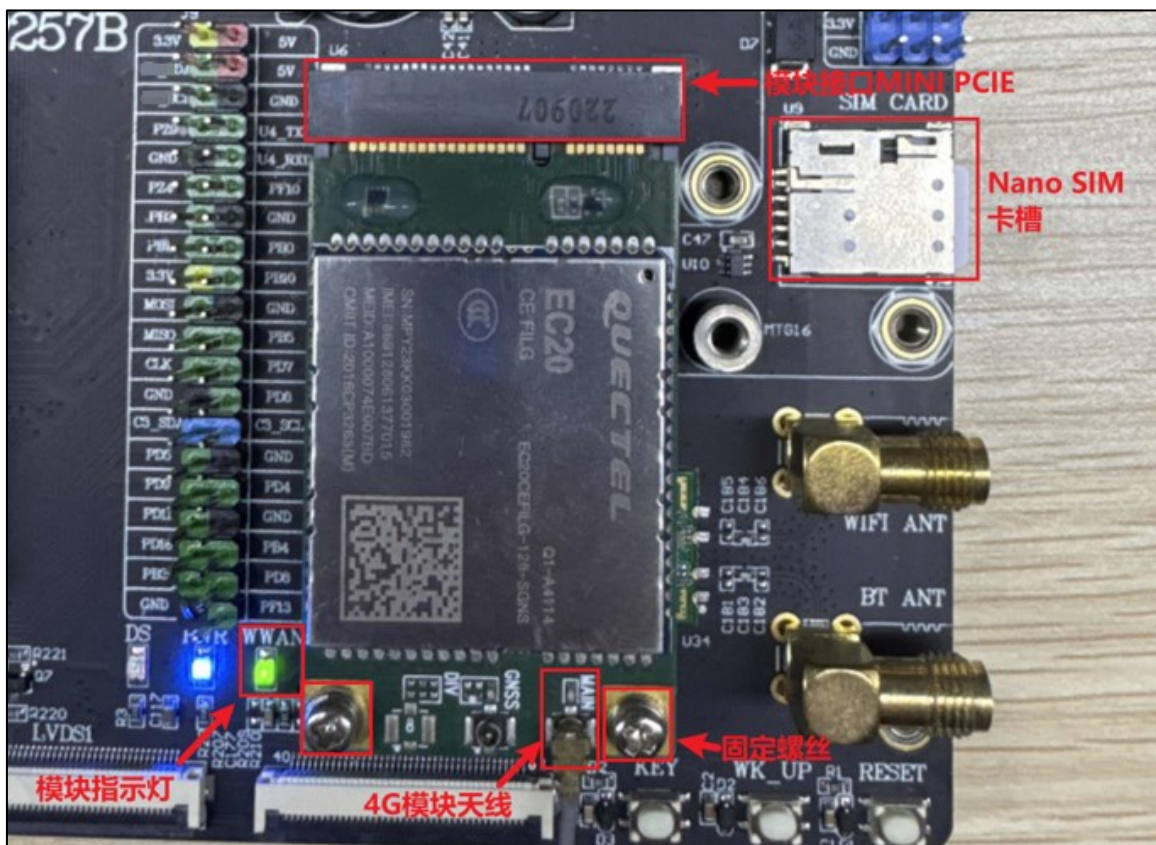


图 26.5-1 开发板 4G 相关接口

正确插入 4G 卡（支持的运营商，请咨询对应模块的卖家，注意有些可能模块不支持物联网卡，请使用普通 4G 卡测试）及插好模块，开发板启动后底板上的 WWAN LED 会亮绿灯。

如果 WWAN LED 绿灯未亮起, 请检查模块是否正确连接插入, 4G 卡是否插入, 天线是否接好, 开发板必须插上配带的 12V 电源。

4G 模块正常加载后, 可以在/dev/下看到有 4 个 ttyUSB*生成。

ls /dev/ttyUSB*

```
root@ATK-DLMP257:~# ls /dev/ttyUSB*
/dev/ttyUSB0 /dev/ttyUSB1 /dev/ttyUSB2 /dev/ttyUSB3
root@ATK-DLMP257:~#
```

图 26.5-2 查看 4G 模块的接口

26.5.1 quectel-CM 拨号上网

使用 quectel-CM 拨号程序工具 (这个工具是我们预先交叉编译好放进文件系统/usr/sbin 目录下面的), 方便用户使用。此驱动由移远提供, 推荐使用此拨号方式。

输入 quectel-CM -h 查看用法。

quectel-CM -h

```
root@ATK-DLMP257:~# quectel-CM -h
[03-03_10:09:14:094] QConnectManager_Linux_V1.6.4
[03-03_10:09:14:095] Usage: quectel-CM [options]
[03-03_10:09:14:095] -s [apn [user password auth]] Set apn/user/password/auth get fro
m your network provider. auth: 1~pap, 2~chap, 3~MsChapV2
[03-03_10:09:14:095] -p pincode Verify sim card pin if sim card is
locked
[03-03_10:09:14:095] -p [quectel-][qmi|mbim]-proxy Request to use proxy
[03-03_10:09:14:095] -f logfilename Save log message of this program t
o file
[03-03_10:09:14:095] -u usbmonlog filename Save usbmon log to file
[03-03_10:09:14:095] -i interface Specify which network interface to
setup data call when multi-modems exists
[03-03_10:09:14:095] -4 Setup IPv4 data call (default)
[03-03_10:09:14:095] -6 Setup IPv6 data call
[03-03_10:09:14:095] -n pdn Specify which pdn to setup data ca
ll (default 1 for QMI, 0 for MBIM)
[03-03_10:09:14:095] -k pdn Specify which pdn to hangup data c
all (by send SIGINT to 'quectel-CM -n pdn')
[03-03_10:09:14:095] -m iface-idx Bind QMI data call to wwan0_<iface
idx> when QMAP used. E.g '-n 7 -m 1' bind pdn-7 data call to wwan0_1
[03-03_10:09:14:095] -b Enable network interface bridge fu
nction (default 0)
[03-03_10:09:14:095] -v Verbose log mode, for debug purpos
e.
[03-03_10:09:14:095] [Examples]
[03-03_10:09:14:095] Example 1: quectel-CM
[03-03_10:09:14:095] Example 2: quectel-CM -s 3gnet
[03-03_10:09:14:095] Example 3: quectel-CM -s 3gnet carl 1234 1 -p 1234 -f gobinet_log.txt
root@ATK-DLMP257:~#
```

图 26.5-3 查看 quectel-CM 工具说明

可以看到-s 参数是指定 apn 类型, 移动卡 apn 一般是 cmnet, 联通卡 apn 一般是 3gnet, 电信卡一般是 ctnet。备注: APN 指一种网络接入技术, 通常是通过手机上网时必须配置的一个参数, 它决定了手机通过哪种接入方式来访问网络。

quectel-CM & // 如果不清楚, 直接输入 quectel-CM, &的作用是后台运行。

```

root@ATK-DLMP257:~# quectel-CM &
root@ATK-DLMP257:~# [03-03_10:10:54:916] QConnectManager_Linux_V1.6.4
[03-03_10:10:54:918] Find /sys/bus/usb/devices/1-1.2 idVendor=0x2c7c idProduct=0x125, bus=0x00
1, dev=0x004
[03-03_10:10:54:919] Auto find qmichannel = /dev/cdc-wdm0
[03-03_10:10:54:919] Auto find usbnet_adapter = wwulu2i4
[03-03_10:10:54:919] netcard driver = qmi_wwan, driver version = 6.1.82
[03-03_10:10:54:919] Modem works in QMI mode
[03-03_10:10:54:931] cdc_wdm fd = 7
[03-03_10:10:55:032] Get_clientWDS = 5
[03-03_10:10:55:064] Get_clientDMS = 1
[03-03_10:10:55:096] Get_clientNAS = 2
[03-03_10:10:55:128] Get_clientUIM = 1
[03-03_10:10:55:160] Get_clientWDA = 1
[03-03_10:10:55:192] requestBaseBandVersion EC20CEFILGR08A01M1G
[03-03_10:10:55:320] requestGetSIMStatus SIMStatus: SIM_READY
[03-03_10:10:55:353] requestGetProfile[1] CMNET///0/IPV4
[03-03_10:10:55:384] requestRegistrationState2 MCC: 460, MNC: 0, PS: Attached, DataCap: LTE
[03-03_10:10:55:417] requestQueryDataCall IPv4ConnectionStatus: DISCONNECTED
[03-03_10:10:55:417] ifconfig wwulu2i4 0.0.0.0
[03-03_10:10:55:427] ifconfig wwulu2i4 down
[03-03_10:10:55:864] requestSetupDataCall WdsConnectionIPv4Handle: 0x86cdb710
[03-03_10:10:55:992] ifconfig wwulu2i4 up
[03-03_10:10:56:001] busybox udhcpc -f -n -q -t 5 -i wwulu2i4
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'wwulu2i4.udhcpc'. Using 'wwulu2i4' (ifindex=9).
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: no lease, failing
[03-03_10:11:11:296] File:ql_raw_ip_mode_check Line:137 udhcpc fail to get ip address, try nex
t:
[03-03_10:11:11:296] ifconfig wwulu2i4 down
[03-03_10:11:11:308] echo Y > /sys/class/net/wwulu2i4/qmi/raw_ip
[03-03_10:11:11:308] ifconfig wwulu2i4 up
[03-03_10:11:11:315] busybox udhcpc -f -n -q -t 5 -i wwulu2i4
udhcpc: started, v1.36.1
Dropped protocol specifier '.udhcpc' from 'wwulu2i4.udhcpc'. Using 'wwulu2i4' (ifindex=9).
udhcpc: broadcasting discover
udhcpc: broadcasting select for 10.38.154.152, server 10.38.154.153
udhcpc: lease of 10.38.154.152 obtained from 10.38.154.153, lease time 7200
RTNETLINK answers: File exists
[03-03_10:11:11:519] /etc/udhcpc.d/50default: Adding DNS 120.196.165.7 ← DNS
[03-03_10:11:11:519] /etc/udhcpc.d/50default: Adding DNS 221.179.38.7
Dropped protocol specifier '.udhcpc' from 'wwulu2i4.udhcpc'. Using 'wwulu2i4' (ifindex=9).
[03-03_10:11:11:580] Warning: No route items found for wwulu2i4

```

图 26.5-4 通过 quectel-CM 工具上网

按 Enter 回到命令行, 输入 ifconfig 命令, 查看 EC20 4G 模块的网络节点名称。wwulu2i4 是 EC20 4G 模块的网络节点名称。

```

root@ATK-DLMP257:~# ifconfig wwulu2i4
wwulu2i4  Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
    inet addr:10.38.154.152  P-t-P:10.38.154.152  Mask:255.255.255.240
    inet6 addr: fe80::2b11:5513:69cf:367f/64  Scope:Link
    UP POINTOPOINT RUNNING NOARP MULTICAST  MTU:1500  Metric:1
    RX packets:2 errors:0 dropped:0 overruns:0 frame:0
    TX packets:97 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:612 (612.0 B)  TX bytes:17012 (16.6 KiB)

root@ATK-DLMP257:~#

```

图 26.5-5 获取到的 IP

ping 百度测试连通性, -I 是指定网卡, 有数据回复表示连通。

ping www.baidu.com -I wwulu2i4

如果提示“bad address 'www.baidu.com'”，那么请检查一下 DNS 服务器地址是否设置正确，打开/etc/resolv.conf 文件，然后加入上面获取到的 DNS 地址，比如笔者的是“221.179.38.7”。

```
echo nameserver 221.179.38.7 >> /etc/resolv.conf
ping www.baidu.com -I wwulu2i4
```

```
root@ATK-DLMP257:~# echo nameserver 221.179.38.7 >> /etc/resolv.conf
root@ATK-DLMP257:~# ping www.baidu.com -I wwulu2i4
PING www.wshifen.com (103.235.47.188) from 10.38.154.152 wwulu2i4: 56(84) bytes of data.
64 bytes from 103.235.47.188 (103.235.47.188): icmp_seq=1 ttl=46 time=89.8 ms
64 bytes from 103.235.47.188 (103.235.47.188): icmp_seq=2 ttl=46 time=99.0 ms
64 bytes from 103.235.47.188 (103.235.47.188): icmp_seq=3 ttl=46 time=76.5 ms
64 bytes from 103.235.47.188 (103.235.47.188): icmp_seq=4 ttl=46 time=77.2 ms
^C64 bytes from 103.235.47.188: icmp_seq=5 ttl=46 time=75.2 ms

--- www.wshifen.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 32475ms
rtt min/avg/max/mdev = 75.199/83.545/98.985/9.342 ms
root@ATK-DLMP257:~#
```

图 26.5-6 添加 dns 到/etc/resolv.conf 并测试

26.5.2 pppd 拨号上网

出厂文件系统/home/root/shell/4G 目录下已经存放有测试 4G 模块的相关脚本。如果前面使用了 quectel-CM 拨号，请重启系统。

进入/home/root/shell/4G/目录下。

```
cd /home/root/shell/4G/
```

使用 ls 指令可以看到此路径下有多个脚本文件，如果没有这样的文件，请及时更新系统固件。

```
root@ATK-DLMP257:~# cd /home/root/shell/4G/
root@ATK-DLMP257:~/shell/4G# ls
disconnect  ECM_DEMO_AUTO  gosuncn_ppp_dialer_10000  gosuncn_ppp_dialer_10086  ppp-on-10010
ECM_DEMO    gosuncn_options  gosuncn_ppp_dialer_10010  ppp-on-10000             ppp-on-10086
root@ATK-DLMP257:~/shell/4G#
```

图 26.5-7 查看脚本

其中尾缀 10000 表示适用于电信卡，10010 表示适用于联通卡，10086 表示适用于移动卡。这里以移动卡为例。执行下面 ppp-on 脚本，进行 4G 模块配置，&的作用是后台运行。

```
./ppp-on-10086 &
```

```

root@ATK-DLMP257:~/shell/4G# ./ppp-on-10086 &
root@ATK-DLMP257:~/shell/4G#
pppd options in effect:
debug                # (from /etc/localtime)
nodetach             # (from /etc/localtime)
persist             # (from /etc/localtime)
dump                # (from /etc/localtime)
noauth              # (from /etc/localtime)
user Anyname        # (from /etc/localtime)
password ??????     # (from /etc/localtime)
/dev/ttyUSB2        # (from /etc/localtime)
l15200              # (from /etc/localtime)
lock                # (from /etc/localtime)
connect chat -v -f /home/root/shell/4G/gosuncn_ppp_dialer_10086 # (from command line)
crtscts             # (from /etc/localtime)
modem               # (from /etc/localtime)
novj                # (from /etc/localtime)
ipcp-accept-local   # (from /etc/localtime)
ipcp-accept-remote  # (from /etc/localtime)
noipdefault         # (from /etc/localtime)
defaultroute        # (from /etc/localtime)
usepeerdns          # (from /etc/localtime)
noccp               # (from /etc/localtime)
nobsdcomp           # (from /etc/localtime)
ATE
OK
ATH
OK
ATP
OK
AT+CGDCONT=1,"IP","CMNET"
OK
ATD*99#
CONNECT
Script chat -v -f /home/root/shell/4G/gosuncn_ppp_dialer_10086 finished (pid 1391), status = 0x0
Serial connection established.
using channel 1
Using interface ppp0
Failed to create pid file /home/alientek/mp2/tools/user_files/4G_files/ppp-2.5.1/_install/var/run/pppd/pp
p0.pid: No such file or directory
Connect: ppp0 <-> /dev/ttyUSB2
sent [LCP ConfReq id=0x1 <asynmap 0x0> <magic 0x57e40a23> <pcomp> <accomp>]
rcvd [LCP ConfReq id=0x2 <asynmap 0x0> <auth chap MD5> <magic 0xacca75cb> <pcomp> <accomp>]
sent [LCP ConfAck id=0x2 <asynmap 0x0> <auth chap MD5> <magic 0xacca75cb> <pcomp> <accomp>]
rcvd [LCP ConfAck id=0x1 <asynmap 0x0> <magic 0x57e40a23> <pcomp> <accomp>]
rcvd [LCP DiscReq id=0x3 magic=0xacca75cb]
rcvd [CHAP Challenge id=0x1 <fb4eabc97f72ab6b23b96c1e5b12860d>, name = "UMTS_CHAP_SRVR"]
sent [CHAP Response id=0x1 <1002a4f238a7eadb74223ec6f876b547>, name = "Anyname"]
rcvd [CHAP Success id=0x1 ""]
CHAP authentication succeeded
CHAP authentication succeeded
sent [IPCP ConfReq id=0x1 <addr 0.0.0.0> <ms-dns1 0.0.0.0> <ms-dns2 0.0.0.0>]
sent [IPV6CP ConfReq id=0x1 <addr fe80::0c2f:5b44:4e61:45c8>]
rcvd [IPCP ConfReq id=0x2]
sent [IPCP ConfNak id=0x2 <addr 0.0.0.0>]
rcvd [IPCP ConfNak id=0x1 <addr 10.38.92.96> <ms-dns1 120.196.165.7> <ms-dns2 221.179.38.7>]
sent [IPCP ConfReq id=0x2 <addr 10.38.92.96> <ms-dns1 120.196.165.7> <ms-dns2 221.179.38.7>]
rcvd [IPCP ConfReq id=0x3]
sent [IPCP ConfAck id=0x3]
rcvd [IPCP ConfAck id=0x2 <addr 10.38.92.96> <ms-dns1 120.196.165.7> <ms-dns2 221.179.38.7>]
Could not determine remote IP address: defaulting to 10.64.64.64
Failed to create /home/alientek/mp2/tools/user_files/4G_files/ppp-2.5.1/_install/etc/ppp/resolv.conf: No
such file or directory
not replacing existing default route via 192.168.6.1 with metric -1
local IP address 10.38.92.96
remote IP address 10.64.64.64
primary DNS address 120.196.165.7
secondary DNS address 221.179.38.7
sent [IPV6CP ConfReq id=0x1 <addr fe80::0c2f:5b44:4e61:45c8>]

```

图 26.5-8 ppp 拨号上网获取到 IP 和 DNS

按下 Enter 键返回命令行, 使用 ifconfig 可查看 4G 模块的拨号节点名称。

```

root@ATK-DLMP257:~/shell/4G# ifconfig ppp0
ppp0      Link encap:Point-to-Point Protocol
          inet addr:10.38.92.96 P-t-P:10.64.64.64 Mask:255.255.255.255
          UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1
          RX packets:4 errors:0 dropped:0 overruns:0 frame:0
          TX packets:24 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:3
          RX bytes:52 (52.0 B) TX bytes:924 (924.0 B)

```

图 26.5-9 查看 4G 网络节点名称

ping 百度测试连通性, -I 是指定网卡, 有数据回复表示连通。按 Ctrl+c 终止指令。

```
ping www.baidu.com -I ppp0
```

如果提示“bad address 'www.baidu.com'”, 那么请检查一下 DNS 服务器地址是否设置正确, 打开/etc/resolv.conf 文件, 然后加入上面获取到的 DNS 地址, 比如笔者的是“221.179.38.7”。

```
root@ATK-DLMP257:~/shell/4G# ping www.baidu.com -I ppp0
PING www.a.shifen.com (183.2.172.42) from 10.38.92.96 ppp0: 56(84) bytes of data.
64 bytes from 183.2.172.42 (183.2.172.42): icmp_seq=1 ttl=47 time=200 ms
64 bytes from 183.2.172.42 (183.2.172.42): icmp_seq=2 ttl=47 time=36.0 ms
64 bytes from 183.2.172.42 (183.2.172.42): icmp_seq=3 ttl=47 time=55.0 ms
64 bytes from 183.2.172.42 (183.2.172.42): icmp_seq=4 ttl=47 time=33.2 ms
64 bytes from 183.2.172.42 (183.2.172.42): icmp_seq=5 ttl=47 time=32.4 ms
64 bytes from 183.2.172.42 (183.2.172.42): icmp_seq=6 ttl=47 time=29.9 ms
```

图 26.5-10 ping 百度测试 4G 网络连接

26.5.3 GPS 功能测试

EC20 还有一个 GPS 的功能, 通过 AT 指令去设置开启 GPS, 本小节只教如何开启和查看原始的 GPS 数据, 其它的 AT 指令请查看【正点原子】STM32MP257 开发板(A 盘)-基础资料\04、参考资料\Quectel_EC2x&EG2x&EG9x&EM05 系列_GNSS_应用指导.pdf。

注意事项: 图中有个 GPS 天线接口, 可以购买带有 GPS 天线版本的 EC20, 无源天线需要接到 GPS 接口, 天线必须放在室外。

使用 GPS 之前先关闭之前的后台拨号操作。运行以下指令进行使用 GPS 功能, 通过 echo 命令把 AT+QGPS=1 数据发送到 USB2。

```
echo -e "AT+QGPS=1\r\n" >/dev/ttyUSB2
cat /dev/ttyUSB1
```

等待搜星结束, 就可以看到 GPS 原始数据, 参考如下:

可按“Ctrl+c”终止 cat 指令, 关闭 GPS 功能可以运行以下代码。

```
echo -e "AT+QGPSEND\r\n" >/dev/ttyUSB2
```

等待几秒钟后 GPS 功能关闭, 这个时候我们可以看到 ttyUSB1 串口不再打印 GPS 的原始数据了。

```
cat /dev/ttyUSB1
```

26.6 4G 模块 ME3630-W 测试

ATK-DLMP257B 开发板预留一个 4G 模块接口 (MINI PCIE 接口), 支持 ME3630-W, EC20 等 4G 模块的安装。准备 ME3630-W 4G 模块, 将 ME3630-W 4G 模块插到 4G 模块接口处, 拧上螺丝。保证 4G 模块与座子接口吻合连接。请使用原装天线, 把天线连接到 4G 模块的 MAIN 接口处。(温馨提示: 4G 模块在开封使用后, 金手指容易氧化, 时间长了上面会有一层氧化膜, 如果多次实验不成功, 请检查 4G 模块是否插好, 或者是否有氧化膜。使用后请保存好 4G 模块!)

正确插入 4G 卡(支持移动、联通、电信 4G 卡)及插好模块, 开发板启动后底板上的 WWAN LED 会亮绿灯。如果 WWAN LED 绿灯未亮起, 请检查模块是否正确连接插入, 4G 卡是否插入, 天线是否接好, 开发板需插上配带的 12V 电源。

4G 模块正常加载后, 可以在/dev/下看到有 3 个 ttyUSB*生成。4G 模块通过 AT 指令访/dev/ttyUSB2 进行通信。

```
ls /dev/ttyUSB*
```

```
root@ATK-DLMP257:~# ls /dev/ttyUSB*
/dev/ttyUSB0 /dev/ttyUSB1 /dev/ttyUSB2
root@ATK-DLMP257:~#
```

图 26.6-1 查看 4G 模块节点

26.6.1 ECM 上网

在操作系统看来，CDC ECM 设备就是一个虚拟以太网卡，包含标准网卡需要的 MAC 地址和 IP 地址。

ECM_DEMO_AUTO 和 ECM_DEMO 是高兴 ECM 上网的程序（我们不需要知道源码，因为它只是一个工具），详细的参数可以执行 ECM_DEMO -h 查看。

ECM_DEMO_AUTO 和 ECM_DEMO 参数是一样的，区别是 ECM_DEMO 执行完流程就会退出，ECM_DEMO_AUTO 会一直运行并每隔一段时间会检查是否断网，断网了会自动重连。ECM_DEMO_AUTO 默认会自动执行拨号流程，故适合做开机自启的程序。

要配置 ECM 模式上网，如果运行了 pppd 上网，请先执行 disconnect 脚本断开 pppd 拨号上网，再执行下面的指令配置成 ECM 模式连接网络。

```
cd shell/4G
./disconnect
./ECM_DEMO -t up
```

```
root@ATK-DLMP257:~# cd shell/4G/
root@ATK-DLMP257:~/shell/4G# ls
disconnect      gosuncn_options      gosuncn_ppp_dialer_10086  ppp-on-10086
ECM_DEMO        gosuncn_ppp_dialer_10000  ppp-on-10000
ECM_DEMO_AUTO   gosuncn_ppp_dialer_10010  ppp-on-10010
root@ATK-DLMP257:~/shell/4G# ./disconnect
killall: pppd: no process killed
root@ATK-DLMP257:~/shell/4G# ./ECM_DEMO -t up
[03_03_09:50:55] ECM_DEMO success
root@ATK-DLMP257:~/shell/4G#
```

图 26.6-2 先断开 ppp-on 上网，再执行 ecm-on 上网

使用 ifconfig 指令查看 enulu2i3 获取的 ip 地址，如果没有获取到 ip 地址，请使用 udhcpc -i enulu2i3 获取。

```
root@ATK-DLMP257:~/shell/4G# ifconfig
br0      Link encap:Ethernet  HWaddr 1E:79:DA:41:FF:FA
        inet6 addr: fe80::1c79:daff:fe41:fffa/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:0  errors:0  dropped:0  overruns:0  frame:0
        TX packets:32  errors:0  dropped:0  overruns:0  carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:0 (0.0 B)  TX bytes:4624 (4.5 KiB)

end0     Link encap:Ethernet  HWaddr 62:6F:C6:77:E6:DD
        inet addr:192.168.6.156 Bcast:192.168.6.255 Mask:255.255.255.0
        inet6 addr: fe80::606f:c6ff:fe77:e6dd/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:5386  errors:0  dropped:0  overruns:0  frame:0
        TX packets:154  errors:0  dropped:0  overruns:0  carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:397296 (387.9 KiB)  TX bytes:30615 (29.8 KiB)
        Interrupt:72

end1     Link encap:Ethernet  HWaddr 52:D9:06:53:A9:0F
        inet6 addr: fe80::50d9:6ff:fe53:a90f/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:163  errors:0  dropped:0  overruns:0  frame:0
        TX packets:296  errors:0  dropped:0  overruns:0  carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:24136 (23.5 KiB)  TX bytes:43803 (42.7 KiB)
        Interrupt:70 Base address:0x4000

enulu2i3 Link encap:Ethernet  HWaddr B2:B5:44:E0:41:39
        inet addr:10.14.17.96 Bcast:10.14.17.127 Mask:255.255.255.192
        inet6 addr: fe80::b0b5:44ff:fee0:4139/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:18  errors:0  dropped:0  overruns:0  frame:0
        TX packets:51  errors:0  dropped:0  overruns:0  carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:1551 (1.5 KiB)  TX bytes:8262 (8.0 KiB)
```

图 26.6-3 查看 ip 地址

ping 百度测试连通性, -I 是指定网卡, 有数据回复表示连通。按 Ctrl+c 终止指令。

ping www.baidu.com -I enulu2i3 // -I 参数是指定网卡名 usb0 (4G 网络)

```
root@ATK-DLMP257:~/shell/4G# ping www.baidu.com -I enulu2i3
PING www.baidu.com (183.240.98.161) from 10.14.17.96 enulu2i3: 56(84) bytes of data.
64 bytes from 183.240.98.161 (183.240.98.161): icmp_seq=1 ttl=51 time=67.3 ms
64 bytes from 183.240.98.161 (183.240.98.161): icmp_seq=2 ttl=51 time=45.7 ms
64 bytes from 183.240.98.161 (183.240.98.161): icmp_seq=3 ttl=51 time=40.6 ms
64 bytes from 183.240.98.161 (183.240.98.161): icmp_seq=4 ttl=51 time=38.9 ms
64 bytes from 183.240.98.161 (183.240.98.161): icmp_seq=5 ttl=51 time=48.6 ms
^C
--- www.baidu.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4003ms
rtt min/avg/max/mdev = 38.871/48.234/67.315/10.158 ms
root@ATK-DLMP257:~/shell/4G#
```

图 26.6-4 ping 百度测试上网

如果需要检测 4G 是否在线 (断线重连), 建议使用 ./ECM_DEMO_AUTO & 指令, 放于后台运行。连上后如果没有获取到 ip 地址, 请使用 udhcpc -i enulu2i3 获取。

26.6.2 pppd 拨号上网

出厂文件系统 /home/root/shell/4G 目录下已经存放有测试 4G 模块的相关脚本。如果前面使用了 ECM 拨号, 请重启系统。

进入 /home/root/shell/4G/ 目录下。

```
cd /home/root/shell/4G/
```


使用 `ls` 指令可以看到此路径下有多个脚本文件, 如果没有这样的文件, 请及时更新系统固件。

```
root@ATK-DLMP257:~# cd /home/root/shell/4G/
root@ATK-DLMP257:~/shell/4G# ls
disconnect      gosuncn_options      gosuncn_ppp_dialer_10086  ppp-on-10086
ECM_DEMO        gosuncn_ppp_dialer_10000  ppp-on-10000
ECM_DEMO_AUTO   gosuncn_ppp_dialer_10010  ppp-on-10010
root@ATK-DLMP257:~/shell/4G#
```

图 26.6-5 查看脚本

其中尾缀 10000 表示适用于电信卡, 10010 表示适用于联通卡, 10086 表示适用于移动卡。这里以移动卡为例。执行下面 `ppp-on` 脚本, 进行 4G 模块配置, &的作用是后台运行。

`./ppp-on-10086 &`

```
root@ATK-DLMP257:~/shell/4G# ./ppp-on-10086 &
root@ATK-DLMP257:~/shell/4G#
pppd options in effect:
debug          # (from /etc/localtime)
nodetach       # (from /etc/localtime)
persist       # (from /etc/localtime)
dump          # (from /etc/localtime)
noauth        # (from /etc/localtime)
user Anyname  # (from /etc/localtime)
password ????? # (from /etc/localtime)
/dev/ttyUSB2  # (from /etc/localtime)
115200       # (from /etc/localtime)
lock          # (from /etc/localtime)
connect chat -v -f /home/root/shell/4G/gosuncn_ppp_dialer_10086 # (from command line)
crtscts       # (from /etc/localtime)
modem         # (from /etc/localtime)
novj          # (from /etc/localtime)
ipcp-accept-local # (from /etc/localtime)
ipcp-accept-remote # (from /etc/localtime)
noipdefault   # (from /etc/localtime)
defaultroute  # (from /etc/localtime)
usepeerdns    # (from /etc/localtime)
noccp         # (from /etc/localtime)
nobsdcomp     # (from /etc/localtime)
ATE
OK
ATH
OK
ATP
OK
AT+CGDCONT=1,"IP","CMNET"
OK
ATD*99#
CONNECT
Script chat -v -f /home/root/shell/4G/gosuncn_ppp_dialer_10086 finished (pid 1362), status = 0x0
Serial connection established.
using channel 1
Using interface ppp0
Failed to create pid file /home/alientek/mp2/tools/user_files/4G_files/ppp-2.5.1/_install/var/run/pppd/pp
p0.pid: No such file or directory
Connect: ppp0 <-> /dev/ttyUSB2
sent [LCP ConfReq id=0x1 <asynmap 0x0> <magic 0x8334470a> <pcomp> <accomp>]
rcvd [LCP ConfReq id=0x1 <asynmap 0x0> <magic 0x724676f9> <pcomp> <accomp>]
sent [LCP ConfAck id=0x1 <asynmap 0x0> <magic 0x724676f9> <pcomp> <accomp>]
rcvd [LCP ConfAck id=0x1 <asynmap 0x0> <magic 0x8334470a> <pcomp> <accomp>]
sent [IPCP ConfReq id=0x1 <addr 0.0.0.0> <ms-dns1 0.0.0.0> <ms-dns2 0.0.0.0>]
rcvd [IPV6CP ConfReq id=0x1 <addr fe80::69a7:3043:0d62:2ff1>]
sent [IPCP ConfAck id=0x1 <addr 10.14.17.97>]
rcvd [IPV6CP ConfReq id=0x1 <addr fe80::2cca:53d7:1461:24bb>]
sent [IPV6CP ConfAck id=0x1 <addr fe80::2cca:53d7:1461:24bb>]
rcvd [IPCP ConfNak id=0x1 <addr 10.14.17.96> <ms-dns1 120.196.165.7> <ms-dns2 120.196.165.7>]
sent [IPCP ConfReq id=0x2 <addr 10.14.17.96> <ms-dns1 120.196.165.7> <ms-dns2 120.196.165.7>]
rcvd [IPV6CP ConfAck id=0x1 <addr fe80::69a7:3043:0d62:2ff1>]
local LL address fe80::69a7:3043:0d62:2ff1
remote LL address fe80::2cca:53d7:1461:24bb
rcvd [IPCP ConfAck id=0x2 <addr 10.14.17.96> <ms-dns1 120.196.165.7> <ms-dns2 120.196.165.7>]
Failed to create /home/alientek/mp2/tools/user_files/4G_files/ppp-2.5.1/_install/etc/ppp/resolv.conf: No
such file or directory
not replacing existing default route via 192.168.6.1 with metric -1
local IP address 10.14.17.96
remote IP address 10.14.17.97
primary DNS address 120.196.165.7
secondary DNS address 120.196.165.7
```

获取到的IP信息和DNS

图 26.6-6 ppp 拨号上网获取到 IP 和 DNS



按下 Enter 键返回命令行, 使用 ifconfig 可查看 4G 模块的拨号节点名称。

```
root@ATK-DLMP257:~/shell/4G# ifconfig ppp0
ppp0      Link encap:Point-to-Point Protocol
          inet addr:10.14.17.96 P-t-P:10.14.17.97  Mask:255.255.255.255
          inet6 addr: fe80::69a7:3043:d62:2ff1/128 Scope:Link
          UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500  Metric:1
          RX packets:8  errors:0  dropped:0  overruns:0  frame:0
          TX packets:32  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:3
          RX bytes:226 (226.0 B)  TX bytes:2330 (2.2 KiB)

root@ATK-DLMP257:~/shell/4G#
```

图 26.6-7 查看 4G 网络节点名称

ping 百度测试连通性, -I 是指定网卡, 有数据回复表示连通。按 Ctrl+c 终止指令。

```
ping www.baidu.com -I ppp0
```

如果提示“bad address 'www.baidu.com'”, 那么请检查一下 DNS 服务器地址是否设置正确, 打开/etc/resolv.conf 文件, 然后加入上面获取到的 DNS 地址, 比如笔者的是 221.179.38.7。

```
echo nameserver 221.179.38.7 >> /etc/resolv.conf
```

```
ping www.baidu.com -I ppp0
```

```
root@ATK-DLMP257:~/shell/4G# ping www.baidu.com -I ppp0
PING www.baidu.com (183.2.172.185) from 10.14.17.96 ppp0: 56(84) bytes of data.
64 bytes from 183.2.172.185 (183.2.172.185): icmp_seq=1 ttl=46 time=160 ms
64 bytes from 183.2.172.185 (183.2.172.185): icmp_seq=2 ttl=46 time=38.7 ms
64 bytes from 183.2.172.185 (183.2.172.185): icmp_seq=3 ttl=46 time=52.9 ms
64 bytes from 183.2.172.185 (183.2.172.185): icmp_seq=4 ttl=46 time=32.6 ms
^C
--- www.baidu.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 32.635/71.089/160.171/51.952 ms
root@ATK-DLMP257:~/shell/4G#
```

图 26.6-8 ping 百度测试 4G 网络连接

26.6.3 GPS 功能测试

ME3630 还有一个 GPS 的功能, 通过 AT 指令去设置开启 GPS, 本小节只教如何开启和查看原始的 GPS 数据, 其它的 AT 指令请查看 ME3630 模块资料的《高新兴物联 ME3630&ME3630-W 模组 AT 指令手册.pdf》。

注意事项: 需使用带有 GPS 天线版本的 **ME3630-C3C**, GPS 天线使用**无源**天线, 需要接到模块的 GPS 接口, 天线必须放在室外。注意, ME3630-C3B 版本是不支持 GPS 功能的, 使用有源天线无法定位。

运行以下指令进行使用 GPS 功能, 通过 echo 命令把 AT+QGPS=1 数据发送到 USB2。

```
echo -e "AT+ZGINIT\r\n" >/dev/ttyUSB1 //初始化 GPS
echo -e "AT+ZGMODE=3\r\n" >/dev/ttyUSB1 //设置定位模式
echo -e "AT+ZGPORT=0\r\n" >/dev/ttyUSB1 //定位信息从 AT、MODEM、UART 上报
echo -e "AT+ZGNMEA=31\r\n" >/dev/ttyUSB1 //设置 GPS 数据为 NMEA 格式
echo -e "AT+ZGPSR=1\r\n" >/dev/ttyUSB1 //使能 ZGPSR 数据
echo -e "AT+ZGRUN=2\r\n" >/dev/ttyUSB1 //连续定位模式
```

配置完成后, 等待搜星结束, 就可以看到 GPS 原始数据, 参考如下:

```
cat /dev/ttyUSB1
```

可按“Ctrl + c”终止 cat 指令, 关闭 GPS 功能可以运行以下代码。

```
echo -e "AT+ZGRUN=0\r\n" >/dev/ttyUSB1 //初始化 GPS
```

等待几秒钟后 GPS 功能关闭，这个时候我们可以看到 ttyUSB1 串口不再打印 GPS 的原始数据了。

```
cat /dev/ttyUSB1
```